

HTML & CSS — 0 dan Expertgacha (o'zbek tilida)

Bu qo'llanma **HTML va CSS** ni mutlaqo noldan professional darajagacha o'rgatadi. Hech qanday oldingi tajriba shart emas — birinchi bob "internet qanday ishlaydi?" dan boshlanadi, oxirgi bob esa to'liq responsive sayt quradi.

🎨 Har bob **SVG diagramlar** bilan boyitilgan — box model, flexbox, grid, specificity, positioning kabi tushunchalar ko'z bilan ko'rib o'rganiladi. Jami 22 bob, 86+ diagramma.

Qanday o'qish kerak

1. Boblarni **tartib bilan** o'q (01 → 02 → ...). Har biri oldingisiga tayanadi.
2. Kod misollarini **o'zing yozib ko'r** — faqat o'qib ketma. Bitta `.html` fayl yarat, brauzerda och.
3. Har bob oxiridagi **Mashqlarni** o'zing yech, keyin `<details markdown="1">` ichidagi yechimga qara.
4. Diagrammalar tushunchani tezroq singdiradi — ularga e'tibor ber.

Talab

Kerak	Daraja
Kompyuter va brauzer (Chrome/Firefox/Edge)	Shart
Matn muharriri (VS Code tavsiya etiladi)	Shart
Oldingi dasturlash tajribasi	Shart emas

I qism — HTML (struktura)

#	Bob	Nima o'rganasan
01	01 — Web va HTML asoslari	internet va web qanday ishlashini (klient-server, brauzer, HTTP so'rov-javob, DNS, URL), HTML nimaligini va nima uchun kerakligini, brauzer sahifani qanday chizishini o'rganamiz, birinchi HTML faylimizni yaratib brauzerda ochamiz, hujjat strukturasini (doctype, html, head, body), element/teg/atribut anatomiyasini, bo'sh joy va izohlarni ko'rib chiqamiz, oxirida DevTools bilan tanishamiz.
02	02 — Matn elementlari	sarlavhalar, paragraflar, ro'yxatlar va matnni belgilash elementlari yordamida matnga tuzilma va ma'no berishni o'rganamiz.
03	03 — Havolalar va media	havolalar (<code>a</code>) yordamida sahifalarni bog'lashni, rasmlarni (<code>img</code>) to'g'ri va tez ko'rsatishni, responsive (moslashuvchan) rasm texnikalarini, hamda audio, video, <code>figure</code> va <code>iframe</code> elementlarini noldan ekspert darajagacha o'rganamiz.
04	04 — Jadvallar	ma'lumot jadvallari: <code>table</code> , qator, ustun va birlashtirish.
05	05 — Formalar	foydalanuvchidan ma'lumot olishni o'rganamiz — <code>form</code> elementi, barcha <code>input</code> turlari va brauzer ichidagi validatsiya (tekshiruv).
06	06 — Semantik HTML va accessibility	ma'noli teglar (<code>header</code> / <code>nav</code> / <code>main</code> / <code>article</code>) bilan sahifa qurish va uni hamma — shu jumladan ko'zi ojiz, klaviatura bilan ishlovchi — odam uchun hammabop (a11y) qilish.
07	07 — Head, meta va SEO	sahifaning ko'rinmas qismi — <code><head></code> ichidagi meta teglar, viewport, favicon, CSS/JS ulash, hamda saytni Google va ijtimoiy tarmoqlar uchun tayyorlaydigan SEO va ulashish sozlamalari.

II qism — CSS (ko'rinish va layout)

#	Bob	Nima o'rganasan
08	08 — CSS asoslari	CSS nima ekanini, uning sintaksisi (selector, e'lon, property, value) hamda CSS ni HTMLga ulashning uch

#	Bob	Nima o'rganasan
		usulini (inline, internal, external) o'rganib, birinchi sahifangizni o'z qo'lingiz bilan bezaysiz.
09	09 — Selektorlar	elementlarni aniq tanlash: tur, klass, id, kombinatorlar va psevdolar.
10	10 — Cascade, specificity va inheritance	bir nechta CSS qoidasi bitta elementga teginganda qaysi qoida g'olib chiqishini hal qiladigan uchta mexanizm — kaskad (cascade), aniqlik (specificity) va meros (inheritance) — to'liq o'rganamiz.
11	11 — Box model	har bir HTML element aslida to'rt qatlamli quti ekanini (content, padding, border, margin) o'rganib, ularning o'lchami qanday hisoblanishini (box-sizing), margin / padding / border qisqa yozuvlarini, margin'lar birlashishini (margin collapse), display turlarini (block/inline/inline-block) va overflow ni to'liq tushunib olasiz.
12	12 — O'lchovlar, tipografiya va ranglar	CSS o'lchov birliklari (px, em, rem, %, vw/vh va boshqalar), rang modellari (nomli, hex, rgb, hsl, oklch) va matnni boshqaradigan tipografiya xossalari hamda web shriftlarni to'liq, "nega"si bilan o'rganamiz.
13	13 — Fon, chegara va soyalar	elementlarni jonlantiramiz — background (fon rangi, rasm, gradient), border va border-radius (chegara va yumaloq burchaklar), box-shadow (chuqurlik beruvchi soyalar), outline hamda filter bilan vizual effektlar yaratamiz.
14	14 — Positioning va z-index	elementlarni joylashtirish — static, relative, absolute, fixed, sticky — hamda qatlamlarni boshqaruvchi z-index va stacking context.
15	15 — Flexbox	bir o'lchovli layout vositasi Flexbox bilan tanishamiz — flex konteyner, ikki o'q (main va cross) va elementlarni shu o'qlar bo'ylab tekislash hamda taqsimlashni 0 dan o'rganamiz.

#	Bob	Nima o'rganasan
16	16 – CSS Grid	sahifani ikki o'lchovda – ustun va qator panjarasi sifatida – qurishni o'rganamiz: <code>grid-template-columns/rows</code> , <code>fr</code> birligi, <code>repeat()</code> , <code>gap</code> , chiziqlarga joylashtirish, nomli sohalar (<code>grid-template-areas</code>), tekislash va <code>auto-fill</code> / <code>auto-fit</code> bilan moslashuvchan layout.
17	17 – Responsive dizayn	bitta sahifani har qanday ekranga – telefon, planshet, desktop – moslashtirishni o'rganamiz: media query, mobile-first yondashuv, fluid layout va <code>clamp()</code> bilan moslashuvchan tipografiya.
18	18 – Transitions, transforms va animatsiyalar	harakat va o'zgarishlar – <code>transform</code> bilan elementni siljitish/aylantirish/kattalashtirish, <code>transition</code> bilan o'zgarishlarni silliq qilish va <code>@keyframes</code> bilan to'liq animatsiyalar yaratishni 0 dan o'rganamiz.
19	19 – Zamonaviy CSS	custom properties (CSS o'zgaruvchilar), <code>calc()</code> / <code>clamp()</code> , CSS nesting, <code>:has()</code> "ota" selektor, <code>@layer</code> kaskad qatlamlari va logical properties kabi zamonaviy CSS imkoniyatlarini 0 dan o'rganib, kodimizni qisqa, moslashuvchan va boshqarsa oson qilamiz.
20	20 – CSS arxitekturasi va uslublar	katta loyihada CSS'ni tartibli, boshqarib bo'ladigan qilib yozish – global scope va specificity muammolari nega kelib chiqishi, BEM kabi nomlash konvensiyalari, reset va normalize, fayllarni komponentlarga bo'lish, DRY, design tokens hamda utility-first (Tailwind) yondashuvini semantik usul bilan taqqoslash.
21	21 – Formalar, UI holatlari va kirish imkoniyati	formalarni chiroyli bezashni, interaktiv holatlarni (<code>:hover</code> , <code>:focus-visible</code> , <code>:checked</code> va boshqalar), dark mode'ni va kirish imkoniyatini (accessibility) o'rganamiz – sayt nafaqat ko'rkam, balki har bir foydalanuvchi uchun qulay bo'lishi uchun.
22	22 – Yakuniy loyiha – responsive landing page	shu paytgacha o'rgangan hamma narsani – semantik HTML, custom properties bilan dizayn tizimi, Grid va Flexbox, responsive, dark mode, hover/focus va animatsiya – birlashtirib, noldan to'liq ishlaydigan, ko'chirib qo'yib ishlatga bo'ladigan responsive landing page quramiz.

HTML va CSS — bir og'iz (kontekst uchun)

- **HTML** — sahifaning **skeleti** (sarlavha, paragraf, rasm, forma). "Nima?" degan savolga javob beradi.
- **CSS** — sahifaning **ko'rinishi** (rang, joylashuv, masofa, animatsiya). "Qanday ko'rinadi?" degan savolga javob beradi.

Analogiya: HTML — uyning g'ishtlari va xonalari; CSS — bo'yoq, mebel va dizayn. Avval struktura (01–07), keyin ko'rinish (08–22).

Tayyor bo'lsang — [01-web-html-asoslari.md](#) dan boshla.

01 — Web va HTML asoslari

 [README](#) · [Keyingi: 02 — Matn elementlari](#) ➔

Bu bobda: internet va web qanday ishlashini (klient-server, brauzer, HTTP so'rov-javob, DNS, URL), HTML nimaligini va nima uchun kerakligini, brauzer sahifani qanday chizishini o'rganamiz, birinchi HTML faylimizni yaratib brauzerda ochamiz, hujjat strukturasi (doctype, html, head, body), element/teg/atribut anatomiyasini, bo'sh joy va izohlarni ko'rib chiqamiz, oxirida DevTools bilan tanishamiz.

1.1 Internet va web — bu nima va ular qanday farq qiladi?

Ko'pchilik "internet" va "web" so'zlarini bir xil ma'noda ishlatadi, lekin ular bir xil emas.

- **Internet** — bu butun dunyo bo'ylab millionlab kompyuterlarni bir-biriga ulab turgan ulkan tarmoq (kabellar, optik tolalar, Wi-Fi, sun'iy yo'ldoshlar). Bu — "yo'llar" tizimi.
- **Web (World Wide Web)** — bu internet ustida ishlaydigan xizmatlardan **bittasi**: o'zaro bog'langan sahifalar tizimi. Bu — yo'llardan yuradigan "mashinalar"dan biri.

✦ Internet ustida web'dan tashqari boshqa narsalar ham ishlaydi: elektron pochta (email), video qo'ng'iroqlar, o'yinlar va hokazo. Biz bu kitobda **web** bilan, ya'ni brauzerda ochiladigan sahifalar bilan ishlaymiz.

Nega bu farqni bilish kerak? Chunki HTML aynan web sahifalarini yaratish uchun mo'ljallangan. Siz internetning o'zini emas, balki uning ustida ishlaydigan sahifalarni quryapsiz.

1.2 Klient va server — kim kimga nima qiladi?

Web'ning eng asosiy g'oyasi — bu **klient-server** modeli. Buni restoran orqali tasavvur qiling:

- **Klient** — mijoz (siz). Restoranda menyudan taom so'raysiz.
- **Server** — ofitsiant va oshxona. So'rovingizni tushunadi, taomni tayyorlab keltiradi.

Web'da:

- **Klient** — bu sizning **brauzeringiz** (Chrome, Firefox, Safari, Edge). U biror sahifani "so'raydi".

- **Server** — bu doim yoqilgan, internetga ulangan kuchli kompyuter. Unda sahifaning fayllari saqlanadi. U so'rovni qabul qilib, kerakli faylni "qaytaradi".

```
Klient (brauzer)  --- so'rov --->  Server
Klient (brauzer)  <--- javob ----  Server
```

💡 Siz hozir o'rganayotgan HTML fayllar oxir-oqibat aynan shunday serverda yotadi va dunyoning istalgan joyidagi odam brauzeri orqali ularni so'rab oladi.

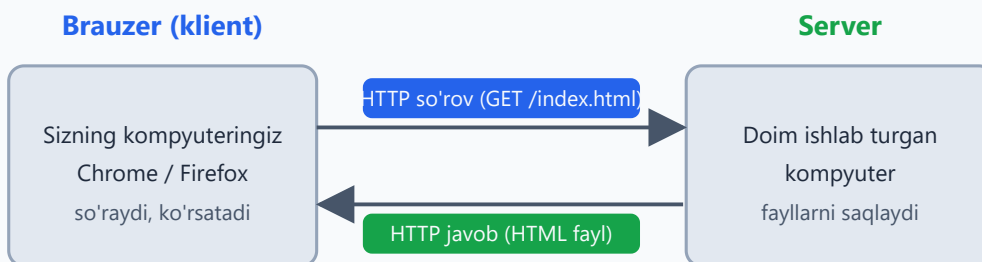
1.3 HTTP so'rov-javob — suhbat qanday kechadi

Brauzer va server bir-biri bilan **HTTP** (HyperText Transfer Protocol — gipermatn uzatish protokoli) deb ataladigan "til" orqali gaplashadi. Protokol — bu ikki tomon kelishib olgan qoidalar to'plami, xuddi ikki kishi bir xil tilda gaplashganidek.

Jarayon oddiy:

1. Siz brauzerga `www.misol.uz` deb yozasiz.
2. Brauzer serverga **HTTP so'rov** (request) yuboradi: "Iltimos, bosh sahifani ber" (`GET /index.html`).
3. Server so'rovni qabul qiladi, kerakli HTML faylni topadi.
4. Server **HTTP javob** (response) qaytaradi: ichida HTML matni bo'lgan xabar.
5. Brauzer bu HTML'ni oladi va ekranga chiroyli sahifa qilib chizadi.

Web qanday ishlaydi: so'rov va javob



1) Brauzer manzilni so'raydi → 2) Server HTML qaytaradi → 3) Brauzer chizadi

✳ `GET` — bu eng ko'p ishlatiladigan so'rov turi: "menga shu narsani **ber**" degani. Boshqa turlari ham bor (`POST` — ma'lumot **yuborish**), lekin hozircha `GET` yetarli.

💡 Har bir javob bilan birga server **holat kodi** (status code) ham yuboradi. Mashhurlari: `200` — "hammasi joyida", `404` — "bunday sahifa topilmadi". `404` xatosini hammamiz ko'rganmiz.

1.4 DNS — manzilni qanday topadi?

Bir muammo bor: kompyuterlar bir-birini nom bilan emas, **raqamli IP-manzil** orqali topadi (masalan, `93.184.216.34`). Lekin biz `www.misol.uz` kabi nomlarni eslab qolamiz, raqamlarni emas.

Bu yerda **DNS** (Domain Name System — domen nomlari tizimi) yordamga keladi. DNS — bu internetning "telefon kitobchasi":

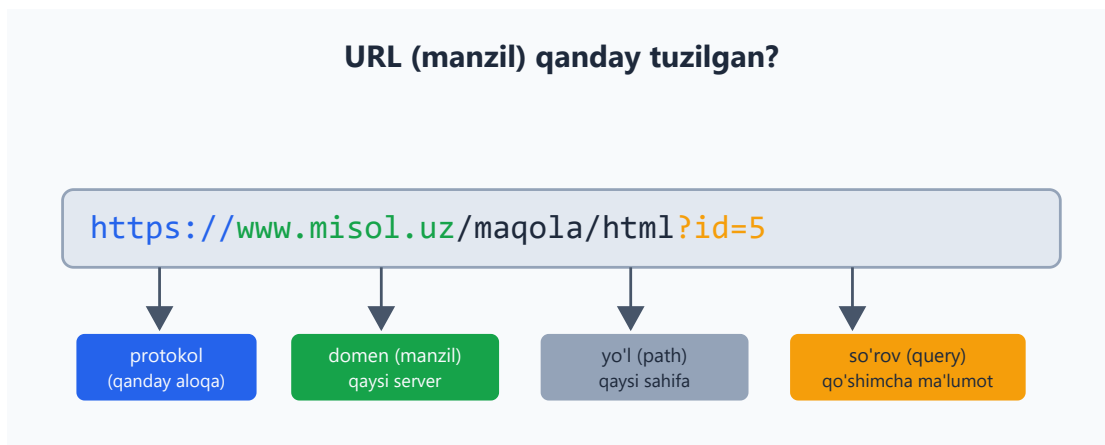
- Siz: "`www.misol.uz` ning IP-manzili nechchi?"
- DNS: "`93.184.216.34`."

Shundan keyingina brauzer aynan o'sha IP-manzildagi serverga so'rov yuboradi.

💡 Buni telefon kontaktlariga o'xshating: siz "Ona" deb qo'ng'iroq qilasiz, telefon esa orqada haqiqiy raqamni topib ulaydi. Siz raqamni eslab qolishingiz shart emas.

1.5 URL — sahifaning to'liq manzili

URL (Uniform Resource Locator) — bu web'dagi har bir resursning (sahifa, rasm, fayl) noyob manzili. Uni qismlarga ajratib ko'raylik:



`https://www.misol.uz/maqola/html?id=5`

Qism	Misol	Ma'nosi
Protokol	<code>https://</code>	Qanday aloqa qilinadi (<code>https</code> — xavfsiz, shifrlangan)
Domen	<code>www.misol.uz</code>	Qaysi serverga murojaat qilinmoqda
Yo'l (path)	<code>/maqola/html</code>	Server ichidagi qaysi sahifa/fayl
So'rov (query)	<code>?id=5</code>	Qo'shimcha ma'lumot (ixtiyoriy)

⚠ Bugungi kunda doim `https` (oxiridagi `s` — secure, xavfsiz) ishlatiladi. Oddiy `http` shifrlanmagan va xavfli hisoblanadi.

✂ `protokol` va `domen` majburiy; `yo'l` va `so'rov` ixtiyoriy. `www.misol.uz` ning o'zi ham yaroqli URL.

1.6 HTML nima va nima uchun kerak?

HTML (HyperText Markup Language — gipermatnli belgilash tili) — bu web sahifaning **strukturasini va mazmunini** tavsiflaydigan til.

E'tibor bering: HTML — bu **dasturlash tili emas**, balki **belgilash (markup) tili**. Ya'ni u "agar bunday bo'lsa, unday qil" kabi mantiqni emas, balki "bu sarlavha", "bu paragraf", "bu rasm" kabi **mazmunni belgilaydi**.

Bir hujjatni tasavvur qiling: siz qog'ozda qaysi qatorni sarlavha, qaysisini oddiy matn, qaysisini ro'yxat qilishni belgilaysiz. HTML aynan shu ishni qiladi — brauzerga "bu element nima ekanligini" aytadi.

Nega aynan HTML kerak?

- Brauzer faqat "matn" ko'rsa, qaysi qism sarlavha, qaysi qism paragraf ekanligini bilmaydi.
- HTML teglari orqali biz brauzerga **ma'no** (semantika) beramiz.
- Bu ma'no nafaqat ko'rinish uchun, balki qidiruv tizimlari (Google) va ekran o'qigichlar (ko'zi ojizlar uchun) uchun ham muhim.

💡 Web sahifa odatda uchta texnologiyadan iborat: - **HTML** — struktura va mazmun (skelet). - **CSS** — ko'rinish, ranglar, joylashuv (kiyim). - **JavaScript** — harakat va interaktivlik (mushaklar).

Biz bu kitobda HTML va CSS bilan shug'ullanamiz. Ushbu bobda — HTML.

1.7 Brauzer HTML'ni qanday o'qiydi (rendering)

Brauzer serverdan HTML matnini olganda, uni shunchaki ko'rsatib qo'ymaydi — avval **qayta ishlaydi**. Bu jarayon **rendering** (chizish) deb ataladi. Qisqacha bosqichlari:

1. **O'qish (parsing)**: brauzer HTML matnini yuqoridan pastga o'qib chiqadi.
2. **DOM qurish**: har bir tegni "tugun" (node) qilib, **daraxt** ko'rinishidagi tuzilma quradi. Bu daraxt **DOM** (Document Object Model) deb ataladi.
3. **Stil qo'shish**: CSS qoidalarini topib, har bir elementga rang, o'lcham va joylashuvni biriktiradi.
4. **Joylashtirish (layout)**: har bir element ekranning qayeriga, qanday o'lchamda joylashishini hisoblaydi.
5. **Chizish (paint)**: nihoyat piksellarni ekranga chizadi.

✦ Eng muhim tushuncha — **DOM daraxti**. Brauzer HTML'ni siz yozgan matn ko'rinishida emas, balki bir-birining ichiga joylashgan tugunlar daraxti ko'rinishida tushunadi. Buni 1.10-bo'limda ko'ramiz.

💡 Brauzer HTML'ni "kechirimli" o'qiydi: agar biror teg unutilgan bo'lsa, u taxmin qilib to'ldirishga harakat qiladi. Bu qulay, lekin xatolarni yashirib qo'yadi — shuning uchun HTML'ni to'g'ri yozish odat qiling.

1.8 Birinchi HTML faylingizni yaratamiz

Endi nazariyadan amaliyotga o'tamiz. HTML fayl yaratish uchun sizga faqat oddiy **matn muharriri** kerak (masalan, VS Code, Notepad). Hech qanday maxsus dastur sotib olish shart emas.

1-qadam. Kompyuteringizda yangi papka oching, masalan `birinchi-sayt`.

2-qadam. Ichida `index.html` nomli fayl yarating.

⚠ Fayl nomi oxiri albatta `.html` bo'lsin. Agar Notepad ishlatsangiz, fayl `index.html.txt` bo'lib qolmasligiga e'tibor bering ("Save as type" da "All Files" tanlang).

✦ Nega `index.html`? Chunki serverlar an'anaviy ravishda papkadagi `index.html` ni "bosh sahifa" deb qabul qiladi. Bu — kelishilgan nom.

3-qadam. Faylga quyidagini yozing:

```
<!DOCTYPE html>
<html lang="uz">
  <head>
    <meta charset="UTF-8">
    <title>Birinchii sahifam</title>
  </head>
  <body>
    <h1>Salom, dunyo!</h1>
    <p>Bu mening birinchi HTML sahifam.</p>
  </body>
</html>
```

4-qadam. Faylni saqlang va ustiga ikki marta bosing (yoki brauzerga sudrab tashlang). Brauzer ochiladi.

Natija: brauzerda katta qalin sarlavha "Salom, dunyo!" va uning ostida "Bu mening birinchi HTML sahifam." matni ko'rinadi. Tabriklaymiz — siz birinchi web sahifangizni yaratdingiz!

💡 Brauzer manzil satrida `file:///C:/.../index.html` kabi yo'l ko'rinadi. Bu — fayl serverdan emas, to'g'ridan-to'g'ri kompyuteringizdan ochilayotganini bildiradi. O'rganish uchun bu mutlaqo yetarli.

1.9 Hujjat strukturasi: doctype, html, head, body

Yuqoridagi kodning har bir qatori muhim. Endi uni qismlarga ajratib tushunamiz.

```
<!DOCTYPE html>
<html lang="uz">
  <head>
    <meta charset="UTF-8">
    <title>Birinchii sahifam</title>
  </head>
  <body>
    <h1>Salom, dunyo!</h1>
  </body>
</html>
```

```
<!DOCTYPE html>
```

Bu — hujjatning birinchi qatori. U brauzerga "men zamonaviy HTML (HTML5) ishlataman" deb aytadi. Bu teg emas, balki **e'lon** (declaration).

⚠️ Agar `<!DOCTYPE html>` ni unutsangiz, brauzer "quirks mode" (g'alati rejim) ga o'tib, sahifani noto'g'ri ko'rsatishi mumkin. Doim birinchi qatorga yozing.

`<html>`

Bu — butun hujjatning **ildiz (root)** elementi. Boshqa hamma narsa shuning ichida joylashadi. `lang="uz"` atributi sahifaning tili o'zbekcha ekanligini bildiradi — bu qidiruv tizimlari va ekran o'qigichlari uchun foydali.

`<head>` — boshqaruv qismi

`<head>` ichidagi narsalar **ekranda ko'rinmaydi**. Bu — sahifa haqidagi "meta ma'lumot" (sahifa haqidagi ma'lumot):

- `<meta charset="UTF-8">` — sahifa kodlash usuli. Bu o'zbekcha harflar (o' , g' , sh) va boshqa belgilar to'g'ri ko'rinishini ta'minlaydi.
- `<title>` — brauzer **tab (varaqa)** sarlavhasida ko'rinadigan nom.

⚠ `<meta charset="UTF-8">` ni unutsangiz, o'zbekcha matn brauzerda tushunarsiz belgilarga aylanib ketishi mumkin. Doim yozing.

`<body>` — ko'rinadigan qism

`<body>` ichidagi hamma narsa **ekranda ko'rinadi**: sarlavhalar, paragraflar, rasmlar, tugmalar — sahifaning butun mazmuni shu yerda.

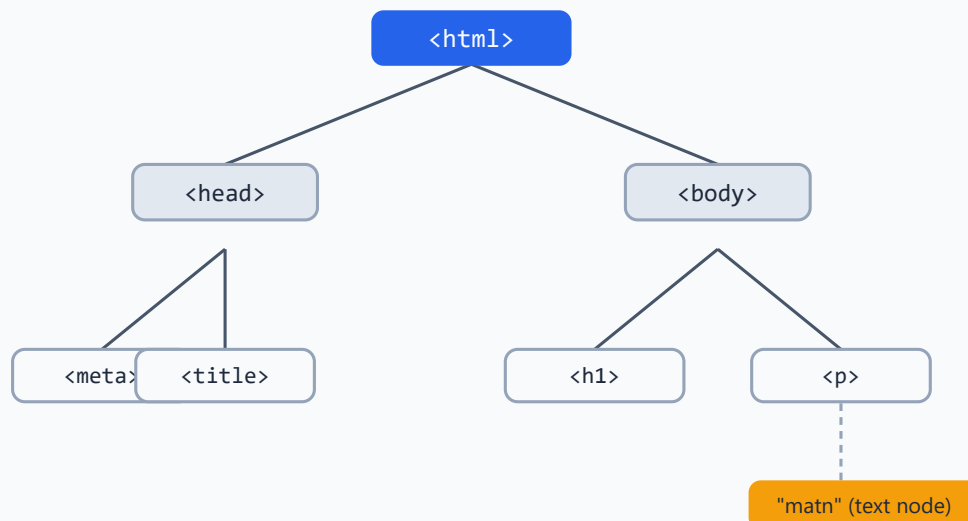
✳ Sodda qoida: `head` — **ko'rinmaydi**, `body` — **ko'rinadi**.

1.10 Hujjat daraxti (DOM tree)

1.7-bo'limda aytganimizdek, brauzer HTML'ni **daraxt** ko'rinishida tushunadi. Yuqoridagi kodimiz daraxt sifatida shunday ko'rinadi:

Hujjat daraxti (DOM): elementlar bir-birining ichida

ildiz (root) element



E'tibor bering:

- `<html>` — eng yuqorida, **ildiz (root)**. Hammasi shuning ichida.
- `<html>` ning ichida ikkita "bola" (child): `<head>` va `<body>`.
- `<head>` ning ichida `<meta>` va `<title>`.
- `<body>` ning ichida `<h1>` va `<p>`.
- `<p>` ning ichida esa oddiy **matn** ("text node").

Bu munosabatlarni oilaviy atamalar bilan aytamiz:

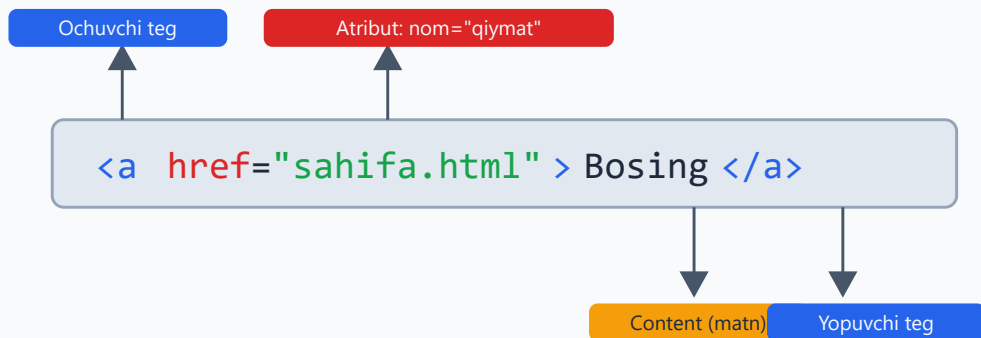
- `<head>` va `<body>` — bir-biriga **opa-singil (siblings)**, chunki bir ota ostida.
- `<html>` ularning **otasi (parent)**.
- `<meta>` — `<head>` ning **bolasi (child)**.

💡 Nega bu muhim? Chunki keyinroq CSS bilan stil berganda va JavaScript bilan ishlaganda biz aynan shu daraxt bo'ylab harakatlanamiz ("bu elementning otasini top", "bolalarini o'zgartir"). Daraxtni tushunish — keyingi hamma narsaning poydevori.

1.11 Element, teg, atribut — anatomiya

HTML butunlay **elementlardan** quriladi. Bitta elementning tuzilishini batafsil ko'raylik:

Bitta HTML element nimalardan iborat?



Ochuvchi teg + content + yopuvchi teg = bitta element. Yopuvchi tegda qiya chiziq /

```
<a href="sahifa.html">Bosing</a>
```

Bu yerda:

- `<a>` — bu **ochuvchi teg** (opening tag). Burchakli qavslar `< >` ichida element nomi.
- `href="sahifa.html"` — bu **atribut** (attribute). Element haqida qo'shimcha ma'lumot. U `nom="qiymat"` ko'rinishida:
- `href` — atribut **nomi**,
- `"sahifa.html"` — atribut **qiymati** (qo'shtirnoq ichida).
- `Bosing` — bu **content** (mazmun), ya'ni ochuvchi va yopuvchi teg orasidagi matn.
- `</a>` — bu **yopuvchi teg** (closing tag). Ochuvchi tegdan farqi — element nomidan oldin **qiya chiziq** / bor.

Demak: **ochuvchi teg + content + yopuvchi teg = bitta element.**

Void (yopilmaydigan) elementlar

Ba'zi elementlarda content bo'lmaydi, shuning uchun **yopuvchi tegi ham yo'q**. Ularni **void element** deyiladi. Misollar:

```
<br>
<hr>

<meta charset="UTF-8">
```

- `<br>` — qator tashlash (line break).
- `<hr>` — gorizontal ajratuvchi chiziq.

- `<img>` — rasm (faqat atributlar bor, ichida matn yo'q).

✦ Void elementlar uchun `</br>` kabi yopuvchi teg **yozi olmaydi** — bu xato bo'ladi.

Bir nechta atribut

Bitta elementda bir nechta atribut bo'lishi mumkin, ular bo'sh joy bilan ajratiladi:

```

```

💡 `alt` atributi rasm yuklanmaganda yoki ekran o'qigich uchun rasmni so'z bilan tasvirlaydi. Doim yozish tavsiya etiladi.

⚠️ Atribut qiymatini doim **qo'shtirnoq** ichiga oling: `href="..."`. Texnik jihatdan ba'zan qo'shtirnoqsiz ham ishlaydi, lekin bu xatolarga olib keladi — odat qilmang.

1.12 Ichma-ich joylashtirish (nesting)

Elementlar bir-birining ichiga joylashtiriladi (nesting). Buni **to'g'ri tartibda** yopish juda muhim.

```
<p>Bu <strong>juda muhim</strong> gap.</p>
```

Bu yerda `<strong>` elementi `<p>` ning ichida joylashgan. Qoida: **eng oxiri ochilgan teg birinchi yopiladi** — xuddi qutilarni bir-birining ichiga joylaganday.

⚠️ Noto'g'ri (teglar kesishib ketgan):

```
<p>Bu <strong>juda muhim</p></strong>
```

To'g'ri (ichkari teg avval yopiladi):

```
<p>Bu <strong>juda muhim</strong></p>
```

💡 Kodni o'qishni osonlashtirish uchun ichki elementlarni **chekinish (indentation)** bilan, ya'ni bir oz ichkariroqdan yozing. Brauzer bu bo'sh joylarni e'tiborsiz qoldiradi, lekin sizga kod tushunarli bo'ladi.

1.13 Bo'sh joy (whitespace)

HTML'da **bo'sh joy** (whitespace — probel, tab, qator tashlash) maxsus tarzda ishlanadi: ketma-ket kelgan bir nechta bo'sh joy brauzer tomonidan **bitta** probelga aylantiriladi.

```
<p>Salom      dunyo</p>
```

Natija: brauzerda "Salom dunyo" — faqat bitta probel bilan ko'rinadi, garchi kodda ko'p probel bo'lsa ham.

Xuddi shunday, qator tashlash ham e'tiborsiz qoldiriladi:

```
<p>Birinchi qator  
ikkinchi qator</p>
```

Natija: "Birinchi qator ikkinchi qator" — bitta qatorda ko'rinadi.

💡 Bu aslida foydali: siz kodni xohlagancha chekinish va qator bilan chiroyli yozasiz, brauzer esa ortiqcha bo'sh joyni o'zi tozalaydi. Agar haqiqiy qator tashlash kerak bo'lsa, `<br>` ishlatib; agar bir nechta probel kerak bo'lsa, keyinroq CSS yordamida hal qilinadi.

1.14 Izohlar (comments)

Ba'zan kodga **izoh** (comment) yozish kerak bo'ladi — o'zingizga yoki boshqa dasturchiga eslatma qoldirish uchun. Izohlar brauzerda **ko'rinmaydi** va sahifaga ta'sir qilmaydi.

HTML izohi quyidagicha yoziladi:

```
<!-- Bu izoh. Brauzerda ko'rinmaydi. -->  
<p>Bu esa ko'rinadi.</p>  
  
<!-- Pastdagi blok bosh sahifaning yuqori qismi -->  
<h1>Xush kelibsiz</h1>
```

Izoh `<!--` bilan boshlanib, `-->` bilan tugaydi. Orasidagi hamma narsa e'tiborsiz qoldiriladi.

Izohlar nima uchun kerak?

- Kodning biror qismini **vaqtincha o'chirib** turish (sinash uchun).
- Murakkab joyni **tushuntirib** qo'yish.
- Katta faylda bo'limlarni **belgilab** qo'yish.

⚠ Izoh ichiga maxfiy ma'lumot (parol, kalit) yozmang! Izohlar sahifaning manba kodida hamma uchun ko'rinadi — istalgan odam "View Source" orqali o'qishi mumkin.

✂ Izohni ichma-ich (`<!-- ichida <!-- boshqa izoh --> -->`) yozib bo'lmaydi — bu xato beradi.

1.15 Brauzer DevTools bilan tanishish

Har bir zamonaviy brauzerda **DevTools** (Developer Tools — dasturchi vositalari) bor. Bu — web sahifani ichidan ko'rish va tekshirish uchun kuchli asbob. Dasturchining "rentgen ko'zi" deyish mumkin.

Ochish:

- Sahifada **sichqoncha o'ng tugmasi** → **Inspect** (yoki "Tekshirish").
- Yoki klaviaturada **F12**.
- Yoki **Ctrl + Shift + I** (Windows) / **Cmd + Option + I** (Mac).

Ochilgan panelda bir nechta bo'lim bor. Boshlovchi uchun eng muhimi — **Elements** (Elementlar) paneli.

Elements paneli nima ko'rsatadi?

- Sahifaning **jonli DOM daraxtini** — ya'ni brauzer hozir tushunib turgan HTML strukturasini.
- Istalgan elementga bossangiz, u sahifada **yoritib (highlight)** ko'rsatiladi.
- Element ustiga ikki marta bosib, HTML'ni **vaqtincha o'zgartirib** ko'rishingiz mumkin (faylga ta'sir qilmaydi, faqat ko'rinishda).

💡 **Birinchi mashqingiz:** istalgan saytni oching (masalan, sevimli yangiliklar sayti), F12 bosing, Elements panelida `<h1>` yoki `<p>` elementini topib, ustiga bosing — sahifada qaysi qism yonishini ko'ring. Bu HTML strukturasini "his qilish"ning eng yaxshi yo'li.

✂ DevTools'dagi o'zgarishlar **vaqtinchalik**: sahifani yangilasangiz (F5), hamma narsa asl holiga qaytadi. Shuning uchun bimalol tajriba qiling — hech narsa buzilmaydi.

⚠ DevTools'da ba'zan "Console" (konsol) bo'limida qizil xato xabarlar ko'rinadi. Hozircha ulardan qo'rqmang — keyingi boblarda nimaligini tushunamiz.

Mashqlar

Quyidagi mashqlarni ketma-ket bajaring. Har birida avval o'zingiz urinib ko'ring, keyin yechimni oching.

Mashq 1 — Birinchi sahifa

`salom.html` nomli fayl yarating. Unda brauzer tabida "Mening saytim" deb ko'rinsin, sahifada esa katta sarlavha "Assalomu alaykum!" va ostida bitta paragraf bo'lsin. Faylni brauzerda oching.

 **Yechim** 

```
<!DOCTYPE html>
<html lang="uz">
  <head>
    <meta charset="UTF-8">
    <title>Mening saytim</title>
  </head>
  <body>
    <h1>Assalomu alaykum!</h1>
    <p>Bu mening yangi sahifam.</p>
  </body>
</html>
```



Mashq 2 — Qismlarni nomlash

Quyidagi qatorda har bir bo'lakni nomlang: ochuvchi teg, atribut nomi, atribut qiymati, content, yopuvchi teg.

```
<a href="https://misol.uz">Saytga o'tish</a>
```



 **Yechim** 

- Ochuvchi teg: `<a ...>`
- Atribut nomi: `href`
- Atribut qiymati: `"https://misol.uz"`
- Content: `Saytga o'tish`
- Yopuvchi teg: `</a>`

Mashq 3 — Xatoni toping

Quyidagi kodda kamida ikkita xato bor. Ularni toping va tuzating.



```
<html>
  <body>
    <h1>Salom<h1>
    <p>Matn</strong>
  </body>
</html>
```

Yechim

Xatolar: 1. `<!DOCTYPE html>`, `<head>` va `<meta charset>` yo'q. 2. `<h1>` ochilgan, lekin yopuvchisi `</h1>` o'rniga yana `<h1>` yozilgan. 3. `<p>` ochilgan, lekin yopuvchisi `</strong>` — mos kelmaydi.

To'g'ri variant:

```
<!DOCTYPE html>
<html lang="uz">
  <head>
    <meta charset="UTF-8">
    <title>Tuzatilgan</title>
  </head>
  <body>
    <h1>Salom</h1>
    <p>Matn</p>
  </body>
</html>
```

Mashq 4 — Void elementni ayting

Quyidagilardan qaysilari void (yopuvchi tegi yo'q) element? `<p>`, `<br>`, `<img>`, `<h1>`, `<hr>`, `<a>`, `<meta>`.

Yechim

Void elementlar: `<br>`, `<img>`, `<hr>`, `<meta>` — bularda content va yopuvchi teg yo'q.

`<p>`, `<h1>`, `<a>` esa oddiy elementlar — ularda content bo'ladi va yopuvchi teg kerak.

Mashq 5 — Izoh qo'shish

3-mashqdagi to'g'ri kodga izoh qo'shing: `<body>` ichida "Sahifaning asosiy mazmuni" deb yozilgan izoh bo'lsin. Brauzerda u ko'rinmasligini tekshiring.

 **Yechim** 

```
<body>
  <!-- Sahifaning asosiy mazmuni -->
  <h1>Salom</h1>
  <p>Matn</p>
</body>
```

Izoh `<!--` va `-->` orasida — brauzerda ko'rinmaydi.

Mashq 6 — Bo'sh joyni sinash

Quyidagi kod brauzerda qanday ko'rinadi? Nima uchun?

```
<p>Bir      ikki      uch</p>
```

 **Yechim** 

Brauzerda **"Bir ikki uch"** — har bir so'z orasida faqat bitta probel bilan ko'rinadi.

Sababi: HTML ketma-ket kelgan bir nechta bo'sh joyni avtomatik ravishda bitta probelga "qisqartiradi" (whitespace collapsing).

Mashq 7 — URL qismlari

`https://blog.misol.uz/maqolalar/html?bob=1` URL'ini qismlarga ajrating: protokol, domen, yo'l, so'rov.

 **Yechim** 

- Protokol: `https://`
- Domen: `blog.misol.uz`
- Yo'l (path): `/maqolalar/html`
- So'rov (query): `?bob=1`

Mashq 8 — DevTools bilan tekshirish

Sevimli saytingizni oching, **F12** bosing va **Elements** panelida sahifaning sarlavha (`<h1>` yoki `<h2>`) elementini toping. Uning ichidagi matnni DevTools'da o'zgartirib ko'ring. Sahifani yangilang (F5) — nima bo'ladi?



Yechim



Elements panelida elementni topib, matn ustiga ikki marta bosib o'zgartirsangiz, sahifada darhol yangi matn ko'rinadi.

Lekin **F5** bosib sahifani yangilaganingizda hamma narsa asl holiga qaytadi — chunki DevTools faqat brauzerdagi vaqtinchalik nusxani o'zgartiradi, serverdagi haqiqiy faylga tegmaydi.



[README](#) · Keyingi: 02 — Matn elementlari ➔

02 — Matn elementlari

← Oldingi: 01 — Web va HTML asoslari · 🏠 README · Keyingi: 03 — Havolalar va media →

Bu bobda: sarlavhalar, paragraflar, ro'yxatlar va matnni belgilash elementlari yordamida matnga tuzilma va ma'no berishni o'rganamiz.

2.1 Nega matn elementlari kerak?

Tasavvur qiling, do'stingizga uzun xat yozyapsiz. Agar xatning hammasini bitta uzun, uzluksiz qatorda yozsangiz — sarlavhasiz, abzatslarsiz, ro'yxatlarsiz — uni o'qish juda qiyin bo'ladi. Aksincha, **sarlavha**, **abzatslar** va **ro'yxatlar** matnni tartibga soladi va uni tushunarli qiladi.

HTML'da ham xuddi shunday. Brauzer matningizni ekranga chiqaradi, lekin u **qaysi qism sarlavha, qaysi qism abzats, qaysi qism ro'yxat ekanini** faqat siz qo'ygan teglar (teg — `<...>` ko'rinishidagi belgi) orqali biladi.

Bu yerda eng muhim g'oya bor, uni butun bob davomida takrorlaymiz:

✦ **HTML ma'no (semantika) bilan ishlaydi, ko'rinish bilan emas.** Siz brauzerga "bu eng muhim sarlavha", "bu abzats", "bu ro'yxat" deb aytasiz. Qanday ko'rinishini (rang, o'lcham, joylashuv) keyinroq CSS hal qiladi. Bu ajratish — HTML'ning asosiy falsafasi.

Nega bu muhim? Chunki HTML'ingizni faqat odamlar emas, **mashinalar** ham o'qiydi:

- **Qidiruv tizimlari** (Google) sahifangizni tushunish uchun teglarga qaraydi.
- **Skrinriderlar** (ko'zi ojiz odamlar uchun matnni ovoz bilan o'qiydigan dasturlar) tuzilmaga tayanadi.
- **Brauzerlar** "kelasi sarlavhaga o'tish" kabi funksiyalarni teglarga qarab beradi.

To'g'ri teg tanlash — bu nafaqat chiroyli ko'rinish, balki **hamma uchun ishlaydigan** sahifa demakdir.

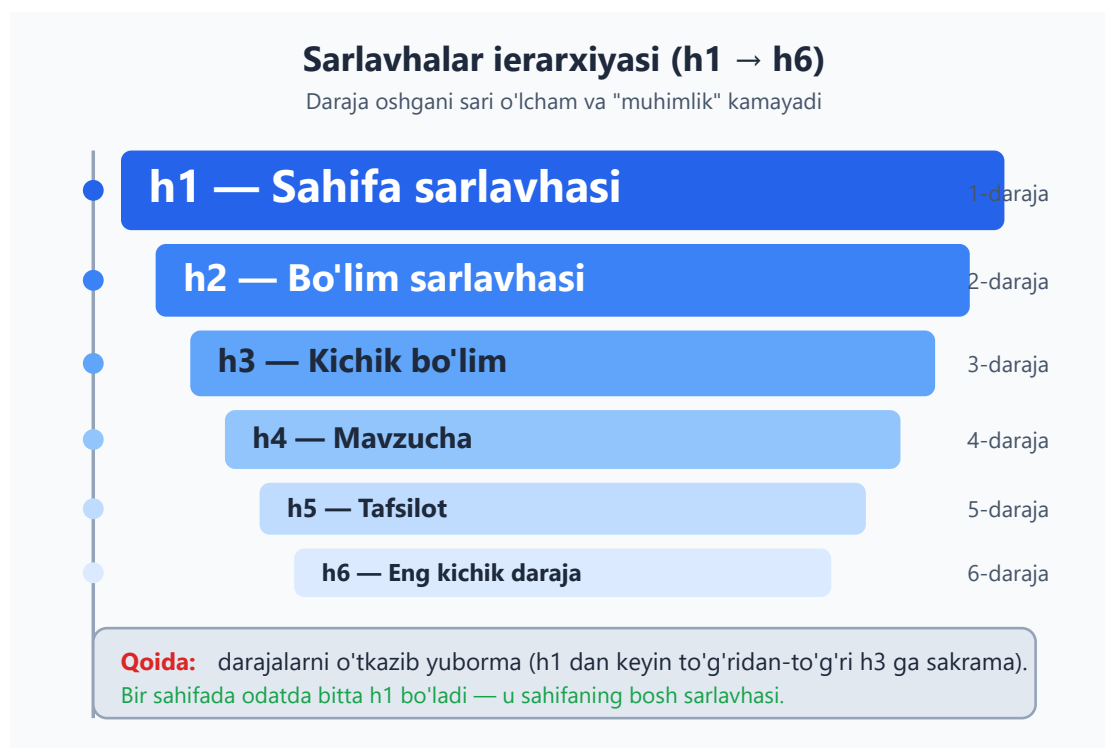
2.2 Sarlavhalar: `<h1>` dan `<h6>` gacha

Sarlavha — matnning bir bo'limining nomi, xuddi kitobdagi bob nomi yoki gazetadagi maqola sarlavhasi kabi. HTML'da **olti daraja** sarlavha bor: `<h1>` (eng muhim, eng

katta) dan `<h6>` (eng kichik) gacha. `h` — "heading" (sarlavha) so'zidan.

```
<h1>HTML qo'llanmasi</h1>
<h2>Matn elementlari</h2>
<h3>Sarlavhalar</h3>
<h4>h1 dan h6 gacha</h4>
<h5>Misol</h5>
<h6>Kichik izoh</h6>
```

Natija: brauzer `<h1>` ni eng katta va qalin qilib, `<h6>` ni eng kichik qilib ko'rsatadi. Lekin yodda tuting — bu standart ko'rinish, uni CSS bilan istalgancha o'zgartirish mumkin. O'lcham emas, **daraja** muhim.



Ierarxiya nima va nega tartib muhim?

Ierarxiya (ierarxiya — daraja bo'yicha tizim) — bu sarlavhalarning bir-biriga bo'ysunishi. Buni kitob mundariyasi (mundarija — kontent ro'yxati) kabi tasavvur qiling:

```
1. HTML asoslari      ← h1
  1.1. Teglar nima    ← h2
    Yopuvchi teg      ← h3
  1.2. Atributlar      ← h2
2. CSS asoslari        ← h1 (lekin sahifada odatda bitta h1)
```

Asosiy qoidalar:

1. **Bir sahifada odatda bitta `<h1>` bo'ladi** — u butun sahifaning bosh sarlavhasi. Buni kitob nomi deb tasavvur qiling: bir kitobda bitta nom bo'ladi.
2. **Darajalarni tartib bilan ishlating** — `<h1>` dan keyin `<h2>` , undan keyin `<h3>` .
3. **Darajani o'tkazib yubormang** — `<h1>` dan to'g'ridan-to'g'ri `<h3>` ga sakramang.

⚠ **Eng keng tarqalgan xato:** "Menga shunchaki katta matn kerak edi, shuning uchun `<h2>` o'rniga `<h1>` ishlatdim" yoki "bu sarlavha kichikroq ko'rinsin deb `<h2>` o'rniga `<h4>` qo'ydim". **Bunday qilmang!** Sarlavha darajasini ma'no bo'yicha tanlang, ko'rinish bo'yicha emas. Matn kattaroq yoki kichikroq ko'rinishi kerak bo'lsa — buni CSS hal qiladi.

Nega bu SEO va accessibility uchun muhim?

SEO (Search Engine Optimization — qidiruv tizimi uchun moslashtirish) nuqtai nazaridan: Google sahifangizning `<h1>` va `<h2>` larini o'qib, sahifa nima haqida ekanini tushunadi. To'g'ri tuzilgan sarlavhalar — qidiruvda yuqori chiqishga yordam beradi.

Accessibility (accessibility — imkoniyati cheklanganlar uchun qulaylik) nuqtai nazaridan: skrinrider foydalanuvchilari ko'pincha "sarlavhadan sarlavhaga sakrab" sahifani tez ko'zdan kechirishadi — xuddi ko'zi ochiq odam sahifani varaqlagandek. Agar sarlavhalar tartibsiz bo'lsa yoki katta matn uchun noto'g'ri ishlatilsa, bu navigatsiyani buzadi.

💡 **Maslahat:** sahifa yozishdan oldin uning mundarijasini qog'ozda chizib oling. Qaysi sarlavha qaysisiga bo'ysunadi? Shu tuzilma sizning `<h1>` — `<h6>` darajalaringizni belgilaydi.

2.3 Paragraf `<p>` , qator uzilishi `<br>` , gorizontaal chiziq `<hr>`

Paragraf — `<p>`

Paragraf (paragraf — abzats) — matnning bir-biriga bog'liq jumladan iborat bo'lagi. Bu eng ko'p ishlatiladigan teglardan biri. `p` — "paragraph" so'zidan.

```
<p>Bu birinchi abzats. U bir nechta jumladan iborat bo'lishi mumkin.
Brauzer satrlarni o'zi avtomatik o'raydi.</p>

<p>Bu ikkinchi abzats. Brauzer abzatslar orasiga avtomatik bo'sh joy
qo'shadi.</p>
```

Natija: ikki abzats alohida bloklar bo'lib, orasida bo'sh joy bilan ko'rinadi.

🚩 **Muhim nuqta — bo'sh joy "siqiladi".** HTML'da kodingizda nechta probel (bo'sh joy) yoki yangi qator qo'ysangiz ham, brauzer ularni **bitta probelga** aylantiradi. Masalan:

```
<p>Salom          dunyo!</p>
```

Brauzerda baribir `Salom dunyo!` (bitta probel bilan) chiqadi. Shuning uchun matnni qatorlarga bo'lish uchun probel yoki Enter bosish ishlamaydi — bunga maxsus teglar kerak.

Qator uzilishi — `<br>`

`<br>` (break — uzilish) matnni **o'sha joyda** keyingi qatorga o'tkazadi. Bu yangi abzats yaratmaydi, faqat qatorni uzadi.

```
<p>Toshkent shahri,<br>
Amir Temur ko'chasi, 12-uy<br>
100000</p>
```

Natija:

```
Toshkent shahri,
Amir Temur ko'chasi, 12-uy
100000
```

`<br>` — **bo'sh element** (void element), ya'ni uning ichida kontent yo'q va yopuvchi teg (`</br>`) kerak emas. Faqat `<br>` yozasiz.

⚠️ **Xato:** `<br>` ni abzatslar orasiga bo'sh joy qo'shish uchun ketma-ket ishlatmang (`<br><br><br>`). Bu uchun alohida abzatslar (`<p>`) ishlatib, masofani esa keyin CSS bilan sozlaysiz. `<br>` faqat **bitta blok ichida** ma'noli qator uzilishi kerak bo'lganda ishlatiladi — masalan she'r, manzil yoki adres.

Gorizontal chiziq — `<hr>`

`<hr>` (horizontal rule — gorizontal chiziq) mavzular o'rtasidagi **tematik o'zgarishni** bildiradi — masalan, bir hikoyadan boshqasiga o'tish. Brauzer buni gorizontal chiziq sifatida ko'rsatadi.

```
<p>Birinchii mavzu haqida matn...</p>

<hr>
```

```
<p>Endi butunlay boshqa mavzu haqida...</p>
```

`<hr>` ham bo'sh element — yopuvchi teg kerak emas.

💡 **Nega "tematik o'zgarish" deyiladi?** Avval `<hr>` faqat dekorativ chiziq edi. Hozirgi HTML'da unga **ma'no** berilgan: u "bu yerda mavzu o'zgaradi" degan semantik signal. Shunchaki chiroyli chiziq kerak bo'lsa — buni CSS chegarasi (border) bilan qiling, `<hr>` bilan emas.

2.4 Tartibsiz ro'yxat `<ul>` va tartibli ro'yxat `<ol>`

Ro'yxatlar — bir turdagi elementlarni guruhlash uchun. HTML'da uchta asosiy ro'yxat turi bor. Avval ikkitasini ko'ramiz.

Ro'yxatning uch turi

<code></code> — tartibsiz	<code></code> — tartibli	<code><dl></code> — tavsif
tartib muhim emas	tartib muhim (qadamlar)	atama + izoh juftligi
● Olma ● Banan ● Uzum	1. Suvni qaynat 2. Choy sol 3. Damla	HTML tuzilma tili CSS ko'rinish tili
<pre> Olma Banan Uzum </pre>	<pre> Qaynat Sol Damla </pre>	<pre><dl> <dt>HTML</dt> <dd>...</dd> </dl></pre>

Tartibsiz ro'yxat — `<ul>`

`<ul>` (unordered list — tartibsiz ro'yxat) — elementlar tartibi **muhim bo'lmaganda** ishlatiladi. Har bir element `<li>` (list item — ro'yxat elementi) ichiga yoziladi. Brauzer har birining oldiga nuqta (•) qo'yadi.

```
<ul>
  <li>Olma</li>
  <li>Banan</li>
  <li>Uzum</li>
</ul>
```

Natija: - Olma - Banan - Uzum

Bu yerda mevalarni qaysi tartibda yozsangiz ham farqi yo'q — shuning uchun "tartibsiz".

Tartibli ro'yxat — `<ol>`

`<ol>` (ordered list — tartibli ro'yxat) — tartib **muhim bo'lganda** ishlatiladi, masalan retsept qadamlari yoki ko'rsatmalar. Brauzer har bir `<li>` oldiga raqam qo'yadi.

```
<ol>
  <li>Suvni qaynating</li>
  <li>Choy soling</li>
  <li>5 daqiqa damlang</li>
</ol>
```

Natija: 1. Suvni qaynating 2. Choy soling 3. 5 daqiqa damlang

💡 **Qaysi birini tanlash?** O'zingizdan so'rang: "Agar bu elementlarning o'rnini almashtirsam, ma'no buziladimi?" Agar **ha** (retsept qadamlari) — `<ol>`. Agar **yo'q** (xarid ro'yxati) — `<ul>`.

`<ol>` ning foydali atributlari

`<ol>` ning raqamlash usulini o'zgartiruvchi uchta atribut bor:

type — raqam o'rniga harf yoki rim raqamlari:

```
<ol type="A">
  <li>Birinchi</li>
  <li>Ikkinchi</li>
</ol>
```

Natija: A. Birinchi B. Ikkinchi

type qiymatlari: `1` (raqam, standart), `A` (katta harf), `a` (kichik harf), `I` (katta rim raqami), `i` (kichik rim raqami).

start — sanoqni 1 dan emas, boshqa sondan boshlash:

```
<ol start="5">
  <li>Beshinchi element</li>
  <li>Oltinchi element</li>
</ol>
```

Natija: 5. Beshinchi element 6. Oltinchi element

reversed — teskari tartibda sanash (kattadan kichikka):

```
<ol reversed>
  <li>Uchinchi o'rin</li>
  <li>Ikkinchi o'rin</li>
  <li>Birinchi o'rin</li>
</ol>
```

Natija: 3. Uchinchi o'rin 2. Ikkinchi o'rin 1. Birinchi o'rin

reversed — **mantiqiy atribut** (boolean attribute): u shunchaki bor yoki yo'q, qiymat yozish shart emas.

🔴 **Diqqat:** `type` va `start` raqamlarning **ko'rinishi va boshlanish nuqtasini** o'zgartiradi, lekin elementlar tartibining ma'nosini emas. Bu HTML atributlari hisoblanadi va ma'noli — masalan, "musobaqaning 5-o'rindan 8-o'rinigacha" ro'yxati uchun `start="5"` to'g'ri yechim.

2.5 Ichma-ich (nested) ro'yxatlar

Ro'yxat ichiga ro'yxat joylashtirib, daraxtsimon tuzilma yaratish mumkin. Buni **nested** (ichma-ich) ro'yxat deyiladi.

Eng muhim qoida: **ichki ro'yxat doim `<li>` ning ichida bo'ladi**, `<li>` lar orasida emas.

```
<ul>
  <li>Mevalar
    <ul>
      <li>Olma</li>
      <li>Banan</li>
    </ul>
  </li>
  <li>Sabzavotlar
    <ul>
      <li>Sabzi</li>
      <li>Kartoshka</li>
    </ul>
  </li>
</ul>
```

Natija: - Mevalar - Olma - Banan - Sabzavotlar - Sabzi - Kartoshka

Brauzer ichki ro'yxatni biroz o'ngga suradi (otstep beradi) va belgisini o'zgartiradi.

⚠️ **Tez-tez uchraydigan xato:** ichki `<ul>` ni ota `<li>` ning **tashqarisiga** qo'yish:


```

<!-- NOTO'G'RI -->
<ul>
  <li>Mevalar</li>
  <ul>                                <!-- li ning tashqarisida! -->
    <li>Olma</li>
  </ul>
</ul>

```

Bu kod ba'zi brauzerlarda "ishlayotgandek" ko'rinishi mumkin, lekin u **noto'g'ri tuzilma** va kelajakda muammo keltiradi. Doim ichki ro'yxatni ota `<li>` ning **ichida** yoping.

💡 Tartibsiz va tartibli ro'yxatlarni aralashtirish mumkin — masalan, `<ol>` ichida `<ul>` yoki aksincha. Mantiqqa qarab tanlang.

2.6 Tavsif ro'yxati — `<dl>`, `<dt>`, `<dd>`

Uchinchi ro'yxat turi — **tavsif ro'yxati** (description list). U **atama + tushuntirish** juftliklari uchun ishlatiladi: lug'at, savol-javob, xususiyatlar ro'yxati va h.k.

Uchta teg ishlatiladi: - `<dl>` (description list) — butun ro'yxatni o'rab turadi. - `<dt>` (description term) — atama (term). - `<dd>` (description details) — atamaning tavsifi.

```

<dl>
  <dt>HTML</dt>
  <dd>Sahifaning tuzilmasini belgilovchi til.</dd>

  <dt>CSS</dt>
  <dd>Sahifaning ko'rinishini (rang, o'lcham) belgilovchi til.</dd>
</dl>

```

Natija:

HTML Sahifaning tuzilmasini belgilovchi til. **CSS** Sahifaning ko'rinishini (rang, o'lcham) belgilovchi til.

Brauzer odatda `<dd>` ni `<dt>` dan biroz o'ngga suradi.

💡 **Moslashuvchanlik:** bitta `<dt>` ga bir nechta `<dd>` berishingiz mumkin (bir atamaning bir nechta ma'nosi), yoki bir nechta `<dt>` ga bitta `<dd>` (bir nechta atama uchun bitta tavsif).

```

<dl>
  <dt>Sichqoncha</dt>
  <dd>Kompyuter boshqaruv qurilmasi.</dd>

```

```
<dd>Kichik kemiruvchi hayvon.</dd>  
</dl>
```

✦ Tavsif ro'yxati ko'pincha unutiladi, lekin u "atama va izoh" juftliklari uchun eng to'g'ri (semantik) tanlovdir — masalan FAQ (tez-tez so'raladigan savollar) yoki metama'lumotlar (sana, muallif) ro'yxati uchun.

2.7 Matnni belgilash: ma'no vs ko'rinish

Endi eng nozik mavzuga keldik. Matn ichida ayrim so'zlarni **ajratib ko'rsatish** uchun teglar bor. Bu yerda asosiy tushuncha: ba'zi teglar **ma'no** beradi (semantik), ba'zilari faqat **ko'rinishni** o'zgartiradi (vizual).

Ma'no (semantika) va ko'rinish (vizual)

Bir xil ko'rinadi, lekin mashina (skrinrider, qidiruv) uchun farqli

Semantik — ma'noli	Vizual — faqat ko'rinish
<code></code> = "muhim" (qalin) <code></code> = "urg'u" (qiya) Skrinrider ovozini o'zgartiradi. Qidiruv tizimi ahamiyatni tushunadi.	<code></code> = shunchaki qalin <code><i></code> = shunchaki qiya Hech qanday qo'shimcha ma'no bermaydi. Faqat harf shaklini o'zgartiradi.

`<strong>` vs `<b>` — qalin matn

Ikkalasi ham brauzerda **qalin** ko'rinadi, lekin ma'nosi farqli:

- `<strong>` — matn **muhim, jiddiy yoki shoshilinch** ekanini bildiradi. Bu **semantik** teg. Skrinrider buni ovoz urg'usi bilan o'qishi mumkin.
- `<b>` — shunchaki **e'tiborni tortish** uchun qalin, lekin qo'shimcha ahamiyat yo'q (masalan, maqola boshidagi kalit so'z).

```
<p><strong>Diqqat:</strong> gaz pechini o'chirishni unutmang!</p>  
<p>Bu retseptda asosiy masalliq — <b>guruch</b>.</p>
```



Natija: ikkalasida ham so'z qalin chiqadi, lekin birinchisi "muhim ogohlantirish", ikkinchisi "shunchaki ajratilgan so'z".

`<em>` vs `<i>` — qiya matn

Ikkalasi ham **qiya** (italic — yotiq) ko'rinadi:

- `<em>` (emphasis — urg'u) — gapdagi **ohang urg'usini** bildiradi. "Men *bugun* keldim" — bugun so'ziga urg'u. Bu **semantik**.
- `<i>` — qo'shimcha ohangsiz qiya matn: chet so'z, ilmiy nom, fikr yoki texnik atama.

```
<p>Men <em>hozir</em> ketishim kerak, ertaga emas!</p>
<p>U <i>déjà vu</i> hissini tuydi.</p>
```

✦ **Qoida — avval semantik tegni tanlang.** Agar so'z haqiqatan muhim bo'lsa — `<strong>`. Agar urg'u bo'lsa — `<em>`. `<b>` va `<i>` ni faqat boshqa semantik teg mos kelmaganda ishlatish. Sababi: semantik teglar skrinrider va qidiruv tizimlari uchun foydali, vizual teglar esa faqat ko'zga ta'sir qiladi.

`<mark>` — sariq belgilash

`<mark>` matnni **marker bilan bo'yalgandek** ajratib ko'rsatadi (odatda sariq fon). U **dolzarblik yoki tegishlilikni** bildiradi — masalan, qidiruv natijasida topilgan so'z.

```
<p>Qidiruvingiz bo'yicha <mark>HTML</mark> so'zi topildi.</p>
```

`<small>` — kichik matn

`<small>` **ikkinchi darajali** matn uchun: mualliflik huquqi, qonuniy izohlar, mayda shartlar.

```
<p><small>&copy; 2026 Mening saytim. Barcha huquqlar himoyalangan.</small></p>
```

`<sub>` va `<sup>` — pastki va yuqori indeks

- `<sub>` (subscript — pastki indeks) — matnni pastga tushiradi: kimyoviy formulalar.
- `<sup>` (superscript — yuqori indeks) — matnni yuqoriga ko'taradi: daraja, izoh raqami.

```
<p>Suv formulasi: H<sub>2</sub>O</p>
```

<p>Maydon: 5 m²</p>

Natija: H₂O va 5 m² ko'rinishida (indekslar bilan).

<code>, <pre>, <kbd> — texnik matnlar

- **<code>** — **kod bo'lagi** (bir qator yoki so'z). Odatda monospace (teng kenglikdagi) shrift bilan ko'rinadi.
- **<pre>** (preformatted — oldindan formatlangan) — matndagi **probellar va qator uzilishlari saqlanadi**. Eslang, 2.3 da aytdikki, brauzer odatda bo'sh joyni siqadi — **<pre>** bu qoidadan istisno.
- **<kbd>** (keyboard — klaviatura) — foydalanuvchi bosadigan **klavishlar**.

```
<p>O'zgaruvchi e'lon qilish uchun <code>let x = 5;</code> yozing.</p>

<pre>
function salom() {
  console.log("Salom");
}
</pre>

<p>Nusxa olish uchun <kbd>Ctrl</kbd> + <kbd>C</kbd> bosning.</p>
```

💡 Ko'p qatorli kodni ko'rsatish uchun **<pre>** va **<code>** ni birga ishlatish keng tarqalgan: **<pre><code>...</code></pre>**. **<pre>** qator uzilishlarini saqlaydi, **<code>** esa "bu kod" degan ma'no beradi.

<blockquote>, <q>, <cite> — iqtiboslar

- **<blockquote>** — **uzun, blok iqtibos** (boshqa manbadan olingan abzats). Brauzer uni odatda chetga suradi.
- **<q>** (quote — qisqa iqtibos) — **gap ichidagi qisqa iqtibos**. Brauzer atrofiga qo'shtirnoq qo'yadi.
- **<cite>** — **asarning nomi** (kitob, film, maqola). Odatda qiya ko'rinadi.

```
<blockquote>
  <p>Bilim — eng kuchli qurol bo'lib, u bilan dunyoni o'zgartirish
  mumkin.</p>
</blockquote>

<p>U <q>ertaga kelaman</q> dedi.</p>

<p>Bu fikr <cite>O'tkan kunlar</cite> romanidan olingan.</p>
```

💡 `<blockquote>` va `<q>` da `cite` **atributi** ham bor (iqtibos manbasining URL manzili uchun): `<blockquote cite="https://...">`. Bu `<cite>` **tegidan** farqli narsa — atribut ko'rinmaydi, faqat manba havolasini saqlaydi.

`<abbr>` — qisqartma

`<abbr>` (abbreviation — qisqartma) qisqartmaning to'liq ma'nosini `title` atributida saqlaydi. Foydalanuvchi sichqonchani ustiga olib kelganda to'liq matn ko'rinadi.

```
<p><abbr title="HyperText Markup Language">HTML</abbr> – veb tili.</p>
```

`<time>` — sana va vaqt

`<time>` sana yoki vaqtni **mashina o'qiy oladigan** formatda saqlaydi. Odam ko'radigan matn istalgancha bo'lishi mumkin, lekin `datetime` atributi standart formatda yoziladi.

```
<p>Tadbir <time datetime="2026-06-09">9-iyun</time> kuni bo'ladi.</p>
```

🔗 **Nega bu foydali?** `datetime="2026-06-09"` ni kalendar dasturlari va qidiruv tizimlari to'g'ri tushunadi, garchi ekranda "9-iyun" yozilgan bo'lsa ham. Bu yana o'sha g'oya: **ma'no (mashina uchun) va ko'rinish (odam uchun) ajratilgan**.

2.8 HTML entities (maxsus belgilar)

Ba'zi belgilarni HTML'ga to'g'ridan-to'g'ri yozib bo'lmaydi. Masalan, `<` belgisini yozsangiz, brauzer uni teg boshlanishi deb o'ylaydi. Bunday belgilar uchun **HTML entity** (HTML mavjudligi — maxsus belgi kodi) ishlatiladi.

Entity `&` bilan boshlanib `;` bilan tugaydi. Eng kerakli beshtasi:

Yozasiz	Chiqadi	Nima uchun
<code>&lt;</code>	<code><</code>	"kichik" belgisi (less than) — teg bilan adashmasligi uchun
<code>&gt;</code>	<code>></code>	"katta" belgisi (greater than)
<code>&amp;</code>	<code>&</code>	ampersand — entity boshi bilan adashmasligi uchun
<code>&copy;</code>	<code>©</code>	mualliflik huquqi belgisi

Yozasiz	Chiqadi	Nima uchun
<code>&nbsp;</code>	(bo'sh joy)	uzilmas probel (non-breaking space)

```
<p>HTML 'da teg shunday yoziladi: &lt;p></p>
<p>Choy &amp; qahva</p>
<p>&copy; 2026 Mening saytim</p>
```



Natija:

```
HTML 'da teg shunday yoziladi: <p>
Choy & qahva
© 2026 Mening saytim
```



` ` — uzilmas probel

` ` (non-breaking space) — oddiy probelga o'xshaydi, lekin ikki muhim farqi bor: 1. U **siqilmaydi** (oddiy probellar bittaga siqilishini eslang). 2. Brauzer uning o'rnida qatorni **uzmaydi** — chap va o'ng tomonidagi so'zlar doim birga qoladi.

```
<p>Narx: 100&nbsp;so'm</p>
```



Bu yerda "100" va "so'm" doim bir qatorda qoladi, hatto satr oxiriga to'g'ri kelsa ham.

⚠️ ` ` **ni masofa qo'yish uchun ko'p ishlatmang** (`   `). Bo'sh joy va masofa CSS ning vazifasi. ` ` ni faqat **so'zlar birga turishi kerak bo'lganda** ishlatng.

💡 Yuzlab boshqa entity'lar bor (`♥`, `→`, `€` va h.k.), lekin yuqoridagi beshtasi kundalik ishingizning 95% ini qoplaydi.

2.9 Yig'ib: semantika — asosiy g'oya

Butun bob davomida bir g'oyani takrorladik, uni mustahkamlaymiz:

Har doim "bu nima?" deb so'rang, "bu qanday ko'rinadi?" deb emas.

- Sarlavhami? → `<h1>` — `<h6>` (darajaga qarab)
- Abzatsmi? → `<p>`
- Muhim ogohlantirishmi? → `<strong>`

- Shunchaki qalin so'zmi? → `<b>`
- Kodmi? → `<code>`
- Iqtibosmi? → `<blockquote>` yoki `<q>`

To'g'ri teg tanlash sahifangizni **hammaga** — odamlarga, mashinalarga, skrinriderlarga, qidiruv tizimlariga — tushunarli qiladi. Ko'rinishni (rang, o'lcham, masofa) keyingi boblarda CSS bilan to'liq nazorat qilamiz. Tuzilma va ma'no — HTML'da; ko'rinish — CSS'da. Bu ajratishni o'zlashtirsangiz, professional veb-dasturchi yo'lining yarmini bosib o'tgan bo'lasiz.

Mashqlar

Quyidagi mashqlarni o'zingiz yozib ko'ring. Har birida yechimni ochishdan oldin o'zingiz urinib ko'ring — shundagina haqiqiy o'rganasiz.

Mashq 1 — Sarlavhalar ierarxiyasi

Bir "Oshxona retseptlari" sahifasi uchun sarlavhalarni to'g'ri darajalarda yozing: sahifa nomi "Oshxona retseptlari", uning ostida "Birinchi taomlar" va "Shirinliklar" bo'limlari, "Birinchi taomlar" ostida esa "Sho'rva" mavzusi bo'lsin.

 **Yechim** 

```
<h1>Oshxona retseptlari</h1>

<h2>Birinchi taomlar</h2>
<h3>Sho' rva</h3>

<h2>Shirinliklar</h2>
```

Sahifada bitta `<h1>`, ikkita `<h2>` (asosiy bo'limlar) va `<h3>` (kichik mavzu) ishlatildi. Darajalar o'tkazib yuborilmadi.

Mashq 2 — To'g'ri ro'yxat turini tanlang

Quyidagilar uchun `<ul>` yoki `<ol>` dan qaysi biri to'g'ri? Sababini ayting: 1. Pitsa tayyorlash qadamlari 2. Yoqtirgan ranglaringiz 3. Musobaqa g'oliblari (1-, 2-, 3-o'rin)



Yechim



1. `<ol>` — qadamlar tartibi muhim, oldin xamir, keyin sous.
2. `<ul>` — ranglar tartibi muhim emas.
3. `<ol>` — o'rinlar tartibli (1, 2, 3).

```
<ol>
  <li>Xamir yoying</li>
  <li>Sous suring</li>
  <li>Pishiring</li>
</ol>
```



Mashq 3 — `start` va `type` atributlari

10 dan boshlanadigan va kichik harflar bilan raqamlanadigan ro'yxat yozing (a, b, c emas — 10-elementdan boshlanadigan raqamli ro'yxat). Keyin alohida ro'yxat: katta rim raqamlari bilan.



Yechim



```
<!-- 10 dan boshlanuvchi -->
<ol start="10">
  <li>0'ninchi</li>
  <li>0'n birinchi</li>
</ol>

<!-- Katta rim raqamlari -->
<ol type="I">
  <li>Birinchi</li>
  <li>Ikkinchi</li>
  <li>Uchinchi</li>
</ol>
```



Birinchisi 10, 11 deb; ikkinchisi I, II, III deb chiqadi.

Mashq 4 — Ichma-ich ro'yxat

Quyidagi tuzilmani HTML'da yozing: - Dasturlash tillari - Veb: HTML, CSS - Backend: Python, PHP

 **Yechim** 

```
<ul>
  <li>Dasturlash tillari
    <ul>
      <li>Veb: HTML, CSS</li>
      <li>Backend: Python, PHP</li>
    </ul>
  </li>
</ul>
```

Ichki `<ul>` ota `<li>` ning **ichida** yopilganiga e'tibor bering.

Mashq 5 — Tavsif ro'yxati

Uchta veb atamasi uchun tavsif ro'yxati (`<dl>`) yozing: "Teg", "Atribut", "Element". Har biriga qisqa izoh bering.

 **Yechim** 

```
<dl>
  <dt>Teg</dt>
  <dd>Burchakli qavslar ichidagi belgi, masalan &lt;p&gt;.</dd>

  <dt>Atribut</dt>
  <dd>Tegga qo'shimcha ma'lumot beruvchi qism, masalan class="...".</dd>

  <dt>Element</dt>
  <dd>Ochuvchi teg, kontent va yopuvchi tegning birgalikdagi to'plami.</dd>
</dl>
```

Mashq 6 — Semantik teglarni to'g'rilang

Quyidagi kodda vizual teglar noto'g'ri ishlatilgan. Semantik variantga o'zgartiring:

```
<p><b>Ogohlantirish:</b> bu havola xavfli.</p>
<p>Men <i>albatta</i> kelaman, ishonchingiz komil bo'lsin!</p>
```



Yechim



```
<p><strong>Ogohlantirish:</strong> bu havola xavfli.</p>
<p>Men <em>albatta</em> kelaman, ishonchingiz komil bo'lsin!</p>
```



"Ogohlantirish" — muhim ma'no, shuning uchun `<strong>`. "albatta" — gapdagi ohang urg'usi, shuning uchun `<em>`.

Mashq 7 — HTML entities

Quyidagi matnni brauzerda **aynan shunday** chiqadigan qilib HTML'da yozing (teglar matn sifatida ko'rinsin):

```
Sevimli teglarim: <h1> & <p>
© 2026
```



Yechim



```
<p>Sevimli teglarim: &lt;h1>& &amp; &lt;p>&lt;/p>
<p>&copy; 2026</p>
```



`<` uchun `<`, `>` uchun `>`, `&` uchun `&`, `©` uchun `©` ishlatildi.

Mashq 8 — Aralash matn elementlari

Bitta abzatsda quyidagilarni birlashtiring: kimyoviy formula (H_2O), klaviatura klavishi (Enter), kichik mualliflik izohi va kod bo'lagi (`document.write`).



Yechim



```
<p>
  Suv — bu H<sub>2</sub>O. Davom etish uchun <kbd>Enter</kbd> bosing.
  Konsolda <code>document.write</code> dan foydalaning.
  <small>&copy; 2026 Qo'llanma</small>
</p>
```



`<sub>` indeks uchun, `<kbd>` klavish uchun, `<code>` kod uchun, `<small>` ikkinchi darajali izoh uchun ishlatildi.

[← Oldingi: 01 — Web va HTML asoslari](#) · [🏠 README](#) · [Keyingi: 03 — Havolalar va media](#) [→](#)

03 — Havolalar va media

◀ Oldingi: 02 — Matn elementlari · 🏠 README · Keyingi: 04 — Jadvallar ▶

Bu bobda: havolalar (`a`) yordamida sahifalarni bog'lashni, rasmlarni (`img`) to'g'ri va tez ko'rsatishni, responsive (moslashuvchan) rasm texnikalarini, hamda audio, video, `figure` va `iframe` elementlarini noldan ekspert darajagacha o'rganamiz.

Oldingi boblarda biz HTML'ning matn elementlarini ko'rdik. Lekin internetni "internet" qiladigan narsa — bu **havolalar**. Aynan havolalar tufayli bir sahifadan ikkinchisiga o'tamiz, butun dunyo saytlari bir-biriga ulanadi. So'ngra esa sahifalarni jonlantiruvchi **media** keladi: rasmlar, ovoz, video.

Bu bobda biz ikki katta mavzuni yopamiz:

1. **Havolalar** — `<a>` elementi va uning barcha imkoniyatlari (`href` , `target` , `download` , xavfsizlik).
2. **Media** — `<img>` rasmlari, responsive rasmlar, `<figure>` , `<audio>` , `<video>` va `<iframe>` .

Oddiydan boshlab, asta-sekin ekspert nozikliklariga yetamiz. Har bir narsada "**nega aynan shunday?**" degan savolga javob beramiz — chunki sababini bilsang, qoidani yodlash shart emas, o'zing chiqarib olasan.

3.1 Havola nima va nega kerak?

Tasavvur qiling, har bir veb-sahifa — bu kitobdagi alohida varaq. **Havola** (link) — bu bir varaqdan ikkinchisiga olib boruvchi "eshik". Foydalanuvchi ko'k, ostiga chizilgan matnni bosadi va boshqa sahifaga o'tadi.

HTML'da havola **anchor** (langar) elementi — `<a>` bilan yaratiladi:

```
<a href="https://uzbekistan.uz">Bizning sayt</a>
```

Bu kod brauzerda "**Bizning sayt**" degan bosiladigan matn ko'rsatadi. Bosilganda foydalanuvchi `https://uzbekistan.uz` manziliga o'tadi.

Bu yerda eng muhim qism — `href` atributi (HTML "**hypertext reference**" — gipermatn havolasi degani). U **qayerga** o'tishni belgilaydi. Ochilish tegi (`<a ...>`) bilan yopilish tegi (`</a>`) orasidagi matn esa foydalanuvchi **ko'radigan** va **bosadigan** qism.

```
<a href="QAYERGA">NIMA KO'RINADI (bosiladigan matn)</a>
```

💡 `<a>` — bu **inline** (qatoriy) element. Ya'ni u matn ichida, gap o'rtasida ham tura oladi:

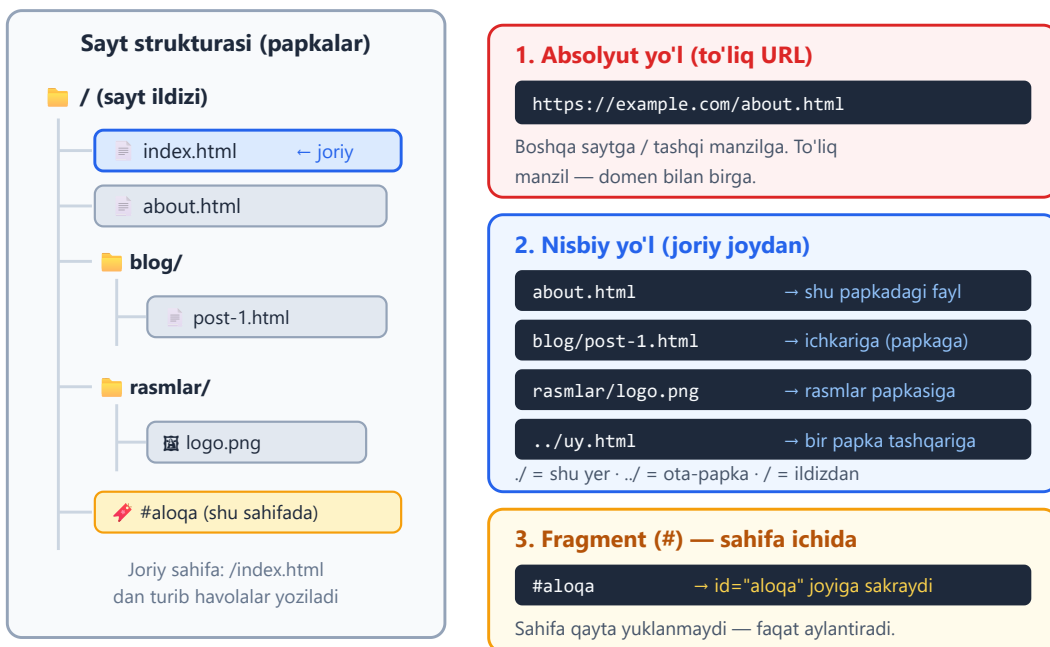
```
<p>Ko'proq ma'lumot uchun <a href="yordam.html">yordam sahifasiga</a> o'ting.</p>
```

⚠️ **Eng ko'p uchraydigan boshlang'ich xato:** `href` ni unutib qo'yish. `<a>Matn</a>` deb yozsangiz, matn ko'rinadi-yu, lekin u **bosilmaydigan** bo'lib qoladi — chunki "qayerga" o'tishni bermadingiz.

3.2 `href` qiymatlari: yo'l turlari

`href` ga turli xil qiymatlar bera olamiz. Eng muhim turlarini ko'rib chiqamiz. Bularni tushunish — bobning eng asosiy poydevori, shuning uchun shoshilmang.

Havola yo'l turlari



Absolyut yo'l (to'liq URL)

Absolyut yo'l — bu **to'liq manzil**, `https://` (yoki `http://`) va domen nomi bilan birga. Bu xuddi xatdagi to'liq pochta manzili kabi: "Toshkent shahri, Amir Temur ko'chasi, 1-uy" — bu manzilni dunyoning istalgan joyidan yozsangiz ham topiladi.

```
<a href="https://developer.mozilla.org">MDN hujjatlari</a>  
<a href="https://www.google.com/search?q=html">Google'da qidirish</a>
```

Nega kerak? Absolyut yo'l **boshqa saytga** (tashqi manzilga) havola berishning yagona to'g'ri yo'li. Boshqa saytda qaysi papkada turishingizni bilmaysiz, shuning uchun to'liq manzil yozasiz.

Nisbiy yo'l (joriy joydan)

Nisbiy yo'l — bu **hozir turgan faylga nisbatan** manzil. Domen yozilmaydi. Bu xuddi "qo'shni xona" yoki "tepadagi qavat" deyishga o'xshaydi — faqat hozir qayerda turganingizni bilsa, joyni topib olasiz.

Aytaylik, hozir `index.html` faylidamiz va u bilan **bir papkada** `about.html` bor:

```
<a href="about.html">Biz haqimizda</a>
```

Endi yo'lning "navigatsiya" belgilarini ko'ramiz — bular eng chalkashtiradigan, ammo eng muhim qism:

Belgi	Ma'nosi	Misol
<code>fayl.html</code>	Shu papkadagi fayl	<code>about.html</code>
<code>papka/fayl.html</code>	Ichkariga — papka ichiga kirish	<code>blog/post-1.html</code>
<code>./</code>	"Aynan shu papka" (ko'pincha tushiriladi)	<code>./about.html</code> = <code>about.html</code>
<code>../</code>	Bir papka tashqariga (ota-papkaga) chiqish	<code>../index.html</code>
<code>../..</code>	Ikki papka tashqariga	<code>../.. /uy.html</code>
<code>/</code> bilan boshlanishi	Sayt ildizidan (domen tubidan)	<code>/rasmlar/logo.png</code>

Misollar bilan tushunamiz. Faraz qiling, fayl manzillari shunday:

```
/index.html  
/about.html  
/blog/post-1.html  
/blog/rasmlar/sarlavha.jpg
```

Agar siz `/blog/post-1.html` ichida bo'lsangiz:

```
<!-- Shu papkadagi rasmlar/ ichiga kirish (ichkariga) -->
<a href="rasmlar/sarlavha.jpg">Sarlavha rasmi</a>

<!-- Bir papka tashqariga chiqib, index.html ga -->
<a href="../index.html">Bosh sahifa</a>

<!-- Sayt ildizidan boshlanadi (qaysi faylda bo'lsangiz ham ishlaydi) -
->
<a href="/about.html">Biz haqimizda</a>
```

💡 `../` va `../` ni qanday eslab qolish kerak? Bitta nuqta `.` = "shu yer" (joriy papka). Ikkita nuqta `..` = "bir qadam orqaga" (ota-papka). Telefon kontaktlaridagi "orqaga" tugmasiga o'xshaydi: har bir `../` sizni bir daraja yuqoriga ko'taradi.

⚠️ **Tez-tez uchraydigan xato:** havola ishlamasa, ko'pincha sabab — yo'lning noto'g'riligi. `../` ni ortiqcha yoki kam yozib qo'yish eng keng tarqalgan muammo. Brauzerning manzil qatoriga qarang: u qaysi faylni izlayotganini ko'rsatadi.

✳️ **Absolyut yo'l (/ dan boshlanuvchi) qachon kerak?** Katta saytlarda fayllar turli chuqurlikdagi papkalarda turadi. `/rasmlar/logo.png` deb yozsangiz, bu **har qaysi faylda** bir xil ishlaydi — chunki u har doim ildizdan hisoblaydi. Nisbiy `../../rasmlar/logo.png` esa fayl joyiga bog'liq bo'lib, ko'chirilganda buziladi.

Fragment havolasi (#) — bitta sahifa ichida sakrash

bilan boshlanuvchi havola **boshqa sahifaga o'tmaydi** — u **shu sahifaning** ma'lum bir joyiga "sakraydi". Bu uzun sahifalarda juda foydali (masalan, "Yuqoriga qaytish" tugmasi yoki mundarija).

Bu ikki qismdan iborat: **maqsad** (id bilan belgilangan joy) va **havola** (# bilan unga ishora qiluvchi):

```
<!-- Havola: bosilganda #aloqa joyiga sakraydi -->
<a href="#aloqa">Aloqa bo'limiga o'tish</a>

<!-- ... uzun matn ... -->

<!-- Maqsad: id="aloqa" bo'lgan element -->
<section id="aloqa">
  <h2>Biz bilan bog'laning</h2>
</section>
```

Nega ishlaydi? Brauzer `#aloqa` ni ko'rib, sahifada `id="aloqa"` bo'lgan elementni qidiradi va o'sha joyga aylantirib (scroll qilib) boradi. Sahifa **qayta yuklanmaydi** — shuning uchun juda tez.

```
<!-- Sahifaning eng tepasiga qaytish uchun maxsus havola -->  
<a href="#">Yuqoriga</a>
```



Fragmentni absolyut yoki nisbiy yo'l bilan ham birlashtirsa bo'ladi:

`href="about.html#jamo"` — `about.html` sahifasini ochib, darhol `id="jamo"` joyiga sakraydi.

Maxsus protokollar: `mailto:` va `tel:`

Havola faqat sahifalarga emas, harakatlarga ham olib borishi mumkin:

```
<!-- Bosilganda foydalanuvchining email dasturini ochadi -->  
<a href="mailto:info@example.com">Bizga yozing</a>  
  
<!-- Telefon (asosan mobil qurilmalarda) — bosilsa qo'ng'iroq oynasi  
ochiladi -->  
<a href="tel:+998901234567">+998 90 123 45 67</a>
```

`mailto:` ga qo'shimcha ma'lumotlar ham berish mumkin (mavzu, matn):

```
<a href="mailto:info@example.com?subject=Savol&body=Salom, menda savol  
bor">  
  Savol berish  
</a>
```



`tel:` raqamida bo'sh joy yoki qavs ishlatmang, faqat `+` va raqamlar:

`tel:+998901234567`. Lekin **ko'rinadigan** matnda chiroyli formatlasangiz bo'ladi.

3.3 `target` va xavfsizlik: yangi tabda ochish

Odatda havola bosilganda yangi sahifa **xuddi shu tabda** ochiladi (eski sahifa o'rnini egallaydi). Agar yangi tabda ochilishini istasangiz, `target="_blank"` ishlatasiz:

```
<a href="https://example.com" target="_blank">Yangi tabda ochish</a>
```

Nega kerak? Ko'pincha **tashqi** saytlarga havola berganda yangi tabda ochasiz — shunda foydalanuvchi sizning saytingizni yo'qotmaydi. Lekin shu yerda muhim **xavfsizlik** masalasi bor.

`rel="noopener"` — nega bu muhim?

`target="_blank"` bilan ochilgan yangi tab, eski versiyalarda, JavaScript orqali **sizning sahifangizni boshqarib** olishi mumkin edi (`window.opener` deb ataluvchi bog'lanish orqali). Yomon niyatli sayt sizning tabingizni soxta "parolni kiriting" sahifasiga yo'naltirib, foydalanuvchini aldab qo'yishi mumkin edi. Bu hujum **tabnabbing** deb ataladi.

target="_blank" va rel="noopener"



Yechim — `rel="noopener"` atributi. U yangi tabning eski tabga bo'lgan bog'lanishini **uzadi** (`window.opener = null` qiladi):

```
<a href="https://example.com" target="_blank" rel="noopener  
noreferrer">  
  Xavfsiz tashqi havola  
</a>
```

- `noopener` — yangi tab sizning sahifangizni boshqara olmasligini ta'minlaydi (asosiy himoya).
- `noreferrer` — qo'shimcha: yangi saytga "qaysi sahifadan keldingiz" ma'lumotini ham yubormaydi.

✦ **Yaxshi yangilik:** zamonaviy brauzerlar `target="_blank"` ishlatilganda `noopener` ni **avtomatik** qo'shadi. Lekin baribir uni o'zingiz yozish — **yaxshi odat** (eski brauzerlar va aniqlik uchun). Ko'pchilik kod tekshiruvchi vositalar uni talab qiladi.

⚠️ Har bir havolaga `target="_blank"` qo'yaverish — yomon amaliyot. Foydalanuvchining "orqaga" tugmasi ishlamay qoladi va o'nlab tablar ochilib ketadi. Faqat **kerakli** joylarda (asosan tashqi saytlar yoki PDF/hujjat) ishlating.

3.4 Boshqa foydali atributlar: `download` va `title`

download — faylni yuklab olish

Odatda brauzer ochib ko'ra oladigan fayllarni (rasm, PDF) **ochadi**. Agar uni darhol **yuklab olishga** majburlamoqchi bo'lsangiz, `download` atributini qo'ying:

```
<!-- Bosilganda fayl ochilmaydi, balki yuklab olinadi -->  
<a href="hujjat.pdf" download>Hujjatni yuklab olish</a>  
  
<!-- Yuklanayotgan faylga boshqa nom berish -->  
<a href="report-2026-final-v3.pdf" download="hisobot.pdf">Hisobotni  
olish</a>
```

✦ `download` faqat **sizning saytingizdagi** (yoki ruxsat berilgan) fayllar uchun ishlaydi — xavfsizlik sababli boshqa domendagi fayllarga ta'sir qilmasligi mumkin.

title — qo'shimcha ma'lumot (tooltip)

`title` atributi sichqoncha havola ustida turganda paydo bo'ladigan kichik izoh (tooltip) beradi:

```
<a href="hisobot.pdf" title="2026-yil yillik hisoboti, PDF, 2  
MB">Hisobot</a>
```

⚠ `title` ga **ortiqcha tayanmang**. U mobil qurilmalarda umuman ko'rinmaydi va ekran o'qiydigan dasturlar (screen reader) uni ishonchli o'qimaydi. Muhim ma'lumotni faqat `title` ga yashirib qo'ymang — uni ko'rinadigan matnga ham yozing.

3.5 Rasmlar: `<img>` elementi

Endi mediaga o'tamiz. Sahifaga rasm qo'yish uchun `<img>` elementi ishlatiladi. U ikki muhim narsa bilan ajralib turadi:

1. **Bo'sh element** (void) — yopilish tegi yo'q. Faqat `<img ...>`.
2. **Replaced element** — uning "ichida" hech narsa yozilmaydi; brauzer o'rniga rasmni "joylashtiradi".

```

```

Bu yerda ikki **majburiy** atribut bor: `src` va `alt`. Ularni alohida ko'rib chiqamiz.

`src` — rasm manzili

`src` (source — manba) — bu rasm faylining yo'li. U havoladagi `href` kabi ishlaydi: absolyut yoki nisbiy bo'lishi mumkin.

```
      <!-- nisbiy -->
 <!-- absolyut -->
```

`alt` — muqobil matn (eng muhim atribut!)

`alt` (alternative text — muqobil matn) — bu rasmni **so'z bilan tasvirlash**. Agar rasm yuklanmasa yoki foydalanuvchi ko'ra olmasa, shu matn ishlatiladi.

Nega `alt` shunchalik muhim? Uch sabab bor:

1. **Accessibility (qulaylik):** ko'zi oqiz foydalanuvchilar ekran o'qiydigan dastur (screen reader) ishlatadi. Bu dastur rasmni "ko'rolmaydi" — u faqat `alt` matnini ovoz chiqarib o'qiydi. `alt` bo'lmasa, foydalanuvchi rasmda nima borligini umuman bilolmaydi.
2. **Rasm yuklanmaganda:** internet sekin bo'lsa yoki fayl topilmasa, brauzer rasm o'rniga `alt` matnini ko'rsatadi — foydalanuvchi nima bo'lishi kerakligini tushunadi.
3. **SEO (qidiruv tizimlari):** Google rasmni "ko'rmaydi", lekin `alt` matnini o'qiydi. Yaxshi `alt` — rasm qidiruvda topilishiga yordam beradi.

```
<!-- ✓ Yaxshi: aniq, mazmunli -->


<!-- X Yomon: ma'nosiz -->


```

💡 **`alt` qanday yoziladi?** Rasmni telefonda ko'ra olmagan do'stingizga tasvirlab beryapsiz deb tasavvur qiling. "Rasm" yoki "foto" deb yozish foydasiz — rasmda **nima** borligini ayting.

🚫 **Bezak (dekorativ) rasmlar uchun bo'sh `alt`**. Agar rasm faqat bezak bo'lsa va hech qanday ma'no tashimasa (masalan, ajratuvchi chiziq), `alt=""` (bo'sh) yozing. Bunda screen reader uni butunlay e'tiborsiz qoldiradi. ⚠️ Lekin `alt` ni **butunlay tashlab ketmang** — bo'sh `alt=""` bilan `alt` yo'qligi har xil narsa. Yo'qligida screen reader fayl nomini o'qib, foydalanuvchini chalkashtiradi.

`width` va `height` — nega ularni yozish kerak?

`width` (kenglik) va `height` (balandlik) atributlari rasmning o'lchamini **piksellarda** beradi:

```

```

Ko'pchilik buni "ortiqcha" deb o'ylaydi, chunki o'lchamni CSS bilan ham berish mumkin. Lekin **nega baribir HTML'da yozish kerak?**

Sabab — **layout shift** (sahifaning sakrashi) muammosi. Tasavvur qiling: matn yuklandi, lekin rasm hali kelmadi. Brauzer rasm **qanchalik joy** egallashini bilmaydi, shuning uchun unga joy ajratmaydi. Rasm yuklanganda esa "birdan" paydo bo'lib, pastdagi matnni surib yuboradi — foydalanuvchi o'qiyotgan joyi sakrab ketadi. Bu juda bezovta qiladi (va Google ham buni "yomon tajriba" deb baholaydi).

Agar `width` va `height` ni oldindan bersangiz, brauzer rasmga **kerakli joyni oldindan ajratadi** — rasm kelguncha joy bo'sh turadi, kelganda esa hech narsa sakramaydi.

```
<!-- Brauzer 16:9 nisbatni hisoblab, joyni oldindan band qiladi -->

```

💡 **Muhim nozik nuqta:** `width` va `height` ni yozsangiz ham, CSS bilan rasmni moslashuvchan qila olasiz. Ushbu CSS responsive saytlarda deyarli har doim ishlatiladi:

```
img {
  max-width: 100%; /* konteynerdan keng bo'lib ketmasin */
  height: auto;    /* nisbatni saqlab, balandlik o'zi moslashsin */
}
```

Bunda HTML'dagi raqamlar **nisbatni** (aspect ratio) belgilaydi, CSS esa real ko'rinadigan o'lchamni boshqaradi. Eng zo'r kombinatsiya.

`loading="lazy"` — kerak bo'lganda yuklash

Uzun sahifada o'nlab rasm bo'lsa, ularning hammasini birdan yuklash sahifani sekinlashtiradi. `loading="lazy"` brauzerga "bu rasmni faqat foydalanuvchi unga **yaqinlashganda** yukla" deb aytadi:

```

```

Nega kerak? Foydalanuvchi sahifaning pastiga umuman tushmasligi mumkin. Unda pastdagi rasmlarni yuklash — bekorga sarflangan trafik va vaqt. `lazy` bilan faqat

ko'rinadigan (yoki yaqinlashayotgan) rasmlar yuklanadi.

⚠ **Diqqat:** sahifaning eng tepasidagi (birinchi ko'rinadigan) muhim rasmga `loading="lazy"` **qo'ymang** — u darhol kerak, kechiktirish faqat zarar qiladi. `lazy` — sahifaning pastidagi rasmlar uchun.

3.6 Responsive rasmlar: `srcset` va `sizes`

Bitta muammo: telefon ekrani kichik (masalan 360px keng), desktop monitori esa katta (1920px+). Agar siz hammaga bitta katta `2000px` rasm bersangiz, telefon foydalanuvchisi o'sha ulkan faylni **bekorga** yuklaydi — sekin va mobil internet trafigini behuda sarflaydi. Aksincha, kichik rasmni katta ekranda ko'rsatsangiz, u xira (pixellashgan) chiqadi.

Yechim: bir nechta o'lchamdagi rasm tayyorlab, brauzerga "o'zing eng mosini tanla" deyish. Buni `srcset` qiladi.

`srcset` — bitta rasm, turli fayllar

Brauzer ekran kengligiga qarab eng mos faylni o'zi tanlaydi



Natija: kichik ekranlar kichik faylni yuklaydi — tezroq, internet trafigi tejaladi

`srcset` — variantlar ro'yxati

```

```

Buni qism-qism ochib beramiz:

- **src** — "zaxira" rasm. **srcset** ni tushunmaydigan juda eski brauzerlar shuni ishlatadi.
- **srcset** — vergul bilan ajratilgan variantlar ro'yxati. Har biri: `fayl-nomi <kenglik>w`. Bu yerda `640w` "bu rasmning haqiqiy kengligi 640 piksel" degani (`px` emas, `w` — width descriptor).
- **sizes** — rasm ekranda **qancha joy** egallashini brauzerga aytadi.

sizes ni qanday o'qish kerak?

`sizes="(max-width: 700px) 100vw, 50vw"` ni so'zlar bilan o'qisak:

"Agar ekran 700px dan tor bo'lsa, rasm ekran kengligining **100%** ini (`100vw`) egallaydi. Aks holda — **50%** ini (`50vw`)."

(`vw` = "viewport width" — ekran kengligining foizi. `100vw` = to'liq ekran kengligi.)

Brauzer qanday tanlaydi? U ikki narsani biladi: 1. Ekran/qurilma o'lchami (va piksel zichligi). 2. **sizes** dan — rasm qancha joy egallashi.

Shulardan kelib chiqib, **srcset** dagi variantlardan **eng mosini** o'zi tanlaydi. Sizning ishingiz — variantlarni va o'lchamlarni to'g'ri berish. Qaror brauzerniki, chunki u foydalanuvchining real holatini (ekran, internet tezligi) biladi.

💡 **Ikki xil holat:** - **Faqat o'lcham farq qilsa** (turli ekranlarga turli o'lchamdagi *bir xil* rasm) — **srcset** + **sizes** yetarli, yuqoridagidek. - **Rasm o'zi boshqacha bo'lishi kerak bo'lsa** (masalan telefonda kesilgan, fokuslangan variant) — keyingi bo'limdagi `<picture>` kerak.

3.7 `<picture>` — art direction va format tanlash

`<picture>` elementi **srcset** dan ham kuchliroq nazorat beradi. U ikki vazifani bajaradi:

1. **Art direction** — turli ekranlarda **butunlay boshqacha** rasm ko'rsatish (faqat o'lcham emas).
2. **Format tanlash** — brauzer qo'llab-quvvatlasa, zamonaviy (kichikroq) formatni berish (`webp`, `avif`).

`<picture>` ichida bir nechta `<source>` va oxirida bitta `<img>` bo'ladi. Brauzer `<source>` larni **yuqoridan pastga** tekshiradi va **birinchi mos kelganini** ishlatadi. Hech biri mos kelmasa — `<img>` ga tushadi (u **majburiy**, "zaxira" sifatida).

Art direction misoli

```
<picture>
  <!-- Tor ekran (telefon): vertikal, yaqindan olingan rasm -->
  <source media="(max-width: 600px)" srcset="portret-mobil.jpg">

  <!-- Aks holda (desktop): keng, panoramali rasm -->
  <source media="(min-width: 601px)" srcset="panorama-desktop.jpg">

  <!-- Zaxira: barcha brauzerlar tushunadi -->
  
</picture>
```

Nega kerak? Keng panoramali rasm telefonning tor ekranida juda kichik va tushunarsiz bo'lib qoladi. Telefon uchun yaqindan olingan, vertikal variant berish — ancha yaxshi tajriba.

Format tanlash (webp / avif)

`webp` va `avif` — zamonaviy rasm formatlari. Ular `jpg` / `png` ga qaraganda bir xil sifatda **ancha kichikroq** fayl beradi (tezroq yuklanadi). Lekin juda eski brauzerlar ularni tushunmasligi mumkin. `<picture>` bilan "tushunsang yangisini, tushunmasang eskisini" deya olamiz:

```
<picture>
  <source type="image/avif" srcset="rasm.avif">
  <source type="image/webp" srcset="rasm.webp">
  
</picture>
```

Brauzer `avif` ni tushunsa — uni oladi (eng kichik). Tushunmasa, `webp` ga, u ham bo'lmasa — eski `jpg` ga tushadi. Foydalanuvchi har doim ishlaydigan variant oladi, qo'llab-quvvatlovchilar esa eng tejamkorini.

✦ **srcset vs <picture>** — qaysi birini ishlataman? - Faqat **o'lcham** uchun (bir xil rasm, turli kattalik) → `<img srcset sizes>`. Soddaroq. - **Boshqa rasm yoki format tanlash** kerak bo'lsa → `<picture>`.

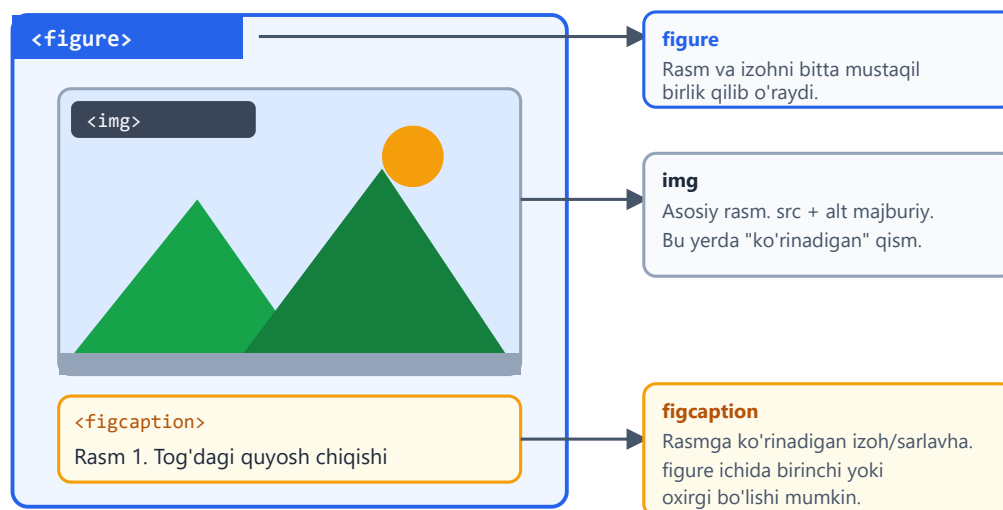
3.8 <figure> va <figcaption>

Ko'pincha rasmga **izoh** (caption) kerak bo'ladi — "Rasm 1. ...". Buni shunchaki `<p>` bilan yozsa bo'lardi, lekin u holda brauzer va qidiruv tizimlari rasm bilan izohning **bog'liqligini** bilmaydi.

`<figure>` elementi rasmni va uning izohini **bitta mustaqil birlik** sifatida o'raydi. Izoh esa `<figcaption>` ichida yoziladi.

<figure> anatomiyasi

Rasm + izoh bir butun sifatida guruhlanadi



```
<figure>
  
  <figcaption>Rasm 1. 2026-yilda sotuvlar 40% ga oshdi.</figcaption>
</figure>
```

Nega <figure> ishlatamiz (oddiy o'rniga)?

- **Semantik bog'lanish:** brauzer va screen reader izoh aynan shu rasmga tegishli ekanini biladi.
- **Mustaqillik:** `<figure>` — "mustaqil" mazmun. Uni matn oqimidan chiqarib, yon tomonga qo'ysangiz ham, ma'no buzilmaydi (xuddi kitobdagi rasm-jadvallar kabi).
- `<figcaption>` rasmning **tepasida** yoki **pastida** bo'lishi mumkin (`<figure>` ichida birinchi yoki oxirgi element).

💡 `<figure>` faqat rasm uchun emas! U kod parchasi, jadval, diagramma, hatto she'r uchun ham ishlatiladi — har qanday "mustaqil, izohli mazmun" uchun.

⚠️ `alt` va `<figcaption>` **bir xil emas**. `alt` — rasmni *tasvirlaydi* (ko'rolmaganlar uchun). `figcaption` — ko'rinadigan *izoh/sarlavha* (hammaga). Ularni bir xil qilib takrorlamang; bir-birini to'ldirsin.

3.9 Video: `<video>` elementi

Sahifaga video qo'yish uchun `<video>` ishlatiladi:

```
<video controls width="640" height="360" poster="muqova.jpg">
  <source src="dars.mp4" type="video/mp4">
  <source src="dars.webm" type="video/webm">
  <p>Brauzeringiz videoni qo'llab-quvvatlamaydi.
    <a href="dars.mp4">Videoni yuklab oling.</a></p>
</video>
```

Asosiy atributlar:

- `controls` — ijro tugmalarini (play, pauza, ovoz, vaqt) ko'rsatadi. ⚠️ Buni unutsangiz, foydalanuvchi videoni boshqara olmaydi!
- `width` / `height` — o'lchami (yana layout shift oldini oladi).
- `poster` — video boshlangunga qadar ko'rsatiladigan "muqova" rasmi.

Ichidagi `<source>` elementlari — `srcset` / `picture` dagi mantiqqa o'xshaydi: brauzer **birinchi qo'llab-quvvatlaydigan** formatni tanlaydi. `mp4` deyarli hamma joyda ishlaydi, `webm` esa kichikroq fayl. Eng oxiridagi matn/havola — video umuman ishlamasa ko'rinadigan "zaxira".

Boshqa foydali atributlar:

```
<!-- Avtomatik boshlanadigan, ovozsiz, takrorlanadigan fon videosi -->
<video autoplay muted loop playsinline width="100%">
  <source src="fon.mp4" type="video/mp4">
</video>
```

- `autoplay` — sahifa ochilishi bilan o'zi boshlanadi.
- `muted` — ovozsiz. ⚠️ **Muhim:** brauzerlar **ovozli** avtoijroni bloklaydi (foydalanuvchini bezovta qilmaslik uchun). Shuning uchun `autoplay` deyarli har doim `muted` bilan birga keladi.
- `loop` — tugagach qaytadan boshlaydi.
- `playsinline` — mobil qurilmada videoni to'liq ekranga o'tkazmay, joyida ijro etadi.

✦ **Katta videolarni o'z saytingizda saqlamang.** Video fayllari juda og'ir va serverni sekinlashtiradi. Amalda ko'pincha YouTube/Vimeo'ga yuklab, `<iframe>` bilan joylashtiriladi (3.11-bo'limga qarang).

3.10 Audio va subtitrlar (`<track>`)

Audio (musiqa, podkast) uchun `<audio>` ishlatiladi — `<video>` ga juda o'xshash:

```
<audio controls>
  <source src="podkast.mp3" type="audio/mpeg">
  <source src="podkast.ogg" type="audio/ogg">
  <p>Brauzeringiz audioni qo'llab-quvvatlamaydi.</p>
</audio>
```

Mantiq aynan video bilan bir xil: `controls` tugmalarni ko'rsatadi, `<source>` lar — brauzer tanlaydigan formatlar.

`<track>` — subtitrlar va matnli izohlar

Video va audioni **hamma uchun qulay** qilish uchun matnli izohlar qo'shiladi. Buni `<track>` elementi qiladi:

```
<video controls width="640" height="360">
  <source src="dars.mp4" type="video/mp4">

  <!-- O'zbekcha subtitrlar (standart yoqilgan) -->
  <track src="subtitrlar-uz.vtt" kind="subtitles" srclang="uz"
label="O'zbekcha" default>

  <!-- Inglizcha subtitrlar -->
  <track src="subtitrlar-en.vtt" kind="subtitles" srclang="en"
label="English">
</video>
```

- `src` — subtitrlar fayli (`.vtt` formatida — WebVTT).
- `kind` — turi: `subtitles` (subtitrlar), `captions` (eshitish qiyinligi bo'lganlar uchun, ovozlarni ham tasvirlaydi), `descriptions` (vizual tasvir).
- `srclang` — til kodi (`uz` , `en` , `ru`).
- `label` — foydalanuvchi menyusida ko'rinadigan nom.
- `default` — qaysi subtitrlar boshida yoqilgan bo'lishini belgilaydi.

Nega kerak? Subtitrlar nafaqat eshitish qiyinligi bo'lganlarga, balki ovozsiz tomosha qiluvchilarga (masalan, jamoat transportida) va boshqa tilda gaplashuvchilarga ham

yordam beradi. Bu — accessibility'ning muhim qismi.

3.11 `<iframe>` — boshqa sahifani joylashtirish

`<iframe>` (inline frame) — bu sizning sahifangiz ichiga **boshqa veb-sahifani** "deraza" sifatida joylashtirish. Eng ko'p ishlatiladigan holat — YouTube videosi yoki Google xaritasini qo'yish:

```
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  width="560" height="315"
  title="O'quv video darsi"
  allowfullscreen
  loading="lazy">
</iframe>
```

- `src` — joylashtiriladigan sahifa manzili.
- `title` — ⚠ accessibility uchun **muhim**: screen reader iframe nima ekanini shu orqali tushuntiradi. Har doim yozing.
- `allowfullscreen` — to'liq ekranga o'tkazishga ruxsat (videolar uchun).
- `loading="lazy"` — iframe og'ir bo'lgani uchun, kerak bo'lganda yuklash foydali.

Xavfsizlik: `sandbox`

⚠ **Diqqat**: iframe ichidagi sahifa — bu **begona** kod. Agar ishonchsiz manbadan sahifa joylashtirsangiz, u sizning sahifangizga zarar yetkazishga urinishi mumkin (oynalarni ochish, yo'naltirish va h.k.).

`sandbox` atributi iframe'ni "qamoqqa oladi" — unga faqat **siz ruxsat bergan** narsalarga yo'l beradi:

```
<!-- Bo'sh sandbox: deyarli hamma narsa bloklangan (eng xavfsiz) -->
<iframe src="https://reklama.example.com" sandbox title="Reklama">
</iframe>

<!-- Faqat skript va formaga ruxsat -->
<iframe src="https://widget.example.com"
  sandbox="allow-scripts allow-forms"
  title="Vidget"></iframe>
```

`sandbox` ni **bo'sh** yozsangiz — eng qattiq cheklov (skript ishlamaydi, forma yuborilmaydi, navigatsiya bloklanadi). Keyin kerakli ruxsatlarni bittalab qo'shasiz (`allow-scripts`, `allow-forms`, `allow-same-origin` va h.k.).

✦ **Asosiy qoida:** `iframe`'ga faqat **kerakli minimum** ruxsat bering. "Hammasiga ruxsat berib qo'ya qolay" — xavfsizlik teshigi. Ishonchsiz manbalar uchun `sandbox` har doim ishlating.

💡 **Maslahat:** YouTube/Google kabi ishonchli xizmatlar o'zlari "embed" (joylashtirish) kodi beradi — uni shunchaki nusxa ko'chiring. O'zingiz qo'lda yozish shart emas.

Mashqlar

Quyidagi mashqlarni ketma-ket bajaring. Avval o'zingiz urinib ko'ring, keyin yechimni oching.

Mashq 1 — Uch xil havola

Bitta `index.html` faylida quyidagi uch havolani yarating: 1. Tashqi saytga (`https://developer.mozilla.org`). 2. Bir papka tashqaridagi `uy.html` faylga. 3. Shu sahifadagi `id="aloqa"` bo'limiga.

 **Yechim** 

```
<a href="https://developer.mozilla.org">MDN hujjatlari</a>
<a href=" ../uy.html">Bosh sahifa</a>
<a href="#aloqa">Aloqa bo'limi</a>

<!-- ... sahifa pastida ... -->
<section id="aloqa">
  <h2>Biz bilan bog'laning</h2>
</section>
```

Mashq 2 — Xavfsiz tashqi havola

Tashqi saytga (`https://example.com`) **yangi tabda** ochiladigan va **xavfsiz** havola yozing.

 **Yechim** 

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">
  Tashqi sayt
</a>
```

`target="_blank"` yangi tabda ochadi, `rel="noopener noreferrer"` esa tabnabbing hujumidan himoya qiladi.

Mashq 3 — `alt` ni to'g'irlash

Quyidagi rasmlar `alt` matnlari yomon. Har birini to'g'rilang:

```


```

 **Yechim** 

```
<!-- Mazmunli rasm: tasvirlaymiz -->


<!-- Bezak (dekorativ) rasm: bo'sh alt -->

```

Birinchisi mazmun tashiydi — uni tasvirlaymiz. Ikkinchisi faqat bezak — bo'sh `alt=""` bilan screen reader uni e'tiborsiz qoldiradi.

Mashq 4 — Layout shift'siz rasm

Internet sekin bo'lsa ham sahifa "sakramaydigan" qilib, `banner.jpg` (1600×900) rasmni joylashtiring. Rasm konteynerdan keng bo'lib ketmasin.

 **Yechim** 

```

```

```
img {
  max-width: 100%;
  height: auto;
}
```

HTML'dagi `width / height` brauzerga nisbatni (16:9) beradi — joy oldindan band qilinadi. CSS esa rasmni moslashuvchan qiladi.

Mashq 5 — Responsive rasm

`manzara` rasmining uch o'lchami bor: `manzara-480.jpg`, `manzara-960.jpg`, `manzara-1440.jpg`. Brauzer ekran kengligiga qarab o'zi tanlaydigan `<img>` yozing. Rasm har doim ekranning to'liq kengligini egallaydi deb faraz qiling.

 **Yechim** 

```

```

`sizes="100vw"` — rasm har doim to'liq ekran kengligini egallaydi, shuning uchun brauzer ekranga qarab eng mos faylni tanlaydi.

Mashq 6 — `<figure>` bilan izohli rasm

`jamoaj.jpg` rasmini ko'rinadigan "Rasm 1. Bizning jamoa 2026-yilda" izohi bilan, semantik to'g'ri tarzda joylashtiring.

 **Yechim** 

```
<figure>
  
  <figcaption>Rasm 1. Bizning jamoa 2026-yilda</figcaption>
</figure>
```

Diqqat: `alt` rasmni *tasvirlaydi* (ko'rolmaganlar uchun), `figcaption` esa ko'rinadigan *izoh* — ular bir xil emas.

Mashq 7 — Subtitrl video

`mahsulot.mp4` videosini ijro tugmalari bilan, o'zbekcha subtitrl (`uz.vtt`, standart yoqilgan) qo'shib joylashtiring. Muqova rasmi — `muqova.jpg`.

 **Yechim** 

```
<video controls width="640" height="360" poster="muqoval.jpg">
  <source src="mahsulot.mp4" type="video/mp4">
  <track src="uz.vtt" kind="subtitles" srclang="uz" label="O'zbekcha"
default>
  <p>Brauzeringiz videoni qo'llab-quvvatlamaydi.</p>
</video>
```

`controls` — tugmalar, `poster` — muqova, `<track ... default>` — boshida yoqilgan o'zbekcha subtitrl.

Mashq 8 — Xavfsiz iframe

Ishonchsiz `https://reklama.example.com` sahifasini `iframe` bilan joylashtiring, lekin u **faqat skript** ishlata olsun, boshqa hech narsa qila olmasin. `title` ni unutmang.



Yechim



```
<iframe
  src="https://reklama.example.com"
  title="Reklama bloki"
  sandbox="allow-scripts"
  width="300" height="250"
  loading="lazy">
</iframe>
```



`sandbox="allow-scripts"` — faqat skriptga ruxsat, qolgan hamma narsa (forma, navigatsiya, popup) bloklangan. Bo'sh `sandbox` dan ham qattiqroq nazorat — minimum kerakli ruxsat beriladi.



Oldingi: 02 — Matn elementlari



README



Keyingi: 04 — Jadvallar



04 — Jadvallar

← Oldingi: 03 — Havolalar va media · 🏠 README · Keyingi: 05 — Formalar →

Bu bobda: ma'lumot jadvallari: `table`, qator, ustun va birlashtirish.

Tasavvur qil: sinfdoshlarningning ismi, yoshi va shahri ro'yxatini yozyapsan. Buni daftarga qanday tushirasan? Albatta — **jadval** chizib. Yuqoriga sarlavhalarni (Ism, Yosh, Shahar), keyin har bir o'quvchiga bitta qator yozasan. Web sahifada ham xuddi shu narsa kerak bo'ladi, va HTML buni `<table>` yorlig'i bilan beradi.

Bu bobda jadvallarni eng oddiy 2x2 katakdan boshlab, birlashgan yacheykalar, sarlavha guruhlar va skrin-riderlar uchun moslashtirilgan "mukammal" jadvalgacha bosqichma-bosqich o'rganamiz. Eng muhimi — **nega** har bir element kerakligini tushunamiz, shunda yodlash emas, bilib ishlatadigan bo'lasan.

4.1 Jadval nima va u qachon kerak?

Jadval (table) — bu ma'lumotni **qatorlar (rows)** va **ustunlar (columns)** ko'rinishida tartibga soluvchi struktura. Ikkalasining kesishuvi **yacheyka (cell)** deyiladi.

Jadvalning bosh qoidasi bitta:

✦ Jadval faqat **tabular data** (ikki o'lchovli ma'lumot) uchun. Ya'ni har bir qiymat ham bir qatorga, ham bir ustunga tegishli bo'lsa.

Jadvalga **mos** keladigan misollar: - Mahsulotlar narxi: har qatorda mahsulot, ustunlarda nom / narx / miqdor. - Dars jadvali: qatorlar — soatlar, ustunlar — hafta kunlari. - Moliyaviy hisobot: oylar va daromad/xarajat.

Jadvalga **mos kelmaydigan** misollar: - Bitta ro'yxat (faqat ismlar) — bu uchun `<ul>` yoki `<ol>` to'g'riroq. - Bitta xatboshi matn — bu `<p>`.

"Layout uchun jadval ishlatma" — nega?

Yillar oldin (taxminan 1998–2008) dasturchilar butun sahifa **ko'rinishini** (chap menyu, o'rtada kontent, o'ngda reklama) jadvallar bilan yasashgan. Bugun bu **katta xato** hisoblanadi. Sabablarini bilib qo'y, chunki bu eng muhim "nega"lardan biri:

1. **Ma'no buziladi (semantika).** `<table>` brauzer va yordamchi texnologiyalarga "bu ikki o'lchovli ma'lumot" deb aytadi. Agar sen u bilan tugmalar va logotipni joylashtirsang, skrin-rider (ko'rlar uchun ovozli o'qigich) "5 ustunli jadval, 1-qator, 1-ustun" deb adashtiruvchi narsalarni o'qiydi. Foydalanuvchi sahifada nima borligini tushunolmaydi.
2. **Moslashuvchanlik yo'q (responsive).** Jadval qatorlari telefon ekranida bukila olmaydi. Layout jadval bilan qurilsa, kichik ekranda hammasi siqilib, o'qib bo'lmaydigan holga keladi.
3. **Tuzatish qiyin.** Layout jadvallari ichma-ich joylashgan o'nlab `<tr>` / `<td>` ga aylanadi — keyinchalik o'zgartirish azobga aylanadi.

💡 **Yodda tut:** Sahifa **ko'rinishi** (joylashuv) uchun `CSS` (Flexbox, Grid — keyingi boblarda) bor. `<table>` esa faqat **ma'lumot** uchun. Bu ikkisini aralashtirma.

⚠️ Oddiy tekshiruv: "Bu ma'lumotni Excel jadvaliga joylab, ustun sarlavhalari mantiqan to'g'ri chiqar miydi?" Agar ha — `<table>` to'g'ri tanlov. Agar yo'q — boshqa element kerak.

4.2 Eng oddiy jadval: `table`, `tr`, `td`, `th`

Jadval to'rt asosiy yorliqdan quriladi. Ularni esda saqlash uchun ingliz so'zlarini bilib qo'yamiz:

Yorliq	Inglizcha	Vazifasi
<code><table></code>	table	Butun jadvalni o'rab turadi (idish).
<code><tr></code>	table row	Bitta qator (gorizontal chiziq).
<code><td></code>	table data	Oddiy yacheyka (ma'lumot katagi).
<code><th></code>	table header	Sarlavha yacheyka (ustun/qator nomi).

Tartibni eslab qol: `<table>` ichida `<tr>` (qatorlar) bo'ladi, har bir `<tr>` ichida esa `<td>` yoki `<th>` (yacheykalar) bo'ladi. Bu **ichma-ich** (nested) tuzilma.

Mana eng oddiy jadval — 3 ustun, 3 qator:

```
<table>
  <tr>
```



```

<th>Ism</th>
<th>Yosh</th>
<th>Shahar</th>
</tr>
<tr>
<td>Ali</td>
<td>25</td>
<td>Buxoro</td>
</tr>
<tr>
<td>Lola</td>
<td>30</td>
<td>Toshkent</td>
</tr>
</table>

```

Natija: brauzer 3 qatorli jadval chizadi. Birinchi qatordagi `<th>` matnlari odatda **qalin (bold)** va **markazlangan** ko'rinadi (bu brauzerning standart uslubi), `<td>` esa oddiy va chapga tekislangan.

✦ Diqqat: standart holatda jadvalda **chiziqlar (border) ko'rinmaydi!** Yacheykalar bir-biriga yopishib turadi. Chiziqlarni biz keyinroq (4.9) CSS bilan qo'shamiz. Hozircha strukturaga e'tibor ber.

`<th>` va `<td>` — farqi shunchaki ko'rinishda emas

Yangi boshlovchilar ko'pincha "`<th>` shunchaki qalin `<td>` -ku, har joyda `<td>` ishlataveraman" deb o'ylaydi. Bu xato. Farq **ma'noda**:

- `<th>` brauzer va skrin-riderga "**bu sarlavha, qolgan yacheykalarni tavsiflaydi**" deb aytadi.
- `<td>` esa oddiy ma'lumot.

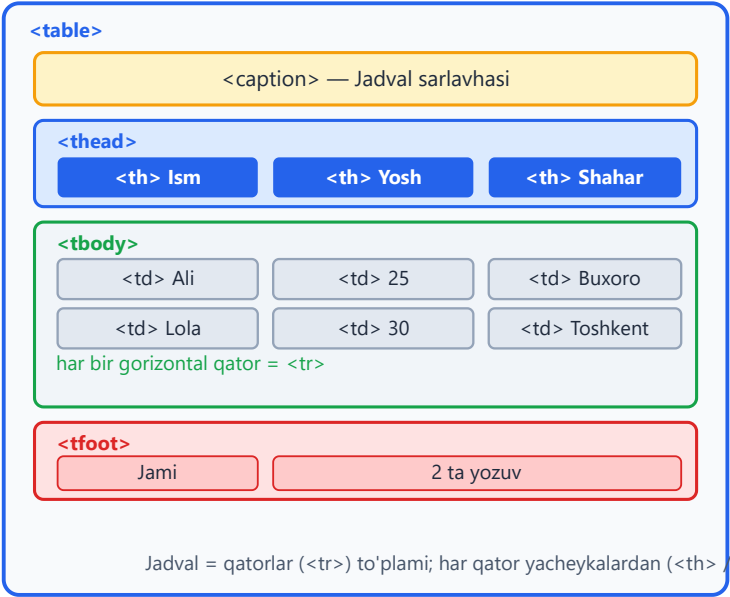
Skrin-rider `<td>` ni o'qiganda, unga tegishli `<th>` ni ham aytadi: masalan "Yosh: 25". Agar sarlavhalarini `<td>` qilib qo'ysang, ko'r foydalanuvchi "25" sonining nimaga tegishli ekanini bilolmaydi. Shuning uchun sarlavhalar **doim** `<th>` bo'lishi shart.

💡 `<th>` ni faqat yuqori qatordagina emas, chap ustunda ham ishlatish mumkin (masalan har qatorning "ismi" — bu qatorning sarlavhasi). Bu haqda 4.5 da gaplashamiz.

4.3 Jadval anatomiyasi: `caption`, `thead`, `tbody`, `tfoot`

Yuqoridagi oddiy jadval ishlaydi, lekin "professional" jadvalning yana to'rtta qismi bor. Ular jadvalni **mantiqiy bo'laklarga** ajratadi:

Jadval anatomiyasi (<table>)



Belgilar

- <th>** sarlavha yacheyka
- <td>** oddiy ma'lumot

- **<caption>** = nom
- **<thead>** = bosh
- **<tbody>** = tana
- **<tfoot>** = oyoq

Tartib (HTML):
caption → thead
→ tbody → tfoot

brauzer tfoot ni
baribir pastda chizadi

Element	Vazifasi
<code><caption></code>	Jadvalning nomi/sarlavhasi (butun jadval nima haqida ekani).
<code><thead></code>	Jadval boshi — sarlavha qatori(lari) shu yerda.
<code><tbody></code>	Jadval tanasi — asosiy ma'lumot qatorlari.
<code><tfoot></code>	Jadval oyog'i — yakuniy qator (masalan "Jami").

<caption> — jadvalga nom berish

`<caption>` `<table>` ning **ENG BIRINCHI** bolasi bo'lishi shart. U jadval ustida nom sifatida ko'rinadi:

```
<table>
  <caption>2026-yil o'quvchilar ro'yxati</caption>
  <tr>
    <th>Ism</th>
    <th>Yosh</th>
  </tr>
  <tr>
    <td>Ali</td>
    <td>25</td>
```

```
</tr>
</table>
```

Natija: jadval ustida "2026-yil o'quvchilar ro'yxati" matni chiqadi.

Nega kerak? Skrimin-rider foydalanuvchisi jadvalga kirganda darhol "Bu jadval nima haqida?" deb eshitadi. Sarlavhasiz jadval — bu nomsiz papka kabi. `<caption>` ni `<h3>` bilan almashtirish mumkin emas, chunki `<caption>` jadval bilan **bog'langan** (krimin-rider buni jadval nomi deb biladi).

`<thead>`, `<tbody>`, `<tfoot>` — qatorlarni guruhlash

Bu uchta element qatorlarni mantiqiy guruhlarga ajratadi:

```
<table>
  <caption>Oylik byudjet</caption>
  <thead>
    <tr>
      <th>Modda</th>
      <th>Summa</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ovqat</td>
      <td>1 200 000</td>
    </tr>
    <tr>
      <td>Transport</td>
      <td>300 000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th>Jami</th>
      <td>1 500 000</td>
    </tr>
  </tfoot>
</table>
```

Nega bu guruhlar foydali?

1. **CSS bilan bezash oson.** "Faqat sarlavhaga kulrang fon ber" desang, `thead { ... }` deb bitta joyga yozasan — har bir `<th>` ni alohida belgilamaysan.
2. **Uzun jadvalni chop etish.** Agar jadval bir necha sahifaga cho'zilsa, brauzer `<thead>` ni har bir sahifa tepasida **takrorlaydi**, shunda ustun nomlari ko'zdan qochmaydi.

3. **Aniqlik.** Kod o'qiganga jadvalning qaysi qismi sarlavha, qaysi qismi ma'lumot ekani darhol ko'rinadi.

📌 **Qiziq nuqta — tartib:** HTML kodida tartib `caption` → `thead` → `tbody` → `tfoot` bo'lishi tavsiya etiladi. Agar `<tfoot>` ni `<tbody>` dan **oldin** yozsang ham, brauzer uni **baribir pastda** ko'rsatadi — chunki `tfoot` ning ma'nosi "oyoq" bo'lib, joylashuvi mazmuni bilan belgilanadi. Lekin chalkashmaslik uchun mantiqiy tartibda yoz.

💡 `<thead>` / `<tbody>` / `<tfoot>` ichida ham `<tr>` bo'ladi — bu guruhlar `<tr>` ni o'rab turuvchi qo'shimcha qatlam, xolos. `<tbody>` ichida bir nechta `<tr>` bo'lishi mumkin (odatda shunday).

4.4 Yacheykalarni birlashtirish: `colspan` va `rowspan`

Ba'zan bitta yacheyka **bir nechta** ustun yoki qatorni egallashi kerak bo'ladi. Masalan, ikkita ustunning umumiy sarlavhasi, yoki bir necha qator uchun bitta umumiy katak. Buning uchun ikkita atribut bor:

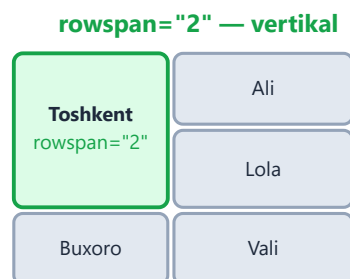
- `colspan="N"` — yacheyka **N ta ustunni** gorizontal egallaydi (column span).
- `rowspan="N"` — yacheyka **N ta qatorni** vertikal egallaydi (row span).

Yacheykalarni birlashtirish



Bitta yacheyka 2 ustunni egallaydi

```
<tr>
  <th colspan="2">Sarlavha
</tr>
<tr>
  <td>A1</td> <td>B1</td>
</tr>
```



Bitta yacheyka 2 qatorni egallaydi

```
<tr>
  <td rowspan="2">Toshkent
  <td>Ali</td> </tr>
<tr>
  <td>Lola</td>
  ⚠ bu qatorda 1 ta td!
```

`colspan` — gorizontal birlashtirish

```
<table>
  <tr>
```



```

    <th colspan="2">Shaxsiy ma'lumot</th>
  </tr>
  <tr>
    <td>Ali</td>
    <td>25 yosh</td>
  </tr>
</table>

```

Natija: birinchi qatorda "Shaxsiy ma'lumot" matni ikkala ustun ustidan cho'zilib turadi. Pastda esa ikkita alohida yacheyka.

E'tibor ber: birinchi `<tr>` ichida atigi **bitta** `<th>` bor, lekin u `colspan="2"` tufayli ikkita ustun enini egallaydi. Agar adashib u yerga yana bitta yacheyka qo'shsang, jadval 3 ustun keng bo'lib ketadi.

rowspan — vertikal birlashtirish

```

<table>
  <tr>
    <td rowspan="2">Toshkent</td>
    <td>Ali</td>
  </tr>
  <tr>
    <td>Lola</td>
  </tr>
</table>

```

Natija: "Toshkent" yacheykasi ikkita qator balandligini egallaydi. Uning yonida birinchi qatorda "Ali", ikkinchi qatorda "Lola" bo'ladi.

⚠ **Eng ko'p uchraydigan xato:** ikkinchi `<tr>` ga e'tibor ber — unda **faqat bitta** `<td>` bor ("Lola")! Chunki chap ustunni "Toshkent" yacheykasi `rowspan` bilan allaqachon egallab turibdi. Agar bu yerga ham ikkita `<td>` yozsang, jadval qiyshayib, ortiqcha katak paydo bo'ladi.

💡 **Oltin qoida:** `rowspan="N"` qo'ygan yacheykadan keyingi `N-1` ta qatorda — o'sha ustun uchun yacheyka YOZMA. Birlashgan yacheyka o'sha joyni o'zi to'ldiradi.

Ikkalasini birga ishlatish

Murakkab jadvallarda `colspan` va `rowspan` birga kelishi mumkin. Mana misol — fan bo'yicha imtihon natijasi:

```

<table>
  <caption>Imtihon natijalari</caption>
  <thead>
    <tr>

```

```

    <th rowspan="2">Talaba</th>
    <th colspan="2">Imtihon</th>
  </tr>
  <tr>
    <th>Yozma</th>
    <th>Og'zaki</th>
  </tr>
</thead>
<tbody>
  <tr>
    <td>Ali</td>
    <td>85</td>
    <td>90</td>
  </tr>
</tbody>
</table>

```

Natija: "Talaba" sarlavhasi ikki qator balandlikda (`rowspan="2"`), "Imtihon" esa ikki ustun kengligida (`colspan="2"`) bo'lib, uning ostida "Yozma" va "Og'zaki" pastki sarlavhalari joylashadi. Bu — ikki darajali (ichma-ich) sarlavha tuzilmasi.

✦ Hisoblash maslahati: chalkashib ketmaslik uchun jadvalni **qog'ozda chizib ol**. Har bir katakni belgila, qaysilari birlashganini ko'rsat, keyin kodga ko'chir. Tajriba ortgach buni xayolan qila olasan.

4.5 Accessibility (kirish imkoniyati): `th scope`

Endi jadvalni nafaqat ko'rinadigan, balki **ko'rlar ham foydalana oladigan** qilamiz. Bu — "expert" darajaning birinchi bosqichi va ko'pincha unutiladigan, lekin juda muhim mavzu.

Muammo shunda: oddiy `<th>` skrin-riderga "bu sarlavha" deydi, lekin **qaysi yo'nalishdagi** sarlavha ekanini aytmaydi. Ustunning sarlavhasimi yoki qatorning? `scope` atributi aynan shuni aniqlaydi:

- `scope="col"` — bu `<th>` **ustun** sarlavhasi (pastdagi butun ustunga tegishli).
- `scope="row"` — bu `<th>` **qator** sarlavhasi (o'ngdagi butun qatorga tegishli).

th scope — sarlavha qaysi yacheykalarga tegishli?

	Yosh scope="col"	Shahar scope="col"
Ali scope="row"	25	Buxoro
Lola scope="row"	30	Toshkent

scope="col" → pastdagi butun ustunga tegishli

< scope="row" → o'ngdagi butun qatorga tegishli

Skrin-rider "Lola, Yosh: 30" deb o'qiydi — har son qaysi sarlavhaga oid ekani aniq
scope yacheyka va sarlavhani bog'laydi — ko'r foydalanuvchilar uchun zarur

```
<table>
  <caption>0' quvchilar</caption>
  <thead>
    <tr>
      <td></td>
      <th scope="col">Yosh</th>
      <th scope="col">Shahar</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Ali</th>
      <td>25</td>
      <td>Buxoro</td>
    </tr>
    <tr>
      <th scope="row">Lola</th>
      <td>30</td>
      <td>Toshkent</td>
    </tr>
  </tbody>
</table>
```

Nega bu ajoyib? Skrin-rider "30" yacheykasini o'qiganda, uning **ustun** sarlavhasi ("Yosh") va **qator** sarlavhasi ("Lola") ni topib, "Lola, Yosh: 30" deb o'qiydi. Foydalanuvchi har bir sonning aniq ma'nosini biladi — ko'zlari bilan jadvalni "ko'rmasa" ham.

E'tibor ber: chap yuqori burchakdagi bo'sh katak `<td></td>` qoldirildi (u na ustun, na qator sarlavhasi — shunchaki bo'sh burchak).

💡 **Soddagina qoida:** har bir oddiy jadvalda yuqori qatordagi `<th>` larga `scope="col"`, chap ustundagi `<th>` larga `scope="row"` qo'y. Bu deyarli barcha jadvallar uchun yetarli va jadvalingni to'liq accessible qiladi.

4.6 Murakkab jadvallar uchun: `headers` va `id` bog'lash

`scope` oddiy jadvallar uchun mukammal. Ammo jadval **murakkab** bo'lsa — masalan birlashgan, ikki darajali sarlavhalar bo'lsa — `scope` ba'zan yetarli aniq bog'lay olmaydi. Bunday holatlar uchun eng kuchli usul bor: har bir yacheykani uning sarlavhalari bilan **`id` orqali aniq** bog'lash.

Ishlash tartibi: 1. Har bir sarlavha `<th>` ga noyob `id` ber. 2. Har bir ma'lumot `<td>` da `headers` atributiga o'sha `id` larni (bo'shliq bilan ajratib) yoz.

```
<table>
  <caption>Mahsulot narxlari</caption>
  <tr>
    <th id="mahsulot">Mahsulot</th>
    <th id="narx">Narx</th>
  </tr>
  <tr>
    <td headers="mahsulot">Olma</td>
    <td headers="narx">12 000</td>
  </tr>
  <tr>
    <td headers="mahsulot">Non</td>
    <td headers="narx">4 000</td>
  </tr>
</table>
```

Bir yacheyka **bir nechta** sarlavhaga bog'lansa, `headers` ichiga bir nechta `id` ni bo'shliq bilan yozasan:

```
<td headers="imtihon yozma">85</td>
```

Bu "85 — Imtihon, Yozma" degani.

🚩 **`scope` ni qachon, `headers/id` ni qachon ishlatish kerak?**

Holat	Tavsiya
Oddiy jadval (bir qator/ustun sarlavha)	<code>scope="col"</code> / <code>scope="row"</code>
Murakkab, birlashgan, ko'p darajali sarlavhalar	<code>headers</code> + <code>id</code>

⚠️ `headers` / `id` ko'proq yozishni talab qiladi va `id` lar **butun sahifada noyob** bo'lishi shart. Shuning uchun keraksiz joyda ishlatma — oddiy jadvalga `scope` yetarli. "Eng oddiy yetarli usulni tanla" tamoyiliga amal qil.

4.7 Ustunlarni guruhlash: `colgroup` va `col`

Hozirgacha biz **qatorlarni** (`tr`, `thead` / `tbody`) boshqardik. Lekin ba'zan butun **ustunga** xususiyat bermoqchi bo'lamiz — masalan ikkinchi ustunni och fon bilan ajratib ko'rsatish. HTML da yacheykalar qatorma-qator yoziladi, "ustun" degan to'g'ridan-to'g'ri element yo'q. Shu bois ustunlar uchun maxsus `<colgroup>` va `<col>` mavjud.

- `<colgroup>` — ustunlar guruhini e'lon qiladi (`<table>` ichida, `<caption>` dan keyin, `<thead>` dan oldin).
- `<col>` — bitta ustunni ifodalaydi (yopiluvchi yorliq kerak emas).

```
<table>
  <caption>Narxlar jadvali</caption>
  <colgroup>
    <col>
    <col style="background-color: #f8fafc;">
    <col span="2" style="background-color: #fef3c7;">
  </colgroup>
  <thead>
    <tr>
      <th>Mahsulot</th>
      <th>Narx</th>
      <th>Soni</th>
      <th>Jami</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Olma</td>
      <td>12 000</td>
      <td>3</td>
      <td>36 000</td>
    </tr>
  </tbody>
</table>
```

Natija: ikkinchi ustun ("Narx") och kulrang fonli, uchinchi va to'rtinchi ustunlar ("Soni", "Jami") sariq fonli bo'ladi. `<col span="2">` bitta `<col>` bilan ikki ustunni qamrab oldi.

Nega foydali? Butun ustunga uslub bermoqchi bo'lsang, har bir `<td>` ga alohida `class` qo'shish o'rniga bitta `<col>` yetadi — kod qisqaradi.

⚠ **Muhim cheklov:** `<col>` ga faqat **cheklangan** CSS xususiyatlar ta'sir qiladi: `background`, `border`, `width`, `visibility`. Masalan `color` (matn rangi) yoki `font-weight` `<col>` orqali **ishlamaydi** — chunki matn yacheykaga (`<td>`) tegishli, ustunga emas. Bu narsalar uchun yacheykalarning o'ziga uslub berish kerak.

💡 `<col>` eng ko'p ishlatiladigan joyi — ustun **kengligini** (`width`) belgilash: `<col style="width: 40%;">`.

4.8 To'liq, "to'g'ri qurilgan" jadval namunasi

Endi o'rganganlarimizning hammasini bitta puxta jadvalga jamlaymiz. Bu — sen real loyihada yozishing kerak bo'lgan "etalon" jadval:

```
<table>
  <caption>2026-yil yarim yillik savdo hisoboti</caption>

  <colgroup>
    <col>
    <col span="2">
  </colgroup>

  <thead>
    <tr>
      <th scope="col">Hudud</th>
      <th scope="col">1-chorak</th>
      <th scope="col">2-chorak</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <th scope="row">Toshkent</th>
      <td>4 500</td>
      <td>5 200</td>
    </tr>
    <tr>
      <th scope="row">Samarqand</th>
      <td>3 100</td>
      <td>3 400</td>
    </tr>
  </tbody>

  <tfoot>
    <tr>
      <th scope="row">Jami</th>
      <td>7 600</td>
      <td>8 600</td>
    </tr>
  </tfoot>
</table>
```

Bu jadval nimasi bilan yaxshi? - `<caption>` — jadvalning aniq nomi bor. -

`<thead>` / `<tbody>` / `<tfoot>` — mantiqiy bo'laklarga ajratilgan. - `scope="col"` va

`scope="row"` — har bir son qaysi hudud va qaysi chorakka oid ekani skrin-riderga aniq.
- `<tfoot>` dagi "Jami" — yakuniy hisob alohida ajratilgan.

✦ Mana shu skelet — deyarli har qanday ma'lumot jadvali uchun ishlaydi. Yangi jadval yozayotganda shu namunadan boshla, keyin ustun/qatorlarni o'zingnikiga moslab o'zgartir.

4.9 Jadvalni CSS bilan bezashga kirish (oldindan eslatma)

Yuqorida aytib o'tdik: standart jadvalda chiziqlar ko'rinmaydi va yacheykalar bir-biriga yopishadi. Buni CSS hal qiladi. To'liq bezashni keyingi CSS boblarida o'rganamiz, lekin eng zarur ikki narsani hozir ko'rib qo'yamiz, chunki ularsiz jadval xunuk ko'rinadi.

```
<style>
  table {
    border-collapse: collapse; /* eng muhim qator — pastga qara */
    width: 100%;
  }
  th, td {
    border: 1px solid #94a3b8; /* har yacheykaga chiziq */
    padding: 8px 12px;        /* ichki bo'shliq */
    text-align: left;
  }
  thead {
    background-color: #dbeafe; /* sarlavhaga fon */
  }
</style>
```

`border-collapse` — nega bu eng muhim qator?

Standart holatda har bir yacheyka **o'zining alohida** chizig'iga ega. Natijada ikki qo'shni yacheyka orasida **ikkita** chiziq paydo bo'ladi (biri u yacheykadan, biri bu yacheykadan) — qo'sh chiziqli, eski ko'rinishdagi jadval chiqadi.

`border-collapse: collapse;` brauzerga "qo'shni chiziqlarni **bitta** qilib birlashtir" deydi. Natijada toza, bir piksellik chiziqli zamonaviy jadval hosil bo'ladi.

💡 Amaliyotda jadval bezayotganda deyarli **har doim** `border-collapse: collapse;` dan boshlanadi. Buni "jadval CSS ining birinchi qadami" deb yodda tut.

`padding` esa yacheyka ichidagi matnni chetidan biroz uzoqlashtiradi — matn chiziqqa yopishib turmaydi, o'qish qulay bo'ladi.

⚠ Diqqat: chiroyli ko'rinish uchun chiziqni `<table>` ga emas, balki `<th>` va `<td>` ga ber. Faqat `<table>` ga `border` bersang, ichki yacheykalar orasida chiziq bo'lmaydi —

atrofidagina ramka chiqadi.

Qolgani — ranglar, zebra (qator-ma-qator turli fon), `:hover` effektlari va responsive jadvallar — CSS boblarida batafsil. Hozircha jadvalning **strukturasini** mukammal qurishni o'rganib oldik; ko'rinish — keyingi qadam.

Xulosa

- `<table>` faqat **tabular data** (ikki o'lchovli ma'lumot) uchun; layout uchun emas (CSS Flexbox/Grid bor).
 - Asosiy yorliqlar: `<table>` (idish), `<tr>` (qator), `<td>` (ma'lumot), `<th>` (sarlavha).
 - Struktura: `<caption>` (nom), `<thead>` / `<tbody>` / `<tfoot>` (bosh/tana/oyoq) jadvalni mantiqiy bo'laklaydi.
 - `colspan` — gorizontal, `rowspan` — vertikal birlashtirish; birlashgan qatorda ortiqcha yacheyka yozma.
 - Accessibility: oddiy jadvalga `scope="col" / scope="row"`, murakkabga `headers + id`.
 - `<colgroup>` / `<col>` — butun ustunga (fon, kenglik) uslub berish uchun.
 - CSS bezashda `border-collapse: collapse;` dan boshla.
-

Mashqlar

Quyidagi mashqlarni tartib bilan bajar. Har birida avval o'zing urinib ko'r, keyin yechimni och.

Mashq 1 — Eng oddiy jadval

3 ustunli (Ism, Yosh, Kasb) va 2 ta ma'lumot qatori bo'lgan jadval yoz. Yuqori qator `<th>` lardan iborat bo'lsin.

 Yechim

```
<table>
  <tr>
    <th>Ism</th>
    <th>Yosh</th>
    <th>Kasb</th>
  </tr>
  <tr>
    <td>Ali</td>
    <td>25</td>
    <td>Dasturchi</td>
  </tr>
  <tr>
    <td>Lola</td>
    <td>30</td>
    <td>Shifokor</td>
  </tr>
</table>
```

Mashq 2 — caption qo'shish

1-mashqdagi jadvalga "Xodimlar ro'yxati" degan `<caption>` qo'sh. U `<table>` ning birinchi bolasi bo'lishini unutma.

 Yechim

```
<table>
  <caption>Xodimlar ro'yxati</caption>
  <tr>
    <th>Ism</th>
    <th>Yosh</th>
    <th>Kasb</th>
  </tr>
  <tr>
    <td>Ali</td>
    <td>25</td>
    <td>Dasturchi</td>
  </tr>
</table>
```

`<caption>` har doim `<table>` ichidagi eng birinchi element bo'lishi shart.

Mashq 3 — thead / tbody / tfoot bilan struktura

Quyidagi byudjet jadvalini to'liq strukturali (caption + thead + tbody + tfoot) holda yoz:
nom "Oylik byudjet"; sarlavhalar "Modda" va "Summa"; tana qatorlari: Ovqat — 1 200 000, Transport — 300 000; oyoq qatori: Jami — 1 500 000.

Yechim

```
<table>
  <caption>Oylik byudjet</caption>
  <thead>
    <tr>
      <th>Modda</th>
      <th>Summa</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ovqat</td>
      <td>1 200 000</td>
    </tr>
    <tr>
      <td>Transport</td>
      <td>300 000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th>Jami</th>
      <td>1 500 000</td>
    </tr>
  </tfoot>
</table>
```

Mashq 4 — `colspan` bilan umumiy sarlavha

Ikki ustunli ("Telefon", "Email") jadval yoz, lekin ularning ustida bitta umumiy "Bog'lanish" sarlavhasi `colspan` bilan ikkala ustunni qamrasin.

Yechim

```
<table>
  <tr>
    <th colspan="2">Bog'lanish</th>
  </tr>
  <tr>
    <th>Telefon</th>
    <th>Email</th>
  </tr>
  <tr>
    <td>+998 90 123 45 67</td>
    <td>ali@example.com</td>
  </tr>
</table>
```

Birinchi qatorda atigi bitta `<th>` borligiga e'tibor ber — `colspan="2"` uni ikki ustunga cho'zadi.

Mashq 5 — `rowspan` bilan birlashtirish

Shahar bo'yicha guruhlangan jadval yoz: "Toshkent" yacheykasi `rowspan="2"` bilan ikki qatorni egallasin, yonida Ali va Lola bo'lsin.

 **Yechim** 

```
<table>
  <tr>
    <th>Shahar</th>
    <th>Ism</th>
  </tr>
  <tr>
    <td rowspan="2">Toshkent</td>
    <td>Ali</td>
  </tr>
  <tr>
    <td>Lola</td>
  </tr>
</table>
```

Ikkinchi ma'lumot qatorida (Lola) faqat bitta `<td>` borligiga diqqat qil — chap ustunni "Toshkent" allaqachon egallab turibdi.

Mashq 6 — `scope` bilan accessibility

Quyidagi jadvalni accessible qil: yuqori qator `<th>` lariga `scope="col"`, har qatorning birinchi (ism) yacheykasini `<th scope="row">` qil.

```
<table>
  <tr><th>Ism</th><th>Yosh</th></tr>
  <tr><td>Ali</td><td>25</td></tr>
</table>
```

**Yechim**

```
<table>
  <thead>
    <tr>
      <th scope="col">Ism</th>
      <th scope="col">Yosh</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Ali</th>
      <td>25</td>
    </tr>
  </tbody>
</table>
```

Endi skrin-rider "25" ni "Ali, Yosh: 25" deb o'qiydi. Ism yacheykasi `<td>` dan `<th scope="row">` ga aylanganiga e'tibor ber.

Mashq 7 — `colgroup` bilan ustunni ajratish

Uch ustunli (Mahsulot, Narx, Soni) jadval yoz va `<colgroup>` / `<col>` yordamida o'rtadagi "Narx" ustuniga sariq (`#fef3c7`) fon ber.

**Yechim**

```
<table>
  <caption>Mahsulotlar</caption>
  <colgroup>
    <col>
    <col style="background-color: #fef3c7;">
    <col>
  </colgroup>
  <thead>
    <tr>
      <th scope="col">Mahsulot</th>
      <th scope="col">Narx</th>
      <th scope="col">Soni</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Olma</td>
      <td>12 000</td>
      <td>3</td>
    </tr>
  </tbody>
</table>
```

`<colgroup>` ichidagi ikkinchi `<col>` o'rtadagi ustunga mos keladi va faqat unga fon beradi.

Mashq 8 — To'liq jadval (yakuniy sinov)

Quyidagi shartlarning HAMMASIGA javob beradigan bitta jadval yoz: 1. `<caption>` "Sinf natijalari". 2. `<thead>` da ikki darajali sarlavha: "Talaba" (`rowspan="2"`) va "Imtihon" (`colspan="2"`), uning ostida "Yozma" va "Og'zaki". 3. `<tbody>` da kamida bitta talaba (ism `<th scope="row">`, ikkita baho `<td>`). 4. CSS bilan `border-collapse: collapse;` va yacheykalarga chiziqli qo'sh.

Yechim

```
<style>
  table {
    border-collapse: collapse;
  }
  th, td {
    border: 1px solid #94a3b8;
    padding: 8px 12px;
    text-align: center;
  }
  thead {
    background-color: #dbeafe;
  }
</style>

<table>
  <caption>Sinf natijalari</caption>
  <thead>
    <tr>
      <th rowspan="2" scope="col">Talaba</th>
      <th colspan="2" scope="colgroup">Imtihon</th>
    </tr>
    <tr>
      <th scope="col">Yozma</th>
      <th scope="col">Og'zaki</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Ali</th>
      <td>85</td>
      <td>90</td>
    </tr>
    <tr>
      <th scope="row">Lola</th>
      <td>78</td>
      <td>88</td>
    </tr>
  </tbody>
</table>
```

Bu yechim bobning deyarli barcha tushunchalarini birlashtiradi: `caption`, `thead / tbody`, ikki darajali sarlavha (`rowspan + colspan`), `scope`, va asosiy CSS (`border-collapse`). "Imtihon" sarlavhasiga `scope="colgroup"` berildi — chunki u bir nechta ustunni qamragan guruh sarlavhasi.

[← Oldingi: 03 — Havolalar va media](#) · [🏠 README](#) · [Keyingi: 05 — Formalar →](#)

05 — Formalar

← Oldingi: 04 — Jadvallar · 🏠 README · Keyingi: 06 — Semantik HTML va accessibility →

Bu bobda: foydalanuvchidan ma'lumot olishni o'rganamiz — `form` elementi, barcha `input` turlari va brauzer ichidagi validatsiya (tekshiruv).

Hozirgacha biz sahifaga matn, rasm, havola **chiqarib bera oldik**. Lekin internet faqat o'qish uchun emas — biz unga ma'lumot **kiritamiz** ham: login qilamiz, izoh yozamiz, qidiramiz, buyurtma beramiz. Bularning hammasi orqasida **forma (form)** turadi.

Formani oddiy hayotdagi **anketa qog'oziga** o'xshating: bir nechta katakcha (maydon), har birining yonida nima yozish kerakligini aytuvchi yozuv (yorliq), va oxirida "Topshirish" tugmasi. Foydalanuvchi to'ldiradi, tugmani bosadi — va qog'oz **birovga** (serverga) yetkaziladi. Mana shu bob aynan shu jarayonni HTML'da qanday yasashni o'rgatadi.

✦ Bu bobda biz asosan **HTML tomonini** o'rganamiz: maydonlarni qanday yasash va brauzer ularni qanday tekshiradi. Ma'lumotni serverda **qabul qilish va saqlash** (PHP, JS) keyingi qo'llanmalar mavzusi — lekin "nega" tushunarli bo'lishi uchun server tomonida nima bo'lishini ham qisqacha izohlaymiz.

5.1 `form` elementi — hamma narsaning idishi

Har qanday ma'lumot kiritish maydoni `<form>` ichida turishi kerak. `form` — bu maydonlarni **bir guruhga** birlashtiruvchi va ularni qayerga yuborishni biluvchi idish.

```
<form action="/royxatdan-otish" method="post">
  <input type="text" name="ism">
  <button type="submit">Jo'natish</button>
</form>
```

Bu yerda ikkita eng muhim atribut bor: `action` va `method`.

`action` — ma'lumot QAYERGA boradi

`action` — bu **manzil (URL)**: foydalanuvchi tugmani bosganda, to'ldirilgan ma'lumot aynan shu manzilga jo'natiladi.

```
<form action="/login">          <!-- shu sayt ichidagi /login manziliga -->
->
<form action="https://misol.uz/saqlash">  <!-- to'liq manzil -->
<form action="">          <!-- bo'sh bo'lsa: shu sahifaning o'ziga
qaytadi -->
```

💡 Hozircha bizda haqiqiy server yo'q — shuning uchun tugmani bosganingizda "sahifa topilmadi" chiqishi normal. Muhimi — **mexanizmni** tushunish.

method — ma'lumot QANDAY boradi: GET vs POST

method so'rovning **turini** belgilaydi. Ikkita asosiy qiymat bor va ular orasidagi farq juda muhim:

	method="get"	method="post"
Ma'lumot qayerda ketadi	URL'ga qo'shiladi: ? ism=Ali&shahar=Toshkent	So'rov tanasida (body) yashirin
URL'da ko'rinadimi	Ha, manzil qatorida ko'rinadi	Yo'q
Xatcho'p qilsa bo'ladimi	Ha (qidiruv natijasi kabi)	Yo'q
Maxfiy ma'lumot uchun	❌ Yaramaydi (parol URL'da ko'rinadi!)	✅ To'g'ri
Qachon ishlatiladi	O'qish/qidirish (hech narsa o'zgartirmaydi)	Yozish/o'zgartirish (ma'lumot saqlash)

Oddiy qoida: - Faqat ma'lumot **olib kelyapsizmi** (qidiruv, filtrlash)? → `get` . - Ma'lumot **saqlayapsizmi/o'zgartiryapsizmi** (ro'yxatdan o'tish, izoh qo'shish, parol)? → `post` .

```
<!-- Qidiruv: get to'g'ri, chunki natijani xatcho'p qilsa bo'ladi -->
<form action="/qidiruv" method="get">
  <input type="search" name="q">
  <button>Qidirish</button>
</form>
<!-- Tugma bosilgach manzil: /qidiruv?q=telefon -->
```

```
<!-- Login: post to'g'ri, chunki parol URL'da ko'rinmasligi kerak -->
<form action="/login" method="post">
  <input type="password" name="parol">
```

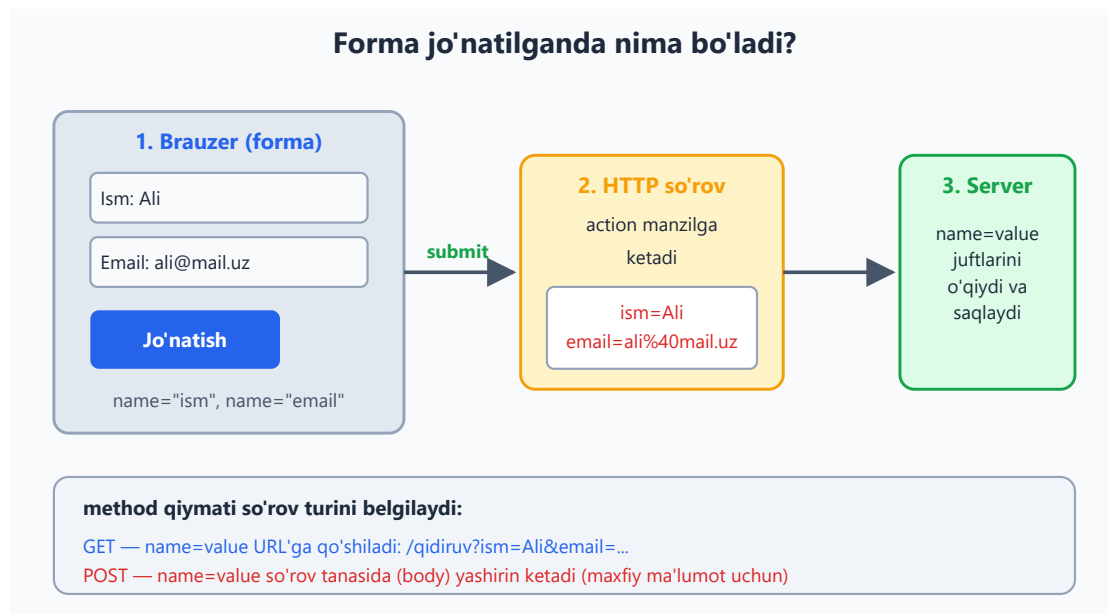
```
<button>Kirish</button>  
</form>
```

⚠ **Eng katta xato:** parol yoki shaxsiy ma'lumotni `get` bilan yuborish. U holda parol manzil qatorida (`/login?parol=12345`) ochiq ko'rinadi, brauzer tarixiga yoziladi va xavfsiz emas. Maxfiy narsa uchun **doim** `post` .

✦ `method` yozilmasa, brauzer `get` deb hisoblaydi.

5.2 Forma jo'natilganda nima bo'ladi? (umumiy oqim)

"Jo'natish" tugmasi bosilganda quyidagilar ketma-ket yuz beradi:



1. **Brauzer** har bir maydonni `name=value` ko'rinishidagi juftlikka aylantiradi (`ism=Ali` , `email=ali@mail.uz`).
2. Brauzer (agar validatsiya o'tib bo'lsa) bu juftlarni `action` **manziliga**, `method` turida jo'natadi.
3. **Server** kelgan `name=value` juftlarini o'qiydi, qayta ishlaydi (saqlaydi, tekshiradi) va javob qaytaradi.

Bu yerda eng asosiy tushuncha — `name` . Server maydonni **nomi orqali** tanib oladi. Buni keyingi bo'limda batafsil ko'ramiz.

5.3 `name` va `value` — server ma'lumotni qanday "ko'radi"

Brauzer uchun maydon — bu ekrandagi quti. Lekin **server uchun** maydon — bu shunchaki bitta `name=value` juftligi. Ekranda nima yozilgani serverga ko'rinmaydi; serverga faqat shu juftlik boradi.

```
<form action="/saqlash" method="post">
  <input type="text" name="ism" value="Ali">
  <input type="email" name="email">
</form>
```

- **name** — maydonning **kaliti (key)**. Server ma'lumotni shu nom orqali oladi. **name bo'lmasa, maydon umuman jo'natilmaydi!**
- **value** — maydonning **qiymati**. `value` atributi boshlang'ich qiymatni belgilaydi; foydalanuvchi yozsa, `value` uning yozgani bo'ladi.

Agar foydalanuvchi ism maydoniga `Olim` deb yozsa, serverga shu boradi:

```
ism=Olim
email=olim@mail.uz
```

💡 Serverda (masalan PHP'da) bu `$_POST['ism']` orqali olinadi — ya'ni `name` qiymati bilan. Shuning uchun `name` larni **mazmunli va ingliz/lotin harflarda, bo'shliqsiz** yozing: `name="ism"`, `name="email"`, `name="phone_number"`.

⚠️ Eng ko'p uchraydigan boshlovchi xatosi: chiroyli forma yasab, lekin `name` atributini unutib qo'yish. Tashqaridan hammasi joyida ko'rinadi, lekin serverga **hech narsa kelmaydi**, chunki nomsiz maydon yuborilmaydi.

5.4 `label` — maydonning yorlig'i (va nega u shart)

`label` — bu maydon **nima uchun ekanini** tushuntiruvchi yozuv ("Ism", "Email", "Parol"). Uni input bilan **bog'lash** kerak:

label va input qanday bog'lanadi?

```
<label for="email" >
Email manzil
</label>

<input id="email"
type="email">
```

for == id

Ekranda natija

Email manzil

Yorliqqa bosilsa, maydon fokuslanadi

Nega muhim?

Skrin-reader yorliqni o'qiydi · bosish maydoni kattalashadi · forma qulayroq va accessible bo'ladi

Bog'lashning ikki usuli bor:

1-usul — for va id orqali (tavsiya etiladi):

```
<label for="email">Email manzilingiz</label>
<input type="email" id="email" name="email">
```

label ning for qiymati input'ning id qiymatiga **aynan teng** bo'lishi kerak — shunda ular bog'lanadi.

2-usul — input'ni label ichiga o'rash:

```
<label>
  Email manzilingiz
  <input type="email" name="email">
</label>
```

Nega label muhim? (uchta sabab)

1. **Bosish maydoni kattalashadi** — foydalanuvchi yorliq matniga bossa ham, maydon aktivlashadi (fokuslanadi). Mobil telefonda kichkina checkbox'ni topish o'rniga, uzun matnga bosish ancha qulay.
2. **Accessibility (qulaylik)** — ko'zi ojiz foydalanuvchilar **skrin-reader** (ekranni o'quvchi dastur) ishlatadi. U bog'langan label ni o'qib, "Email manzilingiz, matn maydoni" deb aytadi. Bog'lanmagan maydon esa "nomsiz maydon" bo'lib qoladi.
3. **Tushunarlilik** — yorliqsiz forma jumboq kabi: foydalanuvchi har bir katakka nima yozishni bilmaydi.

✳️ Qoida: **har bir maydonning bog'langan label i bo'lsin**. Bu professional formaning birinchi belgisi.

5.5 input turlari — type atributi

`input` — formaning eng asosiy elementi. Uning ko'rinishi va xulqi `type` atributiga bog'liq. To'g'ri `type` tanlash — ham qulaylik, ham bepul validatsiya beradi.

input turlari katalogi (type="...")
type atributi brauzerga maydonni qanday ko'rsatish va tekshirishni aytadi

type="text" Oddiy matn	type="password" 	type="email" <input type="email" value="ali@mail.uz"/>
type="number" <input type="number" value="42"/>	type="date" <input type="date" value="09.06.2026"/>	type="tel" <input type="tel" value="+998 90 123 45 67"/>
type="range" <input type="range" value="50"/>	type="color" <input type="color" value="#008000"/> rang tanlash	type="search" <input type="search" value="Q qidirish..."/>
type="checkbox" (ko'p tanlov) ☒ PHP ☐ JS	type="radio" (bittasi) ☒ Erkak ☐ Ayol	type="file" <input type="file" value="Fayl tanlash"/> rasm.png
select (ro'yxatdan tanlash) Toshkent	textarea (ko'p qatorli matn) Uzun izoh matni bir necha qatorda...	button (submit) Tasdiqlash

type noto'g'ri bo'lsa, brauzer "text" deb hisoblaydi — shuning uchun har doim mos type tanlang

Matn kiritish turlari

```
<label for="ism">Ism</label>
<input type="text" id="ism" name="ism">

<label for="parol">Parol</label>
<input type="password" id="parol" name="parol">

<label for="email">Email</label>
<input type="email" id="email" name="email">

<label for="sayt">Veb-sayt</label>
<input type="url" id="sayt" name="sayt">

<label for="tel">Telefon</label>
<input type="tel" id="tel" name="tel">

<label for="qidiruv">Qidirish</label>
<input type="search" id="qidiruv" name="q">
```

- **text** — oddiy bir qatorli matn. Eng universal tur.
- **password** — yozilgan belgilar **nuqtalar bilan yashiriladi** (••••), boshqalar yelka ortidan ko'rmasligi uchun. ⚠ Bu faqat *ekranda* yashiradi — internetda xavfsiz uzatish uchun **HTTPS** kerak (bu boshqa mavzu).
- **email** — brauzer @ belgisi borligini avtomatik tekshiradi. Mobil telefonda klaviaturada @ tugmasi chiqadi.
- **url** — to'liq manzil (https://...) formatini tekshiradi.
- **tel** — telefon raqami. Brauzer formatni tekshirmaydi (chunki har davlatda har xil), lekin mobilda **raqamli klaviatura** ochiladi.
- **search** — qidiruv maydoni. Odatda yumaloq chetli va ichida "tozalash" (x) tugmasi bilan keladi.

💡 **Nega aniq type ?** type="email" yozsangiz, mobil foydalanuvchi avtomatik @ belgili klaviatura oladi va brauzer formatni tekshiradi — hammasi **bepul**, hech qanday kod yozmasdan. Hammasini text qilib qo'ysangiz, bu imkoniyatlarni yo'qotasiz.

Raqam, sana va diapazon turlari

```
<label for="yosh">Yoshingiz</label>
<input type="number" id="yosh" name="yosh" min="0" max="120">

<label for="tugilgan">Tug'ilgan sana</label>
<input type="date" id="tugilgan" name="tugilgan">

<label for="ovoz">Ovoz balandligi</label>
<input type="range" id="ovoz" name="ovoz" min="0" max="100" value="50">

<label for="rang">Sevimli rang</label>
<input type="color" id="rang" name="rang">
```

- **number** — faqat raqam. Yonida ▲ ▼ strelkalar chiqadi, min / max bilan chegaralanadi.
- **date** — sana tanlash. Brauzer **kalendar** (taqvim) ochib beradi — qo'lda yozish shart emas.
- **range** — surgich (slider). Aniq raqam muhim bo'lmaganda, "kam-ko'p" tanlash uchun (ovoz, yorqinlik).
- **color** — rang tanlovchi panel ochadi, natija #rrggbb ko'rinishida keladi.

✳ Yana bor: type="datetime-local" (sana + vaqt), type="time" (faqat vaqt), type="month", type="week". Ularning hammasi shu mantiqda ishlaydi.

Tanlov turlari: checkbox va radio

Bu ikkalasi orasidagi farq boshlovchilarni eng ko'p chalg'itadi:

```
<!-- checkbox: bir nechtasini belgilash mumkin -->
<p>Qaysi tillarni bilasiz?</p>
<label><input type="checkbox" name="tillar[]" value="php"> PHP</label>
<label><input type="checkbox" name="tillar[]" value="js">
JavaScript</label>
<label><input type="checkbox" name="tillar[]" value="py">
Python</label>

<!-- radio: faqat BITTASINI tanlash mumkin -->
<p>Jinsingiz:</p>
<label><input type="radio" name="jins" value="erkak"> Erkak</label>
<label><input type="radio" name="jins" value="ayol"> Ayol</label>
```

- **checkbox** — har biri **mustaqil**: 0 ta, 1 ta yoki hammasi belgilanishi mumkin. "Hammasidan birortasini" tanlash kabi.
- **radio** — bir guruhdan **faqat bittasi**. Sirning kaliti: bir guruhdagi radio'larning **name'i bir xil** bo'lishi shart (`name="jins"`). Brauzer bir xil **name**'li radio'larni guruh deb biladi va faqat bittasiga ruxsat beradi.

💡 Radio'da **value** **shart**. Server qaysi tanlanganini bilishi uchun har bir radio'da boshqacha **value** bo'lishi kerak (`value="erkak"` , `value="ayol"`). Tanlangan radio'ning **value**'si jo'natiladi.

✚ Birortasini **avval belgilangan** qilib qo'yish uchun **checked** atributini qo'shing:

```
<label><input type="radio" name="jins" value="erkak" checked>
Erkak</label>
```

Maxsus turlar: **file** va **hidden**

```
<!-- file: foydalanuvchi kompyuteridan fayl tanlaydi -->
<label for="rasm">Rasm yuklang</label>
<input type="file" id="rasm" name="rasm" accept="image/*">

<!-- hidden: ko'rinmaydi, lekin serverga jo'natiladi -->
<input type="hidden" name="user_id" value="42">
```

- **file** — rasm, hujjat va h.k. yuklash uchun. `accept="image/*"` faqat rasmlarga ruxsat beradi. ⚠ Fayl yuklash uchun **form** ga `enctype="multipart/form-data"` qo'shish **shart** (aks holda fayl jo'natilmaydi):

```
<form action="/yuklash" method="post" enctype="multipart/form-data">
  <input type="file" name="rasm">
```

```
<button>Yuklash</button>
</form>
```

- `hidden` — foydalanuvchiga **ko'rinmaydi**, lekin u bilan birga serverga jo'natiladi. Masalan, qaysi foydalanuvchi tahrir qilayotganini bildiruvchi `user_id` kabi yashirin ma'lumotlar uchun. ⚠️ "Yashirin" so'ziga aldanmang — uni HTML'ni ko'rgan har kim o'qiy oladi, shuning uchun **parol kabi maxfiy narsani u yerga yozmang**.

5.6 `textarea` — ko'p qatorli matn

`input type="text"` bir qatorli. Uzun matn (izoh, xabar, manzil) uchun `textarea` ishlatiladi:

```
<label for="izoh">Izohingiz</label>
<textarea id="izoh" name="izoh" rows="5" cols="40" placeholder="Bu
yerga yozing..."></textarea>
```

- `rows` — necha qator balandlikda ko'rinishi, `cols` — kengligi.
- ⚠️ `textarea` ning `value` atributi **yo'q!** Boshlang'ich matn **ochuvchi va yopuvchi teg orasiga** yoziladi:

```
<textarea name="izoh">Bu matn boshidan turadi.</textarea>
```

✳️ `<textarea></textarea>` ni har doim **yoping**, hatto bo'sh bo'lsa ham. Yopilmasa, undan keyingi hamma narsa `textarea` ichiga "yutilib" ketadi.

5.7 `select` — ro'yxatdan tanlash

Ko'p variantdan bittasini (yoki bir nechtasini) tanlash uchun `select` ishlatiladi. Bu ekranda joy tejaydi:

```
<label for="shahar">Shahringiz</label>
<select id="shahar" name="shahar">
  <option value="">- tanlang -</option>
  <option value="tosh">Toshkent</option>
  <option value="sam">Samarqand</option>
  <option value="buxoro" selected>Buxoro</option>
</select>
```

- Har bir variant — `<option>`. Serverga uning `value` 'si jo'natiladi (ekrandagi matn emas!). Agar `value` yozilmasa, ichidagi matn jo'natiladi.

- `selected` — qaysi variant **avvaldan tanlangan** bo'lishini belgilaydi.
- Birinchi bo'sh `option` ("— tanlang —") — foydalanuvchini majburlash uchun foydali hiyla.

`optgroup` — variantlarni guruhlash

Variantlar ko'p bo'lsa, ularni mavzu bo'yicha guruhlash mumkin:

```
<select name="taom">
  <optgroup label="Issiq taomlar">
    <option value="osh">Osh</option>
    <option value="manti">Manti</option>
  </optgroup>
  <optgroup label="Ichimliklar">
    <option value="choy">Choy</option>
    <option value="kofe">Kofe</option>
  </optgroup>
</select>
```

`optgroup` ning `label` i — guruh sarlavhasi (tanlab bo'lmaydi, faqat ko'rsatkich).

`datalist` — yozish + taklif

`datalist` — `input` va `select` orasidagi gibridd: foydalanuvchi **erkin yozadi**, lekin brauzer **takliflar** ham ko'rsatadi:

```
<label for="brauzer">Sevimli brauzer</label>
<input type="text" id="brauzer" name="brauzer" list="brauzerlar">
<datalist id="brauzerlar">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
</datalist>
```

`input` ning `list` atributi `datalist` ning `id` 'si bilan bog'lanadi. Foydalanuvchi taklif ichidan tanlashi **yoki** o'zinikini yozishi mumkin — `select` da esa faqat berilganlardan tanlash mumkin. Mana asosiy farq.

5.8 `button` — tugmalar va `type` ning ahamiyati

Tugma ikki xil yoziladi: `<button>` yoki `<input type="...">`. `<button>` qulayroq, chunki ichiga matn, rasm, ikonka qo'yish mumkin. Eng muhimi — tugmaning `type` atributi:

```

<form action="/saqlash" method="post">
  <input type="text" name="ism">

  <button type="submit">Jo'natish</button>    <!-- formani jo'natadi -->
  <button type="reset">Tozalash</button>      <!-- maydonlarni
bo'shatadi -->
  <button type="button">Ko'rsatish</button>   <!-- hech narsa qilmaydi
(JS uchun) -->
</form>

```

- **type="submit"** — formani jo'natadi. Bu **standart** qiymat: agar **type** yozilmasa, tugma **submit** bo'ladi.
- **type="reset"** — barcha maydonlarni boshlang'ich holatiga qaytaradi. ⚠ Ehtiyot bo'ling: foydalanuvchi uzun formani to'ldirib, adashib bosib yuborsa, hammasi yo'qoladi. Shuning uchun zamonaviy saytlarda kam ishlatiladi.
- **type="button"** — o'zicha hech narsa qilmaydi. JavaScript bilan boshqariladigan tugmalar uchun (masalan "parolni ko'rsatish").

⚠ **Juda muhim tuzoq:** `<button>` da **type** yozmasangiz, u **avtomatik submit** bo'ladi! Forma ichidagi har qanday oddiy tugma (masalan "+1 qo'shish") **type="button"** yozmasangiz, bosilganda butun formani jo'natib yuboradi. Forma ichidagi har bir tugmaning **type** 'ini ataylab yozing.

5.9 **fieldset** va **legend** — maydonlarni guruhlash

Uzun formani mantiqiy bo'laklarga bo'lish uchun **fieldset** (guruh) va **legend** (guruh sarlavhasi) ishlatiladi:

```

<form action="/royxat" method="post">
  <fieldset>
    <legend>Shaxsiy ma'lumotlar</legend>
    <label for="ism">Ism</label>
    <input type="text" id="ism" name="ism">
    <label for="yosh">Yosh</label>
    <input type="number" id="yosh" name="yosh">
  </fieldset>

  <fieldset>
    <legend>Aloqa</legend>
    <label for="email">Email</label>
    <input type="email" id="email" name="email">
  </fieldset>

  <button type="submit">Ro'yxatdan o'tish</button>
</form>

```

- `fieldset` maydonlar atrofiga **ramka** chizadi va ularni bitta guruh sifatida ko'rsatadi.
- `legend` — shu ramkaning sarlavhasi.
- **Nega muhim?** Skrin-reader `legend` ni har bir maydon bilan birga o'qiydi ("Aloqa — Email"). Bu radio guruhi uchun ayniqsa foydali — guruh nima haqida ekanini bildiradi.

💡 `fieldset` ga `disabled` atributini qo'ysangiz, ichidagi **barcha** maydonlar bir vaqtda o'chiriladi (faolsizlanadi). Qadamli ("wizard") formalarda qulay.

5.10 Validatsiya atributlari — brauzer ichidagi tekshiruv

Brauzer formani jo'natishdan **oldin** maydonlarni tekshira oladi — biz hech qanday kod yozmasdan. Bu **HTML5 validatsiyasi** deyiladi. Asosiy atributlar:

```
<form action="/royxat" method="post">
  <label for="ism">Ism (majburiy)</label>
  <input type="text" id="ism" name="ism"
    required
    minlength="2" maxlength="30"
    placeholder="Ismingizni yozing">

  <label for="parol">Parol</label>
  <input type="password" id="parol" name="parol"
    required minlength="8">

  <label for="yosh">Yosh</label>
  <input type="number" id="yosh" name="yosh" min="18" max="99">

  <label for="tel">Telefon</label>
  <input type="tel" id="tel" name="tel"
    pattern="+998[0-9]{9}"
    placeholder="+998901234567">

  <button type="submit">Yuborish</button>
</form>
```

Atribut	Ma'nosi	Qaysi maydonlarda
<code>required</code>	Maydon bo'sh qolmasligi kerak	barchasida
<code>minlength</code> / <code>maxlength</code>	Matn uzunligining eng kam / eng ko'p belgisi	matn maydonlari

Atribut	Ma'nosi	Qaysi maydonlarda
<code>min / max</code>	Eng kichik / eng katta qiymat	<code>number</code> , <code>range</code> , <code>date</code>
<code>pattern</code>	Matn berilgan shablon (regex)ga mos kelishi shart	matn maydonlari
<code>placeholder</code>	Maydon bo'sh paytdagi xira namuna matn	matn maydonlari

Agar foydalanuvchi `required` maydonni bo'sh qoldirib jo'natmoqchi bo'lsa, brauzer **jo'natmaydi** va "Bu maydonni to'ldiring" degan xabar ko'rsatadi — bularning hammasi avtomatik.

⚠ **placeholder ≠ label!** `placeholder` — maydon ichidagi xira namuna matn. U **yorliq o'rnini bosa olmaydi**, chunki foydalanuvchi yoza boshlashi bilan yo'qoladi va keyin maydon nima uchun ekanini eslay olmaydi. Doim **label + placeholder** birga ishlatiladi:

```
<label for="email">Email</label>
<input type="email" id="email" name="email" placeholder="masalan:
ali@mail.uz">
```

⚠ Eng muhim ogohlantirish: brauzer validatsiyasi YETARLI EMAS

HTML validatsiyasi — bu faqat **qulaylik** (foydalanuvchiga tez xabar berish). U **xavfsizlik emas!** Sababi: tajribali odam brauzer vositalarida `required` ni o'chirib tashlashi yoki formani umuman chetlab o'tib serverga to'g'ridan-to'g'ri yomon ma'lumot yuborishi mumkin.

✳ **Oltin qoida:** mijoz (brauzer) tomonida tekshiruv — qulaylik uchun; **server tomonida** tekshiruv — xavfsizlik uchun. **Har doim ikkalasini ham qiling.** Serverda tekshirishni keyingi qo'llanmalarda (PHP/JS) o'rganasiz, lekin tamoyilni hozir yodda tuting.

5.11 Accessibility va `autocomplete` — formani hamma uchun qulay qilish

Yaxshi forma — hamma (jumladan ko'zi ojizlar, klaviatura bilan ishlovchilar) foydalana oladigan forma. Asosiy amaliyotlar:

1. **Har maydonda bog'langan label** (5.4'da ko'rdik) — bu eng muhimi.

2. **autocomplete — brauzer yodlab, to'ldirib beradi.** Standart nomlar berilsa, brauzer foydalanuvchining oldingi ma'lumotini avtomatik taklif qiladi:

```
<label for="ism">To'liq ism</label>
<input type="text" id="ism" name="ism" autocomplete="name">

<label for="email">Email</label>
<input type="email" id="email" name="email" autocomplete="email">

<label for="tel">Telefon</label>
<input type="tel" id="tel" name="tel" autocomplete="tel">
```

autocomplete="name", "email", "tel", "street-address", "postal-code" — bular brauzer biladigan **standart** nomlar. Foydalanuvchi qayta-qayta yozmaydi — bir bosishda to'ldiriladi.

3. **aria atributlari — qiyin holatlar uchun.** Agar yorliq yashirin bo'lishi kerak bo'lsa (masalan dizayn uchun), `aria-label` bilan skrin-reader uchun nom berish mumkin:

```
<input type="search" name="q" aria-label="Saytdan qidirish">
```

Xato xabarini maydon bilan bog'lash uchun `aria-describedby`:

```
<input type="email" id="email" name="email" aria-describedby="email-hint">
<small id="email-hint">Hech kimga ko'rsatilmaydi.</small>
```

💡 **Tartib:** birinchi navbatda haqiqiy `label`, `fieldset`, to'g'ri `type` ishlatish. `aria` — bu **qo'shimcha**, oddiy HTML yeta olmagan joyni "yamash" uchun. To'g'ri HTML allaqachon ko'p accessibility'ni bepul beradi.

5.12 To'liq misol — barchasini birlashtiramiz

Quyida shu bobdagi tushunchalarni jamlagan haqiqiy ro'yxatdan o'tish formasi:

```
<form action="/royxatdan-otish" method="post">
  <fieldset>
    <legend>Hisob yaratish</legend>

    <p>
      <label for="ism">To'liq ism</label><br>
      <input type="text" id="ism" name="ism"
        required minlength="2"
```

```

        autocomplete="name"
        placeholder="Ali Valiyev">
    </p>

    <p>
        <label for="email">Email</label><br>
        <input type="email" id="email" name="email"
            required autocomplete="email"
            placeholder="ali@mail.uz">
    </p>

    <p>
        <label for="parol">Parol</label><br>
        <input type="password" id="parol" name="parol"
            required minlength="8">
    </p>

    <p>
        <label for="shahar">Shahar</label><br>
        <select id="shahar" name="shahar" required>
            <option value="">- tanlang -</option>
            <option value="tosh">Toshkent</option>
            <option value="sam">Samarqand</option>
        </select>
    </p>

    <p>Jinsingiz:</p>
    <p>
        <label><input type="radio" name="jins" value="erkak" required>
        Erkak</label>
        <label><input type="radio" name="jins" value="ayol"> Ayol</label>
    </p>

    <p>
        <label>
            <input type="checkbox" name="shartlar" value="ha" required>
            Shartlarga roziman
        </label>
    </p>
</fieldset>

<button type="submit">Ro'yxatdan o'tish</button>
</form>

```

Natija: ramka ichida tartibli forma; har maydonda yorliq; bo'sh qoldirilsa yoki parol 8 belgidan qisqa bo'lsa — brauzer jo'natmaydi va xabar beradi; jo'natilsa, serverga `ism=...&email=...&parol=...&shahar=tosh&jins=erkak&shartlar=ha` ko'rinishida boradi.

Mashqlar

Quyidagi mashqlarni tartib bilan bajaring — har biri oldingisining ustiga quradi.

Mashq 1. Bitta matn maydoni (`name="ism"`) va bitta "Jo'natish" tugmasidan iborat forma yarating. `action="/test"`, `method="post"` bo'lsin. Maydonda bog'langan `label` ("Ism") bo'lsin.

Yechim

```
<form action="/test" method="post">
  <label for="ism">Ism</label>
  <input type="text" id="ism" name="ism">
  <button type="submit">Jo'natish</button>
</form>
```

Mashq 2. Login formasi yarating: email maydoni (`type="email"`) va parol maydoni (`type="password"`). Ikkalasi ham `required`. Parol kamida 8 belgidan iborat bo'lsin (`minlength`). Nega bu forma `method="post"` bo'lishi kerakligini bir jumlada izohlang.

Yechim

```
<form action="/login" method="post">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>

  <label for="parol">Parol</label>
  <input type="password" id="parol" name="parol" required minlength="8">

  <button type="submit">Kirish</button>
</form>
```

`post` kerak, chunki parol maxfiy — `get` bo'lsa u URL manzil qatorida ochiq ko'rinib qolardi va brauzer tarixiga yozilardi.

Mashq 3. Bir guruh `radio` tugma yarating: "To'lov usuli" — Naqd, Karta, Pul o'tkazma. Faqat bittasini tanlash mumkin bo'lsin va "Karta" avvaldan belgilangan bo'lsin. (Maslahat: `name` bir xil, `value` 'lar har xil bo'lsin.)



Yechim



```
<fieldset>
  <legend>To'lov usuli</legend>
  <label><input type="radio" name="tolov" value="naqd"> Naqd</label>
  <label><input type="radio" name="tolov" value="karta" checked>
Karta</label>
  <label><input type="radio" name="tolov" value="otkazma"> Pul
o'tkazma</label>
</fieldset>
```

name bir xil (tolov) bo'lgani uchun brauzer ularni bitta guruh deb biladi va faqat bittasiga ruxsat beradi. checked — boshlang'ich tanlovni belgilaydi.

Mashq 4. select yarating: "Mamlakat" ro'yxati optgroup bilan ikki guruhga bo'lingan bo'lsin — "Markaziy Osiyo" (O'zbekiston, Qozog'iston) va "Yevropa" (Germaniya, Fransiya). Birinchi option — tanlang — bo'lsin.



Yechim



```
<label for="mamlakat">Mamlakat</label>
<select id="mamlakat" name="mamlakat">
  <option value="">- tanlang -</option>
  <optgroup label="Markaziy Osiyo">
    <option value="uz">O'zbekiston</option>
    <option value="kz">Qozog'iston</option>
  </optgroup>
  <optgroup label="Yevropa">
    <option value="de">Germaniya</option>
    <option value="fr">Fransiya</option>
  </optgroup>
</select>
```

Mashq 5. Quyidagi formada uchta xato bor. Ularni toping va tuzating:



```
<form>
  <label>Email</label>
  <input type="text" email>
  <button type="reset">Yuborish</button>
</form>
```



Yechim



Xatolar: 1. `input` da **name** **yo'q** — serverga hech narsa jo'natilmaydi. 2. `type="text"` email noto'g'ri — email maydoni uchun `type="email"` bo'lishi kerak (`email` o'zicha atribut emas). 3. "Yuborish" tugmasi `type="reset"` — bu formani **tozalaydi**, jo'natmaydi. `type="submit"` bo'lishi kerak.

Tuzatilgan variant:

```
<form action="/jonatish" method="post">
  <label for="email">Email</label>
  <input type="email" id="email" name="email">
  <button type="submit">Yuborish</button>
</form>
```

(`label` ni `for` / `id` bilan bog'lash va `form` ga `action` / `method` qo'shish ham yaxshilab qo'yildi.)

Mashq 6. Izoh qoldirish formasi yarating: ism (`text`, majburiy), baho (`select` 1 dan 5 gacha), va ko'p qatorli izoh (`textarea`, kamida 10 belgi). Har maydonda yorliq bo'lsin.



Yechim



```
<form action="/izoh" method="post">
  <p>
    <label for="ism">Ism</label><br>
    <input type="text" id="ism" name="ism" required>
  </p>
  <p>
    <label for="baho">Baho</label><br>
    <select id="baho" name="baho">
      <option value="5">5 — A'lo</option>
      <option value="4">4</option>
      <option value="3">3</option>
      <option value="2">2</option>
      <option value="1">1 — Yomon</option>
    </select>
  </p>
  <p>
    <label for="izoh">Izoh</label><br>
    <textarea id="izoh" name="izoh" rows="4" minlength="10"
      placeholder="Fikringizni yozing..."></textarea>
  </p>
  <button type="submit">Yuborish</button>
</form>
```

Eslatma: `textarea` ning boshlang'ich qiymati teglar **orasiga** yoziladi, `value` atributi yo'q.

Mashq 7. Fayl yuklash formasi yarating: faqat rasm qabul qilsin (`accept="image/*"`). `form` ga fayl jo'natish ishlashi uchun **qaysi ikkita** atribut shart? (Maslahat: biri `method`,

biri enctype.)

Yechim

```
<form action="/yuklash" method="post" enctype="multipart/form-data">
  <label for="rasm">Rasm tanlang</label>
  <input type="file" id="rasm" name="rasm" accept="image/*">
  <button type="submit">Yuklash</button>
</form>
```

Fayl yuklash uchun ikkita shart: `method="post"` (fayl URL'ga sig'maydi) va `enctype="multipart/form-data"` (faylni to'g'ri "qadoqlash" usuli). `enctype` bo'lmasa, server faqat fayl nomini oladi, faylning o'zini emas.

Mashq 8. (Mushkulroq) Telefon raqami maydoni yarating: `type="tel"`, `required`, va `pattern` bilan faqat `+998` bilan boshlanib, keyin to'qqizta raqam bo'lishi shart bo'lsin. `placeholder` namuna ko'rsatsin. Nega `pattern` yetarli emasligini (xavfsizlik nuqtai nazaridan) bir jumlada yozing.

Yechim

```
<label for="tel">Telefon</label>
<input type="tel" id="tel" name="tel"
  required
  pattern="\+998[0-9]{9}"
  placeholder="+9989901234567"
  autocomplete="tel">
```

`pattern` izohi: `\+998` — `+998` bilan boshlanadi, `[0-9]{9}` — keyin aniq 9 ta raqam.

Nega yetarli emas: `pattern` faqat **brauzerda** tekshiriladi va uni chetlab o'tish oson (foydalanuvchi formani o'zgartirib yoki to'g'ridan-to'g'ri serverga so'rov yuborib). Shuning uchun raqamni **serverda ham** qayta tekshirish shart — brauzer validatsiyasi qulaylik uchun, xavfsizlik server tomonida ta'minlanadi.

06 — Semantik HTML va accessibility

← Oldingi: 05 — Formalar · 🏠 README · Keyingi: 07 — Head, meta va SEO →

Bu bobda: ma'noli teglar (`header` / `nav` / `main` / `article`) bilan sahifa qurish va uni hamma — shu jumladan ko'zi ojiz, klaviatura bilan ishlovchi — odam uchun hammabop (a11y) qilish.

6.1 Semantik HTML nima va nega `div` 'dan yaxshi

Tasavvur qil: kimdir senga qutilarga joylangan ko'chish yukini berdi, lekin har bir qutida faqat "QUTI" deb yozilgan. Oshxona buyumlari qayerda? Kitoblar qayerda? Bilmaysan — hammasini ochib ko'rishga to'g'ri keladi.

Endi boshqa holatni tasavvur qil: qutilarda "OSHXONA", "KITOBLAR", "KIYIM" deb yozilgan. Bir qarashda nima qayerda ekanini bilasan.

HTML'da ham xuddi shunday. `<div>` — bu "QUTI" deb yozilgan quti: u **hech qanday ma'no bermaydi**, shunchaki bir bo'lak. `<header>`, `<nav>`, `<article>` esa "yorlig'i bor" qutilar — teg nomining o'zi shu bo'lakning **vazifasini** aytadi.

Semantik so'zi "ma'noga oid" degani. Semantik HTML — bu teglarni *ko'rinishi* uchun emas, *ma'nosi* uchun tanlash.

```
<!-- Yomon: ma'nosiz qutilar -->
<div class="header">...</div>
<div class="nav">...</div>
<div class="main-content">...</div>
<div class="footer">...</div>

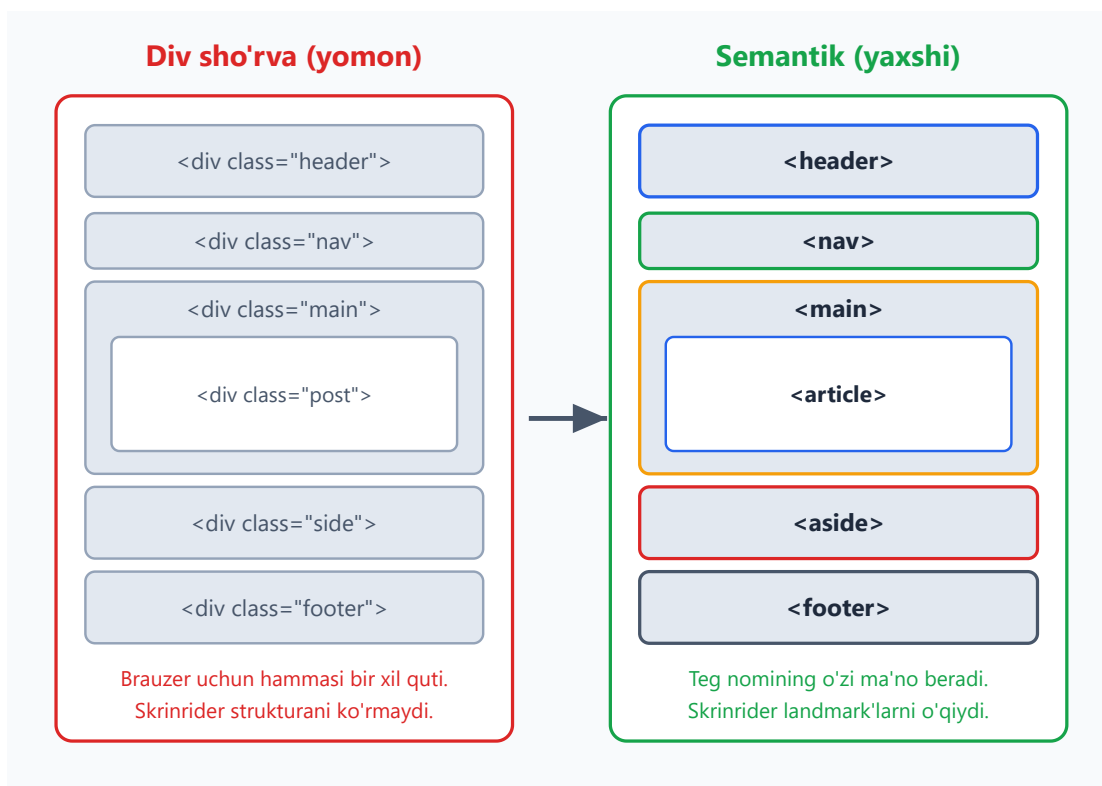
<!-- Yaxshi: teg nomining o'zi ma'no beradi -->
<header>...</header>
<nav>...</nav>
<main>...</main>
<footer>...</footer>
```

Ikkalasi ham brauzerda **bir xil ko'rinadi** (chunki `<div>` ham, `<header>` ham standartda bir xil blok element). Unda nega ovora bo'lamiz? Mana **nega**:

- **Skrinrider (screen reader)** — ko'zi ojiz odam ishlatadigan dastur — `<nav>` ni ko'rib "bu navigatsiya" deydi va foydalanuvchiga uni o'tkazib yuborish imkonini beradi. `<div class="nav">` ni esa oddiy quti deb biladi.

- **Qidiruv tizimlari (SEO)** `<article>`, `<main>` orqali sahifaning asosiy mazmuni qayerdaligini tushunadi.
- **Boshqa dasturchilar** (va 6 oydan keyingi sen o'zing) kodni o'qiganda `<footer>` ni ko'rib darrov tushunadi. `class="footer"` ni esa CSS faylga qarab tekshirish kerak.
- **Brauzer** ba'zi teglarga tayyor xulq-atvor beradi (masalan `<button>` Enter/Probel bilan bosiladi).

💡 **Oltin qoida:** avval mazmunga to'g'ri keladigan semantik tegni tanla. Agar hech qaysi maxsus teg to'g'ri kelmasa — *o'shanda* `<div>` (blok uchun) yoki `<span>` (matn ichidagi bo'lak uchun) ishlat. `<div>` "oxirgi chora", birinchi tanlov emas.



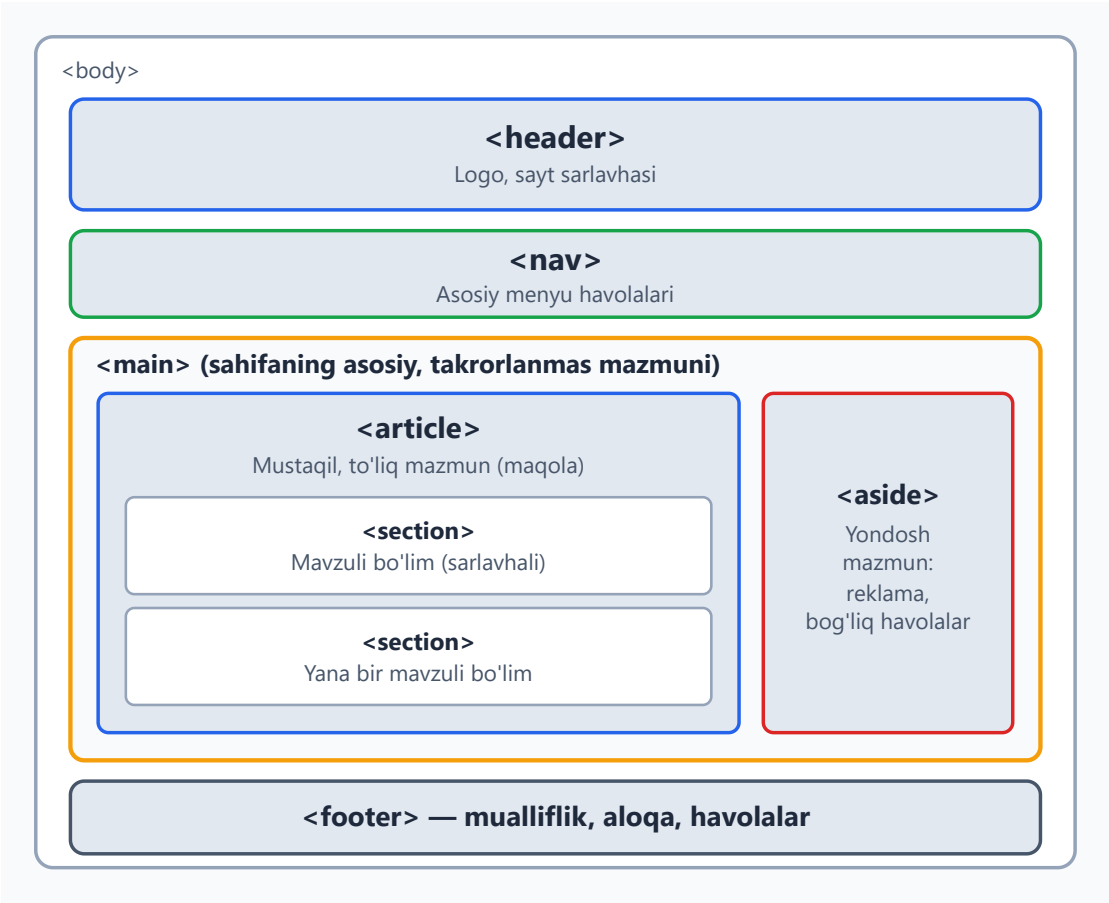
Yuqoridagi rasmda chap tomon "div sho'rva" (div soup) deb ataladi — bu hammasi `<div>` dan iborat, ma'nosiz kod. O'ng tomonda esa har bir bo'lak o'z ma'noli tegiga ega. Ikkalasi piksel-pikselgacha bir xil ko'rinishi mumkin, lekin "ichki sifati" yer bilan osmoncha farq qiladi.

6.2 Sahifa strukturasi: asosiy semantik teglar

HTML5 bizga sahifaning katta bo'laklarini belgilash uchun bir nechta tayyor teg beradi. Mana ularning vazifasi:

Teg	Vazifasi	Analogiya
<code><header></code>	Bo'limning yuqori qismi: logo, sarlavha, kirish	Gazetaning peshlavhasi
<code><nav></code>	Asosiy navigatsiya havolalari	Kitobning mundarijasi
<code><main></code>	Sahifaning asosiy, takrorlanmas mazmuni	Maqolaning o'zi
<code><article></code>	Mustaqil, o'zicha to'liq mazmun bo'lagi	Bitta blog posti
<code><section></code>	Mavzuli bo'lim (odatda sarlavhali)	Kitob bobi
<code><aside></code>	Asosiy mazmunga yondosh, ikkilamchi mazmun	Gazetadagi yon ustun
<code><footer></code>	Bo'limning pastki qismi: mualliflik, aloqa	Kitobning oxirgi sahifasi

Mana ularning tipik joylashuvi:



To'liq misol:

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>Mening blogim</title>
</head>
<body>
  <header>
    <h1>Mening blogim</h1>
    <p>Web texnologiyalari haqida</p>
  </header>

  <nav>
    <a href="/">Bosh sahifa</a>
    <a href="/maqolalar">Maqolalar</a>
    <a href="/aloqa">Aloqa</a>
  </nav>

  <main>
    <article>
      <h2>Semantik HTML nima?</h2>
      <p>Semantik HTML – bu teglarni ma'nosi uchun tanlash...</p>
    </article>
  </main>

  <aside>
    <h2>Mashhur maqolalar</h2>
    <ul>
      <li><a href="#">CSS Flexbox</a></li>
      <li><a href="#">Grid asoslari</a></li>
    </ul>
  </aside>

  <footer>
    <p>&copy; 2026 Mening blogim</p>
  </footer>
</body>
</html>
```

✦ **<main> haqida muhim qoida:** bir sahifada faqat **bitta** ko'rinadigan `<main>` bo'lishi kerak. U sahifaning *o'sha sahifaga* xos mazmunini o'rab oladi — `<header>`, `<nav>`, `<footer>` esa odatda hamma sahifada bir xil bo'lgani uchun `<main>` dan **tashqarida** turadi.

⚠ **Tez-tez uchraydigan xato:** `<header>` ni faqat sahifaning eng tepasidagi blok deb o'ylash. Aslida `<header>` va `<footer>` har qanday bo'lim ichida bo'lishi mumkin — masalan `<article>` ichida ham o'zining `<header>` (maqola sarlavhasi, sanasi) va `<footer>` (teglar, muallif) bo'ladi.

6.3 `<article>` va `<section>` farqi — eng ko'p chalkashtiriladigan joy

Bu ikkisi boshlovchilarni juda chalkashtiradi. Oddiy mezon bilan ajratamiz.

`<article>` — "Bu bo'lakni olib, butunlay boshqa joyga (masalan RSS lentaga yoki boshqa saytga) ko'chirсам, u **o'zicha mantiqan to'liq** bo'ladimi?" Agar HA bo'lsa — `<article>`.

Misollar: blog posti, yangilik maqolasi, forum xabari, mahsulot kartochkasi, foydalanuvchi sharhi (har bir sharh alohida `<article>`).

`<section>` — bu mavzu bo'yicha guruhlangan mazmun bo'lagi, lekin o'zicha mustaqil emas. U odatda **sarlavhaga ega** ("Bu bo'lim nima haqida?" degan savolga javob beradi).

Misollar: bitta maqola ichidagi "Kirish", "Asosiy qism", "Xulosa" bo'limlari; bosh sahifadagi "Bizning xizmatlar", "Mijozlar fikri" kabi bloklar.

```
<article>
  <h1>HTML semantikasini o'rganish</h1>

  <section>
    <h2>Kirish</h2>
    <p>Bu maqolada semantik teglar bilan tanishamiz...</p>
  </section>

  <section>
    <h2>Asosiy teglar</h2>
    <p>header, nav, main va boshqalar...</p>
  </section>

  <section>
    <h2>Xulosa</h2>
    <p>Semantik HTML kodingni hamma uchun yaxshiroq qiladi.</p>
  </section>
</article>
```

Bu yerda butun maqola bitta `<article>` (uni butunlay ko'chirish mumkin), uning ichidagi mavzuli qismlar esa `<section>`.

💡 **Amaliy maslahat:** `<section>` ning deyarli har doim **sarlavhasi** (`<h2>`, `<h3>` ...) bo'lishi kerak. Agar bir blokga sarlavha qo'yolmaysan va u shunchaki "ko'rinish uchun o'ralgan" bo'lsa — bu `<section>` emas, oddiy `<div>` bo'lishi kerak.

⚠️ **`<article>` ichida `<article>` bo'lishi mumkin.** Masalan blog posti — `<article>`, uning ostidagi har bir izoh (comment) ham alohida `<article>` (chunki izohni alohida ko'chirib, "bu kimdir aytgan to'liq fikr" deb tushunsa bo'ladi).

6.4 Hujjat outline (struktura) g'oyasi va sarlavhalar ierarxiyasi

Har qanday HTML hujjat **sarlavhalardan** (`<h1>` – `<h6>`) tashkil topgan ko'rinmas "mundarija"ga ega. Buni **document outline** (hujjat strukturas) deyiladi.

Sarlavhalar — bu shunchaki "katta qalin matn" emas. Ular hujjatning **ierarxik tuzilishini** bildiradi, xuddi kitobdagi boblar va bo'limlar kabi:

```
h1 Saytim haqida (eng yuqori daraja – sahifaning bosh sarlavhasi)
├ h2 Bizning tarix
├ h2 Xizmatlar
│   ├── h3 Veb-dizayn
│   └ h3 Dasturlash
└ h2 Aloqa
```

Asosiy qoidalar (nega muhimligi bilan):

1. **Sahifada bir `<h1>`** — sahifaning bosh mavzusi. (Skrinrider foydalanuvchisi "bu sahifa nima haqida?" degani — javobi `<h1>` .)
2. **Darajalarni o'tkazib yuborma.** `<h2>` dan keyin to'g'ridan-to'g'ri `<h4>` ga sakrama. Bu xuddi kitobning 1.2-bo'limidan keyin 1.2.1 emas, 1.2.0.5 kabi bo'limga o'tishdek g'alati. Skrinrider foydalanuvchisi "bir daraja yo'qoldimi?" deb adashadi.
3. **Sarlavhani o'lcham uchun tanlama.** Agar matn kichikroq ko'rinishini xohlasang — CSS bilan `font-size` o'zgartir, `<h4>` o'rniga `<h2>` ishlatma.

```
<!-- Yomon: daraja o'tkazib yuborilgan, faqat ko'rinish uchun tanlangan -->
<h1>Maqola</h1>
<h4>Kichikroq ko'rinsin deb h4 oldim</h4>

<!-- Yaxshi: ketma-ket ierarxiya -->
<h1>Maqola</h1>
<h2>Birinchi bo'lim</h2>
<h3>Kichik bo'limcha</h3>
```

💡 Ko'zi ojiz foydalanuvchilar ko'pincha sarlavhalar bo'yicha sakrab yuradi (skrinrider'da H klavishi). To'g'ri ierarxiya ularning sahifani tez "ko'zdan kechirishi"ga yordam beradi — xuddi ko'rar odam ko'z bilan sahifaga shoshilib qaragandek.

6.5 Accessibility (a11y) nima va nega muhim

Accessibility (qisqacha **a11y** — chunki "accessibility" so'zida **a** va **y** orasida 11 ta harf bor) — bu web saytni **hamma** odam, jumladan turli imkoniyatga ega bo'lganlar ham ishlata olishini ta'minlash.

Kim haqida gap ketyapti?

- **Ko'rmaydigan yoki kam ko'radigan** odamlar — skrinrider yoki ekranni kattalashtiruvchi dastur ishlatadi.
- **Qo'l harakati cheklangan** odamlar — sichqonchani ishlata olmaydi, faqat klaviatura yoki maxsus qurilma bilan ishlaydi.
- **Eshitmaydigan** odamlar — videodagi subtitr/transkriptga muhtoj.
- **Kognitiv qiyinchiliklari** bor odamlar — sodda, izchil interfeys kerak.
- **Vaqtinchalik cheklov** — qo'li singan, yorug' quyoshda ekran ko'rinmayotgan, yoki bir qo'lida chaqaloq ko'targan oddiy foydalanuvchi ham.

Nega bu sening ishing?

1. **Insoniy sabab:** internet — bu zamonaviy hayotning bir qismi (bank, davlat xizmatlari, ta'lim). Uni hammaga ochiq qilish — adolat masalasi.
2. **Qonuniy sabab:** ko'p mamlakatlarda a11y qonun bilan talab qilinadi; nomuvofiqlik uchun sudga berishadi.
3. **Amaliy sabab:** a11y'ni yaxshilaydigan narsalar (to'g'ri struktura, aniq matn) **hamma** uchun saytni yaxshilaydi — SEO ham, ko'rар foydalanuvchi tajribasi ham.

✦ **Eng yaxshi xabar:** bu bobning birinchi yarmidagi semantik HTML — a11y'ning **eng katta** qismi. To'g'ri teglar ishlatishning o'zi saytingni allaqachon ko'p odamga ochiq qiladi. A11y "qo'shimcha og'ir ish" emas — to'g'ri HTML yozishning tabiiy natijasi.

6.6 Skrinrider sahifani qanday "o'qiydi"

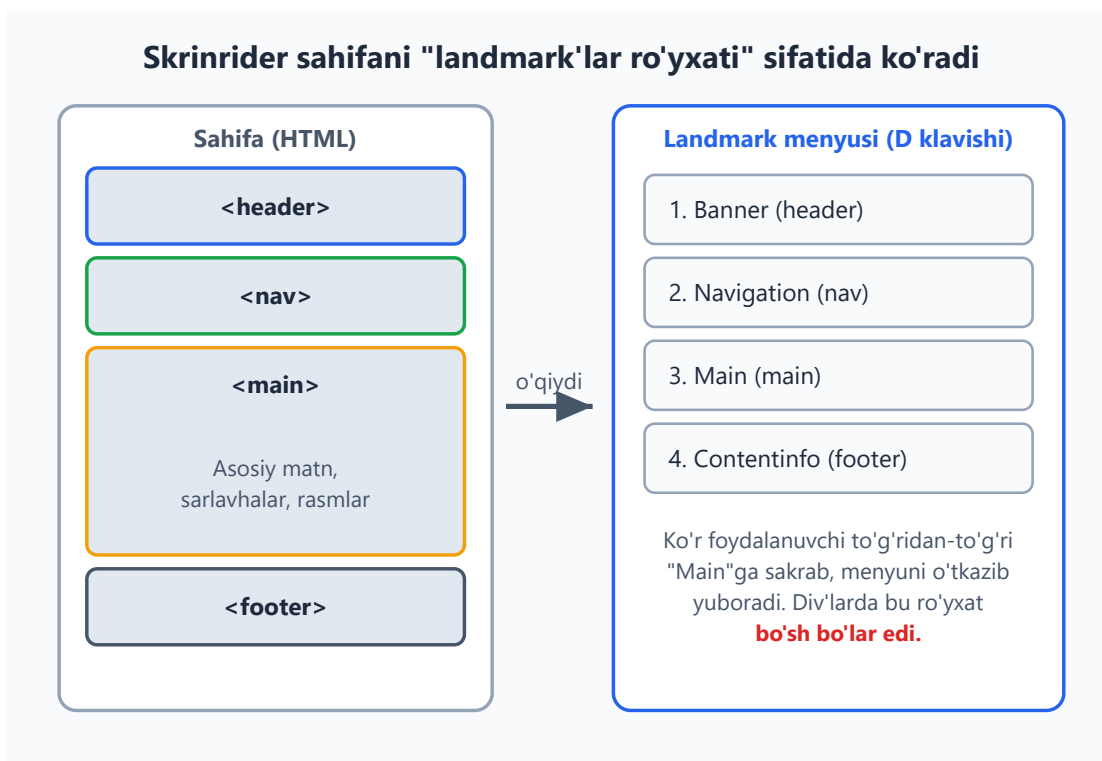
Buni tushunsang, a11y'ning ko'p qoidolari mantiqan ravshan bo'ladi.

Skrinrider — bu ekrandagi narsalarni **ovoz bilan o'qib beradigan** dastur (masalan NVDA, JAWS, VoiceOver). Ko'zi ojiz foydalanuvchi ekranni ko'rmaydi — u faqat eshitadi va klaviatura bilan boshqaradi.

Muhim tushuncha: skrinrider sening **vizual dizayningni ko'rmaydi**. U HTML **strukturasini** o'qiydi. Agar struktura ma'noli bo'lsa — foydalanuvchi sahifada erkin harakat qiladi; agar "div sho'rva" bo'lsa — u adashadi.

Skrinrider foydalanuvchisi sahifada bir necha usulda harakat qiladi:

- **Ketma-ket** (yuqoridan pastga element-element).
- **Landmark'lar bo'yicha** — `<nav>`, `<main>`, `<footer>` kabi katta bo'laklar orasida sakrash.
- **Sarlavhalar bo'yicha** — `<h1>` — `<h6>` orasida sakrash (yuqorida ko'rdik).
- **Havolalar/tugmalar ro'yxati bilan** — barcha bosiladigan elementlarni bir ro'yxatga yig'ib.



Rasmda ko'rinib turibdiki, semantik teglar skrinrider uchun avtomatik "tezkor sayohat menyusi" yaratadi. Foydalanuvchi bir tugma bosib to'g'ridan-to'g'ri "Main"ga o'tib, takrorlanadigan navigatsiyani har sahifada qaytadan eshitmaydi.

Tajriba uchun maslahat: Windows'da NVDA (bepul), Mac'da VoiceOver (`Cmd+F5`) o'rnatilgan. Bir marta ko'zingni yumib, sahifangni faqat ovoz bilan boshqarib ko'r — bu seni a11y'ni "his qilishga" o'rgatadi.

6.7 `alt` matn va `lang` atributi — eng oddiy, eng katta ta'sir

A11y'da eng arzon, lekin eng katta foyda beradigan ikki narsa.

Rasmlar uchun `alt`

`<img>` ning `alt` atributi — rasm ko'rinmaganda (skrinrider o'qiydi, yoki rasm yuklanmaydi) uning **o'rnini bosadigan matn**.

```
<!-- Yaxshi: rasm nimani ko'rsatishini tasvirlaydi -->


<!-- alt yo'q — skrinrider fayl nomini o'qiydi: "mushuk nuqta jpg" — foydasiz -->

```

alt qanday yoziladi: - Rasm *nimani bildirishini* yoz, "rasm", "surat" so'zini qo'shma (skrinrider o'zi "rasm" deydi). - Agar rasm sof **bezak** bo'lsa (ma'no bermaydi) — `alt=""` (bo'sh) qoldir. Bu skrinrider'ga "buni o'tkazib yubor" deydi. ⚠ `alt` ni butunlay tashlab ketma — bo'sh `alt=""` va `alt` siz `<img>` farq qiladi.

```
<!-- Bezak rasm: bo'sh alt — skrinrider e'tibor bermaydi -->

```

Til uchun `lang`

`<html lang="uz">` — sahifa qaysi tilda ekanini bildiradi. Skrinrider buni o'qib, **to'g'ri talaffuz** mexanizmini tanlaydi. `lang` siz inglizcha skrinrider o'zbekcha matnni ingliz aksenti bilan o'qishga urinadi — natija tushunarsiz bo'ladi.

```
<html lang="uz">
...
<!-- Sahifa ichidagi boshqa tildagi bo'lakni belgilash -->
<p>Bu inglizcha ibora: <span lang="en">Hello world</span></p>
</html>
```

💡 `lang="uz"` qo'yish — bir soniyalik ish, lekin o'zbekcha sayt uchun katta farq qiladi. Har doim `<html>` tegiga `lang` qo'y.

6.8 Klaviatura navigatsiyasi, `tabindex` va focus tartibi

Ko'p odamlar saytni **sichqonchasiz**, faqat klaviatura bilan ishlatadi (qo'l harakati cheklanganlar, kuchli foydalanuvchilar, skrinrider foydalanuvchilari). Sayting klaviatura bilan to'liq ishlashi shart.

Asosiy klavishlar: - `Tab` — keyingi interaktiv elementga o'tish (havola, tugma, input). - `Shift + Tab` — orqaga. - `Enter` / `Probel` — bosish (aktivlashtirish).

Focus — bu hozir klaviatura "qaysi elementda turganini" bildiradi. Brauzer fokuslangan elementga ko'rinadigan **chegara (focus ring)** chizadi.

⚠ **Eng katta a11y xatosi:** dizayn "chiroyli" ko'rinsin deb focus ringni o'chirib qo'yish:

```
/* Hech qachon bunday qilma! Klaviatura foydalanuvchisi qayerdaligini  
bilmay qoladi */  
:focus { outline: none; }
```

Agar standart ringni almashtirmoqchi bo'lsang — uni **boshqa** ko'rinadigan belgi bilan **almashtir**, butunlay o'chirma:

```
/* Yaxshi: standart ringni o'z uslubing bilan almashtir */  
:focus-visible {  
  outline: 3px solid #2563eb;  
  outline-offset: 2px;  
}
```

📌 **:focus-visible** — bu zamonaviy, aqlli variant: faqat klaviatura bilan kelganda ring ko'rsatadi, sichqoncha bilan bosganda yo'q (ko'pchilik sichqoncha foydalanuvchisini bezovta qiladigan ring kerak emas).

tabindex

tabindex elementning klaviatura tartibidagi o'rnini boshqaradi:

Qiymat	Ma'nosi	Qachon
<code>tabindex="0"</code>	Tabindex tartibiga qo'sh (tabiiy joyida)	Maxsus interaktiv element (kamdan-kam)
<code>tabindex="-1"</code>	Tab bilan o'tib bo'lmaydi , lekin JS bilan fokus berish mumkin	Modal/xabarga programma orqali fokus
<code>tabindex="1"</code> va katta	Tartibni majburan o'zgartirish	⚠ Deyarli hech qachon — ishlatma!

⚠ **tabindex musbat son (1, 2, 3...) ishlatma.** U tabiiy tartibni buzadi va boshqarish dahshatli bo'lib ketadi. Tabiiy DOM tartibi deyarli har doim to'g'ri tartib — elementlarni HTML'da **mantiqiy ketma-ketlikda** yozsang, focus tartibi o'zidan to'g'ri bo'ladi.

💡 **Nega "to'g'ri teg" muhim:** `<button>`, `<a href>`, `<input>` kabi tabiiy interaktiv teglar **o'zidan** klaviaturaga fokuslanadi va Enter/Probel bilan ishlaydi. Agar `<div onclick="...">` bilan "tugma" yarasang — u klaviatura uchun ko'rinmas bo'ladi,

`tabindex`, `role`, klavish-ishlovchi — hammasini qo'lda qo'shishingga to'g'ri keladi. Shuning uchun **bosiladigan narsa uchun** `<button>` ishlat, `<div>` emas.

6.9 ARIA asoslari — qachon kerak, qachon kerak emas

ARIA (Accessible Rich Internet Applications) — bu HTML'ga qo'shiladigan maxsus atributlar to'plami. Ular yordamchi texnologiyalarga (skrinrider) elementning **roli**, **holati** va **xususiyatlari** haqida qo'shimcha ma'lumot beradi.

Lekin ARIA haqida gapirishdan **oldin** eng muhim qoidani aytamiz:

✦ **Birinchi ARIA qoidasi:** *"Hech qanday ARIA — yomon ARIA'dan yaxshiroq"* (No ARIA is better than bad ARIA). Agar maqsadga **tabiiy semantik HTML** bilan yetsang — ARIA **ishlatma**. ARIA hech qanday ko'rinadigan xulq qo'shmaydi, faqat yordamchi texnologiyaga "va'da" beradi — agar bu va'da noto'g'ri bo'lsa, foydalanuvchini chalg'itadi.

Bu nega shunchalik muhim? Chunki ko'p ARIA "tashqaridan o'rganilgan" sahifalarda noto'g'ri ishlatiladi va **mavjud, ishlaydigan** semantikani **buzadi**:

```
<!-- Yomon: keraksiz role — h1 allaqachon "heading" -->
<h1 role="heading">Sarlavha</h1>

<!-- Yomon: button allaqachon "button" rolida, takror -->
<button role="button">Bosing</button>

<!-- Yaxshi: hech narsa qo'shilmaydi, teg o'zi yetarli -->
<h1>Sarlavha</h1>
<button>Bosing</button>
```

Unda ARIA qachon kerak?

ARIA **HTML'da tabiiy teg yo'q** bo'lganda kerak — masalan, JavaScript bilan yasalgan murakkab vidjet (tab paneli, drag-and-drop, maxsus dropdown). Boshlovchi sifatida senga eng ko'p kerak bo'ladigan uchasi:

1. `aria-label` — ko'rinadigan matni bo'lmagan elementga skrinrider uchun nom beradi:

```
<!-- Faqat ikonkali tugma — skrinrider o'qiy oladigan matn yo'q -->
<button aria-label="Qidiruvni yopish">&times;</button>

<!-- nav'lar ajratish uchun -->
```

```
<nav aria-label="Asosiy menyu">...</nav>
<nav aria-label="Pastki menyu">...</nav>
```

2. **aria-hidden="true"** — elementni skrinrider'dan **yashiradi** (lekin ekranda ko'rinadi). Sof bezak ikonkalar uchun:

```
<!-- Ikonka bezak; yonidagi matn ma'noni beradi -->
<button>
  <span aria-hidden="true">🔍</span>
  Qidirish
</button>
```

3. **role** — elementning rolini bildiradi. Faqat tabiiy teg yo'q bo'lganda:

```
<!-- Ogohlantirish: muhim, darhol o'qilishi kerak bo'lgan xabar -->
<div role="alert">Parol noto'g'ri!</div>
```

⚠️ **aria-label** ni `<div>` yoki `<span>` kabi *ma'nosi yo'q* teglarga qo'yma — u e'tiborga olinmasligi mumkin. U interaktiv elementlarga (`<button>` , `<a>` , `<input>`) yoki landmark'larga (`<nav>`) ishlaydi.

Xulosa: semantik HTML'ni birinchi tanla. ARIA — tabiiy HTML yetmagan kamdan-kam holatlar uchun "yamoq". Kuchli HTML poydevori bo'lsa, senga ARIA juda kam kerak bo'ladi.

6.10 Landmark'lar — sahifaning "yo'l belgilari"

Landmark (mo'ljal, yo'l belgisi) — bu skrinrider foydalanuvchisi to'g'ridan-to'g'ri sakrab o'tishi mumkin bo'lgan sahifaning katta bo'laklari. Ajoyib tomoni: **semantik teglar o'zi avtomatik landmark yaratadi** — qo'shimcha hech narsa qilish shart emas.

Semantik teg	Avtomatik landmark roli
<code><header></code> (sahifa darajasida)	<code>banner</code>
<code><nav></code>	<code>navigation</code>
<code><main></code>	<code>main</code>
<code><aside></code>	<code>complementary</code>
<code><footer></code> (sahifa darajasida)	<code>contentinfo</code>

Semantik teg	Avtomatik landmark roli
<code><form></code> (nomli)	<code>form</code>
<code><section></code> (nomli)	<code>region</code>

Mana **nega** bu kuchli: skrinrider foydalanuvchisi bitta klavish bilan butun navigatsiyani o'tkazib, to'g'ridan-to'g'ri `<main>` ga o'tadi. `<div>` larda bunday "tezkor sayohat" mumkin emas.

Bir nechta bir xil landmark bo'lsa — ularni nomla. Masalan sahifada ikkita `<nav>` bo'lsa (yuqori menyu va pastki menyu), skrinrider ularni "navigation, navigation" deb o'qiya — qaysi biri qaysi? `aria-label` bilan ajratamiz:

```
<nav aria-label="Asosiy navigatsiya">
  <a href="/">Bosh sahifa</a>
  <a href="/maqolalar">Maqolalar</a>
</nav>

<main>...</main>

<nav aria-label="Footer havolalari">
  <a href="/maxfiylik">Maxfiylik</a>
  <a href="/shartlar">Shartlar</a>
</nav>
```

Endi skrinrider "Asosiy navigatsiya" va "Footer havolalari" deb aniq aytadi.

"Skip link" — kuchli a11y nayrang

Klaviatura/skrinrider foydalanuvchisi har sahifada birinchi navbatda navigatsiya menyusiga duch keladi — har safar o'nlab havolani o'tib, asosiy mazmunga yetib borishi zerikarli. **Skip link** ("o'tkazib yuborish havolasi") buni hal qiladi:

```
<body>
  <a href="#main-content" class="skip-link">Asosiy mazmunga o'tish</a>

  <header>...</header>
  <nav>...</nav>

  <main id="main-content">
    ...
  </main>
</body>
```

```

/* Odatda yashirin; faqat klaviatura bilan fokuslanganda ko'rinadi */
.skip-link {
  position: absolute;
  left: -9999px;
}
.skip-link:focus {
  left: 8px;
  top: 8px;
  background: #2563eb;
  color: #fff;
  padding: 8px 12px;
}

```

Tab bosgan birinchi foydalanuvchi shu havolani ko'radi; Enter bosib to'g'ridan-to'g'ri `<main>` ga sakraydi. Ko'rar sichqoncha foydalanuvchisi uni umuman sezmaydi. Bu — a11y'ning "hammaga to'sqinlik qilmasdan, muhtojlarga yordam berish" falsafasining ajoyib namunasi.

6.11 Hammasini birlashtirish — to'liq hammabop sahifa

Bu bobda o'rganganlarimizning hammasini bitta misolda ko'ramiz:

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Blog - Semantik HTML</title>
</head>
<body>
  <!-- Skip link: klaviatura uchun tezkor o'tish -->
  <a href="#asosiy" class="skip-link">Asosiy mazmunga o'tish</a>

  <header>
    <h1>Web Dasturchi Blogi</h1>
  </header>

  <nav aria-label="Asosiy navigatsiya">
    <a href="/">Bosh sahifa</a>
    <a href="/maqolalar">Maqolalar</a>
    <a href="/aloqa">Aloqa</a>
  </nav>

  <main id="asosiy">
    <article>
      <header>
        <h2>Semantik HTML nima uchun muhim?</h2>

```

```

    <p>Nashr sanasi: <time datetime="2026-06-09">9-iyun,
2026</time></p>
  </header>

  <section>
    <h3>Kirish</h3>
    <p>Semantik teglar kodingni hammaga ochiq qiladi...</p>
    
  </section>

  <section>
    <h3>Xulosa</h3>
    <p>To'g'ri HTML – a11y'ning poydevori.</p>
  </section>

  <footer>
    <p>Muallif: Ali Valiyev</p>
  </footer>
</article>
</main>

<aside aria-label="Bog'liq maqolalar">
  <h2>O'qish tavsiya etiladi</h2>
  <ul>
    <li><a href="#">CSS Flexbox asoslari</a></li>
    <li><a href="#">Responsive dizayn</a></li>
  </ul>
</aside>

<footer>
  <p>&copy; 2026 Web Dasturchi Blogi. Barcha huquqlar himoyalangan.
</p>
</footer>
</body>
</html>

```

Diqqat qil: bu sahifa **bironta ham role ishlatmaydi** — semantik teglarning o'zi hamma landmark'ni beradi. `aria-label` faqat takrorlanadigan `<nav>` / `<aside>` ni ajratish uchun ishlatilgan. `<time datetime="...">` — sanaga mashinaga tushunarli format beradi. Bu — "no ARIA is better than bad ARIA" falsafasining amaliy ko'rinishi.

💡 **Tekshiruvchi vositalar:** brauzer DevTools'da **Lighthouse** (Accessibility hisoboti) va **axe DevTools** kengaytmasi sahifangdagi a11y muammolarini avtomatik topadi. Boshlash uchun ajoyib — lekin ular hamma narsani topa olmaydi (masalan, `alt` matni *mazmunan* to'g'rimi yo'qmi — buni faqat odam tekshiradi).

Mashqlar

Quyidagi mashqlarni ketma-ket bajar. Har birida o'zing kod yozib, keyin yechimni och.

Mashq 1 — Div sho'rvani semantikaga aylantir

Quyidagi "div sho'rva"ni to'g'ri semantik teglar bilan qayta yoz:

```
<div class="top">
  <div class="title">Mening Saytim</div>
</div>
<div class="menu">
  <a href="/">Uy</a>
  <a href="/blog">Blog</a>
</div>
<div class="content">
  <div class="post">
    <h2>Birinch post</h2>
    <p>Salom dunyo!</p>
  </div>
</div>
<div class="bottom">© 2026</div>
```

Yechim

```
<header>
  <h1>Mening Saytim</h1>
</header>
<nav>
  <a href="/">Uy</a>
  <a href="/blog">Blog</a>
</nav>
<main>
  <article>
    <h2>Birinch post</h2>
    <p>Salom dunyo!</p>
  </article>
</main>
<footer>© 2026</footer>
```

`class="title"` ichidagi sarlavha matn `<h1>` bo'ldi (sahifa bosh sarlavhasi). `<main>` faqat o'sha sahifaga xos mazmun (post)ni o'rab oldi.

Mashq 2 — `article` yoki `section` ?

Quyidagi har bir holat uchun qaysi teg to'g'ri kelishini ayt va **negaligini** izohla:

1. Yangiliklar saytidagi bitta yangilik xabari.
2. Bitta uzun maqola ichidagi "Tarix" deb nomlangan bo'lim.

3. Onlayn do'kondagi bitta mahsulot kartochkasi.
4. Bosh sahifadagi "Tez-tez so'raladigan savollar" bloki.



Yechim



1. `<article>` — yangilik o'zicha mustaqil, uni RSS lentaga yoki boshqa joyga ko'chirsa ham mantiqan to'liq.
2. `<section>` — maqolaning bir qismi, o'zicha mustaqil emas, sarlavhasi bor ("Tarix").
3. `<article>` — mahsulot kartochkasini olib boshqa joyga (masalan tavsiyalar ro'yxatiga) ko'chirsa ham to'liq.
4. `<section>` — bosh sahifaning sarlavhali mavzuli bo'lagi.

Mashq 3 — Sarlavhalar ierarxiyasini tuzat

Bu kodda ierarxiya buzilgan. Xatoni top va tuzat:

```
<h1>Pishiriqlar retsepti</h1>
<h3>Kerakli mahsulotlar</h3>
<h4>Xamir uchun</h4>
<h2>Tayyorlash bosqichlari</h2>
```



Yechim



```
<h1>Pishiriqlar retsepti</h1>
<h2>Kerakli mahsulotlar</h2>
<h3>Xamir uchun</h3>
<h2>Tayyorlash bosqichlari</h2>
```



Xato: `<h1>` dan keyin to'g'ridan-to'g'ri `<h3>` ga sakrash (darajani o'tkazib yuborish). "Kerakli mahsulotlar" — bosh mavzuning to'g'ridan-to'g'ri bo'limi, demak `<h2>`. "Xamir uchun" esa uning ichidagi kichik bo'limcha, demak `<h3>`. Sarlavhani **ko'rinish** uchun emas, **ierarxiya** uchun tanlaymiz.

Mashq 4 — `alt` matnini yoz

Quyidagi rasmlar uchun mos `alt` matnini yoz:

1. Maqola ichidagi grafik: 2025-yil sotuvlarining o'sish diagrammasi.
2. Saytni chiroyli qilish uchun qo'yilgan to'lqinsimon ajratuvchi chiziq (sof bezak).

3. "Yuklab olish" tugmasi ichidagi pastga qaragan strelka ikonkasi (yonida "Yuklab olish" matni bor).

Yechim

```
<!-- 1: ma'noli rasm - nima ko'rsatilayotganini yoz -->


<!-- 2: sof bezak - bo'sh alt, skrinrider o'tkazib yuboradi -->


<!-- 3: yonida matn bor - ikonka takror ma'no bermaydi, bo'sh alt -->
<button>
  
  Yuklab olish
</button>
```

3-holatda "Yuklab olish" so'zi allaqachon bor, shuning uchun ikonkaning `alt` ini bo'sh qoldiramiz — aks holda skrinrider "Yuklab olish Yuklab olish" deb takrorlaydi.

Mashq 5 — Focus ringni to'g'rilash

Bir dasturchi dizayn "tozaroq" ko'rinsin deb shunday yozgan. Muammo nimada va qanday tuzatasan?

```
button:focus,
a:focus {
  outline: none;
}
```

Yechim

Muammo: focus ring butunlay o'chirilgan. Klaviatura foydalanuvchisi `Tab` bosganda qaysi elementda turganini **ko'rmaydi** — sayt ular uchun ishlab bo'lmaydigan holatga keladi.

Yechim: ringni o'chirma — uni o'z uslubing bilan **almashtir** (va imkon bo'lsa `:focus-visible` ishlat):

```
button:focus-visible,
a:focus-visible {
  outline: 3px solid #2563eb;
  outline-offset: 2px;
}
```

Mashq 6 — ARIA: kerakmi yoki keraksizmi?

Quyidagi har bir qator uchun ARIA to'g'ri ishlatilganmi yoki keraksizmi (yoki noto'g'rimi) ayt:

```
<a href="/about" role="link">Biz haqimizda</a>
<button aria-label="Menyuni yopish">&times;</button>
<nav role="navigation">...</nav>
<div role="button" onclick="...">Bosing</div>
```

Yechim

1. `<a role="link">` — **keraksiz**. `<a href>` allaqachon "link" rolda. ARIA olib tashlanadi.
2. `<button aria-label="Menyuni yopish">×</button>` — **to'g'ri**. Tugmada faqat `x` belgisi bor, o'qiladigan matn yo'q; `aria-label` skrinrider'ga ma'no beradi.
3. `<nav role="navigation">` — **keraksiz**. `<nav>` o'zi "navigation" landmark'ini yaratadi. ARIA olib tashlanadi.
4. `<div role="button">` — **ishlaydi, lekin yomon yondashuv**. Bu yerda `<button>` ishlatish kerak edi — u tabiiy fokuslanadi va Enter/Probel bilan ishlaydi. `<div>` bilan bularni qo'lda qo'shishga to'g'ri keladi. *Tabiiy teg yo'q bo'lgandagina* `role` ishlat.

Mashq 7 — Skip link qo'sh

Quyidagi sahifaga klaviatura foydalanuvchisi navigatsiyani o'tkazib, to'g'ridan-to'g'ri asosiy mazmunga sakrashi uchun "skip link" qo'sh (HTML va CSS):

```
<body>
  <nav>...</nav>
  <main>...</main>
</body>
```



Yechim



```
<body>
  <a href="#main" class="skip-link">Asosiy mazmunga o'tish</a>
  <nav>...</nav>
  <main id="main">...</main>
</body>
```

```
.skip-link {
  position: absolute;
  left: -9999px;      /* odatda ekrandan tashqarida - yashirin */
}
.skip-link:focus {
  left: 8px;          /* faqat fokuslanganda ko'rinadi */
  top: 8px;
  background: #2563eb;
  color: #fff;
  padding: 8px 12px;
}
```

`<main>` ga `id="main"` qo'shildi, havola `href="#main"` orqali unga ishora qiladi.

Mashq 8 — To'liq semantik sahifa qur

Noldan boshlab quyidagini o'z ichiga olgan to'liq, hammabop HTML sahifa yoz: `lang` atributi, skip link, `<header>` + `<h1>`, nomlangan `<nav>`, ichida `<article>` bo'lgan `<main>`, `<aside>`, `<footer>`. Maqolada kamida bitta `alt` matnli rasm bo'lsin.



Yechim namunasi



```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sayohat blogi</title>
</head>
<body>
  <a href="#asosiy" class="skip-link">Asosiy mazmunga o'tish</a>

  <header>
    <h1>Sayohat Blogi</h1>
  </header>

  <nav aria-label="Asosiy navigatsiya">
    <a href="/">Bosh sahifa</a>
    <a href="/yozuvlar">Yozuvlar</a>
  </nav>

  <main id="asosiy">
    <article>
      <h2>Samarqandga sayohat</h2>
      
      <p>Bu safarda men Registonni ziyorat qildim...</p>
    </article>
  </main>

  <aside aria-label="Bog'liq yozuvlar">
    <h2>Yana o'qing</h2>
    <ul>
      <li><a href="#">Buxoroga sayohat</a></li>
    </ul>
  </aside>

  <footer>
    <p>&copy; 2026 Sayohat Blogi</p>
  </footer>
</body>
</html>
```

Hech qanday `role` ishlatilmadi — semantik teglar yetarli. `aria-label` faqat takrorlanadigan landmark'larni ajratish uchun ishlatildi.



Oldingi: 05 — Formalar ·  README · Keyingi: 07 — Head, meta va SEO



07 — Head, meta va SEO

← Oldingi: 06 — Semantik HTML va accessibility · 🏠 README · Keyingi: 08 — CSS asoslari →

Bu bobda: sahifaning ko'rinmas qismi — `<head>` ichidagi meta teglar, viewport, favicon, CSS/JS ulash, hamda saytni Google va ijtimoiy tarmoqlar uchun tayyorlaydigan SEO va ulashish sozlamalari.

Oldingi boblarda biz sahifaning **ko'rinadigan** qismini — `<body>` ichidagi matn, rasm, ro'yxat va havolalarni o'rgandik. Endi sahifaning **ko'rinmas** qismiga — `<head>` ga o'tamiz.

Tasavvur qiling: kitobning muqovasi va ichidagi pasport ma'lumotlari (kim yozgan, qaysi tilda, qaysi yilda) bor. O'quvchi asosan ichki matnni o'qiydi, lekin kutubxonachi kitobni javonga to'g'ri qo'yish, kataloglashtirish va izlash uchun aynan o'sha pasport ma'lumotlariga qaraydi. `<head>` — bu sahifaning pasporti. Uni odam ko'rmaydi, lekin **brauzer, Google va ijtimoiy tarmoqlar** o'qiydi.

Bu bob ko'rinishidan kichik, lekin amalda u **professional sayt** bilan **havaskor sahifa** o'rtasidagi farqni belgilaydi. Boshladik.

7.1 `<head>` nima va u qayerda turadi

Har bir HTML hujjat ikki qismdan iborat: `<head>` (bosh) va `<body>` (tana). Ikkalasi `<html>` ichida yashaydi:

```
<!DOCTYPE html>
<html lang="uz">
  <head>
    <!-- Bu yer KO'RINMAYDI: meta ma'lumotlar -->
    <meta charset="UTF-8">
    <title>Mening saytim</title>
  </head>
  <body>
    <!-- Bu yer KO'RINADI: sahifaning haqiqiy mazmuni -->
    <h1>Salom, dunyo!</h1>
  </body>
</html>
```

Nega ular ajratilgan? Chunki ularning vazifasi boshqacha:

- `<body>` — odamga ko'rsatiladigan mazmun (matn, rasm, tugma).
- `<head>` — mashinalar uchun ma'lumot: "Bu sahifa qaysi tilda? Nomi nima? Qaysi CSS faylni ulash kerak? Google'da qanday tasvirlanadi?"

`<head>` ichiga yozilgan deyarli hech narsa ekranga chiqmaydi (yagona istisno — `<title>`, u tab nomida ko'rinadi). Shuning uchun yangi boshlovchilar ko'pincha uni e'tiborsiz qoldiradi. Bu xato — `<head>` to'g'ri to'ldirilmagan sayt Google'da topilmaydi va telefonda buzilib ko'rinadi.

Quyidagi diagramma `<head>` ichida odatda nimalar turishini ko'rsatadi:



✦ `<head>` ichidagi teglarning **ko'pchiligi yopilmaydi** (`<meta>`, `<link>` — yakka teg). Faqat `<title>` ochilib-yopiladi.

7.2 `<title>` — tab nomi va Google sarlavhasi

`<title>` — `<head>` ichidagi eng muhim teglardan biri. U **ikki joyda** ko'rinadi:

1. Brauzer **tab** (yorliq) ustida — siz qaysi sahifada ekaningizni shu yerdan bilasiz.
2. Google qidiruv natijasida **ko'k katta sarlavha** sifatida — odamlar shuni bosadi.

```

<head>
  <meta charset="UTF-8">
  
```

```
<title>Pishloqli pitsa retsepti – Oshxonam</title>
</head>
```

Natija: tabda "Pishloqli pitsa retsepti — Oshxonam" yoziladi; Google'da ham xuddi shu matn ko'k havola bo'lib chiqadi.

Nega title shunchalik muhim? Chunki odam Google'da 10 ta natijani ko'radi va aynan **title**ga qarab qaysi birini bosishni hal qiladi. Yomon title = kam tashrif.

Yaxshi `<title>` qoidalari:

- **Aniq va mazmunli** bo'lsin: "Bosh sahifa" emas, "Toshkentda pitsa yetkazib berish — Oshxonam".
- **Qisqa**: Google taxminan 55–60 belgini ko'rsatadi, qolgani `...` bo'lib kesiladi.
- **Har sahifada noyob** bo'lsin: ikki sahifa bir xil title'ga ega bo'lmasin.
- Odatda formati: `Sahifa nomi – Brend nomi`.

⚠ Eng keng tarqalgan xato — `<title>` ni umuman yozmaslik. Unda tabda xunuk fayl yo'li (`index.html`) chiqadi, Google esa o'zicha biror matn tanlaydi — odatda yomon tanlaydi.

💡 `<title>` `<body>` ichidagi `<h1>` dan farq qiladi: `<title>` brauzer/Google uchun, `<h1>` esa sahifa ichidagi ko'rinadigan bosh sarlavha. Ikkalasi o'xshash bo'lishi mumkin, lekin bir xil bo'lishi shart emas.

7.3 `<meta charset="UTF-8">` — harflar to'g'ri ko'rinishi uchun

Kompyuter aslida faqat raqamlarni biladi. Har bir harf — bu raqam. "Qaysi raqam qaysi harf" degan jadval **kodlash** (encoding) deyiladi. `charset` — sahifa qaysi kodlashda yozilganini brauzerga aytadi.

```
<head>
  <meta charset="UTF-8">
  <title>Salom dunyo</title>
</head>
```

Nega aynan UTF-8? Chunki UTF-8 — dunyodagi deyarli barcha harflarni (o'zbek `o'`, `g'`, `sh`, `ch`, emoji, xitoy ieroglifi) qamrab oladigan yagona standart. Boshqa eski kodlashlar faqat ingliz alifbosini biladi.

Agar charset bo'lmasa nima bo'ladi? Brauzer kodlashni o'zicha taxmin qiladi va ko'pincha adashadi. Natijada o'zbekcha o'rniga ekranda Å²Ã§Ã© kabi tushunarsiz belgilar (ularni "krakozyabra" yoki "mojibake" deyishadi) chiqadi.

Ikki muhim qoida:

1. `charset` `<head>` ning **eng birinchi qatori** bo'lishi kerak (`<title>` dan ham oldin). Sababi: brauzer harflarni to'g'ri o'qishni boshlashi uchun kodlashni *darrov* bilishi shart.
2. Doimo **UTF-8** ishlating. Boshqasini tanlashga sabab yo'q.

✦ Hozirgi barcha matn muharrirlari (VS Code) faylni avtomatik UTF-8 da saqlaydi, shuning uchun bu teg bilan fayl kodlashi mos keladi — siz faqat tegni yozishni unutmang.

7.4 `<meta name="viewport">` — telefonda to'g'ri ko'rinish

Bu eng muhim meta teglardan biri va boshlovchilar uni eng ko'p unutadi. U **mobil moslashuv** (responsive design) uchun shart.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Muammoni tushunamiz. Telefonlar paydo bo'lganda, ko'p saytlar faqat katta kompyuter ekrani uchun yozilgan edi. Telefon brauzerlari shu saytlarni buzmaslik uchun bir hiyla o'ylab topdi: ular o'zini **980px enli keng ekran** kabi tutadi, keyin butun sahifani kichraytirib telefon ekraniga sig'diradi.

Natija: matn juda mayda bo'lib, o'qish uchun barmoq bilan kattalashtirish (zoom) va chapga-o'ngga surish kerak bo'ladi.

`viewport` tegi brauzerga aytadi: **"Yo'q, soxta 980px kerak emas. Sahifa enini haqiqiy qurilma eniga (`device-width`) teng qil."** Shunda CSS sizning telefon ekraningizni haqiqiy o'lchamda ko'radi va mazmun ekranga moslashadi.

`content` ichidagi ikki qism:

- `width=device-width` — sahifa eni = qurilmaning haqiqiy eni. **Eng muhim qism.**
- `initial-scale=1.0` — sahifa ochilganda zoom darajasi 1 ga teng (kattalashtirilmagan, kichraytirilmagan).

Quyidagi taqqos bu farqni yaqqol ko'rsatadi:

Viewport meta tegi: bor va yo'q holat

Viewport YO'Q

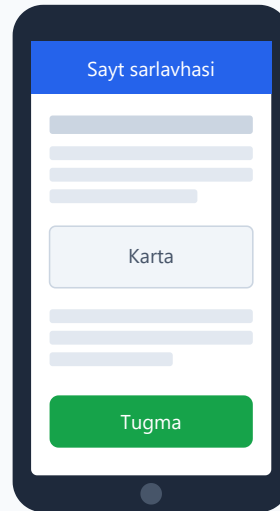
brauzer 980px deb o'ylaydi



Matn juda mayda, gorizontal scroll

Viewport BOR

width=device-width



Ekranga moslashgan, o'qish oson

⚠ Agar `viewport` tegi bo'lmasa, hatto eng zo'r responsive CSS ham telefonda ishlamaydi — chunki brauzer hali ham o'zini 980px deb o'ylaydi. Bu teg responsive dizaynning **kaliti**.

💡 Mobil moslashuvning o'zini (media query, flexbox bilan) keyingi boblarda chuqur o'rganamiz. Hozircha eslab qoling: har bir sahifaga bu bitta qatorni qo'shing, u bepul va majburiy.

7.5 `<meta name="description">` — Google'dagi qisqa tavsif

`description` — sahifa haqida bir-ikki jumlati tavsif. U **ekranda ko'rinmaydi**, lekin Google qidiruv natijasida title ostidagi **kulrang matn** sifatida ishlatilishi mumkin.

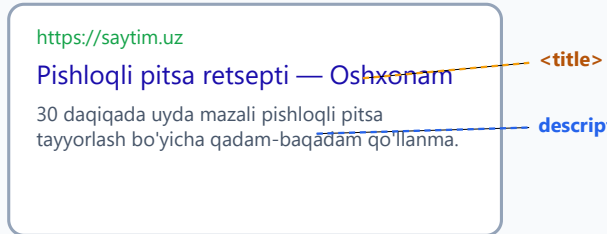
```
<meta name="description" content="30 daqiqada uyda mazali pishloqli  
pitsa tayyorlash bo'yicha qadam-baqadam qo'llanma.">
```

Nega kerak? Odam Google'da natijani ko'rganda, title'dan keyin shu tavsifni o'qiydi va "bu menga keraklimi?" deb hal qiladi. Yaxshi tavsif ko'proq bosish (klik) keltiradi.

Quyidagi diagramma title va description Google natijasida qanday joylashishini ko'rsatadi (chap tomon):

Meta teglar tashqarida qanday ko'rinadi

Google qidiruv natijasi



Ulashish kartasi (Open Graph)



Bitta sahifa — uch xil joyda

Tab nomi (title) · Google natijasi (title + description) · ijtimoiy karta (og:*)
Ularning hammasi <head> ichidagi ko'rinmas teglardan o'qiladi.

Yaxshi description qoidalari:

- Uzunligi taxminan **150–160 belgi** (Google shuncha ko'rsatadi).
- Sahifaning haqiqiy mazmunini tavsiflasin, aldamasin.
- Har sahifada noyob bo'lsin.
- Foydaga e'tibor bersin: "nima o'rganasiz / nima topasiz".

✦ Muhim nuans: **description reyting** (Google'da yuqori chiqish) ga to'g'ridan-to'g'ri ta'sir qilmaydi. Lekin u **bosish foiziga** (CTR — click-through rate) ta'sir qiladi: ko'proq odam bossa, sayt foydaliroq deb baholanadi. Shuning uchun yozishga arziydi.

7.6 Favicon — tabdagi kichik belgi

Favicon (favorite icon) — brauzer tabida, sahifa nomi yonida ko'rinadigan **kichkina belgi/logo**. Masalan, YouTube tabidagi qizil tugma yoki GitHub tabidagi mushukcha.

```
<link rel="icon" href="/favicon.ico">
```



Yoki zamonaviy PNG/SVG bilan:

```
<link rel="icon" type="image/png" href="/favicon.png">
```



Nega kerak? Favicon — saytingizning **brendi**. Odam 20 ta tab ochib o'tirganda, aynan kichik belgiga qarab sizning saytingizni tez topadi. Faviconsiz sayt esa bo'sh, tugallanmagandek ko'rinadi.

`<link>` tegining qismlari:

- `rel="icon"` — bu havolaning vazifasi: "bu fayl — sahifa belgisi (icon)".
- `href="..."` — belgi faylining manzili.
- `type="..."` — fayl turi (ixtiyoriy, lekin foydali).

💡 Eng oson yo'l: 32x32 yoki 48x48 px kvadrat rasm tayyorlab, uni `favicon.png` deb saytning bosh papkasiga qo'ying. Onlayn "favicon generator" saytlari bitta rasmdan barcha kerakli o'lchamlarni yasab beradi.

✦ Brauzerlar standart bo'yicha `/favicon.ico` faylini avtomatik izlaydi. Lekin `<link rel="icon">` ni aniq yozish — ishonchli va to'g'ri yo'l.

7.7 CSS va JS ni ulash: `<link>` va `<script>`

`<head>` ning yana bir asosiy vazifasi — sahifaga **tashqi fayllarni** (CSS uslublari va JavaScript kodi) ulash.

CSS ulash — `<link rel="stylesheet">`

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Mening saytim</title>
  <link rel="stylesheet" href="style.css">
</head>
```

Bu yer brauzerga aytadi: "`style.css` faylini yukla va uning uslublarni shu sahifaga qo'lla." `rel="stylesheet"` — havolaning turi (uslublar jadvali), `href` — CSS faylning manzili.

✦ CSS ni doimo `<head>` ichiga qo'ying. Sababi: brauzer mazmunni chizishdan **oldin** uslublarni bilishi kerak, aks holda sahifa avval bezaksiz, keyin to'satdan bezakli "sakrab" ko'rinadi (buni FOUC — Flash of Unstyled Content deyishadi).

JavaScript ulash — `<script>`

JavaScript faylni ulashning uchta usuli bor va ular tezlik uchun juda muhim:

```
<!-- 1) Oddiy (eskicha) -->
<script src="app.js"></script>

<!-- 2) defer -->
<script src="app.js" defer></script>

<!-- 3) async -->
<script src="app.js" async></script>
```

Farqni tushunish uchun brauzer sahifani **yuqoridan pastga** o'qishini eslang. U `<script>` ga yetganda, oddiy holatda **to'xtaydi**, JS faylni yuklab, ishga tushiradi, keyingina davom etadi.

Usul	Brauzer xulqi	Qachon ishlatish
Oddiy <code><script></code>	HTML o'qishni TO'XTATIB, JS ni yuklab- ishlatadi	Imkon qadar qochish kerak
<code>defer</code>	JS fonda yuklanadi, lekin butun HTML o'qib bo'lingach ishga tushadi	Eng yaxshi tanlov (deyarli har doim)
<code>async</code>	JS fonda yuklanadi va tayyor bo'lishi bilan darrov ishlaydi (tartibsiz)	Mustaqil skriptlar (masalan, analitika)

Nega `defer` eng yaxshi?

1. JS fayl HTML bilan **parallel** yuklanadi — sahifa tez ochiladi.
2. JS faqat HTML to'liq tayyor bo'lgach ishlaydi — demak, kodingiz sahifadagi elementlarni topa oladi (ular allaqachon mavjud).
3. Bir nechta `defer` skript yozilgan tartibida ishga tushadi — bashorat qilish oson.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Mening saytim</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
```

⚠ Klassik maslahat "skriptni `</body>` dan oldin qo'ying" eski va hali ham ishlaydi, lekin `<head>` da `defer` bilan yozish — zamonaviy va tozaroq usul. Ikkalasi ham JS ni sahifa tayyor bo'lgach ishga tushiradi.

💡 JavaScript'ning o'zini bu qo'llanmada o'rganmaymiz (u alohida til), lekin uni HTML ga **qanday ulashni** bilish muhim, chunki interaktiv saytlar shu bilan ishlaydi.

7.8 Open Graph — ijtimoiy tarmoqlarda chiroyli ulashish

Telegram, Facebook yoki WhatsApp'da biror sayt havolasini yuborganingizni eslang — ba'zan u shunchaki ko'k matn bo'lib qoladi, ba'zan esa **chiroyli karta**: rasm, sarlavha va qisqa tavsif bilan. Bu farqni **Open Graph** (qisqacha OG) teglari belgilaydi.

Open Graph — bu Facebook o'ylab topgan, hozir hamma ishlatadigan standart. U ijtimoiy tarmoqqa aytadi: "Bu sahifa ulashilganda mana shu rasm, shu sarlavha va shu tavsifni ko'rsat."

```
<head>
  <meta charset="UTF-8">
  <title>Pishloqli pitsa retsepti - Oshxonam</title>

  <!-- Open Graph teglari -->
  <meta property="og:title" content="Pishloqli pitsa retsepti">
  <meta property="og:description" content="30 daqiqada uyda mazali
pitsa tayyorlash.">
  <meta property="og:image"
content="https://saytim.uz/rasmlar/pitsa.jpg">
  <meta property="og:url" content="https://saytim.uz/pitsa">
  <meta property="og:type" content="website">
</head>
```

Asosiy to'rtta OG tegi:

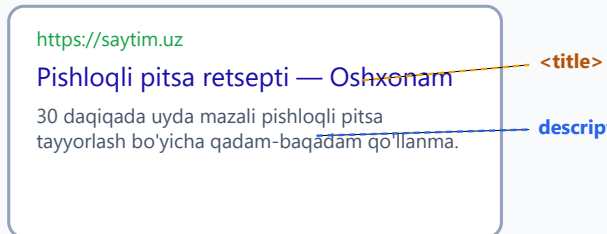
- `og:title` — kartadagi sarlavha (title'dan farqli, qisqaroq bo'lishi mumkin).
- `og:description` — kartadagi qisqa tavsif.
- `og:image` — kartadagi **rasm** (eng ko'p e'tibor tortadigan qism). Manzili **to'liq** (`https://...` bilan) bo'lishi shart.
- `og:url` — sahifaning rasmiy manzili.

Diqqat: `property`, `name` **emas**. Oddiy meta teglarda `name="..."` ishlatamiz, lekin OG teglarida `property="og:..."` ishlatiladi. Bu OG standartining talabi.

Quyidagi diagrammaning o'ng tomoni ulashish kartasi qanday ko'rinishini ko'rsatadi:

Meta teglar tashqarida qanday ko'rinadi

Google qidiruv natijasi



Ulashish kartasi (Open Graph)



Bitta sahifa — uch xil joyda

Tab nomi (title) · Google natijasi (title + description) · ijtimoiy karta (og:*)
Ularning hammasi <head> ichidagi ko'rinmas teglardan o'qiladi.

💡 `og:image` uchun tavsiya etilgan o'lcham — **1200x630 px**. Kichik yoki noto'g'ri nisbatdagi rasm karta ichida xunuk kesiladi.

✚ OG teglari saytingizni Google'da yuqori ko'tarmaydi, lekin odamlar havolani ulashganda ko'proq bosishadi — bu bepul reklama. Jiddiy har bir sayt OG teglarini yozadi.

7.9 Twitter Cards — qisqacha

Twitter/X o'zining qo'shimcha meta teglariga ega, lekin yaxshi xabar: agar Open Graph teglaringiz bo'lsa, Twitter ham ko'p hollarda ulardan foydalanadi. Faqat karta turini aniqlash uchun bir-ikki qator qo'shsangiz kifoya:

```
<meta name="twitter:card" content="summary_large_image">  
<meta name="twitter:title" content="Pishloqli pitsa retsepti">  
<meta name="twitter:image"  
content="https://saytim.uz/rasmlar/pitsa.jpg">
```

- `twitter:card` — karta turi. `summary_large_image` katta rasimli chiroyli karta beradi.
- Qolgan `twitter:*` teglari ixtiyoriy — yozmasangiz, Twitter `og:*` dan oladi.

✚ Diqqat: bu yerda yana `name="..."` ishlatiladi (OG'dagi `property` emas). Twitter standarti shunaqa.

💡 Boshlovchi uchun amaliy yondashuv: avval to'liq **Open Graph** yozing, keyin bitta `twitter:card` qatorini qo'shing — bu Twitter, Facebook va Telegram'ning hammasini qoplaydi.

7.10 SEO asoslari — Google sizni qanday topadi

SEO (Search Engine Optimization — qidiruv tizimi uchun moslash) — bu saytni Google'da yuqoriroq chiqishi uchun qiladigan ishlaringiz. SEO katta mavzu, lekin uning poydevori — **toza, ma'noli HTML**. Eng muhim asoslar:

1. Semantik HTML ishlating. Google `<div>` lar to'plamini emas, **ma'noli teglarni** yaxshi tushunadi: `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>`. Ular sahifaning tuzilishini robotga aytadi. (Bu teglarni keyingi bobda chuqur o'rganamiz.)

2. Sarlavhalarni to'g'ri ishlating. Har sahifada **bitta** `<h1>` bo'lsin (asosiy mavzu), keyin `<h2>`, `<h3>` ierarxiya bilan kichraysin. Sarlavha darajalarini bezak uchun emas, **ma'no** uchun tanlang.

```
<h1>Pishloqli pitsa retsepti</h1>
<h2>Kerakli masalliq</h2>
<h2>Tayyorlash bosqichlari</h2>
  <h3>1-bosqich: xamir</h3>
  <h3>2-bosqich: sous</h3>
```

3. Rasmlarga alt yozing. Har bir `<img>` ga tavsifiy `alt` matni qo'shing. U ko'r odamlar (skrin-rider) uchun ham, Google rasm qidiruvi uchun ham muhim.

```

```

4. Sayt tez ochilsin. Sekin sayt ham foydalanuvchini, ham Google'ni bezdiradi. Tezlikka ta'sir qiladigan oddiy narsalar: rasmlarni siqish, `defer` ishlatish, ortiqcha fayllarni kamaytirish.

5. Mobil moslashuv (`viewport`). Google **birinchi navbatda telefon** versiyasini baholaydi ("mobile-first"). Demak, 7.4-bo'limdagi `viewport` tegi — to'g'ridan-to'g'ri SEO masalasi.

✳️ Yodda tuting: SEO uchun birinchi va eng kuchli qadam — **odamlar uchun foydali, toza yozilgan sahifa**. Mashina o'qiydigan teglar (title, description) bu poydevorga qo'shimcha, uning o'rnini bosmaydi.

7.11 lang atributi — sahifaning tili

Bu kichik, lekin SEO va qulaylik (accessibility) uchun muhim atribut. U `<head>` ichida emas, balki `<html>` tegida turadi:

```
<html lang="uz">
```



`lang="uz"` brauzerga va Google'ga aytadi: "Bu sahifa **o'zbek tilida**." Bu nega kerak:

- **Skrin-rider** (ko'r odamlar uchun ekran o'qigich) matnni to'g'ri talaffuz qiladi — o'zbekcha matnni inglizcha urg'u bilan o'qimaydi.
- Brauzer to'g'ri **tarjima** taklif qiladi.
- Google sahifani to'g'ri tilga ajratadi.

💡 Boshqa tillar uchun: ingliz — `lang="en"`, rus — `lang="ru"`. Sahifa qaysi tilda yozilgan bo'lsa, shuni yozing.

7.12 canonical va robots — qaysi sahifani indekslash

Bular biroz ilg'or, lekin professional saytlarda doim uchraydi.

canonical — "asosiy" manzilni belgilash

Ba'zan bir xil sahifa bir nechta turli manzilda ochilishi mumkin (masalan `saytim.uz/pitsa` va `saytim.uz/pitsa?ref=telegram`). Google buni **takror mazmun** deb o'ylab, chalkashishi mumkin. `canonical` tegi aytadi: "Aslida asosiy manzil mana bu, shuni hisobga ol."

```
<link rel="canonical" href="https://saytim.uz/pitsa">
```



Nega muhim? Takror sahifalar SEO "kuchi"ni bo'lib yuboradi. Canonical hamma kuchni bitta asosiy manzilga jamlaydi.

robots — indekslashni boshqarish

`robots` tegi qidiruv robotiga sahifa bilan nima qilishni aytadi:

```
<!-- Indeks la va havolalarga ergash (standart, odatda yozish shart emas) -->  
<meta name="robots" content="index, follow">
```




```
<!-- Bu sahifani Google'ga qo'shma -->
<meta name="robots" content="noindex">
```

- `index` / `noindex` — sahifani qidiruvga qo'shish / qo'shmaslik.
- `follow` / `nofollow` — sahifadagi havolalarga ergashish / ergashmaslik.

Qachon `noindex` kerak? Masalan, "rahmat" sahifasi, admin paneli yoki test sahifasi — ularning Google natijasida chiqishi shart emas.

⚠ Ehtiyot bo'ling: butun saytga adashib `noindex` qo'ysangiz, sayt Google'dan butunlay yo'qoladi. Bu juda keng tarqalgan va og'ir xato.

7.13 JSON-LD — struktura ma'lumotlari (qisqacha)

Bu — boshlovchi uchun ilg'or, lekin **nima ekanini bilib qo'yish** kerak bo'lgan mavzu.

Struktura ma'lumotlari (structured data) — Google'ga sahifa mazmunini **aniq, mashina tushunadigan** shaklda aytib berish usuli. Masalan: "Bu sahifa — retsept; nomi 'Pitsa'; tayyorlash vaqti 30 daqiqa; reyting 4.8".

Eng tavsiya etiladigan format — **JSON-LD** — `<script>` ichiga yoziladigan maxsus ma'lumot bloki:

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Recipe",
  "name": "Pishloqli pitsa",
  "cookTime": "PT30M",
  "recipeIngredient": ["Xamir", "Pomidor sousi", "Pishloq"]
}
</script>
```

Nega buni qilish kerak? Chunki Google bu ma'lumotni o'qib, qidiruv natijasida **boyitilgan ko'rinish** (rich result) ko'rsatadi: yulduzli reyting, tayyorlash vaqti, rasm va h.k. Bunday natija oddiy ko'k havoladan ko'ra ko'proq e'tibor tortadi.

@type qiymatlari ko'p: `Recipe` (retsept), `Article` (maqola), `Product` (mahsulot), `Organization` (tashkilot) va boshqalar. To'liq ro'yxat **schema.org** saytida bor.

💡 Boshlovchi sifatida buni yodlash shart emas. Faqat shuni bilib qo'ying: "agar Google'da yulduzcha/narx/vaqt bilan chiroyli natija chiqarmoqchi bo'lsam, JSON-LD struktura ma'lumotlarini qo'shaman."

7.14 To'liq, professional `<head>` namunasi

Endi o'rgangan hamma narsani **bitta to'g'ri tartibli** `<head>` ga jamlaymiz. Buni shablon sifatida ishlatishingiz mumkin:

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <!-- 1) Kodlash (eng birinchi) -->
  <meta charset="UTF-8">

  <!-- 2) Mobil moslashuv -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- 3) Sahifa nomi va tavsifi -->
  <title>Pishloqli pitsa retsepti - Oshxonam</title>
  <meta name="description" content="30 daqiqada uyda mazali pishloqli pitsa tayyorlash bo'yicha qadam-baqadam qo'llanma.">

  <!-- 4) Favicon -->
  <link rel="icon" type="image/png" href="/favicon.png">

  <!-- 5) Ulashish (Open Graph) -->
  <meta property="og:title" content="Pishloqli pitsa retsepti">
  <meta property="og:description" content="30 daqiqada uyda mazali pitsa.">
  <meta property="og:image" content="https://saytim.uz/rasmlar/pitsa.jpg">
  <meta property="og:url" content="https://saytim.uz/pitsa">
  <meta property="og:type" content="website">
  <meta name="twitter:card" content="summary_large_image">

  <!-- 6) SEO yordamchilari -->
  <link rel="canonical" href="https://saytim.uz/pitsa">

  <!-- 7) CSS va JS -->
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <h1>Pishloqli pitsa retsepti</h1>
  <!-- ... -->
</body>
</html>
```

Bu tartib mantiqiy: avval brauzer ishlashi uchun zarur narsalar (`charset`, `viewport`), keyin sahifa ma'lumotlari (`title`, `description`), so'ng tashqi xizmatlar uchun (OG, canonical), oxirida resurslar (CSS, JS).

✦ Har bir teg bu yerda nima uchun turibdi — endi siz bilasiz. Bu shablonni yodlamang; tushunib, har sahifaga moslab to'ldiring.

Mashqlar

Quyidagi mashqlarni VS Code'da yangi `.html` fayl yaratib bajaring. Brauzerda ochib tekshiring.

1-mashq. Yangi `index.html` yarating va minimal, lekin to'g'ri `<head>` yozing: `charset`, `viewport` va mazmunli `<title>`. Faylni brauzerda oching va tab nomida title'ni ko'ring.

 **Yechim** 

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mening birinchi saytim</title>
</head>
<body>
  <h1>Salom!</h1>
</body>
</html>
```

2-mashq. Avvalgi sahifaga `<meta name="description">` qo'shing. Tavsif 150 belgi atrofida, sahifa mazmunini aldamasdan tasvirlasin.

 **Yechim** 

```
<meta name="description" content="Bu mening HTML o'rganish jarayonidagi birinchi sahifam. Bu yerda men head, meta teglar va SEO asoslarini mashq qilyapman.">
```

3-mashq. `viewport` tegini sahifadan **vaqtincha o'chiring**, telefon rejimida (brauzer DevTools'da F12 → qurilma rejimi) qanday ko'rinishini ko'ring, keyin qaytarib qo'ying. Farqni o'z so'zlaringiz bilan tushuntiring.



Yechim / maslahat



Viewport o'chirilganda sahifa telefonda kichraytirilgan, mayda matnli va gorizontal aylantirishli ko'rinadi (brauzer o'zini ~980px deb tutadi). Qaytarsangiz, mazmun ekranga moslashadi va matn o'qilarli bo'ladi. Demak, `width=device-width` responsive uchun shart.

4-mashq. Sahifaga favicon ulang. 32x32 px kichik PNG rasm tayyorlang (yoki onlayn generator ishlatib), uni fayl yoniga qo'ying va `<link rel="icon">` bilan ulang. Tabda belgi ko'rinishini tekshiring.



Yechim



```
<link rel="icon" type="image/png" href="favicon.png">
```



Rasm fayli (`favicon.png`) HTML fayl bilan bir papkada bo'lsin. Brauzerni yangilang (ba'zan Ctrl+Shift+R kerak bo'ladi).

5-mashq. Tashqi `style.css` faylni `<link>` bilan, `app.js` faylni esa `defer` bilan `<script>` orqali `<head>` ga ulang. Nega `defer` ishlatilganini izohlang.



Yechim



```
<link rel="stylesheet" href="style.css">
<script src="app.js" defer></script>
```



`defer` JS faylni HTML bilan parallel yuklaydi, lekin HTML to'liq o'qib bo'lingach ishga tushiradi — sahifa tez ochiladi va JS sahifadagi elementlarni topa oladi.

6-mashq. Sahifaga to'liq Open Graph teglarini qo'shing: `og:title`, `og:description`, `og:image` (to'liq `https://` manzil bilan), `og:url`, `og:type`. Keyin bitta `twitter:card` qatorini ham qo'shing.



Yechim



```
<meta property="og:title" content="Mening saytim">
<meta property="og:description" content="HTML o'rganish bo'yicha shaxsiy sahifam.">
<meta property="og:image" content="https://saytim.uz/rasmlar/cover.jpg">
<meta property="og:url" content="https://saytim.uz">
<meta property="og:type" content="website">
<meta name="twitter:card" content="summary_large_image">
```



Diqqat: OG'da `property=`, Twitter'da esa `name=` ishlatiladi.

7-mashq. Quyidagi `<head>` da **uchta xato** bor. Ularni toping va tuzating.

```
<head>
  <title>Sayt</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="initial-scale=1.0">
  <meta name="og:title" content="Mening saytim">
</head>
```

Yechim

1. `<meta charset="UTF-8">` `<title>` dan **oldin**, eng birinchi bo'lishi kerak.
2. `viewport` da eng muhim qism — `width=device-width` — yetishmaydi.
3. Open Graph tegi `name=` emas, `property=` bilan yoziladi.

To'g'ri varianti:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sayt</title>
  <meta property="og:title" content="Mening saytim">
</head>
```

8-mashq. Bir "rahmat sahifasi" (`rahmat.html`) tasavvur qiling — uning Google natijasida chiqishi shart emas. Bu sahifani indeksdan chiqaradigan meta tegni yozing.

Yechim

```
<meta name="robots" content="noindex">
```

Bu Google'ga aytadi: bu sahifani qidiruv natijalariga qo'shma. (Ehtiyot bo'ling — buni faqat kerakli sahifaga qo'yning, butun saytga emas.)

08 — CSS asoslari

← Oldingi: 07 — Head, meta va SEO · 🏠 README · Keyingi: 09 — Selektorlar →

Bu bobda: CSS nima ekanini, uning sintaksisi (selector, e'lon, property, value) hamda CSS ni HTMLga ulashning uch usulini (inline, internal, external) o'rganib, birinchi sahifangizni o'z qo'lingiz bilan bezaysiz.

8.1 CSS nima va nega kerak?

Shu paytgacha HTML bilan ishladingiz. HTML sahifangizning **skeleti** edi: sarlavhalar, paragraflar, ro'yxatlar, rasmlar, havolalar. Lekin faqat HTML bilan yozilgan sahifa juda **xunuk** ko'rinadi — qora matn, ko'k havolalar, oq fon. Hech qanday rang, masofa, shrift yoki tartib yo'q.

CSS (Cascading Style Sheets — "kaskadli uslublar jadvallari") aynan shu yerda kuchga kiradi. CSS — bu HTML elementlari **qanday ko'rinishini** belgilaydigan til: rang, o'lcham, shrift, masofa, joylashuv, fon va boshqalar.

Buni shunday tasavvur qiling:

- **HTML** — uyning **g'isht va devorlari** (struktura: nima qayerda).
- **CSS** — uyning **bo'yog'i, mebeli, dizayni** (ko'rinish: qanday ko'rinadi).
- **JavaScript** (keyingi boblarda) — uyning **elektr va mexanikasi** (xatti-harakat: nima ish qiladi).

Nega strukturani ko'rinishdan ajratamiz?

Bu eng muhim "nega" savoli. Texnik atama bilan bu **separation of concerns** (vazifalarni ajratish) deyiladi. Ajratishning sabablari:

- **Tartib.** HTML faqat mazmun bilan, CSS faqat dizayn bilan shug'ullanadi. Kodni o'qish va tushunish osonlashadi.
- **Qayta ishlatish.** Bitta CSS faylni 100 ta HTML sahifaga ulash mumkin. Dizaynni bir joyda o'zgartirsangiz — 100 sahifada bir vaqtda o'zgaradi.
- **Hamkorlik.** Bir dasturchi HTML, ikkinchisi CSS ustida ishlashi mumkin, bir-biriga xalaqit bermay.
- **Tezlik.** Brauzer CSS faylni bir marta yuklab, keshlab (xotirada saqlab) qoladi. Keyingi sahifalar tezroq ochiladi.

💡 **Analogiya:** Word hujjatini tasavvur qiling. Matn — bu HTML. "Sarlavha 1" uslubi (font, rang, o'lcham) — bu CSS. Agar uslubni o'zgartirsangiz, shu uslubdagi BARCHA sarlavhalar bir vaqtda o'zgaradi. Aynan shu printsipl CSS da ishlaydi.

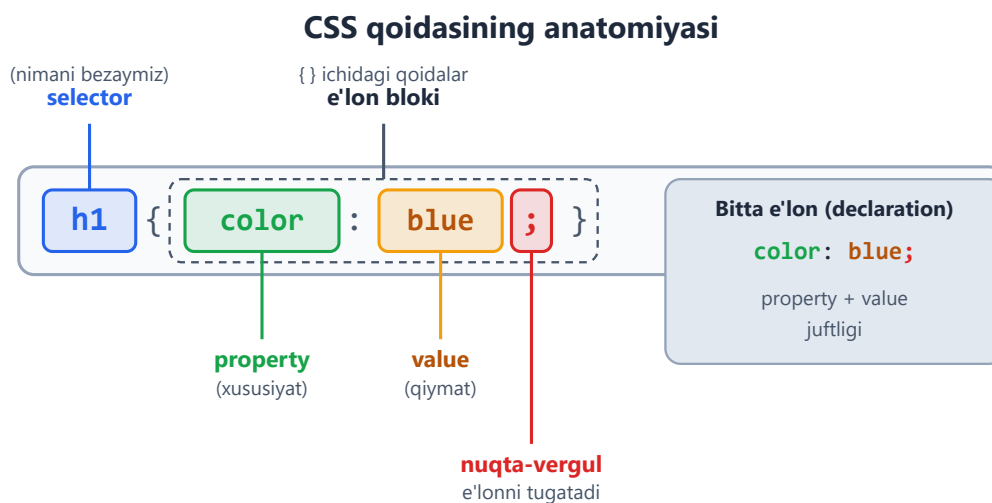
8.2 CSS sintaksisi: qoidaning anatomiyasi

CSS ni o'rganish uchun avval uning **sintaksisini** (yozilish qoidasini) tushunish kerak. Yaxshi xabar: CSS sintaksisi juda **oddiy va izchil**. Bir marta tushunsangiz, butun CSS shu naqshga asoslanadi.

CSS ning eng kichik birligi — **qoida (rule)**. Mana bitta qoida:

```
h1 {  
  color: blue;  
}
```

Bu qoida shuni anglatadi: "Barcha `<h1>` elementlarining matn rangini ko'k (blue) qil". Keling, bu qoidani qismlarga ajratamiz:



Har bir qismni alohida ko'rib chiqaylik:

Qism	Inglizcha	Bu nima	Misol
Selector	selector	Qaysi element(lar)ni bezashni belgilaydi	<code>h1</code>
E'lon bloki	declaration block	{ } qavslar ichidagi barcha qoidalar	<code>{ color: blue; }</code>

Qism	Inglizcha	Bu nima	Misol
Property	property	Qaysi xususiyatni o'zgartirish (rang, o'lcham...)	<code>color</code>
Value	value	Xususiyatga beriladigan qiymat	<code>blue</code>
E'lon	declaration	Bitta <code>property: value;</code> juftligi	<code>color: blue;</code>

Sintaksis qoidalari (eslab qoling)

1. **Select**ordan keyin **ochuvchi qavs** `{` keladi.
2. Har bir **e'lon** ichida property va value **ikki nuqta** `:` bilan ajratiladi.
3. Har bir e'lon **nuqta-vergul** `;` bilan tugaydi.
4. Hammasi **yopuvchi qavs** `}` bilan yakunlanadi.

Bitta selectorga bir nechta e'lon berish mumkin — har birini alohida qatorga yozish odat:

```
h1 {
  color: blue;
  font-size: 32px;
  text-align: center;
}
```

Bu qoida `<h1>` larni: ko'k rangga bo'yaydi, 32 piksel kattalikda qiladi va markazga tekislaydi.

⚠ **Eng ko'p uchraydigan xato — nuqta-vergulni unutish.** `color: blue` dan keyin `;` qo'ymasangiz, brauzer keyingi e'lonni shu bilan birga o'qishga urinadi va ikkalasi ham ishlamay qoladi. Har bir e'londan keyin `;` qo'yishni odat qiling — hatto blokda oxirgi e'londan keyin ham (keyinroq qo'shganda xato bo'lmaydi).

💡 **"Nega bunday sintaksis?"** Bu format mashinaga ham, odamga ham qulay. Brauzer `{`, `}`, `:`, `;` belgilariga qarab kodni aniq bo'laklarga ajrata oladi; siz esa har qaysi qoida qayerda boshlanib, qayerda tugashini bir qarashda ko'rasiz.

8.3 CSS izohlari (comments)

HTML dagi `<!-- -->` kabi, CSS da ham **izoh** (comment) yozish mumkin. Izoh — bu brauzer **e'tiborsiz qoldiradigan**, faqat odam o'qishi uchun yozilgan matn.

CSS da izoh `/*` bilan boshlanib, `*/` bilan tugaydi:

```
/* Bu izoh — brauzer buni o'qimaydi */

h1 {
  color: blue; /* sarlavha rangi */
  /* font-size: 50px; <- bu qator vaqtincha o'chirilgan */
}
```

Izohlar nega kerak?

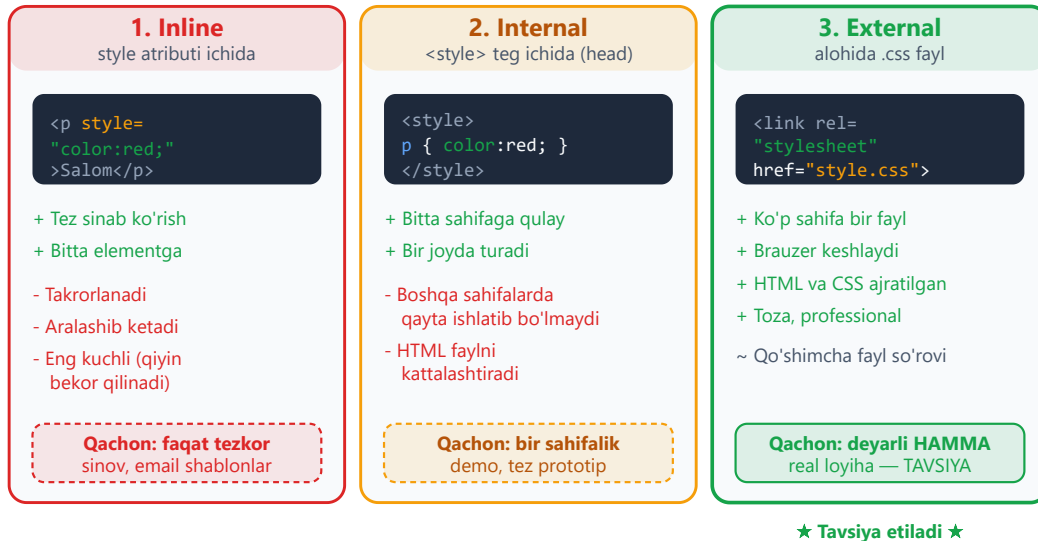
- **O'zingizga eslatma.** "Bu rang nega tanlandi?", "bu blok nimaga javob beradi?".
- **Vaqtincha o'chirish.** Bir e'lonni o'chirib tashlamasdan, izohga aylantirib (`/* ... */`) sinab ko'rish mumkin.
- **Bo'limlarga ajratish.** Katta CSS faylda izoh bilan bo'limlarga (masalan "Header uslublari", "Footer uslublari") sarlavha qo'yish.

✦ HTML izohi `<!-- -->`, CSS izohi `/* */`. Ularni adashtirmang — CSS faylda HTML izohi ishlamaydi va aksincha.

8.4 CSS ni HTMLga ulashning uch usuli

CSS yozdingiz — endi uni HTML bilan **bog'lash** kerak, aks holda brauzer uni ko'rmaydi. Buning **uch usuli** bor. Har birining o'rni bor, lekin amalda bittasi eng ko'p ishlatiladi.

CSS ni HTMLga ulashning uch usuli



1-usul: Inline (style atributi)

CSS to'g'ridan-to'g'ri elementning `style` atributi ichiga yoziladi. Bu yerda selector va qavslar **kerak emas** — chunki uslub allaqachon shu elementga tegishli:

```
<p style="color: red; font-size: 18px;">Bu paragraf qizil va kattaroq.</p>
```

- ✓ **Afzalligi:** tez sinab ko'rish; faqat bitta aniq elementni o'zgartirish.
- ✗ **Kamchiligi:** har bir elementga qaytadan yozish kerak (takrorlanish); HTML va dizayn aralashib ketadi; o'qish qiyinlashadi; keyinchalik o'zgartirish azob.

⚠ Inline uslubdan **iloji boricha qochasiz**. U "eng kuchli" hisoblanadi (8.6 ga qarang), shuning uchun uni boshqa qoida bilan bekor qilish qiyin. Asosan tezkor sinov yoki email shablonlarida ishlatiladi.

2-usul: Internal (sahifa ichidagi `<style>`)

CSS HTML faylning `<head>` qismidagi `<style>` tegi ichiga yoziladi. Bu yerda **to'liq sintaksis** (selector, qavslar) ishlatiladi:

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>Internal CSS</title>
  <style>
    p {
```

```

        color: red;
        font-size: 18px;
    }
</style>
</head>
<body>
    <p>Bu paragraf qizil – internal CSS orqali.</p>
    <p>Bu ham qizil, chunki selector "p" barcha paragraflarga tegishli.
</p>
</body>
</html>

```

-  **Afzalligi:** bitta sahifa uchun barcha uslublar bir joyda; takrorlash yo'q (selector bir marta yoziladi, hamma `<p>` ga ta'sir qiladi).
-  **Kamchiligi:** uslublar faqat **shu bitta** HTML faylga tegishli – boshqa sahifalarda qayta ishlatib bo'lmaydi; HTML fayl kattalashadi.

 `<style>` tegi har doim `<head>` ichida turishi kerak, `<body>` da emas.

3-usul: External (alohida `.css` fayl) – TAVSIYA ETILADI

CSS butunlay alohida faylga (masalan `style.css`) yoziladi, HTML esa unga `<link>` tegi orqali ulanadi. Bu eng professional va eng ko'p ishlatiladigan usul.

Avval `style.css` faylini yarating (faqat CSS, HTML yo'q):

```

/* style.css fayli */
p {
    color: red;
    font-size: 18px;
}

```

Keyin `index.html` faylida `<head>` ichida unga ulaning:

```

<!DOCTYPE html>
<html lang="uz">
<head>
    <meta charset="UTF-8">
    <title>External CSS</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <p>Bu paragraf alohida CSS fayldan bezatildi.</p>
</body>
</html>

```

`<link>` tegidagi atributlarni tushunaylik:

- `rel="stylesheet"` — "bu bog'langan fayl turi: uslublar jadvali" (relationship = aloqa turi).
- `href="style.css"` — CSS faylning manzili (HTML fayl bilan bir papkada bo'lsa, shunchaki nomi yetadi).

Afzalliklari (nega aynan shu usul afzal):

-  **Bir fayl — ko'p sahifa.** Bitta `style.css` ni o'nlab HTML sahifaga ulash mumkin. Saytning ko'rinishini bir joydan boshqaring.
-  **Kesh.** Brauzer CSS faylni bir marta yuklab, xotirada saqlaydi. Foydalanuvchi keyingi sahifaga o'tganda CSS qaytadan yuklanmaydi — sayt tezroq ishlaydi.
-  **Toza ajratish.** HTML faqat struktura, CSS faqat dizayn. Bu 8.1 da gaplashgan "separation of concerns".
-  Yagona kichik "narx": brauzer qo'shimcha bitta fayl so'rovi yuboradi — lekin kesh tufayli bu bir martalik va arzimmas.

 **Qaysi usulni qachon ishlataman? - External** — deyarli har doim, har qanday real loyihada. Standart tanlov. - **Internal** — bir sahifalik tez demo yoki prototip uchun, alohida fayl ortiqcha bo'lsa. - **Inline** — faqat tezkor sinov yoki email kabi maxsus holatlarda. Doimiy kodda qochish kerak.

8.5 Brauzer default uslublari (user-agent stylesheet)

Hech qanday CSS yozmasangiz ham, sahifangiz **butunlay uslubsiz** emas. Sinab ko'ring: bo'sh HTML faylga `<h1>Salom</h1>` va `<p>Matn</p>` yozing — sarlavha **katta va qalin**, paragraf esa kichikroq chiqadi. Ro'yxatlarda esa nuqtalar (•) paydo bo'ladi.

Bu uslublar qayerdan keladi? Ularni **brauzerning o'zi** beradi. Har bir brauzer (Chrome, Firefox, Safari) ichida **default CSS** mavjud — bu **user-agent stylesheet** ("foydalanuvchi-agenti uslublar jadvali") deb ataladi. "User agent" — bu brauzerning texnik nomi.

User-agent uslublari nima qiladi:

- `<h1>` ni katta va qalin qiladi, `<h2>` ni undan kichikroq va hokazo.
- Paragraflar orasiga avtomatik bo'sh joy (margin) qo'yadi.
- Havolalarni ko'k va tagiga chizilgan qiladi.
- `<ul>` ro'yxatlariga nuqta, `<ol>` ga raqam qo'shadi.

 **Nega buni bilish muhim?** Chunki siz ba'zan "men hech narsa yozmadim-ku, bu masofa qayerdan keldi?" deb hayron bo'lasiz. Javob: ko'pincha user-agent

uslublaridan. Sizning CSS qoidalaringiz aslida brauzer default'ining **ustiga** yoziladi yoki uni bekor qiladi.

💡 Ko'p dasturchilar loyiha boshida brauzerlar orasidagi farqni tenglashtirish uchun "CSS reset" yoki "normalize.css" deb nomlangan kichik CSS qo'shadi. Buni hozircha bilib qo'ying — keyingi boblarda batafsil ko'ramiz.

8.6 "Cascade" so'zining ma'nosi

CSS ning to'liq nomi — **Cascading Style Sheets**. Bu yerdagi **"cascade"** (kaskad, sharshara) so'zi tasodifiy emas — u CSS ning eng muhim tushunchalaridan birini bildiradi.

Tasavvur qiling: bitta `<p>` elementiga **bir nechta joydan** rang berildi:

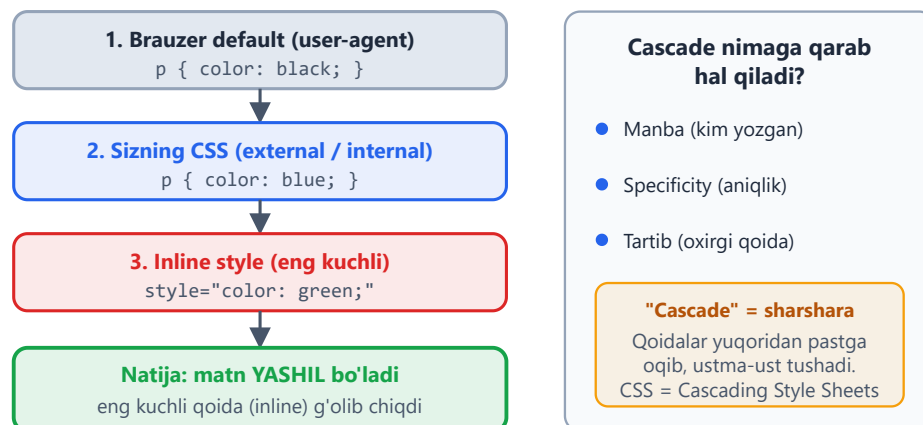
- Brauzer default'i — qora.
- Sizning external CSS — ko'k.
- Inline style — yashil.

Endi savol: paragraf **qaysi rangda** chiqadi? Bir vaqtda uchta rang bo'lishi mumkin emas-ku!

Aynan shu yerda **cascade** ishga tushadi. Cascade — bu CSS ning bir element uchun **ziddiyatli qoidalar**dan qaysi birini tanlash mexanizmi. So'z "sharshara" ma'nosini beradi: qoidalar yuqoridan pastga oqib, ustma-ust tushadi va eng **kuchli** yoki **oxirgi** qoida g'olib chiqadi.

Cascade — uslublar "sharsharadek" oqadi

Bir xil elementga bir xil property kelsa, OXIRGI / KUCHLIROQ qoida g'olib bo'ladi



Yuqoridagi misolda paragraf **yashil** bo'ladi, chunki inline style eng kuchli. Agar inline bo'lmasa — **ko'k** (sizning CSS) g'olib bo'lardi, brauzer default'i (qora) emas.

Cascade qaror qabul qilishda taxminan shu uch narsaga qaraydi:

1. **Manba va muhimlik** — qoidani kim yozgan (brauzer default'imi yoki sizmi).
2. **Specificity (aniqlik)** — qoida qanchalik "aniq" element ko'rsatgan (bu haqda alohida bobda batafsil).
3. **Tartib** — bir xil kuchdagi ikki qoidadan **oxirgisi** g'olib bo'ladi.

✦ Hozircha ikkita oddiy qoidani eslab qoling: - Inline > internal/external > brauzer default (kuchi bo'yicha). - Bir xil kuchdagi ikki qoida bo'lsa — **keyingisi** (faylda pastroq turgan) g'olib chiqadi.

💡 Cascade — CSS ning "yuragi". Hozir uni chuqur tushunish shart emas; faqat **g'oyani** — "qoidalar to'qnashganda CSS qaysidir tartibda g'olibni tanlaydi" degan tushunchani — qabul qilib qo'ying. Keyingi boblarda specificity bilan birga batafsil qaytamiz.

8.7 Amaliy misol: birinchi sahifangizni bezash

Nazariyani amalda sinab ko'raylik. Quyida **external CSS** usuli bilan bezatilgan to'liq, ishlaydigan sahifa keltirilgan. Ikki fayl yarating.

1-fayl: `index.html`

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Mening birinchi bezatilgan sahifam</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Salom, dunyo!</h1>
  <p>Bu mening CSS bilan bezatgan birinchi sahifam.</p>
  <p class="muhim">Bu paragraf alohida ajratilgan.</p>
  <ul>
    <li>Birinchi punkt</li>
    <li>Ikkinchi punkt</li>
    <li>Uchinchi punkt</li>
  </ul>
</body>
</html>
```

2-fayl: style.css (xuddi shu papkada)

```
/* ===== Umumiy sahifa uslublari ===== */
body {
  font-family: "Segoe UI", system-ui, sans-serif;
  background-color: #f8faff; /* och kulrang fon */
  color: #1e293b;           /* matn rangi (deyarli qora) */
  margin: 40px;
}

/* ===== Sarlavha ===== */
h1 {
  color: #2563eb; /* ko'k */
  text-align: center;
}

/* ===== Paragraflar ===== */
p {
  font-size: 18px;
  line-height: 1.6; /* qatorlar orasi kengligi (o'qish uchun qulay) */
}

/* "muhim" class li paragraf – alohida fon va chetdan masofa */
.muhim {
  background-color: #fef3c7; /* och sariq */
  padding: 12px;
  border-radius: 8px;
}

/* ===== Ro'yxat ===== */
li {
  color: #16a34a; /* yashil */
}
```

Natija: Sahifa och kulrang fonli, markazlashgan ko'k sarlavhali bo'ladi. Paragraflar kattaroq va o'qishga qulay; "muhim" klassli paragraf och sariq fon va yumaloq burchaklarga ega bo'ladi; ro'yxat punktlari yashil rangda chiqadi.

E'tibor bering: HTML da `class="muhim"` deb yozdik, CSS da esa unga `.muhim` (nuqta bilan) selector orqali murojaat qildik. `class` va selectorlar (`.class`, `#id`, element nomi) — keyingi bobning asosiy mavzusi. Hozircha asosiy g'oya: **HTML ga class qo'yib, CSS dan unga uslub berish mumkin.**

💡 **Sinab ko'ring:** `style.css` dagi `h1` rangini boshqa rangga (masalan `#dc2626` — qizil) o'zgartiring va faylni saqlab, brauzerni yangilang. Faqat CSS faylni o'zgartirib, butun sahifa ko'rinishini boshqarayotganingizni his qiling — bu external CSS ning kuchi.

⚠ Ikki fayl **bir papkada** bo'lishi va `<link>` dagi `href="style.css"` nomi fayl nomiga **aynan mos** kelishi shart (katta-kichik harf ham muhim). Aks holda CSS ulanmaydi va sahifa bezaksiz ko'rinadi.

Mashqlar

Quyidagi mashqlarni bajarib, CSS asoslarini mustahkamlang. Har biri uchun yangi `.html` (va kerak bo'lsa `.css`) fayl yarating, brauzerda oching va natijani ko'ring.

Mashq 1 — Qoida qismlarini nomlang

Quyidagi CSS qoidasida selector, property, value va e'lonni aniqlang:

```
h2 {  
  color: green;  
  font-size: 24px;  
}
```

Yechim

- **Selector:** `h2`
- **E'lon bloki:** `{ color: green; font-size: 24px; }`
- Bu blokda **ikkita e'lon** bor:
 - 1-e'lon: `color: green;` → property = `color`, value = `green`
 - 2-e'lon: `font-size: 24px;` → property = `font-size`, value = `24px`

Mashq 2 — Uchta usulni sinab ko'ring

Bitta paragrafni **uchala usul** bilan qizil rangga bo'yang: bittada inline, bittada internal, bittada external. Har birini alohida HTML faylda yarating va natija bir xilligiga ishonch hosil qiling.

 **Yechim** 

Inline (inline.html):

```
<p style="color: red;">Inline orqali qizil.</p>
```

Internal (internal.html):

```
<head>
  <style>
    p { color: red; }
  </style>
</head>
<body>
  <p>Internal orqali qizil.</p>
</body>
```

External (external.html + style.css):

```
<!-- external.html -->
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p>External orqali qizil.</p>
</body>
```

```
/* style.css */
p { color: red; }
```

Mashq 3 — Nuqta-vergul xatosini toping

Quyidagi CSS da xato bor. Uni toping va tuzating:

```
h1 {
  color: blue
  font-size: 30px;
}
```

 **Yechim** 

`color: blue` dan keyin **nuqta-vergul** `;` yo'q. Shu sababli brauzer `blue font-size: 30px` ni bitta noto'g'ri qiymat deb o'qiydi va **ikkala** e'lon ham ishlamaydi. To'g'risi:

```
h1 {
  color: blue;
  font-size: 30px;
}
```



Mashq 4 — Izoh bilan vaqtincha o'chirish

Quyidagi CSS dagi `font-size` e'lonini **o'chirib tashlamasdan**, izoh yordamida vaqtincha ishlamas qilib qo'ying:

```
p {
  color: navy;
  font-size: 20px;
}
```



 **Yechim** 

```
p {
  color: navy;
  /* font-size: 20px; */
}
```

Endi `font-size` e'loni izoh ichida — brauzer uni e'tiborsiz qoldiradi, lekin kod o'chmagan. `/* */` ni olib tashlasangiz, qaytadan ishlaydi.

Mashq 5 — External CSS bilan sahifa bezang

`index.html` va `style.css` yarating. Sahifada bitta `<h1>`, bitta `<p>` va uch elementli `<ul>` bo'lsin. External CSS orqali: sarlavhani markazga tekislang va ko'k qiling; paragraf o'lchamini 18px qiling; ro'yxat punktlarini yashil rangga bo'yang.

 **Yechim** 

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>Mashq 5</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Mening sahifam</h1>
  <p>Bu external CSS bilan bezatilgan paragraf.</p>
  <ul>
    <li>Olma</li>
    <li>Banan</li>
    <li>Uzum</li>
  </ul>
</body>
</html>
```

```
/* style.css */
h1 {
  color: blue;
  text-align: center;
}
p {
  font-size: 18px;
}
li {
  color: green;
}
```

Mashq 6 — Cascade: qaysi rang g'olib?

Quyidagi kodda paragraf **qaysi rangda** chiqadi? Nega?

```
<head>
  <style>
    p { color: blue; }
  </style>
</head>
<body>
  <p style="color: green;">Qaysi rang?</p>
</body>
```

 **Yechim** 

Paragraf **yashil** (green) chiqadi. Sababi cascade: **inline style** (`style="color: green;"`) internal CSS dan (`p { color: blue; }`) **kuchliroq**, shuning uchun u g'olib bo'ladi.

Mashq 7 — Tartib qoidasi

Quyidagi CSS da paragraf qaysi rangda chiqadi? Nega?

```
p { color: red; }  
p { color: green; }
```



Yechim



Paragraf **yashil** (green) chiqadi. Ikkala qoidaning **kuchi (specificity) bir xil** — ikkalasi ham oddiy `p` selectori. Bunday holatda cascade'ning **tartib** qoidasi ishlaydi: faylda **oxirgi** kelgan qoida g'olib bo'ladi. `color: green;` keyin kelgani uchun u qizilni bekor qiladi.

Mashq 8 — Mini "biznes kartochka" sahifasi

External CSS yordamida o'zingiz haqingizdagi kichik "biznes kartochka" sahifasini yarating. Talablar: ismingiz `<h1>` da va markazlashgan; bitta kasb/qiziqish paragrafi; uchta sevimli narsangiz `<ul>` ro'yxatida; sahifaga och fon rang; matn rangi to'q kulrang. Iloji bo'lsa, paragrafga `class` berib, unga fon rang va `padding` qo'shing.



Yechim (namuna)



```
<!-- index.html -->
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>Biznes kartochka</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Ali Valiyev</h1>
  <p class="kasb">Boshlovchi veb-dasturchi va HTML/CSS ixlosmandi.</p>
  <ul>
    <li>Kodlash</li>
    <li>Kitob o'qish</li>
    <li>Sayohat</li>
  </ul>
</body>
</html>
```

```
/* style.css */
body {
  font-family: "Segoe UI", system-ui, sans-serif;
  background-color: #f8fafc;
  color: #1e293b;
  margin: 40px;
}
h1 {
  text-align: center;
  color: #2563eb;
}
.kasb {
  background-color: #e2e8f0;
  padding: 12px;
  border-radius: 8px;
  text-align: center;
}
li {
  font-size: 18px;
}
```

Bu yechim shunchaki namuna — ranglar, matn va ro'yxatni o'zingizga moslab o'zgartiring. Asosiysi: external CSS, markazlashgan sarlavha, `class` bilan bezatilgan paragraf.

09 — Selektorlar

← Oldingi: 08 — CSS asoslari · 🏠 README · Keyingi: 10 — Cascade, specificity va inheritance →

Bu bobda: elementlarni aniq tanlash: tur, klass, id, kombinatorlar va psevdolar.

CSS yozishning yarmi — qoidalarni yozish (`color: red`), ikkinchi yarmi esa **qaysi elementga** shu qoidani qo'llashni aytish. Mana shu "qaysi elementga" degan savolga javob beradigan vosita **selektor** (selector — tanlovchi) deb ataladi.

Tasavvur qil: sen sinfdagi o'qituvchisan. "Birinchi qatordagilar turing", "ko'ylagi qizil bo'lganlar qo'l ko'taring", "Aliyevning yonidagi o'tirgan o'quvchi" — bularning har biri bir guruh odamni **aniqlab beradi**. Selektor ham xuddi shunday: sahifadagi minglab elementdan kerakligini ajratib oladi. Bu bob seni eng sodda selektordan (`p`) to expert darajadagi murakkab tanlovlargacha (`li:not(:last-child)::after`) bosqichma-bosqich olib boradi.

💡 Selektorni yaxshi bilish — CSS'da eng kuchli ko'nikma. Ko'p odam CSS "ishlamayapti" deb shikoyat qiladi, aslida muammo qoidada emas, **selektor noto'g'ri elementni** tanlayotganida bo'ladi.

9.1 Selektor nima va u qanday ishlaydi

Har bir CSS qoidasi ikki qismdan iborat: **selektor** va **deklaratsiya bloki**.

```
p {  
    color: blue; /* { ... } - deklaratsiya bloki */  
}
```

Bu qoida shunday o'qiladi: "Sahifadagi **har bir** `<p>` elementini top va matn rangini ko'k qil."

Brauzer HTML'ni o'qiyotganda sahifani **daraxt** (DOM — Document Object Model) ko'rinishida tuzadi: `<html>` ildiz, ichida `<body>`, uning ichida boshqa elementlar va hokazo. Selektor — shu daraxtdan kerakli "shoxlar"ni terib oladigan filtr.

✦ Bitta element bir nechta selektorga mos kelishi mumkin (masalan, `<p class="muhim">` ham `p`, ham `.muhim`, ham `*` selektoriga tushadi). Qaysi qoida g'olib

chiqishi **spesifiklik** (specificity) qoidalariga bog'liq — buni keyingi bobda ko'ramiz. Hozircha selektorlarning o'zini puxta o'rganamiz.

9.2 Asosiy selektorlar: tur, klass, id

Bu uchta — CSS'ning kundalik noni. 95% holatda shulardan foydalanasan.

Tur selektori (element/type selector)

Element nomini shundoq yozasan — hech qanday belgi qo'ymasdan.

```
p      { line-height: 1.6; } /* barcha <p> */
h1     { font-size: 32px; } /* barcha <h1> */
button { cursor: pointer; } /* barcha <button> */
```

Nega kerak? Sahifadagi bir TURDAGI barcha elementga umumiy ko'rinish berish uchun. Masalan, barcha paragraflarning satrlar orasini kengaytirish.

⚠ Tur selektori **juda keng**. `p { color: red; }` desang, sahifadagi tagidagi eng kichik paragrafga ham ta'sir qiladi. Aniqroq nishon kerak bo'lsa — klassdan foydalan.

Klass selektori (class selector)

HTML'da elementga `class="..."` atributini berasan, CSS'da esa **nuqta** `.` bilan murojaat qilasan.

```
<p class="ogohlantirish">Diqqat!</p>
<span class="ogohlantirish">Bu ham diqqat.</span>
```

```
.ogohlantirish {
  color: #dc2626;
  font-weight: bold;
}
```

Nega klass eng ko'p ishlatiladi? - Bitta klassni **istalgancha ko'p** elementga berish mumkin (yuqorida `<p>` ham, `<span>` ham). - Bitta elementga **bir nechta** klass berish mumkin: `<div class="karta katta faol">`. - Element turidan mustaqil — bugun `<p>`, ertaga `<div>` bo'lsa ham klass o'sha-o'sha ishlaydi.

💡 Amaliyotda CSS'ning aksariyati klasslar ustiga quriladi. Bu element turiga bog'lanib qolmaslik va qayta foydalanish imkonini beradi.

Id selektori (id selector)

`id="..."` atributiga **panjara** `#` bilan murojaat qilasan.

```
<header id="bosh-qism">...</header>
```

```
#bosh-qism {  
  background: #f8fafc;  
}
```

Muhim qoida: bitta `id` sahifada **faqat bir marta** ishlatilishi kerak — u noyob identifikator. `class` esa qayta-qayta ishlatiladi.

⚠ Id selektorni stil berish uchun ishlatish **tavsiya etilmaydi**. Sababi: id juda "kuchli" (yuqori spesifiklik), keyinroq uni klass bilan qayta yozish qiyin bo'ladi. Id'lar ko'proq JavaScript va sahifa ichi havolalari (`<a href="#bosh-qism">`) uchun ishlatiladi. Stil uchun — klass.

Qiyoslash jadvali

Selektor	Yozilishi	Necha marta	Asosiy maqsad
Tur	<code>p</code>	cheksiz	bir turdagi barcha element
Klass	<code>.box</code>	cheksiz	qayta ishlatiluvchi stil
Id	<code>#main</code>	bir marta	noyob element, JS, havola

9.3 Universal va guruhlash selektorlari

Universal selektor `*`

Yulduzcha **hamma narsani** tanlaydi — istisnosiz.

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```


Nega kerak? Eng ko'p ishlatiladigan joyi — yuqoridagi kabi "reset": brauzer har bir elementga beradigan standart bo'shliqlarni nolga tushirish va `box-sizing` ni hamma joyda bir xil qilish.

⚠️ `*` ni keng qo'llashdan ehtiyot bo'l. U **har bir** elementga tegadi, shuning uchun unga og'ir qoidalar (masalan `transition: all`) yozsang, sahifa sekinlashishi mumkin. Reset uchun yaxshi, qolgan hollarda aniqroq selektor afzal.

Guruhlash (vergul bilan)

Bir nechta selektorga **bir xil** qoida berish kerak bo'lsa, ularni **vergul** bilan sanaysan.

```
h1, h2, h3 {  
  font-family: "Segoe UI", sans-serif;  
  color: #1e293b;  
}
```

Bu — quyidagining qisqa shakli:

```
h1 { font-family: "Segoe UI", sans-serif; color: #1e293b; }  
h2 { font-family: "Segoe UI", sans-serif; color: #1e293b; }  
h3 { font-family: "Segoe UI", sans-serif; color: #1e293b; }
```

Nega vergul muhim? Vergulni unutsang, ma'no butunlay o'zgaradi:

```
h1 h2 { ... } /* h1 ICHIDAGI h2 (kombinator!) — odatda mavjud emas */  
h1, h2 { ... } /* h1 VA h2 (guruhlash) */
```

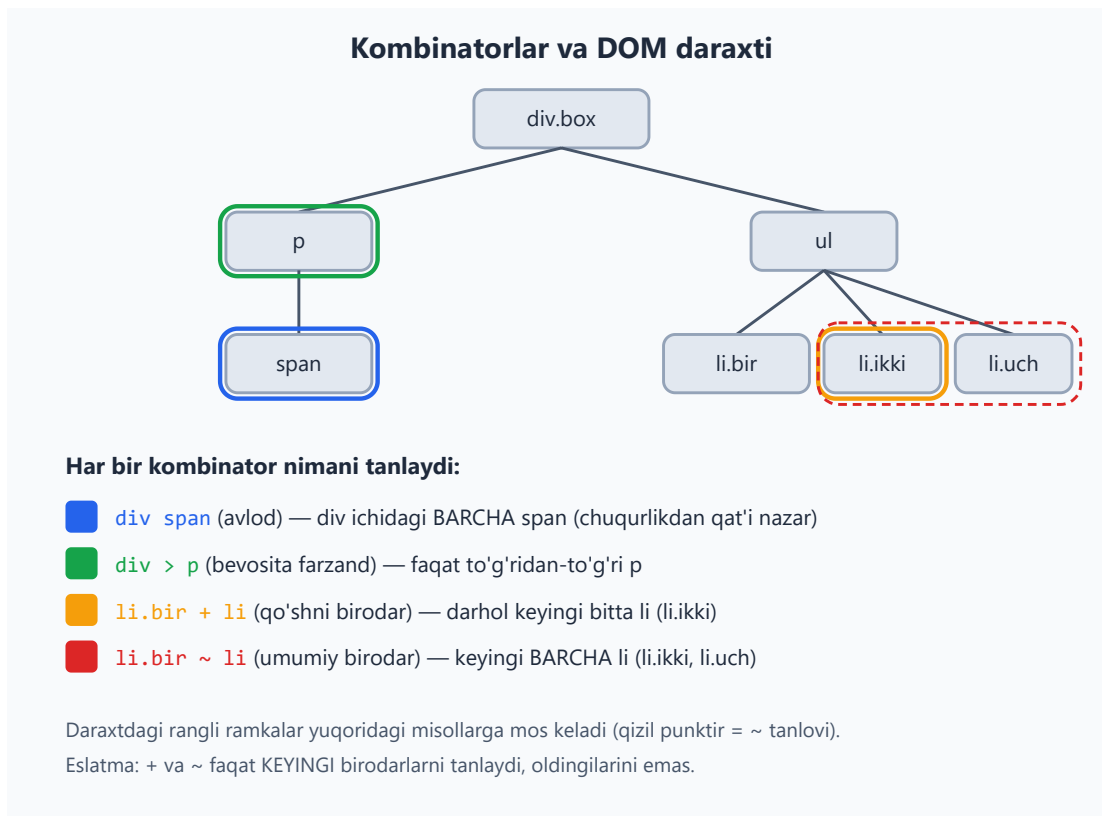
⚠️ Bu juda tez-tez uchraydigan xato: vergulni tushirib qoldirsang, brauzer xato bermaydi — shunchaki butunlay boshqa narsani tanlaydi va sen nima uchun "stil ishlamayapti" deb hayron bo'lasan.

💡 Bitta umumiy maslahat: agar vergulli ro'yxatdagi **bironta** selektor noto'g'ri (brauzer tushunmaydigan) bo'lsa, eski brauzerlarda butun qoida e'tiborsiz qoldirilishi mumkin. Yangi xususiyatlarni alohida yozish xavfsizroq.

9.4 Kombinatorlar: elementlarni munosabati bo'yicha tanlash

Hozirgacha alohida elementlarni tanladik. **Kombinatorlar** esa elementlarni **bir-biriga nisbatan joylashuvi** bo'yicha tanlash imkonini beradi: "shu element ICHIDAGI", "to'g'ridan-to'g'ri farzandi", "darhol keyingisi" va hokazo.

Quyidagi diagramma to'rt kombinatorni bitta DOM daraxtida ko'rsatadi — har biri qaysi elementni tanlashini rang bilan ajratdik.



Endi har birini alohida ko'ramiz. Misollar uchun shu HTML'dan foydalanamiz:

```
<div class="box">
  <p>Birinchii <span>matn</span></p>
  <ul>
    <li class="bir">Bir</li>
    <li class="ikki">Ikki</li>
    <li class="uch">Uch</li>
  </ul>
</div>
```

Avlod kombinatori — bo'shliq a b

Ikki selektor orasiga **bo'shliq** qo'ysang, bu "birinchisi ICHIDAGI ikkinchisi" degani — qanchalik chuqurda bo'lishidan qat'i nazar.

```
.box span { color: blue; } /* .box ichidagi BARCHA span */
```

`<span>` `<p>` ning ichida, `<p>` esa `.box` ning ichida. Span to'g'ridan-to'g'ri `.box` ning farzandi bo'lmasa ham — baribir tanlanadi, chunki u `.box` ning **avlodi** (nabira, evara — farqi yo'q).

Bevosita farzand kombinatori — >

> belgisi faqat **to'g'ridan-to'g'ri farzand**ni tanlaydi (bir pog'ona pastda).

```
.box > p { font-weight: bold; } /* faqat .box ning bevosita farzandi  
p */
```

<p> to'g'ridan-to'g'ri .box ichida — tanlanadi. Ammo .box > span **hech narsani tanlamaydi**, chunki .box ning bevosita farzandi emas (u <p> ichida).

Nega ikkalasi kerak? Bo'shliq — "chuqurligi muhim emas, ichida bo'lsa bas". > — "aniq bitta pog'ona pastda". Murakkab tuzilmalarda > xatolardan saqlaydi: masalan, faqat yuqori darajadagi menyu bandlarini tanlab, ichki ostmenyularga tegmaslik uchun.

Qo'shni birodar kombinatori — +

+ belgisi **darhol keyingi bitta birodar**ni tanlaydi (bir ota-ona ostida, ketma-ket turgan).

```
li.bir + li { color: green; } /* li.bir dan KEYINGI bitta li */
```

Bu li.ikki ni tanlaydi (chunki u li.bir dan darhol keyin keladi). li.uch tanlanmaydi — u keyingi emas, undan keyingisi.

Umumiy birodar kombinatori — ~

~ belgisi **keyingi BARCHA birodar**larni tanlaydi (faqat darhol keyingisini emas).

```
li.bir ~ li { color: red; } /* li.bir dan keyingi BARCHA li */
```

Bu li.ikki **va** li.uch ni tanlaydi.

⚠ Ham +, ham ~ **faqat keyinga** qaraydi. Oldinga (yuqoriga) qaragan kombinator yo'q: li.uch + li yoki li.uch ~ li hech narsani tanlamaydi, chunki li.uch dan keyin birodar yo'q. CSS daraxtni yuqoridan pastga o'qiydi.

Qisqa jadval

Kombinator	O'qilishi	Misol	Tanlaydi
(bo'shliq)	ichidagi (avlod)	div p	div ichidagi barcha p
>	bevosita farzand	div > p	faqat to'g'ridan-to'g'ri p

Kombinator	O'qilishi	Misol	Tanlaydi
+	qo'shni birodar	h2 + p	h2 dan keyingi bitta p
~	umumiy birodar	h2 ~ p	h2 dan keyingi barcha p

9.5 Atribut selektorlari

Elementlarni ularning **atributi** yoki uning **qiymati** bo'yicha tanlash mumkin. Bu kvadrat qavslar [] orqali yoziladi.

Atribut mavjudligi bo'yicha

```
[disabled]      { opacity: 0.5; } /* disabled atributi BOR har qanday
element */
[required]      { border-color: orange; }
input[type]     { ... }          /* type atributi bor input */
```

Aniq qiymat bo'yicha [atr=qiymat]

```
input[type="text"]      { border: 1px solid #94a3b8; }
input[type="password"] { letter-spacing: 2px; }
a[target="_blank"]     { color: #2563eb; }
```

Bu — formalar bilan ishlashda juda qadrlı, chunki barcha `<input>` bir xil teg, lekin `type` ularni farqlaydi.

Qism bo'yicha moslash (kuchli operatorlar)

Mana bu yerda atribut selektorlari haqiqatan kuchli bo'ladi. Qiymatning **bir qismini** ham qidirish mumkin:

Operator	Ma'nosi	Misol	Mos keladi
^=	bilan boshlanadi	[href^="https"]	https://...
\$=	bilan tugaydi	[href\$=".pdf"]	hujjat.pdf

Operator	Ma'nosi	Misol	Mos keladi
<code>*=</code>	ichida bor	<code>[class*="btn"]</code>	<code>btn-katta</code> , <code>mening-btn</code>
<code>~=</code>	bo'sh joy bilan ajralgan so'z	<code>[class~="faol"]</code>	<code>karta faol katta</code>
<code>\ =</code>	qiymat yoki <code>qiymat-...</code>	<code>[lang\ ="uz"]</code>	<code>uz</code> , <code>uz-UZ</code>

Amaliy misollar:

```
/* Tashqi havolalarni ajratish */
a[href^="https://"] { color: #2563eb; }

/* Fayl turini ikonka bilan belgilash */
a[href$=".pdf"]::after { content: " (PDF)"; }

/* Nomida "btn" bo'lgan har qanday klass */
[class*="btn"] { cursor: pointer; }

/* Elektron pochta havolalari */
a[href^="mailto:"] { color: #16a34a; }
```

💡 `^=` (boshi) va `$=` (oxiri) ni eslab qolish uchun: regular expression (matn shabloni) sintaksisida ham `^` "boshi", `$` "oxiri" degani. Bir xil mantiq.

⚠️ Atribut qiymatlarini **qo'shtirnoq** ichiga olish odat tusiga aylansin: `[type="text"]`. Oddiy so'zlar uchun tirnoqsiz ham ishlaydi, lekin qiymatda nuqta, bo'sh joy yoki maxsus belgi bo'lsa, tirnoqsiz buziladi. Doim tirnoq qo'yish — xavfsiz odat.

✳️ Katta-kichik harf: atribut selektorlari odatda qiymatga sezgir (`[type="Text"]` ishlamasligi mumkin). HTML atributlari uchun katta-kichikni e'tiborsiz qoldirishni istasang, yopuvchi qavsdan oldin `i` qo'shasan: `[type="text" i]`.

9.6 Psevdo-klasslar: holatga qarab tanlash

Psevdo-klass (pseudo-class) — elementning HTML'da ko'rinmaydigan **holati** yoki **o'rne** bo'yicha tanlash. Bitta **ikki nuqta** `:` bilan yoziladi. "Psevdo" — "go'yo, soxta" degani: bu HTML'da yozilmagan, lekin go'yoki mavjud klass.

Foydalanuvchi harakati holatlari

Bular foydalanuvchining sichqoncha/klaviatura bilan qilayotgan ishiga javob beradi:

```
a:hover { color: #2563eb; } /* sichqoncha ustida */
a:focus { outline: 2px solid #2563eb; } /* fokusda (Tab bilan) */
a:active { color: #dc2626; } /* bosilayotgan paytda */
```

Nega `:focus` muhim? Klaviatura bilan saytda yuradigan odamlar (sichqonchasiz, yoki maxsus imkoniyatli foydalanuvchilar) qayerda turganini ko'rishi shart. `:focus` stilini olib tashlama — bu qulaylik (accessibility) masalasi.

✦ Tartib muhim: havolalar uchun an'anaviy ketma-ketlik `:link`, `:visited`, `:hover`, `:active` (inglizcha "LoVe HAtE" deb eslanadi). Aks holda `:hover` `:active` ni "bosib qo'yishi" mumkin.

Tuzilma (o'rin) bo'yicha psevdoklasslar

Bular elementning ota-onasi ichidagi **o'rni** bo'yicha tanlaydi:

```
li:first-child { font-weight: bold; } /* ro'yxatdagi BIRINCHI li */
li:last-child { border: none; } /* OXIRGI li */
li:nth-child(2) { color: red; } /* aniq 2-li */
```

`:nth-child()` — eng kuchlisi. Ichiga **formula** yoki kalit so'z yozasan:

```
li:nth-child(odd) { background: #f8fafc; } /* toq: 1,3,5 */
li:nth-child(even) { background: #e2e8f0; } /* juft: 2,4,6 */
li:nth-child(2n) { ... } /* even bilan bir xil */
li:nth-child(3n) { ... } /* har 3-: 3,6,9 */
li:nth-child(3n+1) { ... } /* 1,4,7,10 */
```

Quyidagi diagramma formulani vizual tushuntiradi:

:nth-child() — qaysi qatorlar tanlanadi?

:nth-child(2n)

juft: 2, 4, 6



:nth-child(odd)

toq: 1, 3, 5



Formulani o'qish: $an + b$

$n = 0, 1, 2, 3 \dots$ (har bir qadamda navbatdagi qiymat olinadi)

$3n+1 \rightarrow 1, 4, 7 \dots$ · **odd** = $2n+1$ · **even** = $2n$

Formulani qanday o'qish kerak? $an+b$ da n ketma-ket 0, 1, 2, 3... qiymat oladi: - $2n \rightarrow 2 \cdot 0 = 0$ (yo'q), $2 \cdot 1 = 2$, $2 \cdot 2 = 4$, $2 \cdot 3 = 6 \rightarrow$ **2, 4, 6** - $3n+1 \rightarrow 3 \cdot 0 + 1 = 1$, $3 \cdot 1 + 1 = 4$, $3 \cdot 2 + 1 = 7 \rightarrow$ **1, 4, 7**

💡 Eng mashhur amaliy ishi — jadval qatorlarini almashlab bo'yash ("zebra"): `tr:nth-child(even) { background: #f1f5f9; }`. Ko'z toliqmaydi, qatorlar adashmaydi.

:first-of-type va o'rin psevdolari farqi

`:first-child` "ota-ona ichidagi birinchi farzand bo'lsa" deydi — **agar u o'sha turdagi bo'lsa**. `:first-of-type` esa "o'z TURI bo'yicha birinchi" deydi.

```
<div>
  <h2>Sarlavha</h2>
  <p>Birinchi paragraf</p>
  <p>Ikkinchi paragraf</p>
</div>
```

```
p:first-child { ... } /* HECH NARSA — birinchi farzand h2, p emas */
p:first-of-type { ... } /* "Birinchi paragraf" — p'lar orasida birinchi */
```

⚠ Bu eng ko'p chalkashtiriladigan joy. `p:first-child` "birinchi farzand HAM p bo'lishi shart" demakdir. Agar birinchi farzand boshqa element bo'lsa — hech nima

tanlanmaydi. Aralash bolalarda ko'pincha senga `:first-of-type` kerak bo'ladi.

Inkor: `:not()`

`:not()` — "shunday BO'LMAGAN" elementlarni tanlaydi. Ichiga selektor yozasan.

```
li:not(.faol)           { opacity: 0.6; } /* .faol BO'LMAGAN li */
input:not([type="submit"]) { width: 100%; } /* submit BO'LMAGAN input */
button:not(:last-child) { margin-right: 8px; }
```

Nega kerak? "Oxirgisidan tashqari hammasiga" kabi qoidalar juda ixcham yozadi. Yuqoridagi oxirgi misol: oxirgi tugmadan boshqa hamma tugmaga o'ng tomondan bo'shliq beradi — natijada tugmalar orasida bo'shliq bo'lib, oxirida ortiqcha bo'shliq qolmaydi.

Forma holatlari

```
input:checked { ... } /* belgilangan checkbox/radio */
input:disabled { background: #e2e8f0; } /* o'chirilgan */
input:enabled { ... } /* yoqilgan */
input:required { ... } /* majburiy */
input:valid { border-color: #16a34a; } /* to'g'ri to'ldirilgan */
input:invalid { border-color: #dc2626; } /* xato to'ldirilgan */
```

💡 `:checked` JavaScript'siz juda ko'p narsa qilishga imkon beradi: ochiladigan menyular, tab'lar, akkordeonlar — barchasi yashirin checkbox + `:checked` + birodar kombinatori bilan quriladi.

9.7 Psevdo-elementlar: yangi "soxta" elementlar yaratish

Psevdo-**klass** mavjud elementni tanlardi. Psevdo-**element** esa elementning bir QISMINI tanlaydi yoki **butunlay yangi** (HTML'da yo'q) mazmun qo'shadi. U **ikkita ikki nuqta** `::` bilan yoziladi — shu bilan psevdo-klassdan farqlanadi.

🔴 Tarixiy sabab: eski CSS'da bitta `:` ishlatilardi (`:before`). Hozir ikkitasi (`::before`) standart. Brauzerlar ikkalasini ham tushunadi, lekin yangi kodda doim `::` yoz.

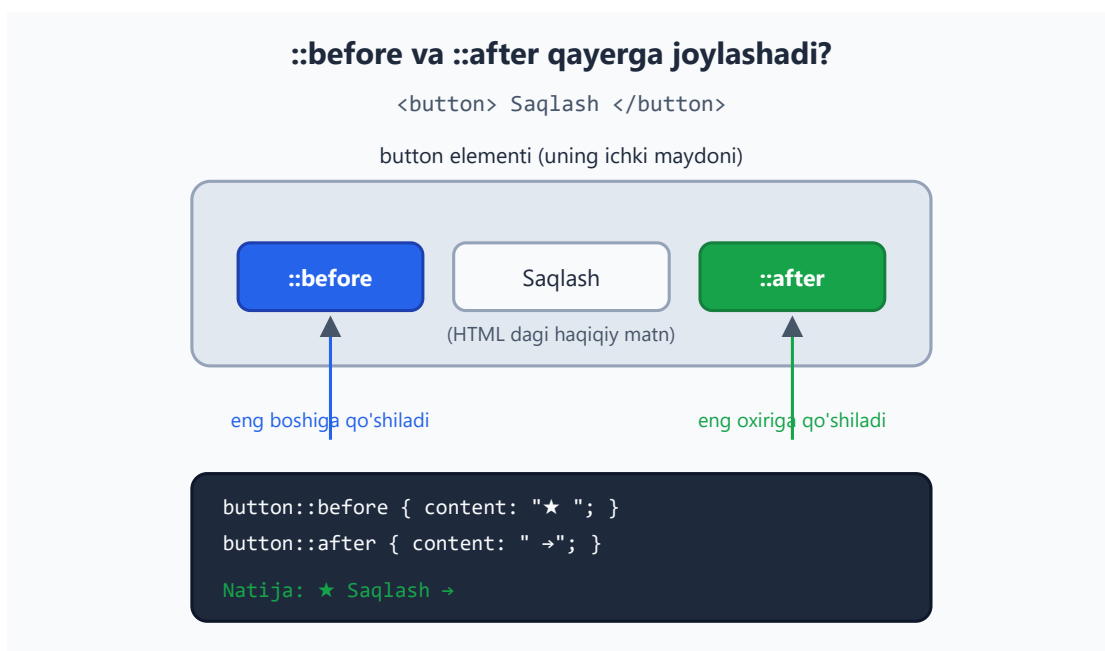
`::before` va `::after` — kontent qo'shish

Bular element ichiga, mazmunining **boshiga** (`::before`) yoki **oxiriga** (`::after`) yangi mazmun qo'shadi. Ular ishlashi uchun **content** xususiyati shart (bo'sh bo'lsa ham:


```
content: "" ).
```

```
.tashqi-havola::after {  
  content: " ↗";  
  color: #2563eb;  
}  
  
.muhim::before {  
  content: "★ ";  
  color: #f59e0b;  
}
```

Quyidagi diagramma `::before` va `::after` element ichida aniq qayerga qo'shilishini ko'rsatadi:



Nega `content` shart? `::before` / `::after` aslida "bo'sh joy" — sen unga nima qo'yishni `content` orqali aytasan. `content` yo'q bo'lsa, psevd-element umuman paydo bo'lmaydi.

Amaliy qo'llanishlar: - Ikonkalar va belgilar qo'shish (yulduzcha, o'q, tirnoq belgilari). - Dekorativ chiziqlar va shakllar (`content: "" + width / height / background`). - Majburiy maydonlarga * qo'shish: `label.majburiy::after { content: " *"; color: #dc2626; } .`

⚠ Muhim qoida: `::before` / `::after` ga qo'shilgan mazmun **dekorativ**. Ekran o'quvchilari (screen reader) uni har doim ham o'qimaydi va u nusxalashda olinmasligi mumkin. Shuning uchun **muhim ma'lumotni** (masalan, asosiy matnni yoki tugma yorlig'ini) bu yerga yozma — faqat bezak uchun ishlat.

Matn qismlarini tanlash: `::first-line`, `::first-letter`

```
p::first-letter {           /* birinchi HARF */
  font-size: 3em;
  float: left;
  color: #2563eb;
}

p::first-line {             /* birinchi SATR */
  font-weight: bold;
}
```

`::first-letter` — gazeta/kitoblardagi katta bosh harf (drop cap) effekti uchun klassik. `::first-line` esa birinchi satrni ajratadi — diqqat: "satr" oyna kengligiga qarab o'zgaradi, shuning uchun bu **dinamik**.

`::selection` — belgilangan matn

Foydalanuvchi sichqoncha bilan ajratib (tanlab) olgan matn ko'rinishini o'zgartiradi:

```
::selection {
  background: #2563eb;
  color: #ffffff;
}
```

💡 Brend ranglaringizga mos tanlov rangi — sayt pishiqiligini ko'rsatadigan kichik, lekin yoqimli detal.

`::placeholder` — input ichidagi ko'rsatma matn

```
input::placeholder {
  color: #94a3b8;
  font-style: italic;
}
```

`<input placeholder="Ismingiz">` ichidagi xira ko'rsatma matnning ko'rinishini boshqaradi.

⚠ Placeholder rangini juda xira qilma — o'qib bo'lmaydigan ko'rsatma yo'q bo'lgani bilan barobar. Va placeholder yorliq (label) o'rnini bosa olmaydi: har bir maydonga haqiqiy `<label>` ber.

Bir nuqta yoki ikki nuqta?

Tur	Belgi	Misol	Nimani
Psevdo-klass	<code>:</code> (bitta)	<code>:hover</code> , <code>:nth-child()</code>	mavjud elementni holat/o'rin bo'yicha
Psevdo-element	<code>::</code> (ikki)	<code>::before</code> , <code>::first-letter</code>	element qismi yoki yangi mazmun

9.8 Murakkab selektorlarni zanjirlash

Haqiqiy loyihalarda selektorlarni **birlashtirib** ishlatasan. Birlashtirishning ikki turi bor: **biriktirish** (bo'shliqsiz) va **kombinator** (bo'shliq yoki belgi bilan).

Bo'shliqsiz biriktirish — "VA" mantiqi

Selektorlarni bo'shliqsiz yozsang, ularning **hammasi** bir elementga tegishi shart:

```
p.muhim { ... } /* p VA klassi muhim */
input[type="text"]:focus { ... } /* input VA type=text VA fokusda */
button.asosiy:hover { ... } /* button VA klassi asosiy VA hover */
li.fao1:first-child { ... } /* li VA fao1 VA birinchi farzand */
```

`p.muhim` — bu `<p class="muhim">` ni tanlaydi, lekin `<div class="muhim">` ni emas (u `p` emas).

⚠ Bo'shliq bor-yoqligi MA'NONI butunlay o'zgartiradi: - `p.muhim` (bo'shliqsiz) = "muhim klassli p". - `p .muhim` (bo'shliq bilan) = "p ichidagi muhim klassli element". Bu butunlay boshqa narsa!

Bu farqni tushunish — boshlovchilarning eng katta sakrashlaridan biri.

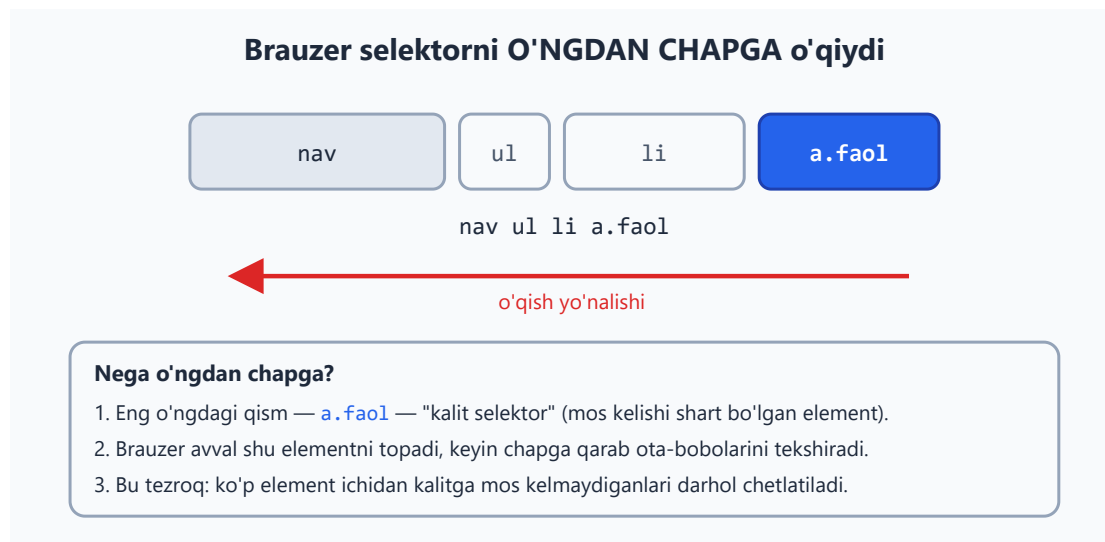
Kombinator bilan birlashtirish

```
nav ul li a { ... } /* nav>ul>li ichidagi a */
.menyu > li:hover { ... } /* .menyu bevosita farzandi li, hover holatda */
form input:not(:disabled){ ... } /* form ichidagi, o'chirilmagan input */
.karta:hover .tugma { ... } /* karta hover bo'lganda ICHIDAGI tugma */
```

Oxirgisi juda kuchli usul: `.karta:hover .tugma` — kartaga sichqoncha kelganda uning ichidagi tugma ko'rinishini o'zgartiradi. Hover bir elementda, ta'sir esa boshqasida.

9.9 Selektorni o'qish strategiyasi: o'ngdan chapga

Endi expert sirini ochamiz. **Brauzer selektorni o'ngdan chapga o'qiydi** — sen yozgan tartibda emas, teskari.



`nav ul li a.fao1` selektorini olaylik. Sen uni chapdan o'ngga "nav, ichida ul, ichida li, ichida faol a" deb o'qiysan. Brauzer esa teskari ishlaydi:

- Eng o'ngdagi qismdan boshlaydi** — `a.fao1`. Bu "kalit selektor" (key selector). Brauzer avval sahifadagi barcha `.fao1` klassli `<a>` ni topadi.
- Keyin har biri uchun **chapga qarab** tekshiradi: "ota-onasi `li` mi? Uning ota-onasi `ul` mi? Uningniki `nav` mi?"
- Agar zanjir uzilsa — o'sha element rad etiladi.

Nega bu seni qiziqtirishi kerak? Ikki sabab:

1) O'qish va tuzatish (debugging) oson bo'ladi. Murakkab selektorni ko'rganda, avval eng o'ngdagi qismga qara — asosan **shu** element stillanadi, qolgani esa shartlar. `div.sidebar ul.menu li a` → "bu a ga tegadi, qolgan hammasi — a qayerda turishi kerakligining sharti".

2) Unumdorlik (performance). Kalit selektor qanchalik aniq bo'lsa, brauzer shuncha kam elementni tekshiradi. `* + *` kabi keng kalit yomon; `.fao1` kabi aniq kalit yaxshi. Kichik saytlarda farqi sezilmaydi, ammo katta sahifalarda muhim.

💡 Amaliy maslahat: selektorni iloji boricha **qisqa va aniq** tut. `body div.wrapper main section.content article p` o'rniga ko'pincha `.content p` yoki shunchaki bitta klass yetarli. Uzun zanjir nafaqat sekin, balki **mo'rt** — HTML tuzilishi biroz o'zgarsa, darhol buziladi.

9.10 Tez-tez uchraydigan xatolar va maslahatlar

✳️ Bobni yakunlashdan oldin, eng ko'p uchraydigan tuzoqlarni bir joyga to'playmiz:

- ⚠️ **Bo'shliqni unutish/qo'shib yuborish.** `.a.b` (ikkala klass bir elementda) va `.a .b` (`b`, `a` ichida) — butunlay boshqa. Bu eng ko'p uchraydigan xato.
 - ⚠️ **Vergulni tushirib qoldirish.** `h1 h2` (`h1` ichidagi `h2`) va `h1, h2` (ikkalasi). Vergulsiz qoida "ishlamaydi" deb ko'rinadi.
 - ⚠️ **`:first-child` va `:first-of-type` ni aralashtirish.** Aralash bolalar orasida ko'pincha `-of-type` kerak.
 - ⚠️ **`content` ni unutish.** `::before / ::after content` siz umuman ko'rinmaydi.
 - ⚠️ **Id'ga stil berish.** Ishlaydi, lekin spesifiklik baland bo'lib, keyin uni qayta yozish azobga aylanadi. Stil uchun klass ishlat.
 - 💡 **`:focus` ni o'chirmagin.** Klaviatura foydalanuvchilari uchun zarur. Standart `outline` xunuk ko'rinsa, uni o'chirma — chiroyliroq qilib qayta chiz.
 - 💡 **Selektorni qisqa tut.** Uzun zanjir sekin va mo'rt. Ko'p hollarda bitta yaxshi nomlangan klass yetarli.
 - 💡 **Atribut qiymatlarini tirnoqqa ol.** `[type="text"]` — har doim xavfsiz odat.
-

Mashqlar

Quyidagi mashqlarni daftarda yoki brauzerda sinab ko'r. Har biri bobning bir bo'limini mustahkamlaydi. Avval o'zing yozib ko'r, keyin yechimni och.

1-mashq. Quyidagi HTML uchun: barcha `<p>` matnini kulrang qil, lekin `class="muhim"` bo'lganlarini qora va qalin qil.

```
<p>Oddiy paragraf.</p>
<p class="muhim">Muhim paragraf.</p>
```





Yechim



```
p { color: #94a3b8; }  
p.muhiim { color: #1e293b; font-weight: bold; }
```



`p.muhiim` — bo'shliqsiz, ya'ni "p VA muhim klassli". `.muhiim` qoidasi `p` dan keyin kelgani uchun (va aniqroq bo'lgani uchun) g'olib chiqadi.

2-mashq. Bir `<ul>` ichidagi `<li>` larni "zebra" qilib bo'yang: toq qatorlar oq, juft qatorlar och-kulrang (`#f1f5f9`).



Yechim



```
li:nth-child(even) {  
  background: #f1f5f9;  
}
```



`even` (yoki `2n`) — 2, 4, 6-qatorlarni tanlaydi. Toq qatorlarga tegmaganimiz uchun ular standart (oq) fonda qoladi.

3-mashq. Faqat `https://` bilan boshlanadigan havolalardan keyin "↗" belgisini chiqaring.



Yechim



```
a[href^="https://"]::after {  
  content: " ↗";  
  color: #2563eb;  
}
```



`[href^="https://"]` — qiymati `https://` bilan **boshlanadigan** href. `::after` + `content` — havoladan keyin belgi qo'shadi.

4-mashq. Tugmalar qatorida har bir tugmaga o'ngdan 8px bo'shliq bering, **lekin** oxirgi tugmaga bermang (oxirida ortiqcha bo'shliq qolmasin).

```
<div class="panel">  
  <button>Bir</button>  
  <button>Ikki</button>  
  <button>Uch</button>  
</div>
```





Yechim



```
.panel button:not(:last-child) {  
  margin-right: 8px;  
}
```



`:not(:last-child)` — oxirgisidan boshqa barcha tugma. Shu bilan oxirgisida margin qolmaydi. (Zamonaviy yechim — ota-onaga `gap` berish, lekin `:not()` mashqi sifatida bu ajoyib.)

5-mashq. Quydagi tuzilmada faqat `.menu` ning **bevosita** farzandi bo'lgan `<li>` larga qalin shrift bering; ichki ostmenu `<li>` lariga tegmang.

```
<ul class="menu">  
  <li>Bosh sahifa</li>  
  <li>Mahsulotlar  
    <ul>  
      <li>Telefonlar</li>  
      <li>Noutbuklar</li>  
    </ul>  
  </li>  
</ul>
```



Yechim



```
.menu > li {  
  font-weight: bold;  
}
```



`>` bevosita farzandni tanlaydi. Ichki `<ul>` dagi `<li>` lar `.menu` ning bevosita farzandi emas (ular ichki `ul` ning farzandi), shuning uchun tegilmaydi. Agar bo'shliq (`.menu li`) ishlatilsa, ularga ham qalin shrift tushardi.

6-mashq. Barcha matnli input maydonlariga fokus tushganda ko'k chegara bering, lekin o'chirilgan (`disabled`) maydonlarga tegmang.



Yechim



```
input[type="text"]:focus:not(:disabled) {  
  border: 2px solid #2563eb;  
  outline: none;  
}
```



Uchta shart zanjirlangan: `input[type="text"]` (matn maydoni) VA `:focus` (fokusda) VA `:not(:disabled)` (o'chirilmagan). Eslatma: `:disabled` element odatda fokus ham ololmaydi, shuning uchun `:not(:disabled)` ko'pincha ortiqcha — lekin niyatni aniq ko'rsatgani uchun foydali mashq.

7-mashq (qiyinroq). Maqola ichidagi birinchi paragrafning birinchi harfini katta "drop cap" qilib bering (3em, ko'k, chapga oqsin). Faqat birinchi paragrafga ta'sir qilsin.

```
<article>  
  <h2>Sarlavha</h2>  
  <p>Birinchi paragraf shu yerdan boshlanadi...</p>  
  <p>Ikkinchi paragraf.</p>  
</article>
```



Yechim



```
article p:first-of-type::first-letter {  
  font-size: 3em;  
  float: left;  
  color: #2563eb;  
  margin-right: 4px;  
}
```



`p:first-of-type` — `<article>` ichidagi p'lar orasidan **birinchisi** (bu yerda `:first-child` ishlamasdi, chunki birinchi farzand `<h2>`). `::first-letter` — uning birinchi harfi. Ikki tushuncha bir zanjirda.

8-mashq (debugging). Quyidagi CSS mo'ljallangandek ishlamayapti. Maqsad: `.karta` ichidagi `.sarlavha` klassli elementni ko'k qilish. Xatoni top va tuzat.

```
.karta.sarlavha {  
  color: #2563eb;  
}
```





Yechim



Xato — bo'shliqning yo'qligi. `.karta.sarlavha` (bo'shliqsiz) = "ham `karta`, ham `sarlavha` klassiga ega BITTA element". Bizga esa " `.karta` ICHIDAGI `.sarlavha` " kerak edi:

```
.karta .sarlavha {  
  color: #2563eb;  
}
```



Yagona bo'shliq butun ma'noni o'zgartiradi — bu bobning eng muhim saboqlaridan biri.

[← Oldingi: 08 — CSS asoslari](#) · [🏠 README](#) · [Keyingi: 10 — Cascade, specificity va inheritance](#) [→](#)

10 — Cascade, specificity va inheritance

← Oldingi: 09 — Selektorlar · 🏠 README · Keyingi: 11 — Box model →

Bu bobda: bir nechta CSS qoidasi bitta elementga teginganda qaysi qoida g'olib chiqishini hal qiladigan uchta mexanizm — kaskad (cascade), aniqlik (specificity) va meros (inheritance) — to'liq o'rganamiz.

10.1 Nega bu bob "CSS sirlarini" ochadi?

Tasavvur qil: sen `<p>` matniga `color: blue` berding, lekin matn baribir qizil chiqyapti. Yoki bir joyda yozgan stilingiz boshqa joyda "ishlamayapti". Boshlovchilar CSS'ni "injiq" deb o'ylaydi — aslida CSS juda **mantiqiy**. U faqat aniq qoidalar bo'yicha qaysi stil g'olib chiqishini hisoblaydi.

Bu bobning kaliti — bitta savol:

Bir elementga bir nechta qoida teginsa, kim yutadi?

Javob uchta omilga bog'liq: **kaskad** (cascade), **aniqlik** (specificity) va **meros** (inheritance). Bularni tushunsang, CSS endi sehr emas, balki tushunarli tizim bo'lib qoladi. Aynan shu bilim seni boshlovchidan professionalga ajratadi.

📌 **Atamalar tarjimasini:** - **cascade** — kaskad, ya'ni "sharshara"/"pog'onali oqim". CSS nomi ham shundan: *Cascading* Style Sheets. - **specificity** — aniqlik, ya'ni selektorning "og'irligi". - **inheritance** — meros, ya'ni ota elementdan farzandga xossa o'tishi.

10.2 Cascade nima?

Cascade (kaskad) — bu bitta elementga tegadigan barcha qoidalarni ko'rib chiqib, ulardan **g'olibini tanlaydigan** algoritm. "Cascading" so'zi "pog'onali tushish" degani: qoidalar bir nechta bosqichdan o'tadi va har bosqichda saralanadi.

Oddiy misol bilan boshlaylik. Bir xil elementga ikkita rang berdik:

```
p {  
  color: blue;  
}  
p {
```



```
color: red;  
}
```

<p>Bu matn qaysi rangda bo'ladi?</p>

Natija: matn **qizil** bo'ladi. Nega? Chunki ikkala qoida ham bir xil "og'irlikda" (ikkalasi ham oddiy `p` selektori), shuning uchun cascade **oxirgi yozilganini** tanlaydi. Bu "manba tartibi" (source order) qoidasi — pastda batafsil ko'ramiz.

💡 **Hayotiy analogiya:** navbatda turgan odamlar — agar ikki kishi bir xil maqomda bo'lsa, oxirgi kelgan emas, balki *kuchliroq dalili borgi* o'tadi. Cascade ham xuddi shunday: avval "kim kuchliroq", agar kuch teng bo'lsa "kim oxirgi" deb hal qiladi.

Cascade uchta omilni **shu tartibda** tekshiradi:

1. **Muhimlik** (origin va `!important`) — qoida qayerdan keldi va `!important` bormi?
2. **Specificity** (aniqlik bali) — selektor qanchalik aniq?
3. **Manba tartibi** (source order) — qoida faylda qayerda yozilgan?

Avval birinchi omil hal qiladi. Agar tenglik bo'lsa, keyingisiga o'tadi. Quyidagi diagramma butun oqimni ko'rsatadi:



Endi har bir omilni alohida ochamiz.

10.3 Birinchi omil: muhimlik (origin va `!important`)

Cascade eng avval qoidaning **muhimligini** (importance) tekshiradi. Muhimlik ikki narsadan iborat: qoida **qayerdan** keldi (origin) va unda `!important` **bormi**.

Origin — qoida qayerdan keladi?

Brauzer stillarni uch manbadan oladi:

Manba (origin)	Bu nima	Misol
User-agent	Brauzerning o'z standart stillari	<code><h1></code> katta va qalin, link ko'k va tagchiziqli
User	Foydalanuvchining shaxsiy sozlamalari	Foydalanuvchi "katta shrift" rejimini yoqqani
Author	Sen yozgan CSS (sayt muallifi)	<code>style.css</code> faylingdagi qoidalar

Oddiy holatda ustunlik tartibi (kuchsizdan kuchliga): **user-agent < user < author**. Ya'ni sen yozgan CSS brauzerning standart stilini bimalol bosib o'tadi — shuning uchun `a { color: green; }` desang, linklar ko'k emas, yashil bo'ladi.

✦ Ko'pincha sen faqat **author** stillari bilan ishlaysiz, shuning uchun amalda origin haqida kam o'ylaysiz. Lekin u cascade'ning eng birinchi bosqichi ekanini bilib qo'y.

`!important` — qoidani "majburan g'olib" qilish

Qoida oxiriga `!important` qo'shsang, u **muhimlik bosqichida** boshqalardan ustun chiqadi:

```
p {  
  color: blue !important;  
}  
p {  
  color: red;  
}
```

Natija: matn **ko'k** bo'ladi — garchi `red` keyin yozilgan bo'lsa ham. `!important` muhimlik bosqichida g'olib bo'lgani uchun, cascade specificity va tartibgacha yetib ham bormaydi.

⚠️ **!important bilan bir nozik gap:** u origin tartibini **teskari aylantiradi**. Ya'ni **!important** qo'shilganlar orasida ustunlik: **author !important < user !important**. Bu foydalanuvchining ehtiyojlarini himoya qilish uchun (masalan, ko'rish qobiliyati cheklangan foydalanuvchi shriftni majburan kattalashtira olishi uchun). Boshlovchi sifatida buni shunchaki "**!important** kuchli" deb eslab qo'ysang bas.

Nega **!important** dan qochish kerak?

!important — bu "atom bomba" tugmasi. U ishlaydi, lekin keyinroq katta muammo tug'diradi:

```
/* Birovning kodida: */
.tugma {
  background: gray !important;
}

/* Sen yangi rang bermoqchisan: */
.tugma.fao1 {
  background: green; /* ISHLAMAYDI! */
}
```

Natija: `.tugma.fao1` aniqroq selektor bo'lsa ham, ishlamaydi — chunki **!important** muhimlik bosqichida g'olib bo'ladi va specificity bosqichigacha yetib bormaydi. Endi sen ham **!important** qo'shishga majbur bo'lasan, keyin yana kimdir... Bu "**!important urushi**" — kod tobora chigallashadi.

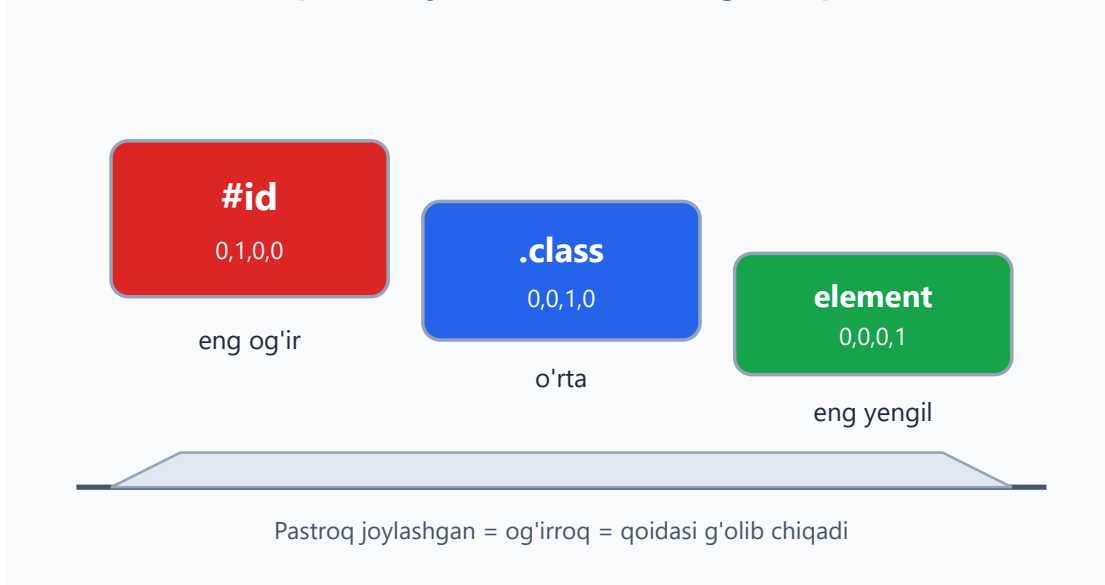
💡 **Maslahat:** **!important** o'rniga **aniqroq selektor** ishlat yoki qoidani to'g'ri tartibda yoz. **!important** ni faqat boshqa iloji bo'lmaganda (masalan, o'zgartira olmaydigan tashqi kutubxona stilini majburan bosishda) ishlat.

10.4 Ikkinchi omil: specificity (aniqlik bali)

Agar ikki qoida **muhimlik bo'yicha teng** bo'lsa (ikkalasida ham **!important** yo'q, ikkalasi ham author stili), cascade ikkinchi omilga — **specificity** ga o'tadi.

Specificity — bu selektorning "og'irligi" yoki "aniqlik bali". Selektor qanchalik aniq nishonni ko'rsatsa, shunchalik kuchli. Mantiq sodda: **#asosiy-sarlavha** aniq bitta elementni ko'rsatadi, **p** esa barcha paragraflarni — shuning uchun aniqroq selektor g'olib bo'lishi adolatli.

Specificity tarozisi — kim og'irroq?



Specificity qanday hisoblanadi?

Specificity **to'rt xonali ball** sifatida tasavvur qilinadi: (a, b, c, d). Har bir tur o'z ustuniga ball qo'shadi:

Ustun	Nima sanaladi	Misol	Ball
a (inline)	<code>style="..."</code> atributi	<code><p style="color:red"></code>	1,0,0,0
b (id)	<code>#id</code> selektorlari	<code>#header</code>	0,1,0,0
c (class/attr/pseudo-class)	<code>.class</code> , <code>[attr]</code> , <code>:hover</code>	<code>.menu</code> , <code>[type="text"]</code> , <code>:hover</code>	0,0,1,0
d (element/pseudo-element)	tag nomi, <code>::before</code>	<code>p</code> , <code>div</code> , <code>::before</code>	0,0,0,1

✦ **Muhim nuqtalar:** - **Universal selektor** `*`, va `:where()` ichidagilar specificity'ga **0** qo'shadi. - `:not()`, `:is()`, `:has()` — ulardan eng kuchli argumenti hisoblanadi (qavslari emas). - **Inline stil** har qanday selektor-qoidadan kuchli (ustun **a**).

Ballarni qanday solishtiramiz?

Ballar **eng chap ustundan boshlab** solishtiriladi. Avval **a** (inline), keyin **b** (id), keyin **c** (class), oxirida **d** (element). Birinchi farq qayerda topilsa, o'sha hal qiladi — qolgan ustunlarga qaramaymiz.

Bu **xona-raqamlar** kabi, lekin har ustun mustaqil. Masalan $(0, 1, 0, 0)$ har doim $(0, 0, 9, 9)$ dan kuchli, chunki id ustunida $1 > 0$ — class va element soni nechta bo'lishidan qat'i nazar.

⚠ **Tez-tez uchraydigan tushunmovchilik:** "11 ta class id'dan kuchlimi?" — YO'Q. $(0, 0, 11, 0)$ baribir $(0, 1, 0, 0)$ dan **kuchsiz**, chunki id ustunidagi $0 < 1$. Ustunlar bir-biriga "qo'shilib ketmaydi".

Ball sanash amaliyoti

Quyidagi jadval to'rtta selektorning balini ko'rsatadi:

Specificity balini qanday sanaymiz?			
Format: (inline, id, class/attr/pseudo-class, element/pseudo-element)			
Selektor	id	class	element
p	0	0	1
.menu	0	1	0
nav .menu li	0	1	2
#header .menu a	1	1	1

Eng katta bal eng chapdan boshlab solishtiriladi:
#header .menu a $(0, 1, 1, 1)$ eng kuchli — id bor.

Endi qo'lda sanab ko'raylik. Quyidagi selektorlarning balini topamiz:

p	/* element: 1 ta → $(0, 0, 0, 1)$ */	
.menu	/* class: 1 ta → $(0, 0, 1, 0)$ */	
nav .menu li	/* element: 2 (nav, li) + class: 1 (.menu) → $(0, 0, 1, 2)$ */	
#header .menu a	/* id: 1 + class: 1 + element: 1 → $(0, 1, 1, 1)$ */	

Bu to'rttasidan **#header .menu a** eng kuchli, chunki uning id ustunida **1** bor, boshqalarda **0**.

Amaliy misol: kim yutadi?



```
.matn {
  color: blue;      /* (0,0,1,0) */
}
p {
  color: green;     /* (0,0,0,1) */
}
```

```
<p class="matn">Qaysi rang?</p>
```

Natija: matn **ko'k** (blue). Nega? `.matn` ning bali `(0,0,1,0)`, `p` ning bali `(0,0,0,1)`. Class ustunida `1 > 0`, shuning uchun `.matn` g'olib — `p` keyin yozilgan bo'lsa ham, ahamiyati yo'q, chunki specificity bosqichida masala hal bo'ldi.

💡 **Asosiy xulosa:** specificity tartibdan **muhimroq**. Manba tartibi faqat specificity teng bo'lgandagina ahamiyatga ega bo'ladi.

10.5 Uchinchi omil: manba tartibi (source order)

Agar ikki qoida **ham muhimlik, ham specificity bo'yicha teng** bo'lsa, cascade oxirgi omilga keladi: **manba tartibi** (source order). Bu eng sodda qoida:

Faylda oxirgi yozilgan qoida g'olib bo'ladi.

```
.tugma {
  background: blue;
}
.tugma {
  background: green; /* g'olib — chunki keyin yozilgan */
}
```

Natija: fon **yashil**. Ikkala selektor ham `(0,0,1,0)` — bir xil specificity, hech qaysisida `!important` yo'q. Shuning uchun oxirgisi yutadi.

✦ **"Faylda oxirgi" nimani anglatadi?** - Bir fayl ichida — pastroqda yozilgani. - Bir nechta CSS faylda — HTML'da `<link>` keyin ulangan fayl kuchliroq. - `<style>` blok va tashqi fayl aralashganda — hujjatdagi kelish tartibi bo'yicha.

💡 **Nega bu foydali?** Ko'pchilik CSS kutubxonalari (masalan Bootstrap) shu tamoyilga tayanadi: avval kutubxona stilini ulaysan, keyin o'z `custom.css` faylingni — natijada sening stiling kutubxonanikidan ustun bo'ladi (teng specificity'da). Shuning uchun **o'z faylingni har doim oxirida ula**.

10.6 Hammasini birlashtirish: to'liq qaror tartibi

Endi cascade'ning to'liq mantig'ini bir joyda jamlaymiz. Bir elementga tegadigan ikki qoida orasidan g'olibni topish uchun cascade **shu tartibda** savol beradi:

1. **Muhimlik teng emasmi?** `!important` borisi yoki kuchliroq origin yutadi. → Hal bo'ldi.
2. **Teng bo'lsa: specificity teng emasmi?** Aniqroq (kattaroq ballik) selektor yutadi. → Hal bo'ldi.
3. **U ham teng bo'lsa: manba tartibi.** Oxirgi yozilgan yutadi.

Bitta to'liq misolda ko'raylik:

```
p {  
  color: green;           /* (0,0,0,1) */  
}  
.maqola p {  
  color: blue;           /* (0,0,1,1) */  
}  
#asosiy .maqola p {  
  color: orange;        /* (0,1,1,1) */  
}
```

```
<div id="asosiy">  
  <div class="maqola">  
    <p>Bu matn qaysi rangda?</p>  
  </div>  
</div>
```

Natija: matn **to'q sariq** (orange). Uchala qoidaning hech birida `!important` yo'q (muhimlik teng), shuning uchun specificity solishtiriladi: $(0,1,1,1) > (0,0,1,1) > (0,0,0,1)$. Eng kattasi — `#asosiy .maqola p` — yutadi.

⚠ **E'tibor ber:** agar qaysidir qoidaga `!important` qo'shsang, bu tartib buziladi. Masalan birinchi `p { color: green !important }` desak, natija **yashil** bo'lardi — chunki `!important` muhimlik bosqichida hammasini bosib o'tadi.

10.7 Inheritance (meros) nima?

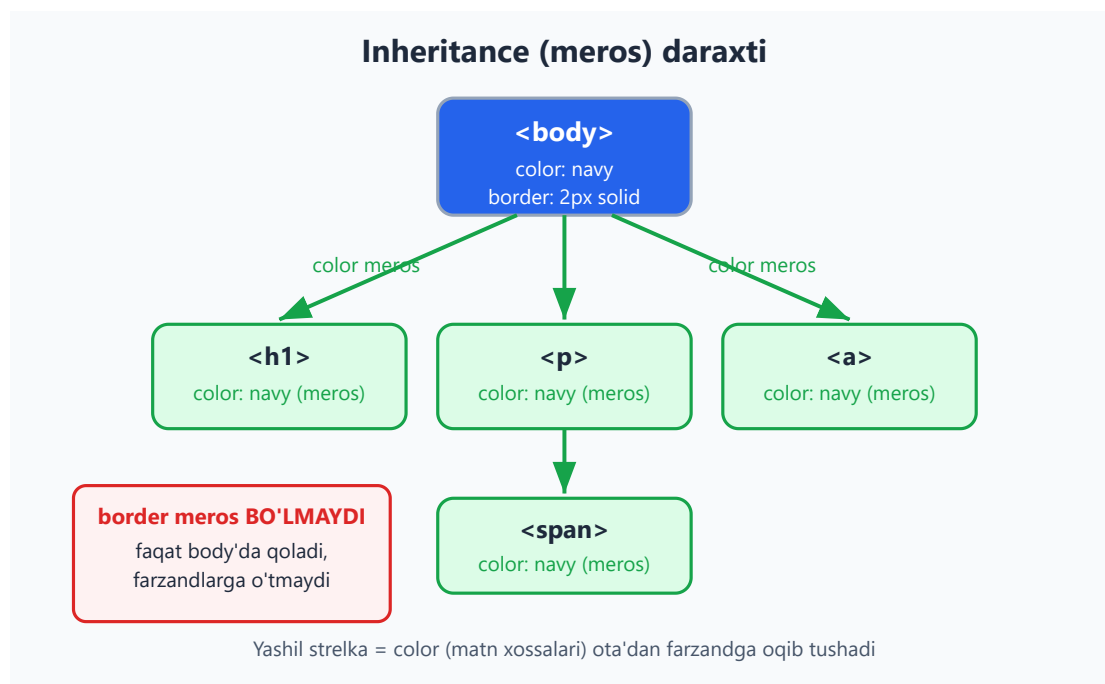
Endi butunlay boshqa mexanizmga o'tamiz. Cascade "bir nechta qoida orasidan g'olibni tanlash" haqida edi. **Inheritance** (meros) esa boshqa savol: agar elementga *hech qanday* qoida tegmasa, u xossani qayerdan oladi?

Inheritance — bu ba'zi xossalarning **ota elementdan farzand elementga avtomatik o'tishi**. Ya'ni `<body>` ga `color: navy` bersang, ichidagi barcha matnlar (`<p>`, `<h1>`, `<span>` ...) avtomatik ko'k-qoramtir bo'ladi — sen ularning har biriga alohida rang bermasang ham.

```
body {  
  color: navy;  
  font-family: Georgia, serif;  
}
```

```
<body>  
  <h1>Sarlavha ham navy</h1>  
  <p>Paragraf ham navy, shrifti ham Georgia.</p>  
</body>
```

Natija: `<h1>` va `<p>` ga umuman tegmadik, lekin ikkalasi ham `navy` rangda va Georgia shriftida chiqadi — `color` va `font-family` **meros bo'ladigan** xossalardir.



💡 **Hayotiy analogiya:** meros — oilaviy familiya kabi. Agar bola o'ziga boshqa familiya tanlamasa, otasinikini oladi. CSS'da ham: farzand element xossani o'zi belgilamasa, otasinikini "meros" qiladi.

Nega meros umuman bor?

Meros — qulaylik uchun. Tasavvur qil, har bir `<p>`, `<span>`, `<li>` ga alohida shrift va rang yozishing kerak bo'lsa edi — bu dahshat bo'lardi. Meros tufayli sen `body` ga bir

marta yozsang, butun sahifa shu shriftni oladi. **Matn bilan bog'liq xossalar** aynan shuning uchun meros qilinadi.

10.8 Qaysi xossalar meros bo'ladi, qaysilar yo'q?

Hamma xossalar ham meros bo'lavermaydi. Umumiy qoida sodda va mantiqiy:

Matn/tipografiya xossalari meros bo'ladi. Tartib/o'lcham (box) xossalari meros bo'lmaydi.

Meros BO'LADI (matn)	Meros BO'LMAYDI (box/tartib)
color	margin
font-family, font-size, font-weight	padding
line-height, letter-spacing	border
text-align	width, height
visibility	background
list-style	display, position
cursor	box-shadow

Nega bunday bo'lingan? O'ylab ko'r: agar `border` meros bo'lganida, `body` ga ramka qo'ysang, ichidagi *har bir* element ham ramkaga o'ralib qolardi — vahima! Yoki `margin` meros bo'lsa, har bir farzand otasining bo'shlig'ini takrorlab, sahifa buzilib ketardi. Shuning uchun **vizual tartib xossalari ataylab meros qilinmaydi** — har element o'z o'lchamini o'zi nazorat qiladi.

Matn xossalari esa aksincha: ular farzandga "oqib tushgani" tabiiy va foydali.

✦ **Eslatma:** `background` meros bo'lmaydi, lekin ko'p hollarda fon **ko'rinadigandek** tuyuladi — chunki farzand elementlarning foni odatda **shaffof** (`transparent`), shuning uchun otasining foni *o'tib ko'rinadi*. Bu meros emas — shunchaki shaffoflik. Nozik, lekin muhim farq.

10.9 Meros kalit so'zlari: `inherit`, `initial`, `unset`, `revert`

CSS senga merosni **qo'lda boshqarish** uchun maxsus kalit so'zlar beradi. Bularni har qanday xossaga qiymat sifatida berish mumkin.

`inherit` — majburan otadan ol

Xossani **majburan** ota elementnikiga tenglashtiradi — meros bo'lmaydigan xossalar uchun ham ishlaydi.

```
.quticha {  
  border: 2px solid;  
}  
.quticha .ichki {  
  border: inherit; /* normalda border meros bo'lmasdi, lekin majburan oldik */  
}
```

Natija: `.ichki` element otasining `border` ini oladi — `inherit` buni majburan amalga oshirdi.

`initial` — standart qiymatga qaytar

Xossani CSS spetsifikatsiyasidagi **boshlang'ich (default) qiymatiga** qaytaradi — ota nima berganidan qat'i nazar.

```
body { color: navy; }  
.maxsus {  
  color: initial; /* navy'ni emas, color'ning standart qiymatini (qora) oladi */  
}
```

Natija: `.maxsus` matni qora bo'ladi (`color` ning initial qiymati — `canvastext`, amalda qora), garchi otasi `navy` bersa ham.

`unset` — "aqli" tiklash

Bu eng aqli kalit so'z: xossa turiga qarab o'zini tutadi: - Agar xossa **meros bo'ladigan** bo'lsa (masalan `color`) → `inherit` kabi ishlaydi. - Agar xossa **meros bo'lmaydigan** bo'lsa (masalan `margin`) → `initial` kabi ishlaydi.

```
.element {  
  color: unset; /* color meros bo'ladi → otadan oladi (inherit kabi)
```

```
*/
margin: unset; /* margin meros bo'lmaydi → standart 0 ga qaytadi
(initial kabi) */
}
```

revert — brauzer standartiga qaytar

Sening (author) stilingni olib tashlab, **brauzerning (user-agent) standart stiliga** qaytaradi. `initial` dan farqi: `initial` CSS spetsifikatsiya qiymatiga, `revert` esa brauzer qiymatiga qaytaradi.

```
h1 { font-size: 12px; } /* h1 ni kichraytirdik */
.asl {
  font-size: revert; /* brauzerning katta h1 standartiga qaytadi
*/
}
```

Natija: `<h1 class="asl">` brauzerning odatdagi katta sarlavha o'lchamini tiklaydi.

💡 Qisqacha farqi:

Kalit so'z	Nimaga qaytaradi
<code>inherit</code>	Otaning qiymatiga
<code>initial</code>	CSS spetsifikatsiya standartiga (ko'pincha "bo'sh"/0/qora)
<code>unset</code>	Meros bo'lsa otaga, bo'lmasa initial'ga
<code>revert</code>	Brauzerning (user-agent) standartiga

10.10 Amaliyot: specificity urushlaridan qanday qochish kerak?

Endi nazariyani amaliy hikmatga aylantiraylik. "Specificity urushi" — bu kodda selektorlar tobora kuchliroq bo'lib boravergan holat: kimdir `.tugma` yozadi, keyin uni bosish uchun boshqa kimdir `.panel .tugma` yozadi, keyin `#sahifa .panel .tugma ...` oxir-oqibat hech kim hech narsani o'zgartira olmay qoladi. Mana qanday qochish kerak:

1. Past va tekis specificity ishlat. Imkon qadar **bitta class** bilan ishla.

`.karta__sarlavha` kabi class `header .karta h2` kabi uzun zanjirdan ancha yaxshi —

chunki uni keyinroq oson bosib o'tasan.

```
/* Yomon – yuqori, qattiq specificity (0,1,2,1) */
#sahifa .panel .tugma span { color: white; }

/* Yaxshi – past, tekis specificity (0,0,1,0) */
.tugma__matn { color: white; }
```

2. id ni stil uchun ishlatma. #id ning specificity'si juda yuqori (0,1,0,0) va uni faqat boshqa id yoki !important bosa oladi. ID'ni JavaScript va anchor (#bo'lim) uchun qoldir, stilni class bilan ber.

3. !important dan qoch. 10.3 da ko'rganimizdek, u keyin "important urushi" ga olib keladi. Faqat tashqi kutubxonani majburan o'zgartirishda, oxirgi chora sifatida ishlat.

4. Manba tartibidan foydalan. Teng specificity'da oxirgi qoida yutadi. Shuning uchun o'z stillaringni to'g'ri tartibda yozsang, specificity'ni oshirmasdan ham g'olib chiqasan. **O'z faylingni har doim oxirida ula.**

5. :where() bilan specificity'ni nolga tushir. Agar qayta yozish oson bo'lishini istasang, :where() ichidagi hamma narsa specificity'ga 0 qo'shadi:

```
:where(.menu) a { color: blue; } /* bali (0,0,0,0) – keyin osongina
bosiladi */
```

💡 **Oltin qoida:** "selektorni mumkin qadar past va tekis tut". Specificity qanchalik past bo'lsa, kodingni boshqarish shunchalik oson — kelajakdagi o'zing senga rahmat aytadi.

Mashqlar

Quyidagi mashqlarni qog'ozda yoki brauzerda (Chrome DevTools'da) sinab ko'r. Avval o'zing javob ber, keyin yechimni och.

Mashq 1 — Ball sanash

Quyidagi selektorlarning specificity balini (a, b, c, d) ko'rinishida yoz:

```
ul li a
.nav .active
#footer p
input[type="text"]:focus
```



Yechim



```
ul li a      → (0,0,0,3) /* 3 ta element */
.nav .active → (0,0,2,0) /* 2 ta class */
#footer p    → (0,1,0,1) /* 1 id + 1 element */
input[type="text"]:focus → (0,0,2,1) /* 1 element + 1 attr + 1 pseudo-class */
```

Mashq 2 — Kim yutadi?

Quyidagi kodda matn qaysi rangda bo'ladi? Nega?

```
p { color: red; }
.matn { color: blue; }
```

```
<p class="matn">Salom</p>
```



Yechim



Matn **ko'k** (blue) bo'ladi. `.matn` ning bali `(0,0,1,0)`, `p` ning bali `(0,0,0,1)`. Class ustunida `1 > 0`, shuning uchun `.matn` g'olib chiqadi. Manba tartibi (kim oldin yozilgani) bu yerda ahamiyatga ega emas, chunki specificity allaqachon farqni hal qildi.

Mashq 3 — `!important` tuzog'i

Bu kodda fon qaysi rangda bo'ladi?

```
.tugma { background: green !important; }
.tugma { background: red; }
```



Yechim



Fon **yashil** (green) bo'ladi. `red` keyin yozilgan bo'lsa ham, `green` da `!important` bor — u muhimlik bosqichida g'olib chiqadi va cascade specificity yoki tartibgacha yetib ham bormaydi.

⚠ Bu — `!important` ning xavfini ko'rsatadigan misol: keyinroq fonni o'zgartirmoqchi bo'lsang, yana `!important` qo'shishga majbur bo'lasan.

Mashq 4 — Meros bo'ladimi?

Quyidagilardan qaysilari `<body>` dan farzand elementlarga meros bo'ladi: `color`, `margin`, `font-size`, `border`, `text-align`, `width`?

Yechim

Meros bo'ladi: `color`, `font-size`, `text-align` (matn/tipografiya xossalari).

Meros bo'lmaydi: `margin`, `border`, `width` (box/tartib xossalari).

Qoida: matn bilan bog'liq xossalar meros bo'ladi, vizual o'lcham va tartib xossalari yo'q — chunki har element o'z o'lchamini o'zi nazorat qilishi kerak.

Mashq 5 — To'liq cascade

Bu kodda matn qaysi rangda bo'ladi? Qadamma-qadam tushuntir.

```
p { color: green; }
.maqola p { color: blue; }
.maqola p { color: purple; }
```

```
<div class="maqola"><p>Test</p></div>
```

Yechim

Matn **binafsha** (`purple`) bo'ladi.

1. **Muhimlik:** hech birida `!important` yo'q, hammasi `author` — teng.
2. **Specificity:** `p` = $(0, 0, 0, 1)$, `.maqola p` = $(0, 0, 1, 1)$. So'nggi ikkalasi tengroq va kuchliroq, `green` tushib qoladi.
3. **Manba tartibi:** ikkita `.maqola p` teng $(0, 0, 1, 1)$ — shuning uchun oxirgi yozilgan, ya'ni `purple` yutadi.

Mashq 6 — `inherit` bilan tuzatish

`.quticha` ning ichidagi `.ichki` element otasining `border` ini olishini xohlaysan, lekin `border` normalda meros bo'lmaydi. Qanday qilib majburlaysan?



Yechim



`inherit` kalit so'zi bilan:

```
.quticha { border: 2px solid navy; }  
.quticha .ichki {  
  border: inherit; /* majburan otadan border'ni oladi */  
}
```



`inherit` har qanday xossani — meros bo'lmaydiganini ham — otadan majburan oldiradi.

Mashq 7 — Specificity urushini tuzatish

Quyidagi selektor juda yuqori specificity'ga ega va keyinroq boshqarish qiyin. Uni **past va tekis** qilib qayta yoz:

```
#sahifa .panel ul li a.havola { color: teal; }
```



Yechim



Bitta mazmunli class bilan almashtir (BEM uslubida):

```
.panel__havola { color: teal; } /* bali (0,0,1,0) — past va oson bosiladi */
```



Eski bali $(0, 1, 2, 3)$ edi — endi $(0, 0, 1, 0)$. Endi bu qoidani keyinroq boshqa bir class bilan osongina bosib o'tish mumkin, `!important` ga ehtiyoj yo'q.

Mashq 8 — `unset` xulq-atvori

`.x` elementga `color: unset` va `margin: unset` berildi. Har biri qanday natija beradi? (Ota element `color: navy`, `margin: 20px` bergan.)



Yechim



- `color: unset` → `color` **meros bo'ladigan** xossa, shuning uchun `inherit` kabi ishlaydi → otadan `navy` ni oladi.
- `margin: unset` → `margin` **meros bo'lmaydigan** xossa, shuning uchun `initial` kabi ishlaydi → standart `0` ga qaytadi (otaning `20px` ini olmaydi).

`unset` "aqlli": xossa turiga qarab `inherit` yoki `initial` kabi yo'l tutadi.

11 — Box model

[← Oldingi: 10 — Cascade, specificity va inheritance](#) · [🏠 README](#) · [Keyingi: 12 — O'lchovlar, tipografiya va ranglar](#) [→](#)

Bu bobda: har bir HTML element aslida to'rt qatlamli quti ekanini (content, padding, border, margin) o'rganib, ularning o'lchami qanday hisoblanishini (`box-sizing`), `margin / padding / border` qisqa yozuvlarini, margin'lar birlashishini (margin collapse), `display` turlarini (block/inline/inline-block) va `overflow` ni to'liq tushunib olasiz.

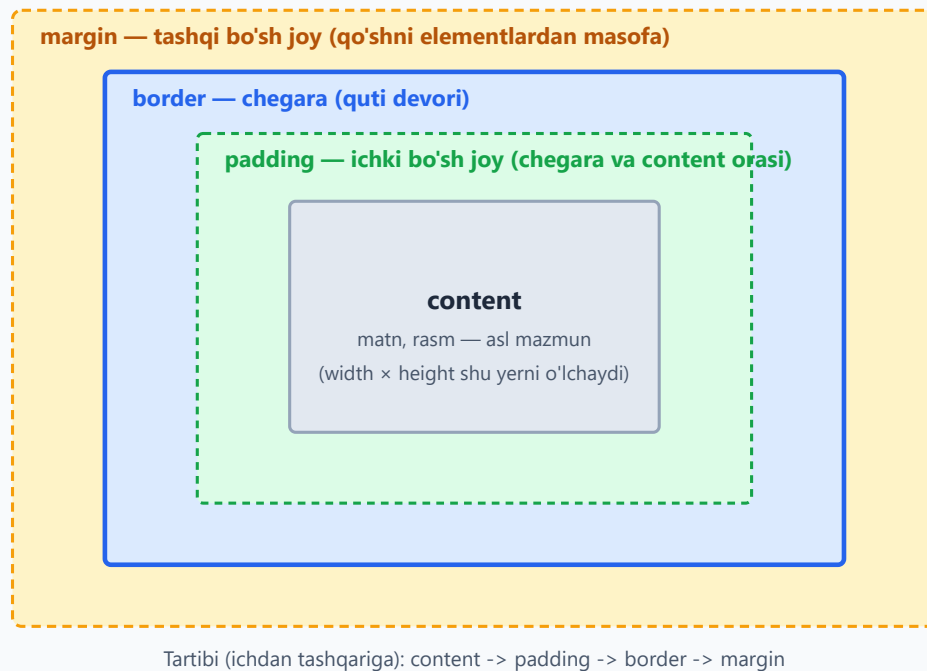
11.1 Asosiy g'oya: har element bir quti

CSS dunyosida eng muhim tushunchalardan biri shu: **brauzer sahifadagi har bir elementni to'rtburchak quti deb ko'radi**. Sarlavha ham, paragraf ham, rasm ham, hatto bitta so'z ham — hammasi quti. Sahifani joylashtirish (layout) — bu shu qutilarni bir-biriga nisbatan tartibga solishdir.

Har bir quti to'rtta qatlamdan iborat. Ichdan tashqariga qarab:

1. **content** (mazmun) — eng ichkaridagi qatlam: matn, rasm yoki boshqa mazmunning o'zi.
2. **padding** (ichki masofa) — content bilan chegara orasidagi bo'sh joy.
3. **border** (chegara) — qutining "devori", ko'rinadigan chiziq.
4. **margin** (tashqi masofa) — quti bilan qo'shni qutilar orasidagi bo'sh joy.

Box model: har element to'rt qatlamdan iborat quti



💡 **Analogiya:** Devorga osilgan rasmni tasavvur qiling. **content** — rasmning o'zi (surat). **padding** — surat bilan ramka orasidagi oq passe-partout (ichki hoshiya). **border** — yog'och ramka. **margin** — bu rasm bilan yonidagi boshqa rasm orasidagi devordagi bo'sh joy. Aynan shu to'rt qatlam har bir HTML elementda mavjud.

Nega buni avval o'rganamiz? Chunki CSS da masofa, o'lcham va joylashuv bilan bog'liq deyarli har bir muammoning ildizi shu to'rt qatlamda. "Nega bu element kattaroq chiqdi?", "Nega oralarida bo'sh joy bor?", "Nega tugma joyiga sig'maydi?" — bularning javobi har doim box model.

✂️ Brauzerda istalgan elementni o'ng tugma bilan bosib "Inspect" (tekshirish) ni tanlasangiz, DevTools'ning pastki qismida aynan shu to'rt qatlamli rangli diagrammani ko'rasiz. Bu — sizning eng yaxshi do'stingiz.

11.2 content: mazmunning o'zi

content — qutining yuragi. Bu yerda elementning haqiqiy mazmuni yashaydi: paragrafdagi matn, `<img>` dagi rasm, tugmadagi yozuv.

Content sohasining o'lchamini ikkita asosiy property boshqaradi:

- `width` — content kengligi (eni).
- `height` — content balandligi (bo'yi).

```
.quti {  
  width: 300px;  
  height: 150px;  
}
```

Bu yerda muhim "nega" bor: **standart holatda width va height faqat content qatlamini o'lchaydi**, padding va border'ni emas. Ya'ni yuqoridagi quti content jihatidan 300px, ammo unga padding va border qo'shsangiz, ekrandagi *haqiqiy* o'lchami kattaroq bo'ladi. Bu ko'p boshlovchini chalg'itadi — buni 11.6 (box-sizing) bo'limida batafsil hal qilamiz.

✦ Aksariyat block elementlarda `width` ni yozmasangiz, u avtomatik ravishda ota-element (parent) eniga **to'liq cho'ziladi**. `height` ni yozmasangiz esa, u mazmunga qarab o'sadi (content qancha bo'lsa, shuncha baland). Shuning uchun ko'pincha `height` ni qo'lda yozmaslik xavfsizroq — matn ko'paysa, quti o'zi o'sadi.

11.3 padding: ichki bo'sh joy

padding — content bilan border orasidagi ichki bo'sh joy. U content'ni chegaradan "itarib" turadi, mazmunga "nafas olish" maydoni beradi.

Padding'ning eng muhim xususiyati: u **elementning fon rangini (background) oladi**. Ya'ni padding qo'shsangiz, fon ham kengayadi. Bu uni margin'dan ajratib turadigan asosiy farq (margin har doim shaffof).

Padding'ni har tomondan alohida boshqarish mumkin:

```
.karta {  
  padding-top: 10px;  
  padding-right: 20px;  
  padding-bottom: 10px;  
  padding-left: 20px;  
}
```

Lekin har doim to'rt qatorni yozish zerikarli. Shuning uchun **qisqa yozuv (shorthand)** mavjud — bitta `padding` property orqali. Qiymatlar soniga qarab ma'nosi o'zgaradi:

```
.a { padding: 20px; } /* 4 ta tomon ham 20px */  
.b { padding: 10px 20px; } /* yuqori/past 10px, chap/o'ng 20px */  
.c { padding: 10px 20px 30px; } /* yuqori 10, chap/o'ng 20, past 30 */  
.d { padding: 10px 20px 30px 40px; } /* yuqori, o'ng, past, chap */
```

Tomonlar tartibini eslab qolishning oson yo'li: **soat strelkasi bo'yicha**, yuqoridan boshlab — yuqori, o'ng, past, chap (ingliz tilida "TRouBLe" — Top, Right, Bottom, Left).

💡 **Ikki qiymatli yozuvni eslab qolish:** `padding: 10px 20px` da birinchi son — *vertikal* (yuqori va past), ikkinchi son — *gorizontal* (chap va o'ng). Bu eng ko'p ishlatiladigan ko'rinish: tugmalarda odatda yon padding tepa-pastdagidan kattaroq bo'ladi.

```
button {  
  padding: 8px 16px; /* tugmaga klassik ko'rinish beradi */  
}
```

11.4 border: chegara (quti devori)

border — content va padding atrofidagi ko'rinadigan chiziq, qutining "devori". Border'ning uchta asosiy xususiyati bor va ularning **uchalasi ham yozilishi kerak** (ayniqsa `style` siz border ko'rinmaydi):

- `border-width` — chiziq qalinligi (masalan `2px`).
- `border-style` — chiziq turi (`solid`, `dashed`, `dotted` va h.k.).
- `border-color` — chiziq rangi.

To'liq yozuv:

```
.quti {  
  border-width: 2px;  
  border-style: solid;  
  border-color: #2563eb;  
}
```

Bu juda uzun. Shuning uchun deyarli har doim **qisqa yozuv** ishlatiladi — uchala qiymatni bitta qatorda berasiz:

```
.quti {  
  border: 2px solid #2563eb; /* qalinlik · turi · rang */  
}
```

⚠️ **Eng ko'p uchraydigan xato:** `border: 2px #2563eb;` deb `style` ni tushirib qoldirish. `border-style` ning standart qiymati `none` — ya'ni `style` bo'lmasa, **border umuman ko'rinmaydi**, qalinligi va rangi qancha bo'lsa ham. Eng muhim qism — `solid` kabi style.

Eng ko'p ishlatiladigan border style'lar:

Qiymat	Ko'rinishi
<code>solid</code>	to'liq, uzluksiz chiziq (eng keng tarqalgan)
<code>dashed</code>	tirelar (chiziqchalar)
<code>dotted</code>	nuqtalar
<code>double</code>	ikki parallel chiziq
<code>none</code>	border yo'q (standart)

Faqat bitta tomonga border qo'yish ham mumkin:

```
.ajratuvchi {
  border-bottom: 1px solid #94a3b8; /* faqat pastki chiziq */
}
```

Va burchaklarni yumaloqlash uchun `border-radius`:

```
.karta {
  border: 1px solid #94a3b8;
  border-radius: 8px; /* yumshoq, zamonaviy burchaklar */
}
```

💡 `border-radius: 50%` ni kvadrat (eni = bo'yi) elementga bersangiz — to'liq **doira** chiqadi. Bu avatarka (profil rasmi) ni doira qilishning klassik usuli.

11.5 margin: tashqi bo'sh joy va markazlash

margin — qutining tashqi qatlami: bu quti bilan **qo'shni qutilar** orasidagi bo'sh joy. U elementlarni bir-biridan uzoqlashtiradi.

Margin'ning padding'dan asosiy farqi: **margin har doim shaffof** — uning foni yo'q. U shunchaki "bo'shliq" yaratadi.

Sintaksis padding bilan **bir xil** — qisqa yozuv ham aynan o'sha qoidalarga bo'ysunadi:

```
.a { margin: 20px; } /* 4 tomon ham 20px */
.b { margin: 10px 20px; } /* vertikal 10px, gorizontal 20px */
.c { margin: 10px 20px 30px 40px; } /* yuqori, o'ng, past, chap */
```

Alohida tomonlar: `margin-top`, `margin-right`, `margin-bottom`, `margin-left`.

margin: auto bilan markazlash

Margin'ning eng foydali hiylasi — **blok elementni gorizontal markazlash**. Buning uchun elementga aniq `width` berib, chap va o'ng margin'ni `auto` qilasiz:

```
.markaz {  
  width: 600px;  
  margin: 0 auto;    /* vertikal 0, gorizontal auto */  
}
```

Nega bu ishlaydi? `auto` brauzerga "qolgan bo'sh joyni teng taqsimla" deydi. Element 600px, ota-element undan keng bo'lsa, ortgan joy chap va o'ngga teng bo'linadi — natijada element o'rtada turadi.

⚠ `margin: 0 auto` markazlash **faqat `width` berilgan block elementda** ishlaydi. Agar `width` bermasangiz, element allaqachon to'liq enni egallaydi — markazlash uchun "ortiqcha joy" qolmaydi. Shuningdek, bu hiyla vertikal markazlashda ishlamaydi (vertikal markazlash uchun keyingi boblarda flexbox'ni ko'ramiz).

💡 Manfiy margin ham mumkin (`margin-top: -10px`) — u elementni qo'shni element ustiga "tortadi". Bu kuchli, lekin chalkash usul; boshda undan qochgan ma'qul.

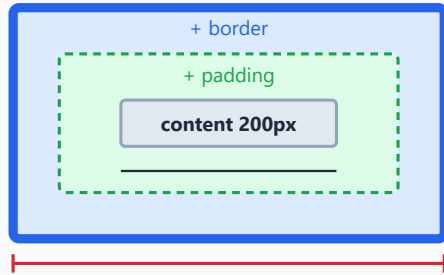
11.6 box-sizing: content-box va border-box

Endi 11.2 da aytilgan muammoga qaytamiz. Savol: agar elementga `width: 200px` bersam, ekranda u **aynan 200px** bo'ladimi?

Standart holatda — **yo'q**. Buning sababi `box-sizing` degan property bo'lib, uning standart qiymati `content-box`.

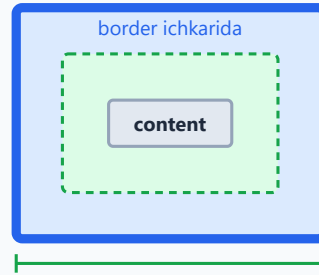
box-sizing: ikkalasida ham width: 200px

content-box (default)



jami eni = 260px
 $200 + 20 \cdot 2 + 10 \cdot 2$

border-box (tavsiya)



jami eni = 200px
padding va border ICHIGA kiradi

Nega border-box afzal?

content-box da padding/border qo'shsangiz quti KENGAYIB ketadi — layout buziladi. border-box da yozgan width o'sha-o'sha qoladi. Shuning uchun ko'pchilik * { box-sizing: border-box } yozadi.

content-box (standart)

`box-sizing: content-box` da `width` va `height` faqat **content** qatlamini o'lchaydi. Padding va border esa shunga **qo'shiladi**:

```
.quti {  
  box-sizing: content-box; /* standart */  
  width: 200px;  
  padding: 20px;  
  border: 10px solid #2563eb;  
}
```

Ekrandagi **haqiqiy eni**:

$200\text{px} (\text{content}) + 20\text{px} \cdot 2 (\text{padding}) + 10\text{px} \cdot 2 (\text{border}) = 260\text{px}$

Ya'ni `200px` so'radingsiz, lekin `260px` oldingiz! Bu layout'ni kutilmaganda buzadi.

border-box (tavsiya etiladi)

`box-sizing: border-box` da `width` **butun qutini** o'lchaydi — padding va border shu o'lcham **ichiga kiradi**:

```
.quti {  
  box-sizing: border-box;  
  width: 200px;  
}
```

```
padding: 20px;
border: 10px solid #2563eb;
}
```

Ekrandagi **haqiqiy eni**:

200px – aynan o'zi. Content avtomatik kichrayadi: $200 - 20 \cdot 2 - 10 \cdot 2 = 140\text{px}$

Nega border-box afzal? Chunki u tabiiyroq: yozgan `width` ekrandagi o'lcham bilan teng bo'ladi. Padding yoki border qo'shganda quti kengayib ketmaydi, layout buzilmaydi. Hisob-kitob soddalashadi.

Aynan shu sababdan deyarli har bir zamonaviy loyiha CSS faylining boshida shu qoidani yozadi:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

`*` (universal selector) "barcha elementlar" degani. Bu bir qatorda butun saytni `border-box` ga o'tkazadi.

💡 Buni "bir marta yozib qo'yiladigan va keyin unutiladigan" sozlama deb biling. Tajribali dasturchilarning aksariyati har yangi loyihada shu uch qatorni avtomatik yozadi.

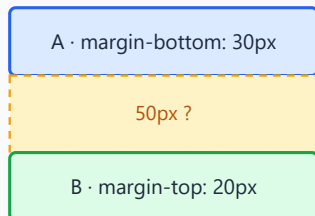
11.7 Margin collapse: vertikal margin'lar birlashishi

Bu — boshlovchilarni eng ko'p hayron qoldiradigan mavzulardan biri. Diqqat bilan ko'ring.

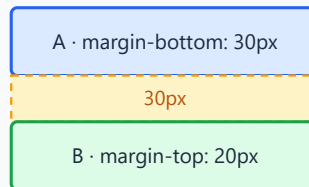
Tasavvur qiling: ikkita paragraf bir-birining ostida turibdi. Birinchisida `margin-bottom: 30px`, ikkinchisida `margin-top: 20px`. Mantiqan ular orasidagi masofa $30 + 20 = 50\text{px}$ bo'lishi kerakdek. Lekin **aslida 30px** chiqadi!

Margin collapse — vertikal margin'lar birlashishi

Kutilgan (noto'g'ri): $30 + 20 = 50\text{px}$



Haqiqat: $\max(30, 20) = 30\text{px}$



Qoida va "nega"

Yonma-yon turgan ikki blokning VERTIKAL margin'lari qo'shilmaydi — faqat KATTAROG'I qoladi (birlashadi = collapse).

Nega: matn oqimida paragraflar orasi ikki barobar kengaymasligi uchun.

Faqat vertikal (yuqori/past) margin'ga taalluqli — gorizonttal birlashmaydi.
padding, border yoki flex/grid orada bo'lsa — birlashmaydi.

Bu hodisa **margin collapse** (margin'lar birlashishi) deb ataladi: yonma-yon turgan ikki block elementning **vertikal** (yuqori va past) margin'lari qo'shilmaydi — ularning faqat **kattarog'i** qoladi.

```
<p class="birinchi">Birinchi paragraf</p>
<p class="ikkinchi">Ikkinchi paragraf</p>
```

```
.birinchi { margin-bottom: 30px; }
.ikkinchi { margin-top: 20px; }
/* Ular orasidagi masofa: max(30, 20) = 30px, 50px EMAS */
```

Nega CSS bunday qiladi? Tarixiy sabab matn hujjatlaridan keladi. Agar har bir paragraf tepa va pastdan margin bersa va ular qo'shilib ketsa, paragraflar orasidagi masofa ikki barobar ortib ketardi va matn siyrak ko'rinardi. Birlashish (collapse) bu masofani bir tekis va kutilgan holda saqlaydi.

Muhim cheklovlar

Margin collapse **faqat** quyidagi shartlarda yuz beradi:

- Faqat **vertikal** margin'larda (yuqori/past). Gorizonttal (chap/o'ng) margin'lar **hech qachon birlashmaydi** — har doim qo'shiladi.
- Faqat normal oqimdagi (normal flow) block elementlarda.

Birlashishni **to'xtatadigan** narsalar (orada to'siq bo'lsa, margin'lar birlashmaydi):

- Ikki margin orasida **border** yoki **padding** bo'lsa.

- Ota-element `display: flex` yoki `display: grid` bo'lsa (flex/grid bolalarida margin collapse yo'q).
- `overflow` qiymati `visible` dan boshqa bo'lsa.

⚠ Bola element margin'i ota-element margin'i bilan ham birlashishi mumkin (masalan, birinchi bolaning `margin-top` i ota'ning `margin-top` i bilan "qochib chiqadi"). Agar ota-elementga `padding-top` yoki `border-top` qo'shsangiz, bu hodisa to'xtaydi. Bu — "nega blok ichidagi margin tashqariga oqib ketdi?" degan jumboqning yechimi.

11.8 display: block, inline va inline-block

Elementlar qutidek joylashadi, dedik — lekin **qanday** joylashadi? Buni `display` property boshqaradi. Eng asosiy uch qiymatni ko'rib chiqamiz.

block · inline · inline-block

block (masalan div, p, h1)

block element 1 — butun qatorni egallaydi

block element 2 — yangi qatordan boshlanadi

width/height ishlaydi · har biri yangi qatorga tushadi

inline (masalan span, a, strong)

inline 1

inline 2

inline 3

yonma-yon turadi · width/height TA'SIR QILMAYDI

inline-block (yonma-yon + o'lcham)

box 1

box 2

box 3

yonma-yon
+ width/height

block

block elementlar to'liq bitta qatorni egallaydi va har biri **yangi qatordan** boshlanadi (xuddi matndagi yangi xatboshi kabi). Standart block elementlar: `<div>`, `<p>`, `<h1>` — `<h6>`, `<ul>`, `<li>`, `<section>`.

- `width` va `height` ishlaydi (qo'lda berish mumkin).

- `width` bermasangiz — to'liq enni egallaydi.
- Yuqori/pastki margin ham, padding ham to'liq ishlaydi.

inline

inline elementlar matn oqimi ichida **yonma-yon** turadi, yangi qator boshlamaydi.

Standart inline elementlar: `<span>`, `<a>`, `<strong>`, `<em>`.

✦ **Istisno — replaced elementlar:** `<img>`, `<video>`, `<input>` ham inline joylashadi, lekin ular *replaced* (almashtirilgan) element — ularda quyidagidan farqli ravishda `width / height` **ishlaydi**. Pastdagi qoidalar oddiy (matnli) inline elementlarga tegishli.

- `width` va `height` **ta'sir qilmaydi** — element mazmuni qanchaga sig'sa, shuncha joy oladi.
- Yuqori/pastki margin va padding **layout'ga to'liq ta'sir qilmaydi** (vizual padding ko'rinishi mumkin, lekin qo'shni qatorlarni surmaydi).
- Faqat chap/o'ng (gorizontal) margin va padding qatorni suradi.

```
<p>Bu matnda <strong>qalin so'z</strong> va <a href="#">havola</a>
yonma-yon.</p>
```

✦ `<strong>` ga `width: 200px` bersangiz — hech narsa o'zgarmaydi, chunki u inline. Bu boshlovchilarni ko'p chalg'itadi: "nega width ishlamayapti?" Javob: element inline.

inline-block

inline-block — ikkalasining "eng yaxshi" tomonini birlashtiradi:

- inline kabi **yonma-yon** turadi (yangi qator boshlamaydi).
- block kabi `width`, `height`, vertikal margin va padding'ni **qabul qiladi**.

Bu navbar tugmalari, "karta" (card) qatorlari kabi yonma-yon turishi kerak bo'lgan, ammo o'lchami nazorat qilinadigan elementlar uchun klassik yechim:

```
.tugma {
  display: inline-block;
  width: 120px;
  padding: 10px;
  margin: 5px;
}
```

Xususiyat	block	inline	inline-block
Yangi qatordan boshlanadi	Ha	Yo'q	Yo'q
width / height ishlaydi	Ha	Yo'q	Ha
Vertikal margin/padding qatorga ta'sir qiladi	Ha	Yo'q	Ha

💡 Bugungi kunda ko'plab yonma-yon joylashuvlar uchun `inline-block` o'rniga **flexbox** yoki **grid** ishlatiladi (keyingi boblarda). Lekin `inline-block` hamon foydali va `display` turlari farqini tushunish layout asoslarining poydevori.

11.9 overflow: mazmun sig'masa nima bo'ladi?

Ba'zan content qutiga **sig'maydi** — masalan, `height` ni belgilab qo'ydingiz, ammo matn undan ko'p. Bunda nima bo'lishini `overflow` property hal qiladi.

```
.quti {
  width: 200px;
  height: 100px;
  overflow: auto;
}
```

To'rtta asosiy qiymat:

Qiymat	Xatti-harakat
<code>visible</code>	Standart. Ortiqcha mazmun qutidan tashqariga tashib ko'rinaveradi.
<code>hidden</code>	Ortiqcha mazmun kesib tashlanadi (ko'rinmaydi), scroll yo'q.
<code>scroll</code>	Har doim scrollbar chiqadi (mazmun sig'sa ham).
<code>auto</code>	Scrollbar faqat kerak bo'lganda (mazmun sig'masada) chiqadi.

Nega `visible` standart? Chunki CSS mazmunni hech qachon "yo'qotmaslik" tarafdori — sukut bo'yicha hammasini ko'rsatadi, hatto bu qutidan tashib chiqsa ham. Mazmunni yashirish — bu sizning ataylab qiladigan tanlovingiz.

💡 **auto vs scroll** : Amalda deyarli har doim **auto** afzal — chunki **scroll** mazmun kam bo'lsa ham bo'sh, xunuk scrollbar ko'rsatadi. **auto** esa faqat zarur bo'lganda chiqaradi.

Gorizontal va vertikalni alohida boshqarish ham mumkin: **overflow-x** (gorizontal) va **overflow-y** (vertikal).

⚠️ **overflow: hidden** mazmunni jimgina kesib tashlaydi — foydalanuvchi uni ko'ra olmaydi va scroll ham qila olmaydi. Agar matn tasodifan kesilib qolsa, sababini shu yerdan qidiring.

11.10 width nazorati: min-width va max-width

Oddiy **width** aniq, qat'iy o'lcham beradi. Lekin zamonaviy, ekranga moslashadigan (responsive) saytlarda ko'pincha **chegaralangan** o'lcham kerak bo'ladi. Buning uchun uchta property bor:

- **width** — afzal ko'rilgan (maqsadli) eni.
- **min-width** — **eng kichik** ruxsat etilgan eni (bundan kichrayolmaydi).
- **max-width** — **eng katta** ruxsat etilgan eni (bundan kattalashmaydi).

Eng ko'p ishlatiladigan naqsh:

```
.kontent {  
  width: 100%;           /* iloji boricha to'liq enni egalla */  
  max-width: 800px;      /* lekin 800px dan oshma */  
  margin: 0 auto;        /* va markazda tur */  
}
```

Nega bu juda foydali? Bu element kichik ekranlarda (telefon) butun enni egallaydi, katta ekranlarda (monitor) esa 800px da to'xtaydi va markazda turadi. Natijada matn juda keng, o'qishga noqulay qatorlar bo'lib ketmaydi. Bu — responsive dizaynning eng asosiy hiylalaridan biri.

min-width esa elementni juda kichrayib, mazmun siqilib qolishidan saqlaydi:

```
.yon-panel {  
  min-width: 250px;      /* hech qachon 250px dan ingichka bo'lmasin */  
}
```

✳️ **height** uchun ham **min-height** va **max-height** bor va xuddi shunday ishlaydi. Amalda **min-height** ko'proq ishlatiladi (masalan **min-height: 100vh** — sahifa butun ekran balandligida bo'lsin).

💡 **Ustuvorlik:** agar `width` va `max-width` zid bo'lsa, `max-width` g'olib chiqadi. Tartib: `min-width` har narsadan kuchli (oxirgi so'z uniki), keyin `max-width`, keyin `width`. Ya'ni element hech qachon `min-width` dan kichik bo'lmaydi, hatto `max-width` past bo'lsa ham.

Mashqlar

Quyidagi mashqlarni bajarib, box model'ni qo'lingiz bilan his qiling. Har biri uchun yangi `.html` fayl yarating, brauzerda oching va DevTools'dagi box diagrammasi bilan solishtiring.

Mashq 1 — Haqiqiy o'lchamni hisoblang

Quyidagi quti `content-box` (standart) rejimida. Uning ekrandagi **haqiqiy eni** necha piksel?

```
.quti {  
  width: 300px;  
  padding: 25px;  
  border: 5px solid black;  
}
```

Yechim

`content-box` da `width` faqat content. Padding va border qo'shiladi:

$$300 \text{ (content)} + 25 \cdot 2 \text{ (padding)} + 5 \cdot 2 \text{ (border)} = 360\text{px}$$

Haqiqiy eni — **360px**. Agar `box-sizing: border-box` bo'lganida, haqiqiy eni aynan **300px** bo'lar, content esa 240px ga kichrayar edi.

Mashq 2 — border-box ga o'tkazing

Bir loyiha boshlayapsiz. Har bir elementga `box-sizing: border-box` ni qo'llaydigan bitta CSS qoidasini yozing.



Yechim



```
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

`*` — barcha elementlar. `::before` va `::after` — psevd-elementlar (keyingi boblarda batafsil), ularni ham qamrab olamiz. Bu qatorni har yangi loyiha boshida yozish odat.

Mashq 3 — Qisqa yozuvni o'qing

Quyidagi har bir e'lon to'rt tomonga qaysi qiymatni beradi? (yuqori / o'ng / past / chap)

```
.a { padding: 15px; }  
.b { padding: 10px 30px; }  
.c { padding: 5px 10px 15px 20px; }
```



Yechim



- `.a` — yuqori **15**, o'ng **15**, past **15**, chap **15** (bitta qiymat = 4 tomon).
- `.b` — yuqori **10**, o'ng **30**, past **10**, chap **30** (vertikal / gorizontal).
- `.c` — yuqori **5**, o'ng **10**, past **15**, chap **20** (soat strelkasi bo'yicha).

Mashq 4 — Blokni markazlang

Eni 500px bo'lgan `<div class="karta">` ni sahifa o'rtasiga (gorizontal markaz) joylashtiring.



Yechim



```
.karta {  
  width: 500px;  
  margin: 0 auto;  
}
```

`width` berilishi shart — aks holda div to'liq enni egallaydi va markazlash uchun bo'sh joy qolmaydi. `margin: 0 auto` da vertikal margin 0, gorizontal `auto` (qolgan joyni teng bo'ladi).

Mashq 5 — Margin collapse jumbog'i

Quyidagi ikki paragraf orasidagi masofa necha piksel?

```
.p1 { margin-bottom: 40px; }  
.p2 { margin-top: 25px; }
```

Yechim

40px. Vertikal margin'lar birlashadi (collapse): qo'shilmaydi ($40 + 25 = 65$ EMAS), faqat kattarog'i — $\max(40, 25) = 40\text{px}$ — qoladi. Agar orada border yoki padding bo'lganida, ular birlashmasdi va masofa 65px bo'lardi.

Mashq 6 — Border nima uchun ko'rinmayapti?

Dasturchi border qo'ydi, lekin u ko'rinmayapti. Xatoni toping va tuzating.

```
.quti {  
  border: 3px #2563eb;  
}
```

Yechim

`border-style` tushib qolgan. `style` ning standart qiymati `none` — shuning uchun border ko'rinmaydi. To'g'risi:

```
.quti {  
  border: 3px solid #2563eb;  
}
```

`solid` (yoki `dashed`, `dotted` va h.k.) — border'ning eng muhim qismi. Qalinlik va rang yetarli emas.

Mashq 7 — display turini tanlang

Ucha tugmani **yonma-yon** joylashtirmoqchisiz, lekin har biriga aniq `width: 100px` va `padding` ham bermoqchisiz. Bu tugmalar `<span>` (inline) elementlari. Qaysi `display` qiymatini ishlatasiz va nega?

 Yechim

display: inline-block ni ishlatasiz.

```
.tugma {
  display: inline-block;
  width: 100px;
  padding: 10px;
}
```

Nega: oddiy inline (span'ning standarti) width va vertikal padding'ni qabul qilmaydi. block esa har birini yangi qatorga tushiradi — yonma-yon turmaydi. inline-block ikkala talabni ham bajaradi: yonma-yon turadi va o'lcham/padding'ni qabul qiladi.

Mashq 8 — Responsive kontent konteyneri

Matn juda keng monitorlarda o'qishga noqulay bo'lib ketmasligi uchun, kontentni: kichik ekranda to'liq engga, katta ekranda eng ko'pi 700px ga cheklab, doim markazda turadigan qilib yozing.

 Yechim

```
.kontent {
  width: 100%;
  max-width: 700px;
  margin: 0 auto;
  padding: 0 16px; /* chetlarga ozgina nafas — ekran tor bo'lsa matn
devorga yopishmaydi */
}
```

width: 100% telefonda to'liq enni egallaydi; max-width: 700px katta ekranda 700px da to'xtaydi; margin: 0 auto markazlaydi. Bu — eng ko'p ishlatiladigan responsive layout naqshlaridan biri.

Mashq 9 — overflow bilan scroll soha

Balandligi 150px bo'lgan, ichida ko'p matn turadigan "izohlar" qutisini yarating. Matn sig'masa, **faqat kerak bo'lganda** vertikal scroll chiqsin.



Yechim



```
.izohlar {  
  height: 150px;  
  overflow-y: auto;      /* faqat kerak bo'lsa vertikal scroll */  
  border: 1px solid #94a3b8;  
  padding: 12px;  
}
```

`auto` ni tanladik (`scroll` emas), chunki matn kam bo'lsa, bo'sh scrollbar chiqmaydi — faqat haqiqatan toshganda paydo bo'ladi. `overflow-y` faqat vertikalni boshqaradi.

[← Oldingi: 10 — Cascade, specificity va inheritance](#) · [🏠 README](#) · [Keyingi: 12 — O'lchovlar, tipografiya va ranglar →](#)

12 — O'lchovlar, tipografiya va ranglar

← Oldingi: 11 — Box model · 🏠 README · Keyingi: 13 — Fon, chegara va soyalar →

Bu bobda: CSS o'lchov birliklari (px, em, rem, %, vw/vh va boshqalar), rang modellari (nomli, hex, rgb, hsl, oklch) va matnni boshqaradigan tipografiya xossalari hamda web shriftlarni to'liq, "nega"si bilan o'rganamiz.

12.1 Nega aynan bu uchta mavzu birga?

Bu uch narsa — **o'lcham**, **rang** va **shrift** — har bir web sahifaning "ko'rinishini" tashkil qiladi. Element qanchalik katta? Qaysi rangda? Matn qanday shriftda va qancha kattalikda? Boshqa hamma narsa (joylashuv, animatsiya, responsiv dizayn) shu uch poydevorga quriladi.

Boshlovchilar ko'pincha shoshib `px` (piksel) bilan hamma narsani yozadi, ranglarni faqat `red` / `blue` deb beradi va `font-family` ni umuman tegmaydi. Natijada sayt har xil ekranda buziladi, ranglar palitrasi tartibsiz bo'ladi, matn esa "import qilingan" emas, "tashlangandek" ko'rinadi. Bu bob seni shu xatolardan qutqaradi va **professional tanlovlar** qilishni o'rgatadi.

✦ **Atamalar tarjimasini:** - **unit** — birlik (o'lchov birligi), masalan `px`, `em`. - **viewport** — ko'rinish maydoni, ya'ni brauzer oynasining ko'rinadigan qismi. - **typography** — tipografiya, ya'ni matnni bezash va joylashtirish san'ati. - **font** — shrift (harflar uslubi).

Keling, eng asosidan — o'lchov birliklaridan boshlaymiz.

12.2 Mutlaq vs nisbiy birliklar

CSS'da o'lchov beradigan har bir joyda (kenglik, balandlik, shrift o'lchami, masofa va h.k.) son yoniga **birlik** yozasan: `16px`, `2em`, `50%`. Birliklar ikki katta turga bo'linadi:

- **Mutlaq (absolute) birliklar** — qiymati har doim bir xil, hech narsaga bog'liq emas. Asosiysi: `px`.
- **Nisbiy (relative) birliklar** — qiymati **boshqa narsaga** nisbatan hisoblanadi (ota element shrifti, root shrifti, ekran o'lchami va h.k.). Masalan: `em`, `rem`, `%`, `vw`.

💡 **Hayotiy analogiya:** "10 metr" — mutlaq, qayerda aytsang ham bir xil. "Bo'yimning yarmi" — nisbiy, kimning bo'yi ekaniga bog'liq. CSS'da mutlaq birlik aniq va oldindan

ma'lum, nisbiy birlik esa moslashuvchan.

Nega nisbiy birliklar muhim? Chunki ular **moslashuvchanlik** (flexibility) beradi. Foydalanuvchi brauzerda shrift o'lchamini kattalashtirsa yoki sayt katta/kichik ekranda ochilsa, nisbiy birliklar avtomatik moslashadi. Mutlaq `px` esa qotib qoladi. Shuning uchun zamonaviy CSS'da nisbiy birliklar afzal — buni bob davomida bir necha bor ko'ramiz.

12.3 px — mutlaq piksel

`px` (piksel) — eng mashhur va eng tushunarli birlik. U ekrandagi bitta nuqtaga (taxminan) teng.

```
.quti {  
  width: 200px;  
  height: 100px;  
  font-size: 16px;  
  border: 1px solid black;  
}
```

Natija: quti aniq 200×100 piksel, matn 16 pikseli, chegara 1 pikseli.

`px` ning **kuchi** — to'liq nazorat va bashorat qilinishi. "1px chegara" har joyda 1px bo'ladi. Shu sababli ingichka chegaralar, kichik soyalar va aniq joylashuv kerak bo'lganda `px` ajoyib.

`px` ning **kamchiligi** — u moslashmaydi. Agar `font-size: 16px` desang va foydalanuvchi brauzer sozlamalarida "katta shrift" tanlagan bo'lsa (masalan ko'zi xira odam), sening matning **baribir 16px** qoladi. Bu — qulaylik (accessibility) muammosi.

⚠ **Tez-tez xato:** matn o'lchamlarini hamma joyda `px` bilan berish. Bu foydalanuvchining shrift kattalashtirish imkonini buzadi. Matn uchun `rem` ishlat (pastda ko'ramiz).

📌 **Eslatma:** CSS `px` — bu "CSS pikseli", jismoniy ekran pikseli emas. Yuqori zichlikdagi (Retina) ekranlarda bitta CSS px bir nechta jismoniy pikseldan iborat bo'lishi mumkin — brauzer buni o'zi hal qiladi, sen tashvishlanma.

12.4 em — ota elementning font-size'iga nisbatan

`em` — bu **nisbiy** birlik. `1em` = element o'zining (yoki ko'p hollarda otasining) `font-size` qiymatiga teng.

Asosiy qoida: - `font-size` xossasida `em` ishlatilsa → u **ota elementning** `font-size` 'iga nisbatan hisoblanadi. - Boshqa xossalarda (masalan `padding`, `margin`, `width`) `em` ishlatilsa → u **shu elementning** `font-size` 'iga nisbatan hisoblanadi.

```
.ota {  
  font-size: 20px;  
}  
.farzand {  
  font-size: 1.5em; /* 1.5 × 20px = 30px */  
  padding: 1em; /* 1 × 30px = 30px (o'zining font-size'i) */  
}
```

Natija: `.farzand` matni 30px, ichki bo'shliq (`padding`) ham 30px.

`em` ning kuchi — komponent ichida hamma narsa shrift o'lchamiga **mutanosib** o'sadi. Tugmaning `font-size` 'ini o'zgartirsang, uning `padding` 'i ham `em` da berilgan bo'lsa, avtomatik moslashadi. Bu tugmalar, beyjlar (badge) kabi "ichki nisbat saqlanishi kerak" bo'lgan komponentlar uchun zo'r.

em ning xavfi: ko'payib ketish (kompaundlanish)

`em` ning eng katta tuzog'i — **ichma-ich (nested) elementlarda** u qatlam-qatlam ko'payib ketadi. Har bir farzand otasining (allaqachon kattalashgan) o'lchamiga nisbatan hisoblaganligi uchun, qiymat tobora o'sib boradi:

```
<div class="daraja">24px (16×1.5)  
  <div class="daraja">36px (24×1.5)  
    <div class="daraja">54px (36×1.5)  
      <div class="daraja">81px! (54×1.5)</div>  
    </div>  
  </div>  
</div>
```

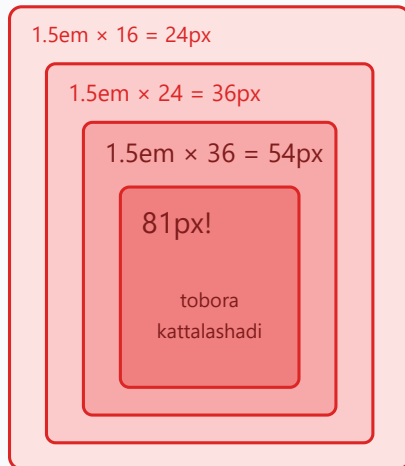
```
html { font-size: 16px; }  
.daraja { font-size: 1.5em; }
```

Natija: har daraja oldingisidan 1.5 baravar katta — 24 → 36 → 54 → 81px. Bu deyarli har doim **xato** — sen barcha `.daraja` larni bir xil kattalikda kutgan eding. Quyidagi diagramma buni `rem` bilan solishtirib ko'rsatadi:

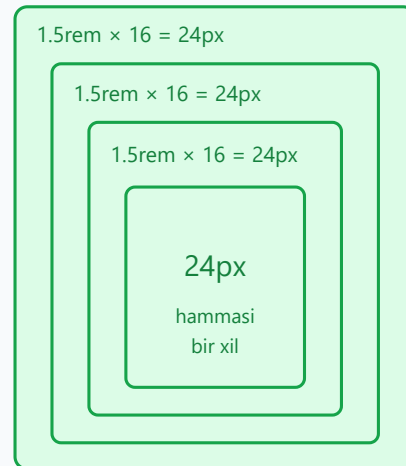
em vs rem — ichma-ich elementlarda nima bo'ladi?

html { font-size: 16px } · har element font-size: 1.5(em yoki rem)

em — otaga bog'liq (ko'payadi)



rem — root'ga bog'liq (barqaror)



em har bosqichda OTA o'lchamiga ko'payadi · rem doim ROOT (16px) ga bog'lanadi

Shuning uchun barqarorlik kerak bo'lsa rem afzal

12.5 rem — root font-size'iga nisbatan (va nega afzal)

rem = "root **em**". U doim **bitta** narsaga — hujjatning ildiz elementi `<html>` ning font-size 'iga nisbatan hisoblanadi. Ota element nima bo'lishidan **qat'i nazar**.

Brauzerda `<html>` ning standart font-size'i odatda **16px**. Demak:

```
html { font-size: 16px; } /* root: 16px (standart) */  
  
h1 { font-size: 2rem; } /* 2 × 16 = 32px */  
p { font-size: 1rem; } /* 1 × 16 = 16px */  
small { font-size: 0.875rem; } /* 0.875 × 16 = 14px */
```

Natija: bu qiymatlar elementlar qancha chuqur ichma-ich joylashganidan **mutlaqo bog'liq emas** — har doim root'ga nisbatan barqaror. Diagrammada ko'rinib turibdiki, rem da uchala daraja ham bir xil 24px qoladi, em da esa 81px'gacha ko'payib ketadi.

Nega rem shrift uchun afzal:

1. **Bashoratlilik.** Hech qachon "ko'payib ketish" yo'q. 1.5rem har joyda bir xil — kodni o'qish va boshqarish oson.
2. **Qulaylik (accessibility).** Foydalanuvchi brauzerda standart shrift o'lchamini kattalashtirsa, root o'zgaradi va butun sayt **mutanosib** kattalashadi. px da bu ishlamaydi.

3. **Bitta joydan boshqarish.** `html { font-size } ni o'zgartirib, butun saytning miqyosini birato'la o'zgartira olasan.`

💡 **Amaliy maslahat — "62.5% hiyla":** ba'zilar `html { font-size: 62.5%; }` yozadi. Buning sababi: $62.5\% \times 16\text{px} = 10\text{px}$. Endi $1\text{rem} = 10\text{px}$ bo'ladi va hisob-kitob osonlashadi — $1.6\text{rem} = 16\text{px}$, $2.4\text{rem} = 24\text{px}$. Bu mashhur, lekin majburiy emas; faqat hisobni soddalashtirish uchun.

✳️ **Qachon `em`, qachon `rem`?** - `rem` — global, barqaror narsalar uchun: shrift o'lchamlari, bo'limlar orasidagi masofa, layout o'lchamlari. - `em` — komponent ichida shriftga mutanosib o'sishi kerak bo'lgan narsalar uchun: tugma `padding`'i, ikonka o'lchami.

12.6 % — ota elementga nisbatan foiz

Foiz (%) ham nisbiy, lekin u **ota element**ning mos xossasiga nisbatan hisoblanadi. "Mos xossa" qaysi xossani berayotganingga bog'liq:

- `width: 50%` → ota elementning **kengligining** yarmi.
- `height: 50%` → ota elementning **balandligining** yarmi (lekin otaning aniq balandligi bo'lishi kerak).
- `font-size: 150%` → ota elementning **font-size**'ining 1.5 baravari (`em` kabi).

```
.ota {  
  width: 400px;  
}  
.farzand {  
  width: 50%;    /* 50% × 400px = 200px */  
}
```

Natija: `.farzand` kengligi 200px. Agar `.ota` keyinroq 600px bo'lsa, `.farzand` avtomatik 300px bo'ladi — bu % ning moslashuvchanligi.

⚠️ **Tez-tez xato:** `height: 100%` ishlamasligi. Foizli balandlik faqat ota elementning **aniq balandligi** bo'lganda ishlaydi. Aks holda brauzer "100% — nimaning 100%i?" deb hisoblay olmaydi va e'tiborsiz qoldiradi.

% asosan **layout** (joylashuv) — kenglik, ustun nisbatlari — uchun ishlatiladi. Shrift uchun `rem` aniqroq.

12.7 Viewport birliklar: vw, vh, vmin, vmax

Bu birliklar to'g'ridan-to'g'ri **viewport**'ga (brauzer oynasining ko'rinadigan qismiga) nisbatan hisoblanadi — ota elementga emas:

Birlik	Ma'nosi
1vw	viewport kengligi ning 1%
1vh	viewport balandligi ning 1%
1vmin	kenglik va balandlikdan kichigi ning 1%
1vmax	kenglik va balandlikdan kattasi ning 1%

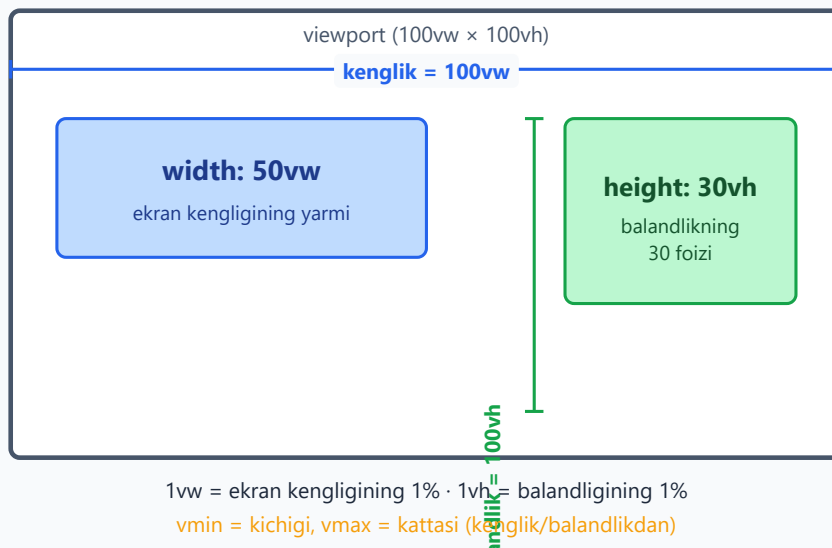
Masalan, ekran 1000px keng bo'lsa, `50vw = 500px`. Agar ekran 800px baland bo'lsa, `100vh = 800px`.

```
.qahramon-bolim {
  width: 100vw;    /* butun ekran kengligi */
  height: 100vh;  /* butun ekran balandligi - "to'liq ekran" bo'lim
  */
}
.katta-sarlavha {
  font-size: 5vw; /* ekran kengaysa sarlavha ham kattalashadi */
}
```

Natija: `.qahramon-bolim` har qanday ekranda to'liq ekranni egallaydi; sarlavha o'lchami ekran kengligiga qarab o'zgaradi. Quyidagi diagramma `vw` va `vh` ning ekranga nisbatini ko'rsatadi:

Viewport birliklar — ekranga nisbatan o'lcham

Viewport = brauzer oynasining ko'rinadigan qismi



💡 **vmin / vmax qachon kerak?** Agar element ekran **aylantirilganda** (portret ↔ landshaft) ham mutanosib qolishini istasang. Masalan kvadrat element: `width: 50vmin; height: 50vmin;` — qaysi tomon kichik bo'lsa, shunga moslashadi va hech qachon ekrandan tashqariga chiqmaydi.

⚠️ **Mobil eslatma:** mobil brauzerlarda manzil paneli paydo bo'lib-yo'qolgani sababli `100vh` ba'zan "sakraydi". Zamonaviy yechim — `100dvh` (dynamic viewport height) — bu nozik mavzu; hozircha `vh` borligini bilib qo'y.

12.8 ch va ex — shriftga bog'liq birliklar

Yana ikkita kam ishlatiladigan, lekin foydali nisbiy birlik bor — ular **joriy shriftning** o'lchamiga bog'liq:

- **ch** — joriy shriftdagi `0` (nol) raqamining kengligi. Asosan **matn ustuni kengligini** belgilashda ajoyib: `max-width: 60ch` taxminan 60 belgilik qator demak — bu o'qish uchun ideal kenglik.
- **ex** — joriy shriftning x-height'i (kichik `x` harfi balandligi). Juda kam ishlatiladi.

```
article {  
  max-width: 65ch; /* o'qish uchun qulay qator uzunligi (~65 belgi)  
  */  
}
```

Natija: maqola juda keng ekranda ham bir qatorga ~65 belgidan ko'p sig'maydi — bu ko'zga eng qulay o'qish kengligi (tipografiya qoidasi: 45–75 belgi).

💡 **Nega ch zo'r?** Chunki o'lchov **belgi sonida** beriladi, shrift o'lchami o'zgarsa ham o'qish qulayligi saqlanadi. Bu — **px** bera olmaydigan narsa.

12.9 Ranglar: nomli ranglar

Eng oddiy usul — rangni **nomi** bilan yozish. CSS'da 140 dan ortiq nomli rang bor:

```
.quti {  
  color: white;  
  background-color: steelblue;  
  border-color: tomato;  
}
```

Natija: matn oq, fon po'lat-ko'k, chegara pomidor-qizil.

Mashhur nomlar: `black`, `white`, `red`, `green`, `blue`, `gray`, `orange`, `purple`, `tomato`, `steelblue`, `gold`. Maxsus qiymat — `transparent` (to'liq shaffof).

Nomli ranglar **o'rganish** va tez prototip qilish uchun qulay, lekin amaliy loyihalarda kamroq ishlatiladi — chunki ular cheklangan va aniq brend ranglarini bermaydi. Professional kod hex yoki hsl ishlatadi.

✦ **Eslatma:** `currentColor` — alohida foydali kalit so'z. U "joriy `color` qiymati" degani. Masalan `border: 1px solid currentColor;` chegarani matn rangiga moslab beradi — matn rangi o'zgarsa, chegara ham o'zgaradi.

12.10 hex (o'n oltilik) ranglar

Hex (hexadecimal, o'n oltilik) — eng keng tarqalgan rang yozuvi. U `#` belgisi va 6 ta belgidan iborat: `#rrggbb`, bunda har juftlik qizil (R), yashil (G) va ko'k (B) miqdorini bildiradi.

Har juftlik `00` dan `ff` gacha (o'nlik tizimda 0–255):

```
.quti {  
  color: #ffffff; /* oq: R=255, G=255, B=255 */  
  background: #2563eb; /* ko'k: R=37, G=99, B=235 */  
  border-color: #000000; /* qora */  
}
```

Natija: oq matn, ko'k fon, qora chegara.

Qisqa #rgb yozuvi

Agar har juftlikning ikki belgisi bir xil bo'lsa, hex'ni uch belgiga qisqartirish mumkin — har belgi ikki marta takrorlangandek o'qiladi:

```
.q { color: #f00; } /* = #ff0000 (sof qizil) */  
.w { color: #fff; } /* = #ffffff (oq) */  
.g { color: #36c; } /* = #3366cc */
```

Natija: #f00 to'liq #ff0000 bilan bir xil — shunchaki qisqaroq yozuv.

💡 **O'n oltilik nega?** Kompyuterlar ranglarni 0–255 oralig'ida saqlaydi va 255 o'n oltilik tizimda aniq ff (ikki belgi). Shuning uchun har kanal aniq ikki belgiga sig'adi — ixcham va aniq. Hex dizayn dasturlaridan (Figma, Photoshop) nusxa olish uchun ham qulay.

⚠️ **Tez-tez xato:** # ni unutish (color: 2563eb; ishlamaydi) yoki noto'g'ri belgi soni (5 yoki 7 belgi). Faqat 3, 6 (yoki alfa bilan 4, 8) belgi to'g'ri.

12.11 rgb() va rgba()

rgb() rangni uch son bilan beradi — qizil, yashil, ko'k miqdori (har biri 0–255). Bu hex bilan **aynan bir xil** model, faqat o'qish osonroq:

```
.quti {  
  color: rgb(255, 255, 255); /* oq */  
  background: rgb(37, 99, 235); /* #2563eb bilan bir xil ko'k */  
}
```

Alfa kanal — shaffoflik

rgba() to'rtinchi qiymat — **alfa** (shaffoflik) qo'shadi. Alfa 0 (to'liq shaffof) dan 1 (to'liq qiyuq — noshaffof) gacha:

```
.yarim-shaffof {  
  background: rgba(0, 0, 0, 0.5); /* 50% shaffof qora */  
}  
.deyarli-korinmas {  
  background: rgba(37, 99, 235, 0.1); /* 10% — ozgina ko'k tus */  
}
```

Natija: birinchi fon orqali ortidagi narsa yarim ko'rinadi; ikkinchisi juda yengil ko'k pardadek.

✦ **Zamonaviy sintaksis:** yangi brauzerlarda vergulsiz va birlashgan yozuv ham ishlaydi: `rgb(37 99 235 / 50%)`. Bunda alfa `/` dan keyin foiz yoki son bilan beriladi. `rgb()` va `rgba()` endi bir xil ishlaydi — `rgb()` ga ham alfa berish mumkin.

💡 **Alfa nega kerak?** Soyalar, qoplama (overlay), hover effektlari uchun. Masalan rasm ustiga `rgba(0, 0, 0, 0.4)` qo'ysang, matn rasm fonida ham o'qiladigan bo'ladi.

12.12 hsl() va hsla() — eng inson-do'st model

`hsl()` rangni mutlaqo boshqacha tasvirlaydi — kompyuter emas, **odam** o'ylaydigandek:

- **H — Hue (tus):** rang g'ildiragidagi burchak, `0` — `360` daraja. `0 / 360` = qizil, `120` = yashil, `240` = ko'k.
- **S — Saturation (to'yinganlik):** rang qanchalik jonli, `0%` (kulrang) dan `100%` (to'liq jonli) gacha.
- **L — Lightness (yorqinlik):** `0%` (qora) dan `100%` (oq) gacha; `50%` — sof rang.

```
.q { color: hsl(221, 83%, 53%); } /* #2563eb bilan bir xil ko'k */  
.qizil { color: hsl(0, 100%, 50%); }  
.yashil { color: hsl(120, 100%, 50%); }  
.kulrang { color: hsl(0, 0%, 60%); } /* to'yinganlik 0 → kulrang */
```

Natija: birinchi qiymat aynan oldingi hex/rgb ko'k bilan bir xil. Quyidagi diagramma bir rangning uchala yozuvda bir xil ekanini ko'rsatadi:

Bitta rang — uch xil yozuv

Bu uchala yozuv AYNAN bir xil ko'k rangni beradi

bir xil natija

HEX

#2563eb

25=R · 63=G · eb=B
16 lik sanoq tizimi

eng ixcham

RGB

rgb(37, 99, 235)

Qizil · Yashil · Ko'k
har biri 0–255

o'qish oson

HSL

hsl(221, 83%, 53%)

Tus · To'yinganlik
· Yorqinlik

odamga tushunarli

HSL'da rangni xayolan tasavvur qilish oson: tus + qanchalik to'yingan + qanchalik yorug'

Nega HSL afzal?

HSL'ning ustunligi — uni **qo'lda o'zgartirish** intuitiv:

- Rangni **to'qroq** qilmoqchimisani? Faqat **L** ni kamaytir: `hsl(221, 83%, 40%)`.
- Och** variant kerakmi (masalan hover yoki fon)? **L** ni oshir: `hsl(221, 83%, 90%)`.
- Bir tonli **palitra** quryapsanmi? **H** ni saqlab, faqat **S** va **L** ni o'zgartir.

Hex'da `#2563eb` ni "10% to'qroq" qilish uchun matematika kerak. HSL'da esa bitta sonni o'zgartirasan. Shuning uchun dizayn tizimlari ko'pincha HSL'da quriladi.

`hsla()` ham `rgba()` kabi alfa qo'shadi: `hsla(221, 83%, 53%, 0.5)`. Zamonaviy yozuvda: `hsl(221 83% 53% / 50%)`.

12.13 Zamonaviy ranglar: oklch (qisqacha)

So'nggi yillarda yangi, kuchliroq rang modeli paydo bo'ldi — **oklch()**. U inson ko'zining rangni qanday idrok etishiga yaqinroq tuzilgan:

```
.q { color: oklch(0.6 0.2 250); }
```

- Birinchi son — **yorqinlik** (L), 0 – 1.
- Ikkinchi — **chroma** (rang to'qligi).
- Uchinchi — **hue** (tus), daraja.

Nega muhim? HSL'da `L` ni o'zgartirsang, ranglar idrok darajasida notekis yorishadi (sariq ko'kdan ko'ra yorqinroq ko'rinadi). OKLCH bu muammoni hal qiladi — `L` ni teng o'zgartirsang, **idrokda ham teng** yorishadi. Bu mukammal palitralar qurish uchun zo'r.

✦ **Hozircha:** `oklch()` zamonaviy brauzerlarda ishlaydi, lekin yangi. Boshlovchi sifatida `hsl` ni o'zlashtir; `oklch` borligini bilib qo'y va kerak bo'lganda o'rganasan.

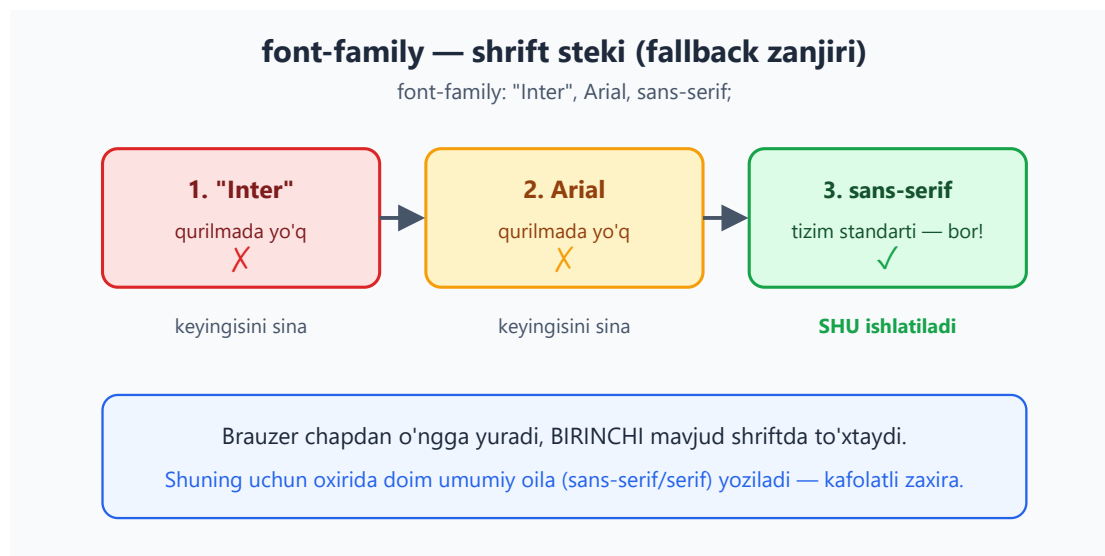
💡 **Qaysi modelni tanlash?** Amaliy maslahat: kundalik ish uchun **hex** (nusxa-ulash oson) yoki **hsl** (qo'lda sozlash oson). Shaffoflik kerak bo'lsa alfa bilan `rgb / hsl`. Kelajak uchun — `oklch`.

12.14 Tipografiya: font-family va shrift steki

Endi matnga o'tamiz. `font-family` matn qaysi **shrift** bilan ko'rsatilishini belgilaydi. Lekin sen bir emas, **ro'yxat** berasan — buni "shrift steki" (font stack) deyiladi:

```
body {  
  font-family: "Inter", Arial, sans-serif;  
}
```

Brauzer ro'yxatni **chapdan o'ngga** sinaydi va birinchi **mavjud** shriftni ishlatadi. "Inter" o'rnatilmagan bo'lsa, `Arial` ga o'tadi; u ham yo'q bo'lsa, tizimning umumiy `sans-serif` shriftiga. Quyidagi diagramma bu zanjirni ko'rsatadi:



Qoidalar: - Nomida bo'sh joy bo'lgan shrift **qo'shtirnoqda**: `"Times New Roman"`. - Ro'yxat oxirida **doim** umumiy oila (generic family) yoz — bu kafolatli zaxira.

Umumiy oilalar (generic families):

Oila	Tavsifi	Misol
<code>serif</code>	Harf uchlarida "oyoqchalar" bor	Times, Georgia
<code>sans-serif</code>	Oyoqchasiz, toza	Arial, Helvetica
<code>monospace</code>	Har harf bir xil kenglikda	Courier, kod uchun
<code>cursive</code>	Yozma uslub	—
<code>system-ui</code>	Foydalanuvchi tizimining standart shrifti	—

⚠ **Tez-tez xato:** umumiy oilani tashlab ketish (`font-family: "Inter";`). Agar shu shrift bo'lmasa, brauzer o'zining standartiga qaytadi — bu sening dizayningga to'g'ri kelmasligi mumkin. Har doim `sans-serif` yoki `serif` bilan tugat.

💡 **"System font stack":** ko'p saytlar hech qanday shriftni yuklamasdan, har qurilmaning o'z chiroyli shriftidan foydalanadi: `font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;`. Bu tez (yuklash yo'q) va har platformada tabiiy ko'rinadi.

12.15 font-size, font-weight, font-style

Bu uchta xossa shriftning asosiy ko'rinishini boshqaradi.

font-size — o'lcham

```
h1 { font-size: 2rem; } /* 32px (root 16px bo'lsa) */
p { font-size: 1rem; } /* 16px */
small { font-size: 0.875rem; } /* 14px */
```

Natija: sarlavha katta, oddiy matn standart, mayda matn kichik. (12.5'da nega `rem` afzalligini ko'rgan eding.)

font-weight — qalinlik

Harf qalinligini belgilaydi. Kalit so'z yoki son (100–900) bilan:

```
.yengil { font-weight: 300; }
.oddii { font-weight: 400; } /* = normal */
```

```
.yarim { font-weight: 500; }  
.qalin { font-weight: 700; } /* = bold */
```

Natija: son qancha katta bo'lsa, harflar shuncha qalin. `normal` = 400, `bold` = 700.

⚠ **Eslatma:** shrift faqat o'zida **bor** qalinliklarni ko'rsata oladi. Agar shrift faqat 400 va 700'ga ega bo'lsa, `font-weight: 500` so'rasang, brauzer eng yaqinini (yoki sun'iy qalinligini) ishlatadi — natija kutilganidek bo'lmasligi mumkin.

font-style — uslub

```
.qiya { font-style: italic; } /* qiya (kursiv) */  
.tik { font-style: normal; } /* tik (standart) */
```

Natija: `italic` matnni qiyalashtiradi — ta'kid yoki sitatalarda ishlatiladi.

12.16 line-height — qator balandligi (va nega unitsiz)

`line-height` matn qatorlari orasidagi vertikal masofani — har qatorning **balandligini** — belgilaydi. To'g'ri `line-height` matnni o'qishni osonlashtiradigan eng muhim xossalardan biri.

```
p {  
  font-size: 16px;  
  line-height: 1.5; /* 1.5 × 16 = 24px qator balandligi */  
}
```

Natija: har qator 24px balandlikda — qatorlar orasida nafas oladigan bo'shliq paydo bo'ladi, matn zich (siqiq) emas.

Nega unitsiz (sonli) qiymat afzal?

`line-height` ni uch xil berish mumkin: `1.5` (unitsiz), `24px`, yoki `150%`. **Unitsiz** afzal — mana nega:

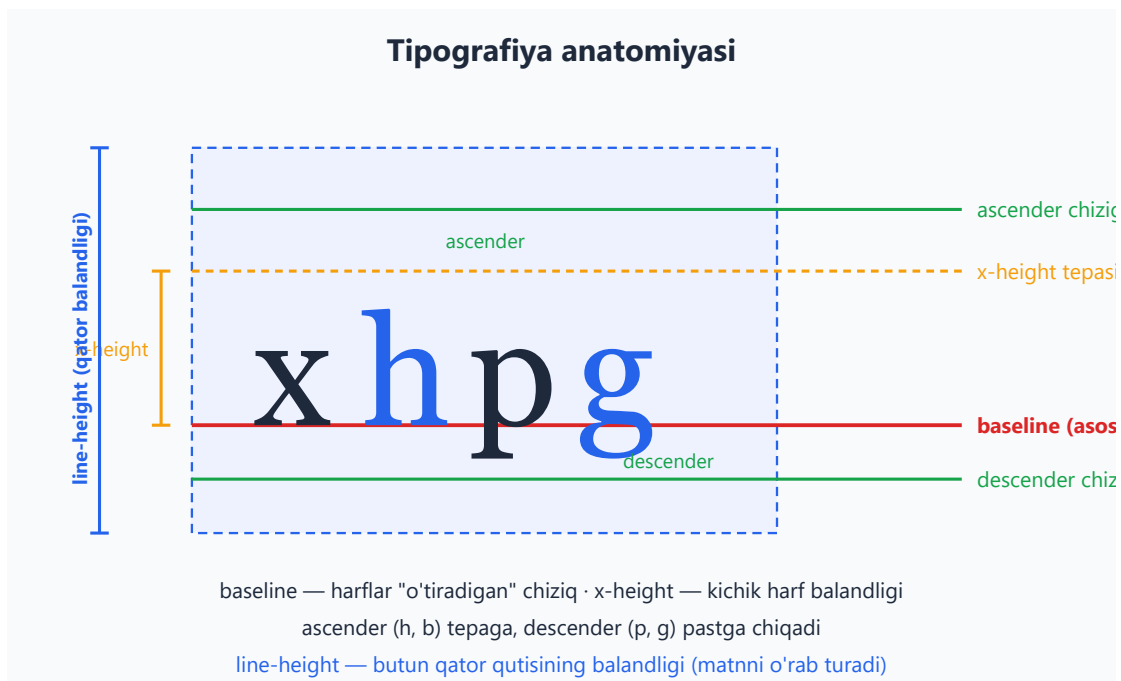
- **Unitsiz 1.5** — bu "joriy element `font-size` 'ining 1.5 baravari" degani va u **har farzandga to'g'ri tarzda** meros bo'ladi: har farzand o'z `font-size` 'iga ko'paytiradi.
- **150% yoki 24px** — *hisoblangan* qiymat meros bo'ladi. Ya'ni ota 16px bo'lsa, `150%` → 24px hisoblanadi va aynan **24px** farzandga o'tadi — farzand 30px bo'lsa ham, qatori baribir 24px qoladi va matn ustma-ust tushib qolishi mumkin.

```
/* Yaxshi — har farzand o'z o'lchamiga moslab hisoblaydi */
body { line-height: 1.5; }

/* Xavfli — hisoblangan 24px hamma farzandga qotib o'tadi */
body { line-height: 24px; }
```

💡 **Oltin qoida:** asosiy matn uchun `line-height: 1.5` atrofida tut (1.4–1.6 oralig'i ideal). Sarlavhalar uchun kichikroq (1.1–1.3), chunki ular kam qatorli.

`line-height` tipografiya anatomiyasining bir qismi — quyidagi diagramma `line-height` ni `baseline`, `x-height` va boshqa chiziqlar bilan ko'rsatadi:



12.17 letter-spacing, text-align, text-decoration

Bu xossalar matnning nozik ko'rinishini sozlaydi.

letter-spacing — harflar orasidagi masofa

```
.kichik-sarlavha {
  text-transform: uppercase;
  letter-spacing: 0.1em; /* harflar orasini ozgina kengaytir */
}
```

Natija: harflar orasida qo'shimcha bo'shliq — KATTA HARFLI sarlavhalarni ko'proq "nafas oladigan" qiladi. `em` da berish afzal (shriftga mutanosib).

text-align — gorizontal tekislash

```
.chap { text-align: left; } /* standart (chapdan-o'ngga  
tillarda) */  
.markaz { text-align: center; }  
.ong { text-align: right; }  
.tekis { text-align: justify; } /* ikkala chetni tekislaydi */
```

Natija: matn mos ravishda chapga, markazga, o'ngga yoki ikki tomonga tekislanadi.

⚠ **Eslatma:** `justify` ni uzun matnda ehtiyot ishlatish kerak — u so'zlar orasida notekis bo'shliq ("daryo"lar) hosil qilishi va o'qishni qiyinlashtirishi mumkin.

text-decoration — chiziqlar

```
a { text-decoration: none; } /* havoladagi tag  
chizig'ini olib tashlash */  
.muhim { text-decoration: underline; } /* tagiga chizish */  
.eski { text-decoration: line-through; } /* ustidan chizish  
(o'chirilgan narx) */  
.boy { text-decoration: underline wavy #dc2626; } /* to'lqinli,  
rangli tag chizig'i */
```

Natija: havoladan chiziq olib tashlanadi; boshqa matnlar tagidan/ustidan chiziladi. To'liq yozuvda chiziq turi, uslubi va rangini belgilash mumkin.

12.18 text-transform va white-space

text-transform — harf registri

Matnni **HTML'ni o'zgartirmasdan**, faqat ko'rinishda katta/kichik harfga aylantiradi:

```
.katta { text-transform: uppercase; } /* HAMMASI KATTA */  
.kichik { text-transform: lowercase; } /* hammasi kichik */  
.bosh { text-transform: capitalize; } /* Har So'z Bosh Harf */
```

Natija: matn ko'rinishi o'zgaradi, lekin HTML'dagi haqiqiy matn (va nusxa olganingda olinadigan matn) o'zgarmaydi.

💡 **Nega vizual?** Agar sarlavhani HTML'da KATTA HARFLI yozsang, skrinrider (screen reader) uni harf-harf o'qishi yoki "qichqirayotgandek" talqin qilishi mumkin. `text-transform: uppercase` esa faqat ko'rinishni o'zgartiradi — HTML toza qoladi, qulaylik saqlanadi.

white-space — bo'sh joy va qator uzilishi

`white-space` matndagi probel va qator uzilishlarining qanday ishlanishini boshqaradi:

Qiymat	Bo'sh joylar	Qator uzilishi
<code>normal</code>	Birlashtiradi	Avtomatik o'raydi
<code>nowrap</code>	Birlashtiradi	O'ramaydi (bir qatorda)
<code>pre</code>	Saqlaydi	Faqat sen yozgan joyda
<code>pre-wrap</code>	Saqlaydi	Saqlaydi + avtomatik o'raydi

```
.bir-qatorda {  
  white-space: nowrap; /* matn o'rالمaydi, bir qatorda qoladi */  
  overflow: hidden;  
  text-overflow: ellipsis; /* sig'magani "... " bo'lib ko'rsatiladi */  
}
```

Natija: uzun matn o'rالمasdan bir qatorda qoladi va sig'magan qismi `...` bilan kesiladi — kartochkalardagi sarlavhalar uchun mashhur usul.

✦ `pre` kod ko'rsatishda foydali — u sening probel va qatorlaringni aynan saqlaydi (`<pre>` tegi ham shuni qiladi).

12.19 Web shriftlar: @font-face, Google Fonts, font-display

Tizim shriftlari cheklangan. Brendga mos **maxsus shrift** kerak bo'lsa, uni saytga yuklash mumkin — buni "web font" deyiladi.

@font-face — shriftni qo'lda ulash

`@font-face` qoidasi brauzerga shrift faylini qayerdan olishni va qanday nomlashni aytadi:

```
@font-face {  
  font-family: "MeningShriftim";  
  src: url("/shriftlar/mening-shrift.woff2") format("woff2");  
  font-weight: 400;  
}
```

```
font-display: swap;
}

body {
  font-family: "MeningShriftim", sans-serif;
}
```

Natija: brauzer shriftni yuklaydi va `body` matnini shu shrift bilan ko'rsatadi. `.woff2` — eng zamonaviy, eng kichik hajmli format (afzal).

Google Fonts — eng oson yo'l

O'zing fayl bilan ovora bo'lmasdan, tayyor bepul shriftlardan foydalanish mumkin.

`<head>` ga bitta `<link>` qo'shasan:

```
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;700&display=swap" rel="stylesheet">
</head>
```

```
body {
  font-family: "Inter", sans-serif;
}
```

Natija: "Inter" shrifti Google serveridan yuklanadi va ishlatiladi. URL'dagi `display=swap` — bu keyingi muhim mavzu.

font-display — yuklanayotganda nima ko'rsatiladi?

Shrift yuklanguncha bir necha yuz millisekund o'tishi mumkin. `font-display` shu paytda nima bo'lishini hal qiladi:

Qiymat	Yuklangunicha	Nega
<code>swap</code>	Zaxira shriftni ko'rsat, tayyor bo'lgach almashtir	Matn darhol o'qiladi — eng ko'p tavsiya etiladigan
<code>block</code>	Qisqa vaqt bo'sh (matn ko'rinmaydi), keyin ko'rsat	Matn kechikadi — qochish kerak
<code>fallback</code>	Qisqa block, keyin swap	Aralash

Qiymat	Yuklangunicha	Nega
optional	Tez bo'lsa ishlat, bo'lmasa zaxirada qol	Sekin internet uchun

💡 **Amaliy tavsiya:** deyarli har doim `font-display: swap` ishlat. Bu "ko'rinmas matn" (FOIT — Flash of Invisible Text) muammosini oldini oladi: foydalanuvchi matnni darhol o'qiy boshlaydi, shrift esa fonda yuklanib, tayyor bo'lgach almashadi.

⚠️ **Ishlash (performance) eslatmasi:** har bir web shrift — qo'shimcha yuklab olinadigan fayl. Faqat **kerakli qalinliklarni** yukla (masalan 400 va 700, hammasini emas) va `.woff2` formatidan foydalan — sayt tez ochiladi.

12.20 Hammasini birga: amaliy misol

Mana shu bobning bilimlarini birlashtirgan kichik, real komponent:

```
:root {
  /* Ranglar HSL'da - sozlash oson */
  --asosiy: hsl(221, 83%, 53%);
  --asosiy-toq: hsl(221, 83%, 43%);
  --matn: hsl(222, 47%, 11%);
}

html { font-size: 16px; } /* root - rem shu yerga bog'lanadi */

.karta {
  max-width: 60ch; /* o'qish uchun qulay kenglik */
  padding: 1.5rem; /* barqaror bo'shliq */
  color: var(--matn);
  font-family: system-ui, "Segoe UI", sans-serif;
  line-height: 1.6; /* unitsiz - to'g'ri meros bo'ladi */
}

.karta h2 {
  font-size: 1.5rem;
  font-weight: 700;
  line-height: 1.2;
  letter-spacing: -0.01em;
}

.karta .tugma {
  padding: 0.6em 1.2em; /* em - tugma shriftiga mutanosib */
  background: var(--asosiy);
  color: #fff;
  border: none;
  border-radius: 0.5rem;
}

.karta .tugma:hover {
```

```
background: var(--asosiy-toq); /* faqat L kamaydi → to'qroq */
}
```

```
<article class="karta">
  <h2>O'lchov va ranglar amalda</h2>
  <p>Bu kartochka rem, em, ch, hsl va to'g'ri line-height'ni
  birlashtiradi.</p>
  <button class="tugma">Bosing</button>
</article>
```

Natija: moslashuvchan, o'qishga qulay va oson sozlanadigan kartochka — har bir o'lchov va rang tanlovi ortida sabab bor. Ranglar HSL o'zgaruvchilarida, shriftlar `rem` da, tugma `padding` 'i `em` da, qator uzunligi `ch` da — bu professional CSS'ning poydevori.

Mashqlar

Quyidagi mashqlarni brauzerda (yoki Chrome DevTools'da) sinab ko'r. Avval o'zing javob ber, keyin yechimni och.

Mashq 1 — em hisobi

`html { font-size: 16px }` bo'lsa va quyidagi struktura berilsa, eng ichki `<span>` matni necha piksel bo'ladi?

```
.a { font-size: 1.25em; }
.b { font-size: 1.25em; }
```

```
<div class="a"><div class="b"><span>matn</span></div></div>
```

Yechim

25px.

- `.a` = $1.25 \times 16 = 20\text{px}$
- `.b` = $1.25 \times 20 = 25\text{px}$ (otasi `.a` ning `20px`'iga nisbatan)
- `<span>` o'zining `font-size` 'i yo'q, shuning uchun otasi `.b` 'dan **25px** meros oladi.

Bu — `em` ning ko'payib ketishi. Agar `.a` va `.b` da `rem` ishlatilganida, ikkalasi ham $1.25 \times 16 = 20\text{px}$ bo'lar edi.

Mashq 2 — Bir rangni uch xil yozish

`rgb(255, 165, 0)` (orange) rangini hex yozuvida yoz. Keyin tushuntir: nega HSL'da uni to'qroq qilish osonroq?

Yechim

Hex: `#ffa500` ($255 = ff, 165 = a5, 0 = 00$).

HSL'da bu rang taxminan `hsl(39, 100%, 50%)`. Uni to'qroq qilish uchun faqat **yorqinlikni** kamaytirasan: `hsl(39, 100%, 35%)`. Hex yoki rgb'da esa uchala kanalni qayta hisoblash kerak bo'lardi — shuning uchun HSL sozlash uchun qulay.

Mashq 3 — line-height tuzog'i

Nega quyidagi kod muammoli? Tuzat.

```
body {  
  font-size: 16px;  
  line-height: 20px;  
}  
h1 { font-size: 40px; }
```

Yechim

Muammo: `line-height: 20px` **hisoblangan 20px** sifatida `<h1>` 'ga meros bo'ladi. Lekin `<h1>` 40px shriftda — 40px harf 20px balandlikdagi qatorga sig'maydi, qatorlar ustma-ust tushadi.

Tuzatish — **unitsiz** qiymat ishlat:

```
body {  
  font-size: 16px;  
  line-height: 1.5; /* har element o'z font-size'iga ko'paytiradi */  
}  
h1 { font-size: 40px; } /* qatori avtomatik 60px bo'ladi */
```

Mashq 4 — To'g'ri shrift steki

Quyidagi `font-family` 'da ikkita xato bor. Topping va tuzating.

```
body {  
  font-family: Times New Roman, Inter;  
}
```



Yechim



Ikki xato: 1. Times New Roman bo'sh joyli nom — **qo'shtirnoq**da bo'lishi kerak. 2. Oxirida **umumiy oila** (generic family) yo'q — zaxira kafolati yo'q.

Tuzatilgan (mantiqan Inter birinchi bo'lishi kerak, keyin zaxiralar):

```
body {  
  font-family: "Inter", "Times New Roman", serif;  
}
```



Mashq 5 — Viewport birliklar

"Qahramon bo'lim" (hero) yarataylik: u har qanday ekranda to'liq ekran balandligini egallasin va kengligi viewport'ning to'liq enini olsun. CSS yoz.



Yechim



```
.qahramon {  
  width: 100vw;      /* viewport kengligi 100% */  
  height: 100vh;     /* viewport balandligi 100% */  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```



100vh ekran balandligiga, 100vw kengligiga bog'lanadi — ekran o'lchami qanday bo'lishidan qat'i nazar to'liq ekranni egallaydi.

Mashq 6 — currentColor

.tugma ning matn rangi `hsl(221, 83%, 53%)` bo'lsin va chegarasi **aynan shu** rangda bo'lsin — lekin rangni faqat **bir marta** yoz. Qanday qilasan?



Yechim



```
.tugma {  
  color: hsl(221, 83%, 53%);  
  border: 2px solid currentColor; /* "joriy color qiymati" */  
  background: transparent;  
}
```



currentColor joriy color qiymatini oladi. Endi color ni o'zgartirsang, chegara ham avtomatik o'zgaradi — takror yozish shart emas.

Mashq 7 — rem bilan miqyoslash

`html { font-size: 16px }` va barcha matnlar `rem` da. Butun saytdagi matnni 12.5% kattalashtirish uchun **bitta qator** o'zgartirish yoz.

Yechim

```
html { font-size: 18px; } /* 16 × 1.125 = 18px */
```

Hamma `rem` qiymatlar root'ga bog'langani uchun, faqat root'ni o'zgartirish butun saytni mutanosib kattalashtiradi. Aynan shu — `rem` ning eng katta afzalligi (12.5'da ko'rgan eding). `px` da har bir o'lchamni alohida o'zgartirishga to'g'ri kelar edi.

Mashq 8 — Matn kesish

Kartochka sarlavhasi uzun bo'lsa, bir qatorda qolib, sig'magani `...` bilan kesilsin. CSS yoz.

Yechim

```
.karta__sarlavha {  
  white-space: nowrap; /* o'ramaslik — bir qatorda */  
  overflow: hidden; /* sig'magan qism yashiriladi */  
  text-overflow: ellipsis; /* yashiringan qism "..." bo'ladi */  
}
```

Uchala xossa birga ishlaydi: `nowrap` matnni bir qatorga majburlaydi, `overflow: hidden` ortiqchasini kesadi, `text-overflow: ellipsis` esa kesilgan joyni `...` bilan belgilaydi.

13 — Fon, chegara va soyalar

← Oldingi: 12 — O'lchovlar, tipografiya va ranglar · 🏠 README · Keyingi: 14 — Positioning va z-index →

Bu bobda: elementlarni jonlantiramiz — `background` (fon rangi, rasm, gradient), `border` va `border-radius` (chegara va yumaloq burchaklar), `box-shadow` (chuqurlik beruvchi soyalar), `outline` hamda `filter` bilan vizual effektlar yaratamiz.

13.1 Nega bu bob dizaynni "tirik" qiladi?

Box model (11-bob) elementga shakl berdi, display va joylashuv (kelgusi boblar) ularni sarlashtiradi. Lekin sahifa hali ham **tekis va jonsiz** ko'rinadi: oq fon, qora chiziq, o'tkir burchak. Aynan shu bobda o'rganadigan xossalar — fon, gradient, yumaloq burchak va soya — sahifaga **chuqurlik, rang va shaxsiyat** beradi.

O'ylab ko'r: zamonaviy tugmalar nega "bosiladigandek" tuyuladi? Kartochkalar nega qog'oz bo'lagidek qalqib turadi? Buning sirliги yo'q — bu yumaloq burchak (`border-radius`) va yumshoq soya (`box-shadow`) ning birgalikdagi ishi. Ushbu bobni tugatganingda, sen oddiy `<div>` ni chiroyli kartochkaga aylantira olasan.

✦ **Atamalar tarjiması:** - **background** — fon (orqa qatlam). - **gradient** — gradient, ya'ni ranglarning silliq o'tishi. - **border** — chegara (quti devori). - **border-radius** — burchak radiusi (yumaloqlik). - **box-shadow** — quti soyasi. - **outline** — kontur (chegaradan tashqaridagi chiziq). - **filter** — filtr (vizual effekt).

13.2 background-color — eng oddiy fon

Eng sodda fon — bitta rang. `background-color` elementning butun fon maydonini (content + padding, odatda border ostini ham) bo'yaydi.

```
.karta {  
  background-color: #e0f2fe; /* och ko'k fon */  
}
```

```
<div class="karta">Men och ko'k fonli quti man.</div>
```

Natija: quti orqasi och ko'k rangga bo'yaladi. Rang har qanday formatda bo'lishi mumkin: nom (`tomato`), HEX (`#e0f2fe`), `rgb()`, `hsl()` yoki shaffoflik bilan `rgba()` / `hsl(... / 0.5)`.

💡 **Maslahat:** fonni shaffof qoldirish uchun `background-color: transparent` ishlat (bu standart qiymat). Yarim shaffof fon uchun `rgba(0, 0, 0, 0.5)` kabi — bu ostidagi narsani biroz ko'rsatadi, modal oynalar oynasi (overlay) uchun juda foydali.

⚠️ **Tez-tez xato:** `color` va `background-color` ni adashtirish. `color` — **matn** rangi, `background-color` — **fon** rangi. Ikkalasini birga ishlatganda kontrastni unutma — och fonda och matn o'qilmaydi.

13.3 background-image — fon sifatida rasm

Fon faqat rang emas — rasm ham bo'la oladi. `background-image` ga `url()` orqali rasm manzilini beramiz.

```
.banner {
  background-image: url("rasmlar/manzara.jpg");
  height: 300px;
  color: white;
}
```

✳️ **Diqqat:** `url()` ichidagi yo'l CSS fayli joylashgan joyga nisbatan hisoblanadi (HTML faylga emas!). Bu boshlovchilarni tez-tez chalg'itadi.

💡 Bir element bir vaqtning o'zida ham `background-color`, ham `background-image` ga ega bo'lishi mumkin. Rasm tepada turadi; agar rasm yuklanmasa yoki shaffof joylari bo'lsa, rang ostidan ko'rinadi. Shu sabab fon rasmlari uchun **doim yedak rang** berish yaxshi odat:

```
.banner {
  background-color: #1e293b; /* rasm yuklanmasa — shu rang */
  /*
  background-image: url("rasmlar/tog.jpg");
  */
}
```

`background-image` ga rasmdan tashqari **gradient** ham berish mumkin (chunki gradient ham texnik jihatdan "rasm"). Buni 13.8 da ko'ramiz.

13.4 background-repeat — rasm takrorlanishi

Standart holatda fon rasmi elementni to'ldirish uchun **ikkala o'q bo'ylab takrorlanadi** (mozaikadek). Buni `background-repeat` boshqaradi:

```
.uslub1 { background-repeat: repeat; } /* standart: x va y bo'ylab takror */
.uslub2 { background-repeat: no-repeat; } /* takrorlanmaydi - bitta nusxa */
.uslub3 { background-repeat: repeat-x; } /* faqat gorizontal takror */
.uslub4 { background-repeat: repeat-y; } /* faqat vertikal takror */
.uslub5 { background-repeat: space; } /* takror + orasida teng bo'sh joy */
.uslub6 { background-repeat: round; } /* takror + cho'zib teng joylash */
```

Nega kerak? Kichik naqsh (pattern, masalan nuqtalar to'ri) bilan butun fonni qoplamoqchi bo'lsang, `repeat` zo'r. Lekin bitta katta surat (banner) qo'yimoqchi bo'lsang, `no-repeat` shart — aks holda surat takrorlanib, xunuk ko'rinadi.

⚠ **Eng keng tarqalgan xato:** banner rasmiga `no-repeat` qo'yishni unutish. Rasm element kattaligidan kichik bo'lsa, u takrorlanib ketadi.

13.5 background-position — rasmni joylashtirish

`background-position` rasmning element ichida **qayerga qo'yilishini** belgilaydi. Kalit so'zlar yoki aniq qiymatlar bilan:

```
.a { background-position: center; } /* o'rta */
.b { background-position: top right; } /* yuqori-o'ngga */
.c { background-position: 20px 50px; } /* chapdan 20px, tepadan 50px */
.d { background-position: 50% 25%; } /* foiz bilan */
```

Ikki qiymat: **birinchisi gorizontal** (chap-o'ng), **ikkinchisi vertikal** (tepa-past).

💡 **Eng foydali kombinatsiya** — banner uchun "rasm doim markazda, takrorlanmaydi, qutini to'ldiradi":

```
.banner {
  background-image: url("rasmlar/tog.jpg");
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover; /* keyingi bo'limda */
}
```

✦ Foiz bilan joylashtirish o'ziga xos: `50% 50%` rasmning **markazini** elementning **markaziga** to'g'rilaydi (oddiy "o'lchamning 50%" emas). Shuning uchun `center` deyarli har doim foizdan tushunarliroq.

13.6 background-size — cover va contain

`background-size` fon rasmining **o'lchamini** boshqaradi. Eng muhim ikki kalit so'z:

```
.cover {  
  background-size: cover; /* rasmni cho'zib qutini TO'LIQ qoplaydi */  
}  
.contain {  
  background-size: contain; /* rasm TO'LIQ ko'rinadi, qutiga sig'adi */  
}
```

Farqini tushunish juda muhim:

- **cover** — rasm nisbatini saqlab, qutini **butunlay** to'ldiradi. Rasm qutidan kattaroq bo'lsa, ortiqcha qismi **kesiladi**. Banner va hero bo'limlar uchun ideal — bo'sh joy qolmaydi.
- **contain** — rasm nisbatini saqlab, **butun rasm ko'rinadigan** qilib joylashtiriladi. Natijada qutida bo'sh joy (bo'shliq) qolishi mumkin. Logotip yoki butun rasmni ko'rsatish kerak bo'lganda ishlat.

Aniq o'lcham ham berish mumkin:

```
.aniq { background-size: 200px 100px; } /* eni 200px, bo'yi 100px */  
.eni { background-size: 100% auto; } /* eni to'liq, bo'yi nisbatan */
```

💡 **Nega "cover" eng mashhur?** Chunki turli ekran o'lchamlarida (telefon, kompyuter) banner doim to'liq to'ladi, bo'sh oq joy qolmaydi. Bu zamonaviy "hero" bo'limlarning asosi.

13.7 background-attachment — fon "yopishishi"

`background-attachment` fon rasmi sahifa bilan birga **siljishini** yoki **joyida qotib turishini** belgilaydi:

```
.scroll { background-attachment: scroll; } /* standart: kontent bilan  
siljiydi */  
.fixed { background-attachment: fixed; } /* joyida qotadi (parallaks  
effekti) */
```

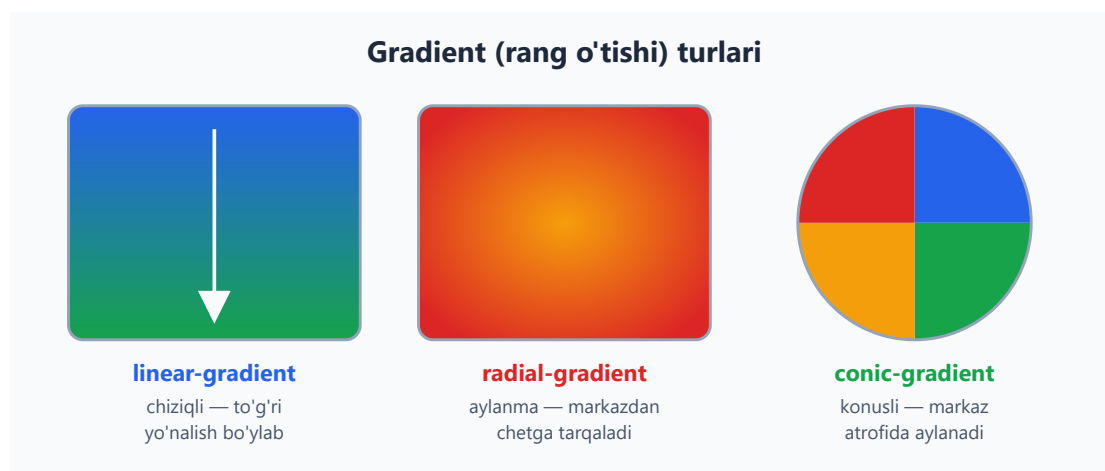
`fixed` da fon ekranga "yopishadi": sahifani aylantirganingda matn siljiydi, lekin fon qimirlamaydi. Bu mashhur **parallaks** (parallax) ta'sirini beradi.

⚠ Mobil brauzerlarda `fixed` har doim ham silliq ishlamaydi va batareyani ko'proq yeydi. Asosiy dizayningni unga bog'lab qo'yma — bu "bezak", zaruriy emas.

13.8 Gradientlar — ranglarning silliq o'tishi

Gradient — bu bir rangdan ikkinchisiga **silliq o'tish**. CSS gradientni bir tur "rasm" deb hisoblaydi, shuning uchun u `background-image` (yoki qisqa `background`) ichida yoziladi — `url()` o'rniga.

Uch asosiy tur bor:



linear-gradient — chiziqli

Ranglar **to'g'ri chiziq** bo'ylab o'tadi. Birinchi argument — **yo'nalish**:

```
.a { background-image: linear-gradient(to right, #2563eb, #16a34a); }  
/* chapdan o'ngga */  
.b { background-image: linear-gradient(to bottom, #2563eb, #16a34a); }  
/* tepadan pastga (standart) */  
.c { background-image: linear-gradient(45deg, #2563eb, #16a34a); }  
/* 45 graduslik burchak */
```


Yo'nalishni **so'z** (to right, to bottom right) yoki **gradus** (90deg) bilan berish mumkin. 0deg — pastdan yuqoriga, 90deg — chapdan o'ngga.

Rang to'xtashlari (color stops) — har rang qayerda boshlanishini aniq belgilash:

```
.uchrang {  
  background-image: linear-gradient(  
    to right,  
    #dc2626 0%,      /* qizil — boshida */  
    #f59e0b 50%,     /* amber — o'rtada */  
    #16a34a 100%    /* yashil — oxirida */  
  );  
}
```

💡 **Keskin chegara** yasash: ikki to'xtashni bir nuqtaga qo'ysang, gradient "o'tish" emas, **chiziq** bo'ladi (bayroq chizish uchun foydali):

```
.bayroq {  
  background-image: linear-gradient(to right, #dc2626 50%, #2563eb  
50%);  
  /* chap yarmi qizil, o'ng yarmi ko'k — silliq o'tishsiz */  
}
```

radial-gradient — aylanma

Ranglar **markazdan chetga** doira/ellips bo'ylab tarqaladi:

```
.a { background-image: radial-gradient(#f59e0b, #dc2626); }  
/* markaz amber -> chet qizil */  
.b { background-image: radial-gradient(circle, #f59e0b, #dc2626); }  
/* aniq doira shaklida */  
.c { background-image: radial-gradient(circle at top left, #f59e0b,  
#dc2626); } /* markaz yuqori-chapda */
```

Bu "yorug'lik manbai" yoki "porlash" effekti uchun zo'r — markaz yorqin, chetlari qorong'i.

conic-gradient — konusli

Ranglar markaz **atrofida aylanadi** (soatdek). Diagrammalar (pie chart), rang g'ildiragi yoki yuklanish indikatorlari uchun ishlatiladi:

```
.pirog {  
  background-image: conic-gradient(  
    #2563eb 0deg 90deg,    /* ko'k chorak */  
    #16a34a 90deg 180deg,  /* yashil chorak */  
    #f59e0b 180deg 270deg, /* amber chorak */  
  );  
}
```

```

    #dc2626 270deg 360deg /* qizil chorak */
  );
  border-radius: 50%;      /* doira qilish uchun */
}

```

✦ **Takrorlanuvchi gradient:** `repeating-linear-gradient` va `repeating-radial-gradient` bilan naqsh (chiziqlar, halqalar) yasash mumkin:

```

.chiziqli {
  background-image: repeating-linear-gradient(
    45deg, #e2e8f0 0 10px, #cbd5e1 10px 20px
  ); /* takroriy diagonal chiziqlar */
}

```

13.9 Bir nechta fon va qisqa background yozuvi

Bir nechta fon (multiple backgrounds)

Bitta elementga **bir nechta fon qatlamini** vergul bilan qo'yish mumkin. **Birinchi yozilgan — eng tepada** turadi:

```

.qatlam {
  background-image:
    url("rasmlar/naqsh.png"), /* 1: eng
    tepada */
    linear-gradient(to bottom, #2563eb, #1e3a8a); /* 2: pastda
    */
}

```

Bu juda kuchli: masalan tepaga shaffof naqsh, ostiga gradient qo'yish bilan boy fonlar yasaladi.

Qisqa `background` yozuvi (shorthand)

Yuqoridagi barcha xossalarni bitta `background` qatorida birlashtirish mumkin:

```

.toliq {
  background: #1e293b url("rasmlar/tog.jpg") no-repeat center / cover
  fixed;
  /*      rang      rasm              repeat  position / size
  attachment */
}

```

✦ **Tartibni eslab qolish shart emas**, lekin bitta nozik qoida bor: `background-size` doim `background-position` dan keyin, / **belgisi bilan** ajratilib yoziladi (`center` / `cover`). Buni unutsang, brauzer qiymatni o'qiy olmaydi.

⚠ **Diqqat:** qisqa `background` yozuvi ko'rsatilmagan barcha sub-xossalarni **standart qiymatga qaytaradi**. Ya'ni avval `background-color: red` yozib, keyin `background: url(...)` yozsang — qizil rang **o'chadi**. Tartib muhim.

13.10 border — chegara va uning turlari

`border` — elementni o'rab turuvchi chiziq. Uch komponentdan iborat: **qalinlik, uslub va rang**.

```
.quti {  
  border-width: 2px;      /* qalinligi */  
  border-style: solid;    /* uslubi (MAJBURIY — bo'lmasa chegara  
                           ko'rinmaydi!) */  
  border-color: #2563eb;  /* rangi */  
}
```

Odatda qisqa yozuv ishlatiladi:

```
.quti { border: 2px solid #2563eb; } /* qalinlik uslub rang */
```

⚠ **Eng ko'p uchraydigan xato:** `border-style` ni unutish. `border: 2px #2563eb` (uslubsiz) — **hech narsa ko'rsatmaydi**, chunki `border-style` ning standart qiymati `none`. Chegara ko'rinishi uchun uslub **shart**.

border-style turlari

```
.s1 { border: 3px solid #475569; } /* to'liq chiziq (eng keng  
tarqalgan) */  
.s2 { border: 3px dashed #475569; } /* tirelar (---) */  
.s3 { border: 3px dotted #475569; } /* nuqtalar (...) */  
.s4 { border: 3px double #475569; } /* ikki parallel chiziq */  
.s5 { border: 4px groove #475569; } /* "o'yilgan" 3D effekt */  
.s6 { border: 4px ridge #475569; } /* "bo'rtgan" 3D effekt */  
.s7 { border: 4px inset #475569; } /* "ichkariga botgan" */  
.s8 { border: 4px outset #475569; } /* "tashqariga chiqqan" */  
.s9 { border: 3px none #475569; } /* yo'q (ko'rinmaydi) */
```

💡 Amalda 90% holatda `solid` ishlatiladi; `dashed` va `dotted` ham foydali. `groove/ridge/inset/outset` eski uslubdagi 3D effektlar — hozir kam ishlatiladi.

Har tomonni alohida boshqarish

Faqat bitta tomonga chegara berish mumkin:

```
.faqat-tag {  
  border-bottom: 2px solid #e2e8f0; /* faqat pastki chegara –  
  ajratuvchi chiziq sifatida */  
}  
.har-xil {  
  border-top: 1px solid #94a3b8;  
  border-left: 4px solid #2563eb; /* chap tomonda qalin urg'u  
  chizig'i */  
}
```

`border-bottom` bilan ajratuvchi chiziq (divider) yasash — ro'yxatlar va menyularda juda keng tarqalgan amaliyot.

13.11 border-radius — yumaloq burchaklar

`border-radius` burchaklarni **yumaloq** qiladi. Bu zamonaviy dizaynning eng muhim detallaridan biri.

border-radius: burchaklarni yumshatish

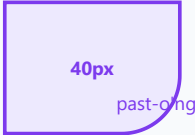


0
o'tkir burchak

16px
yengil yumaloq

40px
kuchli yumaloq

50%
to'liq doira (kvadratda)



40px
past-o'ng

Har burchak alohida
`border-radius: 0 0 40px 0;`
tartib: yuqori-chap, yuqori-o'ng,
past-o'ng, past-chap (soat yo'nalishi)

```
.yengil { border-radius: 8px; } /* yumshoq burchak – tugma, karta  
uchun */  
.kuchli { border-radius: 24px; } /* kuchli yumaloqlik */  
.doira { border-radius: 50%; } /* kvadratni doira qiladi! */
```

💡 **Kvadratdan doira yasash:** kvadrat (eni = bo'yi) elementga `border-radius: 50%` bersang, mukammal **doira** chiqadi. Avatar (profil rasmi) yumaloqlash uchun klassik usul:

```
.avatar {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  object-fit: cover; /* rasm cho'zilmasligi uchun */
}
```

Har burchakni alohida

To'rt qiymat berilsa, ular **soat yo'nalishida** o'qiladi: yuqori-chap, yuqori-o'ng, past-o'ng, past-chap.

```
.bilet {
  border-radius: 0 0 16px 0; /* faqat past-o'ng burchak yumaloq */
}
.qabarcha {
  border-radius: 18px 18px 18px 4px; /* chat xabari "qulog'i" effekti */
}
```

Ellips burchaklar

Har burchakda gorizontaal va vertikal radiusni alohida berish mumkin (/ bilan), bu **elliptik** burchak yasaydi:

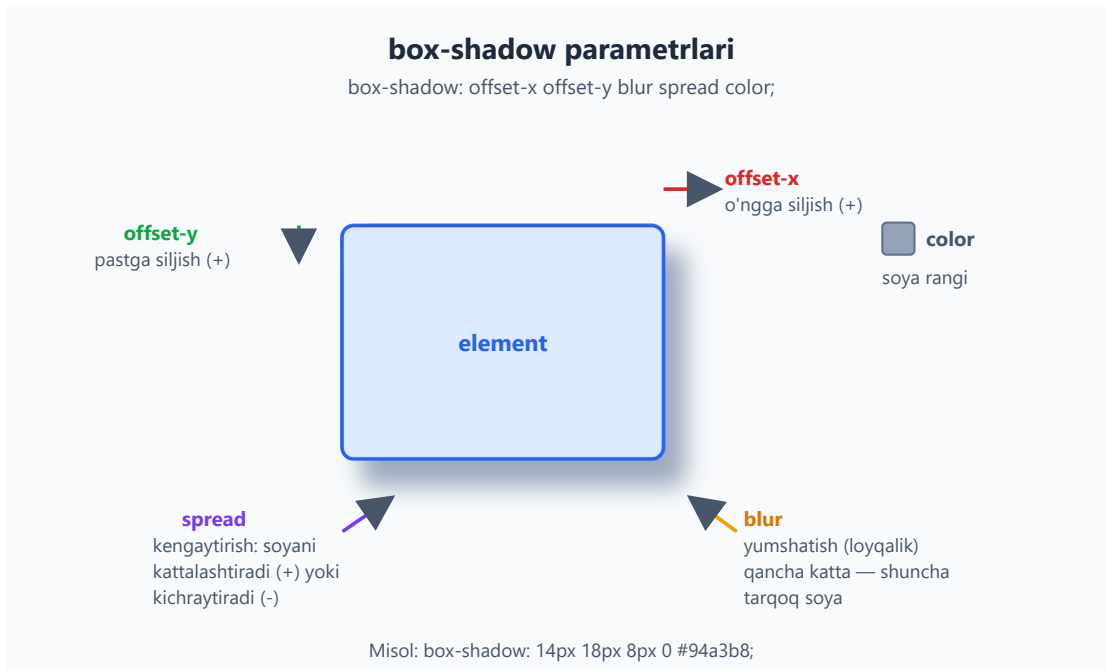
```
.ellips {
  border-radius: 50px / 20px; /* gorizontaal 50px, vertikal 20px - cho'zilgan burchak */
}
```

✦ `border-radius` rasm, gradient va `background-color` ni ham kesadi — fon yumaloq burchak ichida qoladi. Bu juda qulay: bitta xossa hamma narsani yumaloqlaydi.

13.12 box-shadow — chuqurlik beruvchi soya

`box-shadow` elementga **soya** qo'shadi va shu bilan unga "qalqib turish" yoki "chuqurlik" hissini beradi. Bu zamonaviy interfeyslarning asosiy quroli.

To'liq sintaksis besh qismdan iborat:



```
.karta {  
  box-shadow: 0 4px 12px 0 rgba(0, 0, 0, 0.15);  
  /*  
    ↑   ↑   ↑   ↑   ↑  
    |   |   |   |   |  
    |   |   |   |   | color: soya rangi  
    |   |   |   |   | spread: kengayish (0 = o'zgarishsiz)  
    |   |   |   |   | blur: yumshatish (12px loyqalik)  
    |   |   |   |   | offset-y: pastga 4px  
    |   |   |   |   | offset-x: gorizontaal 0 (siljimaydi) */  
}
```

Har parametr nima qiladi:

- **offset-x** — soyani **gorizontal** suradi. Musbat (+) o'ngga, manfiy (-) chapga.
- **offset-y** — soyani **vertikal** suradi. Musbat (+) pastga, manfiy (-) tepaga.
- **blur** — soyaning **chetini yumshatadi**. 0 = keskin (qattiq) soya; katta qiymat = tarqoq, yumshoq soya. Manfiy bo'lolmaydi.
- **spread** — soyani **kattalashtiradi** (+) yoki **kichraytiradi** (-). Ko'pincha 0. (Ixtiyoriy — tushirib qoldirsa bo'ladi.)
- **color** — soya **rangi**. Tabiiy ko'rinish uchun ko'pincha yarim shaffof qora (rgba(0,0,0,0.1)) ishlatiladi.

💡 **Tabiiy soyaning siri:** real hayotda yorug'lik tepadan tushadi, shuning uchun soya **pastda** bo'ladi. Demak `offset-y` musbat (pastga), `offset-x` ko'pincha 0. Yengil blur va past shaffoflik — eng tabiiy natija:

```
.tugma {  
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.12); /* nozik, ishonchli soya
```

```
*/  
}
```

inset — ichki soya

`inset` kalit so'zi soyani **ichkariga** soladi — element "botgan" yoki "o'yilgan" ko'rinadi (bosilgan tugma, kiritish maydoni):

```
.bosilgan {  
  box-shadow: inset 0 2px 5px rgba(0, 0, 0, 0.2); /* ichkariga botgan  
  effekt */  
}
```

Bir nechta soya

Vergul bilan **bir nechta soya** berish mumkin — har biri alohida qatlam. Bu chuqurroq, real soyalar yasaydi:

```
.qalqigan {  
  box-shadow:  
    0 1px 2px rgba(0, 0, 0, 0.08), /* yaqin, keskin soya */  
    0 8px 24px rgba(0, 0, 0, 0.12); /* uzoq, yumshoq soya */  
}
```

💡 Professional dizaynlarda bitta katta soya o'rniga **2-3 ta nozik soya** qatlami ishlatiladi — bu tabiiyroq ko'rinadi.

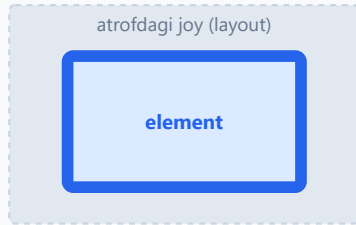
⚠️ `box-shadow` element o'lchamiga **ta'sir qilmaydi** va boshqa elementlarni surmaydi (box model'dan tashqarida chiziladi). Shuning uchun soyalar bilan layout buzilmaydi — bimalol ishlat.

13.13 outline — kontur va undan border farqi

`outline` ham chegaraga o'xshash chiziq, lekin ikki muhim farqi bor. Bu farq tasodifiy emas — har birining o'z vazifasi bor.

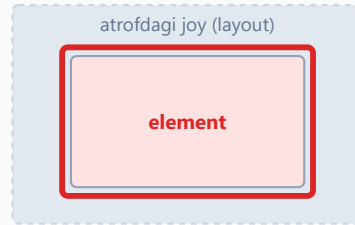
border va outline farqi

border



JOY egallaydi — qo'shni
elementlarni suradi

outline



JOY egallamaydi — ustidan
chiziladi, layout o'zgarmas

```
.kontur {  
  outline: 2px solid #2563eb; /* border kabi: qalinlik uslub rang */  
}
```

`border` dan ikki asosiy farqi:

1. **Joy egallamaydi.** `border` box model'ning bir qismi — element o'lchamiga qo'shiladi va qo'shni elementlarni suradi. `outline` esa **layout'dan tashqarida** chiziladi — qo'shsa ham element kattalashmaydi, sahifa "sakramaydi".
2. **Yumaloqlanmaydi (eski xulq).** An'anaviy ravishda `outline` `border-radius` ni hisobga olmaydi (yangi brauzerlarda bu yaxshilangan, lekin tarixan to'rtburchak qolardi).

Nega `outline` muhim? Chunki u **klaviatura bilan fokus** (`:focus`) ni ko'rsatishning standart usuli. Tab tugmasi bilan harakatlanayotgan foydalanuvchi qaysi elementdaligini aynan outline orqali ko'radi.

⚠ **HECH QACHON shunday qilma:**

```
*:focus { outline: none; } /* ⚠ FALOKAT — accessibility'ni buzadi! */
```

Bu klaviatura foydalanuvchilarini "ko'r" qilib qo'yadi — ular qayerdaligini bilmaydi. Agar standart outline xunuk ko'rinsa, uni o'chirma — **chiroyliroq qil**:

```
.tugma:focus-visible {  
  outline: 3px solid #2563eb;  
  outline-offset: 2px; /* outline'ni elementdan biroz uzoqlashtiradi */  
}
```

outline-offset

`outline-offset` konturni elementdan **uzoqlashtiradi** (musbat) yoki ichiga **kirgizadi** (manfiy):

```
.a { outline: 2px solid #16a34a; outline-offset: 4px; } /* 4px tashqarida - "nafas oladi" */
.b { outline: 2px solid #16a34a; outline-offset: -6px; } /* ichkarida */
```

💡 `border` joyini surishi mumkin bo'lgani uchun fokus ko'rsatkichi sifatida yomon (sahifa sakraydi). `outline` aynan shu muammoni hal qiladi — fokus uchun **doim** `outline` ishlat.

13.14 filter — vizual effektlar (qisqacha)

`filter` xossasi elementga (rasm, fon, butun blok) grafik effektlar qo'llaydi — xuddi rasm tahrirlovchisidagi filtrlar kabi. Eng foydali ikkitasi:

```
.loyqa { filter: blur(4px); } /* loyqalashtirish */
.yorqin { filter: brightness(1.3); } /* yorqinlik: >1 yorqinroq, <1 qorong'i */
.qoraoq { filter: grayscale(100%); } /* qora-oq qiladi */
.kontrast { filter: contrast(1.5); } /* kontrastni oshiradi */
```

Bir nechta filtrni bo'sh joy bilan birlashtirish mumkin:

```
.rasm:hover {
  filter: brightness(1.1) contrast(1.05); /* sichqoncha tushganda jonlanadi */
}
```

💡 **Amaliy misol:** rasmni standart holatda qora-oq qilib, sichqoncha tushganda ranglantirish — galereyalarda mashhur effekt:

```
.galereya img {
  filter: grayscale(100%);
  transition: filter 0.3s; /* silliq o'tish (kelgusi bobda) */
}
.galereya img:hover {
  filter: grayscale(0%); /* asl ranglariga qaytadi */
}
```

✳️ `blur()` ning argumenti `px` da, qolganlari (`brightness`, `contrast`, `grayscale`) raqam yoki foiz bilan: `1` yoki `100%` = "o'zgarishsiz" asos.

⚠ Katta `blur()` qiymatlari yoki katta maydonlarga filtr qo'llash ishlashni (performance) sekinlashtirishi mumkin — me'yorida ishlat.

13.15 Hammasini birlashtirish — chiroyli kartochka

Endi bilimlarni bir joyga yig'amiz. Quyida zamonaviy "karta" komponenti — fon, yumaloq burchak va ko'p qatlamli soya birgalikda:

```
.karta {  
  background: linear-gradient(135deg, #ffffff, #f1f5f9); /* nozik  
gradient fon */  
  border: 1px solid #e2e8f0; /* yengil  
chegara */  
  border-radius: 16px; /* yumaloq  
burchak */  
  padding: 24px;  
  box-shadow: /* ikki  
qatlamli soya */  
    0 1px 2px rgba(0, 0, 0, 0.06),  
    0 10px 30px rgba(0, 0, 0, 0.10);  
}  
.karta:hover {  
  box-shadow: /*  
ko'tarilgandek effekt */  
    0 2px 4px rgba(0, 0, 0, 0.08),  
    0 16px 40px rgba(0, 0, 0, 0.16);  
}
```

```
<div class="karta">  
  <h3>Mahsulot nomi</h3>  
  <p>Ushbu kartochka fon, chegara, yumaloq burchak va soya birgalikda  
qanday ishlashini ko'rsatadi.</p>  
</div>
```

Natija: oq qog'oz varag'idek qalqib turgan, yumshoq soyali, yumaloq burchakli karta. Sichqoncha tushganda u biroz "ko'tariladi". Bu — fon, chegara va soyaning kuchi.

Mashqlar

Mashq 1 — Och fonli ogohlantirish qutisi

`.eslatma` klassiga och sariq fon (`#fef3c7`), chap tomonda 4px qalin amber chegara (`#f59e0b`) va 12px ichki padding bering.



Yechim



```
.eslatma {  
  background-color: #fef3c7;  
  border-left: 4px solid #f59e0b;  
  padding: 12px;  
}
```



Chap tomonda qalin urg'u chizig'i — eslatma/ogohlantirish qutilarining klassik dizayni.

Mashq 2 — Banner foni

`.banner` elementiga "rasmlar/tog.jpg" rasmini fon qiling: takrorlanmasin, markazda tursin va qutini to'liq qoplasin. Yedak rang `#1e293b` bersin.



Yechim



```
.banner {  
  background-color: #1e293b;  
  background-image: url("rasmlar/tog.jpg");  
  background-repeat: no-repeat;  
  background-position: center;  
  background-size: cover;  
}
```



Yoki qisqa yozuvda:

```
.banner {  
  background: #1e293b url("rasmlar/tog.jpg") no-repeat center / cover;  
}
```



Mashq 3 — Uch rangli gradient

`.spektr` elementiga chapdan o'ngga uch rangli linear gradient bering: qizil (`#dc2626`), amber (`#f59e0b`), yashil (`#16a34a`) — teng taqsimlangan.

 Yechim

```
.spektr {  
  background-image: linear-gradient(  
    to right,  
    #dc2626,  
    #f59e0b,  
    #16a34a  
  );  
}
```

Uch rang bo'lsa, brauzer ularni avtomatik teng (0%, 50%, 100%) taqsimlaydi — to'xtashlarni yozish shart emas.

Mashq 4 — Doira avatar

80x80px o'lchamdagi `.avatar` rasmni mukammal doira qiling. Rasm cho'zilmasligi kerak.

 Yechim

```
.avatar {  
  width: 80px;  
  height: 80px;  
  border-radius: 50%;  
  object-fit: cover;  
}
```

Kvadrat (eni = bo'yi) elementga `border-radius: 50%` — mukammal doira beradi. `object-fit: cover` rasmni cho'zmasdan kesib joylaydi.

Mashq 5 — Qalqib turgan tugma

`.tugma` ga ko'k fon (`#2563eb`), oq matn, 8px yumaloq burchak va tabiiy soya (gorizontal 0, pastga 3px, blur 8px, yarim shaffof qora) bering.

 **Yechim** 

```
.tugma {
  background-color: #2563eb;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 10px 20px;
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.2);
}
```

`offset-x: 0, offset-y: 3px` — yorug'lik tepadan kelganidek, soya pastda. Tabiiy ko'rinish shu.

Mashq 6 — Bosilgan (botgan) maydon

`.input` ga ichki soya (`inset`) qo'shing, shunda u "ichkariga botgan" ko'rinsin.

 **Yechim** 

```
.input {
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  padding: 8px;
  box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.08);
}
```

`inset` kalit so'zi soyani element **ichiga** soladi — kiritish maydonlari uchun chuqurlik hissi beradi.

Mashq 7 — Fokusni xavfsiz ko'rsatish

Quyidagi kod accessibility'ni buzadi. Nima xato va qanday tuzatasan?

```
.havola:focus { outline: none; }
```



Yechim



`outline: none` klaviatura foydalanuvchisi qaysi elementdaligini ko'rsatmaydi — bu **falokat**. Outline'ni o'chirish o'rniga uni chiroyliroq qilish kerak:

```
.havola:focus-visible {  
  outline: 3px solid #2563eb;  
  outline-offset: 2px;  
}
```



`outline` joy egallamagani uchun (border'dan farqli) fokus ko'rsatkichi sifatida ideal — sahifa "sakramaydi". `outline-offset` esa konturga "nafas olish" joyi beradi.

Mashq 8 — Ko'p qatlamli karta foni

`.banner` elementiga ikki qatlamli fon bering: tepada shaffof naqsh ("rasmlar/naqsh.png"), ostida tepadan pastga ko'k gradient (`#2563eb` dan `#1e3a8a` gacha).



Yechim



```
.banner {  
  background-image:  
    url("rasmlar/naqsh.png"),  
    linear-gradient(to bottom, #2563eb, #1e3a8a);  
}
```



Birinchi yozilgan qatlam (naqsh) **tepada**, ikkinchisi (gradient) ostida turadi. Shaffof joylaridan gradient ko'rinadi.

Mashq 9 (qiyin) — Pirog diagramma

`.pirog` elementini `conic-gradient` bilan teng to'rt bo'lakka bo'ling: ko'k, yashil, amber, qizil. Uni doira shaklida ko'rsating.



Yechim



```
.pirog {  
  width: 150px;  
  height: 150px;  
  border-radius: 50%;  
  background-image: conic-gradient(  
    #2563eb 0deg 90deg,  
    #16a34a 90deg 180deg,  
    #f59e0b 180deg 270deg,  
    #dc2626 270deg 360deg  
  );  
}
```

`conic-gradient` markaz atrofida aylangani uchun `border-radius: 50%` bilan birga mukammal pirog (pie chart) yasaydi. Har rangga ikki gradus bergani uchun chegaralar keskin chiqadi.

[← Oldingi: 12 — O'lchovlar, tipografiya va ranglar](#) · [🏠 README](#) · [Keyingi: 14 — Positioning va z-index →](#)

14 — Positioning va z-index

← Oldingi: 13 — Fon, chegara va soyalar · 🏠 README · Keyingi: 15 — Flexbox →

Bu bobda: elementlarni joylashtirish — `static`, `relative`, `absolute`, `fixed`, `sticky` — hamda qatlamlarni boshqaruvchi `z-index` va stacking context.

Shu paytgacha biz elementlar sahifada **o'z navbati bilan**, yuqoridan pastga tartib bilan joylashishini ko'rib keldik. Lekin haqiqiy saytlarda bizga ko'pincha bundan ko'prog'i kerak bo'ladi: tugma ustiga "yangi" yorlig'ini yopishtirish, skroll qilganda tepada qotib turadigan menyu, ekran o'rtasida paydo bo'ladigan modal oyna, sichqonchani olib borganda chiqadigan tooltip...

Bularning hammasi **positioning** (joylashtirish) bilan qilinadi. `position` xossasi elementni normal oqimdan "olib chiqib", uni sen xohlagan joyga qo'yish imkonini beradi. Bu bob seni eng sodda `position: static` dan to expert darajadagi stacking context nozikliklarigacha bosqichma-bosqich olib boradi.

💡 Positioning — bu CSS layout (joylashuv) vositalaridan biri. Asosiy layoutni Flexbox va Grid (keyingi boblarda) bilan quramiz; positioning esa alohida elementlarni "aniq nuqtaga" qo'yish yoki bir-biri ustiga qo'yish uchun kerak.

14.1 Avval: normal oqim (normal flow) nima?

`position` ni tushunish uchun avval brauzer elementlarni **standart** holatda qanday joylashtirishini eslab olaylik. Buni **normal oqim** (normal flow) deyiladi.

Normal oqimda:

- **Block elementlar** (`<div>`, `<p>`, `<h1>`, `<section>` ...) bir-birining **ostiga**, yuqoridan pastga tartib bilan tushadi. Har biri butun kenglikni egallaydi.
- **Inline elementlar** (`<span>`, `<a>`, `<strong>` ...) bir-birining **yoniga**, matn kabi chapdan o'ngga joylashadi.

```
<div>Birinchi blok</div>
<div>Ikkinchi blok</div>
<div>Uchinchi blok</div>
```

Natija: uchta blok bir-birining ostida, yuqoridan pastga ketma-ket turadi. Hech kim hech kim bilan ustma-ust tushmaydi — har biriga sahifada o'z joyi ajratiladi.

✦ Normal oqim — bu CSS'ning "asosiy holati". `position` xossasi aslida **shu oqimdan qanday chetga chiqishni** boshqaradi. Shuning uchun "element oqimda turibdimi yoki oqimdan chiqdimi?" degan savol bu bobning eng muhim savoli bo'ladi.

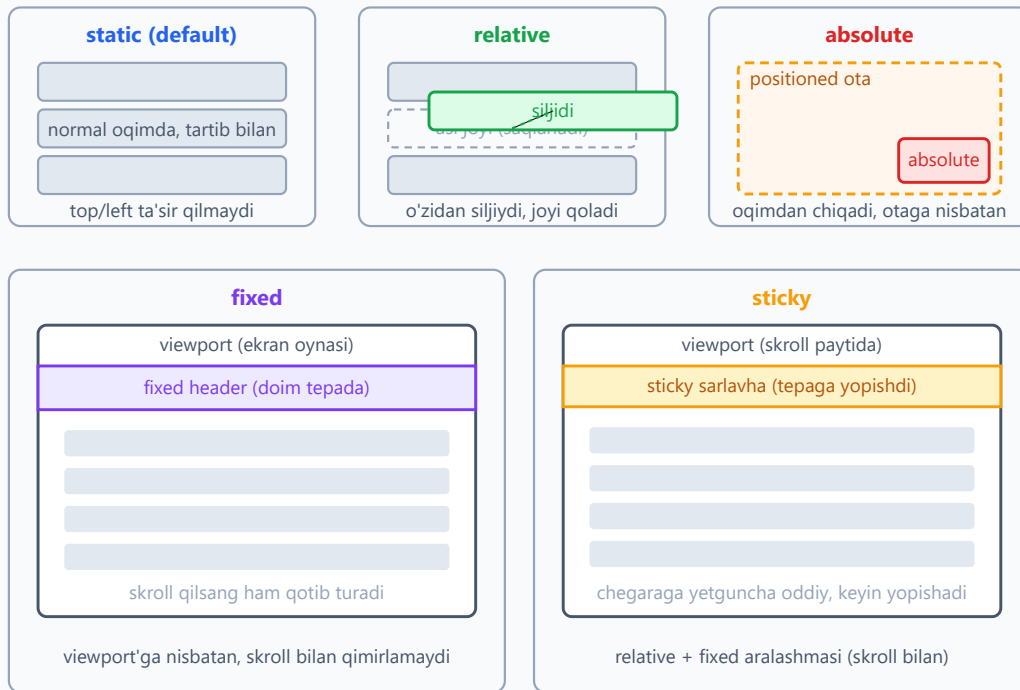
14.2 position xossasi va uning besh qiymati

`position` xossasi elementning qanday joylashishini belgilaydi. Beshta asosiy qiymati bor:

Qiymat	Oqimda qoladimi?	Nimaga nisbatan siljiydi?
<code>static</code>	Ha (default)	Hech nimaga — <code>top/left</code> ishlamaydi
<code>relative</code>	Ha (joyi saqlanadi)	O'zining asl o'rniga
<code>absolute</code>	Yo'q (oqimdan chiqadi)	Eng yaqin positioned ota'ga
<code>fixed</code>	Yo'q (oqimdan chiqadi)	Viewport'ga (ekran oynasiga)
<code>sticky</code>	Ha, keyin yopishadi	Skroll konteyneriga (chegaraga)

Quyidagi diagramma beshtasi sahifada qanday joylashishini ko'rsatadi:

position'ning besh turi: element qayerga joylashadi?



Bu qiymatlarning to'rttasi (`relative`, `absolute`, `fixed`, `sticky`) `top`, `right`, `bottom`, `left` xossalari bilan birga ishlaydi. Endi har birini alohida, sodda misollar bilan ochamiz.

14.3 static — standart holat

`static` — bu **har bir elementning standart** (default) qiymati. Sen `position` ni umuman yozmasang, element `static` bo'ladi.

```
.quti {  
  position: static; /* aslida yozishning hojati yo'q — bu allaqachon  
  default */  
}
```

Static element **normal oqimda** turadi va eng muhimi:

⚠ Static elementda `top`, `right`, `bottom`, `left` va `z-index` xossalari **umuman ishlamaydi**. Ularni yozsang ham brauzer e'tiborsiz qoldiradi.

Nega bilish kerak? Ko'p boshlovchi `top: 20px` yozadi-yu, "element qimirlamayapti" deb hayron bo'ladi. Sababi oddiy: element hali `static` holatida — uni avval boshqa `position` qiymatiga o'tkazish kerak.

💡 Qoidani shunday eslang: **siljish xossalari** (`top/left/...`) **faqat element** `static` bo'lmaganda ishlaydi.

14.4 relative — o'zidan siljish

`position: relative` element uchun ikki narsani anglatadi:

1. Element **hali ham normal oqimda** — uning asl joyi saqlanib qoladi (boshqa elementlar uning bo'sh joyiga kirmaydi).
2. Endi `top/right/bottom/left` bilan elementni **o'zining asl o'rnidan** siljitish mumkin.

```
.belgi {  
  position: relative;  
  top: 15px;    /* yuqoridan 15px pastga suriladi */  
  left: 30px;   /* chapdan 30px o'ngga suriladi */  
}
```

Natija: element o'z joyidan pastga va o'ngga "ko'chadi", lekin u qoldirgan **bo'sh joy o'sha yerda turadi** — boshqa elementlar siljimaydi.

✦ Diqqat qil: `top: 15px` "element yuqorisi 15px bo'lsin" degani EMAS. U "elementni o'z joyidan 15px pastga sur" degani. Ya'ni `top` — bu **yuqoridan beriladigan turtki**. Shuning uchun musbat `top` elementni pastga, musbat `left` o'ngga suradi.

`relative` ning ikki asosiy ishlatilishi bor:

- **Kichik siljitish** — elementni biroz "to'g'rilash" uchun (yuqoridagi misoldek).
- **Tayanch (anchor) bo'lish** — bu eng muhim ishlatilishi! Ichidagi `absolute` bola element shu `relative` ota'ga nisbatan joylashadi. Buni keyingi bo'limda ko'ramiz.

💡 Amalda `relative` ni eng ko'p ikkinchi maqsadda — "ichimdagi absolute element men ichimda qolsin" deb ishlatasan, hatto uni o'zini siljitmasang ham.

14.5 absolute — oqimdan chiqib, ota'ga nisbatan

`position: absolute` eng kuchli va eng ko'p chalkashlik tug'diradigan qiymat. Uni tushunsang, positioning'ning yarmini bilding.

`absolute` element:

1. **Normal oqimdan butunlay chiqadi** — u qoldirgan joy yo'qoladi, boshqa elementlar go'yo u yo'qdek joylashadi.
2. **Eng yaqin "positioned" ota'ga nisbatan** joylashadi (bu tushunchani pastda ochamiz).
3. `top/right/bottom/left` bilan o'sha ota'ning ichida aniq joyga qo'yiladi.

```
<div class="ota">
  <span class="nishon">YANGI</span>
  Mahsulot kartochkasi
</div>
```

```
.ota {
  position: relative; /* MUHIM: tayanch bo'lish uchun */
  width: 300px;
  height: 200px;
}
.nishon {
  position: absolute;
  top: 10px; /* ota'ning tepasidan 10px */
  right: 10px; /* ota'ning o'ngidan 10px */
  background: #dc2626;
  color: white;
  padding: 4px 8px;
}
```

Natija: "YANGI" yorlig'i kartochkaning o'ng yuqori burchagiga aniq yopishadi. Kartochka hajmi o'zgarsa ham, yorliq doim o'ng yuqorida qoladi.

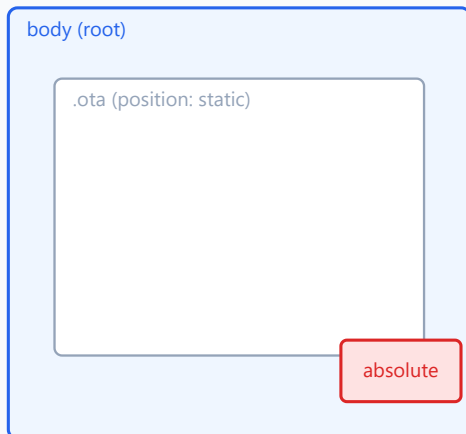
"Positioned ancestor" (positioned ota) nima?

Bu — `absolute` ni tushunishdagi eng muhim tushuncha. `absolute` element joylashish uchun "tayanch" izlaydi: u yuqoriga, ota-bobolari (ancestors) tomon qarab, **position qiymati static bo'lmagan** birinchi ota'ni topadi. Mana shu ota **positioned ancestor** deyiladi.

- Agar bunday ota topilsa — `absolute` element **o'shanga** nisbatan joylashadi.
- Agar **hech qaysi** ota positioned bo'lmasa — element `<body>` ga (aniqrog'i, dastlabki konteynerga / viewport'ga) nisbatan joylashadi.

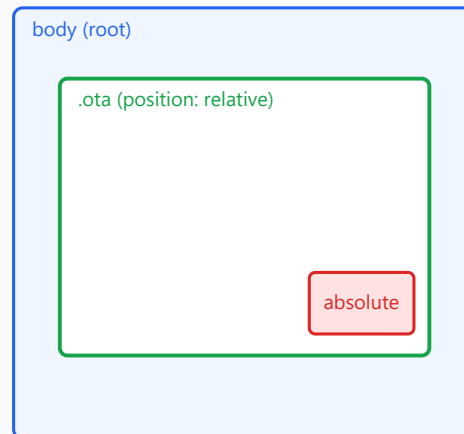
absolute element kimga nisbatan joylashadi?

Ota positioned EMAS



positioned ota yo'q -> body'ga nisbatan

Ota positioned (relative)



eng yaqin positioned ota'ga nisbatan

Maslahat: ota'ga `position: relative` ber — bola `absolute` uni "tayanch" qilib oladi.

⚠ Eng ko'p uchraydigan xato: ota'ga `position: relative` berishni unutish. Natijada `absolute` bola butun sahifa burchagiga "qochib" ketadi. Yodda tut: **absolute bola istasang, ota'ga `position: relative` ber.**

💡 "Positioned" so'zi nimani anglatadi? — `position` qiymati `relative`, `absolute`, `fixed` yoki `sticky` bo'lgan har qanday element. Faqat `static` "positioned emas". Amalda tayanch sifatida deyarli doim `relative` ishlatiladi, chunki u ota'ning o'z joyini o'zgartirmaydi.

14.6 fixed — viewport'ga yopishish

`position: fixed` `absolute` ga juda o'xshaydi — u ham oqimdan chiqadi. Farqi: u **har doim viewport'ga** (brauzer oynasining ko'rinadigan qismiga) nisbatan joylashadi va **skroll qilganda qimirlamaydi**.

```
.tepa-menyu {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  background: #2563eb;  
  color: white;  
}
```

Natija: menyu sahifaning eng tepasiga "qotib" qoladi. Foydalanuvchi pastga skroll qilsa ham, menyu doim ekran tepasida ko'rinib turadi.

`fixed` ning tipik ishlatilishlari:

- Doim ko'rinib turadigan tepa menyu (navbar).
- Pastki burchakda "tepaga qaytish" tugmasi.
- Cookie bildirishnomasi yoki chat tugmasi.

⚠ `fixed` element oqimdan chiqadi, shuning uchun u o'z ortidagi mazmunni **berkitib qo'yishi** mumkin. Masalan, balandligi 60px bo'lgan `fixed` menyu sahifa boshidagi matnni yopadi. Yechim: `<body>` ga shuncha bo'shliq berish, masalan `body { padding-top: 60px; }`.

✳ Nozik nuqta: agar ota elementga `transform`, `filter` yoki `perspective` xossasi berilgan bo'lsa, `fixed` element viewport'ga emas, **o'sha ota'ga** nisbatan joylashib qoladi. Bu kutilmagan xatolarning tez-tez uchraydigan sababi.

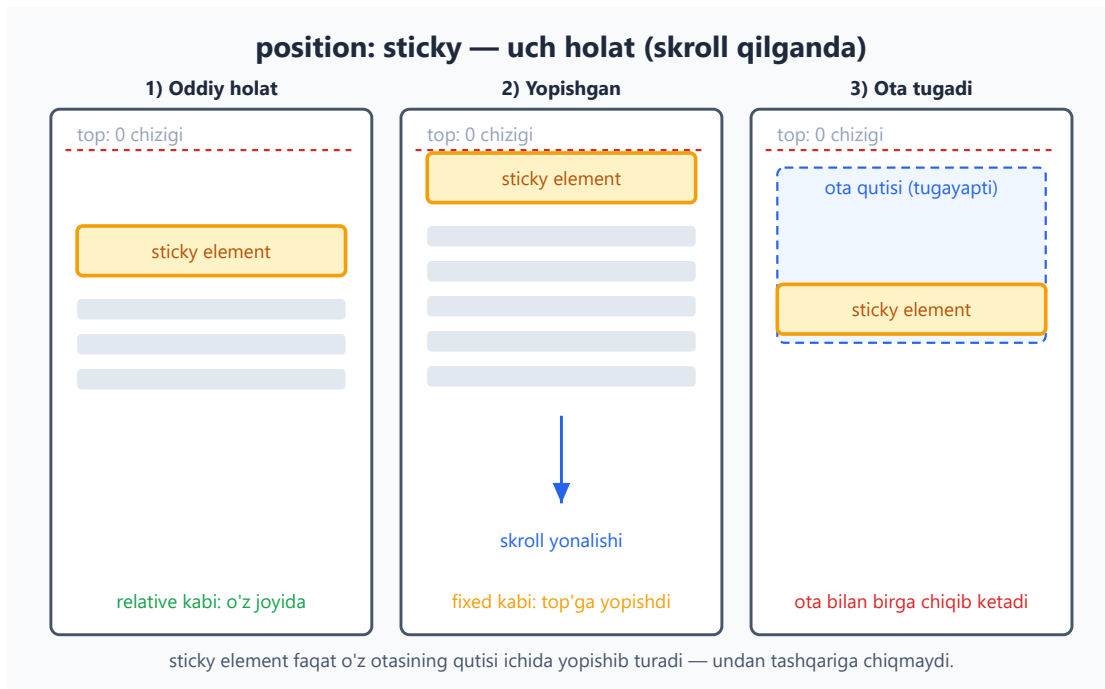
14.7 sticky — skroll bilan yopishadigan element

`position: sticky` — eng "aqlli" qiymat. U `relative` va `fixed` ning aralashmasi kabi ishlaydi:

- Element odatda **normal oqimda**, o'z joyida turadi (`relative` kabi).
- Skroll qilganda u belgilangan **chegaraga** (masalan `top: 0`) yetganda — o'sha yerga **yopishib** qoladi va keyin `fixed` kabi qotib turadi.
- Otasining qutisi tugaganda — element ota bilan birga skroll bo'lib chiqib ketadi.

```
.bolim-sarlavhasi {  
  position: sticky;  
  top: 0; /* tepaga yetganda yopishadi */  
  background: #f59e0b;  
}
```

Natija: bo'lim sarlavhasi skroll bilan birga yuqoriga ko'tariladi, ekran tepasiga yetganda yopishadi va keyingi bo'lim kelguncha o'sha yerda turadi.



⚠️ `sticky` ishlashi uchun `top`, `bottom`, `left` yoki `right` dan kamida bittasi berilishi **shart**. `top: 0` bermasdan faqat `position: sticky` yozsang, hech narsa yopishmaydi.

⚠️ Yana bir tuzoq: agar `sticky` elementning **otasida** `overflow: hidden`, `overflow: auto` yoki `overflow: scroll` bo'lsa, yopishish ko'pincha **ishlamaydi**. Bu boshlovchilarni ko'p qiynaydigan xato — `sticky` "nega ishlamayapti" desang, avval ota elementlarning `overflow` ini tekshir.

💡 Tipik ishlatilishlari: alifbo bo'yicha ro'yxatdagi harf sarlavhalari, jadval ustun sarlavhasi (`<thead>`), uzun sahifadagi bo'lim sarlavhalari, yon menyu.

14.8 z-index va qatlamlanish (stacking)

`position` elementlarni bir-birining ustiga qo'yishi mumkin. Unda **qaysi biri oldinda** ko'rinadi? Buni `z-index` boshqaradi.

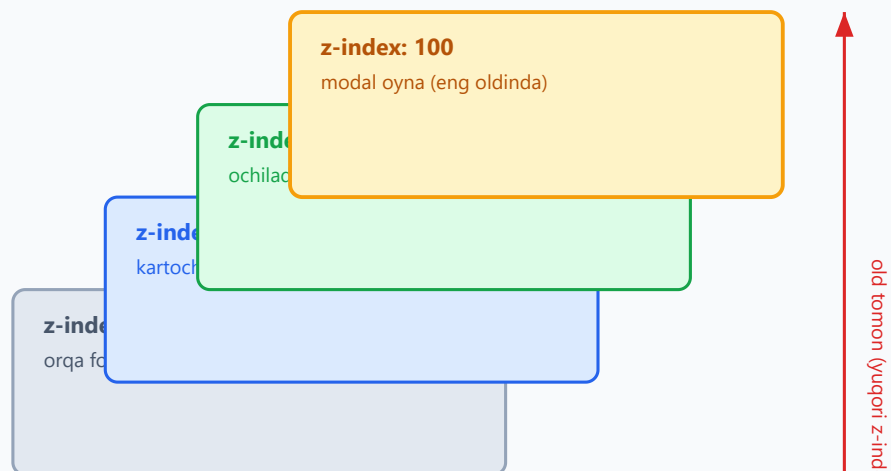
Ekranni 3D deb tasavvur qil: `x` o'qi — chap/o'ng, `y` o'qi — yuqori/past, `z` o'qi esa **senga tomon** — ya'ni "chuqurlik". `z-index` qancha katta bo'lsa, element shuncha **oldinda** (senga yaqin) turadi.

```
.orqa {  
  position: absolute;  
  z-index: 1;      /* pastki qatlam */  
}  
.old {
```

```
position: absolute;
z-index: 10;      /* yuqori qatlam - .orqa ustiga tushadi */
}
```

z-index: kim kimning ustida turadi?

Katta z-index = yuqori qatlam = old tomonda (foydalanuvchiga yaqin)



z-index faqat positioned (static'dan boshqa) elementlarga ishlaydi. Teng bo'lsa — keyin yozilgani ustun.

z-index haqida bilish shart bo'lgan qoidalar:

- z-index **faqat positioned elementlarga** ishlaydi (`static` dan boshqa — `relative/absolute/fixed/sticky`). Static elementda u e'tiborsiz qoladi.
- Qiymat — butun son: musbat (`10`), `0` , yoki manfiy (`-1`) bo'lishi mumkin. Katta son = oldinda.
- Agar z-index berilmasa yoki teng bo'lsa — **HTML'da keyin yozilgan** element ustun bo'ladi (manba tartibi).

📌 **Maslahat:** `z-index: 9999999` kabi ulkan sonlar yozma. Loyihada izchil tizim tut: masalan `1` (content) → `10` (dropdown) → `100` (sticky header) → `1000` (modal). Bu keyinroq "qatlam urushi" muammosidan saqlaydi.

14.9 Stacking context — z-index'ning "yashirin qoidasi"

Bu bobning eng expert qismi. Ko'p tajribali dasturchi ham bu yerda qoqiladi.

Muammo: ba'zan elementga `z-index: 9999` berasan, lekin u baribir boshqa `z-index: 1` element ostida qoladi. Nega?! Javob — **stacking context** (qatlamlanish konteksti) tushunchasida.

Stacking context — bu elementlarning alohida, mustaqil "qatlamlar guruhi". Har bir kontekst o'z ichida alohida saralanadi. Eng muhimi:

Bola elementning `z-index` i faqat **o'z stacking context'i ichida** taqqoslanadi. U tashqaridagi elementlar bilan to'g'ridan-to'g'ri raqobatlasholmaydi.

Hayotiy analogiya: ikkita bino bor. Birinchi binoning 50-qavatidagi odam, ikkinchi binoning 2-qavatidagi odamdan **balandroq emas**, agar birinchi bino umuman pastroq tepalikda joylashgan bo'lsa. "Qavat raqami" (`z-index`) faqat **o'z binosi** (stacking context) ichida ma'noga ega. Binolarning o'zaro tartibi esa boshqacha hal qilinadi.

Yangi stacking context qachon ochiladi?

Element quyidagilardan birortasiga ega bo'lsa, u **yangi stacking context** yaratadi (ro'yxat to'liq emas, eng muhimlari):

- `position: relative/absolute` + `z-index` qiymati (`auto` dan boshqa).
- `position: fixed` yoki `position: sticky` (`z-index`'siz ham).
- `opacity` 1 dan kichik (masalan `opacity: 0.9`).
- `transform`, `filter`, `perspective`, `clip-path` (qiymati `none` dan boshqa).
- `will-change` ba'zi qiymatlar bilan.
- Flex yoki grid bolasida `z-index` berilgan bo'lsa.

```
.ota {
  opacity: 0.99;      /* bu YANGI stacking context ochadi! */
}
.ota .bola {
  position: absolute;
  z-index: 9999;      /* lekin bu faqat .ota ichida ma'noga ega */
}
```

⚠ Yuqoridagi misolda `.bola` ning `z-index: 9999` i `.ota` dan tashqaridagi elementlardan ustun bo'la **olmaydi** — chunki `.ota` o'zining `opacity` tufayli alohida stacking context yaratdi va `.bola` shu "qutiga qamalib" qoldi.

💡 Shuning uchun "z-index'im ishlamayapti" muammosida birinchi tekshiriladigan narsa: **ota elementlardan birortasi stacking context yaratmadimi** (ayniqsa `opacity`, `transform`, yoki `filter` tufayli)? Brauzer DevTools (F12) buni tekshirishga yordam beradi.

14.10 float va clear (legacy — eski layout usuli)

Tarixiy bilim sifatida `float` ni ham qisqacha ko'rib o'tamiz. `float` dastlab gazetadagidek **matnni rasm atrofiga oqizish** uchun yaratilgan edi.

```
img {  
  float: left;          /* rasm chapga suriladi, matn uning o'ngidan  
  oqadi */  
  margin-right: 15px;  
}
```

Natija: rasm chapga "suzadi", undan keyingi matn rasmning o'ng yonidan va ostidan o'rab oqadi — xuddi jurnaldagidek.

`float: left` yoki `float: right` element atrofidan matn oqishiga ruxsat beradi. `clear` esa buning teskarisi — "men float elementlar yonidan emas, ularning **ostidan** boshlanay" deydi:

```
.toza {  
  clear: both;          /* chap va o'ng float'larning ostiga tushadi */  
}
```

Nega bu "legacy" (eski)?

Flexbox va Grid paydo bo'lishidan **oldin** (2017 yilgacha) dasturchilar butun sahifa layoutini — ustunlar, panellar, gazeta tartibi — `float` bilan qurishga majbur edi. Bu juda noqulay edi:

- Float qilingan elementlar otasini "balandligini yo'qotishga" majbur qilardi (collapse muammosi), buni hal qilish uchun `clearfix` degan hiyla ishlatilardi.
- Vertikal markazlash, teng balandlikdagi ustunlar — bularning hammasi azob edi.

⚠ Bugun **layout uchun `float` ishlatma**. Ustunlar va umumiy joylashuv uchun **Flexbox yoki Grid** to'g'ri vosita (keyingi boblar). `float` ni faqat asl maqsadida — matn ichida rasmni oqizish uchun — qoldir.

✳ Bu bo'limni o'qib qo'ydingmi — yetarli. Eski kodlarda `float` layoutini uchratsang, "bu Flexbox'gacha bo'lgan usul ekan" deb tanib olasan, xolos.

14.11 Amaliy misol 1: yopishqoq header (sticky navbar)

Skroll qilganda tepada qotib turadigan menyu — eng ko'p kerak bo'ladigan naqsh. Buni ikki usulda qilish mumkin: `fixed` yoki `sticky`. Hozir `sticky` ni ko'ramiz — u ko'pincha qulayroq, chunki bo'shliq muammosini o'zi hal qiladi.

```
<header class="navbar">Mening saytim</header>
<main>
  <p>Juda uzun matn... (skroll qilish uchun)</p>
  <!-- ... ko'p matn ... -->
</main>
```

```
.navbar {
  position: sticky;
  top: 0; /* tepaga yetganda yopishadi */
  z-index: 100; /* boshqa mazmun ustida turadi */
  background: #2563eb;
  color: white;
  padding: 16px;
}
```

Natija: menyu avval sahifaning eng tepasida turadi; pastga skroll qilganda ekran tepasiga yopishib, doim ko'rinib turadi. `z-index: 100` esa skroll qilinadigan mazmun menyu ustiga "chiqib ketmasligini" ta'minlaydi.

💡 Nega `sticky` `fixed` dan qulay? — `sticky` element o'z joyini oqimda saqlaydi, shuning uchun ostidagi mazmun ostiga "kirib ketmaydi" — `fixed` dagidek qo'shimcha `padding-top` berish shart emas.

14.12 Amaliy misol 2: tooltip (sichqoncha izohi)

Tooltip — element ustiga sichqoncha kelganda chiqadigan kichik izoh. Bu `relative` ota + `absolute` bola naqshining klassik namunasi.

```
<span class="tip">
  Narx
  <span class="tip-matn">Soliq bilan birga hisoblangan</span>
</span>
```

```
.tip {
  position: relative; /* tayanch: absolute bola shu yerga nisbatan */
  cursor: help;
}
.tip-matn {
  position: absolute;
  bottom: 130%; /* matnning tepasida chiqadi */
  left: 50%;
  transform: translateX(-50%); /* gorizontal markazga to'g'rileydi */
  background: #1e293b;
  color: white;
  padding: 6px 10px;
```

```
border-radius: 6px;
white-space: nowrap;
visibility: hidden;          /* odatda yashirin */
z-index: 10;
}
.tip:hover .tip-matn {
  visibility: visible;       /* sichqoncha kelganda ko'rinadi */
}
```

Natija: "Narx" so'zi ustiga sichqonchani olib borsang, ustida qora fonli izoh chiqadi. Izoh "Narx" so'ziga nisbatan markazlashtirilgan, chunki ota `relative`, bola `absolute`.

✦ `transform: translateX(-50%)` hiylasi: `left: 50%` elementning **chap chetini** markazga qo'yadi; `translateX(-50%)` esa uni o'z kengligining yarmiga chapga suradi — natijada element **markazi** to'g'ri keladi. Bu markazlashtirishning juda ko'p ishlatiladigan usuli.

14.13 Amaliy misol 3: modal oyna (overlay bilan)

Modal — ekran o'rtasida paydo bo'lib, ortidagi mazmunni qoramtir parda (overlay) bilan to'sadigan oyna. Bu `fixed` va `z-index` ning eng yaxshi namunasi.

```
<div class="overlay">
  <div class="modal">
    <h2> Tasdiqlaysizmi? </h2>
    <p> Bu amalni bekor qilib bo'lmaydi. </p>
    <button> Ha </button>
  </div>
</div>
```

```
.overlay {
  position: fixed;          /* butun viewport'ni qoplaydi */
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5); /* yarim shaffof qora parda */
  display: flex;
  align-items: center;      /* modalni vertikal markazga */
  justify-content: center; /* gorizontal markazga */
  z-index: 1000;           /* hamma narsadan tepada */
}
.modal {
  background: white;
  padding: 24px;
  border-radius: 12px;
```

```
max-width: 400px;  
}
```

Natija: butun ekranni qoplagan yarim shaffof parda, uning o'rtasida oq modal oyna. `z-index: 1000` modalni sahifadagi har qanday elementdan tepada turishini kafolatlaydi.

💡 Bu yerda ikki texnika birlashdi: `position: fixed` + `width/height: 100%` butun ekranni qoplaydi, Flexbox (`align-items` + `justify-content: center`) esa modalni aniq markazga qo'yadi. `z-index` esa qatlam tartibini boshqaradi.

⚠️ Yodda tut (14.9 bo'limini esla): modal `<body>` ga to'g'ridan-to'g'ri yaqin joyda turishi yaxshi. Agar uni `transform` yoki `opacity` ishlatilgan ota ichiga qo'ysang, yangi stacking context tufayli `z-index: 1000` baribir ishlamay qolishi mumkin.

14.14 Tez-tez uchraydigan xatolar (xulosa)

Bu boblardagi eng ko'p uchraydigan tuzoqlarni bir joyga to'pladik — saqlab qo'y:

Xato	Sabab	Yechim
<code>top: 20px</code> ishlamayapti	Element hali <code>static</code>	<code>position</code> ni <code>relative/absolute/...</code> qil
<code>absolute</code> bola sahifa burchagiga qochib ketdi	Ota positioned emas	Ota'ga <code>position: relative</code> ber
<code>sticky</code> yopishmayapti	<code>top</code> berilmagan	<code>top: 0</code> (yoki boshqa chegara) qo'sh
<code>sticky</code> baribir ishlamayapti	Ota'da <code>overflow: hidden/auto</code>	Ota'dagi <code>overflow</code> ni olib tashla
<code>z-index: 9999</code> baribir ostda	Ota yangi stacking context yaratgan	Ota'dagi <code>opacity/transform/filter</code> ni tekshir
<code>fixed</code> element noto'g'ri joyda	Ota'da <code>transform</code> bor	Ota'dan <code>transform</code> ni olib tashla yoki boshqa joyga ko'chir
<code>fixed</code> menyu mazmunni yopib qo'ydi	Oqimdan chiqdi	<code><body></code> ga shuncha <code>padding-top</code> ber

Mashqlar

Quyidagi mashqlarni o'zing yozib, brauzerda sinab ko'r. Har birida "nega shunday bo'ldi?" deb o'ylab ko'rishni unutma.

Mashq 1 — static tuzoq

Quyidagi kod nima uchun elementni siljitmaydi? Tuzat.

```
.quti {  
  top: 40px;  
  left: 40px;  
}
```

Yechim

Element `static` holatida (default), shuning uchun `top/left` ishlamaydi. `position` qiymatini qo'shish kerak:

```
.quti {  
  position: relative; /* endi top/left ishlaydi */  
  top: 40px;  
  left: 40px;  
}
```

`relative` da element o'z joyidan siljiydi, lekin asl bo'sh joyi saqlanadi.

Mashq 2 — Burchakka nishon yopishtirish

Bir `<div class="kartochka">` ning **o'ng yuqori burchagiga** "-20%" chegirma yorlig'ini joylashtir. Kartochka hajmi o'zgarsa ham yorliq o'ng yuqorida qolsin.

 **Yechim** 

```
<div class="kartochka">
  <span class="chegirma">-20%</span>
  Mahsulot nomi
</div>
```

```
.kartochka {
  position: relative; /* tayanch */
  width: 250px;
  padding: 20px;
  border: 1px solid #94a3b8;
}

.chegirma {
  position: absolute;
  top: 8px;
  right: 8px;
  background: #dc2626;
  color: white;
  padding: 2px 8px;
  border-radius: 4px;
}
```

Kalit: ota'ga `position: relative`, bola'ga `position: absolute` + `top/right`.

Mashq 3 — fixed "tepaga qaytish" tugmasi

Sahifaning **pastki o'ng burchagida** doim ko'rinib turadigan "↑" tugmasini yarat. Skroll qilganda u joyida qolsin.

 **Yechim** 

```
.tepaga {
  position: fixed;
  bottom: 20px;
  right: 20px;
  width: 48px;
  height: 48px;
  border-radius: 50%;
  background: #2563eb;
  color: white;
  z-index: 100;
}
```

`fixed` + `bottom/right` tugmani viewport burchagiga "qotirib" qo'yadi. `z-index` uni boshqa mazmun ustida ushlab turadi.

Mashq 4 — sticky bo'lim sarlavhasi

Uzun ro'yxatda har bo'lim sarlavhasi skroll qilganda tepaga yopishsin (alifbo ro'yxatidagidek). Kerakli CSS ni yoz.

 **Yechim** 

```
.bolim-sarlavha {  
  position: sticky;  
  top: 0;  
  background: #f8fafc;  
  padding: 8px 12px;  
  z-index: 5;  
}
```

⚠ top: 0 shart — busiz yopishmaydi. Va sarlavhaning hech bir ota'sida overflow: hidden/auto bo'lmagligiga ishonch hosil qil, aks holda ishlamaydi.

Mashq 5 — z-index tartibini to'g'rilash

Quyidagi kodda `.menu` `.banner` ostida qolib ketgan. `.menu` ni ustiga chiqar.

```
.banner { position: relative; z-index: 5; }  
.menu { position: absolute; }
```

 **Yechim** 

`.menu` da `z-index` yo'q, shuning uchun u manba tartibiga bog'liq. Uni `.banner` dan katta qilamiz:

```
.banner { position: relative; z-index: 5; }  
.menu { position: absolute; z-index: 10; } /* endi ustida */
```

`z-index` faqat positioned elementga ishlaganini esla — `.menu` `absolute` bo'lgani uchun bu ishlaydi.

Mashq 6 — Stacking context jumbog'i

`.bola` ga `z-index: 9999` berilgan, lekin u baribir `.boshqa` (`z-index: 2`) ostida qolyapti. Quyidagi koddan sababini top:

```
.ota { position: relative; opacity: 0.9; }  
.bola { position: absolute; z-index: 9999; }  
.boshqa { position: absolute; z-index: 2; }
```




Yechim



`.ota` da `opacity: 0.9` bor — bu **yangi stacking context** yaratadi. Shuning uchun `.bola` ning `z-index: 9999` i faqat `.ota` ichida ma'hoga ega; `.ota` dan tashqaridagi `.boshqa` bilan to'g'ridan-to'g'ri raqobatlasholmaydi. Agar `.ota` ning butun konteksti `.boshqa` dan past bo'lsa, `.bola` qancha katta `z-index` olmasin, ostda qoladi.

Yechim: `.ota` dan `opacity` ni olib tashlash (yoki `.ota` ning o'ziga to'g'ri `z-index` berish):

```
.ota { position: relative; } /* opacity olib tashlandi -> alohida kontekst  
yo'q */
```

Mashq 7 — relative bilan kichik to'g'rilash

Matn ichidagi yulduzcha belgisini (`*`) boshqa belgilarni qimirlatmasdan 4px yuqoriga ko'tar.



Yechim



```
.yulduzcha {  
  position: relative;  
  top: -4px; /* manfiy top -> yuqoriga */  
}
```

`relative` element o'z joyini oqimda saqlaydi, shuning uchun atrofidagi matn siljmaydi — faqat yulduzchani o'zi 4px yuqoriga ko'chadi.

Mashq 8 — Markazlashtirilgan modal

`fixed` va `transform` yordamida (Flexbox'siz) modal oynani ekran **aniq markaziga** qo'y.



Yechim



```
.modal {  
  position: fixed;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%); /* o'z o'lchamining yarmiga orqaga */  
  background: white;  
  padding: 24px;  
  z-index: 1000;  
}
```

top/left: 50% modalning **chap-yuqori burchagini** ekran markaziga qo'yadi;
translate(-50%, -50%) esa uni o'z eni va bo'yining yarmiga orqaga suradi — natijada modal **markazi** ekran markaziga to'g'ri keladi. Bu Flexbox'siz markazlashtirishning klassik usuli.

15 — Flexbox

← Oldingi: 14 — Positioning va z-index · 🏠 README · Keyingi: 16 — CSS Grid →

Bu bobda: bir o'lchovli layout vositasi Flexbox bilan tanishamiz — flex konteyner, ikki o'q (main va cross) va elementlarni shu o'qlar bo'ylab tekislash hamda taqsimlashni 0 dan o'rganamiz.

15.1 Flexbox nima va u qanday muammoni yechadi?

Tasavvur qil: sahifaning yuqorisida navbar (navigation bar) yasamoqchisan — chap tomonda logotip, o'ng tomonda menyu havolalari. Yoki bitta tugmani ekranning aynan o'rtasiga — ham gorizontal, ham vertikal — joylashtirmoqchisan. Eski CSS'da bularning hammasi azob edi: `float`, `display: inline-block`, sehrli `margin` hiylalari... Ayniqsa **vertikal markazlash** boshlovchilar uchun haqiqiy bosh og'rig'i bo'lgan.

Flexbox (to'liq nomi *Flexible Box Layout*) — aynan shu muammolarni hal qilish uchun yaratilgan zamonaviy CSS layout tizimi. Uning bitta vazifasi bor: **elementlar guruhini bitta o'q (yo'nalish) bo'ylab tartibga solish** — ularni tekislash, oralarini bo'lish va bo'sh joyni aqlli taqsimlash.

📌 **Atama:** *flex* inglizcha "egiluvchan, moslashuvchan" degani. Nomi shundan: flex elementlari mavjud joyga qarab "egiladi" — kerak bo'lsa cho'ziladi, kerak bo'lsa siqiladi.

💡 **Hayotiy analogiya:** flexbox'ni javondagi kitoblar deb tasavvur qil. Sen "kitoblarni chapga tort", "teng oraliq bilan yoy" yoki "hammasini cho'zib javonni to'ldir" deb buyruq berasan — flexbox aynan shuni qiladi. Sen har bir kitobning aniq joyini hisoblab o'tirmaysan; faqat *qoidani* aytasan, joylashtirishni flexbox o'zi bajaradi.

"Bir o'lchovli" nimani anglatadi?

Bu eng muhim tushuncha. Flexbox **bir o'lchovli** (one-dimensional) tizim: u elementlarni **bir vaqtning o'zida faqat bitta yo'nalishda** — yo **qator** (gorizontal), yo **ustun** (vertikal) — tartibga soladi.

- Navbar, tugmalar qatori, teglar ro'yxati — bir qator -> **flexbox**.
- Butun sahifa maketi (grid: ustunlar VA qatorlar bir vaqtda) — ikki o'lcham -> bu keyingi bobdagi **CSS Grid** ishi.

⚠️ Ko'p boshlovchi xato qiladi: butun murakkab sahifa tarmog'ini faqat flexbox bilan qurmoqchi bo'lib, qiynaladi. Eslab qol: **flexbox = bir qator yoki bir ustun**. Ikki o'lchovli

to'g'ri kerak bo'lsa, Grid ishlatasan.

15.2 Birinchi flex konteyner: `display: flex`

Flexbox'ni yoqish uchun atigi bitta qator kerak — ota elementga `display: flex` berasan. Shu zahoti uning **bevosita farzandlari** flex itemlarga aylanadi.

```
<div class="qator">
  <div class="quti">A</div>
  <div class="quti">B</div>
  <div class="quti">C</div>
</div>
```

```
.qator {
  display: flex;
}
.quti {
  background: #dbeafe;
  padding: 20px;
  border: 2px solid #2563eb;
}
```

Natija: A, B, C qutilar endi tepada-tepada (ustma-ust) emas, balki **yonma-yon** — bitta qatorda joylashadi.

Nega? Chunki `<div>` odatda **block** element — har biri butun qatorni egallab, keyingisini pastga suradi. `display: flex` esa ota'ni flex konteynerga aylantiradi va uning farzandlarini standart holatda **gorizontal qatorga** teradi.

📌 **Eng muhim atamalar — yodda tut:**

Atama	Bu nima
Flex konteyner (container)	<code>display: flex</code> berilgan ota element.
Flex item	Konteynerning bevosita farzandi (har bir bola).

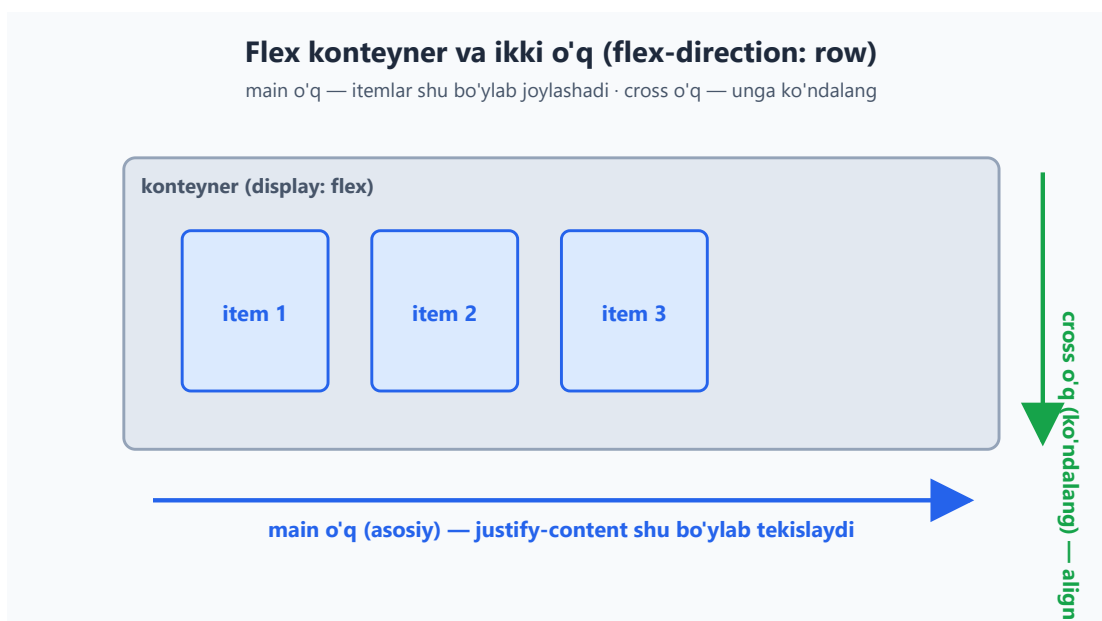
⚠️ Faqat **bevosita** farzandlar flex item bo'ladi. Agar `.quti` ichida yana element bo'lsa, u **nabira** — flexga bo'ysunmaydi (toki u ham flex konteyner bo'lmaguncha).

💡 `display: inline-flex` ham bor: u konteynerni inline element kabi tutadi (atrofidagi matn bilan bir qatorda turadi), lekin ichida o'sha flex qoidalari ishlaydi. Ko'pincha `flex` ishlatasan; `inline-flex` kamroq kerak bo'ladi.

15.3 Ikki o'q: main o'q va cross o'q

Flexbox'ning butun mantig'i **ikki o'q** (axis) tushunchasiga asoslanadi. Buni tushunsang, qolgan hamma narsa joyiga tushadi.

- **Main o'q** (main axis, asosiy o'q) — itemlar **shu yo'nalish bo'ylab** teriladi. Standart holatda gorizontal (chapdan o'ngga).
- **Cross o'q** (cross axis, ko'ndalang o'q) — main o'qqa **perpendikulyar** (ko'ndalang) o'q. Standartda vertikal.



Nega bu shunchalik muhim? Chunki flexbox'dagi deyarli har bir xossa shu ikki o'qdan biriga "ulangan":

- `justify-content` — itemlarni **main o'q** bo'ylab joylashtiradi (taqsimlaydi).
- `align-items` — itemlarni **cross o'q** bo'ylab tekislaydi.

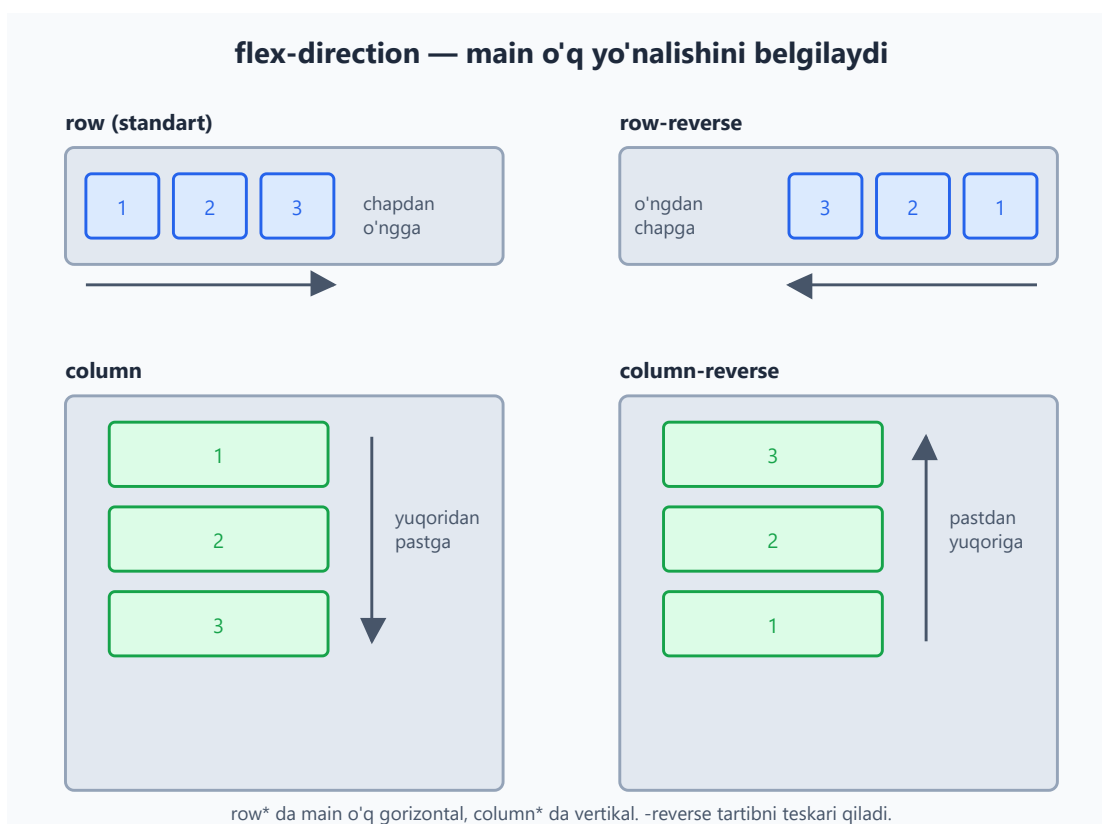
💡 **Eslab qolish hiylasi:** "justify" so'zidagi **j** = "joylashtirish" (main bo'ylab oqim), "align" = "tekislash" (cross bo'ylab). Adashganingda diagrammaga qaytib qara.

⚠️ Eng nozik joy: o'qlar **doimo gorizontal/vertikal emas!** Agar `flex-direction` ni o'zgartirsang (keyingi bo'lim), main o'q vertikal bo'lib qoladi va cross o'q gorizontal bo'ladi — shu bilan birga `justify-content` va `align-items` ning "yo'nalishi" ham aylanadi. Shuning uchun "chap/o'ng" emas, "main/cross" deb o'ylash kerak.

15.4 `flex-direction` — main o'qni burish

`flex-direction` xossasi main o'q **qaysi yo'nalishda** ketishini belgilaydi. To'rt qiymati bor:

Qiymat	Main o'q	Itemlar tartibi
<code>row</code> (standart)	gorizontal	chapdan o'ngga
<code>row-reverse</code>	gorizontal	o'ngdan chapga
<code>column</code>	vertikal	yuqoridan pastga
<code>column-reverse</code>	vertikal	pastdan yuqoriga



```
.qator {  
  display: flex;  
  flex-direction: column; /* endi itemlar ustun bo'lib teriladi */  
}
```

Natija: A, B, C qutilar endi yonma-yon emas, **ustma-ust** (vertikal) joylashadi.

Nega bu muhim? Chunki `flex-direction: column` qilishingiz bilanoq **main o'q vertikal** bo'lib qoladi. Demak endi `justify-content` itemlarni **yuqori-past** bo'ylab

boshqaradi, `align-items` esa **chap-o'ng** bo'ylab. Ko'p boshlovchi shu yerda chalkashadi:

⚠ `flex-direction: column` da matnni vertikal markazlash uchun `align-items: center` emas, `justify-content: center` kerak bo'ladi — chunki o'qlar almashti!

💡 **-reverse haqida:** u faqat **vizual** tartibni teskari qiladi, HTML'dagi haqiqiy tartibni emas. Bu skrinrider (screen reader) va Tab tartibiga ta'sir qilmaydi — shuning uchun `-reverse` ni faqat dizayn uchun ishlat, mazmun mantig'i uchun emas.

15.5 `justify-content` — main o'q bo'ylab taqsimlash

Endi eng ko'p ishlatiladigan xossaga keldik. `justify-content` itemlarni **main o'q bo'ylab** qanday taqsimlashni boshqaradi — ayniqsa konteynerda ortiqcha bo'sh joy bo'lsa.

justify-content — main o'q bo'ylab tekislash
qator (row): itemlar gorizontal bo'sh joyga qanday joylashadi

flex-start	
center	
flex-end	
space-between chetlar yopishadi, oraliq teng	
space-around har item atrofida teng oraliq	
space-evenly barcha oraliq (chet ham) teng	

space-around: chet oraliq = ichki oraliqning yarmi · space-evenly: barcha oraliq bir xil
— = bo'sh joy (oraliq) belgisi

Qiymat	Nima qiladi
<code>flex-start</code> (standart)	itemlar main o'q boshiga yopishadi (row'da: chapga)
<code>center</code>	itemlar markazga to'planadi
<code>flex-end</code>	itemlar main o'q oxiriga yopishadi (row'da: o'ngga)
<code>space-between</code>	birinchi va oxirgi item chetga yopishadi, qolgan bo'sh joy ular orasiga teng bo'linadi
<code>space-around</code>	har item atrofida teng bo'sh joy (chetdagi oraliq ichkisining yarmicha)
<code>space-evenly</code>	barcha oraliq (chetlar ham) bir xil bo'ladi

```
.navbar {
  display: flex;
  justify-content: space-between; /* logo chapda, menyu o'ngda */
}
```

Natija: birinchi element eng chapga, oxirgisi eng o'ngga yopishadi, oradagi bo'sh joy ular orasida qoladi — bu navbar uchun klassik tartib.

✳️ **space-* uchligi farqi (ko'p so'raladigan savol):** - `space-between`: chetlarda **bo'sh joy YO'Q**, faqat itemlar orasida. - `space-around`: har item ikki yonida teng joy oladi — natijada chetdagi bo'sh joy ichki oraliqning **yarmiga** teng ko'rinadi. - `space-evenly`: hamma joy, shu jumladan chetlar ham, **bir xil**.

💡 Agar item'lar konteynerni to'liq egallab tursa (ortiqcha joy yo'q), `justify-content` hech narsa qilmaydi — chunki taqsimlanadigan bo'sh joy yo'q. U faqat **ortgan bo'sh joyni** boshqaradi.

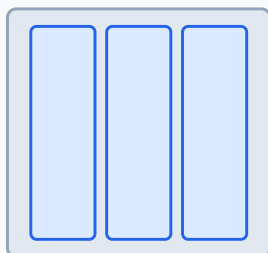
15.6 `align-items` — cross o'q bo'ylab tekislash

`align-items` itemlarni **cross o'q bo'ylab** (row'da: vertikal) tekislaydi. Bu — har bir item konteyner balandligida qayerda turishini hal qiladi.

align-items — cross o'q bo'ylab tekislash

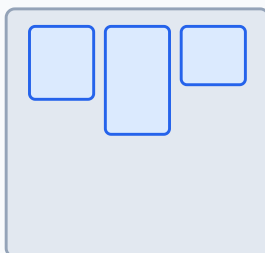
row holatida cross o'q vertikal: itemlar konteyner balandligida qanday turadi

stretch (standart)



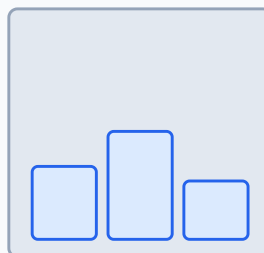
to'liq balandlikka cho'ziladi

flex-start



yuqoriga yopishadi

flex-end



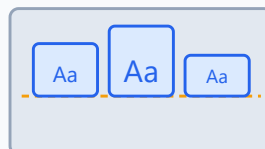
pastga yopishadi

center



cross o'q markaziga

baseline



matn tag chizig'i tekis

Qiymat	Nima qiladi
stretch (standart)	itemlar cross o'q bo'ylab to'liq cho'ziladi (agar balandlik berilmagan bo'lsa)
flex-start	itemlar cross o'q boshiga (row'da: yuqoriga) yopishadi
flex-end	itemlar cross o'q oxiriga (row'da: pastga) yopishadi
center	itemlar cross o'q markaziga keladi
baseline	itemlar matn tag chizig'i (baseline) bo'ylab tekislanadi

```
.qator {  
  display: flex;  
  align-items: center; /* har xil balandlikdagi qutilar vertikal  
  markazda */  
}
```

Natija: turli balandlikdagi kartalar/qutilar endi yuqoridan emas, **markazdan** tekislanadi.

✦ **stretch haqida nega:** standart `stretch` bo'lgani uchun, balandlik bermagan flex itemlar avtomatik konteyner balandligiga cho'ziladi — shuning uchun yonma-yon turgan kartalar ko'pincha o'z-o'zidan teng balandlikda chiqadi. Bu juda foydali xulq.

💡 **baseline qachon kerak:** har xil shrift o'lchamidagi matnlarni bir qatorda tekislaganda — masalan katta sarlavha yonida kichik yorliq — `baseline` ularning matn pastki chizig'ini bir tekisda saqlaydi.

15.7 Sehrli markazlash: `justify-content` + `align-items`

Eski CSS'da eng qiyin masala — bir narsani ekran o'rtasiga to'liq joylashtirish — flexbox'da bor-yo'g'i uch qator:

```
.markaz {  
  display: flex;  
  justify-content: center; /* main bo'ylab markaz (gorizontal) */  
  align-items: center; /* cross bo'ylab markaz (vertikal) */  
  height: 100vh; /* konteyner butun ekran balandligida */  
}
```

```
<div class="markaz">  
  <button>Bosing</button>  
</div>
```

Natija: tugma ekranning aynan markazida — ham gorizontal, ham vertikal — turadi.

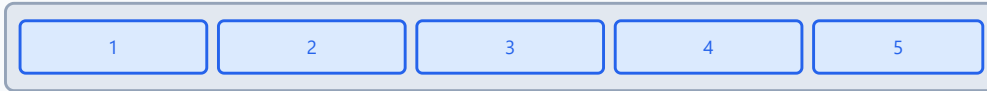
💡 Bu — flexbox'ning "vizit kartochkasi". Yillar davomida dasturchilar izlagan "vertikal markazlash" yechimi shu. `justify-content: center` + `align-items: center` ni yodlab ol — har kuni kerak bo'ladi.

15.8 `flex-wrap` va `align-content` — ko'p qatorli flex

Standart holatda flex itemlar **bitta qatorga** tiqiladi — joy yetmasa siqiladi, lekin pastga tushmaydi. `flex-wrap` shuni o'zgartiradi.

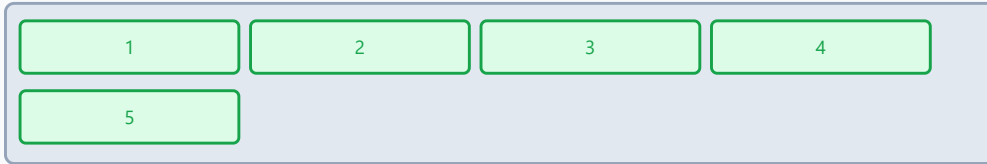
flex-wrap — itemlar sig'masa nima bo'ladi?

nowrap (standart) — hammasi bitta qatorda, siqiladi



itemlar tor joyga zo'rlab tiqiladi (min-content gacha siqiladi)

wrap — sig'magani yangi qatorga tushadi



bir nechta qator hosil bo'ladi · qatorlar orasini align-content boshqaradi

Qiymat	Nima qiladi
<code>nowrap</code> (standart)	hamma item bitta qatorda; joy yetmasa siqiladi
<code>wrap</code>	sig'magan itemlar yangi qatorga tushadi
<code>wrap-reverse</code>	xuddi <code>wrap</code> , lekin yangi qatorlar yuqoriga qarab qo'shiladi

```
.kartalar {  
  display: flex;  
  flex-wrap: wrap; /* tor ekranda kartalar pastga ko'chadi */  
}
```

Natija: ekran torayganda kartalar gorizontal siqilib xunuklashmaydi — ortig'i yangi qatorga tushadi. Bu responsiv (responsive) dizaynning oddiy va kuchli usuli.

align-content — qatorlar orasini boshqarish

`flex-wrap: wrap` tufayli **bir nechta qator** hosil bo'lganda, `align-content` shu **qatorlar guruhini** cross o'q bo'ylab taqsimlaydi. Qiymatlari `justify-content` nikiga o'xshash: `flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`, `stretch`.

⚠ **Muhim farq — adashtirmang:** - `align-items` — har bir **item**'ni o'z **qatori** ichida tekislaydi. - `align-content` — faqat **bir nechta qator bo'lganda** ishlaydi va **qatorlar orasini** boshqaradi. Bitta qatorda u hech narsa qilmaydi.

15.9 gap — itemlar orasiga oson joy

Ilgari flex itemlar orasiga masofa qo'yish uchun har biriga `margin` berib, keyin chetdagi ortiqcha margin'ni o'chirish kerak edi — noqulay. Hozir buning **toza** yechimi bor: `gap`.

```
.qator {  
  display: flex;  
  gap: 16px;          /* har item orasida 16px */  
}
```

Natija: itemlar orasida bir tekis 16px masofa paydo bo'ladi — chetlarda ortiqcha bo'sh joy qoldirmasdan.

```
.qator {  
  gap: 20px 12px;    /* qatorlar orasi 20px, ustunlar orasi 12px (wrap  
  bilan) */  
}
```

💡 `gap` faqat **itemlar orasiga** joy qo'yadi, konteynerning ichki chetiga emas — aynan kerakli xulq. `justify-content: space-*` bilan emas, **aniq, bashorat qilinadigan** masofa kerak bo'lganda `gap` ishlat. Ikkalasini birga ham ishlatsa bo'ladi.

🚩 `gap` Flexbox uchun barcha zamonaviy brauzerlarda ishlaydi — endi `margin` hiylalariga ehtiyoj yo'q.

15.10 Item xossalari: `flex-grow`, `flex-shrink`, `flex-basis`

Hozirgacha biz **konteynerga** beriladigan xossalarni ko'rdik. Endi **alohida item'larga** beriladigan xossalarga o'tamiz — ular item'ning o'lchamini main o'q bo'ylab boshqaradi.

`flex-basis` — boshlang'ich o'lcham

`flex-basis` — item'ning **asl (boshlang'ich) o'lchami** main o'q bo'ylab, o'sish/qisqarish hisobga olinishidan oldin. U `width` o'rnini bosadi (row'da). Standart qiymati `auto` — ya'ni mazmun yoki `width` qancha bo'lsa, shuncha.

`flex-grow` — bo'sh joyni ulushlab olish

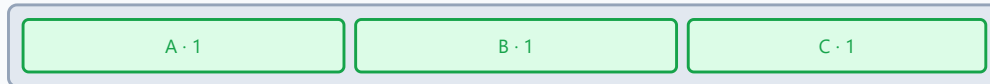
`flex-grow` — konteynerda **ortgan bo'sh joy** bo'lsa, item undan **qancha ulush** olishini belgilaydi. Qiymati son (standart `0` — o'smaydi).

flex-grow — ortgan bo'sh joyni ulushlab bo'lish

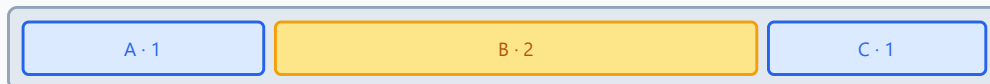
flex-grow: 0 (standart) — itemlar o'smaydi



flex-grow: 1 (hammasiga) — bo'sh joy teng bo'linadi



flex-grow: 1 / 2 / 1 — B ikki barobar ko'p oladi



Qanday hisoblanadi?

1. Konteynerdan itemlarning asl (basis) eni ayriladi -> ortgan bo'sh joy chiqadi.
2. Bo'sh joy grow ulushlariga bo'linadi: $1+2+1 = 4$ ulush. B (2/4) ni, A va C (1/4) dan oladi.
3. Har item: asl en + o'z ulushi. flex-grow faqat ORTGAN joyni bo'ladi, butun enini emas.

```
.item {  
  flex-grow: 1; /* hamma item teng o'sib, butun joyni to'ldiradi */  
}
```

Agar itemlarga `1`, `2`, `1` bersang — ortgan bo'sh joy $1+2+1 = 4$ ulushga bo'linadi: o'rtadagi item ikki barobar ko'p oladi.

⚠ **Eng ko'p uchraydigan tushunmovchilik:** `flex-grow: 2` item'ni "ikki barobar katta" qilmaydi! U faqat **ortgan bo'sh joyning** ikki barobar ulushini oladi. Yakuniy o'lcham = asl (basis) o'lcham + olingan ulush. Diagrammadagi hisob-kitobga e'tibor ber.

flex-shrink — joy yetmaganda qisqarish

`flex-shrink` — joy **yetmaganda** item qanchalik **qisqarishini** belgilaydi (standart `1` — qisqaradi). `0` bersang, item **qisqarmaydi** (asl o'lchamini saqlaydi, kerak bo'lsa toshib chiqadi).

```
.logo {  
  flex-shrink: 0; /* logo hech qachon siqilmasin */  
}
```

💡 `flex-shrink: 0` — logotip, ikonka kabi qisqarib xunuklashmasligi kerak elementlar uchun juda foydali.

15.11 Qisqa `flex` yozuvi va `flex: 1`

Yuqoridagi uchta xossani alohida yozish o'rniga, ularni bitta `flex` qisqa (shorthand) xossasida birlashtirish mumkin — tartibi: `flex: <grow> <shrink> <basis>`.

```
.item {  
  flex: 1 1 0; /* grow:1, shrink:1, basis:0 */  
}
```

Eng ko'p ishlatiladigan yozuv — oddiygina `flex: 1`:

```
.item {  
  flex: 1; /* = flex: 1 1 0% */  
}
```

`flex: 1` ni hamma item'ga bersang, ular **butun konteynerni teng bo'lib** egallaydi — chunki `basis: 0` dan boshlanib, butun joyini teng o'zlashtiradi. Bu teng ustunlar yasashning eng oson usuli.

✦ **Foydali tayyor qiymatlar:**

Yozuv	Ma'nosi
<code>flex: 1</code>	<code>1 1 0</code> — teng o'sadi, bo'sh joyini egallaydi
<code>flex: auto</code>	<code>1 1 auto</code> — mazmunga qarab o'sadi
<code>flex: none</code>	<code>0 0 auto</code> — qotib qoladi (o'smaydi, qisqarmaydi)
<code>flex: 0 0 200px</code>	aniq 200px, o'zgarmaydi

💡 Amalda 99% holatda `flex: 1` yoki `flex: 0 0 <o'lcham>` yetadi. `flex` qisqa yozuvini afzal ko'r — u `flex-basis` ni ham aqlli standartga qo'yadi (aloqida `flex-grow: 1` yozsang, `basis auto` bo'lib qolib, kutilmagan natija berishi mumkin).

15.12 `order` va `align-self` — alohida itemlarni boshqarish

`order` — vizual tartibni o'zgartirish

`order` xossasi item'ning **ko'rsatilish tartibini** o'zgartiradi (standart `0`). Kichikroq `order` oldinroq, kattaroq keyinroq turadi.

```
.muhim {  
  order: -1; /* HTML'da qayerda bo'lishidan qat'i nazar, birinchi  
  ko'rinadi */  
}
```

⚠ `order` faqat **vizual** tartibni o'zgartiradi — HTML manba tartibi, Tab navigatsiyasi va skrinrider tartibi o'zgarmaydi. Shuning uchun mazmun mantig'i uchun emas, faqat vizual moslash uchun ishlat. Aks holda klaviatura foydalanuvchisi "sakrab" yuradigan, chalkash sahifa hosil bo'ladi.

`align-self` — bitta item'ni alohida tekislash

`align-self` — `align-items` ni **bitta item uchun** bekor qiladi. Konteyner hamma item'ni bir xil tekislagan bo'lsa-da, bittasini boshqacha joylashtirishing mumkin.

```
.qator {  
  display: flex;  
  align-items: flex-start; /* hammasi yuqorida */  
}  
.alohida {  
  align-self: flex-end; /* faqat shu item pastda */  
}
```

💡 `align-self` faqat **cross o'q** bo'ylab ishlaydi (`align-items` kabi). Main o'q bo'ylab bitta item'ni surish uchun `margin: auto` hiylasi bor (keyingi bo'lim).

✳ **Bonus** — `margin: auto` **hiylasi**: flex item'ga `margin-left: auto` bersang, u chap tomondagi butun bo'sh joyni "yutadi" va item'ni o'ngga itaradi. Navbarda bitta tugmani o'ng chetga surish uchun ajoyib usul: `.tugma { margin-left: auto; }`.

15.13 Amaliyot: navbar, markazlash va kartalar qatori

Endi o'rganganlarimizni uchta real misolda birlashtiramiz.

1) Responsiv navbar

```
<nav class="navbar">  
  <div class="logo">MySayt</div>  
  <ul class="menu">  
    <li><a href="#">Bosh</a></li>
```

```

    <li><a href="#">Xizmatlar</a></li>
    <li><a href="#">Aloqa</a></li>
  </ul>
</nav>

```

```

.navbar {
  display: flex;
  align-items: center; /* logo va menyu vertikal markazda */
  justify-content: space-between; /* logo chapda, menyu o'ngda */
  padding: 16px 24px;
  background: #f8fafc;
}
.menu {
  display: flex; /* menyu ham flex – havolalar yonma-yon */
  gap: 24px;
  list-style: none; /* nuqtalarni olib tashlash */
  margin: 0;
  padding: 0;
}

```

Natija: chap tomonda logotip, o'ng tomonda yonma-yon, oraliq 24px havolalar — klassik navbar. E'tibor ber: `<nav>` ham, `.menu` ham flex konteyner — ichma-ich flexbox keng tarqalgan amaliyot.

2) To'liq markaz

```

.qahramon {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  text-align: center;
}

```

Natija: ichidagi mazmun (sarlavha, tugma) ekran o'rtasida — qahramon (hero) bo'limlar uchun ideal.

3) Teng kenglikdagi kartalar qatori

```

<div class="kartalar">
  <div class="karta">Karta 1</div>
  <div class="karta">Karta 2</div>
  <div class="karta">Karta 3</div>
</div>

```

```

.kartalar {
  display: flex;
}

```



```

    gap: 20px;
    flex-wrap: wrap; /* tor ekranda pastga ko'chsin */
  }
  .karta {
    flex: 1 1 200px; /* asl 200px, lekin o'sib bo'sh joyni teng
to'ldiradi */
    padding: 24px;
    border: 2px solid #94a3b8;
    border-radius: 8px;
  }

```

Natija: keng ekranda uchta karta yonma-yon teng kenglikda, tor ekranda esa `flex-wrap` tufayli pastga ko'chadi. `flex: 1 1 200px` — har karta kamida 200px bo'lishini istaydi, lekin ortgan joyni teng bo'lib oladi. Bu — eng ko'p ishlatiladigan responsiv naqsh (pattern).

💡 Mana shu uch naqsh (navbar, markaz, kartalar) kundalik frontend ishining katta qismini qoplaydi. Ularni yodlab, o'zgartirib mashq qil — flexbox shu bilan "qo'lingga o'tiradi".

Mashqlar

Mashq 1 — Birinchi qator

Uchta `<div>` ni yonma-yon joylashtir. Hozir ular ustma-ust turibdi. Bitta CSS qatori bilan tuzat.


Yechim


```

.konteyner {
  display: flex;
}

```

`display: flex` ota'ni flex konteynerga aylantiradi va bevosita farzandlarni standart `row` (gorizontal qator) bo'ylab teradi.

Mashq 2 — To'liq markazlash

Bitta tugmani ekranning aynan o'rtasiga (ham gorizontal, ham vertikal) joylashtir.

 **Yechim** 

```
.konteyner {
  display: flex;
  justify-content: center; /* main (gorizontal) markaz */
  align-items: center;     /* cross (vertikal) markaz */
  height: 100vh;          /* butun ekran balandligi */
}
```

`justify-content` main o'q bo'ylab, `align-items` cross o'q bo'ylab markazlaydi. `height` bo'lmasa, konteyner faqat tugma balandligida bo'lib, vertikal markazlash ko'rinmaydi.

Mashq 3 — Navbar tartibi

Navbarda logo chap chetda, menyu havolalari o'ng chetda bo'lsin. Qaysi `justify-content` qiymatini ishlatasan?

 **Yechim** 

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

`space-between` birinchi item'ni (logo) eng chapga, oxirgisini (menyu) eng o'ngga yopishtiradi, oradagi bo'sh joyni esa ular orasiga qo'yadi.

Mashq 4 — Teng ustunlar

Uchta item butun konteyner kengligini **teng** bo'lib egallasin. Eng qisqa yo'li qaysi?

 **Yechim** 

```
.item {
  flex: 1;
}
```

`flex: 1` = `flex: 1 1 0`. `basis: 0` dan boshlanib, hamma item butun bo'sh joyni teng ulush bilan oladi — natijada uchala ustun bir xil kenglikda chiqadi.

Mashq 5 — flex-grow hisobi

Konteyner kengligi 600px. Ucha item: har birining `flex-basis: 100px`, va `flex-grow` mos ravishda 1, 2, 3. Har item yakuniy kengligi qancha bo'ladi?



Yechim



1. Asl egallangan joy: $100 + 100 + 100 = 300\text{px}$.
2. Ortgan bo'sh joy: $600 - 300 = 300\text{px}$.
3. Grow ulushlari: $1 + 2 + 3 = 6$ ulush. Har ulush = $300 / 6 = 50\text{px}$.
4. Yakuniy:
5. Item 1: $100 + 1 \times 50 = 150\text{px}$
6. Item 2: $100 + 2 \times 50 = 200\text{px}$
7. Item 3: $100 + 3 \times 50 = 250\text{px}$

Tekshirish: $150 + 200 + 250 = 600\text{px}$. To'g'ri. `flex-grow` faqat **ortgan** joyini bo'ladi, butun kenglikni emas.

Mashq 6 — Vertikal ustun

Itemlarni yuqoridan pastga (ustun) tartibida joylashtir va ularni gorizontal markazga keltir.



Yechim



```
.konteyner {
  display: flex;
  flex-direction: column; /* main o'q endi vertikal */
  align-items: center; /* cross o'q (gorizontal) bo'ylab markaz */
}
```

⚠ `column` da o'qlar almashdi: `cross` o'q endi gorizontal, shuning uchun gorizontal markazlash uchun `align-items: center` to'g'ri. Agar vertikal taqsimlash kerak bo'lsa, `justify-content` ishlatasan.

Mashq 7 — Qisqarmaydigan logo

Navbarda logo hech qachon siqilmasin, lekin yonidagi qidiruv maydoni qolgan bo'sh joyini egallasin.



Yechim



```
.logo {  
  flex: 0 0 auto; /* o'smaydi, qisqarmaydi - asl o'lchamini saqlaydi */  
}  
.qidiruv {  
  flex: 1; /* qolgan butun joyni egallaydi */  
}
```

`flex: 0 0 auto` (ya'ni `flex: none`) logoni qotirib qo'yadi: `grow:0` o'smaydi, `shrink:0` qisqarmaydi. `flex: 1` esa qidiruvni butun ortgan joyga cho'zadi.

Mashq 8 — Bitta item'ni o'ngga itarish

Navbarda 4 ta tugma bor. Faqat oxirgi tugma ("Chiqish") o'ng chetga yopishsin, qolgan uchta chapda qator tursin. `margin` bilan yech.



Yechim



```
.navbar {  
  display: flex;  
  gap: 12px;  
}  
.chiqish {  
  margin-left: auto; /* chapdagi butun bo'sh joyni yutadi -> o'ngga  
  itaradi */  
}
```

Flex item'ga `margin-left: auto` berilsa, u chap tomondagi mavjud bo'sh joyni to'liq egallaydi va item'ni o'ng chetga suradi. `justify-content` butun guruhga ta'sir qiladi; bitta item'ni surish uchun aynan shu `margin: auto` hiylasi ishlatiladi.

16 — CSS Grid

← Oldingi: 15 — Flexbox · 🏠 README · Keyingi: 17 — Responsive dizayn →

Bu bobda: sahifani ikki o'lchovda — ustun va qator panjarasi sifatida — qurishni o'rganamiz: `grid-template-columns/rows`, `fr` birligi, `repeat()`, `gap`, chiziqlarga joylashtirish, nomli sohalar (`grid-template-areas`), tekislash va `auto-fill` / `auto-fit` bilan moslashuvchan layout.

16.1 Grid nima va nega kerak?

Avvalgi bobda Flexbox bilan tanishding. Flexbox ajoyib — lekin u **bir o'lchovli**: elementlarni yo bitta qatorga, yo bitta ustunga tizadi. Navbar, tugmalar qatori, kartochkalar zanjiri uchun bu yetadi.

Ammo butun sahifani tasavvur qil: yuqorida sarlavha (header), chapda menyu (sidebar), o'rtada asosiy mazmun (main), pastda footer. Bu **ikki o'lchovli** tuzilma — bir vaqtning o'zida ham ustunlar, ham qatorlar bor. Flexbox bilan buni qilish mumkin, lekin ko'p `div` ichiga `div` o'rab, ko'p qatlamli "ichma-ich" tuzilma yasashga to'g'ri keladi.

CSS Grid aynan shu muammoni hal qiladi. Grid — bu shaxmat taxtasi yoki Excel jadvali kabi **panjara**: sen ustunlar va qatorlarni e'lon qilasiz, keyin elementlarni shu panjaraning kerakli kataklariga joylashtirasiz.

✦ Asosiy farq (Flexbox vs Grid):

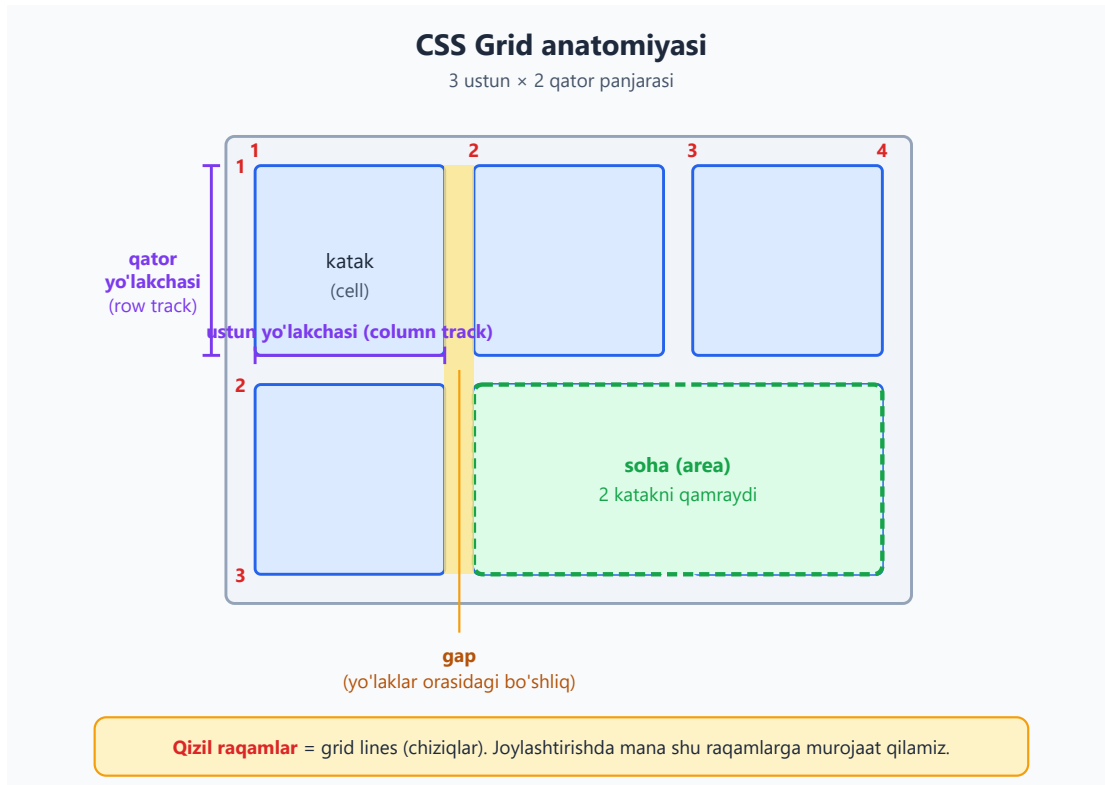
	Flexbox	Grid
O'lcham	Bir o'lchovli (qator YOKI ustun)	Ikki o'lchovli (qator VA ustun)
Boshqaruv markazi	Mazmun boshqaradi (content-first)	Layout boshqaradi (layout-first)
Eng mos	Navbar, tugmalar, ro'yxat	Sahifa skeleti, galereya, dashboard

💡 **Hayotiy analogiya:** Flexbox — bu bir tokchaga kitoblarni tizish (faqat eni bo'yicha). Grid — bu butun kitob javoni: ham gorizontal tokchalar, ham vertikal bo'limlar bir vaqtda.

⚠ Grid Flexbox'ning "raqibi" emas — ular **birga** ishlaydi. Odatda sahifa skeletini Grid bilan, ichidagi mayda elementlarni Flexbox bilan qilamiz. Ikkalasini ham bilish kerak.

16.2 Grid atamalari: chiziq, yo'lak, katak, soha

Grid'ni o'rganishdan oldin uning "tili"ni bilib olaylik. Quyidagi diagrammada barcha asosiy atamalar yorliqlangan:



- **Grid container** (panjara konteyneri) — `display: grid` berilgan ota element. U panjarani belgilaydi.
- **Grid item** (panjara elementi) — konteynerning **bevosita farzandlari**. Faqat shular panjaraga bo'ysunadi.
- **Grid line** (chiziq) — ustun va qatorlarni ajratuvchi ko'rinmas chiziqlar. Ular **1 dan** boshlab raqamlanadi. 3 ta ustun bo'lsa, 4 ta vertikal chiziq bo'ladi (har ustunning ikki tomonida).
- **Grid track** (yo'lakcha) — ikki qo'shni chiziq orasidagi bo'shliq, ya'ni bitta ustun yoki bitta qator.
- **Grid cell** (katak) — bitta ustun va bitta qator kesishgan joy, panjaraning eng kichik birligi (jadvaldagi bitta yacheyka kabi).
- **Grid area** (soha) — bir nechta katakdan iborat to'rtburchak hudud.

- **Gap** (bo'shliq) — yo'laklar orasidagi masofa.

✦ Bu atamalarni hozir yodlash shart emas — quyida har birini amalda ko'rasiz. Lekin chiziqlar **1 dan boshlab raqamlanishini** eslab qol, chunki keyin elementlarni joylashtirishda aynan shu raqamlardan foydalanamiz.

16.3 Birinchi grid: `display: grid` va `grid-template-columns`

Grid'ni yoqish uchun ota elementga `display: grid` beramiz. Bu o'zi hech narsani o'zgartirmaydi — farzandlar baribir ustma-ust (bir ustunda) joylashadi. Sehr `grid-template-columns` bilan boshlanadi: u **nechta ustun** va **har biri qanchaligini** belgilaydi.

```
<div class="panjara">
  <div class="katak">1</div>
  <div class="katak">2</div>
  <div class="katak">3</div>
  <div class="katak">4</div>
  <div class="katak">5</div>
  <div class="katak">6</div>
</div>
```

```
.panjara {
  display: grid;
  grid-template-columns: 100px 100px 100px;
}
.katak {
  background: #dbeafe;
  padding: 20px;
  text-align: center;
}
```

Natija: 6 ta katak **3 ta ustunga** bo'linadi. Birinchi 3 tasi yuqori qatorga, keyingi 3 tasi pastki qatorga tushadi.

Nega bunday bo'ldi? `grid-template-columns: 100px 100px 100px` — "uchta ustun, har biri 100px keng" degani. Grid elementlarni **chapdan o'ngga** ustunlarga joylaydi; ustunlar tugagach, o'zi yangi qator ochib davom etadi. Qatorlar haqida bir og'iz aytmadik — Grid ularni avtomatik yaratdi (bu "implicit grid", 16.12'da ko'ramiz).

💡 Bir xil qiymatni qo'lda uch marta yozish zerikarli. Buni qisqartirish uchun `repeat()` bor (16.5).

16.4 fr birligi — bo'sh joyni ulashish

px bilan ustun belgilash qattiq (q'attiq) o'lcham beradi: ekran katta bo'lsa, o'ng tomonda bo'sh joy qoladi; kichik bo'lsa, ustunlar sig'maydi. Responsive layout uchun bizga **moslashuvchan** birlik kerak — bu `fr`.

`fr` — "fraction" (ulush) so'zining qisqartmasi. U **mavjud bo'sh joyni** qismlarga bo'ladi.

```
.panjara {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
}
```

Bu yerda jami $1 + 2 + 1 = 4$ ulush bor. Grid bo'sh joyni 4 ga bo'ladi: birinchi ustun 1 ulush, ikkinchisi 2 ulush (ikki barobar keng), uchinchisi yana 1 ulush oladi.



`fr` ning eng kuchli tomoni — konteyner kengaysa yoki torayganda, ustunlar **shu nisbatda** birga o'zgaradi. Foiz (%) hisoblashga hojat yo'q.

✦ `fr` ni boshqa birliklar bilan **aralashtirish** mumkin:

```
.panjara {  
  display: grid;  
  grid-template-columns: 200px 1fr 100px;  
}
```

Natija: chap ustun aniq 200px, o'ng ustun aniq 100px, o'rtadagi `1fr` esa **qolgan hamma bo'sh joyni** egallaydi. Bu juda keng tarqalgan naqsh: chetlarda qat'iy o'lchamli paneller, o'rtada cho'ziluvchi mazmun.

⚠️ `fr` "umumiy enining ulushi" emas, balki **qolgan bo'sh joyning** ulushi. Agar `grid-template-columns: 200px 1fr 1fr` desangiz, avval 200px ajratiladi, keyin **qolgani** ikki `fr` orasida teng bo'linadi.

16.5 `repeat()` — takrorni qisqartirish

12 ta bir xil ustun yozish — `1fr 1fr 1fr ...` 12 marta — kulgili. `repeat()` funksiyasi buni qisqartiradi:

```
.panjara {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

`repeat(3, 1fr)` aynan `1fr 1fr 1fr` degani: "1fr ni 3 marta takrorla". Birinchi argument — necha marta, ikkinchisi — nimani.

`repeat()` ichida bir nechta qiymat ham bo'lishi mumkin, va uni boshqa qiymatlar bilan aralashtirsa ham bo'ladi:

```
/* 6 ta ustun: 200px va 1fr navbatma-navbat takrorlanadi */  
grid-template-columns: repeat(3, 200px 1fr);  
  
/* boshida sidebar, keyin 4 ta teng ustun */  
grid-template-columns: 250px repeat(4, 1fr);
```

💡 `repeat()` ayniqsa "12 ustunli grid tizimi" (Bootstrap kabi) yoki bir xil kartochkalar galereyasi uchun ajralmas. Uni `auto-fill` / `auto-fit` bilan birlashtirsa, haqiqiy responsive grid chiqadi (16.11).

`grid-template-rows` ham xuddi shunday ishlaydi — qatorlar balandligini belgilaydi:

```
.panjara {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: 100px 200px; /* 1-qator 100px, 2-qator 200px */  
}
```

16.6 `gap` — kataklar orasidagi bo'shliq

Kataklarni bir-biriga yopishtirib qo'yish chiroyli emas. Ularni ajratish uchun `gap` ishlatamiz:

```
.panjara {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

Natija: har ustun va qator orasida 16px bo'shliq paydo bo'ladi.

✦ `gap` faqat **ichki** bo'shliqlarni qo'shadi — panjaraning tashqi chetlarida bo'shliq yaratmaydi. Bu `margin` dan qulayroq: `margin` bilan "oxirgi elementning ortiqcha bo'shlig'i" muammosi yo'q.

Bo'shliqni alohida boshqarish kerak bo'lsa:

```
.panjara {
  row-gap: 24px; /* qatorlar orasida */
  column-gap: 12px; /* ustunlar orasida */
}
```

`gap: 24px 12px` ham bir xil natija beradi (birinchisi — `row-gap`, ikkinchisi — `column-gap`).

💡 **Tarixiy eslatma:** ilgari bu xossa `grid-gap` deb atalgan edi. Hozir oddiy `gap` standart bo'lib, u Flexbox'da ham ishlaydi. Eski `grid-gap` ni yangi kodga yozmang.

16.7 Elementni joylashtirish: `grid-column` va `grid-row`

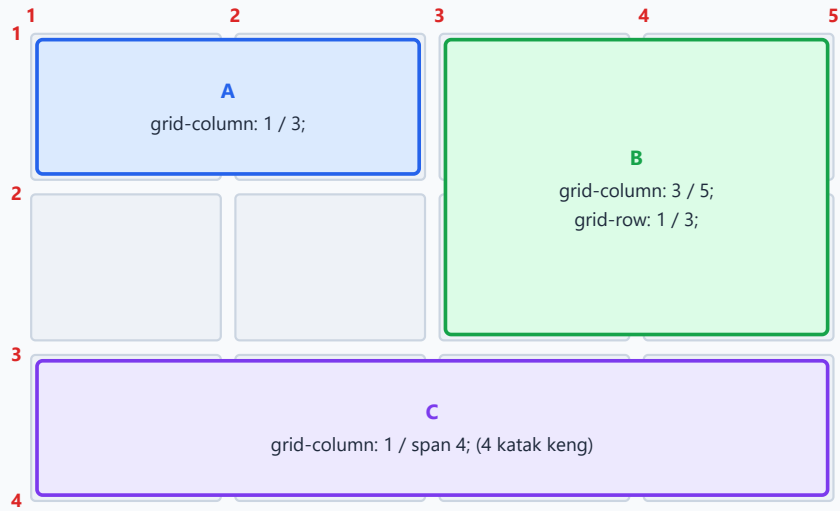
Hozirgacha elementlar avtomatik joylashdi. Lekin Grid'ning kuchi shundaki, **istalgan elementni istalgan katakka** qo'yish mumkin. Buning uchun chiziqlar raqamlaridan foydalanamiz.

Eslab qol: chiziqlar 1 dan boshlanadi. Element qaysi chiziqdan **boshlanishi** va qaysi chiziqda **tugashini** ko'rsatamiz:

```
.element {
  grid-column: 1 / 3; /* 1-chiziqdan 3-chiziqgacha = 2 ta ustun keng */
  grid-row: 1 / 2; /* 1-chiziqdan 2-chiziqgacha = 1 ta qator baland */
}
```

`grid-column: 1 / 3` ni "1-ustun chizig'idan boshla, 3-ustun chizig'ida tugat" deb o'qing. Oraliqda 1 va 2 ustunlar qoladi — demak element 2 ta ustunni egallaydi.

Chiziq raqamlari bilan joylashtirish



1 / 3 = "1-chiziqdan boshla, 3-chiziqda tugat". **span 4** = "shu yerdan 4 katak cho'zil".

span – "shuncha katak cho'z"

Ba'zan element qayerda tugashini emas, **nechta katak egallashini** bilamiz. Shunda span qulayroq:

```
.element {
  grid-column: span 2; /* shu yerdan 2 ta ustun cho'zil */
  grid-row: span 3;    /* shu yerdan 3 ta qator cho'zil */
}
```

`grid-column: 2 / span 3` — "2-chiziqdan boshla va 3 ustun cho'zil" degani. Bu, ayniqsa, element qayerdan boshlanishi muhim, lekin kengligi nisbiy bo'lsa qulay.

-1 — oxirgi chiziq

Manfiy raqamlar **oxiridan** sanaydi: -1 – eng oxirgi chiziq. Demak butun qatorni egallash uchun:

```
.header {
  grid-column: 1 / -1; /* birinchi chiziqdan oxirgisigacha = butun en
*/
}
```

💡 1 / -1 — eng ko'p ishlatiladigan naqshlardan biri: sarlavha yoki footer'ni butun panjara eni bo'ylab cho'zish uchun. Ustunlar soni o'zgarsa ham ishlayveradi.

16.8 grid-template-areas — nomli sohalar bilan layout

Chiziq raqamlari kuchli, lekin `grid-column: 2 / 4` ni o'qiganda nima bo'layotgani darrov ko'rinmaydi. CSS Grid'ning eng chiroyli imkoniyatlaridan biri — **kataklarga nom berib**, layoutni xuddi rasm chizgandek tasvirlash.

Avval konteynerda har qatorni "so'zlar xaritasi" sifatida chizamiz:

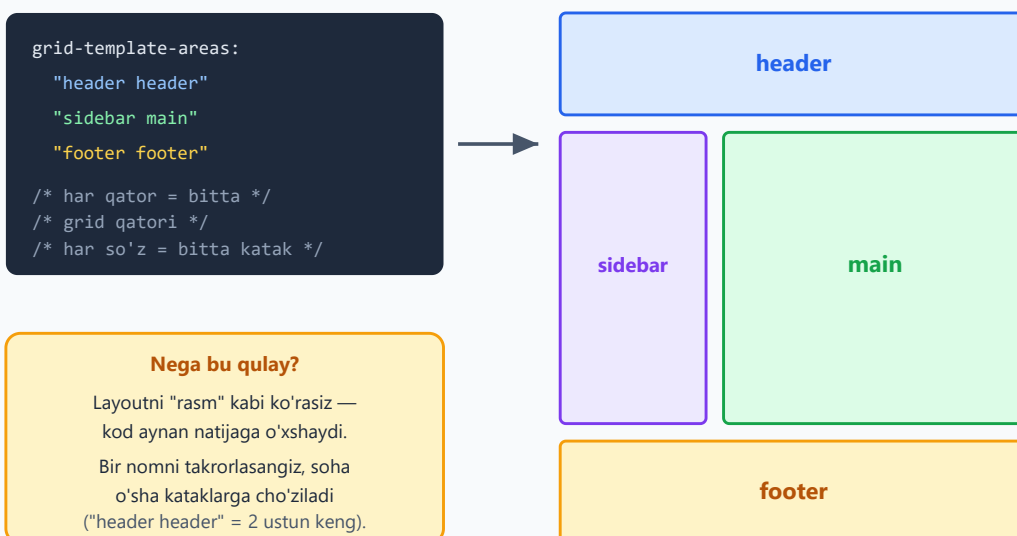
```
.sahifa {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  min-height: 100vh;  
  gap: 12px;  
}
```

Keyin har bir elementni `grid-area` orqali nomga ulaymiz:

```
.head { grid-area: header; }  
.side { grid-area: sidebar; }  
.content { grid-area: main; }  
.foot { grid-area: footer; }
```

```
<div class="sahifa">  
  <header class="head">Header</header>  
  <aside class="side">Sidebar</aside>  
  <main class="content">Asosiy mazmun</main>  
  <footer class="foot">Footer</footer>  
</div>
```

grid-template-areas: nomli sohalar bilan layout



Har element o'zini soha nomiga ulaydi: `.head { grid-area: header; }`
Bo'sh katak kerak bo'lsa nuqta yoziladi: `."` (point).

E'tibor ber: `grid-template-areas` ichidagi matn **aynan natijaga o'xshaydi**. Kodga qarab layoutni "ko'rasiz". Bu — Grid'ning eng o'qiluvchan usuli.

✦ **Qoidalar:** - Har bir tirnoq ichidagi satr — **bitta qator**. Bu yerda 3 ta satr bor, demak 3 ta qator. - Har satrdagi so'zlar soni **ustunlar soniga** teng bo'lishi shart (bu yerda 2 ta). - Bir nomni **takrorlasangiz**, soha o'sha kataklarga cho'ziladi: `"header header"` — header ikki ustunni egallaydi. Takrorlanuvchi kataklar **to'rtburchak** hosil qilishi shart (L-shaklida bo'lmaydi). - **Bo'sh katak** kerak bo'lsa, bir yoki bir nechta **nuqta** (`.`) yoziladi: `"main ."`.

⚠ Har satrni alohida tirnoqqa olishni unutmang. `"header header" "sidebar main"` bitta qatorda yozilsa ham ishlaydi, lekin yuqoridagidek alohida satrlarga yozish ancha o'qiluvchanroq.

💡 Eng zo'r tomoni: media query ichida faqat `grid-template-areas` ni qayta yozib, butun layoutni mobil ko'rinishga o'zgartirishingiz mumkin — elementlarning HTML tartibiga tegmasdan (16.13'da ko'ramiz).

16.9 Tekislash: elementlarni katak ichida joylash

Hozirgacha elementlar o'z kataklarini **to'ldirib** turardi (default `stretch`). Grid'da tekislashni boshqaradigan to'rtta asosiy xossa bor. Ularni ikki guruhga ajratish oson:

- `justify-*` — **gorizontal** o'q (ustun yo'nalishi, chap-o'ng).

- `align-*` — **vertikal** o'q (qator yo'nalishi, yuqori-past).

Elementlarni kataklar ichida (`*-items`)

Konteynerga qo'yiladi, **barcha** elementlarga ta'sir qiladi:

```
.panjara {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: repeat(2, 100px);  
  justify-items: center; /* har element o'z katagida gorizontal  
markazda */  
  align-items: center; /* va vertikal markazda */  
}
```

Qiymatlar: `start`, `end`, `center`, `stretch` (default).

✦ Bitta elementni alohida boshqarish kerak bo'lsa, o'sha elementga `justify-self` / `align-self` beriladi. Ular `*-items` ni shu element uchun bekor qiladi:

```
.alohida {  
  justify-self: end; /* faqat bu element o'ng tomonda */  
  align-self: start; /* faqat bu element yuqorida */  
}
```

Butun panjarani konteyner ichida (`*-content`)

Agar panjaraning umumiy o'lchami konteynerdan **kichik** bo'lsa (masalan, ustunlar `px` bilan belgilangan va joy ortib qolsa), butun panjarani konteyner ichida joylash kerak bo'ladi:

```
.panjara {  
  justify-content: center; /* butun panjara gorizontal markazda */  
  align-content: center; /* butun panjara vertikal markazda */  
}
```

Qiymatlar: `start`, `end`, `center`, `space-between`, `space-around`, `space-evenly`, `stretch`.

💡 **Farqni eslab qolish:** `*-items` / `*-self` — **elementlarni kataklar ichida** joylaydi. `*-content` — **butun panjarani konteyner ichida** joylaydi. `fr` ishlatilgiz panjara konteynerni to'la egallaydi, shuning uchun `*-content` ko'pincha faqat `px` / `auto` ustunlarda seziladi.

16.10 `minmax()` — eng kichik va eng katta o'lcham

Ko'pincha ustun "juda kichik bo'lmasin, lekin imkon bo'lsa kengaysin" deyiladi.

`minmax(min, max)` aynan shuni beradi: ustunning eng kichik va eng katta o'lchamini belgilaydi.

```
.panjara {  
  display: grid;  
  grid-template-columns: 200px minmax(300px, 1fr);  
}
```

Natija: o'ng ustun **kamida** 300px bo'ladi, lekin joy bo'lsa `1fr` bo'lib cho'ziladi. Ekran torayganda ham 300px dan past tushmaydi.

✦ `minmax(0, 1fr)` — muhim hiyla. Default holda `fr` ustunlari ichidagi mazmun (uzun matn, katta rasm) ularni minimal o'lchamdan kengaytirib yuborishi mumkin, bu esa layoutni "portlatadi". `minmax(0, 1fr)` minimumni `0` ga tushirib, ustunlarning haqiqatan teng bo'lishini ta'minlaydi.

```
/* matn juda uzun bo'lsa ham ustunlar teng qoladi */  
grid-template-columns: repeat(3, minmax(0, 1fr));
```

`minmax()` ning eng kuchli qo'llanishi keyingi bo'limda — responsive galereya.

16.11 `auto-fill` va `auto-fit` — chinakam responsive grid

Mana CSS Grid'ning eng sehrli qismi. Tasavvur qil: kartochkalar galereyasi kerak, har kartochka kamida 200px, lekin ekran qanchalik keng bo'lsa, shuncha ko'p kartochka bir qatorga sig'sin — **media query'siz**.

`repeat()` ichida son o'rniga `auto-fill` yoki `auto-fit` yozamiz:

```
.galereya {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  gap: 16px;  
}
```

Buni shunday o'qing: "har biri kamida 200px bo'lgan, joy bo'lsa cho'ziluvchi (`1fr`) ustunlarni **iloji boricha ko'p** yarat". Ekran kengaysa — yangi ustun qo'shiladi; torayganda — ustunlar pastga tushadi. Sehrli responsive grid bitta qatorda!

auto-fill va auto-fit farqi

Ular elementlar **konteynerni to'ldirmaganda** farq qiladi:

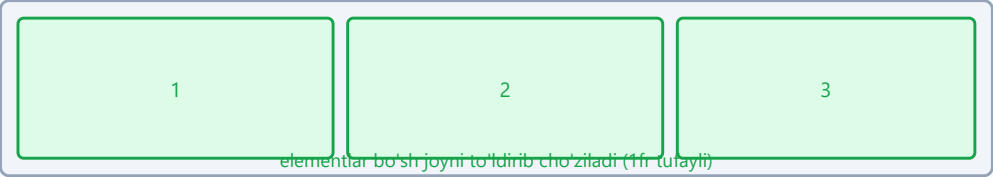
- **auto-fill** — joy yetganicha ustun "uyasi" yaratadi, hatto **bo'sh** bo'lsa ham. Elementlar tugagach, qolgan joyda ko'rinmas bo'sh ustunlar turadi.
- **auto-fit** — bo'sh ustunlarni **yig'ib tashlaydi (collapse)**, mavjud elementlarni `1fr` bilan butun joyga cho'zadi.

auto-fill va auto-fit farqi
`repeat(auto-fill / auto-fit, minmax(150px, 1fr))` — keng konteyner, atigi 3 ta element

auto-fill
bo'sh, ko'rinmas ustunlarni HAM yaratadi



auto-fit
bo'sh ustunlarni YIG'ADI, mavjudlarini cho'zadi



Qachon nima?
Galereyada elementlar kengaysin desangiz — auto-fit. Ustun "to'r"i barqaror qolsin desangiz — auto-fill.

✦ **Qaysi birini tanlash?** - Elementlar bo'sh joyni **to'ldirib cho'zilsin** desangiz — `auto-fit`. Ustun "to'r"i **barqaror** qolsin, elementlar belgilangan kenglikda turaversin desangiz — `auto-fill`.

Ko'p hollarda galereya uchun `auto-fit` tabiiyroq ko'rinadi.

💡 Bu naqsh — "RAM (Repeat, Auto, Minmax)" deb ataladi va zamonaviy CSS'da eng mashhur responsive texnikalardan biri. Bir qatorda butun moslashuvchan galereya tayyor.

16.12 Implicit va explicit grid, `grid-auto-rows`

`grid-template-columns` va `grid-template-rows` bilan e'lon qilingan panjara — **explicit grid** (aniq belgilangan panjara). Lekin elementlar belgilangandan ko'p bo'lsa-

chi?

```
.panjara {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: 100px; /* faqat 1 qator e'lon qildik */  
}
```

Agar bu panjaraga 7 ta element joylasak — birinchi 3 tasi e'lon qilingan 100px qatorga tushadi, qolganlari uchun Grid **avtomatik yangi qatorlar** yaratadi. Bu yangi qatorlar — **implicit grid** (yashirin panjara).

Muammo: avtomatik qatorlar default `auto` balandlikda bo'ladi (ya'ni mazmunga qarab). Ularni bir xil qilish uchun `grid-auto-rows`:

```
.panjara {  
  grid-template-columns: repeat(3, 1fr);  
  grid-auto-rows: 100px; /* avtomatik yaratiladigan har qator 100px */  
}
```

Natija: nechta element bo'lishidan qat'i nazar, hamma qatorlar 100px bo'ladi. Bu — element soni oldindan noma'lum (masalan, ma'lumotlar bazasidan keluvchi ro'yxat) bo'lganda juda foydali.

✦ Xuddi shunday `grid-auto-columns` ham bor — u elementlar gorizontaal yo'nalishda oqsa (`grid-auto-flow: column`) ishlaydi.

`grid-auto-flow` esa elementlar bo'sh kataklarni qanday to'ldirishini boshqaradi: - `row` (default) — qatorma-qator (chapdan o'ngga, keyin pastga). - `column` — ustunma-ustun (yuqoridan pastga, keyin o'ngga). - `dense` — Grid bo'sh qolgan kataklarni "orqaga qaytib" to'ldirishga harakat qiladi (galereyada teshiklarni yopish uchun).

```
.panjara {  
  grid-auto-flow: dense; /* bo'sh joylarni keyingi sig'adigan element  
  bilan to'ldir */  
}
```

⚠ `dense` chiroyli ko'rinsa-da, u elementlarning **vizual tartibini** o'zgartiradi (HTML tartibidan farqli). Bu klaviatura/skrinrider foydalanuvchilari uchun chalkashlik tug'dirishi mumkin, shuning uchun ehtiyot bo'l.

16.13 Amaliy: to'liq sahifa layout

Endi o'rganganlarimizni birlashtirib, klassik "holy grail" sahifa skeletini quramiz: header, sidebar, main, footer — va u mobilda bitta ustunga aylansin.

```
<div class="sahifa">
  <header class="head">Logotip va menyu</header>
  <aside class="side">Yon panel</aside>
  <main class="content">Asosiy mazmun</main>
  <footer class="foot">Footer ma'lumotlari</footer>
</div>
```

```
* { box-sizing: border-box; }

.sahifa {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 16px;
  min-height: 100vh;
  padding: 16px;
}

.head { grid-area: header; background: #dbeafe; padding: 16px; }
.side { grid-area: sidebar; background: #ede9fe; padding: 16px; }
.content { grid-area: main; background: #dcfce7; padding: 16px; }
.foot { grid-area: footer; background: #fef3c7; padding: 16px; }

/* Mobil: 700px dan tor ekranda bitta ustun */
@media (max-width: 700px) {
  .sahifa {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "main"
      "sidebar"
      "footer";
  }
}
```

Diqqat: mobil versiyada biz faqat `grid-template-columns` va `grid-template-areas` ni qayta yozdik. HTML'ga **umuman tegmadik**. Hatto sidebar ni main dan **keyinga** qo'ydik — vizual tartib o'zgardi, lekin HTML tartibi o'sha-o'sha qoldi. Bu — Grid'ning eng kuchli ustunliklaridan biri.

💡 `grid-template-rows: auto 1fr auto` naqshini eslab qol: header va footer mazmuniga qarab (`auto`), o'rtadagi main esa qolgan **butun balandlikni** (`1fr`)

egallaydi. `min-height: 100vh` bilan birga footer hamisha ekran tubida (yoki pastida) turadi — "sticky footer".

16.14 Amaliy: galereya va dashboard

Responsive rasm galereyasi

Media query'siz, o'z-o'zidan moslashuvchi galereya:

```
.galereya {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  grid-auto-rows: 180px;
  gap: 12px;
}
.galereya img {
  width: 100%;
  height: 100%;
  object-fit: cover; /* rasm katakni to'ldirsin, nisbati buzilmasin */
  border-radius: 8px;
}
```

Bitta rasmni **kattaroq** qilib ajratib ko'rsatish (masonry kabi effekt):

```
.galereya .katta {
  grid-column: span 2; /* ikki ustun keng */
  grid-row: span 2;    /* ikki qator baland */
}
```

Dashboard (boshqaruv paneli)

Dashboard — turli o'lchamdagi "kartochkalar" (widgetlar) panjarasi. Grid bu yerda yorqin ko'rinadi:

```
.dashboard {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 160px;
  gap: 20px;
}

/* katta statistika kartasi — 2x2 */
.karta--katta { grid-column: span 2; grid-row: span 2; }

/* keng grafik — butun en bo'ylab */
.karta--grafik { grid-column: 1 / -1; }
```

```
/* oddiy kartalar bitta katakni egallaydi (alohida qoida shart emas) */
```

```
<div class="dashboard">  
  <div class="karta karta--katta">Asosiy KPI</div>  
  <div class="karta">Foydalanuvchilar</div>  
  <div class="karta">Daromad</div>  
  <div class="karta">Buyurtmalar</div>  
  <div class="karta karta--grafik">Oylik grafik</div>  
</div>
```

Natija: turli o'lchamdagi kartalar 4 ustunli panjarada tartibli joylashadi. `span` va `1 / -1` orqali ba'zilar kengroq/balandroq bo'ladi, qolganlari bo'sh kataklarni to'ldiradi.

✦ Diqqat qil: bu yerda Grid sahifa skeletini, har kartaning **ichidagi** mazmun (sarlavha + raqam + ikonka) esa odatda Flexbox bilan tekislanadi. Ikkalasi birga — zamonaviy layoutning poydevori.

16.15 Tez-tez uchraydigan xatolar va maslahatlar

⚠ **gap o'rniga margin ishlatish.** Katakalar orasini `margin` bilan ajratish chetlarda ortiqcha bo'shliq qoldiradi va hisoblashni murakkablashtiradi. `gap` ishlatish — ancha toza yechim.

⚠ **Bevosita bo'lmagan farzandlar.** Faqat konteynerning **to'g'ridan-to'g'ri** farzandlari grid item bo'ladi. Agar `.katak` ni yana bir `div` ichiga o'rasangiz, u panjaraga bo'ysunmaydi.

⚠ **fr ustunlarining "portlashi".** Uzun matn yoki katta element `1fr` ustunni minimal o'lchamdan kengaytirib yuborishi mumkin. Yechim: `minmax(0, 1fr)`.

💡 **DevTools Grid inspektorini ishlat.** Chrome/Firefox DevTools'da `display: grid` elementi yonida kichik "grid" yorlig'i chiqadi. Uni bosing — brauzer chiziq raqamlari, yo'laklar va sohalarni ekranda chizib beradi. Grid'ni o'rganishda bundan foydali narsa yo'q.

💡 **Grid'mi yoki Flexbox'mi?** Agar element **bir yo'nalishda** tizilsa (navbar, tugmalar) — Flexbox. Agar **ham ustun, ham qatorni** bir vaqtda boshqarish kerak bo'lsa (sahifa skeleti, galereya, jadval kabi tuzilma) — Grid. Ikkilanmang: ko'pincha ikkalasini birga ishlatasiz.

✦ **Brauzer qo'llab-quvvatlashi.** CSS Grid 2017-yildan beri barcha zamonaviy brauzerlarda to'liq ishlaydi. Bugun uni xavfsiz ishlatish mumkin — `-prefix` larga ehtiyoj yo'q.

Mashqlar

Quyidagi mashqlarni brauzerda (yoki Chrome DevTools Grid inspektorida) sinab ko'r. Avval o'zing javob ber, keyin yechimni och.

Mashq 1 — Uchta teng ustun

`display: grid` ishlatib, 6 ta kartochkani 3 ta **teng kenglikdagi** ustunga joylashtir, kataklar orasida 16px bo'shliq bo'lsin.

 **Yechim** 

```
.panjara {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 16px;  
}
```

`repeat(3, 1fr)` uchta teng ustun beradi, `gap: 16px` esa bo'shliqni. `1fr` (foiz emas) tanlangani uchun ustunlar har qanday ekranda teng qoladi.

Mashq 2 — Sidebar + cho'ziluvchi mazmun

Chap tomonda aniq **250px** sidebar, o'ng tomonda **qolgan butun joyni** egallaydigan mazmun ustuni yaratuvchi `grid-template-columns` ni yoz.

 **Yechim** 

```
.layout {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
}
```

Sidebar qat'iy 250px, `1fr` esa qolgan bo'sh joyni to'liq egallaydi. Ekran kengaysa, faqat o'ng ustun cho'ziladi.

Mashq 3 — Butun qatorni egallash

4 ustunli panjarada bitta sarlavha elementini **butun en** bo'ylab cho'zmoqchisiz. Ustunlar soni keyin o'zgarsa ham ishlaydigan ikki xil yo'lini yoz.



Yechim



```
/* 1-usul: oxirgi chiziqqa cho'z (ustunlar soniga bog'liq emas) */  
.sarlavha { grid-column: 1 / -1; }  
  
/* 2-usul: span bilan (faqat shu holatda, 4 ustun uchun) */  
.sarlavha { grid-column: span 4; }
```

1 / -1 afzalroq, chunki ustunlar soni o'zgarganda ham avtomatik moslashadi.

Mashq 4 — Nomli sohalar bilan layout

`grid-template-areas` ishlatib quyidagi layoutni yarat: yuqorida butun en bo'ylab `nav`, ostida chapda `sidebar`, o'ngda `main`, eng pastda butun en bo'ylab `footer`.



Yechim



```
.sahifa {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-areas:  
    "nav nav"  
    "sidebar main"  
    "footer footer";  
  gap: 12px;  
}  
.sahifa nav { grid-area: nav; }  
.sahifa aside { grid-area: sidebar; }  
.sahifa main { grid-area: main; }  
.sahifa footer { grid-area: footer; }
```

`nav` va `footer` ikki ustunni qamragani uchun nom ikki marta takrorlandi.

Mashq 5 — Responsive galereya

Media query **ishlatmasdan**, har biri kamida 220px bo'lgan, ekran kengaytganda avtomatik ko'payadigan va bo'sh joyni to'ldirib cho'ziladigan kartochkalar galereyasini yoz.



Yechim



```
.galereya {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 16px;  
}
```

`auto-fit` bo'sh ustunlarni yig'ib, mavjud kartochkalarni `1fr` bilan cho'zadi. `minmax(220px, 1fr)` — "kamida 220px, joy bo'lsa kengay". Bu — "RAM" naqshi.

Mashq 6 — Markazga joylash

Panjarada barcha elementlarni o'z kataklarining ham gorizontal, ham vertikal markaziga joylashtir.



Yechim



```
.panjara {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: repeat(2, 120px);  
  justify-items: center; /* gorizontal markaz */  
  align-items: center; /* vertikal markaz */  
}
```

`justify-items` gorizontal (ustun) o'qni, `align-items` vertikal (qator) o'qni boshqaradi. Faqat bitta element uchun bo'lsa `justify-self` / `align-self` ishlatilardi.

Mashq 7 — Element joylashtirish (chiziq raqamlari)

5 ustunli panjarada bitta elementni **2-ustundan boshlab 3 ustun keng** va **1-qatordan 2 qator baland** qilib joylashtir.



Yechim



```
.element {  
  grid-column: 2 / span 3; /* 2-chiziqdan boshla, 3 ustun cho'zil (2..5) */  
  grid-row: 1 / span 2; /* 1-chiziqdan boshla, 2 qator cho'zil (1..3) */  
}
```

`grid-column: 2 / 5` ham aynan shu natijani beradi (2-chiziqdan 5-chiziqgacha = 3 ustun).

Mashq 8 — Implicit grid balandligi

Element soni oldindan noma'lum bo'lgan ro'yxat bor. 3 ustunli panjarada nechta element bo'lishidan qat'i nazar, **har qator aniq 140px** bo'lishini ta'minla.

Yechim

```
.royxat {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 140px; /* avtomatik yaratiladigan qatorlar 140px */
  gap: 12px;
}
```

`grid-template-rows` emas, balki `grid-auto-rows` ishlatildi, chunki qatorlar (element soniga qarab) avtomatik — ya'ni implicit grid'da yaratiladi.

17 — Responsive dizayn

◀ Oldingi: 16 — CSS Grid · 🏠 README · Keyingi: 18 — Transitions, transforms va animatsiyalar ▶

Bu bobda: bitta sahifani har qanday ekranga — telefon, planshet, desktop — moslashtirishni o'rganamiz: media query, mobile-first yondashuv, fluid layout va `clamp()` bilan moslashuvchan tipografiya.

17.1 Responsive nima va nega kerak?

Tasavvur qil: sen chiroyli sahifa yasading va u 27 dyumli monitoringda ajoyib ko'rinadi. Keyin do'sting uni telefonida ochadi — matn mayda, tugmalar bir-biriga yopishgan, sahifani o'qish uchun ikki barmoq bilan kattalashtirish kerak. Tanish holat, to'g'rimi?

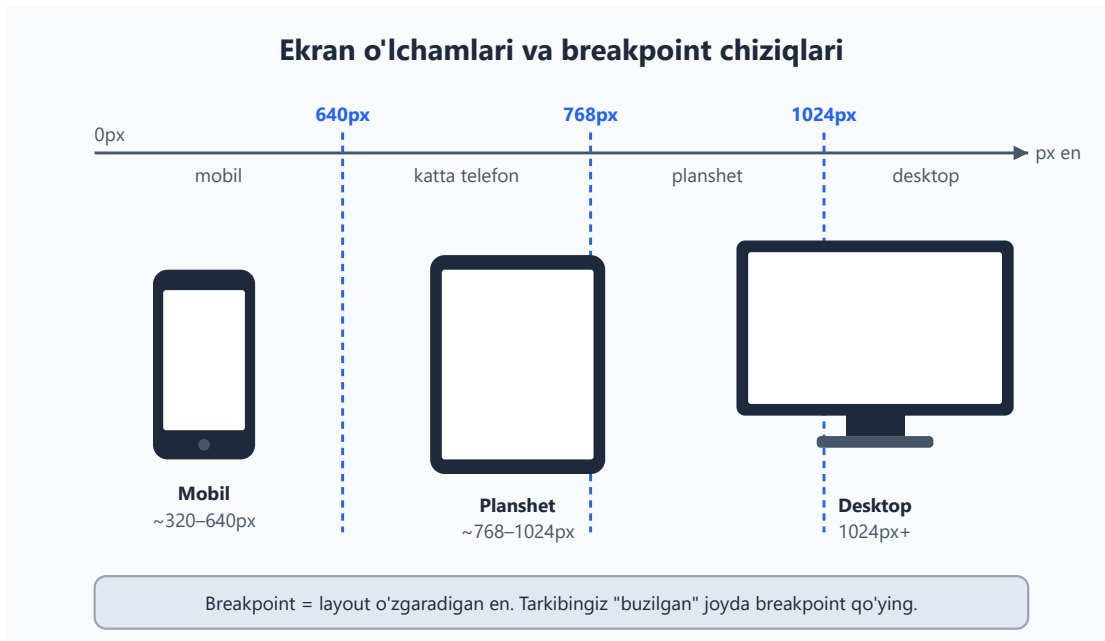
Responsive dizayn (responsive design — "javob beradigan dizayn") — bu bitta sahifaning **o'zi ekran o'lchamiga qarab o'zgarishi**. Telefonda bir ustun, planshetda ikki ustun, desktopda uch ustun — lekin HTML bitta, sahifa ham bitta.

Nega bu shunchalik muhim?

- **Foydalanuvchilarning yarmidan ko'pi telefon orqali kiradi.** Agar saytingiz telefonda noqulay bo'lsa, ular ketadi.
- **Bitta kod — barcha qurilmalar.** "Mobil versiya" va "desktop versiya" deb ikki alohida sayt qilish — eskirgan, qimmat va xatolarga to'la usul.
- **Google mobil-do'stlikni reyting omili sifatida ko'radi.** Yomon moslashgan sayt qidiruvda pastroq turadi.

✦ **Atamalar:** **viewport** — brauzer oynasining sahifa ko'rinadigan qismi (ko'rinish maydoni). **breakpoint** — layout o'zgaradigan ekran eni (sindirish nuqtasi). **fluid** — qattiq piksel o'rniga foiz/nisbat bilan "oqadigan" o'lcham.

Quyidagi diagramma uchta asosiy qurilma sinfini va tipik breakpoint chiziqlarini ko'rsatadi:



17.2 Viewport meta tegi — birinchi qadam (eslatma)

7-bobda viewport meta tegini ko'rgan eding. Bu — responsive dizaynning **po'ydevori**, shuning uchun qisqacha eslatib o'tamiz: usiz hech qanday media query ishlamaydi.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Bu nima qiladi? Telefon brauzerlari tarixiy sabablarga ko'ra sahifani **980px keng** deb hisoblab, keyin uni kichraytirib ko'rsatadi (shuning uchun moslashmagan saytlar telefonda kichkina ko'rinadi). `width=device-width` brauzerga: "sahifa eni qurilmaning **haqiqiy** eni bilan teng bo'lsin" deb aytadi. `initial-scale=1.0` esa boshlang'ich kattalashtirishni 1:1 qilib qo'yadi.

⚠ **Eng keng tarqalgan xato:** responsive CSS yozasiz, lekin viewport tegini `<head>` ga qo'shishni unutasiz. Natijada media query'lar telefonda **umuman ishlamaydi**, chunki brauzer eni hali ham 980px deb o'ylaydi. Yangi sahifa boshlaganda bu teg birinchi yoziladigan narsalardan bo'lsin.

17.3 Media query — sintaksis

Media query (media so'rovi) — bu CSS'ga "agar ekran shu shartga mos kelsa, mana bu stillarni qo'lla" deyish usuli. Sintaksis `@media` bilan boshlanadi:

```

/* Asosiy stil – barcha ekranlarga */
.menu {
  display: block;
}

/* Faqat eni 768px va undan KATTA ekranlarda */
@media (min-width: 768px) {
  .menu {
    display: flex;
  }
}

```

Ichidagi qoidalar **faqat shart bajarilganda** kuchga kiradi. Yuqorida: `.menu` odatda `block` (har element alohida qatorda), lekin ekran 768px dan keng bo'lsa — `flex` (yonma-yon).

Asosiy shartlar

Shart	Ma'nosi
<code>(min-width: 768px)</code>	eni 768px va undan katta bo'lsa
<code>(max-width: 767px)</code>	eni 767px va undan kichik bo'lsa
<code>screen</code>	media turi: ekran (chop etish emas)

and, or va vergul

Shartlarni birlashtirish mumkin:

```

/* HAM min, HAM max – ya'ni 768px dan 1023px gacha bo'lgan oraliq */
@media (min-width: 768px) and (max-width: 1023px) {
  .karta { background: #f8fafc; }
}

/* YOKI – vergul "or" vazifasini bajaradi */
@media (max-width: 600px), (orientation: portrait) {
  .panel { display: none; }
}

```

- `and` — ikkala shart ham bajarilishi kerak.
- Vergul (,) — **or** (yoki): shartlardan **bittasi** bajarilsa kifoya.
- `screen and (min-width: 768px)` — media turini ham qo'shsangiz bo'ladi, lekin amalda `screen` ni ko'pincha tushirib qoldiramiz, chunki standart shart shunday

ham yetarli.

💡 **Eslatma:** zamonaviy CSS yangi sintaksisni ham qo'llab-quvvatlaydi: `@media (width >= 768px)`. Bu `(min-width: 768px)` bilan bir xil, faqat o'qish osonroq. Brauzer qo'llab-quvvatlashini tekshirib, bimalol foydalansangiz bo'ladi.

17.4 Mobile-first yondashuv — nega aynan shunday?

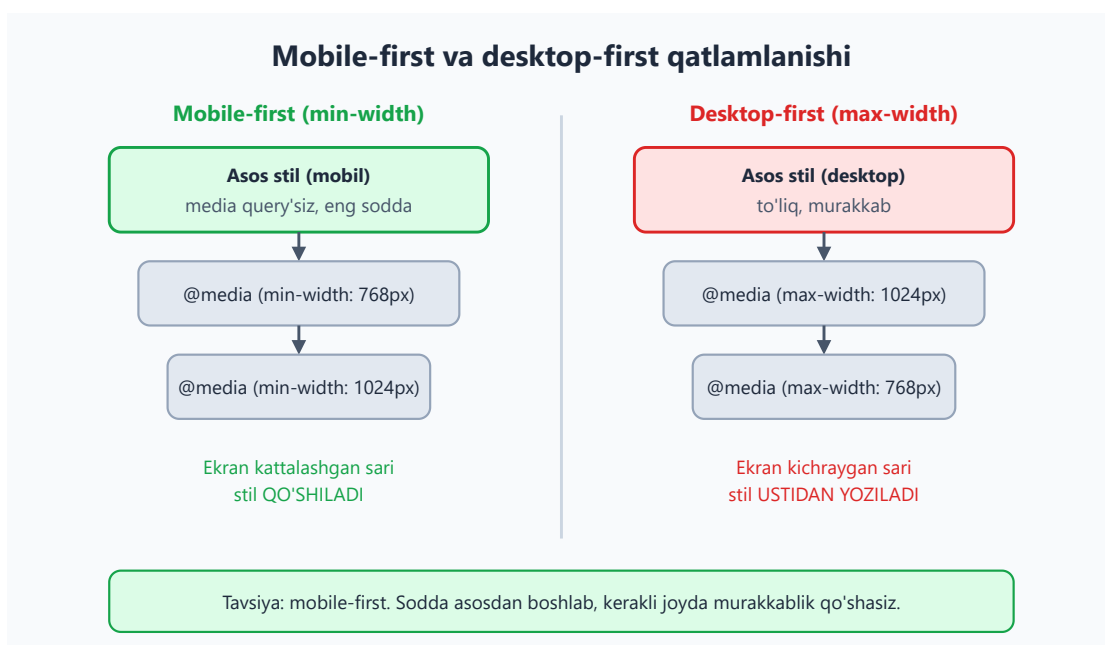
Endi muhim falsafiy savol: media query'larni qaysi tomondan yozish kerak — kichikdan kattagami yoki kattadan kichikkami? Ikki yo'l bor.

Desktop-first (kattadan kichikka): asosiy stilni desktop uchun yozasiz, keyin `max-width` bilan kichik ekranlar uchun ustidan o'zgartirasiz.

Mobile-first (kichikdan kattaga): asosiy stilni eng sodda — mobil — holatga yozasiz, keyin `min-width` bilan kattaroq ekranlarga murakkablik **qo'shasiz**.

Zamonaviy amaliyot — **mobile-first**. Nega?

1. **Sodda asos.** Mobil layout odatda eng oddiy (bir ustun, vertikal). Murakkab narsani qo'shish, murakkabni soddalashtirishdan oson.
2. **Tezlik.** Eski/zaif telefonlar `max-width` ichidagi murakkab desktop qoidalarini baribir o'qib, keyin bekor qiladi. Mobile-first'da telefon faqat o'ziga kerakli sodda qoidalarni oladi.
3. **Mantiqiy o'sish.** "Ekran kattalashdi — endi nima qo'shaman?" deb fikrlash, "ekran kichrayd — nimani olib tashlayman?" dan tabiiyroq.



Mobile-first ko'rinishi:

```
/* ASOS: mobil (media query'siz) */
.grid {
  display: grid;
  grid-template-columns: 1fr; /* bitta ustun */
  gap: 16px;
}

/* Planshet va undan katta: ikki ustun */
@media (min-width: 768px) {
  .grid {
    grid-template-columns: 1fr 1fr;
  }
}

/* Desktop: uch ustun */
@media (min-width: 1024px) {
  .grid {
    grid-template-columns: 1fr 1fr 1fr;
  }
}
```

Natija: telefonda kartalar tik bir ustun bo'lib chiqadi, planshetda ikkitadan, desktopda uchtdan. Diqqat qil — asos qoidasi **eng sodda**, har breakpoint faqat o'zgargan narsani qo'shadi.

17.5 Tipik breakpoint'lar

Qaysi enlarda breakpoint qo'yish kerak? **Sehrli raqamlar yo'q**, lekin amaliyotda quyidagilar keng tarqalgan (ko'p framework'lar shularga yaqin):

Breakpoint	Tipik qurilma
480px	katta telefon
640px	telefon / kichik planshet chegarasi
768px	planshet (portret)
1024px	planshet (landshaft) / kichik laptop
1280px	desktop

💡 **Eng yaxshi maslahat — tarkibga qarab breakpoint qo'ying, qurilmaga emas.**

Brauzer oynasini asta torting: layout qayerda **xunuklashadi** (matn juda cho'zilib ketdi, kartalar siqildi), o'sha joyda breakpoint qo'ying. Aniq telefon modeliga moslashishga urinish behuda — har yili yangi o'lchamlar chiqadi.

⚠️ **Xato:** o'nlab breakpoint qo'yib, har biriga alohida stil yozish. Bu kodni boshqarib bo'lmas holga keltiradi. Odatda 2–3 ta yaxshi tanlangan breakpoint yetarli. Eng yaxshi responsive layout — **breakpoint kam talab qiladigan** layout (keyingi bo'limlarda buni qanday qilishni ko'ramiz).

17.6 Fluid layout — "oqadigan" tartib

Media query — bu **sakrash**: 767px da bir ko'rinish, 768px da boshqasi. Lekin orada-chi? Bu yerda **fluid layout** yordamga keladi: qattiq piksel o'rniga **nisbiy** o'lchamlar ishlatamiz, shunda tartib har eng **silliqlik** moslashadi.

Foiz (%) bilan

```
.container {  
  width: 90%;           /* ota elementning 90% i — har enda mos */  
  max-width: 1200px;    /* lekin 1200px dan oshmasin */  
  margin: 0 auto;       /* markazga */  
}
```

`width: 90%` — eni har doim ota elementning 90% i bo'ladi, ya'ni ekran bilan birga oqadi.
`max-width: 1200px` esa juda keng monitorlarda matn cheksiz cho'zilib ketishining oldini oladi.

✳️ Bu naqsh — `width: 90% + max-width + margin: 0 auto` — eng ko'p ishlatiladigan "konteyner" retsepti. Yodlab oling.

Flexbox va Grid bilan (eng kuchli usul)

15- va 16-boblarda o'rgangan flexbox/grid o'zi tabiatan fluid. `fr` birligi va `flex` xossasi joyni nisbatda taqsimlaydi:

```
/* Grid: ustunlar joyga avtomatik moslashadi, media query'siz! */  
.galereya {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));  
  gap: 16px;  
}
```

Bu sehrli qatorni ochib beraylik:

- `minmax(200px, 1fr)` — har ustun **kamida 200px**, joy bo'lsa **1fr** gacha cho'ziladi.
- `auto-fill` — joyga **qancha** ustun sig'sa, shunchasini yaratadi.

Natija: ekran kengaysa, ustunlar soni o'zi ko'payadi; torayganda — kamayadi. Bitta ham `@media` yozmadik! Aynan shu — modern responsive'ning go'zalligi: **layoutni o'zi moslashuvchan qil, breakpoint'larga kamroq tayan.**

17.7 Fluid tipografiya: `clamp()`, `min()`, `max()`

Tartib oqadi — yaxshi. Ammo shrift o'lchami-chi? Telefonda 32px sarlavha juda katta, desktopda 18px sarlavha juda kichik. Eski yechim: har breakpointda `font-size` ni qo'lda o'zgartirish. Zamonaviy yechim — `clamp()`.

`clamp()` uchta qiymat oladi:

```
font-size: clamp(MIN, AFZAL, MAX);
```

- **MIN** — eng kichik ruxsat etilgan qiymat (pol).
- **AFZAL** — odatda ekranga bog'liq qiymat (masalan `2.5vw`), u o'sib boradi.
- **MAX** — eng katta ruxsat etilgan qiymat (shift).

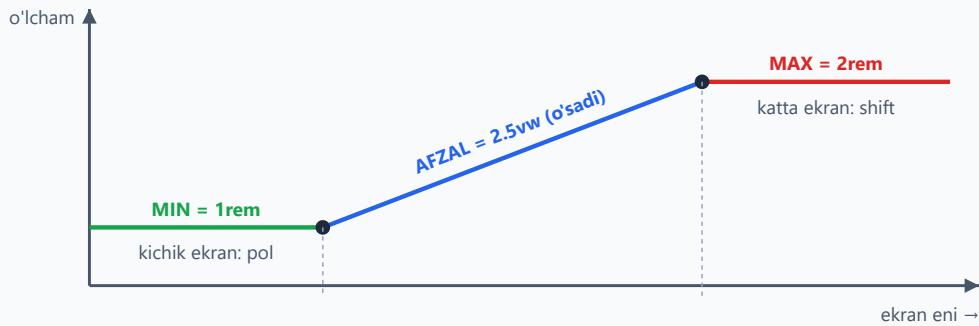
Brauzer AFZAL qiymatni hisoblaydi, lekin uni MIN dan past, MAX dan baland chiqishiga yo'l qo'ymaydi.

```
h1 {  
  font-size: clamp(1.5rem, 5vw, 3rem);  
}
```

Natija: kichik telefonda sarlavha 1.5rem (24px), ekran kengaygan sari `5vw` bilan silliq o'sadi, katta ekranda 3rem (48px) da to'xtaydi. **Bitta ham media query yo'q.**

clamp(MIN, AFZAL, MAX) qanday ishlaydi

font-size: clamp(1rem, 2.5vw, 2rem)



Matn na juda kichik, na juda katta — orada silliq o'zgaradi. Media query'siz!

📌 **vw nima?** vw (viewport width) — viewport enining 1% i. 5vw = ekran enining 5% i. Shuning uchun u ekran bilan birga o'sadi.

min() va max() — yakka tartibda ham foydali

clamp() ning yarim aka-ukalari:

```
/* Kenglik: 90% YOKI 600px — qaysi KICHIK bo'lsa */
.matn { width: min(90%, 600px); }

/* Padding: 16px YOKI 3vw — qaysi KATTA bo'lsa */
.qism { padding: max(16px, 3vw); }
```

- min(a, b) — ikkisidan **kichigini** tanlaydi (shift vazifasi: "bundan kattalashmasin").
- max(a, b) — ikkisidan **kattasini** tanlaydi (pol vazifasi: "bundan kichiklashmasin").

💡 Aslida clamp(A, B, C) aynan max(A, min(B, C)) bilan teng — clamp() shunchaki uni qisqaroq yozish usuli.

⚠️ **Xato:** font-size 'da faqat vw ishlatish (font-size: 5vw). Juda kichik ekranda matn o'qib bo'lmas darajada mayda, juda katta ekranda esa ulkan bo'ladi. **Doim clamp() bilan chegaralang.**

17.8 Responsive rasmlar — qisqacha eslatma

3-bobda batafsil ko'rgan edik, shuning uchun bu yerda faqat eslatib o'tamiz: responsive sahifada rasmlar ham ekranga moslashishi kerak.

Eng asosiy qadam — har rasm konteyneridan oshib ketmasin:


```
img {  
  max-width: 100%;  
  height: auto;  
}
```

`max-width: 100%` — rasm hech qachon ota elementidan keng bo'lmaydi. `height: auto` — nisbat saqlanadi (rasm cho'zilmaydi).

Tezlik uchun esa har qurilmaga **mos o'lchamdagi** faylni yuborish kerak — bu `srcset` va `<picture>` bilan qilinadi (3-bobni eslang):

```

```

Brauzer ekran eni va `sizes` ga qarab `srcset` dan eng mos faylni tanlaydi — telefonga kichik fayl, desktopga katta fayl. Bu trafikni tejaydi va sahifani tezlashtiradi.

17.9 Container queries — element o'lchamiga qarash

Mana, responsive'ning eng yangi va kuchli quroli. Media query'ning bitta cheklovi bor: u doim **butun viewport** o'lchamiga qaraydi. Lekin ko'pincha bizga kerakli narsa — **komponentning o'zi joylashgan joyning** o'lchami.

Misol: bir "mahsulot kartasi" komponenti bor. U keng asosiy hududda turganda — gorizontal (rasm chapda, matn o'ngda). Tor yon ustunda turganda — vertikal (rasm tepada). Media query bu yerda ojiz: viewport bir xil bo'lsa-da, **kartaning eni** ikki joyda har xil!

Container query aynan shu muammoni hal qiladi: u stilni viewport'ga emas, **ota konteyner o'lchamiga** qarab qo'llaydi.

Media query vs Container query



Ikki qadamda ishlaydi. Avval ota elementni "konteyner" deb e'lon qilamiz:

```
.karta-joyi {  
  container-type: inline-size; /* eni kuzatiladigan konteyner */  
}
```

Keyin `@container` bilan farzandni konteyner eniga qarab o'zgartiramiz:

```
/* Asos: vertikal (rasm tepada) */  
.karta {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
/* Konteyner eni 400px dan oshsa: gorizontaL */  
@container (min-width: 400px) {  
  .karta {  
    grid-template-columns: 150px 1fr; /* rasm | matn */  
  }  
}
```

Natija: bitta `.karta` komponenti keng joyda gorizontaL, tor joyda vertikaL ko'rinadi — viewport emas, **o'z konteyneri** qancha keng ekaniga qarab. Endi shu kartani saytning istalgan joyiga qo'ysangiz, u o'zini to'g'ri tutadi.

💡 **Nega bu o'yinni o'zgartiradi?** Komponentlar endi **chinakam mustaqil** bo'ladi. Avval har komponent "men sahifaning qayerida turibman?" deb bosh qotirardi; endi u faqat "menga qancha joy berildi?" deb so'raydi. Bu — qayta ishlatiladigan komponentlar uchun ideal.

✦ Container query'lar 2023 yildan barcha yirik brauzerlarda qo'llab-quvvatlanadi. Eski brauzerlarni qo'llab-quvvatlash zarur bo'lsa, media query'ni zaxira sifatida qoldiring.

17.10 Foydalanuvchi afzalliklari: `prefers-color-scheme` va `prefers-reduced-motion`

Media query faqat ekran o'lchamini emas, foydalanuvchining **tizim sozlamalarini** ham o'qiy oladi. Ikkitasi ayniqsa muhim.

`prefers-color-scheme` — qorong'i/yorug' rejim

Foydalanuvchi telefonini "tungi rejim"ga qo'ygan bo'lsa, saytingiz ham unga moslashishi mumkin:

```
/* Asos: yorug' rejim */
body {
  background: #ffffff;
  color: #1e293b;
}

/* Foydalanuvchi qorong'i rejimni tanlagan bo'lsa */
@media (prefers-color-scheme: dark) {
  body {
    background: #1e293b;
    color: #f8fafc;
  }
}
```

Natija: tizimi qorong'i rejimda bo'lgan foydalanuvchi saytni ham qorong'i ko'radi — ko'zga qulay, batareyani tejaydi.

`prefers-reduced-motion` — kamroq harakat

Ba'zi odamlar uchun ekrandagi animatsiyalar bosh aylanishi yoki ko'ngil aynishiga sabab bo'ladi (vestibulyar buzilish). Ular tizimida "harakatni kamaytir" ni yoqishadi. Bunga hurmat ko'rsatamiz:

```
/* Asos: silliq animatsiya */
.tugma {
  transition: transform 0.3s;
}

/* Foydalanuvchi kamroq harakat so'ragan bo'lsa: animatsiyani o'chir */
@media (prefers-reduced-motion: reduce) {
```

```
.tugma {  
  transition: none;  
}  
}
```

💡 Bu — **accessibility** (qulaylik) masalasi. Responsive dizayn faqat ekran o'lchami emas — bu **har bir foydalanuvchiga** uning ehtiyojiga ko'ra moslashishdir.

17.11 Amaliy: bitta layoutni mobil va desktopga moslash

Endi o'rgangan hamma narsani birlashtirib, to'liq, ishlaydigan responsive sahifa quramiz. Maqsad: mobil-first, fluid, `clamp()` bilan tipografiya.

```
<!DOCTYPE html>  
<html lang="uz">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Responsive sahifa</title>  
</head>  
<body>  
  <header class="sayt-bosh">  
    <div class="logo">Mening Saytim</div>  
    <nav class="menu">  
      <a href="#">Bosh</a>  
      <a href="#">Mahsulotlar</a>  
      <a href="#">Aloqa</a>  
    </nav>  
  </header>  
  
  <main class="grid">  
    <article class="karta">Karta 1</article>  
    <article class="karta">Karta 2</article>  
    <article class="karta">Karta 3</article>  
  </main>  
</body>  
</html>
```

```
/* ===== ASOS: mobil (media query'siz) ===== */  
* { box-sizing: border-box; margin: 0; }  
  
body {  
  font-family: system-ui, sans-serif;  
  color: #1e293b;  
}  
  
/* Sarlavha tipografiyasi — fluid, clamp bilan */
```

```

.logo {
  font-size: clamp(1.25rem, 4vw, 2rem);
  font-weight: bold;
}

/* Header: mobilda tik (logo tepada, menyu pastda) */
.sayt-bosh {
  display: flex;
  flex-direction: column;
  gap: 8px;
  padding: max(16px, 3vw);
  background: #f8fafc;
}

.menu {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

/* Konteyner: fluid, lekin chegaralangan */
.grid {
  width: min(92%, 1100px);
  margin: 24px auto;
  display: grid;
  grid-template-columns: 1fr; /* mobilda 1 ustun */
  gap: 16px;
}

.karta {
  padding: 24px;
  background: #e2e8f0;
  border-radius: 8px;
}

/* ===== PLANSKET: 768px+ ===== */
@media (min-width: 768px) {
  .sayt-bosh {
    flex-direction: row; /* logo va menyu yonma-yon */
    justify-content: space-between;
    align-items: center;
  }
  .menu { flex-direction: row; } /* menyu gorizontal */
  .grid { grid-template-columns: 1fr 1fr; } /* 2 ustun */
}

/* ===== DESKTOP: 1024px+ ===== */
@media (min-width: 1024px) {
  .grid { grid-template-columns: 1fr 1fr 1fr; } /* 3 ustun */
}

```

Natija va tahlil:

- **Telefonda:** logo va menyu tik joylashadi, kartalar bir ustun bo'lib chiqadi. Sarlavha `clamp()` tufayli mos o'lchamda.
- **Planshetda (768px+):** header gorizontal bo'ladi (`flex-direction: row`), menyu yonma-yon, kartalar ikki ustun.
- **Desktopda (1024px+):** kartalar uch ustun.
- **Hamma yerda:** konteyner `min(92%, 1100px)` bilan oqadi, lekin 1100px dan oshmaydi.

Diqqat qil — asos stil **butunlay mobil** uchun yozilgan va media query'siz ham mukammal ishlaydi. Har breakpoint faqat **kerakli o'zgarishni qo'shadi**. Aynan shu — mobile-first'ning amaliy ko'rinishi.

💡 **Sinab ko'rish:** bu kodni faylga saqlang, brauzerda oching va oyna chetidan **astatorting**. Layoutning har breakpointda o'zgarishini o'z ko'zingiz bilan ko'rasiz. DevTools'dagi "qurilma rejimi" (device mode) ham buning uchun ajoyib vosita.

Mashqlar

1-mashq. Quyidagi sahifa telefonda buzilib ko'rinadi — matn mayda, gorizontal scroll chiqadi. `<head>` ga bitta qator qo'shib tuzating.

```
<head>
  <meta charset="UTF-8">
  <title>Mening saytim</title>
</head>
```

Yechim

Viewport meta tegi yetishmayapti. Usiz brauzer sahifani 980px deb hisoblaydi:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mening saytim</title>
</head>
```

Bu — responsive dizaynning birinchi va eng muhim qadami. Usiz hech qanday media query ishlamaydi.

2-mashq. `.qator` klassiga shunday CSS yozing: ekran **600px dan keng** bo'lsa, `display: flex` bo'lsin; aks holda (sukut bo'yicha) `display: block` qolsin. Mobile-first usulda yozing.



Yechim



```
/* Asos (mobil): block */
.qator {
  display: block;
}

/* 600px+ : flex */
@media (min-width: 600px) {
  .qator {
    display: flex;
  }
}
```

Asos qoidasi sodda holatga, media query esa kattaroq ekranga murakkablik qo'shadi — bu mobile-first naqshining mohiyati.

3-mashq. `h1` sarlavhasiga fluid tipografiya bering: hech qachon **20px** (1.25rem) dan kichik va **40px** (2.5rem) dan katta bo'lmasin, orada `6vw` bo'yicha o'ssin.



Yechim



```
h1 {
  font-size: clamp(1.25rem, 6vw, 2.5rem);
}
```

Tartib: `clamp(MIN, AFZAL, MAX)`. MIN — pol, MAX — shift, o'rtadagi `6vw` — silliq o'sib boruvchi afzal qiymat. Bironta ham media query kerak emas.

4-mashq. Grid galereya yarating: ustunlar **kamida 220px**, joy bo'lsa avtomatik ko'paysin, oralig'i 20px. **Bitta ham media query ishlatmang.**



Yechim



```
.galereya {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 20px;
}
```

`auto-fill + minmax(220px, 1fr)` kombinatsiyasi joyga qarab ustunlar sonini o'zi hisoblaydi — bu media query'siz responsive layoutning klassik usuli.

5-mashq. Konteyner shunday bo'lsin: eni ota elementning **90%** i, lekin hech qachon **800px** dan oshmasin, gorizontal markazda tursin. Avval ikki xossa bilan, keyin `min()` bilan bitta qatorda yozing.



Yechim



An'anaviy usul:

```
.container {  
  width: 90%;  
  max-width: 800px;  
  margin: 0 auto;  
}
```



`min()` bilan en bitta qatorda:

```
.container {  
  width: min(90%, 800px);  
  margin: 0 auto;  
}
```



`min(90%, 800px)` — ikkisidan kichigini tanlaydi: tor ekranda 90% (kichik), keng ekranda 800px (kichik). Natija `width` + `max-width` bilan bir xil.

6-mashq. Foydalanuvchi tizimida **qorong'i rejimni** tanlagan bo'lsa, `body` foni `#1e293b`, matn rangi `#f8fafc` bo'lsin. Yorug' rejim asos bo'lib qolsin.



Yechim



```
/* Asos: yorug' rejim */  
body {  
  background: #ffffff;  
  color: #1e293b;  
}  
  
/* Qorong'i rejim */  
@media (prefers-color-scheme: dark) {  
  body {  
    background: #1e293b;  
    color: #f8fafc;  
  }  
}
```



`prefers-color-scheme` foydalanuvchining tizim sozlamasini o'qiydi — ekran o'lchamiga aloqasi yo'q, lekin u ham media query orqali ishlaydi.

7-mashq (qiyinroq). `.karta-joyi` ni eni kuzatiladigan konteynerga aylantiring, so'ng ichidagi `.karta` shu **konteyner eni 350px dan oshganda** ikki ustunli grid (`120px 1fr`) bo'lsin. Boshqa hollarda bir ustunli qolsin.



Yechim



```
/* 1) Otani konteyner deb e'lon qilamiz */
.karta-joyi {
  container-type: inline-size;
}

/* 2) Asos: bir ustun */
.karta {
  display: grid;
  grid-template-columns: 1fr;
}

/* 3) Konteyner 350px dan oshsa: ikki ustun */
@container (min-width: 350px) {
  .karta {
    grid-template-columns: 120px 1fr;
  }
}
```

E'tibor bering: `@container` viewport'ga emas, `.karta-joyi` ning **eniga** qaraydi. Shuning uchun bu kartani saytning keng yoki tor istalgan joyiga qo'ysangiz, u o'zini o'sha joyga moslab tutadi.

8-mashq (debugging). Quyidagi media query telefonda **umuman ishlamayapti**, garchi CSS sintaksisi to'g'ri bo'lsa ham. HTML faylda nima yetishmayotganini toping.

```
<!DOCTYPE html>
<html>
<head>
  <title>Test</title>
  <link rel="stylesheet" href="style.css">
</head>
...
```

```
@media (max-width: 600px) {
  body { font-size: 14px; }
}
```



Yechim



CSS to'g'ri — muammo HTML'da: **viewport meta tegi yo'q**. Usiz telefon brauzeri sahifa enini 980px deb hisoblaydi, shuning uchun `max-width: 600px` sharti **hech qachon** bajarilmaydi ($980 > 600$).

Tuzatish — `<head>` ga qo'shish:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



Bu eng tez-tez uchraydigan responsive xato: CSS to'g'ri yozilgan, lekin viewport tegi unutilgan. Media query "ishlamayotgani"da birinchi shuni tekshiring.

[← Oldingi: 16 — CSS Grid](#) · [🏠 README](#) · [Keyingi: 18 — Transitions, transforms va animatsiyalar →](#)

18 — Transitions, transforms va animatsiyalar

← Oldingi: 17 — Responsive dizayn · 🏠 README · Keyingi: 19 — Zamonaviy CSS →

Bu bobda: harakat va o'zgarishlar — `transform` bilan elementni siljitish/aylantirish/kattalashtirish, `transition` bilan o'zgarishlarni silliq qilish va `@keyframes` bilan to'liq animatsiyalar yaratishni 0 dan o'rganamiz.

18.1 Nega harakat kerak? (va nega ehtiyot bo'lish kerak)

Hozirgacha sen yaratgan sahifalar **statik** edi: tugmaning rangi bir zumda o'zgarardi, modal oyna birdan paydo bo'lardi. Bu ishlaydi, lekin "qo'pol" tuyuladi. Haqiqiy saytlarda tugma ustiga sichqonchani olib borsang, rang **silliq** o'zgaradi; menyu **sirpanib** ochiladi; yuklanish belgisi **aylanadi**.

Bu uchta texnologiya bilan amalga oshadi:

- `transform` — elementning *ko'rinishini* o'zgartiradi: siljitish, aylantirish, kattalashtirish, qiyshaytirish. (Bu o'zi harakat emas — bu shunchaki "yangi holat".)
- `transition` — ikki holat orasini *silliq* o'tkazadi. (Holatdan holatga ko'prik.)
- `@keyframes` + `animation` — ko'p bosqichli, o'zi-o'zidan aylanadigan to'liq animatsiya.

💡 **Hayotiy analogiya:** `transform` — bu rasmning *yangi kadri* (qo'l yuqorida). `transition` — eski kadrda yangisiga *silliq o'tish*. `@keyframes` — bu *butun multfilm* (ko'p kadrlar ketma-ket).

⚠️ **Eng muhim ogohlantirish (oldindan):** harakat foydalanuvchini *xursand* qilishi ham, *bezdirishi* yoki hatto *kasal qilishi* ham mumkin. Ba'zi odamlar uchun ko'p harakat bosh og'rig'i yoki ko'ngil aynishiga sabab bo'ladi (vestibulyar buzilish). Shuning uchun bu bobning oxirida `prefers-reduced-motion` ni o'rganamiz — bu majburiy, "yaxshi bo'lsa qo'shamiz" emas. Animatsiya — bezak, asosiy funktsiya emas: sayt animatsiyasiz ham to'liq ishlashi shart.

18.2 `transform`: translate (siljitish)

`transform` xossasi elementni **vizual jihatdan** o'zgartiradi, lekin uning sahifadagi **haqiqiy o'rnini** (layout'dagi joyini) o'zgartirmaydi. Bu juda muhim nuqta — keyin tushuntiramiz.

Eng sodda transform — `translate` (ko'chirish):

```
.box {  
  transform: translate(40px, 30px);  
}
```

Bu elementni o'ng tomonga `40px`, pastga `30px` siljitadi. Faqat bitta o'qni xohlasang:

```
.a { transform: translateX(40px); } /* faqat gorizontal */  
.b { transform: translateY(-20px); } /* faqat vertikal (yuqoriga) */
```

✦ **Foiz qiymatlar elementning o'z o'lchamiga nisbatan.** `translateX(100%)` elementni *o'z eni qadar* o'ngga siljitadi. Bu markazlashtirish hiylasida juda foydali (`translate(-50%, -50%)`).

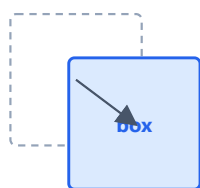
💡 **top/left (position) dan farqi:** 14-bobdagi `position` bilan `top / left` ham elementni siljitadi. Lekin `transform: translate` **ancha tezroq** ishlaydi (nega — 18.6 da) va boshqa elementlarni "turtmaydi". Animatsiya uchun deyarli har doim `translate` ni tanla.

transform turlari: original (kulrang) ustiga ta'siri (ko'k)

har bir panelda punktir kulrang — asl o'ni · to'q ko'k — transformdan keyingi holat

translate(40px, 30px)

elementni siljitadi (joyini o'zgartiradi)



o'ng + past

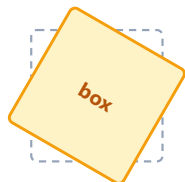
scale(1.5)

o'lchamni kattalashtiradi (markazdan)



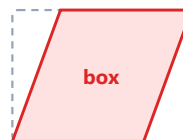
rotate(30deg)

markaz atrofida aylantiradi (soat yo'nalishi)



skewX(20deg)

qiyshaytiradi (parallelogram qiladi)



18.3 transform: scale, rotate, skew

scale — kattalashtirish/kichraytirish

```
.kichik { transform: scale(0.5); } /* ikki barobar kichik */  
.katta { transform: scale(1.5); } /* 1.5 barobar katta */  
.faqatX { transform: scaleX(2); } /* faqat enini cho'zadi */  
.cho'zilgan { transform: scale(1.5, 0.8); } /* en 1.5x, bo'y 0.8x */
```

`scale(1)` — o'zgarishsiz. `scale(0)` — element ko'rinmas (nuqtaga siqiladi). Element **markazidan** kattalashadi (buni `transform-origin` bilan o'zgartiramiz, 18.5).

⚠ **Diqqat:** `scale` element atrofidagi bo'shliqni (layout'ni) o'zgartirmaydi. Kattalashgan element qo'shnilari ustiga **chiqib ketishi** mumkin — bu odatda istalgan natija (masalan hover'da karta "ko'tariladi"), lekin bilib qo'ygan ma'qul.

rotate — aylantirish

```
.ozgina { transform: rotate(15deg); } /* 15 daraja, soat  
yo'nalishida */  
.teskari { transform: rotate(-90deg); } /* manfiy = soatga teskari */  
.tola { transform: rotate(360deg); } /* to'liq aylanish */
```

Burchak `deg` (daraja), `turn` (1turn = 360deg) yoki `rad` (radian) bilan beriladi. Musbat qiymat — soat yo'nalishi bo'yicha.

skew — qiyshaytirish

```
.qiya { transform: skewX(20deg); } /* gorizontal qiyshaytirish */  
.qiyay { transform: skewY(10deg); } /* vertikal */
```

`skew` to'rtburchakni parallelogramga aylantiradi. Amalda kam ishlatiladi (ko'pincha dekorativ "qiya panel" effektlari uchun), lekin transform oilasining bir a'zosi.

✳ **Daraja birligi (deg) ni unutma!** `rotate(45)` ISHLAMAYDI — `rotate(45deg)` bo'lishi kerak.

18.4 Bir nechta transform va tartibning ahamiyati

Bitta `transform` ga bir nechta amalni **bo'sh joy bilan** ketma-ket yozish mumkin:



```
.box {  
  transform: translateX(100px) rotate(45deg) scale(1.2);  
}
```

⚠ **Tartib MUHIM!** Transformlar **o'ngdan chapga emas, chapdan o'ngga** qo'llanadi, lekin har biri elementning *yangi koordinata tizimini* o'zgartiradi. Misol bilan ko'raylik:

```
/* A: avval siljitadi, keyin aylantiradi */  
.a { transform: translateX(100px) rotate(45deg); }  
  
/* B: avval aylantiradi, keyin siljitadi */  
.b { transform: rotate(45deg) translateX(100px); }
```

Natija: `.a` da element 100px o'ngga ketadi, *keyin* o'sha joyda 45° buriladi. `.b` da element *avval* buriladi — endi uning "o'ng tomoni" qiya bo'lib qoladi — keyin shu **qiya yo'nalishda** 100px siljiydi. Ikkalasi butunlay boshqa joyga tushadi!

💡 **Eslab qolish uchun:** har bir transform "elementning o'z dunyosini" o'zgartiradi. `rotate` dan keyingi `translate` aylangan o'qlar bo'ylab harakatlanadi. Shubha bo'lsa, har doim oddiy holatdan boshlab, qadam-baqadam tekshir.

18.5 transform-origin: aylanish/o'lchamning markazi

Standart holatda `rotate` va `scale` element **markazidan** (50% 50%) amalga oshadi. `transform-origin` bu nuqtani o'zgartiradi:

```
.soat-strelka {  
  transform-origin: bottom center; /* pastki o'rtadan aylanadi */  
  transform: rotate(90deg);  
}
```

```
.a { transform-origin: top left; } /* yuqori chap burchakdan */  
.b { transform-origin: 0 0; } /* xuddi shu — chap yuqori */  
.c { transform-origin: 100% 100%; } /* o'ng pastki burchak */
```

Nega kerak? Soat strelkasi *markazidan* emas, *pastki uchidan* aylanishi kerak. Yoki eshikni tasavvur qil — u markazdan emas, *menteshasidan* aylanadi. `transform-origin` aynan shu "mentesha"ni belgilaydi.

💡 Birinchi qiymat — gorizontal (left/center/right yoki px/%), ikkinchisi — vertikal (top/center/bottom).

18.6 Nega `transform` "performant" (tez)?

Bu — professionallarni boshlovchidan ajratadigan bilim. Brauzer sahifani chizishda bir necha bosqichdan o'tadi:

1. **Layout (reflow)** — har bir elementning joyi va o'lchamini hisoblaydi. (Eng qimmat bosqich.)
2. **Paint** — piksellarni bo'yaydi (rang, soya, chegara).
3. **Composite** — bo'yalgan qatlamlarni birlashtiradi.

`width`, `height`, `top`, `margin` kabi xossalarni o'zgartirsang, brauzer **Layout**dan boshlab hammasini qayta hisoblaydi — bu sekin, ayniqsa har kadrda (sekundiga 60 marta) takrorlansa.

Lekin `transform` va `opacity` faqat **Composite** bosqichiga ta'sir qiladi — Layout va Paint qayta hisoblanmaydi. Brauzer bu ikkitasini ko'pincha **GPU**ga (grafik protsessor) topshiradi, natijada animatsiya **silliq** (60fps) ketadi.

Oltin qoida: animatsiya uchun imkon qadar faqat `transform` va `opacity` ni o'zgartir. Bu ikkisi "arzon", qolganlari "qimmat".

⚠ Masalan menyuni `left: -300px` dan `left: 0` ga animatsiya qilish o'rniga, `transform: translateX(-300px)` dan `translateX(0)` ga animatsiya qil — natija bir xil ko'rinadi, lekin ikkinchisi ancha silliq.

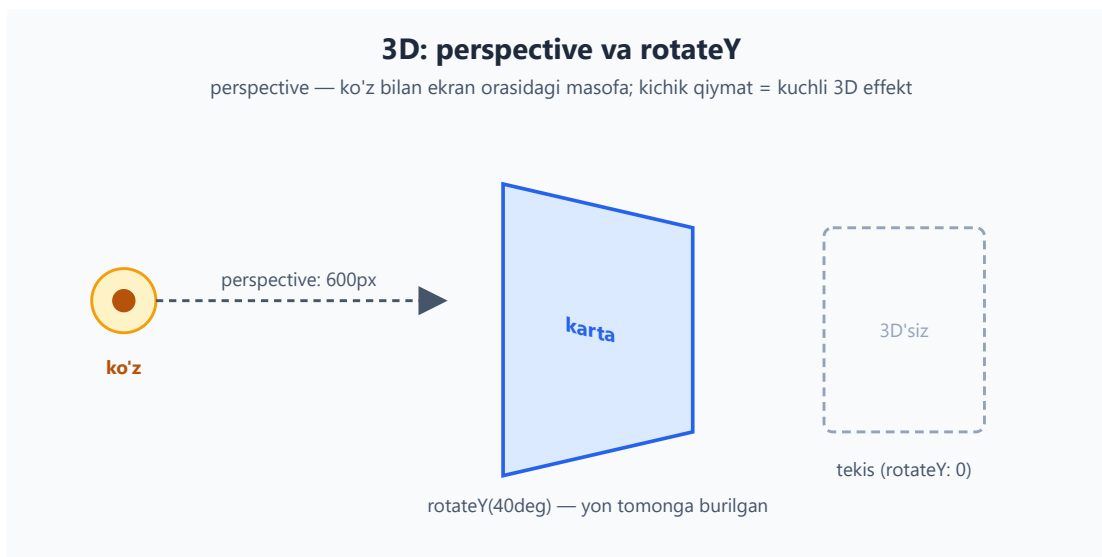
18.7 3D transformlar (qisqacha)

2D yetarli bo'lsa-da, transform 3D fazoda ham ishlaydi. Buning uchun ikki narsa kerak:

```
.sahna {
  perspective: 600px; /* "chuqurlik" hissi — ota elementga beriladi
  */
}
.karta {
  transform: rotateY(40deg); /* Y o'qi atrofida burilish (vertikal
  o'q) */
}
```

- `rotateX(deg)` — gorizontal o'q atrofida buriladi (oldinga/orqaga egiladi).
- `rotateY(deg)` — vertikal o'q atrofida buriladi (chapga/o'ngga, eshik kabi).
- `rotateZ(deg)` — ekran tekisligida buriladi (oddiy 2D `rotate` bilan bir xil).
- `translateZ(px)` — elementni ko'zga yaqin/uzoq qiladi.

perspective eng muhim tushuncha: u kuzatuvchi (ko'z) bilan ekran orasidagi masofani belgilaydi. **Kichik qiymat** (masalan `300px`) = kuchli, "dramatik" 3D effekt; **katta qiymat** (`2000px`) = yumshoq, deyarli tekis. **perspective** aylanadigan elementning **otasiga** beriladi.



💡 Amaliy qo'llanilishi: kartani aylantirib orqa tomonini ko'rsatish ("flip card"), 3D karusel, kub. Boshlovchi sifatida 3D borligini bilib qo'ysang yetarli — keng qo'llanilishi ilg'or mavzu.

18.8 transition: o'zgarishni silliq qilish

transform element holatini *bir zumda* o'zgartiradi. **transition** esa o'sha o'zgarishni **vaqt davomida silliq** qiladi.

Eng oddiy misol — hover'da rang o'zgarishi:

```
.tugma {
  background: #2563eb;
  transition: background 0.3s; /* background 0.3 sekundda o'zgarsin */
}
.tugma:hover {
  background: #1e40af;
}
```

Natija: sichqonchani tugma ustiga olib borsang, rang bir zumda emas, `0.3` sekund davomida **silliq** o'zgaradi. Sichqonchani olib ketsang — yana silliq qaytadi.

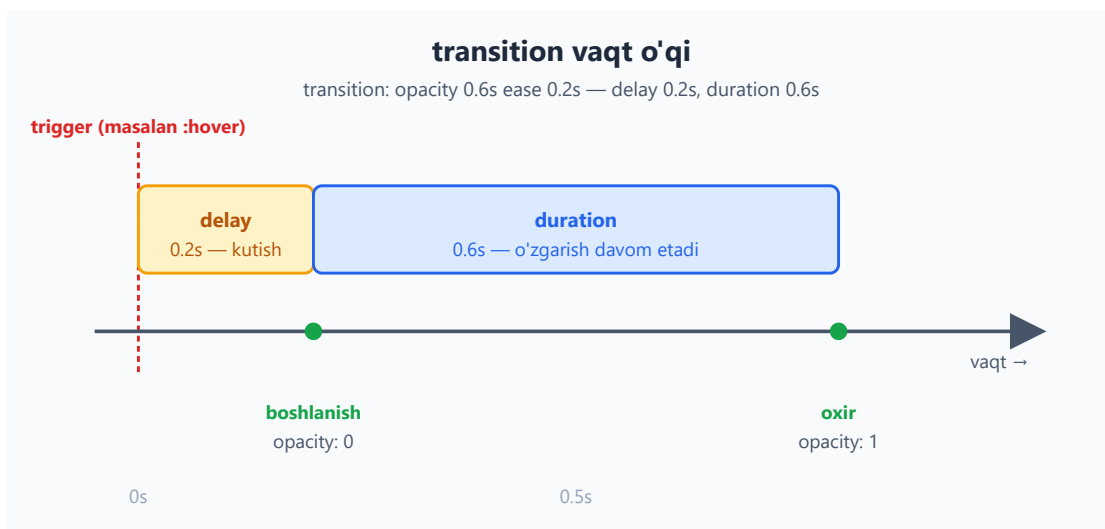
📌 **Eng muhim tushuncha:** **transition** **asosiy (oddiy) holatga** yoziladi, `:hover` ga emas. Chunki u "ikki tomonga" ishlashi kerak — kirishda ham, chiqishda ham. Agar

`:hover` ga yozsang, faqat kirishda silliq, chiqishda esa keskin bo'ladi.

transition'ning to'rt qismi

`transition` to'rtta qiymatdan iborat:

```
.box {  
  transition-property: transform; /* qaysi xossa */  
  transition-duration: 0.4s; /* qancha vaqt */  
  transition-timing-function: ease-in-out; /* qanday tezlik egri */  
  transition-delay: 0.1s; /* qancha kutib boshlasin */  
}
```



Qisqa yozuv (shorthand)

Hammasini bitta qatorda:

```
/* tartib: property duration timing-function delay */  
.box {  
  transition: transform 0.4s ease-in-out 0.1s;  
}
```

⚠ **Vaqt qiymatlarining tartibi:** qisqa yozuvda **birinchi** vaqt qiymati `duration`, **ikkinchisi** `delay`. Ya'ni `transition: opacity 0.5s 0.2s — duration 0.5s, delay 0.2s`.

Bir nechta xossani animatsiya qilish

Vergul bilan ajratib, har biriga alohida sozlama berasan:

```
.karta {  
  transition: transform 0.3s ease, box-shadow 0.3s ease, background  
  0.5s;  
}
```

✦ `transition: all 0.3s` — *barcha* o'zgaradigan xossalarni animatsiya qiladi. Qulay, lekin **ehtiyot bo'l**: u kutilmagan xossalarni ham (masalan `width`, `height`) animatsiya qilib, sekinlikka olib kelishi mumkin. Aniq xossa nomini yozish — yaxshiroq odat.

18.9 Qaysi xossalarni animatsiya qilish mumkin?

Hamma xossa ham `transition` qila olmaydi. Asosiy qoida: **uzluksiz (oraliq qiymatlari bor) xossalar** animatsiya bo'ladi.

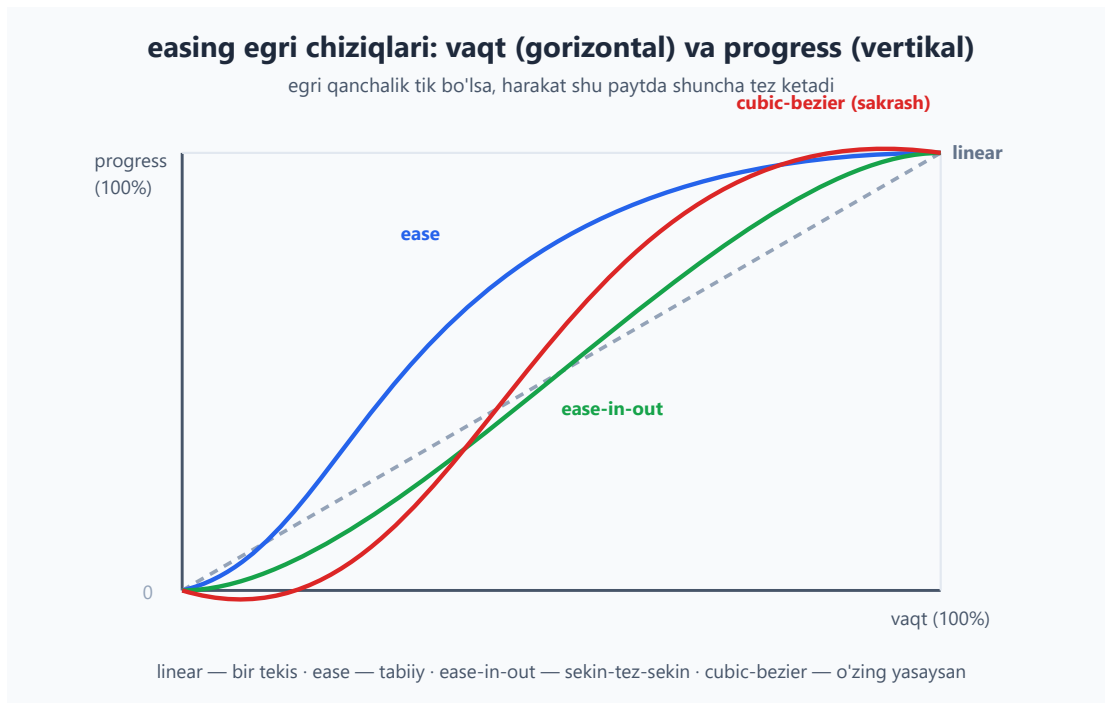
Animatsiya BO'LADI	Animatsiya BO'LMAYDI
<code>opacity</code> (0 → 1 oraliq bor)	<code>display</code> (block/none — oraliq yo'q)
<code>transform</code> (silliqlik o'zgaradi)	<code>position</code> (static/absolute)
<code>color</code> , <code>background-color</code>	<code>font-family</code>
<code>width</code> , <code>height</code> , <code>margin</code> , <code>padding</code>	<code>visibility</code> (faqat maxsus holatda)
<code>box-shadow</code> , <code>border-color</code>	

Nega `display: none` animatsiya bo'lmaydi? Chunki `block` bilan `none` orasida "yarim ko'rinadigan" holat yo'q — u "ha/yo'q" xossa. Shuning uchun elementni silliq yo'qotmoqchi bo'lsang, `display` emas, `opacity` (va kerak bo'lsa `visibility`) ni animatsiya qil.

💡 **Boshlovchi tuzog'i:** `transition: display 0.3s` yozib, "nega ishlamayapti?" deb hayron bo'lish — juda keng tarqalgan. Yodda tut: `display` animatsiya bo'lmaydi.

18.10 Timing funksiyalar (easing)

`transition-timing-function` — o'zgarishning **tezlik egri chizig'i**. Ya'ni harakat boshida tez, oxirida sekinmi? Bir tekismi? Bu animatsiyaning "tabiiyligini" belgilaydi.



Tayyor kalit so'zlar:

```
.a { transition-timing-function: linear; } /* bir tekis tezlik */  
.b { transition-timing-function: ease; } /* default: tez  
boshlanib sekin tugaydi */  
.c { transition-timing-function: ease-in; } /* sekin boshlanadi  
*/  
.d { transition-timing-function: ease-out; } /* sekin tugaydi */  
.e { transition-timing-function: ease-in-out; } /* sekin-tez-sekin  
(eng tabiiy) */
```

💡 **Qaysi birini tanlash?** Real hayotda hech narsa "linear" harakatlanmaydi — mashina ham tezlanib, sekinlashib to'xtaydi. Shuning uchun `ease-out` (kirish menyusi uchun) yoki `ease-in-out` (umumiy) ko'pincha eng tabiiy ko'rinadi. `linear` esa faqat aylanadigan loader kabi *uzluksiz* harakatlar uchun mos.

`cubic-bezier` — o'zingning egri chizig'ing

Tayyor funksiyalar yetmasa, o'zing yasaysan:

```
.maxsus {  
  transition-timing-function: cubic-bezier(0.68, -0.55, 0.27, 1.55);  
}
```

To'rtta son ikkita "tortuvchi nuqta"ni belgilaydi (P1x, P1y, P2x, P2y). Ikkinchi/to'rtinchi son 0 — 1 dan tashqariga chiqsa, harakat "sakraydi" (overshoot) — masalan oxirida

biroz oshib qaytadi. Aslida hamma kalit so'z (`ease` va h.k.) ichida shu `cubic-bezier` yashiringan.

✦ Egri chizig'ingni vizual yasash uchun brauzer DevTools'idagi bezier tahrirlagichi yoki `cubic-bezier.com` saytidan foydalan.

`steps()` — pog'onali harakat

`steps()` harakatni silliq emas, **bo'lak-bo'lak** (sakrash bilan) qiladi:

```
.tayer {  
  transition-timing-function: steps(4);  
}
```

Bu sprite-animatsiya (rasmlar ketma-ketligi), pog'onali sanagich yoki "yozilayotgan matn" effekti uchun ishlatiladi. Silliq emas, "qadam-qadam".

18.11 `@keyframes` va `animation`: to'liq animatsiya

`transition` faqat **ikki holat** orasini bog'laydi va biror **trigger** (hover, class qo'shilishi) talab qiladi. Lekin sahifa ochilishi bilan o'z-o'zidan boshlanadigan, **ko'p bosqichli**, **takrorlanadigan** animatsiya kerak bo'lsa-chi? Buning uchun `@keyframes` bor.

Avval — keyframes'ni e'lon qilish

`@keyframes` animatsiyaning bosqichlarini (foiz bilan) belgilaydi:

```
@keyframes paydoBol {  
  0% {  
    opacity: 0;  
    transform: translateY(20px);  
  }  
  100% {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}
```

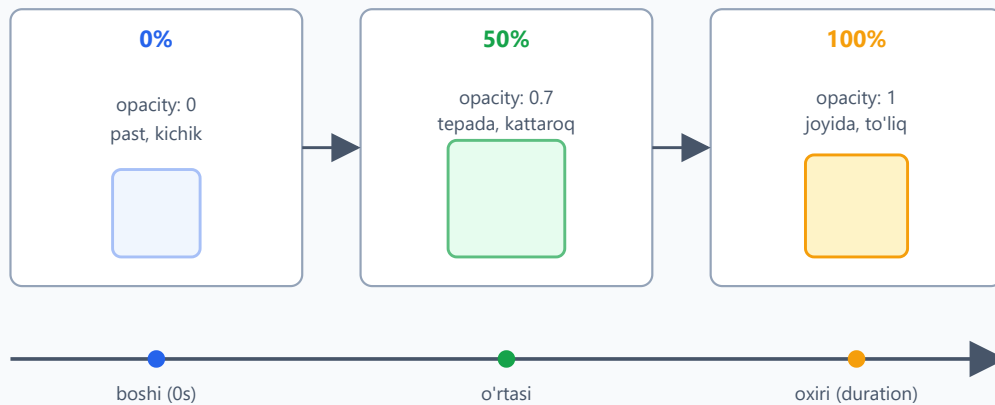
`from` va `to` — `0%` va `100%` ning sinonimlari. Oraliq bosqichlar ham qo'shish mumkin:

```
@keyframes sakra {  
  0% { transform: translateY(0); }  
  50% { transform: translateY(-30px); } /* o'rtada yuqoriga */
```

```
100% { transform: translateY(0); }  
}
```

@keyframes bosqichlari: 0% → 50% → 100%

har foiz — bir tugun (keyframe); brauzer oraliqlarni avtomatik to'ldiradi



animation: paydoBol 2s ease forwards; — 2s ichida 0% dan 100% gacha

Keyin — elementga qo'llash

```
.element {  
  animation: paydoBol 0.8s ease-out;  
}
```

Brauzer 0% va 100% (va oraliq) qiymatlar orasini avtomatik to'ldiradi.

18.12 animation ning to'liq xossalari

animation qisqa yozuvi ichida bir nechta sozlama yashiringan:

```
.element {  
  animation-name: aylan; /* qaysi @keyframes */  
  animation-duration: 2s; /* bir tsikl davomiyligi */  
  animation-timing-function: linear; /* easing (18.10 dagi kabi) */  
  animation-delay: 0.5s; /* boshlashdan oldin kutish */  
  animation-iteration-count: infinite; /* necha marta (yoki cheksiz) */  
  animation-direction: alternate; /* yo'nalish */  
  animation-fill-mode: forwards; /* tugagach qaysi holatda qolsin */  
  /*  
  animation-play-state: running; /* running / paused */  
}
```

Keling, eng ko'p chalkashtiradigan uchtasini chuqurroq ochamiz.

animation-iteration-count

Necha marta takrorlanadi: son (3) yoki `infinite` (cheksiz). Loader uchun deyarli har doim `infinite`.

animation-direction

- `normal` — har safar 0% → 100% (keyin "sakrab" boshiga qaytadi).
- `reverse` — har safar 100% → 0%.
- `alternate` — oldinga, keyin orqaga, keyin yana oldinga... (silliq "nafas olish" effekti).
- `alternate-reverse` — orqadan boshlab almashinadi.

💡 `alternate` — "pulse"/"nafas" effektlari uchun eng foydali: animatsiya oxirida keskin sakramaydi, balki silliq qaytadi.

animation-fill-mode (eng ko'p chalkashtiradigan!)

Animatsiya **boshlanishidan oldin** va **tugagandan keyin** element qaysi holatda bo'ladi?

- `none` (default) — animatsiyadan oldin va keyin element o'zining **asl CSS holatiga** qaytadi. (Tugagach "sakrab" orqaga ketadi — ko'pincha istalmagan.)
- `forwards` — tugagach **oxirgi kadr** (100%) holatida qoladi. (Eng ko'p kerak bo'ladigani.)
- `backwards` — delay vaqtida ham **birinchi kadr** (0%) holatini ko'rsatadi.
- `both` — `forwards` + `backwards`.

⚠️ **Juda keng tarqalgan muammo:** "fade-in animatsiyam tugadi, lekin element yana g'oyib bo'ldi!" — sababi `fill-mode` berilmagan (default `none`). Yechim: `animation-fill-mode: forwards` qo'sh.

animation-play-state

`running` yoki `paused`. Buni JavaScript yoki `:hover` bilan o'zgartirib, animatsiyani to'xtatib/davom ettirish mumkin:

```
.karusel:hover {  
  animation-play-state: paused; /* sichqoncha ustida - to'xtaydi */  
}
```

Qisqa yozuv

```
/* name duration timing-function delay iteration-count direction fill-mode */
.element {
  animation: aylan 2s linear 0s infinite alternate forwards;
}
```

✦ Tartib moslashuvchan, lekin **birinchi vaqt** = duration, **ikkinchi vaqt** = delay (xuddi `transition` dagi kabi).

18.13 `will-change` : brauzerga oldindan ogohlantirish

`will-change` brauzerga "bu element tez orada o'zgaradi, tayyorlanib tur" deb aytadi. Brauzer elementni alohida qatlamga ajratib, animatsiyani silliqroq qiladi.

```
.tez-animatsiya {
  will-change: transform;
}
```

⚠ **Juda ehtiyotkorlik bilan ishlat!** `will-change` xotira (RAM/GPU) sarflaydi. Uni *hamma narsaga* qo'ysang, brauzerni sekinlashtirasiz — foydasi o'rniga zarari tegadi. Qoidalar:

- Faqat **haqiqatan tez-tez animatsiya bo'ladigan** elementlarga qo'y.
- Imkon bo'lsa, animatsiya boshlanishidan *sal oldin* (masalan JavaScript bilan) qo'shib, tugagach olib tashla.
- "Ko'r-ko'rona optimizatsiya" qilma — avval muammo borligini (lag) sezgin, keyin qo'sh.

💡 Aslida ko'p hollarda `transform / opacity` animatsiyasi `will-change` siz ham silliq ishlaydi. Buni faqat aniq sezilarli sekinlik bo'lganda "oxirgi chora" sifatida ishlat.

18.14 Amaliy: hover effektlari

Endi bilimni amalga aylantiramiz. Klassik "ko'tariladigan karta" effekti:

```
.karta {
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
  transition: transform 0.25s ease, box-shadow 0.25s ease;
}
```

```
.karta:hover {
  transform: translateY(-6px) scale(1.02);
  box-shadow: 0 12px 24px rgba(0,0,0,0.18);
}
```

Natija: sichqoncha ustiga kelganda karta biroz **ko'tariladi** va soyasi **chuqurlashadi** — go'yo qog'oz stoldan ko'tarilgandek. Faqat `transform` va `box-shadow` o'zgargani uchun silliq ishlaydi.

💡 `transition` ni `.karta` ga (asosiy holatga) yozganimizga e'tibor ber — shuning uchun sichqoncha ketganda ham silliq qaytadi.

18.15 Amaliy: aylanadigan loader

`@keyframes` ning eng mashhur qo'llanilishi — yuklanish belgisi (spinner):

```
.loader {
  width: 40px;
  height: 40px;
  border: 4px solid #e2e8f0;      /* och kulrang halqa */
  border-top-color: #2563eb;     /* faqat yuqori qism ko'k */
  border-radius: 50%;           /* doira */
  animation: aylanish 0.8s linear infinite;
}

@keyframes aylanish {
  to { transform: rotate(360deg); }
}
```

```
<div class="loader" role="status" aria-label="Yuklanmoqda"></div>
```

Natija: ko'k qismi bo'lgan halqa cheksiz aylanadi — klassik spinner. `linear` ishlatdik, chunki aylanish **bir tekis** bo'lishi kerak (`ease` bo'lsa "sakraydigandek" tuyulardi). `to` — 100% ning qisqasi.

✦ `role="status"` va `aria-label` skrinriderlar uchun: animatsiya ko'rinmaydi, lekin "yuklanmoqda" deb e'lon qilinadi.

18.16 Amaliy: fade-in (paydo bo'lish)

Sahifa ochilganda element silliq paydo bo'lishi:




```
.kirish {
  animation: fadeIn 0.6s ease-out forwards;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(16px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

Natija: element pastdan biroz suzib chiqib, shaffofdan to'liq ko'rinadigan holga keladi. `forwards` muhim — usiz element tugagach yana g'oyib bo'lardi (18.12 dagi tuzoq).

💡 Bir nechta elementni ketma-ket (birin-ketin) paydo qilmoqchi bo'lsang, har biriga turli `animation-delay` ber:

```
.kirish:nth-child(1) { animation-delay: 0.0s; }
.kirish:nth-child(2) { animation-delay: 0.1s; }
.kirish:nth-child(3) { animation-delay: 0.2s; }
```

Bu "shashka" (stagger) effekti — ro'yxat elementlari to'lqin kabi paydo bo'ladi.

18.17 `prefers-reduced-motion`: harakatni hurmat qilish

Bu — bobning **eng muhim** bo'limi, "ixtiyoriy bezak" emas. Ba'zi foydalanuvchilar ko'p harakatdan **jismonan azob chekadi**: bosh aylanishi, ko'ngil aynishi, bosh og'rig'i (vestibulyar buzilish, migren). Ular operatsion tizim sozlamalarida "harakatni kamaytir" rejimini yoqishadi. Sen buni `prefers-reduced-motion` media so'rovi bilan **hurmat qilishing shart**.

```
/* Asosiy animatsiya — hamma uchun */
.karta {
  transition: transform 0.3s ease;
}
.karta:hover {
  transform: translateY(-6px);
}

/* Harakatni kamaytirishni xohlaganlar uchun — o'chiramiz */
@media (prefers-reduced-motion: reduce) {
  .karta {
```

```

    transition: none;
  }
  .karta:hover {
    transform: none;
  }
}

```

Ko'p saytlar butun loyihaga "xavfsizlik to'ri" qo'yadi:

```

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}

```

💡 **Falsafa:** asosiy holatni "harakat bor" deb yozib, keyin reduce rejimida o'chir. Yoki teskari: animatsiyani faqat `prefers-reduced-motion: no-preference` ichida yoq. Ikkala yondashuv ham to'g'ri — muhimi, foydalanuvchining tanlovini **hurmat qilish**.

⚠️ Bu kichik kod bo'lakcha, lekin u sening saytingni ba'zi foydalanuvchilar uchun "boshim og'riydi" dan "qulay" ga aylantiradi. Har bir jiddiy loyihada bo'lishi shart.

18.18 Umumiy xulosa va keng tarqalgan xatolar

Bu uchtasini ajratib eslab qol:

Texnologiya	Nima qiladi	Qachon
<code>transform</code>	Element ko'rinishini o'zgartiradi (joyi/o'lchami/burchagi)	Bir martalik holat o'zgarishi
<code>transition</code>	Ikki holat orasini silliq qiladi	Trigger bor (hover, class)
<code>@keyframes</code> + <code>animation</code>	Ko'p bosqichli, o'zi ketadigan animatsiya	Loader, takroriy, avtomatik

⚠️ **Eng ko'p uchraydigan xatolar:**

1. `rotate(45)` — `deg` unutilgan → `rotate(45deg)`.
2. `transition` ni `:hover` ga yozish → asosiy holatga yoz.
3. `transition: display 0.3s` → `display` animatsiya bo'lmaydi, `opacity` ishlat.
4. Animatsiya tugagach element "sakraydi" → `animation-fill-mode: forwards` qo'sh.
5. `width / height / top` ni animatsiya qilib sekinlik → `transform` ishlat.
6. `prefers-reduced-motion` ni unutish → har doim qo'sh.

Mashqlar

Quyidagilarni brauzerda sinab ko'r. Avval o'zing yoz, keyin yechimni och.

Mashq 1 — Transform ta'sirini aniqlash

`transform: scale(1.5)` berilgan `100px × 100px` kvadrat ekranda qancha o'lchamda ko'rinadi? Uning **layout'dagi** (sahifa oqimidagi) joyi o'zgaradimi?



Yechim



Ekranda `150px × 150px` bo'lib ko'rinadi (markazdan kattalashadi). Lekin **layout'dagi joyi o'zgarmaydi** — sahifa hali ham uni `100px × 100px` deb hisoblaydi, shuning uchun u qo'shnilar ustiga chiqib ketadi. Bu `transform` ning asosiy xususiyati: vizual o'zgartiradi, layout'ni emas.

Mashq 2 — Transition shorthand'ni o'qish

Bu kod nimani anglatadi? Har bir qiymatni nomla:

```
transition: opacity 0.5s ease-in 0.2s;
```





Yechim



- `opacity` — `transition-property` (qaysi xossa animatsiya bo'ladi)
- `0.5s` — `transition-duration` (birinchi vaqt = davomiylilik)
- `ease-in` — `transition-timing-function` (sekin boshlanadi)
- `0.2s` — `transition-delay` (ikkinchi vaqt = kutish)

Ya'ni: `opacity 0.2s` kutib, keyin `0.5s` davomida sekin boshlanib o'zgaradi.

Mashq 3 — Tugma hover effekti

Tugma ustiga sichqoncha kelganda biroz kattalashib (`scale 1.05`) va fon rangi to'qlashishi kerak — ikkalasi ham `0.2s` da silliq. Yoz.



Yechim



```
.tugma {
  background: #2563eb;
  color: #fff;
  padding: 10px 20px;
  border: none;
  border-radius: 8px;
  transition: transform 0.2s ease, background 0.2s ease;
}
.tugma:hover {
  transform: scale(1.05);
  background: #1e40af;
}
```

`transition` asosiy `.tugma` da — shuning uchun sichqoncha ketganda ham silliq qaytadi.

Mashq 4 — fill-mode tuzog'i

Bu fade-in animatsiyasi ishlaydi, lekin tugagach element yana g'oyib bo'ladi. Nega? Tuzat.

```
.element {
  opacity: 0;
  animation: paydo 1s ease;
}
@keyframes paydo {
  to { opacity: 1; }
}
```



Yechim



Sababi: `animation-fill-mode default none`, shuning uchun animatsiya tugagach element o'zining asl `opacity: 0` holatiga **qaytadi**. Yechim — `forwards` qo'shish:

```
.element {  
  opacity: 0;  
  animation: paydo 1s ease forwards; /* oxirgi kadrda qoladi */  
}  
@keyframes paydo {  
  to { opacity: 1; }  
}
```



Mashq 5 — Transform tartibi

Bu ikki qoida bir xil natija beradimi? Nega?

```
.a { transform: rotate(45deg) translateX(50px); }  
.b { transform: translateX(50px) rotate(45deg); }
```



Yechim



Yo'q, har xil natija. Transformlar elementning *o'z koordinata tizimini* navbatma-navbat o'zgartiradi:

- `.a` — avval 45° buriladi, keyin **aylangan** o'q bo'ylab 50px siljiydi (qiya yo'nalishda).
- `.b` — avval gorizontal 50px o'ngga siljiydi, keyin o'sha joyda 45° buriladi.

Tartib har doim muhim — har bir transform keyingisining "dunyasini" o'zgartiradi.

Mashq 6 — "Nafas oladigan" (pulse) animatsiya

Element silliq kattalashib-kichrayib (`scale 1` ↔ `scale 1.1`) cheksiz takrorlanishi kerak. Oxirida "sakramasin". Yoz.

**Yechim**

```
.pulse {  
  animation: nafas 1.5s ease-in-out infinite alternate;  
}  
@keyframes nafas {  
  from { transform: scale(1); }  
  to   { transform: scale(1.1); }  
}
```

Sir — `alternate`: animatsiya 1→1.1 ga borib, keyin teskari 1.1→1 ga qaytadi (sakramaydi).
`infinite` — to'xtamaydi. `ease-in-out` — tabiiy "nafas" hissi.

Mashq 7 — Performant menyu

Yon menyu chapdan sirpanib chiqishi kerak. Boshlovchi `left: -300px` → `left: 0` ishlatibdi, lekin sekin. Tezroq (performant) qil.

**Yechim**

`left` o'rniga `transform: translateX` ishlatish kerak — u Layout'ni qayta hisoblamaydi, GPU'da silliq ishlaydi:

```
.menu {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 300px;  
  transform: translateX(-100%); /* boshida ekrandan tashqarida */  
  transition: transform 0.3s ease-out;  
}  
.menu.ochiq {  
  transform: translateX(0); /* ochilganda joyiga keladi */  
}
```

`translateX(-100%)` — menyuni o'z eni qadar chapga (ekrandan tashqariga) suradi.
JavaScript bilan `.ochiq` class qo'shilganda silliq chiqadi.

Mashq 8 — Reduced motion himoyasi

Sening saytingda ko'p animatsiya bor. Vestibulyar buzilishi bor foydalanuvchilarni himoya qiladigan global "xavfsizlik to'ri" qo'sh.



Yechim



```
@media (prefers-reduced-motion: reduce) {  
  *,  
  *::before,  
  *::after {  
    animation-duration: 0.01ms !important;  
    animation-iteration-count: 1 !important;  
    transition-duration: 0.01ms !important;  
    scroll-behavior: auto !important;  
  }  
}
```

Bu foydalanuvchi OS'da "harakatni kamaytir" ni yoqsa, barcha animatsiya/transition'larni deyarli bir zumda (0.01ms) qiladi — ya'ni amalda o'chiradi, lekin funksiya buzilmaydi. Bu — har bir jiddiy loyihaning majburiy qismi.

19 — Zamonaviy CSS

◀ Oldingi: 18 — Transitions, transforms va animatsiyalar · 🏠 README · Keyingi: 20 — CSS arxitekturasi va uslublar ▶

Bu bobda: custom properties (CSS o'zgaruvchilar), `calc()` / `clamp()`, CSS nesting, `:has()` "ota" selektor, `@layer` kaskad qatlamlari va logical properties kabi zamonaviy CSS imkoniyatlarini 0 dan o'rganib, kodimizni qisqa, moslashuvchan va boshqarsa oson qilamiz.

19.1 Nega "zamonaviy CSS" alohida bob?

Avvalgi boblarda CSS'ning poydevorini qo'ydik: selektorlar, box model, flexbox, grid, ranglar. Bularning ko'pi 10-15 yildan beri mavjud va ular hamon zarur. Lekin so'nggi yillarda CSS'ga shu darajada kuchli imkoniyatlar qo'shildiki, ular ilgari faqat JavaScript yoki maxsus vositalar (Sass kabi preprocessorlar) bilan hal qilinadigan masalalarni endi **toza CSS bilan** yechib beradi.

Bu bob aynan o'sha yangiliklar haqida. Nega ular muhim?

- **Custom properties** — bitta qiymatni (masalan, asosiy rangni) bir joyda saqlab, butun saytda ishlatish va bir lahzada o'zgartirish imkonini beradi. Bu "theming" (mavzu/tema almashtirish — kunduzgi/tungi rejim) ning poydevori.
- `calc()`, `clamp()` — o'lchamlarni qattiq raqam bilan emas, **hisoblanadigan** va **moslashuvchan** qilib yozadi. Responsiv dizaynni ancha soddalashtiradi.
- **Nesting** — CSS'ni HTML tuzilishiga o'xshatib ichma-ich yozish, takrorini kamaytirish.
- `:has()` — CSS tarixida birinchi marta **"ota" elementni farzandiga qarab** tanlash imkonini berdi.
- `@layer` — katta loyihalarda kaskad ustidan **aniq nazorat** o'rnatadi: `!important` urushlarini tugatadi.
- **Logical properties** — saytni boshqa til yo'nalishlariga (masalan, o'ngdan chapga yoziladigan arab tiliga) avtomatik moslashtiradi.

🚀 Bu boblarning aksariyati 2022-2024 yillarda barcha asosiy brauzerlarda (Chrome, Firefox, Safari, Edge) qo'llab-quvvatlana boshladi. Ya'ni bugun ularni bemaolol ishlatishingiz mumkin.

💡 **Maslahat:** har bir yangi xususiyatni o'rganganda "bu menga qaysi muammoni yechadi?" deb so'rang. Zamonaviy CSS — bu nafaqat yangi sintaksis, balki **kamroq kod yozib, ko'proq ish bajarish** falsafasi.

19.2 Custom properties — CSS o'zgaruvchilar

Muammo: takrorlanuvchi qiymatlar

Tasavvur qiling, saytingizning brendi ko'k rangda (#2563eb). Bu rang tugmalarda, havolalarda, sarlavhalarda, chegaralarda — o'nlab joyda ishlatiladi:

```
.tugma { background: #2563eb; }
a { color: #2563eb; }
h1 { border-bottom: 2px solid #2563eb; }
.belgi { color: #2563eb; }
```

Endi brend rangini o'zgartirmoqchi bo'lsangiz, har bir joyni qo'lda topib almashtirishingiz kerak. Bitta joyni unutsangiz — sayt buziladi. Bu xavfli va zerikarli.

Yechim: bir joyda e'lon qiling

Custom property (CSS o'zgaruvchi, "maxsus xossa") — bu o'zingiz nomlagan qiymatni bir joyda saqlab, kerakli joyda chaqirib ishlatish usuli. Ular ikki harf bilan boshlanadi: **-**.

```
:root {
  --asosiy-rang: #2563eb;
}

.tugma { background: var(--asosiy-rang); }
a { color: var(--asosiy-rang); }
h1 { border-bottom: 2px solid var(--asosiy-rang); }
.belgi { color: var(--asosiy-rang); }
```

Endi rangni o'zgartirish uchun faqat **bir qator** — `--asosiy-rang` ning qiymatini almashtirsangiz kifoya. Hamma joy avtomatik yangilanadi.

✦ **Ikki qism: - E'lon (declaration):** `--ism: qiymat;` — o'zgaruvchini yaratadi. `ism` `--` bilan boshlanadi va katta-kichik harfga sezgir (`--Rang` ≠ `--rang`). - **Foydalanish:** `var(--ism)` — o'zgaruvchining qiymatini joylaydi.

Nega `:root` ?

`:root` — bu butun hujjatning eng tepa elementi, ya'ni `<html>` ga teng (lekin specificity'si bir oz balandroq). O'zgaruvchini `:root` da e'lon qilsak, u **butun sahifaga** tarqaladi.

💡 Custom property nomi `--` bilan boshlangani uchun ular oddiy CSS xossalari bilan adashmaydi. `--margin` deb yozsangiz, brauzer buni o'z `margin` xossasi deb o'ylamaydi — bu sizning shaxsiy o'zgaruvchingiz.

⚠️ **Eng keng tarqalgan xato:** `var()` ni unutib qo'yish. `color: --asosiy-rang;` ishlamaydi! To'g'risi: `color: var(--asosiy-rang);`. O'zgaruvchini *o'qish* uchun har doim `var()` kerak.

19.3 Scope va inheritance — o'zgaruvchilar qayerda yashaydi

Custom property'lar oddiy CSS xossalari kabi **meros (inheritance)** orqali pastga tarqaladi. Ya'ni o'zgaruvchini bir elementda e'lon qilsangiz, u o'sha element **va uning barcha farzandlarida** mavjud bo'ladi.

```
:root {
  --rang: blue;
}

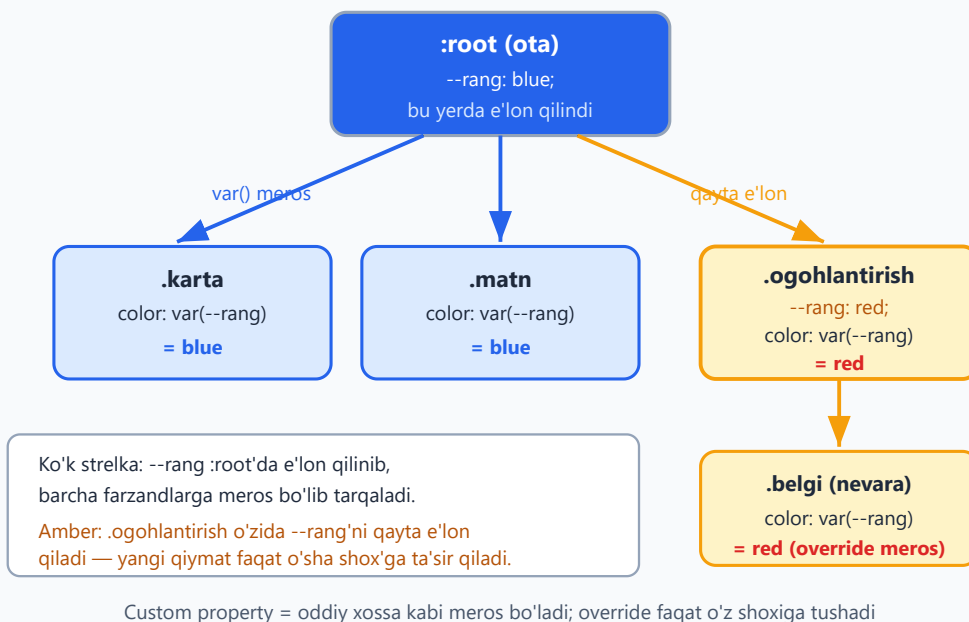
.ogohlantirish {
  --rang: red; /* faqat shu shox uchun qayta e'lon */
}

.karta { color: var(--rang); } /* blue */
.ogohlantirish.belgi { color: var(--rang); } /* red */
```

```
<div class="karta">Bu matn ko'k</div>
<div class="ogohlantirish">
  <span class="belgi">Bu matn qizil</span>
</div>
```

`--rang` `:root` da `blue` deb e'lon qilingan, shuning uchun butun sahifada `blue` mavjud. Lekin `.ogohlantirish` ichida uni `red` ga **override** qildik — bu yangi qiymat faqat `.ogohlantirish` va uning ichidagi elementlarga ta'sir qiladi, boshqa joyga emas.

Custom property scope va meros



Nega bu kuchli? Bitta o'zgaruvchini turli joyda turli qiymatga sozlash mumkin. Masalan, "kartochka" komponenti `--bo'shliq` o'zgaruvchisini ishlatadi, lekin har xil kartochkalar uni o'zicha sozlaydi — komponentni qayta yozmasdan.

```
.karta {
  --bo'shliq: 16px;
  padding: var(--bo'shliq);
}
.karta.zich { --bo'shliq: 8px; } /* zichroq kartochka */
.karta.keng { --bo'shliq: 32px; } /* kengroq kartochka */
```

✦ **Farq:** oddiy preprocessor o'zgaruvchilari (Sass `$rang`) kompilyatsiya paytida "qotib qoladi". CSS custom property'lar esa **brauzerda jonli yashaydi** — meros bo'ladi, real vaqtda o'zgaradi, JavaScript bilan boshqariladi.

19.4 Fallback — zaxira qiymat

`var()` ikkinchi argument qabul qiladi — agar o'zgaruvchi mavjud bo'lmasa ishlatiladigan **zaxira (fallback) qiymat**:

```
.element {
  color: var(--rang, black); /* --rang yo'q bo'lsa, black */
  padding: var(--bo'shliq, 16px); /* --bo'shliq yo'q bo'lsa, 16px */
}
```

Agar `--rang` hech qayerda e'lon qilinmagan bo'lsa, `color` `black` bo'ladi. Bu komponentni mustahkam qiladi: o'zgaruvchi sozlanmasa ham, oqilona standart qiymat ishlaydi.

Fallback ichida yana `var()` ham bo'lishi mumkin (zanjir):

```
.element {  
  /* --asosiy yo'q bo'lsa --zaxira, u ham yo'q bo'lsa gray */  
  color: var(--asosiy, var(--zaxira, gray));  
}
```

💡 Komponent kutubxonasi yozayotganda fallback'lar juda foydali: foydalanuvchi o'zgaruvchini bermasa ham komponent buzilmaydi.

19.5 JavaScript bilan o'qish va yozish

Custom property'lar brauzerda jonli yashagani uchun ularni **JavaScript bilan o'qish va o'zgartirish** mumkin. Bu — preprocessor o'zgaruvchilarida mutlaqo imkonsiz.

```
const root = document.documentElement; // <html>  
  
// O'qish:  
const rang = getComputedStyle(root).getPropertyValue('--asosiy-rang');  
console.log(rang.trim()); // "#2563eb"  
  
// Yozish (o'zgartirish):  
root.style.setProperty('--asosiy-rang', '#dc2626');  
// Endi --asosiy-rang ishlatadigan HAMMA element qizilga aylanadi
```

Bir qator JavaScript butun saytning rangini o'zgartirdi! Bu — tema almashtirgich (kunduzgi/tungi rejim), foydalanuvchi sozlamalari, real vaqtli dizayn boshqaruvi uchun asos.

Amaliy misol — interaktiv slayder:

```
<input type="range" min="8" max="40" value="16" id="bo'shliq">  
<div class="quti">Mening ichki bo'shlig'im o'zgaradi</div>
```

```
.quti { padding: var(--p, 16px); background: #e2e8f0; }
```

```
const slayder = document.getElementById("bo'shliq");  
slayder.addEventListener('input', (e) => {  
  document.documentElement.style.setProperty('--p', e.target.value +
```

```
'px');  
});
```

Foydalanuvchi slayderna surganda `.quti` ning ichki bo'shlig'i jonli o'zgaradi — hech qanday murakkab kod yozmasdan.

19.6 Theming — tema almashtirish

Custom property'larning eng kuchli qo'llanishi — **theming** (kunduzgi/tungi rejim kabi tema almashtirish). Mantiq oddiy: barcha ranglarni o'zgaruvchilarga chiqaramiz, keyin bitta klass bilan ularning qiymatlarini almashtiramiz.

```
:root {  
  --fon: #ffffff;  
  --matn: #1e293b;  
  --karta-fon: #f8faff;  
}  
  
:root.tungi {  
  --fon: #0f172a;  
  --matn: #e2e8f0;  
  --karta-fon: #1e293b;  
}  
  
body { background: var(--fon); color: var(--matn); }  
.karta { background: var(--karta-fon); }
```

```
// Tugma bosilganda temani almashtirish  
document.querySelector('.tema-tugma').addEventListener('click', () => {  
  document.documentElement.classList.toggle('tungi');  
});
```

`<html>` ga `tungi` klassi qo'shilsa, barcha o'zgaruvchilar yangi qiymat oladi va butun sahifa bir lahzada tungi rejimga o'tadi. Har bir elementni alohida o'zgartirish shart emas — bu custom property'larning kuchi.

💡 **Nega bu yondashuv professional?** Yangi rang sxemasi qo'shmoqchi bo'lsangiz (masalan, "sepiya" rejim), faqat yana bir `:root.sepiya { ... }` bloki yozasiz. Komponentlarning hech biriga tegmaysiz.

19.7 `calc()` — CSS ichida hisoblash

`calc()` brauzerga matematik amalni **bajartiradi** — qo'shish (+), ayirish (-), ko'paytirish (*), bo'lish (/). Eng muhimi: u **turli birliklarni aralashtira oladi**, bu boshqa hech qayerda mumkin emas.

```
.element {  
  width: calc(100% - 80px); /* foiz va piksel aralash! */  
  height: calc(100vh - 60px); /* butun ekran balandligidan header'ni  
ayirish */  
  padding: calc(1rem + 2px);  
}
```

`width: calc(100% - 80px)` ni qo'lda hisoblab bo'lmaydi, chunki `100%` ekran o'lchamiga bog'liq, lekin `80px` qat'iy. Faqat brauzer real vaqtda buni hisoblay oladi.

⚠ **Eng keng tarqalgan xato:** operator atrofida bo'sh joy qo'yishni unutish.

`calc(100%-80px)` **ishlamaydi!** + va - atrofida albatta probel kerak: `calc(100% - 80px)`. (* va / uchun shart emas, lekin baribir qo'ying.)

`calc()` custom property'lar bilan ajoyib birikadi:

```
:root { --asos: 8px; }  
.kichik { padding: calc(var(--asos) * 1); } /* 8px */  
.o'rta { padding: calc(var(--asos) * 2); } /* 16px */  
.katta { padding: calc(var(--asos) * 3); } /* 24px */
```

Bu — "8px tarmoq tizimi" (spacing scale): hamma bo'shliqlar bitta asosning ko'paytmasi. Dizaynni izchil qiladi.

💡 `calc()` ichida `calc()` qo'yish ham mumkin, lekin odatda kerak emas — bitta ifoda ichida bir nechta amal yozasiz: `calc((100% - 40px) / 3)`.

19.8 `clamp()`, `min()`, `max()` — moslashuvchan o'lchamlar

Bu uchta funksiya o'lchamni **diapazon ichida** boshqaradi. Ular responsiv dizaynni media query'larsiz ham yechib beradi.

`min()` — eng kichigini tanlaydi

```
.quti { width: min(600px, 100%); }
```

`min()` berilgan qiymatlardan **eng kichigini** oladi. Bu yerda: agar ekran 600px'dan keng bo'lsa, `100%` 600px'dan katta — demak 600px tanlanadi (quti 600px'da to'xtaydi). Tor

ekranda 100% kichikroq — quti ekranga to'la sig'adi. Ya'ni "maksimal 600px, lekin ekrandan oshmasin".

`max()` — eng kattasini tanlaydi

```
.matn { font-size: max(16px, 1vw); }
```

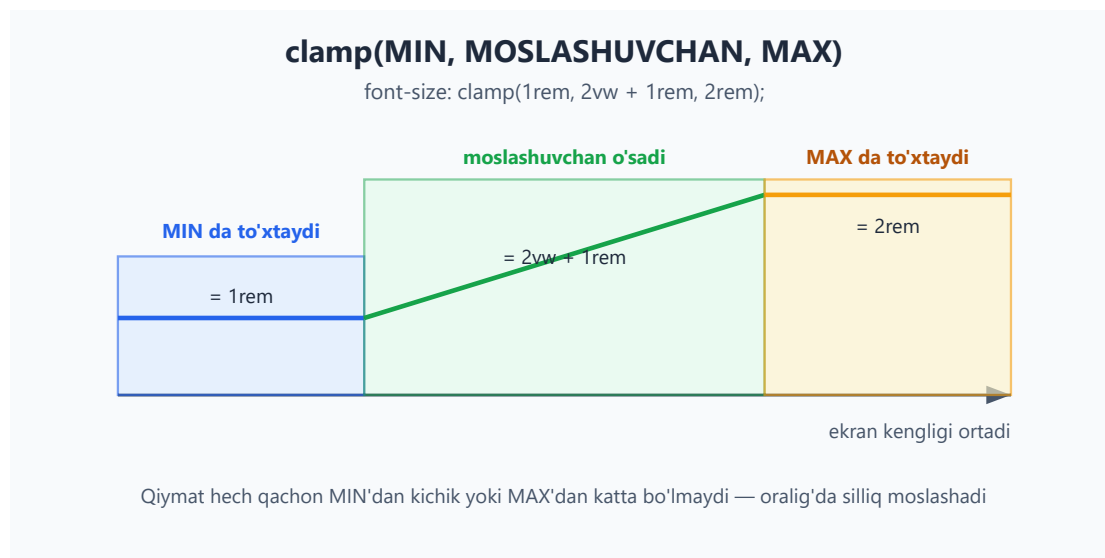
`max()` eng kattasini oladi. Bu yerda shrift hech qachon 16px 'dan kichik bo'lmaydi — kichik ekranda o'qilishi kafolatlanadi.

`clamp()` — min va max orasida ushlab turish

`clamp(MIN, MOSLASHUVCHAN, MAX)` — eng foydalisi. Qiymatni pastdan MIN, yuqoridan MAX bilan cheklab, oraliq'da MOSLASHUVCHAN qiymatni ishlatadi:

```
h1 {  
  font-size: clamp(1.5rem, 4vw, 3rem);  
}
```

Bu qanday ishlaydi: - Kichik ekranda 4vw 1.5rem 'dan kichik bo'ladi → 1.5rem ishlatiladi (juda kichrayib ketmaydi). - Katta ekranda 4vw 3rem 'dan katta bo'ladi → 3rem ishlatiladi (cheksiz kattalashmaydi). - Oraliq'da 4vw silliq o'sadi → shrift ekranga moslashib o'sib boradi.



💡 **Nega bu inqilob?** Ilgari "responsiv shrift" uchun bir nechta `@media` query yozish kerak edi. Endi bitta `clamp()` qatori barchasini hal qiladi va o'tish silliq (savrashsiz) bo'ladi.

`clamp()` ko'pincha `calc()` bilan birga ishlatiladi:

```
h1 { font-size: clamp(1.5rem, 1rem + 2vw, 3rem); }
```

`1rem + 2vw` — bu "asos + ekranga bog'liq qo'shimcha" formulasi, professional fluid tipografiyaning standartidir.

19.9 CSS nesting — ichma-ich yozish

Ilgari ichma-ich (nested) CSS faqat Sass kabi preprocessorlarda bor edi. Endi u **toza CSS** ga kirdi. Nesting — selektorlarni HTML tuzilishiga o'xshatib biri ichiga ikkinchisini yozish.

Nesting'siz (eski uslub):

```
.karta { padding: 16px; }
.karta h2 { color: #2563eb; }
.karta p { color: #475569; }
.karta:hover { box-shadow: 0 4px 12px rgba(0,0,0,0.1); }
```

`.karta` to'rt marta takrorlandi. **Nesting bilan:**

```
.karta {
  padding: 16px;

  h2 { color: #2563eb; }
  p { color: #475569; }

  &:hover {
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
}
```

Endi `.karta` faqat bir marta yozildi, qolgani uning ichiga "uyalandi". Kod HTML tuzilishini aks ettiradi va o'qilishi osonroq.

& belgisi — "joriy selektor"

`&` — bu "shu yerda joriy (ota) selektor turibdi" degani. U asosan ikki holatda kerak:

```
.tugma {
  background: #2563eb;

  &:hover { background: #1d4ed8; } /* .tugma:hover */
  &.faol { background: #16a34a; } /* .tugma.faol (bo'shliqsiz!) */
}
```


`&:hover` → `.tugma:hover`. Bu yerda `&` shart, chunki `:hover` selektorga **bo'shliqsiz** yopishishi kerak. Agar shunchaki `:hover` yozsangiz, ba'zi holatlarda bu `.tugma *:hover` (farzand) deb talqin qilinishi mumkin.

✦ **Qoida:** oddiy farzand element uchun (`h2`, `p`) `&` shart emas. Lekin joriy element'ning o'ziga teginadigan holat (`:hover`, `.faol`, `[disabled]`) uchun `&` kerak.

⚠ Nesting'ni juda chuqurlashtirib yubormang. 3 darajadan oshsa, kod o'qilishi qiyinlashadi va specificity beixtiyor oshib ketadi. Tekis tuzilma — ko'pincha yaxshiroq.

19.10 `:has()` — "ota" selektor

CSS tarixida eng katta yetishmovchiliklardan biri shu edi: **ota elementni farzandiga qarab tanlab bo'lmasdi**. Selektorlar har doim pastga (farzandga) qarab ishlardi, hech qachon yuqoriga emas. `:has()` aynan shu muammoni yechdi.

`:has()` — ota element **ichida** ma'lum bir narsa bor-yo'qligini tekshiradi va shunga qarab ota'ni tanlaydi:

```
/* Ichida <img> bor kartochkalarni tanla */
.karta:has(img) {
  border: 2px solid #16a34a;
}
```

Bu "ichida `<img>` bo'lgan har qanday `.karta` 'ni tanla" degani. Rasmsiz kartochkalar o'zgarishsiz qoladi.

:has() — "ota" selektor

`.karta:has(img) { border: 2px solid green; }`



:has() ota elementni ICHIDAGI farzandiga qarab tanlaydi — CSS tarixida yo'q bo'lgan imkoniyat

Yana foydali misollar:

```
/* Ichida tasdiqlangan checkbox bor label'ni belgila */
label:has(input:checked) {
  font-weight: bold;
  color: #16a34a;
}

/* Forma ichida xato bo'lsa, butun formaga qizil ramka */
form:has(input:invalid) {
  border: 2px solid #dc2626;
}

/* h2 dan keyin darhol p kelsa, h2 ning pastki bo'shlig'ini olib tashla */
h2:has(+ p) {
  margin-bottom: 0;
}
```

💡 **Nega bu shunchalik muhim?** Ilgari bunday holatlarni faqat JavaScript bilan hal qilardik (element'ga klass qo'shib). Endi toza CSS yetadi — tezroq, soddaroq, ishonchliroq.

🚩 `:has()` ichida bir nechta shart vergul bilan ham yoziladi: `.karta:has(img, video)` — ichida `img` **yoki** `video` bo'lsa.

19.11 `:is()` va `:where()` — selektorlarni guruhlash

Bu ikki funksiya uzun, takrorlanuvchi selektorlarni qisqartiradi.

Takrorlanuvchi kod:

```
header a:hover,  
nav a:hover,  
footer a:hover {  
  color: #2563eb;  
}
```

`:is()` bilan:

```
:is(header, nav, footer) a:hover {  
  color: #2563eb;  
}
```

`:is(A, B, C)` — "A yoki B yoki C" degani. Kod ancha qisqardi.

`:is()` va `:where()` farqi — specificity

Ular bir xil tanlashadi, lekin **specificity** (aniqlik bali, 10-bobni eslang) jihatidan tubdan farq qiladi:

- `:is()` — o'z ichidagi **eng kuchli** selektorning specificity'sini oladi. Masalan, `:is(#asosiy, p)` — `#asosiy` (id) borligi uchun butun selektor id-darajada kuchli bo'ladi.
- `:where()` — har doim **nol (0)** specificity beradi. Ichida nima bo'lishidan qat'i nazar, `:where()` "og'irligi" yo'q.

```
:is(#asosiy, .menyu) a { color: red; } /* specificity: id darajasida (kuchli) */  
:where(#asosiy, .menyu) a { color: red; } /* specificity: 0 (juda zaif) */
```

Nega `:where()` foydali? U "yengib o'tilishi oson" standart stillar yozish uchun ideal. Komponent kutubxonasi `:where()` bilan asosiy stil bersa, foydalanuvchi uni hatto oddiy klass bilan ham bimalol o'zgartira oladi — specificity urushiga kirmasdan.

💡 **Qisqacha:** ko'p tanlash kerak va specificity muhim bo'lsa → `:is()`. Yengilishi oson "yumshoq" standart kerak bo'lsa → `:where()`.

19.12 `@layer` — kaskad qatlamlari

Katta loyihalarda eng katta og'riq — **specificity urushlari**. Kim qaysi qoidani yengayotganini nazorat qilish qiyinlashadi, oxiri hamma `!important` yozishni boshlaydi. `@layer` (cascade layers — kaskad qatlamlari) bu muammoni tubdan yechadi.

`@layer` CSS'ni nomlangan **qatlamlarga** ajratadi va qatlamlarning **tartibini** belgilaydi. Eng muhimi:

Qatlamlar tartibi specificity'dan ustun turadi. Keyingi qatlamdagi qoida, avvalgi qatlamdagidan har doim kuchliroq — selektor qanchalik aniq bo'lishidan qat'i nazar.

```
/* Qatlam tartibini e'lon qilamiz (eng muhim qator!) */
@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; box-sizing: border-box; }
}

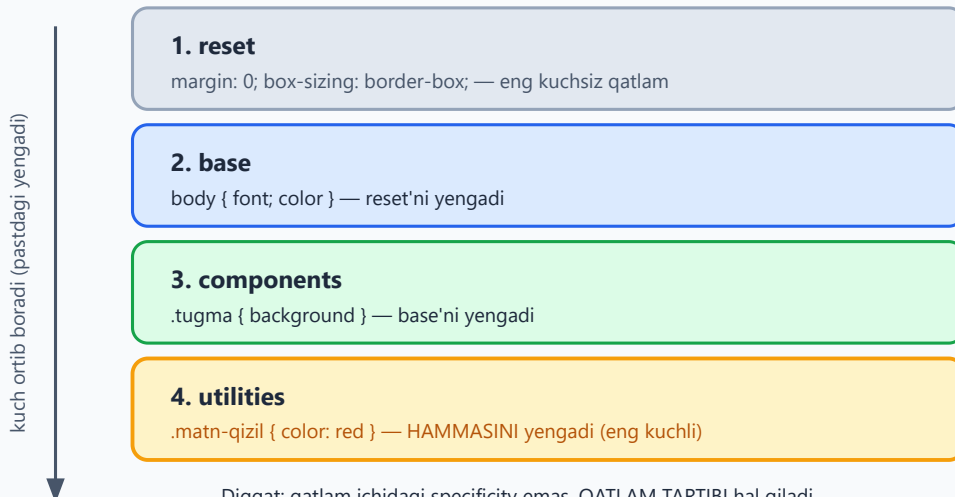
@layer base {
  body { font-family: system-ui; color: #1e293b; }
}

@layer components {
  .tugma#asosiy { background: #2563eb; } /* id ishlatilgan! */
}

@layer utilities {
  .matn-qizil { color: red; } /* oddiy klass */
}
```

@layer tartibi — kim kimni yengadi

@layer reset, base, components, utilities; (e'lon tartibi kuchni belgilaydi)



Diqqat: qatlam ichidagi specificity emas, QATLAM TARTIBI hal qiladi

`.tugma#asosiy` (komponentda) bo'lsa ham, `utilities`'dagi oddiy `.matn-qizil` g'olib chiqadi

Qatlamsiz (unlayered) qoidalar esa barcha qatlamlardan kuchliroq turadi

Diqqat: `components` qatlamida `.tugma#asosiy` (id'li, juda yuqori specificity) bo'lsa ham, `utilities` qatlamidagi oddiy `.matn-qizil` (juda past specificity) **g'olib chiqadi** — chunki `utilities` keyinroq e'lon qilingan qatlamda. Qatlam tartibi specificity'ni "ko'tarib o'tadi".

Nega bu inqilob? Endi siz aniq bilasiz: - `reset` eng kuchsiz (uni hamma yengadi), - `utilities` eng kuchli (u hammani yengadi).

`!important` deyarli kerak emas — tartibni qatlamlar hal qiladi.

✦ **Qatlamsiz qoidalar.** Hech qaysi `@layer` ichiga kirmagan oddiy qoidalar **barcha qatlamlardan kuchliroq** turadi. Shuning uchun muhim qoidalarni qatlamga solmasangiz, ular ustun bo'ladi.

💡 **Tartibni e'lon qilish.** `@layer reset, base, ...;` qatori qatlamlar tartibini **birinchi uchrashida** belgilaydi. Shuning uchun uni faylning eng boshiga yozish odat. Keyin qatlamlarni istalgan tartibda to'ldirsangiz ham, kuch tartibi o'sha e'londa qolaveradi.

19.13 Logical properties — til yo'nalishiga moslashish

`margin-left`, `padding-top`, `text-align: left` — bularning hammasi **jismoniy yo'nalishlar** (chap, o'ng, yuqori, past) ga bog'langan. Ammo dunyoning hamma tili chapdan o'ngga yozmaydi: arab va ibroniy tillari **o'ngdan chapga** (RTL), ba'zi yozuvlar yuqoridan pastga ketadi.

Logical properties (mantiqiy xossalar) jismoniy yo'nalish o'rniga **mantiqiy yo'nalishni** ishlatadi:

Jismoniy (eski)	Mantiqiy (yangi)	Ma'nosi
<code>margin-left</code> / <code>margin-right</code>	<code>margin-inline-start</code> / <code>margin-inline-end</code>	matn oqimi bo'ylab boshi/oxiri
<code>margin-left</code> + <code>margin-right</code>	<code>margin-inline</code>	ikkala yon (qisqartma)
<code>margin-top</code> + <code>margin-bottom</code>	<code>margin-block</code>	yuqori + past (qisqartma)
<code>padding-left</code>	<code>padding-inline-start</code>	ichki bo'shliq, oqim boshi

Jismoniy (eski)	Mantiqiy (yangi)	Ma'nosi
<code>width</code>	<code>inline-size</code>	oqim yo'nalishidagi o'lcham
<code>height</code>	<code>block-size</code>	bloklar yo'nalishidagi o'lcham
<code>text-align: left</code>	<code>text-align: start</code>	matn boshiga tekislash

```
.maqola {
  /* eski: faqat chapdan o'ngga to'g'ri ishlaydi */
  margin-left: 20px;
  padding-left: 16px;

  /* yangi: har qanday til yo'nalishida to'g'ri ishlaydi */
  margin-inline-start: 20px;
  padding-inline-start: 16px;
}
```

Nega bu muhim? `margin-inline-start` ingliz tilida (LTR) chapni, arab tilida (RTL) o'ngni anglatadi — avtomatik. Saytni `dir="rtl"` ga o'zgartirsangiz, butun layout to'g'ri "aks etadi", siz birorta qoidani qo'lda o'zgartirmaysiz.

`margin-inline` va `margin-block` qisqartmalari ham juda qulay:

```
.element {
  margin-inline: auto; /* chap va o'ng margin: auto (markazlash) */
  margin-block: 24px; /* yuqori va past margin: 24px */
  padding-inline: 16px; /* chap va o'ng ichki bo'shliq */
}
```

💡 **Maslahat:** yangi loyihada imkon qadar logical properties ishlatiing. Hatto faqat ingliz/o'zbek tilida ishlasangiz ham, ular kodingizni "kelajakka tayyor" qiladi va `margin-inline: auto` kabi qisqartmalar baribir qulay.

19.14 `aspect-ratio` — tomonlar nisbati

`aspect-ratio` element'ning eni va bo'yi nisbatini saqlaydi. Ilgari bu uchun "padding-hack" degan murakkab nayrang ishlatilardi, endi bitta qator yetadi:

```
.video {
  width: 100%;
```

```

    aspect-ratio: 16 / 9; /* en : bo'y = 16 : 9 */
  }

  .avatar {
    width: 80px;
    aspect-ratio: 1; /* kvadrat (1:1) */
  }

```

`aspect-ratio: 16 / 9` — element eni qancha bo'lishidan qat'i nazar, bo'yi avtomatik eni $\times 9/16$ bo'ladi. Video va rasm konteynerlari uchun ideal: ekran kichraysa ham nisbat buzilmaydi.

💡 **Nega foydali?** Responsiv video joylashtirganda balandlikni qo'lda hisoblash shart emas. `width: 100%` + `aspect-ratio: 16/9` — tamom, video har doim to'g'ri proporsiyada.

19.15 `gap` — hamma joyda bo'shliq

Avvalroq `gap` ni faqat flexbox va grid kontekstida ko'rgan bo'lishingiz mumkin (15- va 16-boblar). U element'lar **orasiga** bo'shliq qo'yadi, lekin chetlarga emas — bu uni `margin` 'dan ustun qiladi:

```

.menyu {
  display: flex;
  gap: 16px; /* har element orasida 16px */
}

.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px 16px; /* qatorlar orasi 24px, ustunlar orasi 16px */
}

```

Nega `gap` > `margin`? `margin` bilan bo'shliq qo'ysangiz, oxirgi element'da ham ortiqcha `margin` qoladi (yoki `:last-child { margin: 0 }` deb tuzatishga to'g'ri keladi). `gap` faqat **orada** bo'shliq qo'yadi — chetlar toza qoladi. Kod soddaroq, xato kamroq.

✳️ Bugun `gap` flexbox'da ham, grid'da ham barcha zamonaviy brauzerlarda ishlaydi. Element orasidagi bo'shliq uchun birinchi tanlovingiz `gap` bo'lsin.

19.16 Zamonaviy ranglar — `oklch()` va `color-mix()`

12-bobda `rgb()` va `hsl()` ranglarini ko'rgansiz. Zamonaviy CSS yana ikki kuchli vositani qo'shdi.

`oklch()` — bir tekis yorqinlik

`oklch(yorqinlik chroma rang-burchagi)` — inson ko'ziga **bir tekis** ko'rinadigan rang modeli:

```
.element {  
  color: oklch(0.7 0.15 250); /* yorqinlik 70%, to'yinganlik, ko'k  
  burchak */  
}
```

- Birinchi son — **yorqinlik** (0 = qora, 1 = oq).
- Ikkinchi — **chroma** (rangning to'yinganligi/quyuqligi).
- Uchinchi — **rang burchagi** (0-360, hsl'dagi kabi).

Nega `oklch`? `hsl()` da yorqinlikni o'zgartirsangiz, turli ranglar turlicha "yorqin" ko'rinadi (sariq och, ko'k quyuq). `oklch` da esa `0.7` yorqinlik — har qanday rang uchun bir xil idrok etiladigan yorqinlik beradi. Bu izchil palitra (rang to'plami) yaratishni osonlashtiradi.

`color-mix()` — ikki rangni aralashtirish

`color-mix()` ikki rangni berilgan nisbatda aralashtiradi:

```
.element {  
  /* 80% ko'k + 20% oq = ochroq ko'k */  
  background: color-mix(in oklch, #2563eb 80%, white);  
  
  /* asosiy rangni 30% qora bilan to'qlashtirish */  
  border-color: color-mix(in oklch, var(--asosiy-rang), black 30%);  
}
```

Nega kuchli? Custom property bilan birga ishlatilsa, bitta asosiy rangdan butun palitra (ochroq, to'qroq variantlar) avtomatik hosil qilinadi. Tugmaning `:hover` rangini qo'lda tanlash o'rniga `color-mix(in oklch, var(--rang), black 15%)` deysiz — har doim mos to'q variant chiqadi.

```
.tugma {  
  --rang: #2563eb;  
  background: var(--rang);  
}  
.tugma:hover {
```



```
background: color-mix(in oklch, var(--rang), black 15%);
}
```

💡 Bu boblar (oklch, color-mix) hozir barcha zamonaviy brauzerlarda ishlaydi. Eski brauzerlarni qo'llab-quvvatlash kerak bo'lsa, fallback rang bering: avval oddiy `background: #2563eb;`, keyin zamonaviy qatorni yozing — eski brauzer tushunmaganini "tashlab ketadi".

19.17 Hammasini birlashtirgan misol

Quyida shu bobdagi imkoniyatlar birga ishlagan kichik komponent. Har bir qatorni o'qib, qaysi xususiyat ekanini taniy olasizmi?

```
@layer components {
  .karta {
    /* custom properties */
    --bo'shliq: clamp(12px, 2vw, 24px);
    --rang: #2563eb;

    /* logical properties + gap */
    padding-block: var(--bo'shliq);
    padding-inline: calc(var(--bo'shliq) * 1.5);
    display: flex;
    flex-direction: column;
    gap: var(--bo'shliq);

    /* nesting */
    h2 { color: var(--rang); }

    /* :has() */
    &:has(img) {
      border-inline-start: 4px solid var(--rang);
    }

    /* nesting + color-mix */
    &:hover {
      background: color-mix(in oklch, var(--rang), white 90%);
    }
  }
}
```

Bu komponent: moslashuvchan bo'shliqli (clamp), rangni o'zgaruvchidan oladigan (custom property), til yo'nalishiga moslashadigan (logical), rasm bo'lsa chap chegara qo'shadigan (:has()), va hover'da silliq to'qlashadigan (color-mix) — barchasi 20 qatorda, toza CSS bilan.

Mashqlar

Quyidagi mashqlarni qog'ozda yoki brauzerda (Chrome DevTools'da) sinab ko'ring. Avval o'zingiz javob bering, keyin yechimni oching.

Mashq 1 — Custom property e'lon qilish

`:root` da `--asosiy-rang` ni `#16a34a` (yashil) qilib e'lon qiling va uni `.tugma` ning fon rangiga, `a` ning matn rangiga bering.

Yechim

```
:root {  
  --asosiy-rang: #16a34a;  
}  
  
.tugma { background: var(--asosiy-rang); }  
a { color: var(--asosiy-rang); }
```

`var()` ni unutmang — o'zgaruvchini o'qish uchun u shart.

Mashq 2 — Fallback qiymat

Quyidagi kodda `--bo'shliq` hech qayerda e'lon qilinmagan. `.quti` ning `padding` qiymati qancha bo'ladi?

```
.quti { padding: var(--bo'shliq, 20px); }
```

Yechim

`padding` **20px** bo'ladi. `--bo'shliq` mavjud emasligi uchun `var()` ning ikkinchi argumenti (fallback) — `20px` ishlatiladi. Fallback aynan shu holat uchun: o'zgaruvchi sozlanmasa ham komponent buzilmaydi.

Mashq 3 — `clamp()` ni tushunish

`font-size: clamp(1rem, 5vw, 2rem)` berilgan. Quyidagi uch holatda shrift qancha bo'ladi (taxminan)? - Juda tor ekran ($5vw = 0.6rem$ ga teng) - O'rta ekran ($5vw = 1.5rem$ ga teng) - Juda keng ekran ($5vw = 3rem$ ga teng)



Yechim



- Tor ekran: `5vw` (`0.6rem`) `1rem` 'dan kichik → `1rem` (MIN bilan cheklandi).
- O'rta ekran: `5vw` (`1.5rem`) oraliqda → `1.5rem` (moslashuvchan qiymat ishlatildi).
- Keng ekran: `5vw` (`3rem`) `2rem` 'dan katta → `2rem` (MAX bilan cheklandi).

`clamp()` hech qachon MIN'dan past yoki MAX'dan baland chiqmaydi.

Mashq 4 — Nesting va `&`

Quyidagi nesting'siz kodni nesting bilan, `&` dan to'g'ri foydalanib qayta yozing:

```
.menu { display: flex; }  
.menu a { color: #1e293b; }  
.menu a:hover { color: #2563eb; }
```



Yechim



```
.menu {  
  display: flex;  
  
  a {  
    color: #1e293b;  
  
    &:hover { color: #2563eb; }  
  }  
}
```



`a` farzand element bo'lgani uchun `&` shart emas, lekin `:hover` joriy element'ga (`a`) yopishishi kerak — shuning uchun `&:hover`.

Mashq 5 — `:has()` ni qo'llash

Ichida `<input:invalid>` (xato to'ldirilgan maydon) bor `form` ga `2px solid #dc2626` qizil chegara beradigan qoidani yozing.



Yechim



```
form:has(input:invalid) {  
  border: 2px solid #dc2626;  
}
```



`:has()` ota element (`form`) ni ichidagi farzandiga (`input:invalid`) qarab tanlaydi. Ilgari buni faqat JavaScript bilan qilish mumkin edi.

Mashq 6 — `:is()` va `:where()` specificity

Quyidagi ikki qoidadan qaysi biri kuchliroq (yuqori specificity'ga ega)? Nega?

```
:is(#header, .nav) a { color: red; }  
:where(#header, .nav) a { color: blue; }
```



`<a>` element `#header` ichida. Qaysi rang g'olib chiqadi?



Yechim



Matn **qizil** (`red`) bo'ladi.

- `:is(#header, .nav)` — ichidagi eng kuchli selektor `#header` (id) bo'lgani uchun id-darajadagi specificity oladi (kuchli).
- `:where(#header, .nav)` — har doim **0** specificity beradi (juda zaif).

Kuchli `:is()` qoidasi zaif `:where()` qoidasini yengadi, manba tartibidan qat'i nazar.

Mashq 7 — `@layer` tartibi

```
@layer base, utilities;  
  
@layer utilities {  
  .matn-kichik { font-size: 12px; }  
}  
  
@layer base {  
  p#muhim { font-size: 24px; }  
}
```



`<p id="muhim" class="matn-kichik">` element shrifti qancha bo'ladi?



Yechim



Shrift **12px** bo'ladi.

`base` qatlamidagi `p#muhim` juda yuqori specificity'ga ega (id bor), lekin `@layer base, utilities;` e'loniga ko'ra `utilities` qatlami **keyin** keladi — demak u kuchliroq. Qatlam tartibi specificity'dan ustun: `utilities` 'dagi oddiy `.matn-kichik` g'olib chiqadi.

Bu — `@layer` ning asosiy g'oyasi: kuchni qatlam tartibi hal qiladi, selektor og'irligi emas.

Mashq 8 — Logical properties

`.maqola` ga: chap va o'ng tomonidan `auto` margin (markazlash), yuqori va pastdan `32px` margin, har ikki yon ichki bo'shlik (`padding`) `16px` bering — faqat **logical properties** ishlatib.



Yechim



```
.maqola {  
  margin-inline: auto; /* chap + o'ng margin: auto */  
  margin-block: 32px; /* yuqori + past margin: 32px */  
  padding-inline: 16px; /* chap + o'ng padding: 16px */  
}
```

`inline` o'qi — matn oqimi bo'ylab (LTR'da chap-o'ng), `block` o'qi — bloklar yo'nalishi (yuqori-past). Bu kod arab tili (RTL) sahifasida ham to'g'ri ishlaydi.

Mashq 9 — Theming (qiyinroq)

Custom property'lardan foydalanib, `<html>` ga `tungi` klassi qo'shilganda `body` foni qora (`#0f172a`), matni och (`#e2e8f0`) bo'ladigan tema tizimini yozing. Klassi yo'q holatda fon oq, matn to'q bo'lsin.



Yechim



```
:root {  
  --fon: #ffffff;  
  --matn: #1e293b;  
}
```

```
:root.tungi {  
  --fon: #0f172a;  
  --matn: #e2e8f0;  
}
```

```
body {  
  background: var(--fon);  
  color: var(--matn);  
}
```

```
document.documentElement.classList.toggle('tungi');
```

`<html>` ga `tungi` klassi qo'shilsa, o'zgaruvchilar yangi qiymat oladi va `body` avtomatik tungi rejimga o'tadi — `body` qoidasiga umuman tegmasdan. Bu custom property theming'hing kuchi.

20 — CSS arxitekturası va uslublar

← Oldingi: 19 — Zamonaviy CSS · 🏠 README · Keyingi: 21 — Formalar, UI holatlari va kirish imkoniyati →

Bu bobda: katta loyihada CSS'ni tartibli, boshqarib bo'ladigan qilib yozish — global scope va specificity muammolari nega kelib chiqishi, BEM kabi nomlash konvensiyalari, reset va normalize, fayllarni komponentlarga bo'lish, DRY, design tokens hamda utility-first (Tailwind) yondashuvini semantik usul bilan taqqoslash.

20.1 Nega bu bob — "katta loyiha" haqida?

Shu paytgacha sen CSS'ning *qurollarini* o'rgandingiz: selektorlar, box model, flexbox, grid, animatsiyalar. Bitta tugma yoki bitta sahifa uchun bu yetarli. Lekin yuzlab komponentdan, o'nlab sahifadan iborat **katta loyiha** boshlaganingda butunlay yangi muammo paydo bo'ladi: kodning o'zi tartibsizlanib ketadi.

Tasavvur qil: loyihada 5000 qator CSS bor. Sen bitta tugmaning rangini o'zgartirasan — va kutilmaganda boshqa sahifadagi forma buziladi. Yoki yangi a'zo jamoaga qo'shiladi, kodga qaraydi va hech narsa tushunmaydi: bu klass nimaga kerak, uni o'chirsam nima buziladi?

Bu bobning savoli: CSS'ni shunday yozish kerakki, u **o'sib borganda ham boshqarib bo'ladigan** bo'lsin.

Buni hal qiladigan qoidalar, naqshlar va konvensiyalar majmuasi **CSS arxitekturası** deyiladi. Bu — sintaksis emas, balki *intizom*. Aynan shu intizom havaskorni professionaldan ajratadi.

✦ **Atamalar:** - **architecture** (arxitektura) — kodni qanday tashkil qilish, nomlash va bo'lish bo'yicha umumiy reja. - **methodology** (metodologiya) — shu rejaning aniq qoidalar to'plami (masalan BEM). - **convention** (konvensiya) — jamoa kelishib olgan yozish uslubi (masalan klass nomlarini qanday yozish).

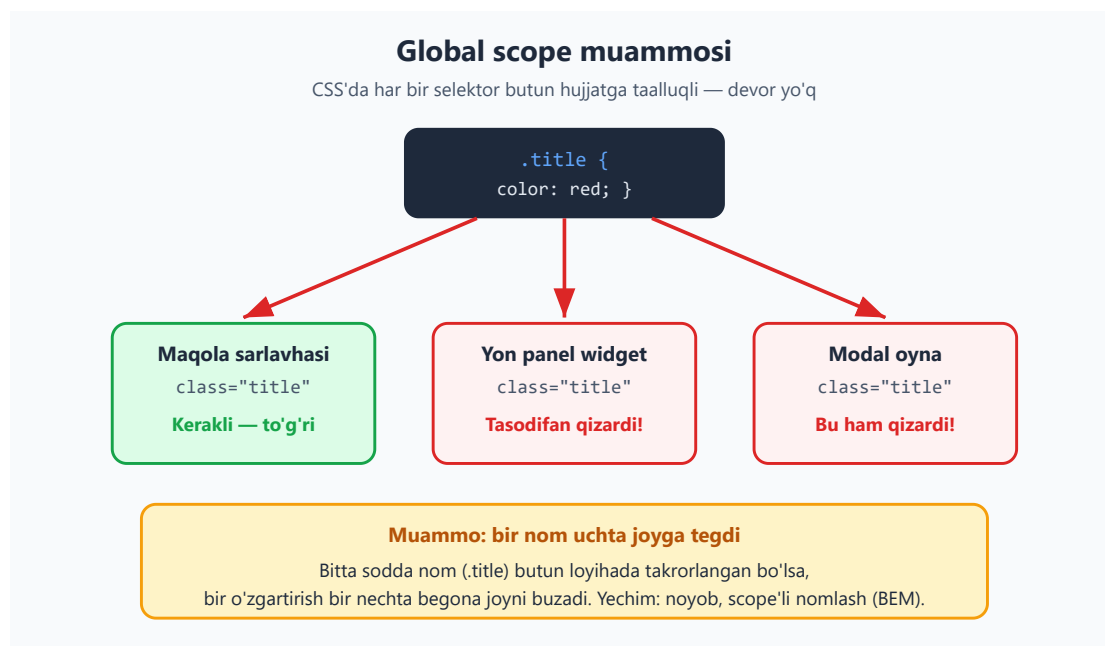
20.2 CSS nega tartibsizlanadi? Ikki tub sabab

CSS o'ziga xos tarzda "qiyin" til — sintaksisi oson bo'lsa-da, kattalashganda chigallashishga **moyil**. Buning ikkita tub sababi bor. Ularni tushunmasdan turib hech qanday metodologiya yordam bermaydi.

Sabab 1 — global scope (umumiy ko'rinish maydoni)

JavaScript yoki boshqa tillarda funksiya ichidagi o'zgaruvchi faqat o'sha funksiya ichida yashaydi — bu **scope** (ko'rinish maydoni). CSS'da esa bunday devor **yo'q**: har qanday selektor butun hujjatga taalluqli.

Sen `.title { color: red; }` deb yozsang, bu qoida sahifadagi **barcha** `class="title"` elementlarga tegadi — maqola sarlavhasiga ham, yon paneldagi widgetga ham, modal oynaga ham. Ularning birortasini ko'zda tutmagan bo'lsang ham.



```
/* "title" oddiy va keng tarqalgan nom */  
.title {  
  color: red;  
}
```

```
<h1 class="title">Maqola</h1>      <!-- buni ko'zladik -->  
<div class="widget">  
  <span class="title">Ob-havo</span>  <!-- bu ham qizardi! -->  
</div>
```

Natija: ikkala matn ham qizil. Birinchisi kerakli edi, ikkinchisi — tasodif. Loyiha o'sgan sari bunday tasodifiy to'qnashuvlar ko'payadi. Bu — **global scope** ning to'g'ridan-to'g'ri oqibati.

💡 **Hayotiy analogiya:** global scope — bu butun shahar uchun bitta umumiy ism ro'yxati. Agar ikki oila bolasiga "Ali" deb nom qo'ysa, "Ali!" deb chaqirganingda ikkalasi ham keladi. Yechim — familiya qo'shish (ya'ni nomni noyob qilish). BEM aynan shuni qiladi.

Sabab 2 — specificity urushlari

10-bobda **specificity** (aniqlik bali) ni o'rgandik: aniqroq selektor g'olib chiqadi. Bu kichik loyihada foydali, lekin kattasida **qurollanish poygasi** ga aylanadi.

Kimdir bir stilni bosib o'tish uchun selektorni biroz aniqroq qiladi. Keyin boshqasi uni bosish uchun yana aniqroq qiladi. Oxir-oqibat hamma `!important` qo'shishga o'tadi:

```
.tugma { background: blue; }  
.panel .tugma { background: green; } /* avvalgisini bosish uchun */  
#sahifa .panel .tugma { background: orange; } /* yana bosish uchun */  
#sahifa .panel .tugma { background: red !important; } /* taslim bo'ldik */
```

Endi bu tugmaning rangini o'zgartirish uchun yangi kelgan kishi **yana** `!important` qo'shishi kerak. Kod tobora qattiqlashadi — har bir o'zgartirish kurashga aylanadi.

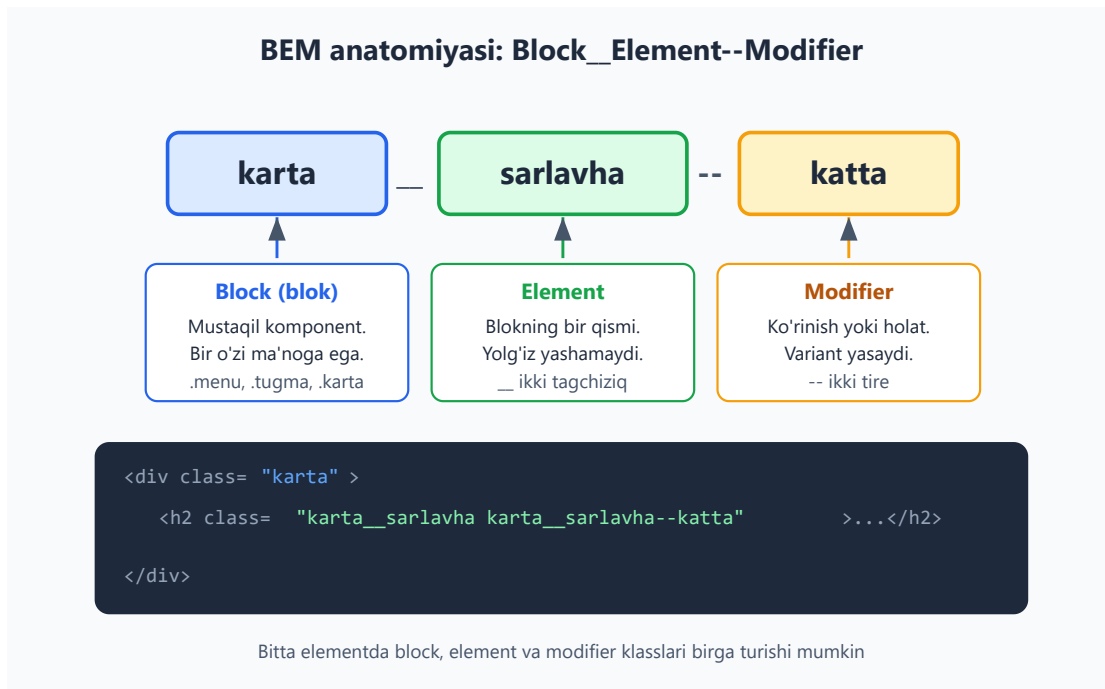
⚠ **Asosiy g'oya:** CSS arxitekturasining katta qismi — **specificity'ni past va tekis tutish**. Agar barcha selektorlar bir xil og'irlikda (masalan bitta class) bo'lsa, urush boshlanmaydi: oxirgi yozilgan oddiygina g'olib bo'ladi.

20.3 Yechim: nomlash konventsiyasi — BEM

Yuqoridagi ikkala muammoning ham ildizi bitta: **nomlar**. Agar nomlar noyob va bashorat qilinadigan bo'lsa, to'qnashuv bo'lmaydi; agar selektorlar tekis (bitta class) bo'lsa, specificity urushi bo'lmaydi.

BEM — eng mashhur nomlash metodologiyasi. Nomi uchta so'zdan: **Block**, **Element**, **Modifier**. U klass nomini aniq tuzilishga soladi:

```
blok__element--modifier
```



Uch qism — anatomiya

1. Block (blok) — mustaqil, qayta ishlatiladigan komponent. U bir o'zi ma'noga ega va boshqa joyga ko'chirsa ham ishlaydi.

```
.karta { }      /* karta komponenti */  
.menu { }      /* menyu komponenti */  
.tugma { }     /* tugma komponenti */
```

2. Element — blokning bir bo'lagi. U yolg'iz yashay olmaydi — faqat o'z bloki ichida ma'no kasb etadi. Blok nomidan keyin **ikkita tagchiziq** (__) bilan yoziladi:

```
.karta__rasm { }      /* kartaning rasmi */  
.karta__sarlavha { }  /* kartaning sarlavhasi */  
.karta__matn { }     /* kartaning matni */
```

3. Modifier — blok yoki elementning **varianti** yoki **holati**. Ko'rinishni yoki holatni o'zgartiradi. **Ikkita tire** (--) bilan yoziladi:

```
.tugma--katta { }    /* katta o'lchamli variant */  
.tugma--ochiq { }    /* och rangli variant */  
.karta--tanlangan { } /* tanlangan holat */
```

To'liq misol

```
<article class="karta karta--tanlangan">
  
  <h2 class="karta__sarlavha">Mahsulot nomi</h2>
  <p class="karta__matn">Tavsif...</p>
  <button class="tugma tugma--katta">Sotib olish</button>
</article>
```

```
.karta { border: 1px solid #ccc; padding: 16px; }
.karta--tanlangan { border-color: #2563eb; }

.karta__sarlavha { font-size: 20px; margin: 8px 0; }
.karta__matn { color: #475569; }

.tugma { padding: 8px 16px; background: #2563eb; color: white; }
.tugma--katta { padding: 12px 24px; font-size: 18px; }
```

BEM nega ikkala muammoni ham hal qiladi?

- **Global scope:** `.karta__sarlavha` nomi noyob — uni boshqa joyda tasodifan ishlatib qo'yish ehtimoli deyarli yo'q. Nom o'zining qaysi blokka tegishli ekanini aytib turadi.
- **Specificity urushi:** har bir BEM klass — bitta class selektori, ya'ni bali $(0, 0, 1, 0)$. Hamma tekis bo'lgani uchun urush boshlanmaydi. Hech qachon `.karta` `.karta__sarlavha` deb ichma-ich yozma — shunchaki `.karta__sarlavha`.

⚠ **Tez-tez xato:** modifierni yolg'iz ishlatish. `tugma--katta` o'zi to'liq tugma emas — u faqat **qo'shimcha**. HTML'da har doim asosiy blok bilan birga yoz: `class="tugma tugma--katta"`, yolg'iz `class="tugma--katta"` emas.

💡 **Eslatma:** element ichida element bo'lsa ham, nomni cho'zma. `.karta__rasm__belgi` emas — `.karta__belgi` deb yoz. BEM faqat **bir** daraja `__` ni tavsiya qiladi; chuqurlik HTML tuzilishida bo'ladi, klass nomida emas.

20.4 Boshqa metodologiyalar: OOCSS va SMACSS

BEM yagona yo'l emas. U paydo bo'lishidan oldin ham g'oyalar bor edi — ularni bilib qo'ygan foydali, chunki ko'p loyihalarda izlarini uchratasan.

OOCSS — Object-Oriented CSS

OOCSS (ob'ektga yo'naltirilgan CSS) ikkita asosiy tamoyilga tayanadi:

1. Tuzilma va ko'rinishni ajratish. "Qanday joylashgan" (o'lcham, padding) bilan "qanday ko'rinadi" (rang, soya) ni alohida klasslarga bo'l:

```
/* tuzilma - qayta ishlatiladi */
.tugma { padding: 8px 16px; border-radius: 6px; }

/* ko'rinish - alohida */
.kok { background: #2563eb; color: white; }
.yashil { background: #16a34a; color: white; }
```

```
<button class="tugma kok">Saqlash</button>
<button class="tugma yashil">Tasdiqlash</button>
```

2. Konteyner va tarkibni ajratish. Element o'zi qayerda turishidan **qat'i nazar** bir xil ko'rinishi kerak. Ya'ni `.sarlavha` har joyda bir xil bo'lsin, `.yonpanel` `.sarlavha` deb joyga bog'lab qo'yma.

✦ OOCSS'ning eng muhim merosi — **takrorlanadigan, kontekstdan mustaqil** komponentlar g'oyasi. Tailwind kabi utility yondashuvlar ham aslida shu ildizdan o'sgan.

SMACSS — Scalable and Modular Architecture for CSS

SMACSS qoidalarini **beshta toifaga** ajratadi va ularni alohida fayllarda saqlashni tavsiya qiladi:

Toifa	Nima	Misol
Base	Teglarning asosiy stillari	<code>body</code> , <code>a</code> , <code>h1</code>
Layout	Sahifaning katta bo'laklari	<code>.header</code> , <code>.sidebar</code> , <code>.grid</code>
Module	Qayta ishlatiladigan komponentlar	<code>.karta</code> , <code>.tugma</code> , <code>.menu</code>
State	Holatlar	<code>.is-active</code> , <code>.is-hidden</code>
Theme	Mavzu (rang sxemasi)	<code>.theme-dark</code>

SMACSS'ning kuchi — **fayl tashkili va holat nomlashi** (`is-active` kabi prefiks). Ko'p jamoalar BEM (nomlash uchun) va SMACSS (fayl tashkili uchun) ni **birga** ishlatadi.

💡 **Xulosa:** bu metodologiyalar raqobatchi emas — bir-birini to'ldiradi. BEM senga *nomlashni* beradi, SMACSS *tashkilni*, OOCSS *qayta ishlatishni*. Aksariyat zamonaviy loyihalar uchalasidan ham bir oz oladi.

20.5 CSS reset va normalize — nega kerak?

Brauzer har bir HTML elementga **o'zining standart stilini** beradi (10-bobda buni "user-agent stillari" degan edik). Masalan `<h1>` katta va qalin, `<ul>` da nuqtalar va chap padding bor, `<body>` da kichik margin bor.

Muammo: bu standartlar **brauzerdan brauzerga biroz farq qiladi**. Chrome va Safari'da bir xil `<button>` boshqacha ko'rinishi mumkin. Natijada sening saying turli brauzerda har xil ko'rinadi.

Yechim ikki xil falsafada keladi:

Reset — hammasini nolga tushir

CSS reset brauzerning standart stillarini deyarli **butunlay o'chiradi**, toza varaqdan boshlash uchun:

```
/* sodda reset namunasi */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Bu yondashuvda `<h1>` ham `<p>` ham bir xil o'lchamda bo'ladi — har bir narsani **noldan o'zing** belgilaysan. To'liq nazorat, lekin ko'proq yozish.

Normalize — farqlarni tekisla, foydalisini saqla

Normalize.css boshqacha: u standartlarni o'chirmaydi, balki brauzerlar **orasidagi farqlarni tekislaydi** va foydali standartlarni (masalan `<h1>` ning kattaligi) saqlab qoladi. Faqat nomuvofiqliklarni tuzatadi.

	Reset	Normalize
Standart stillar	O'chiriladi	Saqlanadi
Falsafa	"Noldan boshla"	"Farqlarni tekisla"
Natija	Hamma element yalang'och	Mantiqiy standartlar qoladi
Qachon	To'liq nazorat kerak bo'lsa	Tez, standartlar yoqsa

✦ **Zamonaviy o'rtacha yo'l.** Bugun ko'pchilik to'liq reset ham, to'liq normalize ham emas, balki kichik **"modern reset"** ishlatadi — eng foydali tuzatishlarni oladi:

```
*, *::before, *::after { box-sizing: border-box; }
* { margin: 0; }
body { line-height: 1.5; -webkit-font-smoothing: antialiased; }
img, picture, video { display: block; max-width: 100%; }
input, button, textarea, select { font: inherit; }
```

Nega bu kerak? `box-sizing: border-box` o'lchovni bashorat qilinadigan qiladi (11-bob), `img { max-width: 100% }` rasmlar konteynerdan toshib chiqmasligini ta'minlaydi, `font: inherit` esa formalaridagi shrift body shriftiga moslashishini ta'minlaydi. Bularsiz har loyihada bir xil "boshlang'ich og'riqlar" takrorlanadi.

💡 **Maslahat:** reset/normalize **eng birinchi** ulanishi kerak — barcha boshqa stillarningdan oldin. Aks holda u senikini bosib qo'yadi.

20.6 Fayl tashkili: komponentlarga bo'lish

Bitta 5000 qatorli `style.css` — kabus. Kerakli joyni topish qiyin, tahrirlash qo'rqinchli. Yechim — kodni **mantiqiy bo'laklarga**, har biri o'z faylida bo'ladigan qilib bo'lish.

Fayl tashkili: bitta dev emas, mantiqiy bo'laklar

Yomon: bitta ulkan fayl

`style.css`

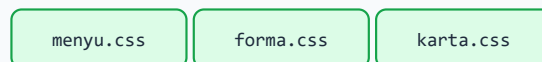
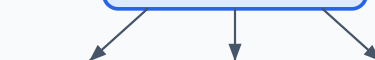
```
/* reset */
/* tugmalar */
/* kartalar */
/* menyu */
/* formalar */
... 4000 qator ...
/* footer */
```

Topish qiyin, qo'rqib tahrirlaysan

Yaxshi: komponentlarga bo'lingan

`main.css`

@import barchasini ulaydi



Har fayl bir vazifa — tez topiladi

```
/* main.css — tartib MUHIM: pastdagi yuqorisini yengadi */
@import "reset.css" ;
@import "tugma.css" ;
/* yoki: @layer reset, components, utilities; */
```

```
styles/
├─ reset.css      ← eng birinchi
├─ tokens.css     ← rang/o'lcham o'zgaruvchilari
├─ base.css       ← body, sarlavhalar, linklar
```

```

├── components/
│   ├── tugma.css
│   ├── karta.css
│   └── menyu.css
└── main.css      ← hammasini birlashtiradi

```

Birlashtirishning ikki usuli

1. @import (eski, sodda). `main.css` boshqa fayllarni ichiga "tortib oladi". Tartib muhim — pastdagi yuqoridagini bosadi:

```

/* main.css */
@import "reset.css";
@import "tokens.css";
@import "base.css";
@import "components/tugma.css";
@import "components/karta.css";

```

⚠ Sof `@import` har faylni alohida yuklaydi (sekinroq). Shuning uchun amalda ko'pincha **build vositasi** (Vite, webpack, Sass) hammasini bitta faylga "qadoqlaydi". Lekin tushuncha bir xil — kodni mantiqan bo'lasan.

2. Cascade layers — @layer (zamonaviy, kuchli). 19-bobda ko'rganimizdek, `@layer` qatlamlarning *kuchini* e'lon tartibiga ko'ra belgilaydi. Bu specificity muammosini ildizidan hal qiladi:

```

/* qatlam tartibini bir marta e'lon qilamiz */
@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; box-sizing: border-box; }
}
@layer components {
  .tugma { background: blue; }
}
@layer utilities {
  .matn-qizil { color: red; } /* components'ni yengadi — qatlam
tartibi tufayli */
}

```

Nega bu kuchli? Endi `.matn-qizil` `components` ichidagi murakkabroq selektordan ham g'olib bo'ladi — chunki **qatlam tartibi specificity'dan ustun turadi**. Specificity urushi tugaydi: kim qaysi qatlamda turishi hal qiladi, selektor uzunligi emas.

20.7 DRY — takrorlanishni kamaytirish

DRY — "Don't Repeat Yourself" (o'zingni takrorlama). Agar bir qiymat yoki naqsh kodda qayta-qayta uchrasa, uni **bir joyga** chiqarish kerak. Aks holda o'zgartirish kerak bo'lganda hamma nusxani qidirib yurasan — va bittasini albatta unutib qoldirasan.

Takrorlanish — muammo

```
.tugma { background: #2563eb; border-radius: 6px; }  
.havola { color: #2563eb; }  
.belgi { border: 2px solid #2563eb; }  
.sarlavha { color: #2563eb; }
```

Bu yerda `#2563eb` to'rt marta takrorlangan. Brendning ko'k rangini o'zgartirmoqchi bo'lsang, to'rttasini ham topib o'zgartirishing kerak. Loyiha kattalashganda bu 40 ta bo'lib ketadi.

Yechim — custom properties (design tokens)

12-bobda ko'rgan CSS custom properties (`--o'zgaruvchi`) aynan shu uchun ideal. Rangni **bir marta** e'lon qilamiz, hamma joyda `var()` bilan ishlatamiz:

```
:root {  
  --rang-asosiy: #2563eb;  
  --radius: 6px;  
}  
  
.tugma { background: var(--rang-asosiy); border-radius: var(--radius); }  
.havola { color: var(--rang-asosiy); }  
.belgi { border: 2px solid var(--rang-asosiy); }  
.sarlavha { color: var(--rang-asosiy); }
```

Endi brend rangini o'zgartirish uchun **bitta** qatorni — `--rang-asosiy` ni — tahrirlaysan, hamma joy avtomatik yangilanadi.

💡 **DRY'ni oshirib yuborma.** Ikki narsa *tasodifan* bir xil bo'lsa, ularni majburan birlashtirma. Masalan tugma va ogohlantirish hozir bir xil padding'ga ega bo'lishi mumkin, lekin ular **boshqa-boshqa narsa** — kelajakda biri o'zgarsa, ikkinchisi o'zgarmasligi kerak. DRY — bir xil *ma'no* uchun, bir xil *qiymat* uchun emas.

20.8 Design tokens — dizaynni o'zgaruvchilarga aylantirish

DRY g'oyasini bir pog'ona yuqoriga ko'tarsak — **design tokens** ga kelamiz. Design token — bu dizayn qarorining (rang, masofa, shrift o'lchami, radius) **nomlangan o'zgaruvchi**

ko'rinishidagi yagona manbai.

G'oya: dizayner aniq #2563eb rangini emas, balki "asosiy rang" tushunchasini beradi. Sen uni bir marta tokenga aylantirib, butun tizimda shu tokenni ishlatasan. Natijada dizayn izchil bo'ladi va o'zgartirish bir nuqtadan boshqariladi.

```
:root {  
  /* ranglar */  
  --rang-asosiy: #2563eb;  
  --rang-matn: #1e293b;  
  --rang-fon: #f8faff;  
  
  /* masofa shkalasi (izchil bo'shliqlar) */  
  --bosh-1: 4px;  
  --bosh-2: 8px;  
  --bosh-3: 16px;  
  --bosh-4: 24px;  
  
  /* tipografiya */  
  --shrif-asosiy: "Segoe UI", system-ui, sans-serif;  
  --olcham-katta: 1.5rem;  
  
  /* boshqalar */  
  --radius: 6px;  
  --soya: 0 2px 8px rgba(0,0,0,0.1);  
}
```

Endi komponentlar faqat tokenlardan foydalanadi — hech qachon "xom" qiymat yozmaydi:

```
.karta {  
  padding: var(--bosh-3);  
  background: var(--rang-fon);  
  border-radius: var(--radius);  
  box-shadow: var(--soya);  
}
```

Nega masofa shkalasi muhim? Agar har bir komponentda 13px, 15px, 17px kabi tasodifiy raqamlar yozsang, sahifa "notekis" ko'rinadi. Cheklangan shkala (4, 8, 16, 24) butun saytni vizual **maromli** qiladi — bu professional dizaynning maxfiy retsepti.

✦ **Tokenlar — dizayn tizimi (design system) ning poydevori.** Katta kompaniyalar (masalan tizimlar) butun mahsulot oilasini shunday tokenlar to'plamiga bog'laydi: bir token o'zgarsa, barcha ilovalar yangilanadi.

20.9 Utility-first yondashuv — Tailwind falsafasi

Endi butunlay boshqacha falsafaga o'tamiz. Hozirgacha ko'rganlarimiz — **semantik** yondashuv: sen `.karta`, `.tugma` kabi *ma'noli* klass yasaysan va unga CSS yozasan.

Utility-first yondashuv buni teskari qiladi: sen CSS **yozmaysan**. Buning o'rniga oldindan tayyor, har biri **bitta vazifa bajaradigan** mayda klasslardan ("utility") to'g'ridan-to'g'ri HTML'da foydalanasan.



```
<!-- Semantik: bitta ma'noli klass, CSS alohida -->
<button class="tugma">Yuborish</button>

<!-- Utility-first: har klass = bitta xossa, CSS yozilmaydi -->
<button class="px-4 py-2 bg-blue-600 text-white rounded">Yuborish</button>
```

Bu yerda `px-4` = gorizontaal padding, `py-2` = vertikal padding, `bg-blue-600` = ko'k fon, `text-white` = oq matn, `rounded` = yumaloq burchak. Hech qaysisi uchun sen CSS yozmaysan — ular **Tailwind CSS** kabi kutubxonada tayyor turadi.

Afzalliklari

- **Tezlik.** Yangi komponent yasash uchun fayl ochib, klass nomi o'ylab, CSS yozish kerak emas — to'g'ridan HTML'da terib ketasan.
- **Nom o'ylamaysan.** "Bu klassni nima deb atasam?" degan azob yo'qoladi (dasturlashning eng qiyin ishlaridan biri — nom topish).

- **Scope muammosi yo'q.** `px-4` har doim aniq bir narsa qiladi va hech qachon o'zgarmaydi — 20.2 dagi global scope to'qnashuvi bo'lmaydi.
- **O'lik kod yo'qoladi.** Komponentni o'chirsang, uning klasslari ham HTML bilan ketadi — "qaerdadir keraksiz CSS qolib ketdi" muammosi kamayadi.

Kamchiliklari

- **HTML uzun bo'ladi.** Bitta elementda 10-15 ta klass to'planishi — o'qish noqulay bo'lishi mumkin.
- **Yangi til.** `px-4`, `flex`, `gap-2` — bularning hammasini o'rganish kerak; bu CSS'ni *yashirmaydi*, balki uning ustiga yana bir qatlam qo'shadi.
- **Takror.** Bir xil tugma 20 joyda bo'lsa, klass ro'yxatini 20 marta nusxalaysan (buni komponent ramkalari yoki `@apply` yumshatadi).

⚠ **Muhim tushunmovchilik:** utility-first "inline stil bilan bir xil" emas.

`style="padding:16px"` haqiqiy inline stil — u takrorlanadi, javob bermaydi (responsive emas) va specificity'si yuqori. Utility klasslar esa cheklangan **design tokenlardan** keladi (`px-4` har doim aynan bir qiymat), `:hover` va breakpoint'larni qo'llaydi (`md:px-8`) va izchillikni majburlaydi. Bu — tartibsizlik emas, tizim.

20.10 Semantik vs utility — qaysi birini tanlash?

Bu — zamonaviy CSS olamidagi eng katta bahslardan biri. Haqiqat shundaki, **ikkalasi ham to'g'ri** — vaziyatga bog'liq. Ularni adolatli taqqoslaylik:

Mezon	Semantik (.tugma)	Utility-first (px-4 py-2)
HTML o'qilishi	Toza, ma'noli	Klasslar bilan to'la
CSS hajmi	O'sib boradi	Deyarli o'zgarmaydi
Yangi komponent	Sekinroq (CSS yoz)	Tez (HTML'da ter)
Nom o'ylash	Kerak	Kerak emas
Qayta ishlatish	Klass orqali oson	Nusxa yoki komponent kerak
Izchillik	Intizomga bog'liq	Token tizimi majburlaydi

Mezon	Semantik (.tugma)	Utility-first (px-4 py-2)
O'rganish	CSS yetarli	+ utility tilini o'rganish

💡 **Amaliy maslahat:** - **Mayda yoki tez prototip** — utility-first juda qulay. - **Murakkab, takrorlanuvchi komponentlar** (karta, modal) — semantik klass (BEM) toza qoladi. - **Eng kuchli kombinatsiya** — ikkalasini birga: tuzilmani BEM komponenti qil, ichidagi mayda sozlashlarni utility bilan ber. Tailwind'ning `@apply` direktivi aynan shu ko'prikn quradi:

```
/* utility'larni semantik klass ichiga "yig'ish" */
.tugma {
  @apply px-4 py-2 bg-blue-600 text-white rounded;
}
```

✳️ **Asosiy xulosa:** "qaysi biri yaxshi?" degan savol noto'g'ri. To'g'ri savol — "bu loyiha, bu jamoa, bu vazifa uchun qaysi biri mos?". Professional ikkalasini ham biladi va o'rinni tanlaydi.

20.11 Maintainable CSS prinsiplari — yakuniy oltin qoidalar

Endi butun bobni amaliy tamoyillar ro'yxatiga jamlaymiz. Bular qaysi metodologiyani tanlashingdan **qat'i nazar** amal qiladi — bu CSS arxitekturasining mohiyati.

- 1. Specificity'ni past va tekis tut.** Imkon qadar bitta class bilan ishla. `id` ni stil uchun ishlatma, ichma-ich selektor zanjirlaridan qoch. Bu — specificity urushining oldini oladigan eng muhim qoida.
- 2. !important dan qoch.** U vaqtinchalik yengillik berib, keyin "important urushi" ga olib keladi. Faqat o'zgartira olmaydigan tashqi kutubxonani majburlashda oxirgi chora sifatida ishlat.
- 3. Bir vazifa — bir joy (DRY).** Takrorlanadigan qiymatlarni design tokenlarga (`var()`) chiqar. Bir narsani o'zgartirish bir nuqtadan boshqarilsin.
- 4. Izchil nomlash.** Bitta konventsiyani tanla (BEM tavsiya etiladi) va **butun jamoa** unga rioya qilsin. Yarmi BEM, yarmi tasodifiy nom — eng yomon holat.
- 5. Komponentlarga bo'l.** Bitta ulkan fayl emas, har komponent o'z faylida. Kerakli joyni topish va xavfsiz tahrirlash oson bo'lsin.

6. Reset/normalize bilan boshla. Brauzer farqlarini birinchi qatorda tekisla — keyin barcha stiling bashorat qilinadigan poydevorga quriladi.

7. Joyga emas, komponentga bog'la. `.yonpanel` `.tugma` deb tugmani joyga bog'lama. Tugma har joyda bir xil bo'lsin; farqi kerak bo'lsa — `tugma--kichik` modifier ber.

8. Kodni o'zgartirish oson bo'lsin. Eng asosiy mezon: "olti oydan keyin yangi kishi (yoki o'zing) bu kodga qarab, qo'rqmasdan o'zgartira oladimi?". Agar javob "ha" bo'lsa — arxitektura yaxshi.

💡 **Yakuniy fikr:** CSS arxitekturasi — bu kelajakdagi o'zingga (va jamoangga) yozilgan sovg'a. Bugun biroz ko'proq intizom — ertaga soatlab azobdan xalos qiladi. Sintaksisni hamma o'rganadi; arxitekturani biladigan kishi esa **professional** deyiladi.

Mashqlar

Quyidagi mashqlarni qog'ozda yoki brauzerda sinab ko'r. Avval o'zing javob ber, keyin yechimni och.

Mashq 1 — BEM ga aylantirish

Quyidagi "loyqa" HTML va CSS'ni BEM konventsiyasiga moslab qayta yoz. Komponent — `profil` kartasi: ichida avatar, ism va "kuzatish" tugmasi (faol holati bor).

```
<div class="card">
  <img class="img">
  <h3 class="name">Aziz</h3>
  <button class="btn active">Kuzatish</button>
</div>
```

 **Yechim** 

```
<div class="profil">
  <img class="profil__avatar">
  <h3 class="profil__ism">Aziz</h3>
  <button class="tugma tugma--faol">Kuzatish</button>
</div>
```

```
.profil { }
.profil__avatar { }
.profil__ism { }
.tugma { }
.tugma--faol { }
```

E'tibor ber: `tugma` alohida blok (har joyda ishlatiladi), shuning uchun `profil__tugma` emas, `tugma` deb nomladik. Faol holat — modifier (`tugma--faol`), HTML'da asosiy `tugma` bilan birga turadi.

Mashq 2 — Reset yoki normalize?

Quyidagi ikki holatda reset yoki normalize'dan qaysi birini tanlagan bo'larding va nega?

a) Brendning o'ziga xos, noldan chizilgan noyob dizaynli marketing sayti. b) Tez yasaladigan blog — brauzerning standart sarlavha/paragraf ko'rinishlari yetarli.

 **Yechim** 

a) **Reset** — noyob dizaynda har bir element to'liq nazoratda bo'lishi kerak; brauzer standartlari faqat xalaqit beradi, shuning uchun ularni noldan o'chirib, hammasini o'zing belgilaysan.

b) **Normalize** (yoki modern reset) — brauzerning mantiqiy standartlari (sarlavha kattaligi, paragraf bo'shliqlari) ayni muddao; faqat brauzerlar orasidagi farqlarni tekislash yetarli, hammasini qaytadan yozish ortiqcha mehnat.

Mashq 3 — Global scope to'qnashuvi

Quyidagi kodda nima xato? Sahifada ikki joyda `class="title"` bor — biri maqola, biri yon panel. Maqola sarlavhasi katta, yon panelniki esa kichik bo'lishi kerak edi, lekin ikkalasi ham katta chiqdi. Sababini va BEM yechimini ayt.

```
.title { font-size: 32px; }
```



Yechim



Sabab: `.title` — global, sodda nom. U sahifadagi *barcha* `class="title"` elementlarga tegadi, shu jumladan yon paneldagisiga ham. Bu — global scope muammosi.

BEM yechimi: har sarlavhani o'z blokiga bog'lab, noyob nom ber:

```
.maqola__sarlavha { font-size: 32px; }  
.yonpanel__sarlavha { font-size: 16px; }
```



Endi nomlar to'qnashmaydi — har biri faqat o'z blokiga tegadi.

Mashq 4 — DRY va design tokens

Quyidagi CSS'da takrorlanish bor. Custom properties (design tokens) bilan DRY qilib qayta yoz.

```
.tugma { background: #16a34a; padding: 12px; }  
.belgi { color: #16a34a; }  
.chiziq { border-bottom: 2px solid #16a34a; }  
.karta { padding: 12px; }
```



Yechim



```
:root {  
  --rang-yashil: #16a34a;  
  --bosh: 12px;  
}  
  
.tugma { background: var(--rang-yashil); padding: var(--bosh); }  
.belgi { color: var(--rang-yashil); }  
.chiziq { border-bottom: 2px solid var(--rang-yashil); }  
.karta { padding: var(--bosh); }
```



Endi yashil rangni yoki bo'shliqni o'zgartirish uchun `:root` dagi bitta qatorni tahrirlaysan — to'rt joy avtomatik yangilanadi.

Mashq 5 — Specificity urushini tinchitish

Quyidagi kod "specificity urushi" ga tushib qolgan. Uni **past, tekis** specificity'li, BEM uslubidagi koda qayta yoz.

```
.panel .ro'yxat li a { color: blue; }  
#asosiy .panel .ro'yxat li a { color: green; }  
#asosiy .panel .ro'yxat li a:hover { color: red !important; }
```





Yechim



```
.panel__havola { color: green; }  
.panel__havola:hover { color: red; }
```



Eski kodda har qoida avvalgisini bosish uchun aniqroq (va oxirida `!important`) qilingan edi. Bitta BEM klass (`(0,0,1,0)`) bilan zanjir yo'qoladi, `!important` ga ehtiyoj qolmaydi, `:hover` esa tabiiy ravishda g'olib chiqadi (bir xil specificity, keyin yozilgan).

Mashq 6 — Semantik vs utility tanlash

Quyidagi uch vaziyatning har biri uchun semantik (BEM) yoki utility-first'dan qaysi biri ko'proq mos kelishini va nega ekanini yoz:

a) Bir kunlik tezkor prototip — mijozga g'oyani ko'rsatish uchun. b) 30 ta sahifada qayta-qayta ishlatiladigan murakkab "mahsulot kartasi". c) Katta jamoa, ko'p odam bir vaqtda turli komponent yozadi, izchillik juda muhim.



Yechim



- a) **Utility-first** — tezlik hal qiluvchi; CSS fayl ochmasdan to'g'ridan HTML'da terib, g'oyani darhol ko'rsatasan.
- b) **Semantik (BEM)** — bir komponent 30 joyda takrorlanganda, bitta `.mahsulot-karta` klassi HTML'ni toza tutadi va o'zgartirishni bir joydan boshqaradi (utility'da klass ro'yxatini 30 marta nusxalashga to'g'ri kelardi).
- c) **Utility-first** (yoki token bilan cheklangan tizim) — chunki u izchillikni **majburlaydi**: hamma bir xil cheklangan tokenlardan foydalanadi, shuning uchun har xil odam yozsa ham natija bir xil maromli chiqadi. Semantik yondashuvda izchillik faqat intizomga bog'liq.

Mashq 7 — Fayllarni tashkil qilish

Senda bitta 3000 qatorli `style.css` bor: ichida reset, sarlavha/link asoslari, tugma, karta, menyu va rang o'zgaruvchilari aralash. Buni qanday fayllarga bo'lasan va `main.css` da qaysi tartibda ulaysan? Nega tartib muhim?



Yechim



```
styles/
├─ reset.css
├─ tokens.css      (rang/o'lcham o'zgaruvchilari)
├─ base.css        (sarlavha, link asoslari)
├─ components/
│   ├── tugma.css
│   ├── karta.css
│   └── menyu.css
└─ main.css
```

```
/* main.css */
@import "reset.css";
@import "tokens.css";
@import "base.css";
@import "components/tugma.css";
@import "components/karta.css";
@import "components/menyu.css";
```

Nega tartib muhim: teng specificity'da oxirgi yozilgan qoida g'olib chiqadi (10-bob). Shuning uchun `reset` eng birinchi (uni hamma narsa bossin), tokenlar undan keyin (komponentlar ulardan foydalanadi), komponentlar oxirida. Tartib buzilsa, reset komponentni bosib qo'yishi mumkin.

Mashq 8 — `@apply` ko'prigi

Loyihada utility-first ishlatyapsan, lekin "Yuborish" tugmasi 25 joyda bor va har safar `px-4 py-2 bg-blue-600 text-white rounded` ni nusxalash zerikarli. Bu takrorlanishni qanday yo'qotasan? (Tushuncha darajasida tushuntir.)



Yechim



Takrorlanadigan utility to'plamini bitta **semantik klass** ichiga `@apply` bilan yig'asan:

```
.tugma {
  @apply px-4 py-2 bg-blue-600 text-white rounded;
}
```

```
<button class="tugma">Yuborish</button>
```

Endi 25 joyda uzun ro'yxat o'rniga bitta `tugma` klassini ishlatasan — bu utility tezligini semantik klassning DRY afzalligi bilan birlashtiradi. Tugmani o'zgartirish kerak bo'lsa, faqat bitta `.tugma` qoidasini tahrirlaysan.

[← Oldingi: 19 — Zamonaviy CSS](#) · [🏠 README](#) · [Keyingi: 21 — Formalar, UI holatlari va kirish imkoniyati](#) [→](#)

21 — Formalar, UI holatlari va kirish imkoniyati

← Oldingi: 20 — CSS arxitekturasini va uslublarni · 🏠 README · Keyingi: 22 — Yakuniy loyiha — responsive landing page →

Bu bobda: formalarni chiroyli bezashni, interaktiv holatlarni (`:hover` , `:focus-visible` , `:checked` va boshqalar), dark mode'ni va kirish imkoniyatini (accessibility) o'rganamiz — sayt nafaqat ko'rkam, balki har bir foydalanuvchi uchun qulay bo'lishi uchun.

21.1 Nega aynan formalar va holatlar?

Sayt — bu shunchaki o'qiladigan matn emas. Foydalanuvchi unga **javob qaytaradi**: tugmani bosadi, maydonga yozadi, katakchani belgilaydi. Aynan shu nuqtada CSS faqat "bezak" bo'lishdan to'xtaydi va **muloqotning bir qismiga** aylanadi.

Tasavvur qil: lift tugmasini bosding, lekin u na yorishdi, na ovoz chiqardi. "Bosildimi yoki yo'qmi?" deb yana bosasan. Vizual javob bo'lmagani uchun ishonchsizlik tug'iladi. Sayt tugmalari ham xuddi shunday: foydalanuvchi sichqonni ustiga olganda rang o'zgarishi, bosilganda biroz "cho'kishi" kerak — bular **interaktiv holatlar** (interaktiv holat — element foydalanuvchi harakatiga qarab o'zgargan ko'rinishi).

Bu bobda uchta katta mavzuni bog'laymiz:

- **Forma elementlarini bezash** — input, textarea, select, checkbox, radio.
- **Holat selektorlari** — `:hover` , `:focus` , `:focus-visible` , `:active` , `:disabled` , `:checked` , `:valid` / `:invalid` .
- **Hammaga qulaylik** — dark mode, kontrast, fokus halqasi (focus ring), harakat sezgirligi (`prefers-reduced-motion`), teginish maydonlari.

✦ **Eslatma:** formalarning HTML tomonini biz 5-bobda o'rgangan edik (`<input>` , `<label>` , `<select>` va h.k.). Bu yerda esa ularning **ko'rinishi** va **xatti-harakati** bilan shug'ullanamiz. Agar HTML qismini unutgan bo'lsang, 5-bobni qayta ko'zdan kechir.

21.2 Brauzer standart ko'rinishini qaytadan boshlash

Birinchi savol: nega input'lar har brauzerda har xil ko'rinadi? Chunki brauzer ularga o'zining **standart stillari** (user-agent stylesheet) ni beradi. Chiroyli, izchil dizayn uchun avval shu standartni "nolga keltirib", keyin o'zimiznikini quramiz.

```
/* Forma elementlari shrift va o'lchamni o'zidan MEROS qilib olmaydi -📄
*/
/* shuni majburlaymiz, aks holda ular brauzer shriftida qoladi */
input,
textarea,
select,
button {
  font: inherit;      /* atrofdagi matn shriftini olsin */
  color: inherit;
}

/* Qutida o'lcham hisobini soddalashtirish (11-bobdagi box-sizing) */
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

⚠️ **Eng keng tarqalgan ajablanish:** `<input>` va `<button>` matn shriftini **avtomatik meros qilmaydi**. Brauzer ularga "Arial 13px" kabi tizimli shriftni beradi. `font: inherit` bo'lmasa, butun saytning shrifti chiroyli, lekin formalar begona ko'rinadi. Bu — birinchi yoziladigan qatorlardan biri.

💡 **Maslahat:** brauzer standartini butunlay tozalash uchun ko'pchilik [normalize.css](#) yoki [modern-normalize](#) deb nomlangan tayyor faylni ulaydi. Hozircha yuqoridagi bir necha qator boshlovchi loyiha uchun yetarli.

21.3 Input va textarea'ni uslublash

Endi maydonni o'zimizcha bezaymiz. Asosiy g'oya — bir nechta **umumiy stil** berib, keyin holatlarda uni o'zgartirish.

```
.maydon {
  width: 100%;
  padding: 10px 12px;
  font: inherit;
  color: #1e293b;
  background: #ffffff;
  border: 1px solid #94a3b8; /* tinch holatdagi chegara */
  border-radius: 8px;
  transition: border-color 0.15s, box-shadow 0.15s; /* silliq o'tish */
}
```

```
/* Placeholder (maydon ichidagi maslahat matni) rangi */
.maydon::placeholder {
  color: #94a3b8;
}
```

```
<label for="email">Email manzilingiz</label>
<input class="maydon" type="email" id="email"
placeholder="ism@misol.uz">
```

Natija: chetlari yumaloq, ichida bo'sh joy (padding) bor, oddiy va toza maydon.

`transition` tufayli keyingi bo'limda qo'shadigan holat o'zgarishlari keskin emas, **silliqlik** bo'ladi.

`textarea` uchun bitta foydali qo'shimcha bor:

```
textarea.maydon {
  min-height: 120px;
  resize: vertical; /* foydalanuvchi faqat balandlikni cho'zsin */
}
```

✦ **Nega `resize: vertical`?** Standartda `textarea` ni har ikki tomonga cho'zish mumkin. Lekin enini cho'zish layout'ni buzadi (maydon ota-elementdan oshib ketadi). `vertical` — eng xavfsiz va eng ko'p ishlatiladigan tanlov.

21.4 `:focus` va `:focus-visible` — eng muhim holat

Foydalanuvchi maydonga yozishni boshlaganda, **qaysi maydon faolligini** ko'rsatish kerak. Bu — `:focus` holati.

```
.maydon:focus {
  outline: none; /* brauzer standart konturini olib
tashlaymiz */
  border-color: #2563eb; /* chegara accent rangga o'tadi */
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.25); /* yumshoq "halqa" */
}
```

Bu yerda muhim nozik nuqta bor. `outline: none` deb yozdik — bu **xavfli** qadam, chunki outline (kontur) klaviatura bilan ishlovchilar uchun hayotiy. Shuning uchun **darhol o'rniga** `box-shadow` bilan o'z halqamizni qo'ydik. Hech qachon hech narsa qo'ymasdan outline'ni o'chirmang.

`:focus` va `:focus-visible` farqi

Mana eng zamonaviy va eng foydali holat. Muammo shunday: agar har **fokus**da halqa chiqarsang, sichqon bilan tugmani bosgan odam ham keraksiz halqa ko'radi (chunki bosish ham fokus beradi). Bu bezovta qiladi.

`:focus-visible` — brauzerga "halqani **faqat kerak bo'lganda**, ya'ni odam klaviatura (Tab) bilan o'tganda ko'rsat" deb aytadi. Sichqon bilan bosganda halqa chiqmaydi.



```
/* Sichqon bosishi: halqa YO'Q. Tab bilan o'tish: halqa BOR. */
.tugma:focus-visible {
  outline: 3px solid #2563eb;
  outline-offset: 2px;
}

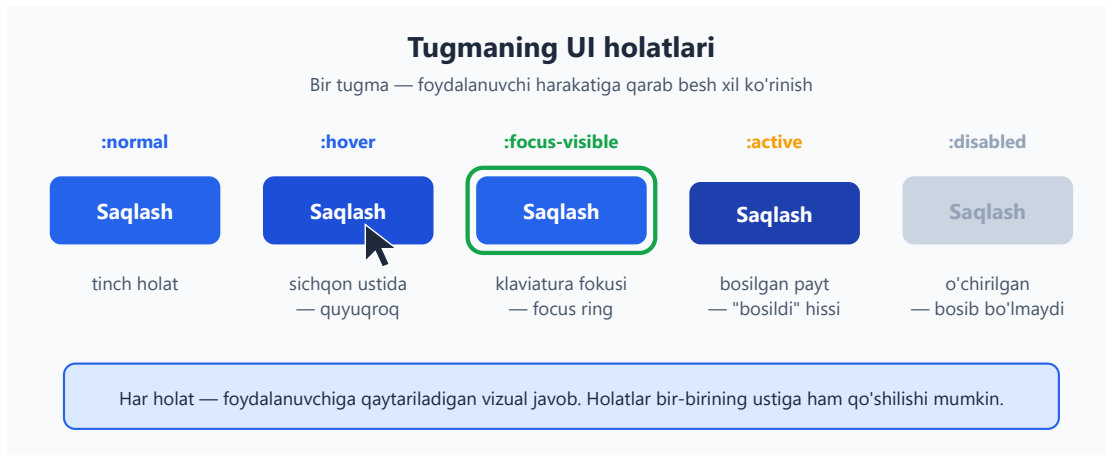
/* Eski brauzerlar :focus-visible ni bilmaydi - ularga oddiy :focus
bilan zaxira beramiz */
.tugma:focus:not(:focus-visible) {
  outline: none; /* yangi brauzerlarda sichqon fokusida halqani
o'chiramiz */
}
```

💡 **Qisqacha qoida:** odat sifatida `:focus-visible` ishlat, oddiy `:focus` emas. Shunda klaviatura foydalanuvchilari halqani ko'radi, sichqon foydalanuvchilari esa bezovta bo'lmaydi — ikkala tomon ham yutadi.

⚠️ **Nega bu muhim?** Klaviatura bilan ishlovchilar (sichqon ishlatolmaydigan, yoki shunchaki tezroq harakat qiluvchi odamlar) hozir **qayerda turganini** faqat focus ring orqali biladi. Uni o'chirib yuborish — ular uchun saytni "ko'r" qilish bilan teng.

21.5 Holat selektorlarining to'liq ro'yxati

Tugma misolida barcha asosiy holatlarni bir joyga yig'aylik. Quyidagi diagramma tugmaning beshta holatini ko'rsatadi:



```
.tugma {
  padding: 10px 20px;
  font: inherit;
  font-weight: 600;
  color: #ffffff;
  background: #2563eb;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: background 0.15s, transform 0.05s;
}

/* :hover - sichqon ustida (quyuqroq rang) */
.tugma:hover {
  background: #1d4ed8;
}

/* :focus-visible - klaviatura fokusi (halqa) */
.tugma:focus-visible {
  outline: 3px solid #2563eb;
  outline-offset: 2px;
}

/* :active - bosib turilgan payt ("cho'kish" hissi) */
.tugma:active {
  background: #1e40af;
  transform: translateY(1px);
}

/* :disabled - o'chirilgan (bosib bo'lmaydi) */
.tugma:disabled {
  background: #cbd5e1;
  color: #94a3b8;
  cursor: not-allowed;
}
```

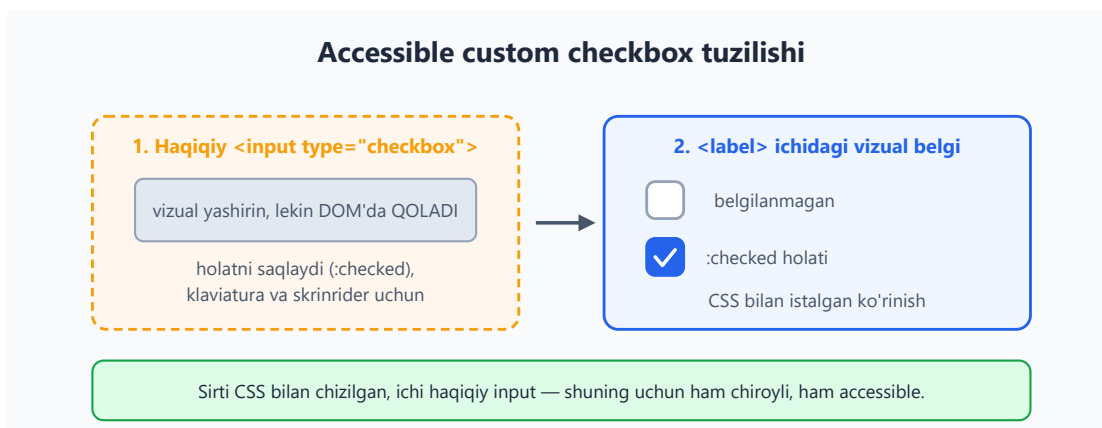
Har bir holatning **ma'nosi**:

Selektor	Qachon faollashadi	Nega kerak
<code>:hover</code>	sichqon element ustida	"bu narsa bosiladigan" signali
<code>:focus</code>	element fokusda (har qanday usulda)	qayerda turganimizni bildiradi
<code>:focus-visible</code>	fokus klaviaturadan kelganda	sichqonni bezovta qilmasdan halqa
<code>:active</code>	element bosib turilgan payt	"bosildi" degan tezkor javob
<code>:disabled</code>	element <code>disabled</code> atributiga ega	harakat hozir mumkin emas
<code>:checked</code>	checkbox/radio belgilangan	tanlangan holatni ko'rsatadi

✦ **Tartib muhim:** agar bir elementga `:hover`, `:focus`, `:active` ni birga yozsang, ularni shu tartibda yozish odat (ba'zan "LVHA/HFA qoidasi" deyiladi). Sababi — specificity teng bo'lganda, keyin yozilgani g'olib chiqadi (10-bobdagi cascade). Masalan `:active` ni `:hover` dan keyin yozsak, bosilgan payt to'g'ri rang ko'rinadi.

21.6 Custom checkbox va radio (accessible)

Standart checkbox kichkina va uni CSS bilan bezash qiyin. Lekin **uni butunlay almashtirib bo'lmaydi** — chunki haqiqiy `<input>` klaviatura va skrinrider (screen reader — ekran o'quvchi) uchun zarur. Yechim: haqiqiy input'ni saqlab, **vizual qismini o'zimiz chizamiz**.



Avval HTML — input `<label>` ichida turadi, shunday qilib label'ning istalgan joyiga bosish input'ni belgilaydi:

```
<label class="tanlov">
  <input type="checkbox" class="tanlov__input">
  <span class="tanlov__belgi" aria-hidden="true"></span>
  <span class="tanlov__matn">Shartlarga roziman</span>
</label>
```

Endi CSS. Asosiy hiyla — haqiqiy input'ni **ko'rinmas qilamiz, lekin o'chirmaymiz**:

```
.tanlov {
  display: inline-flex;
  align-items: center;
  gap: 10px;
  cursor: pointer;
}

/* Haqiqiy input: vizual yashirin, AMMO mavjud va fokus oladi */
.tanlov__input {
  position: absolute;
  opacity: 0;          /* ko'rinmas */
  width: 0;
  height: 0;
}

/* O'zimiz chizgan vizual katakcha */
.tanlov__belgi {
  width: 22px;
  height: 22px;
  border: 2px solid #94a3b8;
  border-radius: 6px;
  background: #ffffff;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  transition: background 0.15s, border-color 0.15s;
}

/* Input belgilangan (:checked) bo'lsa, YONIDAGI belgini bo'yaymiz */
.tanlov__input:checked + .tanlov__belgi {
  background: #2563eb;
  border-color: #2563eb;
}

/* Belgilanganda ichida "tasdiq" (✓) chizig'ini ko'rsatamiz */
.tanlov__input:checked + .tanlov__belgi::after {
  content: "✓";
  width: 6px;
  height: 11px;
  border: solid #ffffff;
  border-width: 0 2px 2px 0;
}
```

```

    transform: rotate(45deg);
    margin-top: -2px;
}

/* MUHIM: input klaviatura bilan fokus olganda, vizual belgida halqa
ko'rsatamiz */
.tanlov__input:focus-visible + .tanlov__belgi {
    outline: 3px solid #2563eb;
    outline-offset: 2px;
}

```

Bu yerda nima sodir bo'ldi, qadam-baqadam:

1. **opacity: 0 bilan yashirdik, display: none BILAN EMAS.** Bu juda muhim farq. `display: none` qilsak, input klaviaturadan fokus ololmaydi va skrinrider uni o'qiy olmaydi — accessibility buziladi. `opacity: 0` esa uni faqat **ko'zga** ko'rinmas qiladi, ammo u hamon "tirik".
2. **+ (qo'shni qardosh) kombinatori.** `.tanlov__input:checked + .tanlov__belgi` — "belgilangan input'ning to'g'ridan-to'g'ri **keyingi** elementi" degani (9-bobdagi kombinatorlarni eslang). Shuning uchun HTML'da `<span class="tanlov__belgi">` aynan input'dan **keyin** turishi shart.
3. **::after bilan ✓ belgisini chizdik** — to'rtburchakning faqat ikki tomonini chegara qilib, 45 daraja burib, "tasdiq" shaklini hosil qildik.
4. **:focus-visible halqasini vizual belgiga ko'chirdik** — chunki haqiqiy input ko'rinmas, halqa ko'rinadigan qismga tushishi kerak.

✳ `aria-hidden="true"` — vizual `<span>` ga qo'ydik, chunki u faqat bezak; skrinrider uni o'qimasligi kerak (haqiqiy input allaqachon "checkbox" deb o'qiladi).

💡 **Radio uchun ham xuddi shu naqsh** ishlaydi — faqat `border-radius` ni 50% qilib doira, va `::after` ichida nuqta chizasiz. `type="radio"` qilsangiz, brauzer "bittasini tanlash" mantig'ini o'zi boshqaradi.

21.7 :valid, :invalid va forma tekshiruvi

Brauzer forma maydonlarini **o'zi tekshiradi** (HTML `type`, `required`, `pattern` asosida). CSS bu natijaga `:valid` va `:invalid` orqali javob bera oladi.

```

<label for="pochta">Email</label>
<input type="email" id="pochta" class="maydon" required>

```

```

/* To'g'ri to'ldirilgan: yashil chegara */
.maydon:valid {

```

```
border-color: #16a34a;
}

/* Xato: qizil chegara */
.maydon:invalid {
border-color: #dc2626;
}
```

Bitta jiddiy muammo bor: sahifa ochilishi bilanoq bo'sh `required` maydon **darhol** qizil bo'lib qoladi — foydalanuvchi hali hech narsa qilmagan bo'lsa ham. Bu adolatsiz va bezovta qiladi.

Yechim — `:user-invalid` (zamonaviy brauzerlar) yoki `:invalid` ni `:focus` bilan birga ishlatish:

```
/* Faqat foydalanuvchi MULOQOT QILGANDAN keyin xatoni ko'rsat */
.maydon:user-invalid {
border-color: #dc2626;
background: #fef2f2;
}
```

⚠ **`:invalid` tuzog'i**: uni yolg'iz ishlatmang. `:user-invalid` (yoki ozgina JavaScript bilan "touched" sinfini qo'shish) foydalanuvchi maydonni tark etgandan keyingina xatoni ko'rsatadi — bu ancha xushmuomala tajriba. Eski brauzerlar uchun `:user-invalid` ni `:invalid` bilan birga, zaxira sifatida bering.

21.8 Karta va tugma uslublari

Endi o'rgangan holatlarni amaliy komponentlarga qo'llaymiz. **Karta** (card) — sayt dizaynidagi eng keng tarqalgan "quti".

```
<article class="karta">
  <h3 class="karta__sarlavha">Mahsulot nomi</h3>
  <p class="karta__matn">Qisqacha tavsif shu yerda turadi.</p>
  <button class="tugma">Batafsil</button>
</article>
```

```
.karta {
padding: 20px;
background: #ffffff;
border: 1px solid #e2e8f0;
border-radius: 12px;
box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
transition: box-shadow 0.2s, transform 0.2s;
}
```

```
/* Sichqon ustida karta biroz "ko'tariladi" */
.karta:hover {
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
  transform: translateY(-2px);
}
```

Natija: karta ustiga sichqon olib borilganda soya kattalashadi va karta yuqoriga ozgina suriladi — bu unga "yuzaga ko'tarilish" tuyg'usini beradi (13-bobdagi soyalar va 18-bobdagi transform'ni eslang).

💡 **Maslahat:** `transform: translateY(-2px)` `margin` o'zgartirishdan yaxshiroq, chunki transform layout'ni qayta hisoblashga majburlamaydi (atrofdagi elementlar joyidan siljimagaydi) — natijada animatsiya silliqroq bo'ladi.

21.9 Navbar — flexbox bilan navigatsiya

Deyarli har sayt yuqorida **navbar** (navigatsiya paneli) ga ega: chap tomonda logotip, o'ng tomonda havolalar. Buni flexbox bilan qilish juda oson (15-bob).

```
<nav class="navbar">
  <a class="navbar__logo" href="/">Mening Saytim</a>
  <ul class="navbar__menu">
    <li><a href="/">Bosh sahifa</a></li>
    <li><a href="/haqida">Biz haqimizda</a></li>
    <li><a href="/aloqa">Aloqa</a></li>
  </ul>
</nav>
```

```
.navbar {
  display: flex;
  align-items: center;          /* vertikal markazlash */
  justify-content: space-between; /* logo chapda, menyu o'ngda */
  padding: 12px 24px;
  background: #ffffff;
  border-bottom: 1px solid #e2e8f0;
}

.navbar__logo {
  font-weight: 700;
  font-size: 1.2rem;
  color: #1e293b;
  text-decoration: none;
}

.navbar__menu {
  display: flex;
  gap: 24px;          /* havolalar orasidagi masofa */
}
```

```

    list-style: none; /* nuqtalarni olib tashlaymiz */
    margin: 0;
    padding: 0;
}

.navbar__menu a {
    color: #475569;
    text-decoration: none;
    padding: 6px 4px;
    transition: color 0.15s;
}

.navbar__menu a:hover {
    color: #2563eb;
}

/* Klaviatura uchun fokus halqasi – UNUTMAYMIZ */
.navbar__menu a:focus-visible {
    outline: 2px solid #2563eb;
    outline-offset: 2px;
    border-radius: 4px;
}

```

`justify-content: space-between` ikkita bolani qutining ikki chetiga itaradi — logo chapga, menyu o'ngga. `gap` esa menyu havolalari orasiga teng masofa qo'yadi. Bu — navbar uchun klassik naqsh.

✦ **Mobil eslatma:** kichik ekranlarda menyuni "gamburger" tugmasiga yashirish kerak bo'ladi — bu odatda JavaScript talab qiladi. CSS tomondan esa `@media (max-width: 600px)` ichida `.navbar__menu` ni `display: none` qilib, JS uni ochadi. Bu kitobimiz doirasidan tashqarida, lekin tuzilma shunday.

21.10 Dark mode — `prefers-color-scheme` va custom property

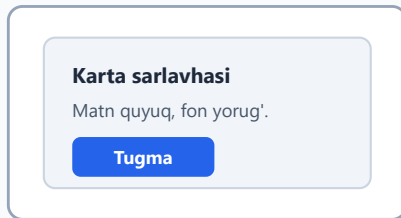
Ko'p foydalanuvchilar **qorong'i tema** (dark mode) ni afzal ko'radi — ayniqsa kechqurun. Operatsion tizim bu afzallikni biladi, va CSS uni `prefers-color-scheme` orqali eshita oladi.

Eng toza usul — barcha ranglarni **custom property** (CSS o'zgaruvchilari, 19-bob) qilib e'lon qilish, keyin temaga qarab faqat **qiymatlarni** almashtirish. Komponentlar o'zgaruvchini ishlatadi, qiymatdan bexabar.

Dark mode: bitta komponent, ikki tema

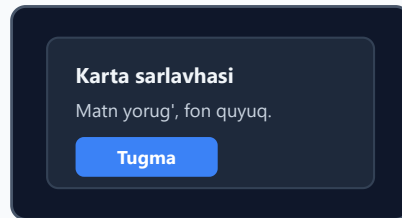
HTML bitta — faqat custom property qiymatlari almashadi

Yorug' tema (light)



tema

Qorong'i tema (dark)



Almashadigan custom property'lar

o'zgaruvchi	light	dark
--fon	#ffffff	#0f172a
--matn	#1e293b	#f1f5f9
--accent	#2563eb	#3b82f6

```
/* 1. Yorug' tema — standart qiymatlar */
:root {
  --fon: #ffffff;
  --yuza: #f1f5f9; /* karta foni */
  --matn: #1e293b;
  --matn-ikkilamchi: #475569;
  --chegara: #cbd5e1;
  --accent: #2563eb;
}

/* 2. Tizim qorong'i temani so'rasa — faqat QIYMATLARNI almashtiramiz */
@media (prefers-color-scheme: dark) {
  :root {
    --fon: #0f172a;
    --yuza: #1e293b;
    --matn: #f1f5f9;
    --matn-ikkilamchi: #cbd5e1;
    --chegara: #334155;
    --accent: #3b82f6; /* qorong'ida ozroq yorqin ko'k */
  }
}

/* 3. Komponentlar FAQAT o'zgaruvchilarni ishlatadi */
body {
  background: var(--fon);
  color: var(--matn);
}

.karta {
  background: var(--yuza);
  color: var(--matn);
  border: 1px solid var(--chegara);
}
```

```
.tugma {  
  background: var(--accent);  
  color: #ffffff;  
}
```

Bu yondashuvning go'zalligi: `.karta` va `.tugma` qoidalariga **umuman tegmaymiz**. Tema almashganda faqat `:root` dagi oltita qiymat o'zgaradi, butun sayt esa o'zi yangilanadi. Bu — DRY (Don't Repeat Yourself — "takrorlamaslik") tamoyilining ajoyib namunasi.

Foydalanuvchi qo'lda almashtirsa-chi?

Ko'pincha saytda "tema almashtirish" tugmasi bo'ladi. Buning uchun temani `:root` ga emas, balki `<html>` dagi atributga bog'laymiz, va JavaScript shu atributni almashtiradi:

```
/* Standart (yorug') */  
:root {  
  --fon: #ffffff;  
  --matn: #1e293b;  
}  
  
/* <html data-tema="dark"> bo'lsa */  
:root[data-tema="dark"] {  
  --fon: #0f172a;  
  --matn: #f1f5f9;  
}
```

```
// Tugma bosilganda atributni almashtiramiz  
const tugma = document.querySelector('.tema-tugma');  
tugma.addEventListener('click', () => {  
  const html = document.documentElement;  
  const hozir = html.getAttribute('data-tema');  
  html.setAttribute('data-tema', hozir === 'dark' ? 'light' : 'dark');  
});
```

💡 **Eng yaxshi amaliyot:** ikkalasini birlashtir — standart bo'yicha `prefers-color-scheme` ni hurmat qil (tizim afzalligi), lekin foydalanuvchi qo'lda almashtirsa, uning tanlovi ustun bo'lsin (`data-tema` atributi orqali). Bu eng xushmuomala xatti-harakat.

21.11 CSS'da kirish imkoniyati (accessibility)

Accessibility (qisqacha **a11y**) — saytni **har bir odam** ishlata olishini ta'minlash: ko'rish qobiliyati cheklangan, sichqon ishlatolmaydigan, harakatga sezgir foydalanuvchilar. CSS bu yerda muhim rol o'ynaydi.

Qoida 1: fokus halqasini hech qachon yo'q qilma

Eng ko'p uchraydigan a11y xatosi — bu:

```
/* JINOYAT — buni hech qachon qilma */
*:focus {
  outline: none;
}
```

Bu klaviatura foydalanuvchilarini "ko'r" qiladi: ular qayerda turganini bilmaydi. Agar standart halqa dizayningga to'g'ri kelmasa, uni **o'chirma — almashtir**:

```
:focus-visible {
  outline: 3px solid #2563eb;
  outline-offset: 2px;
}
```

Qoida 2: yetarli kontrast

Matn va fon orasida yetarli **kontrast** (qarama-qarshilik) bo'lishi shart, aks holda kam ko'radigan odamlar o'qiy olmaydi. WCAG standarti oddiy matn uchun kamida **4.5:1** nisbatni talab qiladi.

```
/* YOMON: och-kulrang matn oq fonda — kontrast ~2:1, o'qib bo'lmaydi */
.yomon { color: #cbd5e1; background: #ffffff; }

/* YAXSHI: quyuq matn — kontrast ~12:1 */
.yaxshi { color: #1e293b; background: #ffffff; }
```

💡 Kontrastni tekshirish uchun brauzer DevTools (rang tanlovchida nisbat ko'rsatiladi) yoki onlayn "contrast checker" dan foydalan.

Qoida 3: harakatga hurmat — prefers-reduced-motion

Ba'zi odamlarda animatsiya **bosh aylanishi yoki ko'ngil aynishi** keltirib chiqaradi (vestibulyar buzilish). Ular tizim sozlamalarida "harakatni kamaytirish" ni yoqadi. Biz buni hurmat qilamiz:

```
/* Standart: silliq animatsiyalar */
.karta {
  transition: transform 0.2s;
}

/* Foydalanuvchi harakatni kamaytirishni so'rasa — animatsiyani o'chiramiz */
@media (prefers-reduced-motion: reduce) {
```



```

*,
*::before,
*::after {
  animation-duration: 0.01ms !important;
  animation-iteration-count: 1 !important;
  transition-duration: 0.01ms !important;
  scroll-behavior: auto !important;
}
}

```

🚀 **Nega !important ?** Bu yerda u o'rinli — bu foydalanuvchining sog'lig'iga oid afzalligi va u **har qanday** komponent stilidan ustun turishi kerak. Bu — `!important` ning kam sonli to'g'ri ishlatilish holatlaridan biri.

Qoida 4: yetarli teginish maydoni

Telefonda barmoq sichqon kursoridan ancha katta. Agar tugma juda kichik bo'lsa, foydalanuvchi noto'g'ri joyni bosadi. Tavsiya — **kamida 44×44 piksel** teginish maydoni.

```

.tugma,
.navbar__menu a {
  min-height: 44px;           /* barmoq uchun yetarli balandlik */
  min-width: 44px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
}

```

⚠️ Vizual jihatdan kichik ikonka tugmasini ham `padding` yoki `min-height` bilan teginish maydonini kattalashtirish kerak — ko'rinish kichik, lekin bosiladigan joy katta bo'lsin.

🚀 **Bonus:** `:focus-within`. Forma ichidagi **biror** maydon fokusda bo'lganda butun konteynerga stil berishingiz mumkin: `.forma:focus-within { border-color: #2563eb; }` — bu foydalanuvchiga "siz hozir shu forma bilan ishlayapsiz" degan kuchli signal beradi.

21.12 Print uslublari — `@media print` (qisqacha)

Foydalanuvchi sahifani **bosib chiqarsa** (yoki PDF qilsa) ham, u chiroyli ko'rinishi kerak. Navbar, tugmalar, fonlar qog'ozda keraksiz. `@media print` shu uchun:

```

@media print {
  /* Qog'ozga kerak bo'lmagan narsalarni yashiramiz */
  .navbar,

```

```

.tugma,
.reklama,
footer {
    display: none;
}

/* Qora matn, oq fon – siyohni tejaymiz */
body {
    color: #000;
    background: #fff;
}

/* Havola manzilini matn yonida ko'rsatamiz (qog'ozda bosib
bo'lmaydi-ku) */
a[href]::after {
    content: " (" attr(href) )";
    font-size: 0.85em;
    color: #555;
}
}

```

💡 `a[href]::after { content: " (" attr(href) )"; }` — bu chiroyli hiyla: chop etilgan qog'ozda havolaning **manzilini** matn yonida ko'rsatadi, chunki qog'ozda havolani bosib bo'lmaydi. `attr(href)` atribut qiymatini o'qiydi.

📌 Print stillarni o'zingiz ko'rish uchun: brauzerda `Ctrl+P` (chop etish oynasi) — u jonli oldindan ko'rsatma beradi.

Mashqlar

Mashqlarni bajarib ko'r, keyin yechimni och. Eng ko'p narsa o'zing urinib ko'rganingda yodda qoladi.

1-mashq. Bitta `<input type="text">` yarat va unga shunday stil ber: tinch holatda kulrang chegara, fokusda ko'k chegara va yumshoq ko'k "halqa" (`box-shadow`). `outline` ni o'chirsang, albatta o'rniga ko'rinadigan narsa qo'y.



Yechim



```
.maydon {
  padding: 10px 12px;
  border: 1px solid #94a3b8;
  border-radius: 8px;
  font: inherit;
  transition: border-color 0.15s, box-shadow 0.15s;
}
.maydon:focus {
  outline: none;
  border-color: #2563eb;
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.25);
}
```

`transition` tufayli fokusga o'tish silliq bo'ladi. `outline: none` ni qo'ydik, lekin darhol `box-shadow` bilan ko'rinadigan halqa berdik — fokus hech qachon "ko'rinmas" qolmaydi.

2-mashq. Tugma yarat va uning **to'rtta holatini** bezab chiq: normal (ko'k), `:hover` (quyuqroq), `:active` (yana quyuqroq + 1px pastga siljish), `:disabled` (kulrang, `cursor: not-allowed`).



Yechim



```
.tugma {
  padding: 10px 20px;
  color: #fff;
  background: #2563eb;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: background 0.15s, transform 0.05s;
}
.tugma:hover { background: #1d4ed8; }
.tugma:active { background: #1e40af; transform: translateY(1px); }
.tugma:disabled {
  background: #cbd5e1;
  color: #94a3b8;
  cursor: not-allowed;
}
```

`transform: translateY(1px)` bosilgan paytda tugmaning "cho'kkani" hissini beradi.

3-mashq. `:focus` va `:focus-visible` farqini o'z so'zing bilan tushuntir. Qaysi birini odatda ishlatish kerak va nega?



Yechim



`:focus` element **har qanday** usulda (sichqon bosishi yoki klaviatura) fokus olganda faollashadi. `:focus-visible` esa fokus **klaviatura** orqali kelganda (yoki brauzer "halqa kerak" deb hisoblaganda) faollashadi — sichqon bosishida halqa chiqmaydi.

Odatda `:focus-visible` ishlatiladi: u klaviatura foydalanuvchilariga zarur halqani beradi, lekin sichqon foydalanuvchilarini keraksiz halqa bilan bezovta qilmaydi. Ikkala tomon ham yutadi.

4-mashq. Accessible custom checkbox yarat: haqiqiy input ko'rinmas bo'lsin (lekin o'chmasin!), yonida o'zing chizgan katakcha turadi, `:checked` da u ko'k bo'lib ichida belgi paydo bo'ladi.



Yechim



```
<label class="tanlov">
  <input type="checkbox" class="tanlov__input">
  <span class="tanlov__belgi" aria-hidden="true"></span>
  <span>Roziman</span>
</label>
```

```
.tanlov { display: inline-flex; align-items: center; gap: 10px; cursor:
pointer; }
.tanlov__input { position: absolute; opacity: 0; width: 0; height: 0; }
.tanlov__belgi {
  width: 22px; height: 22px;
  border: 2px solid #94a3b8; border-radius: 6px;
  display: inline-flex; align-items: center; justify-content: center;
}
.tanlov__input:checked + .tanlov__belgi {
  background: #2563eb; border-color: #2563eb;
}
.tanlov__input:checked + .tanlov__belgi::after {
  content: ""; width: 6px; height: 11px;
  border: solid #fff; border-width: 0 2px 2px 0;
  transform: rotate(45deg); margin-top: -2px;
}
.tanlov__input:focus-visible + .tanlov__belgi {
  outline: 3px solid #2563eb; outline-offset: 2px;
}
```

Asosiy nuqta: `opacity: 0` (`display: none` EMAS) — shunda input klaviatura va skrinrider uchun "tirik" qoladi. + kombinatori belgilangan input'ning keyingi `<span>` ini bo'yaydi.

5-mashq. `:root` da uchta custom property e'lon qil (`--fon`, `--matn`, `--accent`) va `prefers-color-scheme: dark` ichida ularning qiymatlarini almashtir. `body` va bitta `.tugma` shu o'zgaruvchilarni ishlatsin. Sinab ko'r: tizim temasini almashtirganda sayt o'zi o'zgaradimi?



Yechim



```
:root {
  --fon: #ffffff;
  --matn: #1e293b;
  --accent: #2563eb;
}
@media (prefers-color-scheme: dark) {
  :root {
    --fon: #0f172a;
    --matn: #f1f5f9;
    --accent: #3b82f6;
  }
}
body { background: var(--fon); color: var(--matn); }
.tugma { background: var(--accent); color: #fff; }
```

E'tibor ber: `body` va `.tugma` qoidalari **bir marta** yozildi va o'zgartirilmadi. Tema almashganda faqat `:root` dagi qiymatlar yangilanadi — bu DRY tamoyilining go'zal namunasi.

6-mashq. Foydalanuvchi sog'lig'i uchun: `@media (prefers-reduced-motion: reduce)` blokini yoz, u barcha `transition` va `animation` ni deyarli nolga keltirsin. Nega bu yerda `!important` ishlatish o'rinli?



Yechim



```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

`!important` bu yerda o'rinli, chunki bu foydalanuvchining **sog'ligiga oid afzalligi** (animatsiya bosh aylanishi keltirib chiqarishi mumkin) va u har qanday komponent stilidan ustun turishi shart. Bu — `!important` ning kam sonli to'g'ri qo'llanish holatlaridan biri.

7-mashq. Flexbox bilan navbar yasab chiq: chap tomonda logo, o'ng tomonda uch havola. Havolalar `:hover` da rang o'zgartirsin va `:focus-visible` da ko'rinadigan halqa olsun. Teginish maydoni kamida 44px balandlikda bo'lsin.



Yechim



```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 24px;
}
.navbar__menu {
  display: flex; gap: 24px;
  list-style: none; margin: 0; padding: 0;
}
.navbar__menu a {
  display: inline-flex; align-items: center;
  min-height: 44px; /* teginish maydoni */
  color: #475569; text-decoration: none;
  transition: color 0.15s;
}
.navbar__menu a:hover { color: #2563eb; }
.navbar__menu a:focus-visible {
  outline: 2px solid #2563eb;
  outline-offset: 2px;
  border-radius: 4px;
}
```

`justify-content: space-between` logoni chapga, menyuni o'ngga itaradi; `gap` havolalar orasini ajratadi; `min-height: 44px` esa barmoq uchun yetarli teginish maydonini ta'minlaydi.

8-mashq. (Qiyinroq) Forma maydonida `:user-invalid` ni ishlatib, xato faqat foydalanuvchi maydon bilan **muloqot qilgandan keyin** ko'rinsin. Nega oddiy `:invalid` ni yolg'iz ishlatish yomon tajriba beradi?



Yechim



```
<input type="email" class="maydon" required>
```

```
.maydon:user-invalid {
  border-color: #dc2626;
  background: #fef2f2;
}
```

Oddiy `:invalid` ni yolg'iz ishlatsak, sahifa ochilishi bilanoq bo'sh `required` maydon **darhol qizil** bo'ladi — foydalanuvchi hali hech narsa yozmagan bo'lsa ham. Bu adolatsiz va bezovta qiladi. `:user-invalid` esa xatoni faqat foydalanuvchi maydonni to'ldirib, undan chiqqandan **keyin** ko'rsatadi — bu ancha xushmuomala tajriba. (Eski brauzerlar uchun `:user-invalid` ni `:invalid` bilan zaxira sifatida birga berish mumkin.)

[← Oldingi: 20 — CSS arxitekturasi va uslublar](#) · [🏠 README](#) · [Keyingi: 22 — Yakuniy loyiha — responsive landing page →](#)

22 — Yakuniy loyiha — responsive landing page

← Oldingi: 21 — Formalar, UI holatlari va kirish imkoniyati · 🏠 README

Bu bobda: shu paytgacha o'rgangan hamma narsani — semantik HTML, custom properties bilan dizayn tizimi, Grid va Flexbox, responsive, dark mode, hover/focus va animatsiya — birlashtirib, noldan to'liq ishlaydigan, ko'chirib qo'yib ishlatsa bo'ladigan responsive landing page quramiz.

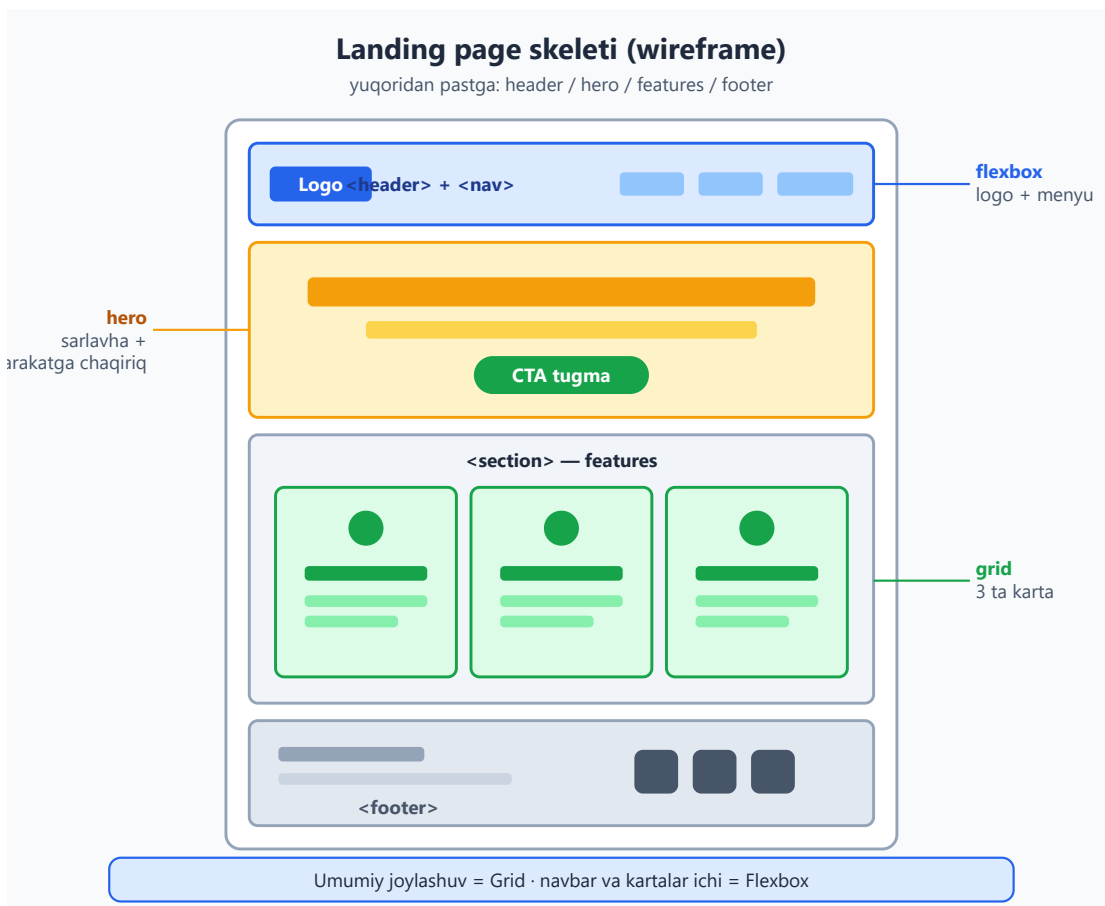
22.1 Loyihaga kirish — nima quramiz va nega

Tabriklaymiz — sen bu yerga yetib kelding! Box model, Flexbox, Grid, responsive, custom properties, animatsiya... ularning har biri alohida bo'lakcha edi. Bu bobda biz ularni **bitta haqiqiy mahsulot** ichida bog'laymiz.

Quradigan narsamiz — **landing page** (qo'nish sahifasi). Bu mahsulot yoki xizmatni reklama qiladigan bitta sahifa: yuqorida diqqatni tortadigan sarlavha, ostida afzalliklar, pastida aloqa. Real hayotda startaplar, kurslar, ilovalar aynan shunaqa sahifadan boshlanadi.

Nega aynan landing page? Chunki u **kichik, lekin to'liq**: bitta sahifaga butun frontend skillni sig'dirib bo'ladi. Katta sayt yasashdan oldin bittasini mukammal qilishni o'rganish — eng to'g'ri yo'l.

Mana quradigan sahifamiz skeletining ko'rinishi (blueprint):



✦ Bo'limlar nima vazifa bajaradi:

Bo'lim	Vazifasi
header + nav	logo va navigatsiya menyusi (yuqori panel)
hero	birinchi ekran: katta sarlavha + harakatga chaqiriq tugmasi
features	mahsulotning 3 ta afzalligi — kartalar tarzida
footer	mualliflik, havolalar, ijtimoiy tarmoqlar

💡 **Ishlash tartibimiz** quyidagicha bo'ladi — har bir qadam alohida bo'limda:

1. Avval **semantik HTML skelet** (mazmun — dizaynsiz).
2. Keyin **dizayn tizimi** (custom properties — design tokens).
3. Keyin **layout** (Grid bilan umumiy, Flexbox bilan ichki).
4. Keyin **responsive** (mobile-first, media query, `clamp()`).
5. Keyin **dark mode**, **interaktivlik** va **animatsiya**.

6. Oxirida **accessibility tekshiruvi** va to'liq birlashtirilgan kod.

⚠ **Eng keng tarqalgan boshlovchi xatosi** — darrov CSS yozishga tushish. Avval HTML to'liq tayyor bo'lsin, mazmun o'qilsin. Stilsiz ham sahifa mantiqan to'g'ri ko'rinsa — poydevor mustahkam degani.

22.2 1-qadam: Semantik HTML skelet

Boshlaymiz. Hech qanday stil yo'q — faqat **mazmun va struktura**. Maqsad: sahifani CSS'siz ochganda ham u mantiqiy, o'qiladigan hujjat bo'lsin.

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FokusApp - diqqatingizni boshqaring</title>
</head>
<body>
  <!-- Yuqori panel: logo + navigatsiya -->
  <header class="site-header">
    <a href="#" class="logo">FokusApp</a>
    <nav aria-label="Asosiy menyu">
      <ul class="nav-list">
        <li><a href="#features">Imkoniyatlar</a></li>
        <li><a href="#narx">Narx</a></li>
        <li><a href="#aloqa">Aloqa</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <!-- Hero: birinchi ekran -->
    <section class="hero">
      <h1>Diqqatingizni qaytaring</h1>
      <p>FokusApp chalg'ituvchilarni bloklaydi va ishingizni kuzatadi - shunda muhim narsaga e'tibor qarata olasiz.</p>
      <a href="#" class="btn btn-primary">Bepul boshlash</a>
    </section>

    <!-- Imkoniyatlar -->
    <section id="features" class="features">
      <h2>Nega FokusApp?</h2>
      <div class="card-grid">
        <article class="card">
          <h3>Chalg'ituvchilarni bloklaydi</h3>
          <p>Ish vaqtida ijtimoiy tarmoqlar va xabarlar avtomatik o'chiradi.</p>
```

```

</article>
<article class="card">
  <h3>Vaqtini kuzatadi</h3>
  <p>Qaysi vazifaga qancha vaqt ketganini aniq ko'rsatadi.</p>
</article>
<article class="card">
  <h3>Hisobotlar beradi</h3>
  <p>Har hafta unumdorligingiz haqida sodda hisobot yuboradi.
</p>
</article>
</div>
</section>
</main>

<!-- Pastki qism -->
<footer class="site-footer">
  <p>&copy; 2026 FokusApp. Barcha huquqlar himoyalangan.</p>
  <nav aria-label="Footer havolalari">
    <a href="#">Maxfiylik</a>
    <a href="#">Shartlar</a>
  </nav>
</footer>
</body>
</html>

```

Nima uchun aynan shunday yozdik — har biri sababli:

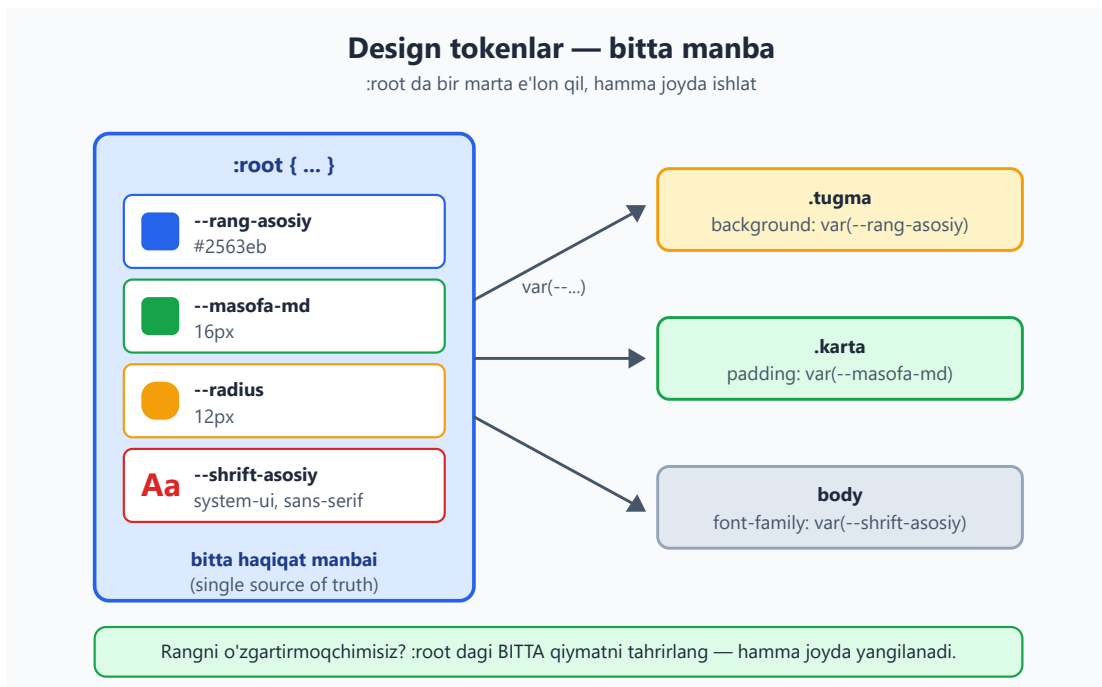
- `<header>`, `<main>`, `<section>`, `<footer>` — bular **semantik landmarklar** (mo'ljall nuqtalari). Skrinrider va Google sahifa tuzilishini shular orqali tushunadi. `<div>` bilan yozsak ham vizual bir xil bo'lardi, lekin **ma'nosi yo'qoladi**.
- `<nav aria-label="...">` — sahifada ikkita `nav` bor (yuqori va footer), shuning uchun har biriga nom berdik. Aks holda skrinriderdan foydalanuvchi "qaysi navigatsiya?" deb adashadi.
- `<h1>` bittagina — sahifaning bosh sarlavhasi. `<h2>` bo'lim sarlavhasi, `<h3>` karta sarlavhasi. **Ierarxiyani buzib o'tkazib yubormaslik** kerak (h1 dan to'g'ridan-to'g'ri h3 ga sakramaymiz).
- Kartalar `<article>` — har biri o'zicha mustaqil ma'noli bo'lak. `<ul>` ichida `<li>` — menyu bu ro'yxat, shuning uchun ro'yxat tegi.

⚠ Hozir sahifa **xunuk** ko'rinadi — qora matn, oq fon, hech qanday joylashuv. Bu **normal**. Stilsiz holatda ham mazmun to'liq o'qilsa, biz to'g'ri yo'ldamiz.

22.3 2-qadam: Dizayn tizimi — custom properties (design tokens)

Endi stil. Lekin to'g'ridan-to'g'ri rang yozishni boshlamaymiz. Avval **dizayn tizimi** quramiz — bir joyda barcha ranglar, masofalar, shriftlarni e'lon qilamiz. Bularni **design token** deyiladi.

Nega? Tasavvur qil: sahifada ko'k rang 15 joyda ishlatilgan. Mijoz "ko'kni binafshaga o'zgartiring" desa — 15 joyni qidirib topishing kerak. Token bilan esa **bitta** qatorni o'zgartirasan, hamma joy yangilanadi.



`:root` — bu butun hujjatning ildizi (`<html>` elementi). U yerda e'lon qilingan custom property **hamma joyda** ko'rinadi:

```
:root {
  /* === RANGLAR === */
  --rang-asosiy: #2563eb; /* asosiy ko'k (brand color) */
  --rang-asosiy-quyuq: #1d4ed8; /* hover holati uchun quyuqroq */
  --rang-matn: #1e293b; /* asosiy matn */
  --rang-matn-och: #475569; /* ikkilamchi matn (xira) */
  --rang-fon: #ffffff; /* sahifa foni */
  --rang-fon-karta: #f8fafc; /* karta/panel foni */
  --rang-chegara: #e2e8f0; /* chiziq va chegaralar */

  /* === MASOFA (spacing) - 4px shkalasi === */
  --masofa-xs: 4px;
  --masofa-sm: 8px;
  --masofa-md: 16px;
  --masofa-lg: 32px;
  --masofa-xl: 64px;

  /* === TIPOGRAFIYA === */
  --shrift-asosiy: "Segoe UI", system-ui, -apple-system, sans-serif;
```

```

/* === BOSHQA === */
--radius: 12px; /* burchak yumaloqligi */
--soya: 0 4px 12px rgba(0, 0, 0, 0.08);
--kenglik-maks: 1100px; /* kontent maksimal eni */
}

```

Nega aynan shunday tuzdik:

- **Rang nomlarini vazifasi bilan atadik** (`--rang-asosiy`), tusi bilan emas (`--kok` emas). Sababi: ertaga ko'kni yashilga o'zgartirsang, `--kok: green` degan ahmoqona nom qolib ketmaydi. Nom **rolni** bildiradi, rangni emas.
- **Masofa shkalasi** `4px` ning karralaridan iborat (4, 8, 16, 32, 64). Bu — tartib. "Ba'zan 7px, ba'zan 13px, ba'zan 22px" degan tasodifiy qiymatlar layoutni notekis qiladi. Shkala bilan hamma masofa o'zaro mutanosib bo'ladi.
- **system-ui** — foydalanuvchi qurilmasining tabiiy shriftini ishlatadi (Windows'da Segoe UI, Mac'da San Francisco). Tashqi shrift yuklamaymiz — sahifa tezroq ochiladi.

✦ **Custom property'ni ishlatish** `var()` orqali bo'ladi: `color: var(--rang-matn);`. Agar token topilmasa, ikkinchi argument zaxira bo'ladi: `var(--rang-yoq, #fff)`.

Endi global "reset" va asosiy stillarni qo'shamiz:

```

/* Hamma elementga box-sizing: border-box — padding eni ichida
hisoblanadi */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--shrift-asosiy);
  color: var(--rang-matn);
  background: var(--rang-fon);
  line-height: 1.6; /* qatorlar orasini biroz keng — o'qish
qulay */
}

img {
  max-width: 100%; /* rasm konteyneridan toshib chiqmasin */
  display: block;
}

a {
  color: inherit; /* havola matn rangini meros oladi */
  text-decoration: none;
}

```

💡 `box-sizing: border-box` — har responsive loyihaning birinchi qatori. Usiz `width: 100%` ga `padding` qo'shilib, element konteynerdan toshib chiqadi. `border-box` bilan `padding` eni **ichida** hisoblanadi.

22.4 3-qadam: Umumiy layout — Grid

Endi sahifa **joylashuvi**. Ikki xil layout vositamiz bor va ularning roli aniq bo'lsin:

- **Grid** — ikki o'lchamli (qator + ustun) umumiy joylashuv uchun. Bizda: features bo'limidagi **kartalar to'ri**.
- **Flexbox** — bir o'lchamli (bir qator yoki bir ustun) uchun. Bizda: **navbar** (logo bir chetda, menyu boshqa chetda) va karta ichidagi elementlar.

✦ **Sodda qoida:** "bir qatorga tizish" — Flexbox. "To'r/panjara qilish" — Grid.

Avval butun sahifani markazga olamiz va bo'limlarga nafas beramiz. Buning uchun takrorlanadigan `.container` klassi yaratamiz:

```
/* Kontentni markazlab, ikki yondan padding beruvchi o'rovchi */
.container {
  max-width: var(--kenglik-maks);
  margin-inline: auto;          /* chap+o'ng margin avtomatik =
markaz */
  padding-inline: var(--masofa-md);
}

/* Har bo'lim yuqori-pastdan nafas olsin */
section {
  padding-block: var(--masofa-xl);
}
```

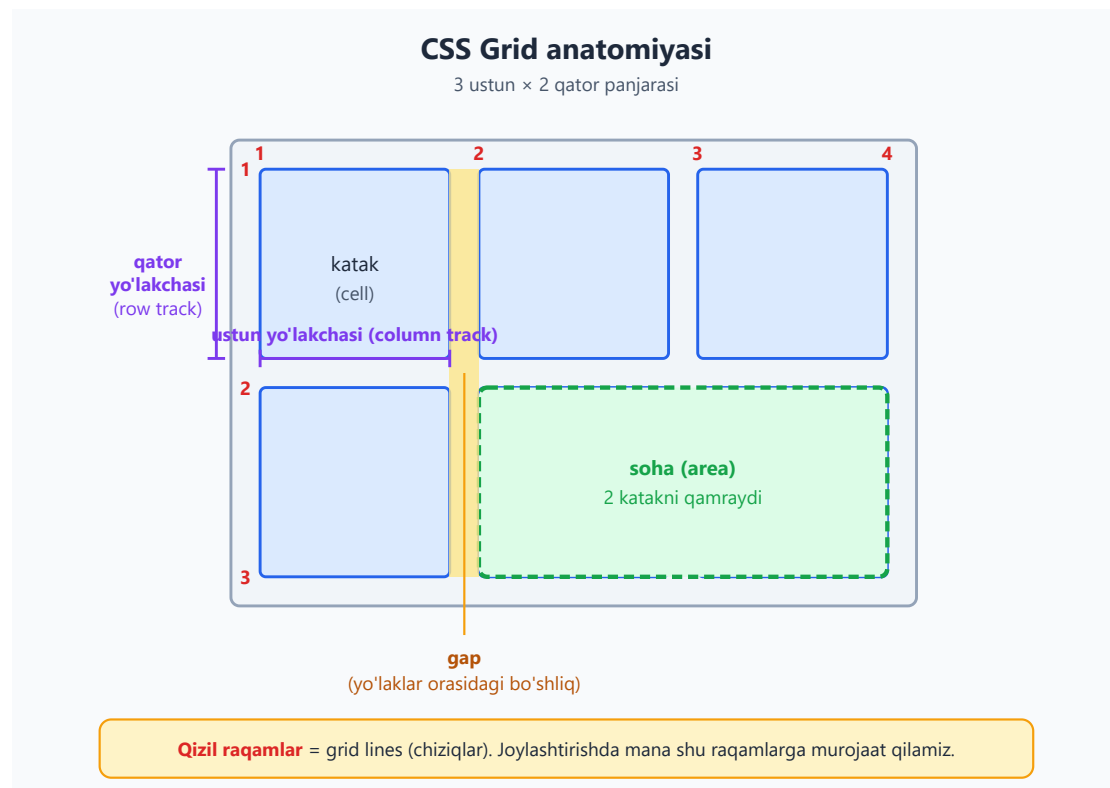
`margin-inline: auto` — `margin-left: auto; margin-right: auto` ning qisqasi. Konteyner `max-width` dan keng ekranda o'rta tushadi.

Endi **features kartalarini Grid bilan to'rga** joylaymiz:

```
.card-grid {
  display: grid;
  /* auto-fit: kartalar joyga qarab o'zi to'rga taqsimlanadi.
  minmax(260px, 1fr): har karta kamida 260px, qolgan joyni teng
  bo'lishadi */
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: var(--masofa-lg);
}
```

Nega auto-fit + minmax? Bu — eng kuchli responsive grid retsepti. Ekran keng bo'lsa, kartalar yonma-yon (260px sig'gancha) tiziladi; ekran torayganda — o'zi avtomatik bir ustunga tushadi. Bunga **media query ham kerak emas** — Grid o'zi hisoblaydi!

Grid'ning ichki tuzilishini eslab olaylik:



💡 **gap** — bu Grid (va Flexbox) ning kataklar orasidagi bo'shlig'i. Eski usuldagi **margin** o'yinlaridan ancha toza: chetki kataklarga ortiqcha margin qo'shilmaydi.

22.5 4-qadam: Navbar va kartalar — Flexbox

Navbar — klassik Flexbox vazifasi: **logo bir chetda, menyu boshqa chetda**.

```
.site-header {  
  display: flex;  
  justify-content: space-between; /* ikki uchini ikki chetga itaradi */  
  align-items: center;           /* vertikal markazlaydi */  
  padding: var(--masofa-md);  
  max-width: var(--kenglik-maks);  
  margin-inline: auto;  
  border-bottom: 1px solid var(--rang-chegara);  
}
```

```

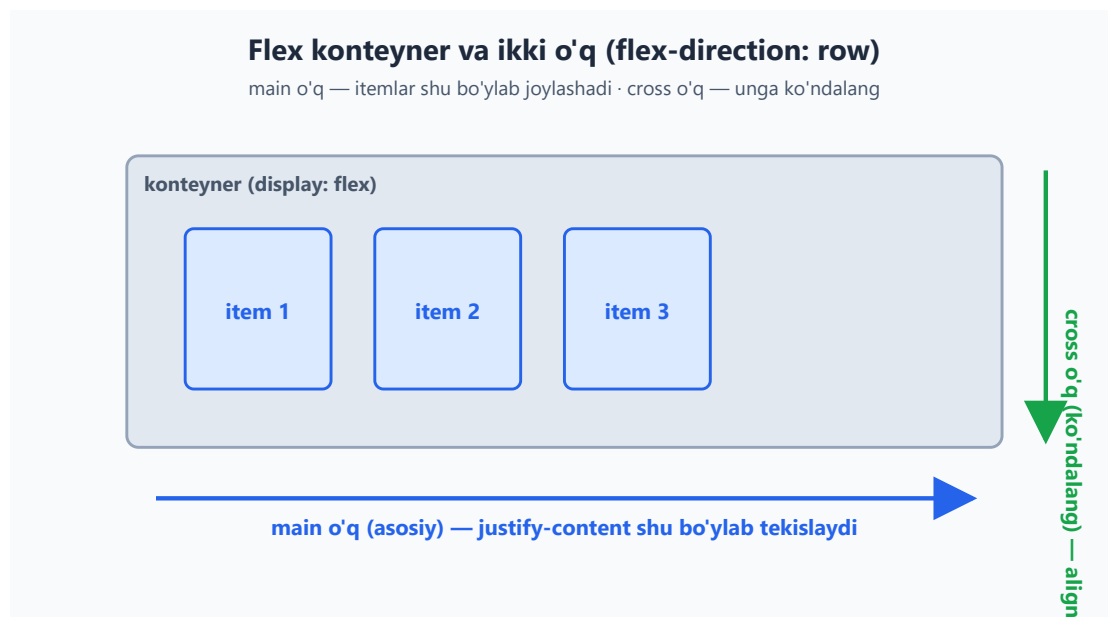
.logo {
  font-size: 1.4rem;
  font-weight: 700;
  color: var(--rang-asosiy);
}

.nav-list {
  display: flex;
  gap: var(--masofa-lg);
  list-style: none;      /* ro'yxat nuqtalarini olib tashlash */
}

```

justify-content: space-between nega kerak? Flexbox'da bu asosiy o'q (main axis) bo'ylab elementlarni taqsimlaydi: birinchisi boshda, oxirgisi oxirda, qolgan bo'shliq orasiga tarqaladi. Bizda ikki element bor — logo va nav — shuning uchun ular ikki chetga itariladi.

Flexbox'ning ikki o'qini eslab olaylik:



Endi **kartaning ichini** Flexbox bilan tartiblaymiz — sarlavha, matn tik joylashsin va kartalar bir xil balandlikda turishi uchun:

```

.card {
  display: flex;
  flex-direction: column;      /* ichidagilar tik (ustun) tartibda */
  gap: var(--masofa-sm);
  padding: var(--masofa-lg);
  background: var(--rang-fon-karta);
  border: 1px solid var(--rang-chegara);
  border-radius: var(--radius);
}

```



```
.card h3 {
  color: var(--rang-asosiy);
  font-size: 1.25rem;
}

.card p {
  color: var(--rang-matn-och);
}
```

Va hero hamda tugmalar uchun stil:

```
.hero {
  text-align: center;
  max-width: 680px;
  margin-inline: auto;
}

.hero h1 {
  font-size: 2.5rem;
  margin-bottom: var(--masofa-md);
  line-height: 1.2;
}

.hero p {
  color: var(--rang-matn-och);
  font-size: 1.15rem;
  margin-bottom: var(--masofa-lg);
}

/* Tugma — qayta ishlatiladigan komponent */
.btn {
  display: inline-block;
  padding: var(--masofa-sm) var(--masofa-lg);
  border-radius: var(--radius);
  font-weight: 600;
  cursor: pointer;
}

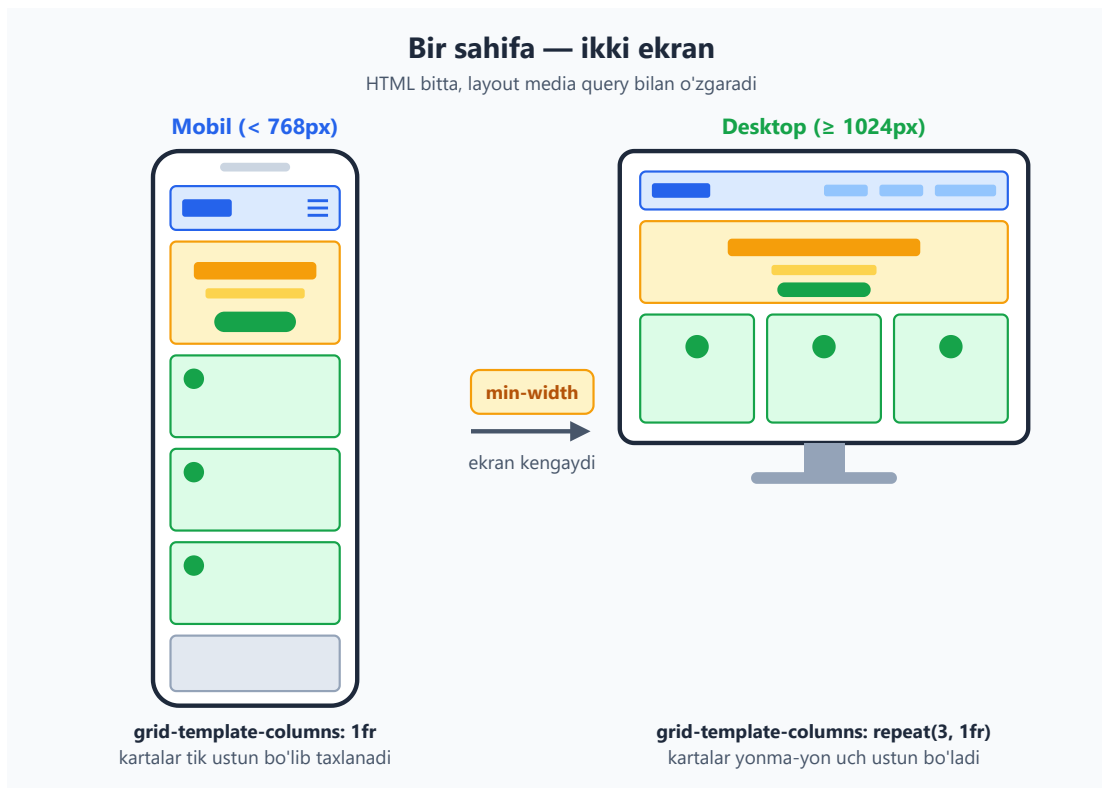
.btn-primary {
  background: var(--rang-asosiy);
  color: #ffffff;
}
```

Natija: navbar yuqorida ikki chetga taqsimlandi, hero markazda, kartalar bir xil balandlikdagi to'ra bo'lib joylashdi. Hammasi token qiymatlaridan — hech qanday "sehrli raqam" yo'q.

22.6 5-qadam: Responsive — mobile-first va `clamp()`

Hozirgi kod desktopda yaxshi ko'rinadi, lekin telefonni tekshiramiz. Asosiy muammo: hero sarlavhasi `2.5rem` — telefonda juda katta, ekrandan toshib chiqishi mumkin. Navbar ham torayganda siqilib qoladi.

Bir xil sahifa ikki ekranda qanday o'zgarishi kerakligi:



Biz **mobile-first** ishlaymiz: asosiy stil eng sodda (mobil) holatga, keyin `min-width` bilan kattaroq ekranga murakkablik **qo'shamiz**.

Birinchi muammo — navbar. Tor ekranda logo va menyuni tik (ustun) qilamiz:

```
/* ASOS (mobil): navbar tik joylashsin */
.site-header {
  flex-direction: column;
  gap: var(--masofa-sm);
}

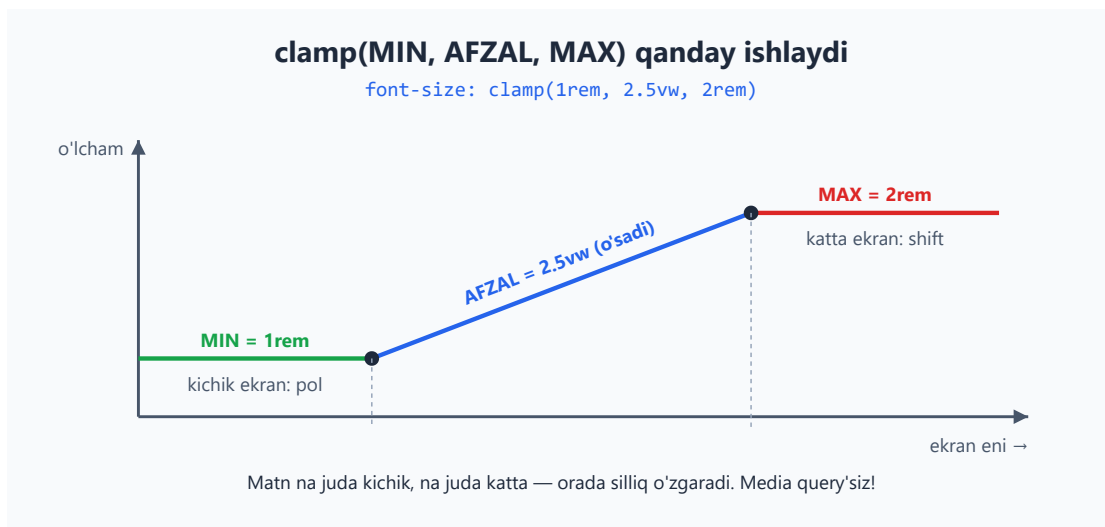
/* Planshet va undan keng: yana yonma-yon */
@media (min-width: 768px) {
  .site-header {
    flex-direction: row;
  }
}
```

Ikkinchi muammo — sarlavha o'lchami. Mana bu yerda `clamp()` ajoyib yechim beradi:

```
.hero h1 {
  /* clamp(MIN, ISTALGAN, MAKS):
    - hech qachon 1.8rem dan kichik bo'lmaydi (telefonda)
    - hech qachon 3.5rem dan katta bo'lmaydi (katta monitor)
    - oralig'da ekran eniga (5vw) qarab silliq o'sadi */
  font-size: clamp(1.8rem, 5vw, 3.5rem);
}
```

Nega `clamp()` shunchalik kuchli? U bitta qatorda **media query'lar zanjirini** almashtiradi. Oldin "640px dan kichikda 1.8rem, 1024px dan kattada 3.5rem..." deb bir nechta breakpoint yozardik. `clamp()` esa o'zi silliq oraligni hisoblaydi — sakrash yo'q, raqamlar uzluksiz o'zgaradi.

`clamp()` ning ish printsiipi:



✦ **card-grid ni eslaymiz** — uni `auto-fit` + `minmax` bilan yozganimiz uchun **u allaqachon responsive!** Telefonda kartalar o'zi bitta ustunga tushadi, kerak emas media query. Mana shuning uchun to'g'ri texnika tanlash media query'lar sonini kamaytiradi.

⚠ **Viewport meta tegini unutma!** HTML'imizda `<meta name="viewport" ...>` bor edi. Usiz telefon sahifani 980px deb hisoblaydi va media query'lar **umuman ishlamaydi**. Bu — eng ko'p uchraydigan responsive xato.

22.7 6-qadam: Dark mode — `prefers-color-scheme`

Zamonaviy foydalanuvchilarning ko'pi qurilmasini tungi (dark) rejimga qo'yib qo'ygan. Sahifamiz buni **avtomatik** sezsa — professional taassurot qoldiradi.

Sirning kaliti shu yerda: biz hamma rangni **token** qilib yozganmiz. Demak, dark mode uchun butun sahifani qayta yozmaymiz — faqat `:root` dagi token qiymatlarini almashtiramiz!

```
/* Foydalanuvchi tizimi tungi rejimda bo'lsa */
@media (prefers-color-scheme: dark) {
  :root {
    --rang-matn: #e2e8f0;          /* matn — och */
    --rang-matn-och: #94a3b8;
    --rang-fon: #0f172a;          /* fon — quyuq ko'k-kulrang */
    --rang-fon-karta: #1e293b;
    --rang-chegara: #334155;
    /* Asosiy ko'kni biroz yorqinroq qilamiz — quyuq fonda yaxshi
    ko'rinadi */
    --rang-asosiy: #3b82f6;
  }
}
```

Bu yerda hech qanday `.card`, `.hero`, `body` qoidasini takrorlamadik — chunki ularning hammasi `var(--rang-...)` orqali rang oladi. Token qiymati o'zgarishi bilan butun sahifa avtomatik qayta bo'yaladi. Mana bu — design token tizimining eng katta mukofoti.

💡 **Nega media query ichida `:root` ?** Custom property'lar **kaskad bo'yicha qayta hisoblanadi**. `prefers-color-scheme: dark` shart bajarilganda, yangi `:root` qoidasi eskisini ustidan yozadi va `var()` ni ishlatgan har element yangi qiymatni oladi.

⚠️ **Diqqat — kontrast.** Dark mode'da matn va fon orasidagi kontrast yetarli bo'lsin. `#e2e8f0` matn `#0f172a` fonda juda yaxshi o'qiladi. Quyuq fonga quyuq matn qo'yib yubormaslik kerak — bu accessibility buzilishi.

22.8 7-qadam: Interaktivlik — hover, focus va nozik animatsiya

Sahifa tirik tuyulishi uchun interaktiv holatlar kerak: sichqoncha tekkanda tugma reaksiya bersin, klaviatura bilan yurganda fokus ko'rinsin.

```
/* Tugma ustiga sichqoncha kelganda */
.btn-primary:hover {
  background: var(--rang-asosiy-quyuq);
  transform: translateY(-2px); /* biroz "ko'tariladi" */
}

/* O'tishni silliq qilish — sakramasin */
.btn {
```

```

    transition: background 0.2s ease, transform 0.2s ease;
}

/* Karta ustiga kelganda – biroz ko'tarilib soya beradi */
.card {
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.card:hover {
    transform: translateY(-4px);
    box-shadow: var(--soya);
}

/* Menyu havolalari hover'da asosiy rangga o'tadi */
.nav-list a:hover {
    color: var(--rang-asosiy);
}

```

transition nega muhim? Usiz `:hover` holati **darrov, sakrab** almashadi — bu qo'pol ko'rinadi. `transition` o'zgarishni 0.2 soniyaga cho'zib silliq qiladi. Inson ko'zi silliq harakatni "professional" deb qabul qiladi.

Endi **fokus holati** — bu accessibility uchun shart, bezak emas:

```

/* Klaviatura (Tab) bilan yurganda fokusni aniq ko'rsat */
.btn:focus-visible,
.nav-list a:focus-visible {
    outline: 3px solid var(--rang-asosiy);
    outline-offset: 3px;
}

```

✦ **`:focus-visible` nega `:focus` emas?** `:focus-visible` faqat **klaviatura** bilan fokuslanganda outline'ni ko'rsatadi, sichqoncha bilan bosilganda ko'rsatmaydi. Shunday qilib klaviatura foydalanuvchisi yo'lini ko'radi, sichqoncha foydalanuvchisi esa keraksiz ramkadan bezovta bo'lmaydi.

⚠ **Hech qachon `outline: none` qilib, o'rniga hech narsa qo'ymaslik kerak!** Bu klaviatura foydalanuvchisini "ko'r" qilib qo'yadi — ular qayerda turganini bilmaydi. Outline'ni o'chirsang, albatta chiroyliroq alternativa (ramka yoki soya) ber.

Nihoyat, bitta **nozik kirish animatsiyasi** — hero sahifa ochilganda yumshoq paydo bo'lsin (fade-in):

```

/* Pastdan yuqoriga suzib chiqib paydo bo'lish */
@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(16px);
    }
    to {
        opacity: 1;
    }
}

```

```

    transform: translateY(0);
  }
}

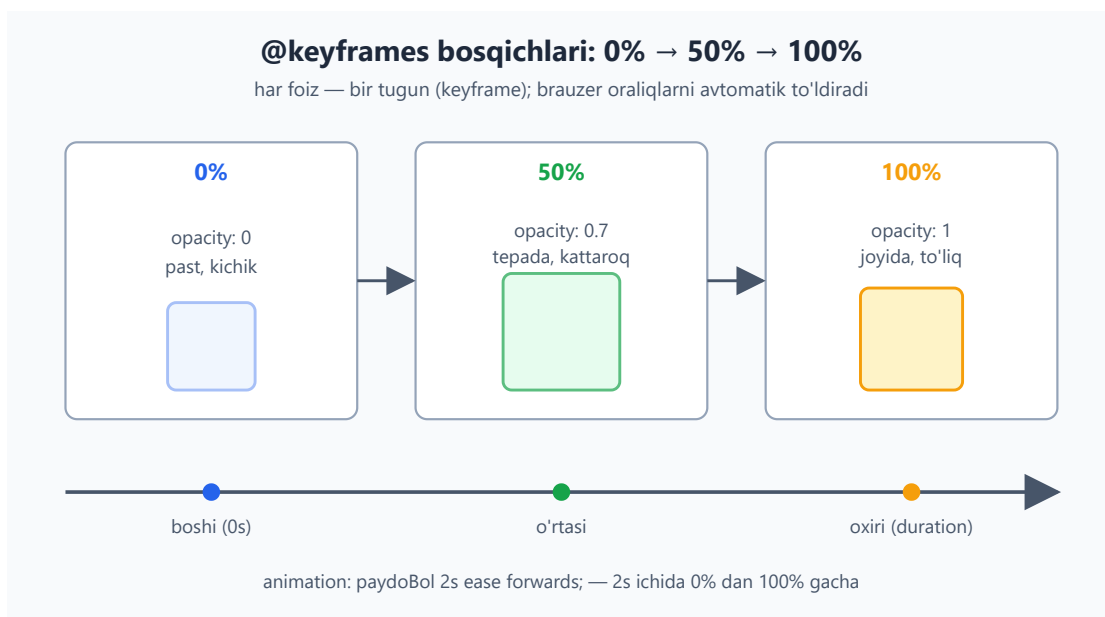
.hero {
  animation: fadeIn 0.6s ease-out;
}

/* Harakatni kamaytirishni so'ragan foydalanuvchiga hurmat */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation: none !important;
    transition: none !important;
  }
}

```

💡 **prefers-reduced-motion nega kerak?** Ba'zi foydalanuvchilar (vestibulyar buzilishi borlar) harakatdan boshi aylanadi yoki ko'ngli aynaydi. Ular tizimda "harakatni kamaytir" ni yoqib qo'yadi. Bu media query shuni sezadi va animatsiyalarni o'chiradi. **Professional sahifa shuni hurmat qiladi** — bu ham accessibility.

@keyframes ning ishlash printsipi (timeline):



22.9 8-qadam: Accessibility tekshiruvi

Sahifa chiroyli, lekin **hamma uchun foydalanish mumkinmi?** Mahsulotni topshirishdan oldin shu ro'yxatdan o'tamiz. Accessibility — qo'shimcha emas, asosiy sifat ko'rsatkichi.

Tekshiruv ro'yxati (checklist):

Tekshiruv	Bizning holat
<code>lang</code> atributi bor	<code><html lang="uz"></code> ✓
Bitta <code><h1></code> , ierarxiya buzilmagan	<code>h1 → h2 → h3</code> ✓
Semantik landmarklar	<code>header / main / footer</code> ✓
Har <code>nav</code> nomlangan	<code>aria-label</code> bor ✓
Klaviatura bilan yurish mumkin	havola/tugma <code><a></code> / <code><button></code> ✓
Fokus ko'rinadi	<code>:focus-visible</code> <code>outline</code> ✓
Rang kontrasti yetarli	matn/fon WCAG AA ✓
Harakatni kamaytirish	<code>prefers-reduced-motion</code> ✓
Rasmlarda <code>alt</code> (bo'lsa)	har <code></code> ga <code>alt</code> ✓

💡 **Tez sinov:** sichqonchani chetga qo'y va faqat **Tab** tugmasi bilan sahifani aylanib chiq. Har bosishda fokus qayerga tushganini ko'ra olyapsanmi? Logodan menyuga, tugmaga, footer havolalariga mantiqiy tartibda o'tyaptimi? Agar ha — klaviatura accessibility joyida.

⚠️ **Eng ko'p unutiladigan narsa — rang kontrasti.** "Xira kulrang matn och kulrang fonda" chiroyli ko'rinadi-yu, lekin ko'plar (ayniqsa quyoshda yoki ko'zi xira odamlar) uni o'qiy olmaydi. WCAG AA standarti: oddiy matn uchun kontrast nisbati kamida **4.5:1**. Brauzer DevTools'da rangni bosib, kontrast nisbatini ko'rsa bo'ladi.

🚫 **color ni yagona ma'lumot manbai qilma.** Masalan "yashil = muvaffaqiyat, qizil = xato" deb faqat rangga tayanma — rang ko'rmaydigan foydalanuvchi farqlay olmaydi. Rang bilan birga matn yoki ikonka ham qo'sh.

22.10 Hammasini birlashtirib — to'liq kod

Mana butun loyiha bitta faylda. Buni `index.html` ga ko'chirib, brauzerda ochsang — tayyor responsive landing page ishlaydi. (Bu yerda CSS'ni `<style>` ichiga qo'ydik soddalik uchun; real loyihada alohida `styles.css` faylga ajratiladi.)



```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>FokusApp – diqqatingizni boshqaring</title>
  <style>
    /* ===== DIZAYN TOKENLARI ===== */
    :root {
      --rang-asosiy: #2563eb;
      --rang-asosiy-quyuq: #1d4ed8;
      --rang-matn: #1e293b;
      --rang-matn-och: #475569;
      --rang-fon: #ffffff;
      --rang-fon-karta: #f8fafc;
      --rang-chegara: #e2e8f0;
      --masofa-sm: 8px;
      --masofa-md: 16px;
      --masofa-lg: 32px;
      --masofa-xl: 64px;
      --shrif-asosiy: "Segoe UI", system-ui, -apple-system, sans-
serif;
      --radius: 12px;
      --soya: 0 4px 12px rgba(0, 0, 0, 0.08);
      --kenglik-maks: 1100px;
    }

    /* ===== DARK MODE ===== */
    @media (prefers-color-scheme: dark) {
      :root {
        --rang-matn: #e2e8f0;
        --rang-matn-och: #94a3b8;
        --rang-fon: #0f172a;
        --rang-fon-karta: #1e293b;
        --rang-chegara: #334155;
        --rang-asosiy: #3b82f6;
      }
    }

    /* ===== RESET va ASOS ===== */
    *, *::before, *::after { box-sizing: border-box; margin: 0;
padding: 0; }
    body {
      font-family: var(--shrif-asosiy);
      color: var(--rang-matn);
      background: var(--rang-fon);
      line-height: 1.6;
    }
    a { color: inherit; text-decoration: none; }

    /* ===== LAYOUT ===== */
    .container, .site-header, main, .site-footer {
      max-width: var(--kenglik-maks);

```



```

    margin-inline: auto;
  }
  section { padding-block: var(--masofa-xl); }
  main, .site-footer { padding-inline: var(--masofa-md); }

  /* ===== HEADER + NAV (Flexbox) ===== */
  .site-header {
    display: flex;
    flex-direction: column;
    gap: var(--masofa-sm);
    align-items: center;
    padding: var(--masofa-md);
    border-bottom: 1px solid var(--rang-chegara);
  }
  .logo { font-size: 1.4rem; font-weight: 700; color: var(--rang-asosiy); }
  .nav-list { display: flex; gap: var(--masofa-lg); list-style: none; }
  .nav-list a:hover { color: var(--rang-asosiy); }
  @media (min-width: 768px) {
    .site-header { flex-direction: row; justify-content: space-between; }
  }

  /* ===== HERO ===== */
  .hero {
    text-align: center;
    max-width: 680px;
    margin-inline: auto;
    animation: fadeIn 0.6s ease-out;
  }
  .hero h1 { font-size: clamp(1.8rem, 5vw, 3.5rem); margin-bottom: var(--masofa-md); line-height: 1.2; }
  .hero p { color: var(--rang-matn-och); font-size: 1.15rem; margin-bottom: var(--masofa-lg); }

  /* ===== TUGMA ===== */
  .btn {
    display: inline-block;
    padding: var(--masofa-sm) var(--masofa-lg);
    border-radius: var(--radius);
    font-weight: 600;
    cursor: pointer;
    transition: background 0.2s ease, transform 0.2s ease;
  }
  .btn-primary { background: var(--rang-asosiy); color: #ffffff; }
  .btn-primary:hover { background: var(--rang-asosiy-quyuq); transform: translateY(-2px); }

  /* ===== FEATURES (Grid) ===== */
  .features h2 { text-align: center; margin-bottom: var(--masofa-lg); }
  .card-grid {
    display: grid;

```

```

        grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
        gap: var(--masofa-lg);
    }
    .card {
        display: flex;
        flex-direction: column;
        gap: var(--masofa-sm);
        padding: var(--masofa-lg);
        background: var(--rang-fon-karta);
        border: 1px solid var(--rang-chegara);
        border-radius: var(--radius);
        transition: transform 0.2s ease, box-shadow 0.2s ease;
    }
    .card:hover { transform: translateY(-4px); box-shadow: var(--soya); }
}

.card h3 { color: var(--rang-asosiy); font-size: 1.25rem; }
.card p { color: var(--rang-matn-och); }

/* ===== FOOTER ===== */
.site-footer {
    display: flex;
    flex-direction: column;
    gap: var(--masofa-sm);
    align-items: center;
    padding-block: var(--masofa-lg);
    border-top: 1px solid var(--rang-chegara);
    color: var(--rang-matn-och);
    text-align: center;
}
.site-footer nav { display: flex; gap: var(--masofa-md); }

/* ===== FOKUS (accessibility) ===== */
.btn:focus-visible, a:focus-visible {
    outline: 3px solid var(--rang-asosiy);
    outline-offset: 3px;
}

/* ===== ANIMATSIYA ===== */
@keyframes fadeIn {
    from { opacity: 0; transform: translateY(16px); }
    to { opacity: 1; transform: translateY(0); }
}
@media (prefers-reduced-motion: reduce) {
    *, *::before, *::after { animation: none !important; transition: none !important; }
}
</style>
</head>
<body>
    <header class="site-header">
        <a href="#" class="logo">FokusApp</a>
        <nav aria-label="Asosiy menyu">
            <ul class="nav-list">
                <li><a href="#features">Imkoniyatlar</a></li>

```

```

        <li><a href="#narx">Narx</a></li>
        <li><a href="#aloqa">Aloqa</a></li>
    </ul>
</nav>
</header>

<main>
    <section class="hero">
        <h1>Diqqatingizni qaytaring</h1>
        <p>FokusApp chalg'ituvchilarni bloklaydi va ishingizni kuzatadi – shunda muhim narsaga e'tibor qarata olasiz.</p>
        <a href="#" class="btn btn-primary">Bepul boshlash</a>
    </section>

    <section id="features" class="features">
        <h2>Nega FokusApp?</h2>
        <div class="card-grid">
            <article class="card">
                <h3>Chalg'ituvchilarni bloklaydi</h3>
                <p>Ish vaqtida ijtimoiy tarmoqlar va xabarlarini avtomatik o'chiradi.</p>
            </article>
            <article class="card">
                <h3>Vaqtni kuzatadi</h3>
                <p>Qaysi vazifaga qancha vaqt ketganini aniq ko'rsatadi.</p>
            </article>
            <article class="card">
                <h3>Hisobotlar beradi</h3>
                <p>Har hafta unumdorligingiz haqida sodda hisobot yuboradi.
        </p>
            </article>
        </div>
    </section>
</main>

<footer class="site-footer">
    <p>&copy; 2026 FokusApp. Barcha huquqlar himoyalangan.</p>
    <nav aria-label="Footer havolalari">
        <a href="#">Maxfiylik</a>
        <a href="#">Shartlar</a>
    </nav>
</footer>
</body>
</html>

```

Natija: to'liq ishlaydigan sahifa. Telefonda — bitta ustun, tik navbar, kichikroq sarlavha. Desktopda — kartalar yonma-yon, navbar ikki chetga taqsimlangan. Tungi rejimda — avtomatik quyuq. Tugma va kartalar hover'da jonlanadi, klaviatura bilan yurish ko'rinadi, hero yumshoq paydo bo'ladi. Hammasi token qiymatlaridan boshqariladi.

22.11 Keyingi qadamlar — qayoqqa o'sish kerak

Tabriklaymiz — bu qo'llanmaning oxirgi bobini ham yakunlading! Endi sen HTML va CSS bilan to'liq, professional sahifa qura olasan. Lekin o'rganish shu yerda tugamaydi. Mana keyingi yo'nalishlar:

1. Mashq qilish — eng muhimi. Bilim faqat amaliyotda mustahkamlanadi. - [Flexbox Froggy](#) — Flexbox'ni o'yin orqali mashq qilish. - [Grid Garden](#) — Grid uchun shunaqa o'yin. - O'zing yoqtirgan saytni (masalan kichik blog yoki portfolio) noldan qaytadan yasab ko'r.

2. Brauzer DevTools'ni o'rgan. `F12` bos — bu sening eng kuchli quroling. - **Elements** panelida HTML/CSS'ni jonli tahrirlash. - **Device toolbar** (telefon ikonkasi) bilan responsive'ni turli ekranlarda sinash. - **Lighthouse** bilan accessibility va tezlikni avtomatik baholash.

3. Real loyihalar qur. Portfolio nazariy bilimdan ko'ra ko'p narsani isbotlaydi. - Shaxsiy portfolio sahifang. - Do'sting yoki kichik biznes uchun bir betlik sayt. - Bu landing page'ni kengaytir: narx jadvali, izohlar bo'limi, aloqa formasi qo'sh.

4. Keyingi bosqich texnologiyalar (HTML/CSS'ni mustahkamlagach): - **JavaScript** — sahifani interaktiv qilish (forma tekshiruvi, dinamik mazmun). - **CSS framework'lar** (Tailwind, Bootstrap) — tezroq ishlash uchun. - **Sass/SCSS** — CSS'ni kuchaytiruvchi vositalar. - **Git va GitHub** — kodni saqlash va loyihalarni nashr qilish (GitHub Pages bilan saytingni bepul internetga chiqar).

💡 **Eng muhim maslahat:** mukammallikni kutma. Birinchi loyihang xunuk bo'ladi — bu normal. Har loyiha bilan yaxshilanasan. Kod yoz, sindir, tuzat, yana yoz. Frontend — amaliyot san'ati.

Mashqlar

Quyidagi mashqlar shu bobdagi loyiha kodi ustida ishlaydi. Har birini brauzerda sinab ko'r.

1-mashq. `:root` ga yangi token `--rang-aksent: #f59e0b;` (amber) qo'sh va undan footer'dagi havolalar rangida foydalan (hover'da). Token tizimining qulayligini his qil.

**Yechim**

```
:root {  
  /* ... mavjud tokenlar ... */  
  --rang-aksent: #f59e0b;  
}  
  
.site-footer nav a:hover {  
  color: var(--rang-aksent);  
}
```

Faqat ikki joyga teging — token bilan boshqarish shunaqa oson.

2-mashq. Hero tugmasi yoniga ikkinchi, "ikkilamchi" tugma qo'sh: `<a href="#" class="btn btn-secondary">Batafsil</a>`. Unga `.btn-secondary` stil ber — shaffof fon, asosiy rangli chegara va matn.

**Yechim**

```
.btn-secondary {  
  background: transparent;  
  color: var(--rang-asosiy);  
  border: 2px solid var(--rang-asosiy);  
}  
  
.btn-secondary:hover {  
  background: var(--rang-asosiy);  
  color: #ffffff;  
}
```

`.btn` umumiy stil (padding, radius, transition) avtomatik meros bo'ladi — faqat farqlarni yozdik.

3-mashq. Kartalar to'rini majburan **3 ustun** qil, lekin faqat ekran 900px dan keng bo'lganda. 900px dan tor ekranda `auto-fit` qoidasi qolsin.

**Yechim**

```
@media (min-width: 900px) {  
  .card-grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

🚩 Eslatma: `auto-fit` ko'pincha media query'siz yetarli. Bu mashq aniq nazorat qachon kerakligini ko'rsatish uchun.

4-mashq. Hero sarlavhasining `clamp()` qiymatlarini o'zgartir: minimal `2rem`, maksimal `4rem`, oraliq `6vw`. Brauzer oynasini toraytirib-kengaytirib, matn qanday silliq o'zgarishini kuzat.

Yechim

```
.hero h1 {  
  font-size: clamp(2rem, 6vw, 4rem);  
}
```

Oyna enini sekin o'zgartirsang, matn sakramay, uzluksiz kattalashib-kichrayadi. Bu — `clamp()` ning sehri.

5-mashq. Dark mode'ni sun'iy yoqib, sinab ko'r. DevTools'da: `F12` → uch nuqta menyu → "More tools" → "Rendering" → "Emulate CSS prefers-color-scheme" ni `dark` qil. Sahifa qaysi ranglar bilan o'zgardi?

Yechim/maslahat

Hech qanday kod yozish shart emas — bu kuzatuv mashqi. Dark rejim yoqilganda `:root` dagi token qiymatlari almashadi, shuning uchun `body`, `.card`, matn ranglari avtomatik quyushladi. Diqqat qil: biz hech bir komponent qoidasini takrorlamadik — token tizimi shuni mumkin qildi.

6-mashq. Kartaga hover'da `transform: translateY(-4px)` bilan birga `border-color: var(--rang-asosiy);` qo'sh — sichqoncha tekkanda chegara asosiy rangga o'tsin. `transition` ga `border-color` ni ham qo'shishni unutma.

Yechim

```
.card {  
  /* ... */  
  transition: transform 0.2s ease, box-shadow 0.2s ease, border-color 0.2s ease;  
}  
.card:hover {  
  transform: translateY(-4px);  
  box-shadow: var(--soya);  
  border-color: var(--rang-asosiy);  
}
```

⚠ `transition` ga `border-color` ni qo'shmasang, chegara rangi sakrab o'zgaradi — silliq bo'lmaydi.

7-mashq. Accessibility tekshiruvi: sichqonchani ishlatmay, faqat **Tab** tugmasi bilan sahifani boshidan oxirigacha aylanib chiq. Fokus har doim ko'rinadimi? Tartib mantiqiyimi? Agar biror element fokusda ko'rinmasa — sababini top.



Yechim/maslahat



To'g'ri yozilgan kodda fokus tartibi: logo → menyu havolalari → hero tugmasi → footer havolalari. Har biri `:focus-visible` outline bilan ko'rinishi kerak. Agar biror havola/tugma `<div onclick>` bilan yasalgan bo'lsa — u Tab bilan fokuslanmaydi. Shuning uchun bosiladigan narsa har doim `<a>` yoki `<button>` bo'lishi kerak.

8-mashq. (Qiyinroq) Footer'ga uchinchi bo'lak — ijtimoiy havolalar ro'yxati qo'sh va `prefers-reduced-motion` ishlayotganini tekshir: DevTools "Rendering" da "Emulate prefers-reduced-motion" ni `reduce` qil, keyin sahifani yangila. Hero animatsiyasi va hover o'tishlari to'xtadimi?



Yechim/maslahat



```
<footer class="site-footer">
  <p>&copy; 2026 FokusApp. Barcha huquqlar himoyalangan.</p>
  <nav aria-label="Footer havolalari">
    <a href="#">Maxfiylik</a>
    <a href="#">Shartlar</a>
  </nav>
  <nav aria-label="Ijtimoiy tarmoqlar">
    <a href="#">Telegram</a>
    <a href="#">Instagram</a>
    <a href="#">YouTube</a>
  </nav>
</footer>
```

`reduce` yoqilganda hero birdan (animatsiyasiz) ko'rinadi, tugma/karta hover'lari ham sakrab o'zgaradi. Bu — to'g'ri xulq: harakatga sezgir foydalanuvchilarni himoya qildik.

